Genetic Diversity in Hybrid Breeding

Measuring it, constraining it, and optimising against it, with R

Eder D. B. S.

Contents

Preface . . . 15
The problem, stated once . . . 15
What is unusual here . . . 15
How to read this . . . 16
Running the code . . . 16

I Foundations

1 Coancestry, and the two matrices . . . 21
1.1 From line to hybrid . . . 21
1.2 The VanRaden matrix, and why it cannot measure diversity . . . 21
1.3 Molecular coancestry . . . 22
1.3.1 Computable form . . . 22
1.3.2 Reading the diagonal . . . 23
1.4 `f` is not an ad hoc substitute . . . 23
1.5 The division of labour . . . 24
Reproduce this chapter . . . 24

2 Diversity in a subdivided population . . . 25
2.1 Notation . . . 25
2.2 The three coancestry blocks . . . 25
2.2.1 f_{ij}, between subpopulations . . . 25
2.2.2 f_{ii}, within a subpopulation . . . 25
2.2.3 s_i and F_i, self-coancestry and molecular inbreeding . . . 26
2.3 Two per-subpopulation metrics . . . 26
2.4 Distances, and what they actually are . . . 26
2.5 The equation that matters most in practice . . . 27
2.6 F statistics and the partition . . . 27
2.6.1 Why the partition changes the strategy by species . . . 27
2.7 The worked example, reproduced . . . 27
2.7.1 Removing an accession can *increase* diversity . . . 28
2.7.2 Optimal contributions . . . 29
2.8 Minimising coancestry is maximising N_e . . . 29
2.9 The honest caveats . . . 29
2.10 Two errata, found while verifying . . . 30
2.11 What manages this quantity, and where . . . 30
Reproduce this chapter . . . 30

3 Response to selection: the other half of the objective 31
3.1 The breeder's equation, and how well it does here 31
3.2 Genic variance, and what it has to do with gene diversity 32
3.3 The decomposition this book already had 33
3.4 What `weights = NULL` is assuming 33
3.5 The α constraint is a restricted index 34
3.6 Where this connects 35
Reproduce this chapter 35

II Simulation

4 Simulating lines, pools and crosses 39
4.1 What has to be produced 39
4.2 Simulating the lines 39
4.2.1 The divergence knob 40
4.3 From lines to hybrids 41
4.4 Predicted traits 42
4.5 The two matrices, both built 42
4.6 The stage-1 contract in one call 43
4.7 What the candidate pool looks like 44
Reproduce this chapter 44

5 Sizing a segregating population, F1 to F4 45
5.1 The simulated space is ancestry, not alleles 45
5.2 The assumptions, one at a time 45
5.3 Verifying the identities live 46
5.4 The results 47
5.4.1 The diversity curve saturates at 0.988, not at 1 48
5.4.2 The bulk is the worst scheme across the whole range 48
5.4.3 Block size does not depend on N 49
5.4.4 The metrics saturate at different N 49
5.5 Sizing the F1 51
5.6 Working numbers 51
5.7 Where this connects 51
Reproduce this chapter 52

6 Introgressing several genes by backcrossing 53
6.1 The operation, in one diagram 53
6.2 The model 54
6.3 Three criteria, and only one of them binds 54
6.4 Which selection index 55
6.5 How long to keep selecting recombinants 57
6.6 Allocating a fixed budget across BC1, BC2 and BC3 57
6.7 Fixing the stack: selfing against doubled haploids 58

6.8 Running the simulation . . . 59
6.9 Assumptions, and what is not modelled . . . 60
6.10 Where this connects . . . 61
Reproduce this chapter . . . 62

7 Simulating a hybrid breeding programme: AlphaSimR from the ground up . . . 63
7.1 Step 1 – Founder genomes: what a founder is, and which engine to use . . . 64
7.2 Step 2 – `SimParam`, and defining a trait with dominance . . . 66
7.2.1 The dominance model, as AlphaSimR writes it . . . 67
7.3 Step 3 – The base population, and where the variance components live . . . 68
7.4 Step 4 – The DNA lottery, heterosis, and why an F1 is not a family . . . 70
7.5 Step 5 – Selfing, doubled haploids, and why hybrid seed is remade every year . . . 72
7.6 Step 6 – Phenotyping and selection: per se, then on the testcross . . . 73
7.6.1 Per se selection and the breeder's equation . . . 74
7.6.2 Selection on the testcross: general combining ability . . . 75
7.6.3 How many testers, how many locations . . . 76
7.7 Step 7 – Two traits, and what equal weighting assumes . . . 78
7.8 Step 8 – The parts assembled: reciprocal recurrent selection . . . 80
7.8.1 Founding two pools with a real coalescent history . . . 80
7.8.2 One cycle . . . 84
7.8.3 The trajectory across cycles, replicated, against a paired control . . . 84
7.8.4 The final hybrid pool through `stage2_select()`, unchanged . . . 88
7.9 Assumptions, and what this adds over Chapter 4. . . . 89
7.10 Where this connects . . . 89
Reproduce this chapter . . . 90

8 A complete plant breeding programme, and what it costs in diversity . . . 91
8.1 The scheme . . . 91
8.2 Why a breeding programme needs a burn-in . . . 92
8.3 One programme, fifteen years . . . 92
8.4 Genomic selection: two levers, not one . . . 94
8.5 The comparison, replicated . . . 94
8.6 The diversity ledger . . . 96
8.7 Assumptions . . . 96
8.8 Where this connects . . . 97
Reproduce this chapter . . . 97

III Mating and testing design

9 How few hybrids are enough? Mating designs for a two-pool programme . . . 101
9.1 Why the crossing plan is the binding decision . . . 101

9.2 Combining ability as an estimation problem . . . 102
9.2.1 The model . . . 103
9.2.2 Four quantities that control a design . . . 103
9.2.3 Connectivity, in one paragraph . . . 104
9.2.4 SCA and the cross-level residual are the same thing in cross means . . . 105
9.3 The classical tester approach . . . 106
9.3.1 What a tester is for, and what it trades . . . 106
9.3.2 Choice of tester . . . 107
9.3.3 What testers cannot deliver . . . 107
9.3.4 When the tester is still right . . . 108
9.4 Sparse and incomplete factorials . . . 108
9.4.1 The design family . . . 108
9.4.2 How sparse is sparse enough – the published evidence . . . 108
9.4.3 Sparse versus tester, at equal plot budget . . . 109
9.4.4 Crosses per line, and structure versus randomness . . . 110
9.4.5 What is lost . . . 111
9.5 The analysis is part of the design . . . 111
9.6 Genomic information for deciding which crosses to make . . . 113
9.6.1 From relatedness to hybrid prediction . . . 114
9.6.2 The T0 / T1 / T2 hierarchy . . . 114
9.6.3 The negative result . . . 114
9.6.4 Which hybrids to phenotype, and which crosses to make . . . 115
9.6.5 What the simulation adds to the literature . . . 116
9.7 Seed production and the cost of a crossing plan . . . 116
9.7.1 Three cost centres . . . 116
9.7.2 Budget-constrained optimisation . . . 117
9.8 A decision framework . . . 119
9.9 Open problems . . . 120
9.10 Reproducibility and assumptions . . . 121
9.11 Where this connects . . . 122
Reproduce this chapter . . . 122

10 Sparse testing: which hybrids go to which locations . . . 123
10.1 The allocation problem of stage 1 . . . 124
10.1.1 The economics of the first stage . . . 124
10.1.2 The breeder's equation as the decision criterion . . . 125
10.2 Foundations . . . 125
10.2.1 The hybrid model: GCA, SCA, and what sparsity threatens . . . 125
10.2.2 Hybrid relationship matrices . . . 126
10.2.3 Multi-environment trials and mixed models . . . 126
10.3 What sparse testing is, and where it came from . . . 126
10.3.1 Definition and genealogy . . . 126
10.3.2 A taxonomy of sparse designs . . . 127
10.4 Validation schemes: what each one answers . . . 129
10.5 The empirical record across crops . . . 130
10.6 The minimum-overlap controversy . . . 132
10.7 Connectivity and checks . . . 133
10.8 Sparse testcrossing and the number of testers . . . 134
10.9 What sparsity does to variance components and to the BLUP . . . 135

10.10 Sparse testing with phenomics . . . 136
10.11 Simulation as a sizing tool . . . 137
10.11.1 Why simulate before designing . . . 137
10.11.2 Realistic parameters: what the literature supplies . . . 137
10.11.3 What the simulation finds . . . 138
10.12 Practical recommendations by stage . . . 145
10.13 Limitations of the evidence, and perspectives . . . 146
10.14 Reproducibility and assumptions . . . 147
10.15 Where this connects . . . 148
Reproduce this chapter . . . 149

11 Which crosses, at which locations? Joint sparse allocation . . . 151
11.1 Why the two decisions are one . . . 151
11.2 The joint model . . . 152
11.3 What the literature offers . . . 152
11.4 Design criteria computable before planting . . . 154
11.5 What the simulation finds . . . 155
11.5.1 1. The budget split: crosses against locations . . . 155
11.5.2 2. Allocation strategy, with the plan fixed . . . 157
11.5.3 3. What the criterion can and cannot see . . . 158
11.5.4 Limitations of the simulation . . . 160
11.6 Practical recommendations . . . 161
11.7 Open problems . . . 161
11.8 Reproducibility and assumptions . . . 161
11.9 Where this connects . . . 162
Reproduce this chapter . . . 162

IV Metrics

12 Diversity metrics in the genomic era . . . 165
12.1 The programme this chapter is written for . . . 165
12.2 Three lenses, not three synonyms . . . 166
12.3 The metrics, one at a time . . . 166
12.3.1 Rate of inbreeding and effective size . . . 166
12.3.2 `F_hom`, homozygosity-based . . . 167
12.3.3 `F_drift`, drift-based . . . 168
12.4 The classical equivalence, and the covariance that destroys it . . . 169
12.4.1 Verifying the identity . . . 169
12.4.2 Why each matrix generates a sign . . . 170
12.5 The other matrices, briefly . . . 172
12.5.1 Side by side . . . 172
12.6 Reference results . . . 172
12.7 The monitoring protocol . . . 173
12.7.1 Step 1 – Freeze p_0 at the founding cycle . . . 173
12.7.2 Step 2 – Compute both lenses in one marker sweep . . . 175
12.7.3 Step 3 – Report the covariance, not just the two lenses . . . 176
12.7.4 Step 4 – Regress $\log(1 - F)$ on cycle, and read R^2 . . . 177

12.7.5 Step 5 – Audit on loci that did not enter the management . . . 180
12.7.6 Step 6 – Read the two pools separately, and never F_{ST} alone . . . 181
12.7.7 Step 7 – Check the matrix you are about to invert . . . 184
12.8 What not to do . . . 185
12.9 Limits of the animal-to-plant transposition . . . 186
12.10 Operational checklist . . . 187
Reproduce this chapter . . . 187

13 Heterosis, dominance, and what divergence buys . . . 189
13.1 The dominance model, minimally . . . 189
13.1.1 The one-locus scale . . . 189
13.1.2 What d_k is, and what it is not . . . 190
13.1.3 The mid-parent difference . . . 190
13.2 Expected heterosis over a pool pair . . . 191
13.3 The identity . . . 191
13.3.1 A third form: one minus the between-pool coancestry . . . 192
13.4 A second identity, at the hybrid level . . . 193
13.5 Inbreeding depression, and why it does not price α . . . 193
13.6 When genetic distance predicts heterosis exactly . . . 194
13.7 Divergence is not a bonus . . . 195
13.7.1 The two terms, term by term . . . 196
13.8 What actually buys heterosis . . . 198
13.8.1 Alignment is a covariance . . . 198
13.8.2 The scenarios . . . 198
13.8.3 Isolating the divergence channel . . . 199
13.9 Reading the S1 panel again . . . 200
13.10 General and specific combining ability . . . 200
13.10.1 GCA is an average effect in the other pool's background . . . 201
13.10.2 SCA is minus twice a weighted parental inner product . . . 202
13.10.3 The variance components . . . 202
13.11 Divergence destroys SCA faster than it builds GCA . . . 203
13.11.1 What this says about ranking crosses . . . 206
13.12 Running this on a real programme . . . 206
13.12.1 Step 1 – Verify the parents are inbred, and price it if they are not . . . 206
13.12.2 Step 2 – Compute pool frequencies, and keep them distinct from p_0 . . . 207
13.12.3 Step 3 – Decide what stands in for d_k, and say so . . . 207
13.12.4 Step 4 – Partition, and cross-check against the diversity partition . . . 208
13.12.5 Step 5 – Log `mean_sq_delta_p` every cycle . . . 209
13.12.6 Step 6 – Check that the crossing layout identifies what you intend to fit . . . 209
13.12.7 Step 7 – Fit the decomposition and check it reconstructs . . . 211
13.12.8 Step 8 – Decide the F_{ST} band on alignment, not on F_{ST} . . . 212
13.13 What this does not change . . . 212
13.13.1 What it would take to use d . . . 213
13.14 Where this connects . . . 213
Reproduce this chapter . . . 213

14 A catalogue of metrics on `f` . . . 215
14.1 Group coancestry, and the off-diagonal problem . . . 215
14.2 Gene diversity: θ and H_e are the same quantity . . . 216

14.3 Status number N_s . . . 216
14.4 Effective number of parents . . . 217
14.5 Allelic richness . . . 217
14.6 Distances: E-NE and A-NE . . . 218
14.7 Effective dimensionality . . . 218
14.8 Decomposition by heterotic group . . . 219
14.8.1 Where this could go next . . . 219
14.9 The correct baseline . . . 220
14.9.1 Why not compare against the whole population . . . 220
14.9.2 Operational definition of loss . . . 220
14.9.3 How to set the α limit . . . 221
14.10 Every metric against the null . . . 221
Reproduce this chapter . . . 221

15 Choosing metrics: collapse, cost, redundancy . . . 223
15.1 Finding 1: the hybrid coancestry matrix is never needed . . . 223
15.1.1 Derivation . . . 223
15.1.2 Four consequences . . . 223
15.1.3 Verification . . . 224
15.2 Finding 2: what each metric costs . . . 225
15.3 Finding 3: the metric space is nearly one-dimensional . . . 226
15.3.1 Reproducing the argument at book scale . . . 227
15.4 Finding 4: the encoding, not the constraint handler, decides whether DE works . . . 229
15.5 Finding 5: the index is not the truth, and its error is not white . . . 230
15.5.1 The correlation structure matters more than the amount . . . 231
15.5.2 What this does to the recommendation . . . 231
15.6 The recommendation, stated . . . 232
15.6.1 Inside the optimiser . . . 232
15.6.2 At validation, on the returned plan only . . . 232
15.6.3 Not recommended . . . 232
15.6.4 Caveats . . . 232
15.7 Drop-in code . . . 232
Reproduce this chapter . . . 233

16 Metrics at each pipeline stage, and five traps . . . 235
16.1 The kernel: one matrix, all four stages . . . 235
16.1.1 Trap 1: modified Rogers distance is the same statistic . . . 235
16.2 S1: conserving the heterotic groups . . . 236
16.2.1 Trap 2: two F_{ST} definitions differ by up to 2x . . . 236
16.2.2 Trap 3: F_{ST} is the wrong target to maximise . . . 236
16.2.3 The S1 monitoring panel . . . 237
16.3 S2: choosing crosses for DH induction . . . 237
16.3.1 Three verified identities . . . 237
16.3.2 Progeny variance, and the usefulness criterion . . . 238
16.3.3 Trap 4: genetic distance is not a proxy for progeny variance . . . 238
16.4 S3: selecting lines per se . . . 239
16.4.1 Trap 5: the incremental greedy constraint silently returns nothing . . . 240

16.5 **S4: hybrid mining** . . . 241
16.6 **Where the diversity actually goes** . . . 241
16.6.1 What the diversity term buys, and what it does not . . . 242
16.7 **Cost summary** . . . 243
16.8 **Checklist** . . . 243
Reproduce this chapter . . . 243

17 **Predicting breeding values: GBLUP, BayesA, BayesB** . . . 245
17.1 **A training population with real noise** . . . 245
17.2 **The truth this chapter is predicting** . . . 246
17.2.1 The generative model is GBLUP's own assumption . . . 246
17.2.2 A hybrid's value is the mean of two parental values . . . 246
17.3 **GBLUP: prediction from the relationship matrix** . . . 247
17.3.1 Where the mixed model equations come from . . . 247
17.3.2 Why the ridge blend is not optional . . . 247
17.3.3 Reliability: the accuracy you can compute without the truth . . . 248
17.4 **What the random split was measuring** . . . 249
17.5 **BayesA and BayesB: prediction from marker effects** . . . 250
17.5.1 The priors, explicitly . . . 251
17.5.2 What the sampler actually selects . . . 251
17.5.3 What BayesB does that BayesA cannot . . . 252
17.6 **Verification: two routes agree** . . . 253
17.7 **Which model wins, and when** . . . 253
17.7.1 `pi` was fixed, not tuned . . . 254
17.8 **From GEBVs to `stage1_build()`** . . . 255
17.9 **Back to Finding 5: a measured accuracy, not an assumed one** . . . 255
Reproduce this chapter . . . 256

V **Optimisation**

18 **Optimal contributions and the α constraint** . . . 259
18.1 **Constraint, not a weighted sum** . . . 259
18.1.1 This argument is older than it looks . . . 259
18.2 **Relation to the standard OCS formulation** . . . 260
18.2.1 The continuous relaxation as an upper bound . . . 260
18.3 **Setting $\alpha_{\max}$** . . . 262
18.3.1 How far $\alpha_{\max}$ moves when $\mathbf{f}$ moves . . . 264
18.4 **A caveat carried over from the OCS literature** . . . 265
18.5 **Quadratic programming, when contributions are continuous** . . . 265
18.6 **What belongs in the inner loop** . . . 266
Reproduce this chapter . . . 266

19 **Combinatorial selection** . . . 267
19.1 **Four baselines before any optimiser** . . . 268
19.2 **Greedy, with incremental updates** . . . 270
19.3 **The encoding problem** . . . 273

19.3.1 A third encoding: one weight per line . . . 274
19.4 Differential evolution . . . 276
19.4.1 The warm start is not optional . . . 277
19.4.2 Checking convergence against the bound . . . 279
19.5 The full frontier . . . 280
19.6 Which strategy to use . . . 282
Reproduce this chapter . . . 283

20 Advanced selection: structure, bounds and scale . . . 285
20.1 Three facts about this problem . . . 285
20.1.1 `f` is a Gram matrix . . . 285
20.1.2 Maximising diversity is minimising the submatrix sum . . . 286
20.1.3 The per-line cap is a partition matroid . . . 286
20.2 What lazy greedy would not buy . . . 287
20.3 The move that was already paid for . . . 287
20.3.1 Against the differential evolution, at equal wall-clock . . . 290
20.3.2 The encoding, wired up at last . . . 292
20.4 The bound was not a bound . . . 294
20.5 Rounding the relaxation into a plan . . . 297
20.6 Exact, and how far the heuristics really are . . . 298
20.7 Scaling to 200 of 10,000 . . . 300
20.7.1 Which routes actually run without `f` . . . 301
20.7.2 The head-to-head at production scale . . . 303
20.8 Multi-objective, and why not NSGA-II . . . 304
20.9 What not to reach for . . . 304
20.10 Which method to use . . . 304
Reproduce this chapter . . . 305

VI Evidence and practice

21 What the plant literature establishes . . . 309
21.1 The headline qualification . . . 309
21.2 Why plant practice differs from animal breeding . . . 310
21.2.1 Distance measures, and the absence of a standard . . . 310
21.3 Heterotic groups . . . 310
21.3.1 Assignment is not a solved problem . . . 310
21.3.2 The empirical trend that justifies the constraint . . . 310
21.4 Selecting within the pool . . . 311
21.4.1 The constraint alone is not sufficient . . . 311
21.4.2 Cycle length, and what goes unreported . . . 311
21.4.3 What is hybrid-specific, and what is not . . . 311
21.5 Genebanks, core collections and introgression . . . 311
21.6 The animal-breeding corpus: which matrix to manage . . . 312
21.7 Optimisation machinery reported in plants . . . 312
21.8 What remains unestablished . . . 313

21.9 Consequences for the pipeline . . . 313
Reproduce this chapter . . . 314

22 Diversity across cycles . . . 315
22.1 The cycle . . . 315
22.2 Freezing p_0, and what happens if you do not . . . 316
22.3 The trajectory . . . 316
22.3.1 It is not an artefact of the cheap selector . . . 318
22.4 The rate of inbreeding, and what N_s is not . . . 318
22.5 Choosing a budget for a horizon . . . 319
22.6 The per-line cap, at last measurable . . . 319
22.7 Injection, and what it costs . . . 320
22.8 What the corpus supports . . . 320
22.9 Where this connects . . . 321
Reproduce this chapter . . . 321

23 The limit, and what the metrics cannot see . . . 323
23.1 Robertson's limit . . . 323
23.2 The R^2 column decides whether the theory applies . . . 323
23.3 What the metrics cannot see . . . 324
23.3.1 The experiment . . . 324
23.3.2 With real linkage . . . 325
23.4 The two costs of selection . . . 325
23.5 Where this connects . . . 326
Reproduce this chapter . . . 326

24 End to end, and the operational checklist . . . 327
24.1 The whole pipeline in one script . . . 327
24.1.1 The reference values . . . 327
24.1.2 Metrics per scenario . . . 328
24.1.3 Both lenses, and the pool partition . . . 328
24.1.4 Distance from the random null . . . 329
24.1.5 The selection table . . . 329
24.2 Reading the frontier . . . 330
24.3 What this simulation cannot tell you . . . 330
24.4 The operational checklist . . . 331
24.4.1 Setting up . . . 331
24.4.2 Choosing the budget . . . 331
24.4.3 Running the optimiser . . . 331
24.4.4 Validating the plan . . . 331
24.4.5 Across cycles . . . 331
24.5 Where to go next . . . 332
Reproduce this chapter . . . 332

Appendices . . . 333

A Notation and glossary . . . 335
A.1 Symbols . . . 335
A.1.1 Matrices . . . 335
A.1.2 Coancestry and diversity . . . 335
A.1.3 The partition . . . 336
A.1.4 Heterosis and cycles . . . 336
A.1.5 Inbreeding, the three lenses . . . 336
A.1.6 Selection and optimisation . . . 336
A.1.7 Genomic prediction . . . 337
A.1.8 Population sizing . . . 337
A.1.9 Mating design . . . 337
A.1.10 Sparse testing . . . 338
A.1.11 Response to selection . . . 338
A.2 Conventions . . . 338
A.3 Terms . . . 339
Reproduce this chapter . . . 339

B The `hybdiv` package . . . 341
B.1 Installation . . . 341
B.2 File map . . . 341
B.3 The two-stage contract . . . 341
B.4 Function index . . . 342
B.4.1 Simulation and context . . . 342
B.4.2 The two matrices . . . 342
B.4.3 Diversity metrics – all `f(idx, ctx) -> scalar` . . . 342
B.4.4 The fast line route . . . 343
B.4.5 Selection strategies . . . 343
B.4.6 Benchmarking . . . 343
B.5 Runnable scripts . . . 344
B.5.1 Pipeline stages S1-S4 . . . 344
B.5.2 Heterosis and cycles . . . 344
B.5.3 Mating design . . . 344
B.5.4 Sparse testing . . . 345
B.5.5 Caballero & Toro . . . 345
B.5.6 Population sizing . . . 345
B.5.7 Reference oracles . . . 345
B.6 Full export list . . . 346
Reproduce this chapter . . . 347

C Verification . . . 349
C.1 The method: slow oracles . . . 349
C.2 The identity sweep . . . 350
C.3 The test suite . . . 350
C.4 Provenance of every figure and table . . . 352
C.4.1 Computed live at render . . . 352
C.4.2 Read from cache . . . 353
C.5 Regenerating the cached results . . . 354
C.6 Reproducing the book . . . 354

C.7 Known limitations of the verification . . . 355
Reproduce this chapter . . . 355

D The bibliographic corpora . . . 357
D.1 What they are . . . 357
D.2 Structure . . . 357
D.3 Relevance and coverage . . . 357
D.4 Themes in the animal corpus . . . 359
D.5 Querying them . . . 359
D.6 Limitations . . . 360
D.7 Classical references . . . 360
Reproduce this chapter . . . 360

References . . . 361

Bibliography . . . 363

Preface

A hybrid breeding programme has two objectives that pull against each other. The first is short-term: pick the crosses with the highest predicted performance. The second is long-term: do not spend the germplasm that will be needed to make the next round of gain possible. Every programme knows this. Very few can say, in a number, how much of the second they traded away to buy the first.

This book is about producing that number, defending it, and putting it inside an optimiser.

The problem, stated once

Given a pool of N candidate F1 hybrids – each a cross between one line from heterotic group A and one from group B, each with a predicted multi-trait index – choose n_{sel} of them so that the mean index is as high as possible, subject to losing no more than a fraction α of the molecular gene diversity that a *random* set of the same size would have carried.

Three of those words carry the entire argument.

Molecular. Diversity is measured on the Caballero & Toro molecular coancestry matrix f, never on a VanRaden genomic relationship matrix G. This is not a stylistic preference. G is centered on its own population by construction, so the sum of all its elements is exactly zero; every diversity quantity computed from it is meaningless. Chapter 1. shows this happening.

Random. The reference for "no loss" is the distribution of random subsets of size n_{sel}, not the full candidate pool. Comparing a selected set of 100 against a pool of 2500 confounds a selection effect with a sampling effect, and the sampling effect alone can look like a loss of half a percent. Section 14.9 separates them.

Subject to. α is a *constraint*, not a term in a weighted sum. When you write $\alpha_{max} = 0.05$ you are saying "I accept losing five percent", which is a statement a breeding committee can approve. A weighted sum says "diversity is worth λ index units", which is a statement nobody can approve because nobody knows what λ means. Chapter 18. makes the distinction precise, and shows the standard optimal-contribution formulation it specialises.

What is unusual here

Most treatments of diversity management come from animal breeding, where the population is randomly mating and the unit of selection is an individual. Plant hybrid breeding breaks both assumptions:

- The parents are **inbred lines**, so their within-individual diversity is zero by construction, and the interesting heterozygosity lives in the F1 product, not in the parents.
- The population is **permanently subdivided** into heterotic groups, and the divergence between them is not a problem to be minimised – it is the engine of heterosis, to be *maintained*. A single pool-wide statistic cannot express "conserve within, hold between". Chapter 13. derives that claim as an exact identity, and then bounds it: divergence pays only where it coincides with dominance, and drifting the pools apart buys nothing at all.
- The candidates are **crosses of fixed lines**, so the progeny variance term that drives cross-selection criteria elsewhere collapses to zero, and the hybrid-by-hybrid coancestry matrix turns out never to be needed at all (Section 15.1).

Each of these changes what should be measured. None of them changes the underlying theory, which is why the book starts with the theory.

How to read this

The book is meant to be run, not only read. Every derivation that could be checked numerically is checked numerically, in a code block you can execute. Where a result is an exact algebraic identity, the deviation from a simulated oracle is printed next to it; where it is a Monte Carlo agreement, the residual is printed.

The argument the parts assemble is a chain, and it is worth stating before the table that lists them:

1. Diversity between the pools is what makes the hybrid worth selling, and diversity within them is what the next cycle has to select on (Chapter 2., Chapter 13.).
2. Those two are in tension: every plan that raises one lowers the other, and the partition of Chapter 14. is what makes the trade visible.
3. Within-pool diversity is worth protecting because it is the raw material of *response*: genic variance is the same sum over the same loci as gene diversity, weighted by squared effect instead of equally (Chapter 3.). That is what makes a diversity constraint a variance constraint in disguise, and what makes "restrict rather than price" a sixty-year-old theorem rather than a preference.
4. So the loss of within-pool diversity is a **budget**, set against what random selection of the same size would have cost (Section 14.9), not a term traded off inside an objective (Chapter 18.).
5. How much diversity there is to spend depends on how the candidates were generated: a panel cannot be selected back up to variation it never sampled (Chapter 5.), and a pool of near-copies cannot be rescued by any optimiser (Chapter 6.). Chapter 4. and Chapter 7. build that panel two different ways, and everything downstream is blind to which – Chapter 7. from the primitives up, so the generative model is something a reader can rebuild rather than accept, and Chapter 8. pointing the same primitives at an inbred-line programme, where the diversity cost of running faster is measured directly. Before any of it, the crossing plan decides which candidates are seen at all (Chapter 9.), and the trial layout which locations each one is seen in (Chapter 10.).
6. Spending the budget optimally is a combinatorial problem with structure worth naming (Chapter 19., Chapter 20.).
7. The budget is worth paying because of what it leaves for later cycles, which is a claim about trajectories and is checked as one (Chapter 22.) – and then predicted, which turns out to work only where the constraint holds (Chapter 23.).

The parts are ordered so that each depends only on what came before:

Part	What it establishes
Foundations	The coancestry kernel, the partition of diversity in a subdivided population, and the equation that says what selection buys
Simulation	How the candidate pool is generated, and how large a segregating population has to be
Mating and testing design	Which few hundred of the 40,000 possible crosses to grow, what each design can estimate, and how to spread them over locations
Metrics	What can be measured, what it costs, and which measurements are redundant
Optimisation	How the constraint enters an optimiser, and which optimiser to use
Evidence and practice	What the literature does and does not establish, and a worked end-to-end run

Readers who only want the operational answer can read Chapter 15. (which metrics), Chapter 16. (at which stage) and Chapter 24. (the whole pipeline in one script), then come back for the derivations.

Running the code

All code is in the `hybdiv` package that ships with this book.

```
# from the repository root
install.packages(c("DEoptim", "quadprog"))
devtools::install(".")
```

```
library(hybdiv)
st1 <- stage1_simulate()                                  # lines -> X, f, hybrids
res <- stage2_select(st1$X, st1$f, st1$hybrids, n_sel = 100, fL = st1$fL)
res$metrics
```

Every live example in the book runs at a deliberately small scale – 25 lines per pool and 2000 markers, giving 625 candidate hybrids – so that a full render takes minutes. The scale is set once, in `_common.R`:

```
str(book_cfg)
#> List of 6
#>  $ n_pool_A: num 25
#>  $ n_pool_B: num 25
#>  $ m       : num 2000
#>  $ n_traits: num 3
#>  $ fst     : num 0.15
#>  $ seed    : num 1
```

Results that are expensive at any scale – the differential-evolution sweeps, the cost benchmark, the population-sizing grid – are precomputed and shipped in `book/data` and `book/figs`. `book/scripts/regenerate.R` regenerates all of them, and Appendix C. says exactly which figures and tables come from where.

A note on numbers: everything reported here is reproducible output from a simulated dataset, not a fixed constant. Change the simulation configuration or a random seed and the numbers move. The *relationships* between them – which metric dominates which, which identity holds exactly – do not.

I Foundations

1 Coancestry, and the two matrices . . 21
1.1 From line to hybrid . . . 21
1.2 The VanRaden matrix, and why it cannot measure diversity . . . 21
1.3 Molecular coancestry . . . 22
1.4 `f` is not an ad hoc substitute . . . 23
1.5 The division of labour . . . 24
1.5 Reproduce this chapter . . . 24

2 Diversity in a subdivided population . . . 25
2.1 Notation . . . 25
2.2 The three coancestry blocks . . . 25
2.3 Two per-subpopulation metrics . . . 26
2.4 Distances, and what they actually are . . . 26
2.5 The equation that matters most in practice . . . 27
2.6 F statistics and the partition . . . 27
2.7 The worked example, reproduced . . . 27
2.8 Minimising coancestry is maximising N_e . . . 29
2.9 The honest caveats . . . 29
2.10 Two errata, found while verifying . . . 30
2.11 What manages this quantity, and where . . . 30
2.11 Reproduce this chapter . . . 30

3 Response to selection: the other half of the objective . . . 31
3.1 The breeder's equation, and how well it does here . . . 31
3.2 Genic variance, and what it has to do with gene diversity . . . 32
3.3 The decomposition this book already had . . . 33
3.4 What `weights = NULL` is assuming . . . 33
3.5 The α constraint is a restricted index . . . 34
3.6 Where this connects . . . 35
3.6 Reproduce this chapter . . . 35

1. Coancestry, and the two matrices

Two matrices are built from the same marker data and they are not interchangeable. One predicts traits. The other measures diversity. Using the first for the second is the most consequential mistake available in this subject, and it fails silently – it returns a number, and the number is meaningless.

This chapter builds both, shows the failure, and derives the matrix used for the rest of the book.

1.1 From line to hybrid

Let $\mathbf{M}^L$ be the $L \times m$ matrix of line genotypes, with $M^L_{ik} \in \{0, 2\}$ the count of the reference allele at marker k. Inbred lines are homozygous, so the value 1 never occurs – residual heterozygotes must be imputed or discarded beforehand.

The F1 hybrid between lines a and b receives one gamete from each parent. Since both parents are homozygous, the gamete from a carries $M^L_{ak}/2$ copies of the reference allele, so

$$M^H_{(ab)k} = \frac{M^L_{ak} + M^L_{bk}}{2} \in \{0, 1, 2\}. \tag{1.1}$$

The value 1 appears exactly at markers where the two lines differ. Those are the hybrid's heterozygous loci, and they are the ones carrying heterosis.

Predicting the hybrid's *genotype* is deterministic, not statistical. It is the one piece of information in the whole pipeline that requires no assumption at all – unlike its phenotype, which requires a great many.

From here on we work with the **within-individual reference-allele frequency**,

$$x_{ik} = \frac{M^H_{ik}}{2} \in \{0, 0.5, 1\}, \tag{1.2}$$

which is the natural quantity for coancestry: it is the probability that an allele drawn at random from individual i at marker k is the reference allele.

```
ctx <- simulate_data(book_cfg)
ctx$X[1:4, 1:8]
#>      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8]
#> [1,]  0.5  0.5  0.5    1    0    1  0.5  0.5
#> [2,]  0.5  0.0  0.0    1    0    1  1.0  0.0
#> [3,]  0.0  0.0  0.5    1    0    1  1.0  0.5
#> [4,]  0.0  0.0  0.0    1    0    1  1.0  0.5
```

Rows are hybrids, columns markers. A 0.5 is a heterozygous locus.

1.2 The VanRaden matrix, and why it cannot measure diversity

With p_k the reference-allele frequency in the hybrid population and $\mathbf{Z} = \mathbf{M}^H - \mathbf{1}\,2\mathbf{p}'$ the centered matrix,

$$\mathbf{G} = \frac{\mathbf{Z}\mathbf{Z}'}{2\sum_k p_k(1 - p_k)}. \tag{1.3}$$

This is VanRaden's first parametrisation. It estimates realised additive relationship and it is the right matrix for GBLUP. It is implemented as `vanraden_G()`:

```
args(vanraden_G)
#> function (M)
#> NULL
```

Now the failure. Centering on $2\mathbf{p}$ uses frequencies from the population itself, so every column of $\mathbf{Z}$ sums to zero:

$$\mathbf{Z}'\mathbf{1} = \mathbf{0} \quad \Rightarrow \quad \mathbf{1}'\mathbf{G}\mathbf{1} = \frac{\|\mathbf{Z}'\mathbf{1}\|^2}{2\sum_k p_k q_k} = 0. \tag{1.4}$$

The sum of **all** elements of $\mathbf{G}$ is identically zero. Group coancestry is proportional to that sum, so it is zero for the whole population, and the status number $N_s = 1/(2\theta)$ diverges:

```
c(sum_G      = sum(ctx$G),
  theta_on_G = mean(ctx$G) / 2,
  Ns_on_G    = 1 / (2 * mean(ctx$G) / 2))
#>         sum_G    theta_on_G       Ns_on_G
#> -4.178879e-11 -5.348298e-17 -9.348769e+15
```

That is not a rounding artefact to be worked around. It is exact, and it is what centering means. In a *subset* the sum is not zero, but the quantity is still centered: it oscillates around zero, takes negative values, is not a probability, and admits no identity-by-descent interpretation.

> ! The root cause is the implicit base population
>
> $\mathbf{G}$ measures relatedness *relative to the current population*. Diversity requires an *absolute* scale. Any matrix whose centering constant is estimated from the same data whose diversity you are measuring will have this problem, whatever it is called.

1.3 Molecular coancestry

Malécot's coancestry between i and j is the probability that two randomly sampled alleles, one from each individual at the same locus, are identical. Replacing identity *by descent* with identity *by state* – which is what markers actually observe – yields molecular coancestry [1], [2]:

$$f_{ij} = \frac{1}{m}\sum_{k=1}^{m}\Big[x_{ik}\,x_{jk} + (1-x_{ik})(1-x_{jk})\Big]\,. \tag{1.5}$$

The first term is the probability that both sampled alleles are the reference allele; the second, that both are the alternative. Three properties follow immediately:

- $f_{ij} \in [0,1]$ always. **A negative value is a symptom of the wrong matrix.**
- Self-coancestry is $f_{ii} = (1+F_i)/2$, with F_i the individual's molecular homozygosity. For a fully heterozygous F1, $f_{ii} = 0.5$.
- No centering, no scaling, no arbitrary base population.

1.3.1 Computable form

The double sum is $O(n^2 m)$ if computed naively. Expanding,

$$\sum_k \big[\, x_{ik}x_{jk} + (1-x_{ik})(1-x_{jk})\big] = 2\sum_k x_{ik}x_{jk} - \sum_k x_{ik} - \sum_k x_{jk} + m, \tag{1.6}$$

so with $\mathbf{s} = \mathbf{X}\mathbf{1}$ the row sums,

$$\mathbf{f} = \frac{1}{m}\Big(\,2\,\mathbf{X}\mathbf{X}' - \mathbf{s}\mathbf{1}' - \mathbf{1}\mathbf{s}' + m\,\mathbf{J}\,\Big), \tag{1.7}$$

a single matrix product – the same cost as building $\mathbf{G}$.

```
molecular_coancestry
#> function (X)
#> {
#>     m <- ncol(X)
#>     s <- rowSums(X)
#>     f <- (2 * tcrossprod(X) - outer(s, rep(1, nrow(X))) - outer(rep(1,
#>         nrow(X)), s) + m)/m
#>     dimnames(f) <- NULL
#>     f
#> }
#> <bytecode: 0x7bd5a5b8c0>
#> <environment: namespace:hybdiv>
```

```
fmt(c(minimum       = min(ctx$f),
      maximum       = max(ctx$f),
      diagonal_mean = mean(diag(ctx$f)),
      offdiag_mean  = mean(ctx$f[upper.tri(ctx$f)])))
#>       minimum       maximum diagonal_mean  offdiag_mean
#>        0.6448        0.8318        0.8192        0.6717
```

Everything is inside $[0, 1]$, as it must be.

1.3.2 Reading the diagonal

The value 0.5 is the floor of the diagonal, reached by a hybrid heterozygous at *every* marker. The excess above 0.5 measures exactly the fraction of markers at which the two parent lines carry the same allele – loci where the F1 is homozygous and therefore contributes nothing to heterosis.

```
fd <- mean(diag(ctx$f))
fmt(c(diagonal_mean         = fd,
      molecular_F           = 2 * fd - 1,     # f_ii = (1 + F_i) / 2
      heterozygous_fraction = 2 * (1 - fd)))
#>         diagonal_mean           molecular_F heterozygous_fraction
#>                0.8192                0.6383                0.3617
```

This fraction depends directly on the divergence between the heterotic pools (`fst` in the configuration): more divergent pools give more heterozygous F1 hybrids and a diagonal closer to 0.5. That relationship is what Chapter 4. puts under experimental control.

1.4 `f` is not an ad hoc substitute

Two independent lines of work converge on this matrix, which is worth stating because it is otherwise easy to read $\mathbf{f}$ as "the thing we used because $\mathbf{G}$ broke".

Nejati-Javaremi et al. (1997) proposed the *allelic-similarity* method, $f_{ij} = \frac{1}{4}\sum_{k=1}^{2}\sum_{l=1}^{2}\delta_{kl}$ with $\delta_{kl} = 1$ when the k-th allele of i equals the l-th of j. That is exactly the identity-by-state average above, generalised from single loci to a genome-wide mean. F. Gómez-Romano, B. Villanueva, J. Fernández, J. A. Woolliams, and R. Pong-Wong [4] contrast it directly against crossproduct-based matrices: the allelic-similarity matrix is guaranteed to stay in $[0, 1]$ by construction, while crossproduct matrices normalised by *current* allele frequencies are not – precisely the failure demonstrated above.

The same matrix arrives from a second direction. M. A. Toro, B. Villanueva, and J. Fernández [5] and T. H. E. Meuwissen, A. K. Sonesson, G. Gebregiwergis, and J. A. Woolliams [6] show that fixing the VanRaden centering frequency at $p_k = 0.5$ for every marker, instead of the sample frequency, produces a matrix – written $\mathbf{G}_{0.5}$ – that is proportional to homozygosity-based inbreeding and molecular coancestry. Structurally that is $\mathbf{f}$: replacing $2p_k$ with 1 removes the population-dependent term that forced $\mathbf{1}'\mathbf{G}\mathbf{1} = 0$.

So $\mathbf{f}$ is the member of the VanRaden family that has an absolute scale, reached independently from two unrelated derivations. That it also happens to be the one that does not break is not a coincidence.

1.5 The division of labour

Matrix	Centering	Range	Sum of all elements	Use
`G` (VanRaden)	sample allele frequencies	centered, signed	exactly 0	trait prediction (GBLUP)
`f` (molecular coancestry)	none	[0, 1]	positive, informative	every diversity quantity

This split is enforced throughout `hybdiv`: every metric takes `ctx$f`, and `ctx$G` appears only where traits are predicted. The test suite asserts `sum(G) == 0` precisely so that a future change which silently starts feeding `G` to a diversity metric fails loudly.

The next chapter takes **f** and asks what happens when the population it describes is not one population, but several.

Reproduce this chapter

All the code of this chapter, in order and with the explanation kept as comments, is one script: **01-coancestry.R**. Download it, install `hybdiv`, and run it top to bottom in an empty directory; cached results are fetched on first use.

2. Diversity in a subdivided population

A breeding programme is never one population. It is heterotic groups, or accessions, or landraces, or cycles – and the diversity of the whole is not the average of the diversity of the parts. A. Caballero and M. A. Toro [1] give the partition that makes this precise, and it is the analytical backbone of everything that follows: the constraint in Chapter 18. is a special case of it, and the within/between asymmetry that makes plant hybrid breeding different from animal breeding is a statement about which term of it you are allowed to minimise.

The paper's central claim is uncomfortable and worth stating first: **a method that ranks accessions by distance alone can invert the correct conclusion.** An accession can be the most distant *because* it is internally fixed, and removing it can *raise* the diversity of the pool. We reproduce that result numerically below.

2.1 Notation

- A metapopulation of n subpopulations; subpopulation i has N_i individuals, $N_T = \sum_i N_i$.
- A tilde, $\tilde{x}$, is a weighted mean **within** subpopulations: $\tilde{f} = \sum_i f_{ii} N_i / N_T$.
- A bar, $\bar{x}$, is a mean over **all pairs** in the metapopulation, between-subpopulation pairs included: $\bar{f} = \sum_{i,j} f_{ij} N_i N_j / N_T^2$.
- Always $\bar{f} \leq \tilde{f} \leq \tilde{s}$. The differences between those three numbers *are* the partition of diversity.
- Everything is computed per locus and then averaged over loci. Never average frequencies first.

> ⚠ The subtle point, and the one most often got wrong
>
> Every coancestry here includes the pairs of an individual with **itself** – group coancestry in the sense of Cockerham. That is what makes $1 - f_{ii}$ exactly Nei's expected heterozygosity, and it is what makes the algebra close. Drop the diagonal and none of the identities below hold.

2.2 The three coancestry blocks

2.2.1 f_{ij}, between subpopulations

The probability that two alleles, one sampled at random in subpopulation i and one in j, are identical. From markers, $f_{ij} = \sum_k \tilde{p}_{k,i} \tilde{p}_{k,j}$.

Read it as **redundancy** between two materials. If you cross plants of i with plants of j, the progeny has expected inbreeding $F = f_{ij}$. It answers: what do I gain by recombining these two accessions?

In maize, f_{ij} between a Stiff Stalk pool and a Non-Stiff Stalk pool is typically low – and that low between-group coancestry is exactly what sustains the heterotic pattern. Between two elite lines of the same family it can exceed 0.6, and the cross generates nothing new.

2.2.2 f_{ii}, within a subpopulation

The case $i = j$. From markers $f_{ii} = \sum_k \tilde{p}_{k,i}^2$, the expected homozygosity, so

$$1 - f_{ii} = H_{e,i} = \text{Nei's gene diversity of accession } \; i. \tag{2.1}$$

A landrace accession regenerated twenty times with forty ears per cycle accumulates coancestry and watches f_{ii} rise silently. It is the cheapest alarm for genetic erosion *inside* a genebank – and it is precisely what a distance tree cannot see.

2.2.3 s_i and F_i, self-coancestry and molecular inbreeding

$$s_i = \sum_k \sum_l \frac{p_{k,l,i}^2}{N_i}, \qquad F_i = 2s_i - 1, \qquad 1 - s_i = H_{o,i} \tag{2.2}$$

with $p_{k,l,i}$ the dosage of allele k in individual l.

> ! F_i here is molecular, and is not F_{IS}
>
> It is measured against a hypothetical base in which all alleles are distinct. It is **not** comparable across studies using different marker panels. For the question "is this population mating at random?", use α_i below.

In soybean, rice and wheat, pure lines have $s_i \to 1$, so $F_i \to 1$ and $H_o \to 0$. That is expected, not a symptom. It changes only *where* the diversity resides.

2.3 Two per-subpopulation metrics

$$\alpha_i = \frac{F_i - f_{ii}}{1 - f_{ii}}, \qquad G_i = \frac{s_i - f_{ii}}{1 - f_{ii}} = \frac{1 + \alpha_i}{2} \tag{2.3}$$

α_i is the subpopulation's analogue of F_{IS}: positive means an excess of homozygotes (selfing, mating among relatives, a Wahlund effect from internal substructure, null alleles); negative means an excess of heterozygotes (dioecy, self-incompatibility, selection, or a freshly formed F1 or synthetic).

G_i is the proportion of the accession's diversity that sits **between** individuals rather than inside them, and it is probably the most underused quantity in the paper. It is a mating-system meter:

G_i	Situation	Typical material
~ 0.5	perfect panmixia	maize, sunflower, open populations
< 0.5	excess of heterozygotes	F1 synthetics, heterozygous clones, self-incompatibles
-> 1.0	all homozygous	inbred lines, doubled haploids, selfing species

Operationally, $1 - G_i$ is the fraction of an accession's diversity stored *inside* each plant, and it decides the sampling design. With $G_i \to 1$ (selfers), one individual captures almost nothing and you need many plants per accession. With $G_i \approx 0.5$ (maize), a few plants already capture half.

2.4 Distances, and what they actually are

Nei's minimum distance between subpopulations is

$$\mathbf{D}_{ij} = \frac{f_{ii} + f_{jj}}{2} - f_{ij} = \sum_k \frac{1}{2}(\tilde{p}_{k,i} - \tilde{p}_{k,j})^2. \tag{2.4}$$

Note what that is: **half the sum of squared allele-frequency differences** – a variance component, not an evolutionary distance calibrated in time. It is the piece that enters F_{ST}, and it is legitimate. The error in a distance-only method is not using $\mathbf{D}_{ij}$; it is using *only* $\mathbf{D}_{ij}$.

> A free $O(N^2)$ saving
>
> The paper's Eq. 13 computes D_{ij} by a double sum over all pairs of individuals, at cost $O(N_i N_j m)$. That is unnecessary: the identity $D_{ij} = (s_i + s_j)/2 - f_{ij}$ is **exact** and reduces the cost to $O(Nm)$. For a 50k-SNP panel and 5000 accessions this is the difference between hours and seconds.

2.5 The equation that matters most in practice

$$\bar{f} = \sum_i \frac{N_i}{N_T} \Big[\underbrace{f_{ii}}_{\text{internal coancestry}} - \underbrace{\sum_j \mathbf{D}_{ij} N_j / N_T}_{\text{mean distance to the others}} \Big] \tag{2.5}$$

Each accession's contribution to global coancestry – that is, to the *loss* of diversity in the pool – has **two terms of opposite sign**:

- **Term 1 (+).** The more internally fixed an accession is, the more it *raises* global coancestry: it dilutes the pool with copies of the same allele.
- **Term 2 (–).** The more distant it is from the others, the more it *reduces* global coancestry.

A distance-only method sees term 2 alone. That is the whole mechanism of the inverted conclusions.

2.6 F statistics and the partition

$$F_{IS} = \frac{\tilde{F} - \tilde{f}}{1 - \tilde{f}}, \qquad F_{ST} = \frac{\tilde{f} - \bar{f}}{1 - \bar{f}} = \frac{\bar{D}}{1 - \bar{f}}, \qquad F_{IT} = \frac{\tilde{F} - \bar{f}}{1 - \bar{f}} \tag{2.6}$$

with $(1 - F_{IT}) = (1 - F_{IS})(1 - F_{ST})$.

The partition itself is the heart of the paper:

$$(1 - \bar{f}) = \underbrace{(1 - \tilde{s})}_{GD_{WI}} + \underbrace{(\tilde{s} - \tilde{f})}_{GD_{BI}} + \underbrace{(\tilde{f} - \bar{f})}_{GD_{BS}} \tag{2.7}$$

Component	Definition	Biologically
GD_T = 1 - f_bar	total diversity	He of the whole pool if everything were recombined
GD_WI = 1 - s_til	within individuals	heterozygosity carried inside each plant
GD_BI = s_til - f_til	between individuals, within accession	segregating variation available to within-population selection
GD_WS = 1 - f_til	within subpopulations	GD_WI + GD_BI
GD_BS = f_til - f_bar	between subpopulations	divergence; the only component a distance method sees

with the clean identities

$$\frac{GD_{WI}}{GD_T} = (1 - G)(1 - F_{ST}), \quad \frac{GD_{BI}}{GD_T} = G(1 - F_{ST}), \quad \frac{GD_{WS}}{GD_T} = 1 - F_{ST}, \quad \frac{GD_{BS}}{GD_T} = F_{ST} \tag{2.8}$$

2.6.1 Why the partition changes the strategy by species

- **Outcrossers** (maize, sunflower, forages, forest trees): GD_{WI} is large. The diversity is literally *inside* the plants. Conserving means maintaining heterozygosity – regenerate with many individuals and balanced contributions.
- **Selfers** (soybean, wheat, rice, beans) **and DH or line collections**: $GD_{WI} \approx 0$. All diversity is $GD_{BI} + GD_{BS}$. Conserving means maintaining the *number of distinct lines*; heterozygosity is not a target. Regenerating with few plants per line is acceptable; losing lines is not.
- **Clonal crops** (potato, sugarcane, fruit trees): GD_{WI} is high and **frozen** – it does not decay by drift while the clone exists. The N_e algebra does not apply without adaptation.

This is the reason a hybrid maize programme cannot manage diversity with a single pool-wide number: it needs GD_{WS} defended and GD_{BS} held, and those two pull in opposite directions under selection.

2.7 The worked example, reproduced

The paper's metapopulation: four subpopulations of N = 12, 4, 8, 6, three loci with 5, 3 and 3 alleles. Its published coancestry matrix:

```
f <- matrix(c(0.3391, 0.3021, 0.2535, 0.3229,
              0.3021, 0.8750, 0.3073, 0.3889,
              0.2535, 0.3073, 0.4766, 0.2483,
              0.3229, 0.3889, 0.2483, 0.3704), 4, 4, byrow = TRUE,
            dimnames = list(paste0("Sub", 1:4), paste0("Sub", 1:4)))
N <- c(12, 4, 8, 6)
w <- N / sum(N)
```

Note subpopulation 2: $f_{22} = 0.875$, so $H_e = 0.125$. It is nearly fixed internally. Now the global quantities, with the paper's published values beside them:

```
f_bar <- drop(w %*% f %*% w)
f_til <- sum(w * diag(f))
s_til <- 0.7056                          # published; needs individual genotypes

data.frame(
  quantity = c("f_bar", "f_til", "GD_T", "GD_WI", "GD_BI", "GD_BS", "F_ST"),
  computed = round(c(f_bar, f_til, 1 - f_bar, 1 - s_til, s_til - f_til,
                     f_til - f_bar, (f_til - f_bar) / (1 - f_bar)), 4),
  published = c(0.3256, 0.4535, 0.6744, 0.2944, 0.2521, 0.1279, 0.1897)
)
```

quantity	computed	published
f_bar	0.3256	0.3256
f_til	0.4535	0.4535
GD_T	0.6744	0.6744
GD_WI	0.2944	0.2944
GD_BI	0.2521	0.2521
GD_BS	0.1279	0.1279
F_ST	0.1897	0.1897

Every value reproduces exactly. Note that GD_{BS} is only 19% of the total: the between-subpopulation divergence – the only thing a distance method measures – accounts for less than a fifth of the diversity in this metapopulation.

2.7.1 Removing an accession can *increase* diversity

```
res <- list(f = f, by_subpop = data.frame(N = N))
cbind(ct_loss_gain(res),
      delta_pct_equal = round(ct_loss_gain(res, "equal")$delta_pct, 2))
```

pop	GD_without	delta_pct	delta_pct_equal
Sub1	0.6296049	−6.6468853	−6.92
Sub2	0.6868734	1.8444502	6.47
Sub3	0.6480760	−3.9081294	−6.94
Sub4	0.6689694	−0.8102105	−3.56

The number to look at is subpopulation 2. A Weitzman-style analysis calls it the most valuable in the set – losing it would cost 65.5% of the diversity, because it is the most divergent. The coancestry analysis says removing it **raises** pool diversity by 1.8% (census weights) or 6.5% (equal weights).

It is distant *because* it is fixed. Alleles fixed in a sample of four individuals are not a genetic resource; they are the result of drift. The same pattern appears in the paper's reanalysis of eleven European pig breeds: the breed a distance method nominates as most valuable turns out to raise diversity by 0.67% when removed.

What this does and does not license

For a genebank: a rare accession with few plants and high homozygosity – the archetypal "unique" accession on a distance tree – may be contributing *negatively* to the pool. That does **not** mean discard it. It means the correct priority may be to **regenerate it at larger N**, not to replicate it into a core collection. Storing seed is cheap; extinction is irreversible.

2.7.2 Optimal contributions

Maximising $GD_T = 1 - \sum_{i,j} f_{ij} c_i c_j$ subject to $c_i \geq 0$ and $\sum_i c_i = 1$ is a quadratic programme:

```
oc <- optimal_contributions(f)
oc$c_i
#>     Sub1     Sub2     Sub3     Sub4
#> 0.522398 0.000000 0.269890 0.207711
c(GD_before = 1 - f_bar, GD_after = oc$GD_max,
  gain_pct = 100 * (oc$GD_max - (1 - f_bar)) / (1 - f_bar))
#> GD_before  GD_after  gain_pct
#> 0.6744338 0.6873675 1.9177220
```

Subpopulation 2 receives weight **zero**, and total diversity rises from 0.6744 to 0.6874. This reproduces the paper's published solution $\mathbf{c} = (0.522, 0, 0.270, 0.208)$ exactly.

Three direct uses in plant breeding:

- **Base population for recurrent selection.** c_i gives the seed proportion from each source that maximises long-term additive variance.
- **Core collection.** Instead of clustering heuristics plus proportional sampling, solve the quadratic programme directly with operational constraints ($c_i \leq$ multiplication capacity, $c_i \geq$ a policy minimum for accessions of cultural importance).
- **Optimal contribution selection.** Replace the objective with $\max \mathbf{c}'\mathbf{g} - \lambda\, \mathbf{c}'\mathbf{F}\mathbf{c}$ and you have Meuwissen's framework, now standard in genomic programmes. This chapter is the case $\lambda \to \infty$, pure diversity. A breeder operates in the middle of the curve – which is Chapter 18..

2.8 Minimising coancestry is maximising N_e

Since $N_s = 1/(2\bar{f})$, minimising group coancestry and maximising effective size are the same operation. The paper's Eq. 14 makes the operational consequence explicit. With **equal** contributions ($S^2_W = S^2_B = 0$), $N_e \approx 2Nn/(1 - F_{IT})$ – roughly **double** the N_e obtained under random (Poisson) contributions.

> Harvesting the same number of seeds from every plant, and the same number of plants from every family, doubles the effective size. No genotyping, no technology, no cost.

It is probably the best cost-benefit intervention available in germplasm management. The opposite – bulk harvest from a plot, which in practice samples the most productive plants disproportionately – inflates S^2_W and collapses N_e.

2.9 The honest caveats

The paper makes these, and a breeder should take them seriously.

Criteria based on within-population variation systematically **favour larger populations**. A $\bar{f}$ criterion can push a genebank toward abundant elite material – the opposite of the conservation objective. Beyond that:

- Neutral markers do not measure adaptive value, disease resistance, stress adaptation or cultural value. A fixed accession may carry the only resistance allele to an emerging rust.
- **Allelic richness** is more sensitive to bottlenecks than heterozygosity and is often the more relevant criterion for pre-breeding. The argument that minimising $\bar{f}$ maximises allelic richness in the long run

(via the effective number of alleles being the inverse of mean coancestry) is valid asymptotically but does **not** protect rare alleles in the short term. This is why allelic richness survives the redundancy analysis in Chapter 15. as the one genuinely independent axis.
- Weighting between-group variation more heavily is arbitrary. A more defensible fix is to optimise **c** with per-accession minimum constraints.

Rule of thumb: use $\bar{f}$ to decide *proportions and regeneration effort*; never use $\bar{f}$ alone to decide *discard*.

2.10 Two errata, found while verifying

Worth recording, since the numbers above were checked against the published tables.

- Table 2b lists $F_2 = 0.8383$, but $F_i = 2s_i - 1 = 2(0.9167) - 1 = 0.8333$. A typographical error. Every other value in Tables 2 and 3 reproduces exactly.
- Conceptually, the Discussion states that equal contributions ($S_B^2 = S_W^2 = 0$) are required "to minimise effective size" and "maximise mean coancestry". That is inverted: Eq. 15 and the entire argument of the paper show that equal contributions **maximise** N_e and **minimise** $\bar{f}$.

2.11 What manages this quantity, and where

This chapter is the book's keystone, and it is easy to read it as a self-contained treatment of Caballero & Toro. The partition it derives is not background: it is the object every later chapter operates on. Stated once, so the rest of the book can refer back rather than re-introduce it:

Term	What it is	Where it is managed
GD_{WI}	diversity within individuals	exactly zero with inbred parents (Chapter 14.); the hybrid-level $GD_{WI\text{ - }hyb}$ is what carries signal
GD_{BI}	between individuals, within pool	the quantity α budgets (Chapter 18.), and the one recurrent selection erodes (Chapter 22.)
GD_{BS}	between pools	the heterosis engine, to be **preserved** not minimised – derived as an exact identity in Chapter 13.

Three consequences follow, and each is a chapter.

A single pool-wide statistic cannot express "conserve within, hold between". That is why Chapter 14. reports the partition rather than only GD_T, and why Chapter 16. makes S1 a separate monitoring stage instead of folding pool conservation into the hybrid-selection constraint.

The between term is not a bonus to be maximised. Section 13.7 shows that under neutral drift every unit of GD_{BS} gained is paid for out of the shared term, so pushing the pools apart buys nothing. The partition tells you where diversity *is*, not which direction to move it.

The within term is the one with a future. GD_{BI} is the raw material of next cycle's response, which Chapter 3. makes precise: it is the same sum over the same loci as genic variance, weighted equally instead of by squared effect. That is the entire reason a diversity constraint is worth writing down, and Chapter 23. is where the equivalence is pushed until it breaks.

Reproduce this chapter

All the code of this chapter, in order and with the explanation kept as comments, is one script: **02-subdivided.R**. Download it, install `hybdiv`, and run it top to bottom in an empty directory; cached results are fetched on first use.

3. Response to selection: the other half of the objective

Every chapter so far measures what selection *spends*. None of them writes down what it *buys*.

That asymmetry is not cosmetic. Chapter 22. will show a constrained programme overtaking an unconstrained one after eight cycles, and the book can report that number because it simulated it – not because anything in the preceding chapters predicts it. A reader who wants to know whether the crossover arrives at cycle 8 in *their* programme has no way to find out except to re-run the simulation with their own parameters.

The quantitative genetics that answers such questions is older than any of the genomic machinery here, and this chapter installs it: the variance selection acts on, the equation that turns that variance into gain, and the classical index theory that turns out to be the direct ancestor of this book's α constraint.

```
sim <- simulate_pools(n_A = book_cfg$n_pool_A, n_B = book_cfg$n_pool_B,
                      m = book_cfg$m, fst = 0.15, n_qtl = 300, h2 = 0.5, seed = 1)
X  <- sim$X                          # N x m hybrids, 0/0.5/1
gv <- drop(X %*% sim$beta)           # true genetic value, known here by construction
I  <- sim$index                      # what the breeder actually sees
c(hybrids = nrow(X), markers = ncol(X), qtl = sum(sim$beta != 0))
#> hybrids markers     qtl
#>     625    2000     300
```

3.1 The breeder's equation, and how well it does here

Response to one round of truncation selection is

$$R = i\, r\, \sigma_A \tag{3.1}$$

with i the selection intensity (a property of the selected fraction alone), r the accuracy of the criterion as a predictor of genetic value, and σ_A the additive standard deviation. `sel_intensity()` has been in this package since Chapter 16.; it has never once been used to compute a response.

```
n_sel <- 60
N     <- nrow(X)
i_norm <- sel_intensity(n_sel / N)
r      <- cor(I, gv)
sel    <- order(I, decreasing = TRUE)[seq_len(n_sel)]

fmt(data.frame(i = i_norm, r = r, sigma_A = sd(gv),
               predicted = i_norm * r * sd(gv),
               realised  = mean(gv[sel]) - mean(gv)), 3)
```

i	r	sigma_A	predicted	realised
1.774	0.702	4.742	5.908	5.116

The equation overpredicts, by about 13 percent. That is worth taking apart rather than rounding away, because both halves of the discrepancy are instructive.

```
i_real <- (mean(I[sel]) - mean(I)) / sd(I)
b      <- cov(gv, I) / var(I)                # regression of value on criterion
fmt(data.frame(
  i_normal_theory = i_norm,
  i_realised      = i_real,
```

```
  R_with_realised_i = i_real * r * sd(gv),
  R_from_regression = b * (mean(I[sel]) - mean(I)),
  R_actual          = mean(gv[sel]) - mean(gv),
  index_skew        = mean(scale(I)^3)), 3)
```

i_normal_theory	i_realised	R_with_realised_i	R_from_regression	R_actual	index_skew
1.774	1.701	5.662	5.662	5.116	−0.17

Two separate failures, and they are different in kind.

The realised intensity is below normal theory. $i = \varphi(z)/q$ is exact for a normal criterion. This index is slightly left-skewed, so its upper tail is thinner than a normal's and the top 60 individuals sit closer to the mean than the formula assumes. This is a property of *this* population, not an error.

The regression is exact, and still wrong. `R_from_regression` and `R_with_realised_i` agree to machine precision – they are the same quantity written twice, since $b\,S_I = i\,r\,\sigma_A$ identically. Both still exceed the realised response. What fails is not the algebra but the assumption underneath it: $R = b\,S_I$ requires $E[g \mid I]$ to be linear in I *in the selected tail*, which is a bivariate-normality claim, and bivariate normality is exactly what a two-pool hybrid population does not have.

> ! Read the breeder's equation as an upper bound here
>
> Every response number in this book's cycle simulations is a *realised* response, measured from the simulation. Where a predicted response is quoted, it will run above the realised one for the reason above. The gap is not noise and does not shrink with more markers; it shrinks when the criterion becomes more normal and the population less structured.

3.2 Genic variance, and what it has to do with gene diversity

The additive variance in the equation above is not a fixed property of the population. Split it in two.

Genic variance is what the loci would contribute if they were independent:

$$V_{\text{genic}} = \sum_k \beta_k^2 \, \mathrm{Var}(x_k) \tag{3.2}$$

Realised variance is what the population actually has, $\mathrm{Var}(\mathbf{X}\boldsymbol{\beta})$, which is the sum of the *whole* covariance matrix rather than only its diagonal. The difference is exactly the gametic-phase disequilibrium:

$$\mathrm{Var}(\mathbf{X}\boldsymbol{\beta}) - V_{\text{genic}} = 2 \sum_{k<l} \beta_k \beta_l \, \mathrm{Cov}(x_k, x_l) \tag{3.3}$$

```
c(realised = var(gv),
  genic    = genic_var(X, sim$beta),
  gap      = var(gv) - genic_var(X, sim$beta),
  oracle_deviation = abs(genic_var(X, sim$beta) - ref_genic_var(X, sim$beta)))
#>         realised            genic              gap oracle_deviation
#>     2.248375e+01     2.520510e+01    -2.721357e+00     1.065814e-14
```

Now the sentence that ties this chapter to every one that follows.

V_{genic} is $\sum_k \beta_k^2 p_k q_k$ up to a coding constant, and gene diversity is $\frac{1}{m}\sum_k 2p_k q_k$. **They are the same sum over the same loci, weighted differently** – one by squared effect, one by nothing at all. That is the precise sense in which managing gene diversity manages the raw material of response, and it is why a diversity constraint is a defensible proxy for a variance constraint in the first place.

It is also the precise sense in which the proxy is incomplete. Gene diversity weights every locus equally, so it cannot distinguish a marker carrying half the genetic variance from one carrying none. And – more seriously –

> ⚠ No metric in this book can see the disequilibrium term
>
> θ, GD, N_s, allelic richness, `F_hom` and `F_drift` are all functions of allele frequencies **one locus at a time**. The covariance term above does not move a single allele frequency. A flat gene diversity trajectory therefore does not imply flat additive variance, and Chapter 23. measures how far apart the two can travel.

3.3 The decomposition this book already had

Additive variance is the variance of *breeding values*, not of genotypic values, and in a two-pool hybrid population the average effect of an allele substitution depends on which pool it is substituted into. That sounds like it needs new machinery. It does not: Section 13.10 already built the exact orthogonal split, and general combining ability *is* the additive part.

```
set.seed(7)
d  <- rnorm(ncol(X), 0.3, 0.1)                  # dominance deviations
U  <- sim$GL[sim$pool == "A", ]
V  <- sim$GL[sim$pool == "B", ]
gs <- heterosis_gca_sca(U, V, d)
H  <- heterosis_value(X, d)

nA <- nrow(U); nB <- nrow(V)
gA <- rep(gs$gca_A, times = nB)                 # ped varies pool A fastest
gB <- rep(gs$gca_B, each  = nA)
sc <- as.vector(gs$sca)

c(reconstruction_dev = max(abs(H - (gs$mu + gA + gB + sc))),
  var_H              = var(H),
  sum_of_components  = var(gA) + var(gB) + var(sc),
  identity_dev       = abs(var(H) - (var(gA) + var(gB) + var(sc))))
#> reconstruction_dev              var_H  sum_of_components       identity_dev
#>       5.115908e-13       3.811311e+01       3.811311e+01       9.237056e-14
```

The three components are orthogonal, so their variances add exactly. Read the shares:

```
p <- c(gca_A = var(gA), gca_B = var(gB), sca = var(sc))
fmt(data.frame(component = names(p), variance = p,
               share = p / sum(p)), 3)
```

	component	variance	share
gca_A	gca_A	9.064	0.238
gca_B	gca_B	8.673	0.228
sca	sca	20.376	0.535

`gca_A` and `gca_B` are the part of heterosis an additive model can express and that therefore behaves like additive variance under selection. `sca` is the part it cannot, and it does not accumulate across cycles the way the general components do. A programme selecting on an additive index is selecting on the first two and getting the third by luck.

3.4 What `weights = NULL` is assuming

`stage2_select()` defaults to equal weight on every trait, and every chapter of this book including Chapter 24. takes that default. It is a choice, not a neutral one.

The classical answer is the Smith-Hazel index (Smith 1936; Hazel 1943): the weights maximising the correlation between the index and the aggregate genotype $H = \mathbf{a}'\mathbf{g}$ are $\mathbf{b} = \mathbf{P}^{-1}\mathbf{G}\mathbf{a}$.

```r
P <- matrix(c(4.0, 1.2, 1.2, 1.0), 2)      # phenotypic covariance
G <- matrix(c(2.0, 0.9, 0.9, 0.3), 2)      # genetic covariance
a <- c(1, 1)                               # equal economic weight

fmt(data.frame(trait = c("yield", "moisture"),
               economic_weight = a,
               smith_hazel     = index_smith_hazel(P, G, a)), 3)
```

trait	economic_weight	smith_hazel
yield	1	0.570
moisture	1	0.516

Equal economic weights do **not** give equal index weights. They coincide only when $\mathbf{P}^{-1}\mathbf{G}$ is proportional to the identity, which requires the traits to be equally variable, equally heritable and mutually uncorrelated. Nothing guarantees that, and where it fails the index being maximised is not the index anyone intended.

3.5 The α constraint is a restricted index

Chapter 18. will argue that a diversity *budget* is approvable where a weighted sum is not: "I accept losing five percent" is a sentence a breeding committee can sign, and "diversity is worth λ index units" is not.

That argument is usually presented as a point about human decision-making. It is older and harder than that. Kempthorne & Nordskog (1959) posed exactly the same problem – maximise aggregate gain subject to **zero expected change** in a specified direction – and solved it:

$$\mathbf{b} = \mathbf{P}^{-1}\left[\mathbf{G}\mathbf{a} - \mathbf{C}(\mathbf{C}'\mathbf{P}^{-1}\mathbf{C})^{-1}\mathbf{C}'\mathbf{P}^{-1}\mathbf{G}\mathbf{a}\right], \qquad \mathbf{C} = \mathbf{G}\mathbf{R} \tag{3.4}$$

```r
R  <- cbind(c(0, 1))                       # hold moisture at zero gain
b_r <- index_restricted(P, G, a, R)
b_u <- index_smith_hazel(P, G, a)

fmt(data.frame(
  index = c("Smith-Hazel", "restricted"),
  w_yield    = c(b_u[1], b_r[1]),
  w_moisture = c(b_u[2], b_r[2]),
  gain_in_restricted_trait = c(drop(crossprod(R, G %*% b_u)),
                               drop(crossprod(R, G %*% b_r)))), 4)
```

index	w_yield	w_moisture	gain_in_restricted_trait
Smith-Hazel	0.5703	0.5156	0.668
restricted	−0.1207	0.3621	0.000

The restricted index moves the restricted trait by exactly zero – not approximately, not on average. That is the property distinguishing a restriction from a penalty, and it is the property α inherits.

> **i** What is genuinely new in this book, then
>
> Not the idea of constraining rather than pricing: that is 1959. What is new is the restricted direction. Kempthorne and Nordskog restricted a *trait*. This book restricts the loss of molecular gene diversity, which is not a trait, has no economic weight, and is measured on a matrix rather than a phenotype. The algebra of restriction carries over; the object being restricted does not.
>
> Stating the lineage makes Chapter 18. stronger, not weaker. A sixty-year-old theorem stands behind the move.

3.6 Where this connects

Chapter 12. and Chapter 14. measure the frequencies whose weighted sum is the genic variance defined here, which is why a diversity constraint is a variance constraint in disguise. 3 measures the r this chapter took as given, and Finding 5 of Chapter 15. shows what happens when it is assumed rather than measured. Chapter 18. is where the restricted index becomes the α constraint, and Chapter 23. is where the disequilibrium term this chapter isolated turns out to matter more than the equation suggests.

Reproduce this chapter

All the code of this chapter, in order and with the explanation kept as comments, is one script: **02b-response.R**. Download it, install `hybdiv`, and run it top to bottom in an empty directory; cached results are fetched on first use.

II Simulation

to F4 45
5.1 The simulated space is ancestry, not alleles 45
5.2 The assumptions, one at a time 45
5.3 Verifying the identities live 46
5.4 The results 47
5.5 Sizing the F1 51
5.6 Working numbers 51
5.7 Where this connects 51
5.7 Reproduce this chapter 52

6 Introgressing several genes by backcrossing 53
6.1 The operation, in one diagram 53
6.2 The model 54
6.3 Three criteria, and only one of them binds 54
6.4 Which selection index 55
6.5 How long to keep selecting recombinants 57
6.6 Allocating a fixed budget across BC1, BC2 and BC3 57
6.7 Fixing the stack: selfing against doubled haploids 58
6.8 Running the simulation 59
6.9 Assumptions, and what is not modelled 60
6.10 Where this connects 61
6.10 Reproduce this chapter 62

7 Simulating a hybrid breeding programme: AlphaSimR from the ground up 63
7.1 Step 1 – Founder genomes: what a founder is, and which engine to use 64
7.2 Step 2 – `SimParam`, and defining a trait with dominance 66
7.3 Step 3 – The base population, and where the variance components live 68
7.4 Step 4 – The DNA lottery, heterosis, and why an F1 is not a family 70
7.5 Step 5 – Selfing, doubled haploids, and why hybrid seed is remade every year 72
7.6 Step 6 – Phenotyping and selection: per se, then on the testcross 73
7.7 Step 7 – Two traits, and what equal weighting assumes 78
7.8 Step 8 – The parts assembled: reciprocal recurrent selection 80
7.9 Assumptions, and what this adds over Chapter 4. 89
7.10 Where this connects 89
7.10 Reproduce this chapter 90

8 A complete plant breeding programme, and what it costs in diversity 91
8.1 The scheme 91
8.2 Why a breeding programme needs a burn-in . . . 92
8.3 One programme, fifteen years 92
8.4 Genomic selection: two levers, not one 94
8.5 The comparison, replicated 94
8.6 The diversity ledger 96
8.7 Assumptions 96
8.8 Where this connects 97
8.8 Reproduce this chapter 97

4. Simulating lines, pools and crosses

Everything downstream of this chapter consumes three objects and knows nothing about where they came from. That is deliberate: it is the seam at which a simulation is swapped for real data, and it is the only place that has to change when it is.

This chapter builds those objects. It also builds the candidate pool the rest of the book selects from, so the choices made here – how divergent the pools are, how the index is constructed – determine what the later chapters have to work with.

4.1 What has to be produced

Object	Shape	Content	Needed for
`X`	N x m	hybrid marker matrix, values in 0/0.5/1	every frequency-based metric
`f`	N x N	molecular coancestry of the hybrids	every coancestry-based metric
`hybrids`	N x (3 + n_traits)	hybrid id, the two parent lines, predicted traits	the index, and the line-usage vector
`fL`	L x L, optional	coancestry BETWEEN LINES	only the heterotic-group decomposition

4.2 Simulating the lines

Two heterotic pools are generated with a Balding-Nichols model. Ancestral allele frequencies are drawn uniform on $[0.05, 0.95]$, then each pool's frequency at marker k is drawn as

$$p_{A,k} \sim \mathrm{Beta}\left(p_k \frac{1-F_{ST}}{F_{ST}}, (1-p_k)\frac{1-F_{ST}}{F_{ST}}\right), \tag{4.1}$$

and independently for pool B. The Beta has mean p_k and variance $F_{ST}\,p_k(1-p_k)$, so the parameter `fst` in the configuration *is* the expected between-pool divergence – it is not a proxy for it.

Genotypes are then drawn as $2 \times \mathrm{Bernoulli}(p)$, which makes every line fully homozygous, as an inbred line should be.

```
simulate_lines
#> function (cfg)
#> {
#>     set.seed(cfg$seed)
#>     p_anc <- stats::runif(cfg$m, 0.05, 0.95)
#>     bn <- function(p, fst) {
#>         stats::rbeta(length(p), p * (1 - fst)/fst, (1 - p) *
#>             (1 - fst)/fst)
#>     }
#>     pA <- bn(p_anc, cfg$fst)
#>     pB <- bn(p_anc, cfg$fst)
#>     draw <- function(n, p) {
#>         matrix(2L * stats::rbinom(n * length(p), 1, rep(p, each = n)),
#>             nrow = n)
#>     }
#>     L <- rbind(draw(cfg$n_pool_A, pA), draw(cfg$n_pool_B, pB))
#>     storage.mode(L) <- "double"
#>     L
#> }
```

```
#> <bytecode: 0x7942b80ba0>
#> <environment: namespace:hybdiv>
```

```
L <- simulate_lines(book_cfg)
dim(L)
#> [1]   50 2000
table(as.vector(L[1:5, 1:200]))
#>
#>   0   2
#> 480 520
```

Only 0 and 2 appear: no residual heterozygotes. With real data those must be imputed or dropped before this point, because the hybrid-genotype identity in Chapter 1. assumes homozygous parents.

4.2.1 The divergence knob

`fst` is the single most consequential parameter in the simulation, because it sets how much of the pools' diversity sits *between* them rather than within. What that buys is less obvious than it looks:

```
fst_grid <- c(0.02, 0.05, 0.10, 0.15, 0.25, 0.40)
div <- vapply(fst_grid, function(fst) {
  cfg <- modifyList(book_cfg, list(fst = fst, m = 1500))
  Lx  <- simulate_lines(cfg)
  fL  <- molecular_coancestry(Lx / 2)
  A   <- seq_len(cfg$n_pool_A); B <- cfg$n_pool_A + seq_len(cfg$n_pool_B)
  # a hybrid A x B is heterozygous wherever its parents differ
  c(between_pool_f = mean(fL[A, B]),
    within_pool_f  = mean(c(fL[A, A], fL[B, B])),
    het_fraction   = 1 - mean(fL[A, B]))
}, numeric(3))

d <- data.frame(fst = fst_grid, t(div))
plot(d$fst, d$het_fraction, type = "b", pch = 19, ylim = c(0, 1),
     xlab = expression(F[ST]~"between pools"),
     ylab = "expected heterozygous fraction of the F1")
grid()
```

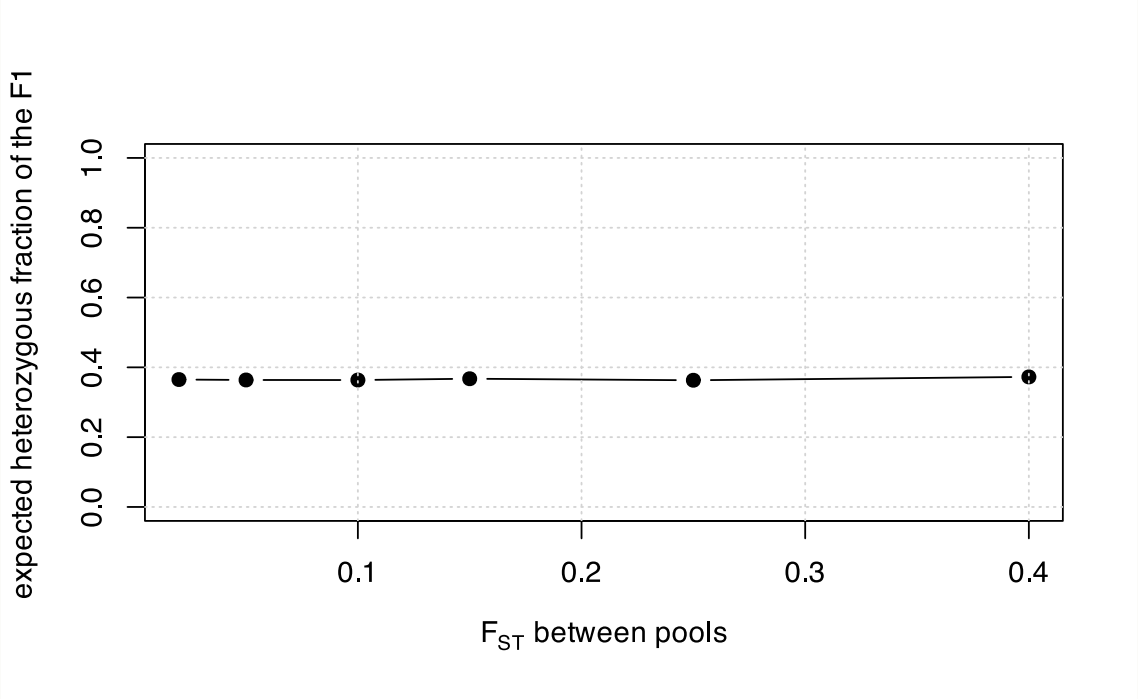

Figure 4.1: Driving the pools apart leaves the expected heterozygous fraction of the F1 flat. What moves is the within-pool coancestry, which climbs throughout. Section 13.7 derives why the two terms must trade one for one.

```
fmt(d)
```

fst	between_pool_f	within_pool_f	het_fraction
0.02	0.6349	0.6564	0.3651
0.05	0.6362	0.6684	0.3638
0.10	0.6364	0.6877	0.3636
0.15	0.6326	0.7052	0.3674
0.25	0.6370	0.7387	0.3630
0.40	0.6274	0.7898	0.3726

The heterozygous fraction does not move. It is flat across a twentyfold range of `fst`, while `within_pool_f` climbs steadily: the divergence is being paid for out of within-pool diversity, unit for unit.

That is not an artefact of this grid. The expected heterozygous fraction of an A x B hybrid is exactly `1 - theta_AB`, and Section 13.7 shows that under neutral drift every unit gained by the between-pool term is subtracted from the shared one, leaving the total fixed. Neutral divergence relocates diversity; it does not create heterosis. Chapter 2. is why `GD_BS` is nonetheless held rather than minimised, and Section 13.8 is what has to be true of the divergence for it to be worth anything.

4.3 From lines to hybrids

Every A x B combination is formed. With 25 lines per pool that is 625 candidates; at the production scale of 71 per pool it is 5041.

```
cfg <- book_cfg
a_ids <- seq_len(cfg$n_pool_A)
b_ids <- cfg$n_pool_A + seq_len(cfg$n_pool_B)
ped <- expand.grid(a = a_ids, b = b_ids)
M   <- (L[ped$a, , drop = FALSE] + L[ped$b, , drop = FALSE]) / 2   # 0/1/2
dim(M)
```

```
#> [1]  625 2000
head(ped, 3)
```

a	b
1	26
2	26
3	26

The factorial structure matters more than it looks. Because every line appears in many candidate hybrids, selecting hybrids is really selecting *line usage* – and that observation, made precise in Section 15.1, is what makes the whole problem tractable at production scale.

4.4 Predicted traits

Marker effects are drawn correlated across traits, through a Cholesky factor of a fixed correlation matrix, and the predicted values are the centered marker matrix times those effects:

```
set.seed(cfg$seed + 99)
R <- matrix(c(1, 0.3, -0.3,
              0.3, 1,  0.1,
             -0.3, 0.1, 1), 3, 3)
vr <- vanraden_G(M)
B <- matrix(rnorm(cfg$m * cfg$n_traits), cfg$m) %*% chol(R)
traits <- scale(vr$Z %*% B)
round(cor(traits), 3)
#>        [,1]  [,2]   [,3]
#> [1,]  1.000 0.259 -0.440
#> [2,]  0.259 1.000  0.049
#> [3,] -0.440 0.049  1.000
```

The realised trait correlations recover the target `R`, which is the point of the Cholesky step: without it, a multi-trait index would be a weighted sum of independent noise, and the trade-off it induces against diversity would be unrealistically easy.

> **i** What this simulation deliberately does not model
>
> These predicted values are constructed from *known* marker effects, so they carry no prediction error. Real GEBVs do. The book therefore treats `ctx$index` as given throughout, and the reader should read every gain figure as an upper bound on what a real programme would realise. The additive model also carries no dominance, so it cannot express specific combining ability – see the caveats in Chapter 24..

4.5 The two matrices, both built

```
X <- M / 2
ctx <- build_ctx(X = X, f = molecular_coancestry(X), G = vr$G, traits = traits,
                 ped = ped, n_lines = nrow(L),
                 pool = rep(c("A", "B"), c(cfg$n_pool_A, cfg$n_pool_B)),
                 fL = molecular_coancestry(L / 2), cfg = cfg)

c(N = ctx$N, m = ctx$m, n_lines = ctx$n_lines,
  sum_G = sum(ctx$G), min_f = min(ctx$f), max_f = max(ctx$f))
#>             N             m       n_lines         sum_G         min_f
#>  6.250000e+02  2.000000e+03  5.000000e+01 -4.178879e-11  6.447500e-01
#>         max_f
#>  8.317500e-01
```

`sum(G)` is zero to machine precision and `f` is inside $[0, 1]$: the two properties from Chapter 1., on this dataset.

The context also freezes two things that must never be recomputed per scenario:

```
str(ctx[c("p0", "maf_pop", "index")], max.level = 1)
#> List of 3
#>  $ p0     : num [1:2000] 0.44 0.3 0.56 0.9 0.12 0.98 0.98 0.7 0.64 0.22 ...
#>  $ maf_pop: num [1:2000] 0.44 0.3 0.44 0.1 0.12 0.02 0.02 0.3 0.36 0.22 ...
#>  $ index  : num [1:625] -0.291 0.498 -0.361 -0.186 -0.348 ...
```

`p0` is the candidate-pool allele frequency vector. It is the **base** against which `F_hom` and `F_drift` are measured in Chapter 12., and recomputing it per scenario would silently zero out the historical memory of erosion – the single most common error in applying these metrics.

4.6 The stage-1 contract in one call

All of the above is `stage1_simulate()`:

```
stage1_build
#> function (X, ped, traits, fL = NULL)
#> {
#>     if (max(X) > 1)
#>         X <- X/2
#>     stopifnot(nrow(X) == nrow(ped), nrow(X) == nrow(traits))
#>     hybrids <- data.frame(hybrid = seq_len(nrow(X)), line_A = as.integer(ped$a),
#>         line_B = as.integer(ped$b), traits)
#>     list(X = X, f = molecular_coancestry(X), hybrids = hybrids,
#>         fL = fL)
#> }
#> <bytecode: 0x7944608a18>
#> <environment: namespace:hybdiv>
```

```
st1 <- stage1_simulate(book_cfg)
str(st1, max.level = 1)
#> List of 4
#>  $ X      : num [1:625, 1:2000] 0.5 0.5 0 0 0 0.5 0.5 0 0 0 ...
#>  $ f      : num [1:625, 1:625] 0.818 0.74 0.743 0.738 0.745 ...
#>  $ hybrids:'data.frame': 625 obs. of  6 variables:
#>  $ fL     : num [1:50, 1:50] 1 0.677 0.692 0.674 0.684 ...
head(st1$hybrids, 3)
```

hybrid	line_A	line_B	trait1	trait2	trait3
1	1	26	−0.6880637	0.1907069	0.0153955
2	2	26	−0.6102738	−0.0320787	1.4661012
3	3	26	−0.3094099	−0.5013539	0.2137806

For real data the same function is called with your own matrices:

```
st1 <- stage1_build(
  X      = hybrid_markers,   # N x m, coded 0/1/2 or 0/0.5/1 (auto-detected)
  ped    = pedigree,         # data.frame with columns a, b
  traits = predicted_traits, # N x n_traits
  fL     = line_coancestry   # optional
)
```

`stage1_build()` detects the marker coding and rescales, so both conventions give an identical result:

```
tr <- st1$hybrids[, grep("^trait", names(st1$hybrids))]
pd <- data.frame(a = st1$hybrids$line_A, b = st1$hybrids$line_B)
identical(stage1_build(st1$X, pd, tr)$f,
          stage1_build(st1$X * 2, pd, tr)$f)
#> [1] TRUE
```

4.7 What the candidate pool looks like

```
op <- par(mfrow = c(1, 2), mar = c(4.2, 4.2, 2.5, 1))
hist(ctx$index, breaks = 40, col = "grey85", border = "white",
     main = "multi-trait index", xlab = "index (s.d. units)")
gd_random <- replicate(400, gene_diversity(sample.int(ctx$N, 60), ctx))
hist(gd_random, breaks = 30, col = "grey85", border = "white",
     main = "GD of random subsets of 60", xlab = "gene diversity")
abline(v = gene_diversity(seq_len(ctx$N), ctx), col = "#b2182b", lwd = 2)
```

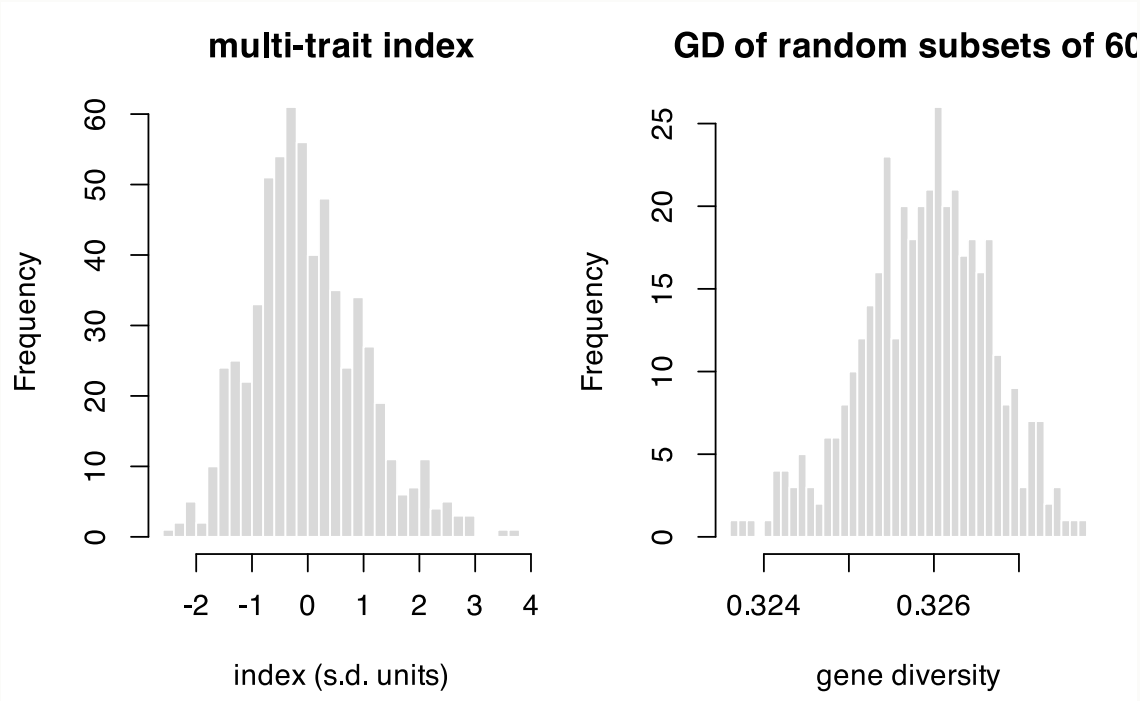

Figure 4.2: The candidate pool. Left: the index the selection maximises. Right: gene diversity of random subsets of 60, the baseline every later chapter measures against.

```
par(op)
```

The red line is the gene diversity of the *whole* pool. It does not sit at the centre of the distribution of random subsets of 60 – it sits above it. That gap is pure sampling, it has nothing to do with selection, and mistaking it for a loss is the subject of Section 14.9.

```
c(GD_population = gene_diversity(seq_len(ctx$N), ctx),
  GD_random_60  = mean(gd_random),
  apparent_loss_pct = 100 * (gene_diversity(seq_len(ctx$N), ctx) - mean(gd_random)) /
                           gene_diversity(seq_len(ctx$N), ctx))
#>     GD_population      GD_random_60 apparent_loss_pct
#>         0.3280928         0.3259023         0.6676555
```

Reproduce this chapter

All the code of this chapter, in order and with the explanation kept as comments, is one script: **03-simulating-crosses.R**. Download it, install `hybdiv`, and run it top to bottom in an empty directory; cached results are fetched on first use.

5. Sizing a segregating population, F1 to F4

The previous chapters took the candidate pool as given. This one asks where it comes from: a wide cross between two divergent parents, carried F1 -> F2 -> F3 -> F4, and the question every programme has to answer before it plants anything – **how many plants?**

The answer turns out to depend entirely on what you are sizing *for*. There is no single optimal N, and the most common failure is to size for the objective that saturates earliest and then be surprised when a different objective was never met.

5.1 The simulated space is ancestry, not alleles

Each genome position is coded 0 (from parent P1) or 1 (from parent P2). For a wide biparental cross between two divergent heterotic groups this is the correct abstraction: essentially every locus polymorphic between the parents is biallelic, so ancestry is a sufficient statistic. Throughout this chapter, "allele frequency" means "frequency of P1 ancestry".

5.2 The assumptions, one at a time

These are the assumptions the numbers rest on. Read them before trusting any of the numbers.

P1. Ancestry, not alleles. As above.

P2. Parents fully homozygous and fixed for alternative alleles. Consequently every F1 plant is genetically identical and the genetic N_e of the F1 is 1. Sizing the F1 is a **logistical** decision, not a genetic one – unless the parents carry residual heterozygosity, which is handled separately below.

P3. The genetic map. Ten chromosomes, lengths from a classical composite maize map, about 1670 cM (~17 Morgans). Do **not** use IBM-type map lengths from intermated RILs: they are inflated 5-10x by construction and will make every answer wrong.

```
map <- build_map(maize_chr_len, spacing_cM = 2)
c(chromosomes = length(map$chr_len), markers = map$m,
  total_cM = map$total_cM, morgans = map$total_cM / 100)
#> chromosomes     markers    total_cM     morgans
#>        10.0       846.0      1670.0        16.7
```

P4. Haldane recombination, no interference, identical in male and female. The probability of a haplotype switch between adjacent markers d cM apart is $r = 0.5(1 - e^{-2d/100})$; the first marker of each chromosome gets $r = 0.5$, which produces independent segregation between chromosomes automatically.

Limitation: without interference the variance in crossover number is inflated (Poisson). Maize has strong positive interference, so the simulated block-length distribution has a slightly longer tail than reality. Mean block lengths and allele frequencies are unaffected.

P5. Segregation distortion by rejection sampling, therefore exact rather than approximate. Two mechanisms: *gametophytic* (ga1/Ga1-s type, pollen incompatibility) acting only on the male gamete, and *zygotic* (hybrid lethality or subvitality) as genotype fitnesses. Because rejection acts on the **whole gamete**, the region linked to the distorter is dragged along – which is exactly the phenomenon of interest.

```
f2_distorters
```

chr	pos_cM	type	k	w00	w01	w11	label
4	75	gametic	0.80	NA	NA	NA	ga1-like (4.05)
1	160	zygotic	NA	1	1	0.45	partial lethal (1.10)

chr	pos_cM	type	k	w00	w01	w11	label
5	60	gametic	0.68	NA	NA	NA	weak gametic (5.03)

> ⚠ The distorters are plausible, not measured
>
> This is the parameter that moves the results most. Replace `f2_distorters` with your own F2 genotyping data before using any number here to decide field area.

P6. Five schemes, all ending with $H = 0.125$ so they are directly comparable: `ssd` (one seed per plant per generation – the neutral reference, no selection, no drift beyond the meiotic), `ssd_attr` (SSD with 15% random loss per generation: vigour, partial sterility, flowering asynchrony – common in wide crosses with exotic material), `bulk` (differential seed contribution, a Wright-Fisher model with weights $w_i = e^{\beta(z_i - \bar{z})}$ where z_i is the share of genome from the adapted parent – a *phenomenological* model of polygenic natural selection, not a model of adaptation loci), and `syn1`/`syn2` (one or two random intercrossing cycles in the F2 before the two selfings).

P7. The F4 panel is not a RIL panel. Residual heterozygosity is $H = 0.125$, and that changes every theoretical expectation:

Quantity	Formula	Value in F4
Var(dosage per line)	(1 - H) / 4	0.21875
SE(p_hat)	0.5 sqrt((1 - H) / N)	0.468 / sqrt(N)
Diversity retained D	1 - (1 - H) / N	1 - 0.875 / N
P(P2 homozygote per locus)	(1 - H) / 2	0.4375
Junctions per haplotype	A_2 = L; A_(t+1) = A_t + L H_t	1.75 L

P8. The standard error is estimated between replicates, not between markers. This is essential: linked markers are strongly autocorrelated, so the spread across markers within one replicate is not an estimator of sampling error.

P9. What is not modelled, and therefore cannot be answered here: deliberate artificial selection, QTL architecture, phenotypic inbreeding depression, mutation (irrelevant on this timescale), and structural variation. If you know there is a large inversion between your parents, zero the recombination rate over that interval in `build_map()`.

Known bias: at 2 cM spacing, double crossovers within an interval are not counted. The bias is -2.0% in junction number, always downward. Use `spacing_cM = 1` if you need a faithful count.

5.3 Verifying the identities live

The theory above is not decoration – it is checkable, and the simulator is only trustworthy if it reproduces it.

```
map10 <- build_map(maize_chr_len, spacing_cM = 10)
neutral <- make_distortion(map10, NULL, active = FALSE)

set.seed(4)
F1 <- make_F1(map10, 400)
F2 <- cross_indices(F1, 1:400, 1:400, map10, neutral)
F3 <- cross_indices(F2, 1:400, 1:400, map10, neutral)
F4 <- cross_indices(F3, 1:400, 1:400, map10, neutral)

data.frame(
  generation = c("F1", "F2", "F3", "F4"),
  H_observed = round(c(panel_het(F1), panel_het(F2), panel_het(F3), panel_het(F4)),
4),
  H_expected = c(1, 0.5, 0.25, 0.125)
)
```

generation	H_observed	H_expected
F1	1.0000	1.000

generation	H_observed	H_expected
F2	0.4968	0.500
F3	0.2541	0.250
F4	0.1268	0.125

Heterozygosity halves at every selfing, as it must. And the diversity identity:

```
set.seed(5)
check <- do.call(rbind, lapply(c(20, 50, 100, 200), function(N) {
  pop <- run_scheme("ssd", N, map10, neutral, f2_opt)
  H <- panel_het(pop)
  data.frame(N = N,
             D_observed = diversity_retained(panel_freq(pop)),
             D_expected = 1 - (1 - H) / N)
}))
fmt(check, 4)
```

N	D_observed	D_expected
20	0.9582	0.9569
50	0.9845	0.9825
100	0.9904	0.9912
200	0.9954	0.9956

Map expansion is an algebraic prediction, and the payoff is a result many programmes get wrong:

```
Lm <- map$total_cM / 100
rbind(SSD  = expected_junctions("ssd",  map),
      Syn1 = expected_junctions("syn1", map),
      Syn2 = expected_junctions("syn2", map))
#>       junctions mean_block_cM
#> SSD      29.225      42.57489
#> Syn1     37.575      35.10247
#> Syn2     45.925      29.86142
```

Each Syn cycle adds only $0.5L$ junctions, not $1.0L$. The extra meiosis creates a junction only where the parent is heterozygous, and $H = 0.5$ at the F2/Syn stage. Budgeting an intercrossing cycle as "one more map length of recombination" overstates what it buys by a factor of two.

5.4 The results

The full grid – five schemes, 20 population sizes, adaptive replication – takes several minutes and is shipped precomputed.

N	D	SE(p_hat)	skewed genome	min(p, 1-p)	N equivalent
10	0.903	0.148	42.8%	0.069	9
30	0.961	0.084	14.4%	0.187	23
50	0.972	0.066	7.9%	0.209	32
70	0.978	0.056	5.6%	0.221	40
90	0.981	0.047	4.3%	0.233	47
110	0.982	0.044	4.2%	0.232	48
150	0.984	0.039	3.8%	0.236	56
190	0.985	0.032	3.7%	0.227	60
290	0.988	0.027	3.3%	0.240	70
390	0.989	0.023	3.1%	0.242	77

5.4.1 The diversity curve saturates at 0.988, not at 1

```
schemes <- unique(f2$scheme)
cols <- c(ssd = "#1f5f8b", ssd_attr = "#7fb3d5", bulk = "#c0392b",
          syn1 = "#27ae60", syn2 = "#16a085")
plot(NA, xlim = range(f2$N), ylim = c(0.88, 1.0),
     xlab = expression(N[F2]~"plants"), ylab = "diversity retained, D")
grid()
for (s in schemes) {
  d <- f2[f2$scheme == s, ]
  lines(d$N, d$div, type = "b", pch = 19, cex = 0.6, col = cols[s], lwd = 1.8)
}
curve(1 - 0.875 / x, add = TRUE, lty = 2, col = "grey30", lwd = 1.5)
legend("bottomright", c(schemes, "theory, no distortion"),
       col = c(cols[schemes], "grey30"), lwd = c(rep(1.8, length(schemes)), 1.5),
       lty = c(rep(1, length(schemes)), 2), pch = c(rep(19, length(schemes)), NA),
       bty = "n", cex = 0.8)
```

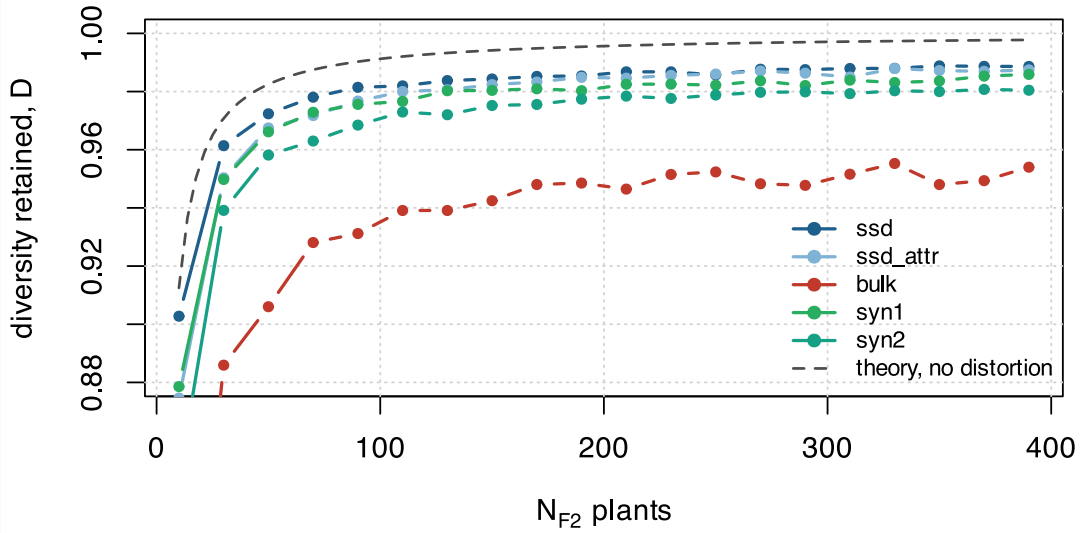

Figure 5.1: Diversity retained against population size, five schemes. The theoretical curve is 1 - (1 - H)/N under no distortion.

The gap between the observed curve and the theoretical one is **segregation distortion** – a deterministic bias that does not shrink with more plants. Adding plants buys you drift protection; it does not buy you distortion protection.

5.4.2 The bulk is the worst scheme across the whole range

comparison	N	D
bulk, largest N	390	0.954
SSD, N = 30	30	0.961

Bulk at the largest population size retains **less** diversity than SSD at 30 plants. With a less adapted donor parent, natural selection in the bulk removes precisely the germplasm the wide cross was made to introduce. If the objective is introgression, the bulk is not a cheaper SSD – it is a different experiment.

5.4.3 Block size does not depend on N

```
map4 <- build_map(maize_chr_len, spacing_cM = 4)
dist <- make_distortion(map4, f2_distorters)
set.seed(6)
small <- panel_freq(run_scheme("ssd", 20,  map4, dist, f2_opt))
large <- panel_freq(run_scheme("ssd", 300, map4, dist, f2_opt))

plot(small, type = "l", col = "#e08214", ylim = c(0, 1),
     xlab = "genome (chromosomes 1–10)", ylab = "P1 ancestry frequency", xaxt = "n")
lines(large, col = "#1f5f8b", lwd = 1.6)
abline(h = 0.5, lty = 2, col = "grey50")
# chromosome boundaries and distorter positions
bnd <- which(map4$first)
axis(1, at = (c(bnd, map4$m + 1)[-1] + bnd) / 2, labels = seq_along(map4$chr_len))
abline(v = bnd[-1] - 0.5, col = "grey85")
for (i in seq_len(nrow(f2_distorters)))
  abline(v = marker_index(map4, f2_distorters$chr[i], f2_distorters$pos_cM[i]),
         col = "#b2182b", lty = 3)
legend("topleft", c("N = 20", "N = 300", "distorter"), col = c("#e08214", "#1f5f8b",
"#b2182b"),
       lty = c(1, 1, 3), lwd = c(1, 1.6, 1), bty = "n", cex = 0.8, horiz = TRUE)
```

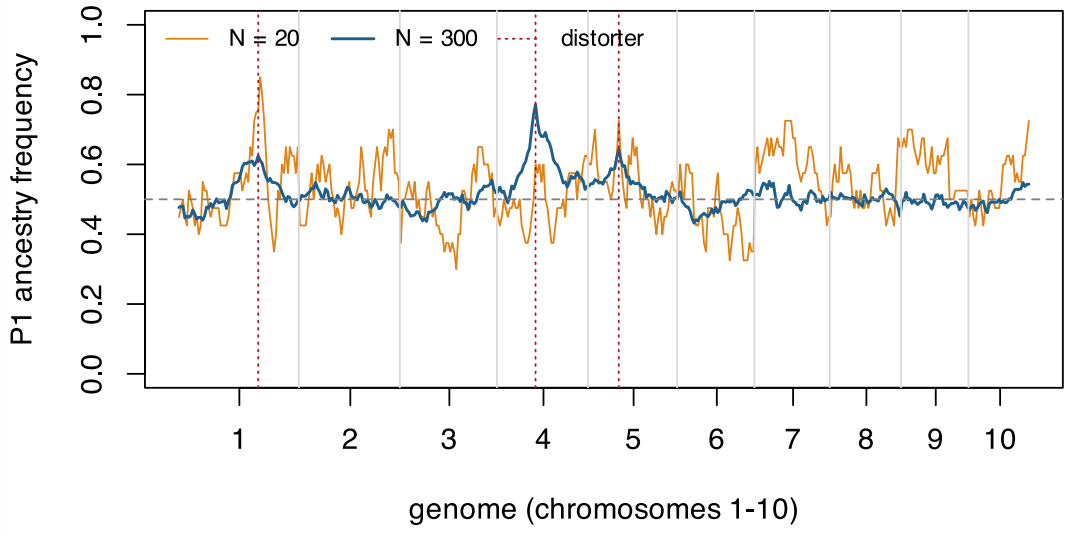

Figure 5.2: Ancestry frequency along the ten chromosomes, at N = 20 and N = 300. At N = 20 the whole curve is noisy: that is drift, and more plants remove it. At N = 300 the noise is gone and three peaks remain, exactly at the distorters. Those are the deterministic bias, and no number of plants removes them.

Block length depends only on the number of meioses. Against linkage drag the instrument is an **intercrossing cycle, not field area** – which is what the Syn curves in the theory table above quantify.

5.4.4 The metrics saturate at different N

This is the central practical result, and the reason the question "what is the optimal N?" has no single answer.

Objective	Saturates near
Diversity retained (D)	100-120
Worst region of the genome, min(p, 1-p)	60-80
Precision of the panel's p_hat	**never** – falls as 1/sqrt(N)
Recovery of 5 loci	200-260
Recovery of 8-10 loci	**not saturated** at the largest N tested

```
kk <- sort(unique(ml$k))
plot(NA, xlim = range(ml$N), ylim = c(0, 1), xlab = expression(N[F2]),
     ylab = "P(at least one line)")
grid()
for (i in seq_along(kk)) {
  d <- ml[ml$k == kk[i], ]
  lines(d$N, d$P, type = "b", pch = 19, cex = 0.6, col = i + 1, lwd = 1.8)
}
legend("topright", paste("k =", kk), col = seq_along(kk) + 1, lwd = 1.8,
       pch = 19, bty = "n")
```

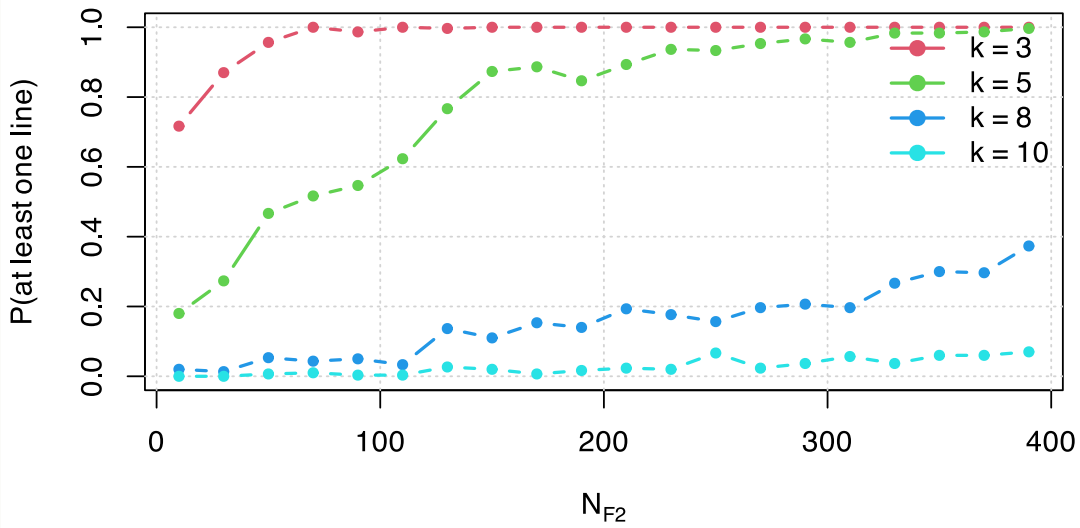

Figure 5.3: Multilocus recovery: P(at least one line homozygous for P2 at k loci). This is the objective that keeps paying for larger N.

The marginal gain in D tells the same story from the other side:

step	gain_in_D
10 -> 30	0.0586
30 -> 50	0.0110
50 -> 70	0.0057
70 -> 90	0.0034
90 -> 110	0.0005
110 -> 130	0.0018

The knee is early. Past it, twenty more plants buy thousandths.

5.5 Sizing the F1

With fully homozygous parents the F1 is genetically uniform and the decision is logistical. With residual heterozygosity – $F \approx 0.95$ is common in programme lines – each parent still segregates at a fraction of loci, and each F1 plant receives one random gamete from each parent. At a segregating locus the chance of losing one of the two parental alleles with n F1 plants is $2(1/2)^n$.

```
s <- f1_sizing(n_F1 = 1:20, h_res = 0.05, n_loci_eff = 5000)
s[s$n_F1 %in% c(4, 8, 10, 12, 15, 20), ]
```

	n_F1	p_loss_per_locus	expected_loci_lost
4	4	0.1250000	31.2500000
8	8	0.0078125	1.9531250
10	10	0.0019531	0.4882812
12	12	0.0004883	0.1220703
15	15	0.0000610	0.0152588
20	20	0.0000019	0.0004768

Twelve selfed F1 plants already put the expected number of lost loci below one. That is the whole F1 sizing argument.

5.6 Working numbers

Generation	Plants	Rationale
F1	8-15 selfed	Ne = 1, a logistical call. With 5% residual heterozygosity, 12 plants put expected lost loci below 1
F2	**120** to retain diversity	Above this, +280 plants buy +0.005 of D
	250-400 for multilocus selection or panel frequencies	These are the objectives that keep gaining with N
F3	same N (SSD, one seed per plant)	Does not create diversity, only avoids losing it
F4	lines x 3-8 plants	Seed multiplication. Plants of the same line are not independent replicates: they share 87.5% of the genome

> On the "equivalent N"
>
> $N_{eq} = (1 - H)/(1 - D)$ answers "how many F4 lines without distortion would give the diversity actually observed". It is a **communication device, not a Wright effective size**: it converts a deterministic bias into a drift equivalent. Use it to compare scenarios; never substitute it into an N_e formula.

5.7 Where this connects

The panel this chapter sizes is the raw material for everything else, and the claim that its diversity is a *ceiling* should be measured rather than asserted. The rest of the book works on one kernel – molecular coancestry f and $GD = 1 - \theta$ (Chapter 1.). The F4 panel can be pushed straight into it:

```
set.seed(3)
map4  <- build_map(spacing_cM = 10)
dist4 <- make_distortion(map4, active = TRUE)

bridge <- do.call(rbind, lapply(c(30, 120, 480), function(n) {
  pop <- run_scheme("SSD", n, map4, dist4, opt = f2_opt)
  X   <- t((pop$H1 + pop$H2) / 2)          # lines x markers, 0/0.5/1
```

```
  th  <- mean(molecular_coancestry(X))
  data.frame(n_F2 = n, D = diversity_retained(panel_freq(pop)),
             theta = th, GD = 1 - th)
}))
fmt(bridge, 4)
```

n_F2	D	theta	GD
30	0.9643	0.5179	0.4821
120	0.9795	0.5103	0.4897
480	0.9899	0.5050	0.4950

D, the chapter's own diversity-retained statistic, and GD, the quantity every later chapter constrains, move together: they are the same loss in two coordinate systems. The sizing decision made here therefore sets the GD that Chapter 15., Chapter 18. and Chapter 19. spend the rest of the book budgeting, and it sets it *once*. An optimiser cannot select back up to variation the panel never sampled; the best it can do is spend what is left more slowly.

> **i** N_s is not informative inside a biparental cross
>
> Computing the status number on these panels returns approximately 1 whatever the sizing, and that is correct rather than broken. Every line descends from two parents, so the panel has the genetic base of a single diploid individual, which is exactly what $N_s = 1/(2\theta)$ reports. Section 14.3 warns that N_s describes a base rather than projecting drift; here the base is genuinely one cross.
>
> Use D and GD to compare sizing scenarios within a cross, and N_s only once material from several crosses has been pooled.

Chapter 16. puts a number on how the loss is distributed across the whole pipeline, and this stage is where the cheapest insurance is available – more plants, once, at the start. Chapter 23. is where the same argument is made about the *variance* rather than the frequencies, and reaches a different conclusion about what is recoverable.

Reproduce this chapter

All the code of this chapter, in order and with the explanation kept as comments, is one script: **04-population-sizing.R**. Download it, install `hybdiv`, and run it top to bottom in an empty directory; cached results are fetched on first use.

6. Introgressing several genes by backcrossing

Chapter 5. sized a segregating population for *diversity*: the objective was to sample as much of the cross as possible. This chapter is the mirror image. A donor line carries k genes worth having and an elite line carries everything else worth having, and the job is to move the k genes across while **throwing the rest of the donor away**. Here donor genome is contamination, and the only diversity that is wanted is the k target loci themselves.

The operation is marker-assisted backcrossing (MABC). The numbers below come from a simulator of it: a marker grid at 5 cM on ten maize chromosomes (1670 cM), one selected plant advanced per generation, $k = 1$ to 4 recessive targets from a single donor, carried to BC3. It ships with the package – the engine and the selection index are `R/08_mabc.R`, the experiment driver is `inst/scripts/run_mabc.R` – and Section 6.8 shows how to run it.

6.1 The operation, in one diagram

Before any sizing question can be asked, the crossing operations have to be unambiguous – which plant is pollinated by which, and what is genotyped at each step.

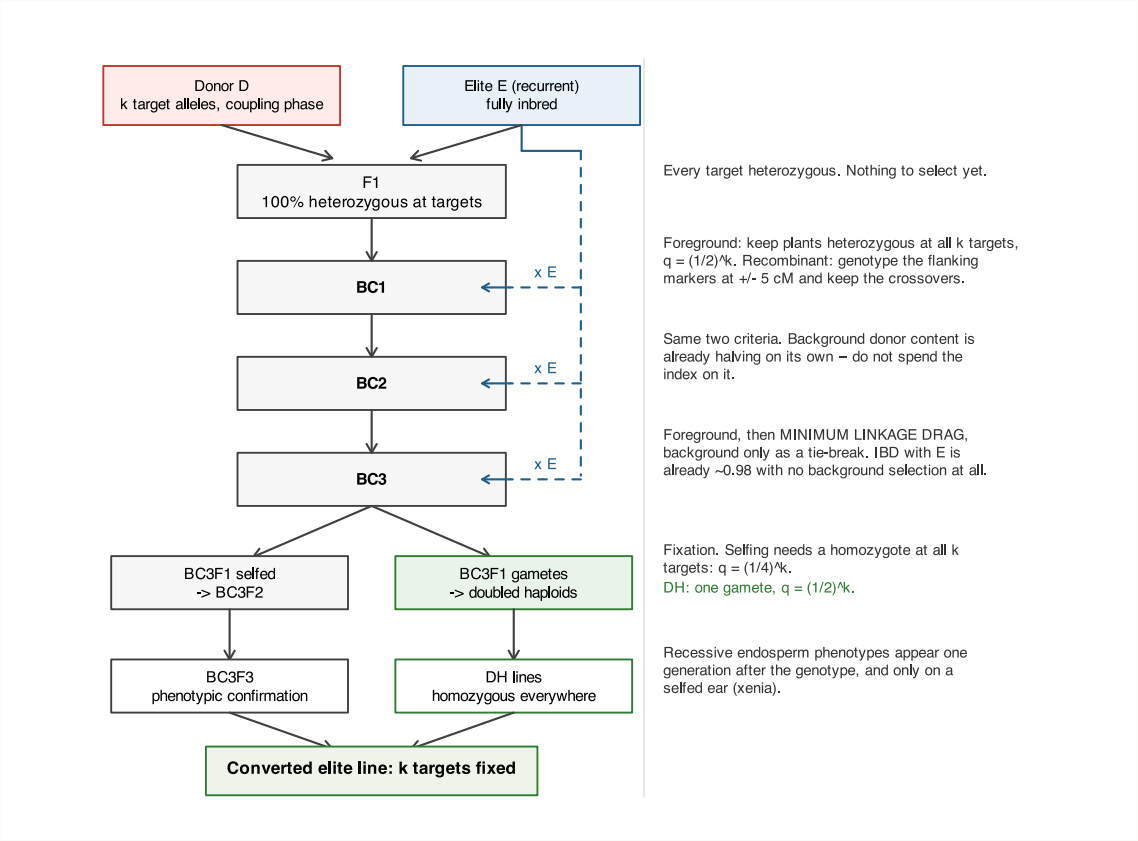

Figure 6.1: The MABC operation. Left column: the cross made in each generation. Right column: what is genotyped and selected on, and the population size that criterion demands. The recurrent parent E re-enters at every backcross; the fork at the bottom is the fixation step, where doubled haploids and selfing differ by more than an order of magnitude in plants required.

Two features of that diagram do all the work in the rest of the chapter. The recurrent parent re-enters at every backcross, so donor content halves every generation *whether or not anyone selects for it*; and the fork at the bottom is where the plant numbers explode, because fixation asks for a homozygote and backcrossing never does.

6.2 The model

Two states, one haplotype. The recurrent parent is a fully homozygous inbred, so the gamete it contributes is invariant and only the donor-derived haplotype has to be tracked: a binary vector over 334 markers. Meiosis is a per-interval Bernoulli switch with Haldane's

$$r = \frac{1}{2}\left(1 - e^{-2d/100}\right), \tag{6.1}$$

which is exact for this chain precisely because Haldane is memoryless (see the limitation in Section 6.9).

Donor dosage and IBD are different quantities. Donor and elite already share a fraction s of the genome by descent – they are both maize, and often both from the same heterotic group. Donor genome inside a shared tract costs nothing. So with D the donor dosage,

$$\mathrm{IBD}_E = 1 - (1 - s)\,D, \qquad \mathrm{IBD}_E + \text{donor-specific} = 1. \tag{6.2}$$

Both identities hold exactly in the simulator and are asserted in its test battery. Everything below uses $s = 0.6$.

Coupling phase. All k targets come from one donor, on the same haplotype. Two consequences, and the second is easy to get wrong. Linked targets travel *together*, so linkage drag is the **union** of the donor segments, not their sum. And conditional on both flanking targets being donor-derived, an *even* number of crossovers is forced in the interval between them, so the probability that they sit in one continuous donor segment is not e^{-L} but

$$P_{\text{same}} = \frac{e^{-L}}{\left(1 + e^{-2L}\right)/2}, \qquad L = d/100 \ \text{ Morgans}\ . \tag{6.3}$$

```
data.frame(d_cM = c(20, 60, 400),
           naive = exp(-c(20, 60, 400) / 100),
           conditional = p_same_segment(c(20, 60, 400)))
```

d_cM	naive	conditional
20	0.8187308	0.9803280
60	0.5488116	0.8435507
400	0.0183156	0.0366190

At 20 cM that is 0.98 against a naive 0.82. Published stacking simulations get this wrong often enough that it is worth stating separately.

> ! If the alleles come from different donors, none of this applies
>
> Coupling phase is an assumption, not a convenience. Two linked targets in repulsion, from two donors, cannot both be recovered without a recombinant in the interval, and the cost explodes. That programme needs an intercross between the donors before any backcrossing starts, and this model does not describe it.

6.3 Three criteria, and only one of them binds

A conversion programme is usually written with three success conditions at BC3: do not lose the genes, recover the recurrent parent, and do not drag a chunk of donor chromosome along with each target. They are not remotely equal in cost.

Not losing the genes is exact and cheap. A plant is foreground-positive with probability $q = (1/2)^k$, so n plants fail with probability $(1-q)^n$:

```
data.frame(genes = mb_sz$genes, q = mb_sz$q_foreground,
           n_BC_per_gen = vapply(mb_sz$q_foreground, n_for_count, integer(1)))
```

genes	q	n_BC_per_gen
1	0.5000	5
2	0.2500	11
3	0.1250	23
4	0.0625	47

Fewer than 50 plants per generation at four genes, and that is the whole foreground problem.

Recovering the elite is free by BC3. With no background selection at all, expected donor dosage after 3 backcrosses is $(1/2)^4 = 0.0625$, so

$$\mathrm{IBD}_E = 1 - (1-s)\,(1/2)^4 = 0.975 \tag{6.4}$$

at $s = 0.6$ – comfortably above a 90% target before a single background marker is scored.

genes	N = 20	N = 80	N = 200	N = 400
1	0.984	0.995	0.997	0.998
2	0.970	0.985	0.990	0.992
3	0.956	0.973	0.980	0.984
4	0.944	0.954	0.969	0.975

That leaves **linkage drag** – the donor chromosome still attached to each target – as the only criterion that a large population can buy. It is the criterion the rest of the chapter is about.

> **i** The reframing
>
> An index that spends its decisive stage on background recovery is spending it on the criterion that was already satisfied. This is not a hypothetical: it was the defect the technical review found in the first version of this simulator, and correcting it reverses the headline conclusion. The next section is that correction.

6.4 Which selection index

Four policies, all applied to foreground-positive plants only:

Key	Rule
`bg`	minimise background donor content
`rec12`	recombinant selection in BC1-BC2 only (classical practice), then background
`rec_bg`	recombinant selection throughout, **background** decides among survivors
`rec_drag`	recombinant selection throughout, **linkage drag** decides, background breaks ties

`rec_bg` is the natural-looking two-stage rule: truncate on drag, then pick the cleanest background among the survivors. Stage 2 discards stage 1. The plant with the least drag survives the truncation and then loses to a sibling that happens to carry less donor genome somewhere irrelevant.

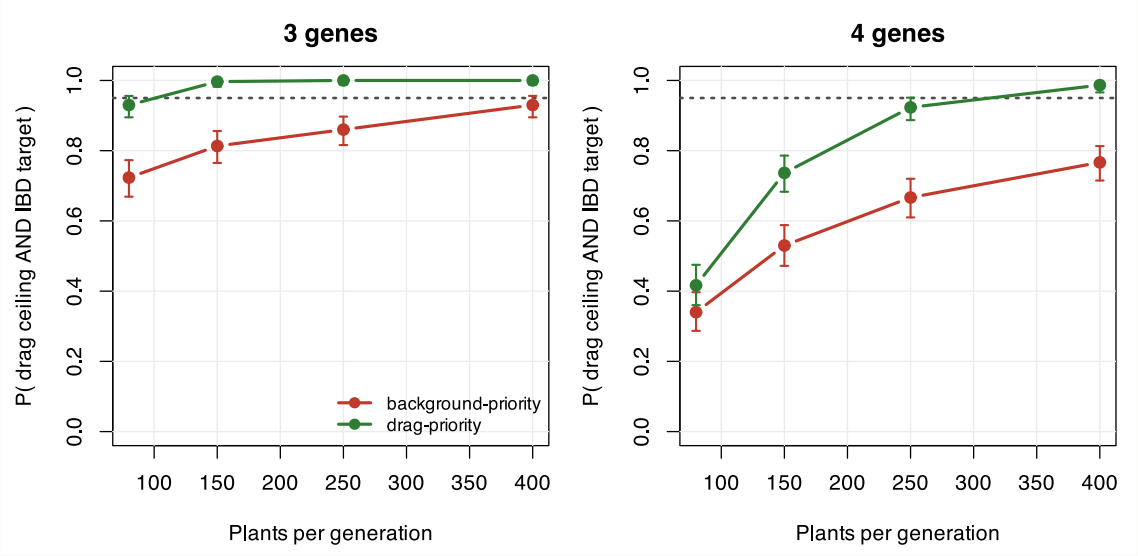

Figure 6.2: Paired comparison (common random numbers, 300 replicates, s = 0.60) of the two-stage background-priority index against the drag-priority index. Bars are Clopper-Pearson 95% intervals. The criterion is drag <= 30 cM per gene AND IBD >= 0.90 at BC3.

genes	N/gen	drag cM	IBD	P(both)	drag cM	IBD	P(both)
3	80	78.7	0.972	0.723	64.6	0.962	0.930
3	150	68.8	0.979	0.813	49.4	0.966	0.997
3	250	65.7	0.982	0.860	41.5	0.968	1.000
3	400	59.0	0.984	0.930	36.0	0.969	1.000
4	80	138.9	0.956	0.340	130.2	0.952	0.417
4	150	120.9	0.965	0.530	105.7	0.958	0.737
4	250	107.9	0.971	0.667	87.6	0.960	0.923
4	400	103.9	0.973	0.767	75.3	0.961	0.987

The trade is 1.1 percentage points of IBD for 19.2 cM of drag, and since the IBD constraint was met in every replicate with margin, it is free. Four genes reach 0.987 at 400 plants per generation. Under the background-priority index the same cell reaches 0.767, which is what produced the original – and wrong – conclusion that the ceiling is unreachable at four genes and the programme must be extended to BC4-BC5 or split in two. **The wall was a property of the index, not of maize.**

6.5 How long to keep selecting recombinants

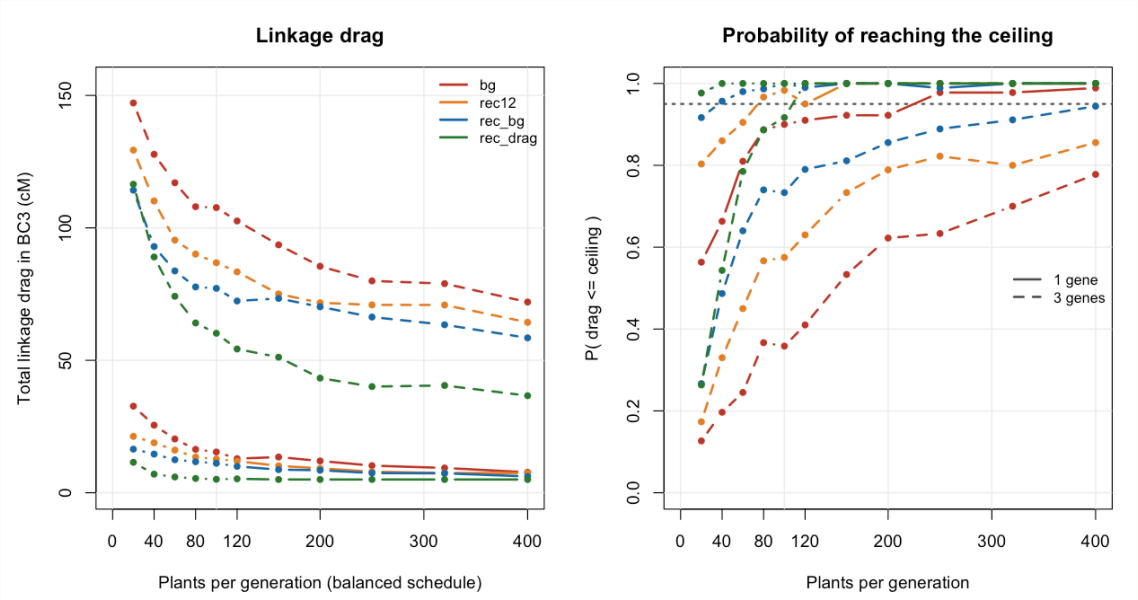

Figure 6.3: Linkage drag at BC3 (left) and probability of meeting the ceiling (right) under the four policies, s = 0.60. Solid = 1 gene, dashed = 3 genes. Extending recombinant selection into BC3 costs no extra plants, only the flanking markers on plants that are being genotyped anyway.

Classical practice stops recombinant selection after BC2, on the reasoning that BC3 is for background clean-up. By BC3 the background needs no clean-up, and the expected distance to the nearest crossover from a target shrinks as $1/g$ with generation g – so BC3 is the generation where a plant buys the *most* drag reduction. Both of the free changes in Figure 6.3 point the same way: genotype the flanking markers in BC3 as well, and let drag decide.

6.6 Allocating a fixed budget across BC1, BC2 and BC3

Given a total number of plants to grow, how should they be split between the three backcrosses?

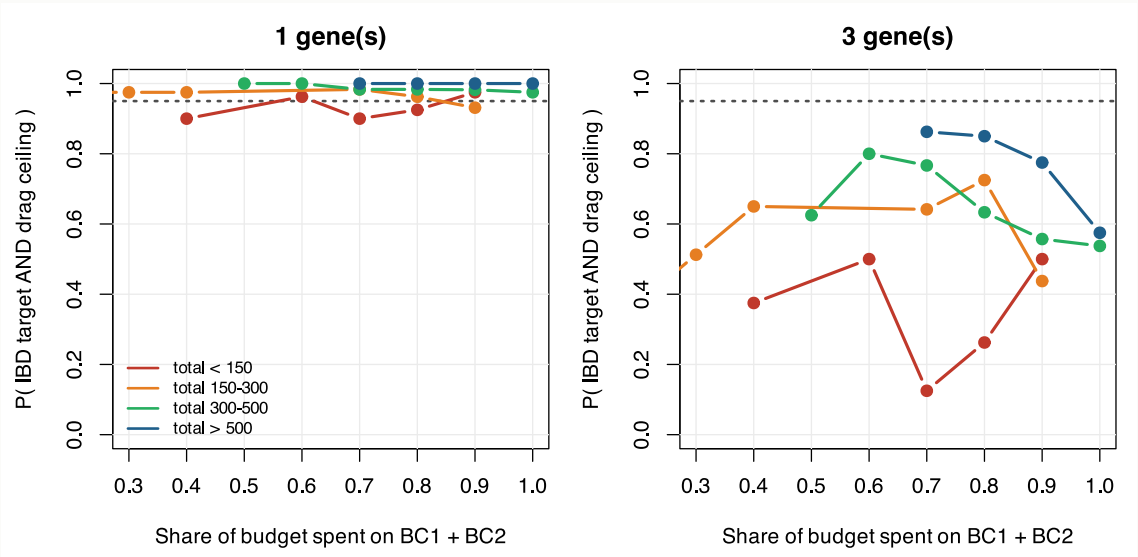

Figure 6.4: P(criterion met) against the share of a fixed budget spent early, for 1 and 3 genes, binned by total plants. Front-loading BC1 is the worst use of a fixed budget.

A paired re-run at a fixed budget of 360 plants, three genes, makes the ordering explicit:

BC1/BC2/BC3	drag cM	P(both)	lo	hi	drag cM	P(both)	lo	hi
300/40/20	93.0	0.448	0.385	0.512	75.6	0.656	0.594	0.715
120/120/120	72.6	0.772	0.715	0.823	53.5	0.988	0.965	0.998
20/150/190	78.7	0.640	0.577	0.700	58.9	0.876	0.829	0.914
40/120/200	74.3	0.748	0.689	0.801	58.1	0.940	0.903	0.966
200/120/40	79.7	0.680	0.618	0.737	59.3	0.932	0.893	0.960

The balanced schedule wins under both indices, and 300/40/20 – the intuitive "cast a wide net early" schedule – is the worst. The mechanism is the $1/g$ shrinkage again: a plant grown in BC3 is offered a much shorter donor segment to trim than a plant grown in BC1, so it converts into drag reduction far more efficiently.

6.7 Fixing the stack: selfing against doubled haploids

Backcrossing only ever asks for a heterozygote. Fixation asks for a homozygote at all k targets, and that is where the plant numbers move by an order of magnitude. Selfing a BC3F1 gives $q = (1/4)^k$; a doubled haploid derived from a BC3F1 gamete is homozygous everywhere, so it carries all k targets with $q = (1/2)^k$.

And the requirement is not one plant but 3, which is a *cumulative binomial* $P(\mathrm{Bin}(n, q) \geq 3) \geq 0.95$, not $1 - (1 - q)^n$:

genes	q (selfing)	BC3F2 plants	q (DH)	DH lines
1	0.25000	23	0.5000	11
2	0.06250	99	0.2500	23
3	0.01562	401	0.1250	49
4	0.00391	1610	0.0625	99

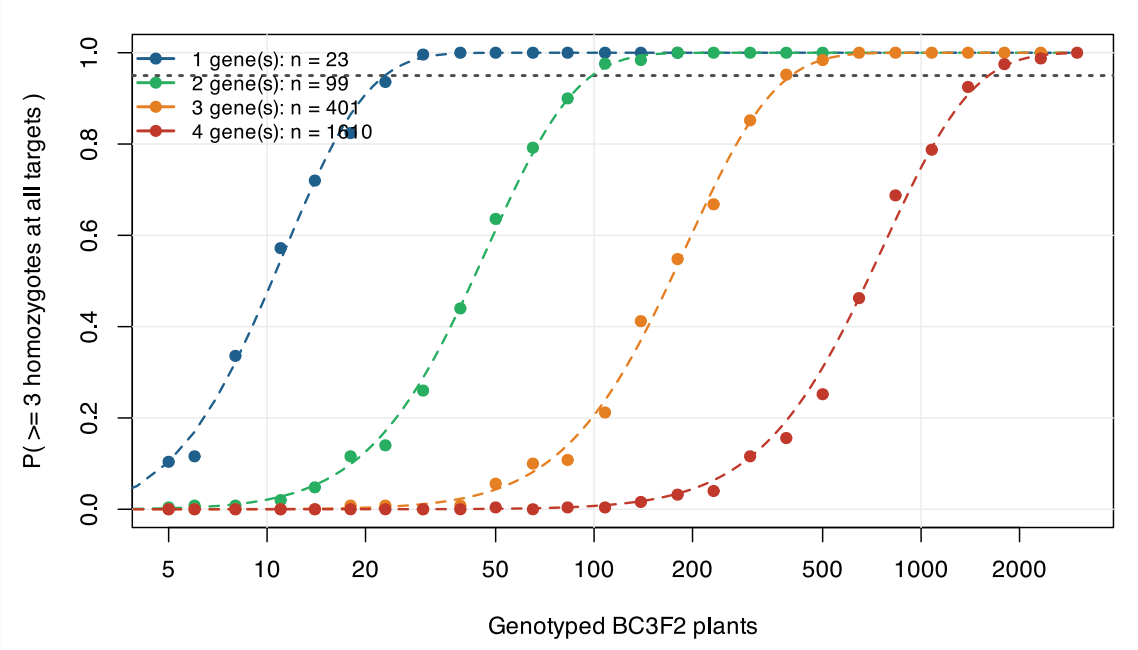

Figure 6.5: Points: simulation. Dashed: the cumulative binomial. Log scale. Each extra gene multiplies the BC3F2 requirement by four.

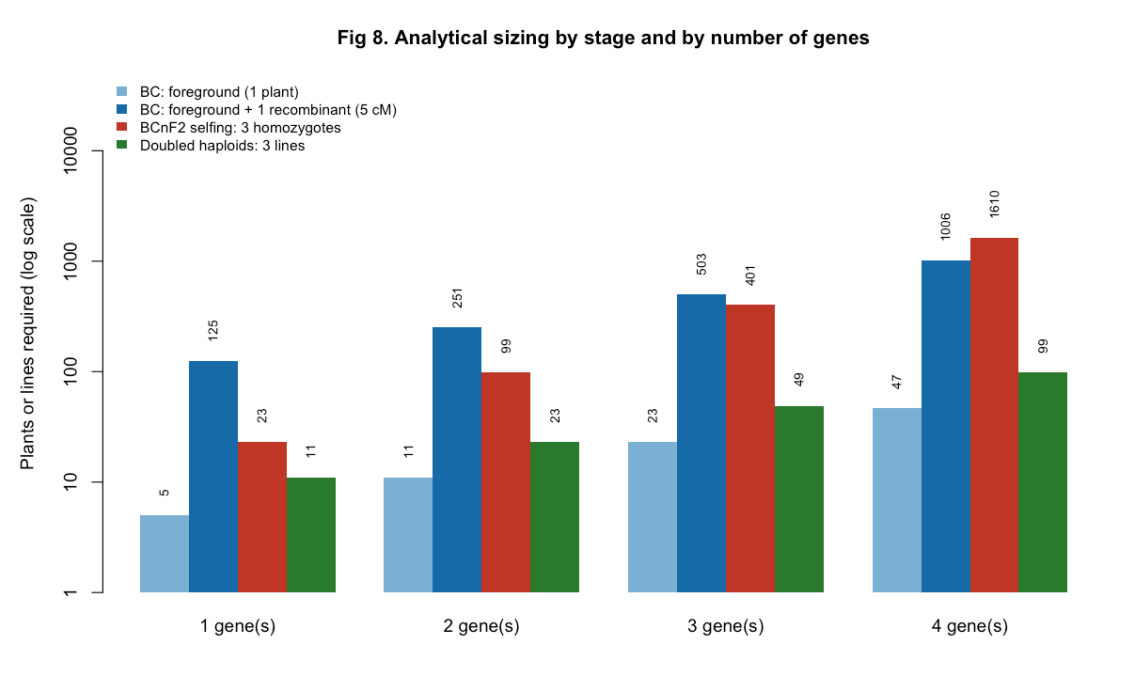

Figure 6.6: Analytical sizing by stage, log scale. The two blue bars are backcross generations; red is fixation by selfing; green is fixation by doubled haploids.

At four genes that is 1610 BC3F2 plants against 99 DH lines, a factor of 16.3. For maize, where DH is routine, the alarming number is not the real bottleneck.

Without DH there is a staggered alternative. Do not ask one generation to fix everything. Select BC3F2 plants homozygous for two targets and heterozygous for the other two (roughly 330 plants), then close out the remaining two at BC3F3 (roughly 100). Two stages of hundreds instead of one of thousands, at the cost of one generation.

6.8 Running the simulation

Everything above is reproducible from the package. The engine and the decision layer are in `R/08_mabc.R`; the experiment driver that produced the cached tables is `inst/scripts/run_mabc.R`.

One programme, interactively. Build a map, place the targets, draw the donor-elite sharing, and run the three backcrosses:

```
map <- build_map(maize_chr_len, spacing_cM = 5)
w   <- marker_weights(map)
tg  <- target_indices(map, k = 3)              # the first three of maize_targets
sh  <- make_shared_ibd(map, s = 0.60, mw = w, target_idx = tg)

pr <- run_bc_program(n_sched = c(120, 120, 120), s = 0.60,
                     strategy = "rec_drag", map = map, opt = mabc_opt,
                     tg = tg, w = w, shared = sh)
fmt(pr$traj, 3)
```

gen	ibd	dose	drag	n_fg	ibd_pop_mean	fail
1	0.878	0.251	162.5	13	0.891	FALSE
2	0.934	0.127	97.5	12	0.925	FALSE
3	0.951	0.094	70.0	15	0.952	FALSE

`n_sched` is the schedule, one entry per backcross, so `c(300, 40, 20)` and `c(120, 120, 120)` are the two allocations compared above. Passing the same `shared` mask to two calls is what makes a comparison between selection indices paired rather than confounded with the donor-elite pair.

Comparing indices on common random numbers. Reset the seed before each strategy so the two programmes see the same meioses:

```
drag_of <- function(strategy) {
  set.seed(7)
  run_bc_program(c(120, 120, 120), 0.60, strategy, map, mabc_opt, tg, w,
                 shared = sh)$traj$drag[mabc_opt$n_bc]
}
c(rec_bg = drag_of("rec_bg"), rec_drag = drag_of("rec_drag"))
#>   rec_bg rec_drag
#>       60       45
```

A single replicate is noise; the tables above are 300 of these per cell, which is the driver's job.

The analytical sizing needs no simulation at all. It is a cumulative binomial, so it is exact and instant:

```
sizing_table(k_max = 4, flank_cM = 5, P = 0.95, n_keep = 3)
```

genes	q_foreground	n_BC_1plant	n_BC_1rec	q_F2	n_F2_1plant	n_F2_3plants	q_DH	n_DH_3lines
1	0.5000	5	125	0.2500000	11	23	0.5000	11
2	0.2500	11	251	0.0625000	47	99	0.2500	23
3	0.1250	23	503	0.0156250	191	401	0.1250	49
4	0.0625	47	1006	0.0039063	766	1610	0.0625	99

The full scans, from the shell. The driver runs the whole grid – four policies by four gene numbers by eleven population sizes by three values of s, plus the allocation grid and the BCnF2 stage – and writes the CSVs, the nine figures and a self-contained HTML report:

```
Rscript inst/scripts/run_mabc.R --test              # 35 checks, ~10 s
Rscript inst/scripts/run_mabc.R --quick --out=DIR   # reduced grid, ~30 s
Rscript inst/scripts/run_mabc.R --out=report/mabc   # the full run, ~3 min
```

The cached tables this chapter reads are the full run's `results_uniform.csv`, `results_allocation.csv`, `results_bcnf2.csv` and `analytical_sizing.csv`, copied into `book/data` as `mabc_*.csv`; the header of the driver lists the copy commands.

Pointing it at your own programme. Three things change and nothing else:

```
# 1. your map
map <- build_map(my_chr_len_cM, spacing_cM = 5)

# 2. your targets -- same columns as maize_targets
my_targets <- data.frame(gene = c("A", "B"), chr = c(1, 5),
                         pos_cM = c(40, 110), label = c("A", "B"))
tg <- target_indices(map, k = 2, targets = my_targets)

# 3. your criteria
opt <- utils::modifyList(mabc_opt,
                         list(n_bc = 4, drag_per_gene = 20, target_ibd = 0.95))
```

`s` – how much of the genome your donor and your elite already share – is the parameter worth measuring rather than assuming. It scales the whole cost: at $s = 0.7$ most of the donor genome that survives is free, at $s = 0$ none of it is.

6.9 Assumptions, and what is not modelled

These matter more than usual here, because unlike the diversity chapters this one is used to decide field area.

No crossover interference (material, and it cuts against the results above). Meiosis is Haldane, so crossovers are independent between intervals. Maize has strong positive interference. For the event this simulation depends on most – a double recombinant in the two 5 cM intervals flanking a target, which is what turns a long donor segment into a short one – Haldane gives 2.26×10^{-3} against 4.90×10^{-4} under a coincidence model. **The model over-produces tight recombinants by about 4.6x**, so every

population size quoted for close-in trimming is a lower bound. Note the direction: the index correction makes the problem easier, interference makes it harder, and they do not cancel in any principled way. The correct fix is a crossover-position sampler – chiasmata as a stationary gamma renewal process with shape ν – which reduces to Haldane exactly at $\nu = 1$ and so is backward compatible; maize estimates put ν near 3.

One plant advanced per generation. Real programmes advance 3-10 and cross each to the recurrent parent. Every probability in this chapter is therefore a **single-lineage** probability: it understates achievable progress and it removes lineage-level variance from every distribution shown.

Uniform recombination rate, no distortion, no genotyping error. Maize has 5-10x rate variation and strongly suppressed pericentromeric regions; a target in a cold region carries drag no feasible population will trim. Gametophyte factors shift the foreground frequency away from $1/2$. Map positions here are illustrative and not pinned to a named consensus map – and IBM-type maps must be rescaled before use, exactly as in Chapter 5..

Triploid endosperm genetics is not modelled. The illustrative targets are endosperm genes. The endosperm is 2 maternal : 1 paternal, so a recessive kernel phenotype does not appear on the ear of a heterozygous plant: phenotypic confirmation lags the genotype by one generation, and xenia means it requires a self-pollination, not open pollination.

The gene set is illustrative, not a breeding recommendation. `sh2` and `bt2` encode the two subunits of the same enzyme, so stacking both is largely redundant – dropping one turns the hard case ($k = 4$) into the comfortable one ($k = 3$). `o2` alone gives opaque endosperm, not QPM: the hardness modifiers are polygenic, unlinked and unmodelled here, which changes the problem class from "stack k Mendelian genes" to "stack k genes plus a QTL background".

⚠ Two statistical habits this chapter had to be repaired for

Select on the interval, not the point estimate. The first version of the sizing tables took the first grid point whose *point estimate* crossed 0.95. With 30-150 replicates the standard error is 0.02-0.09, so that is a winner's curse: the tables were non-monotone in N (0.97, 0.97, then 0.93 at a larger N), which is a monotone quantity reading noise. Selection is now on the Clopper-Pearson lower bound, and intervals are reported everywhere.

Never type a number into the prose. Every figure in the original narrative was a string constant written against an earlier configuration; re-runs put the true values outside the reported confidence intervals, and the same document contradicted itself in three places. Every number in this chapter is interpolated from the cached tables at render time. Appendix C. says which table each one comes from.

6.10 Where this connects

Chapter 5. sized a population to *keep* diversity; this chapter sized one to discard it everywhere except at k loci. The two share their machinery – the same map, the same Haldane meiosis, the same distinction between what is segregating and what is fixed – and they share the lesson that dominates both: **size for the objective that binds.** In Chapter 5. that meant not sizing for gene diversity, which saturates early. Here it means not sizing for background recovery, which is already satisfied at BC3, and putting the plants into the one criterion that a population can actually buy.

The converted lines this chapter produces re-enter the pool the rest of the book optimises over. A conversion programme run for its own sake reduces diversity by construction: it makes an elite line into a near-copy of itself. If several conversions are run in parallel off the same recurrent parent, the resulting lines are close to each other.

That has been asserted for a chapter and a half. It is cheap to measure, and the size of it is the argument. Eight BC3 conversions off one recurrent parent, against eight unrelated elite lines, put through the kernel of Chapter 1.:

```
set.seed(11)
m_c  <- 2000
elite <- matrix(rbinom(40 * m_c, 1, 0.5), 40, m_c)   # unrelated elite lines
rec   <- elite[1, ]                                   # one recurrent parent
donor <- elite[40, ]
```

```
ibd   <- expected_ibd(3, 0.5)                          # recurrent genome at BC3

conv <- t(sapply(1:8, function(i)
  ifelse(rbinom(m_c, 1, ibd) == 1, rec, donor)))

cost <- data.frame(
  panel = c("8 BC3 conversions, one recurrent parent", "8 unrelated elite lines"),
  theta = c(mean(molecular_coancestry(conv)),
            mean(molecular_coancestry(elite[1:8, ]))))
cost$GD <- 1 - cost$theta
fmt(cost, 4)
```

panel	theta	GD
8 BC3 conversions, one recurrent parent	0.9714	0.0286
8 unrelated elite lines	0.5617	0.4383

The converted panel carries about 0.07 times the gene diversity of the unrelated one – not a marginal erosion but a different order of magnitude, and it follows directly from `expected_ibd(3, 0.5)` = 0.969. Conversion is *designed* to make lines identical everywhere except the target, and it succeeds.

Two consequences for the rest of the book. A conversion programme must be counted against the diversity budget of Chapter 18. rather than run alongside it, because θ cannot distinguish eight conversions from one line entered eight times. And Chapter 16.‘ per-line usage cap is the mechanism that already handles this: capping how often a line is used caps how often its near-copies are, provided the conversions are recognised as near-copies rather than entered as eight independent parents.

> ⚠ Do not enter a converted panel as independent lines
>
> The failure mode is administrative, not statistical. Conversions arrive with distinct line codes and distinct pedigree entries, so a pipeline that trusts the codes will treat them as eight parents. The coancestry matrix will not, and Chapter 15.‘s recommendation to compute θ on markers rather than on pedigree is what protects against exactly this.

Reproduce this chapter

All the code of this chapter, in order and with the explanation kept as comments, is one script: **04b-backcross-introgression.R**. Download it, install `hybdiv`, and run it top to bottom in an empty directory; cached results are fetched on first use.

7. Simulating a hybrid breeding programme: AlphaSimR from the ground up

Chapter 4. built the `{X, f, hybrids, fL}` contract with a deliberately fast simulator: Balding-Nichols allele frequencies, independent markers, no recombination, no dominance. That is enough to exercise the diversity and selection machinery, but it is not how a hybrid breeding programme is actually run, and it is not how anyone would build one from scratch.

This chapter builds one. The crop in mind is maize, the canonical allogamous hybrid crop: two heterotic pools, inbred (doubled-haploid) lines drawn from each, lines judged by their testcrosses onto the other pool rather than by themselves, pools improved by reciprocal recurrent selection (RRS; R. E. Comstock, H. F. Robinson, and P. H. Harvey [7]), and a commercial product that is a single cross between one line from each pool. Steps 1 to 7 are a **step-by-step construction** with AlphaSimR (R. C. Gaynor, G. Gorjanc, and J. M. Hickey [8]), written out in the primitive calls – `runMacs()`, `SimParam`, `addTraitAD()`, `makeCross()`, `hybridCross()`, `makeDH()`, `setPheno()`, `selectInd()` – with a plant breeder's reading of each argument and a measurement at each step. Step 8 assembles those primitives into RRS, replicates it against a paired no-selection control, and shows that the same stage-1 contract comes out the other end, so `stage2_select()` runs on it exactly as it does on the book's own simulated data.

Why this machinery, and why it has no counterpart in a self-pollinated programme. In an autogamous (self-pollinated) crop – wheat, rice, soybean, common bean – recombination happens once, in the F1, and the programme then narrows a segregating population toward homozygosity by selfing or single-seed descent, selecting throughout on a line's own (per se) phenotype, because a self-pollinated genotype breeds true and its own performance *is* what a farmer will grow: the released variety is very often the same homozygous line the breeder selected. Chapter 8. builds exactly that programme with the same AlphaSimR primitives – `makeDH()`, `setPheno()`, `selectInd()` – and is the chapter to read this one against, step by step. None of Steps 2, 4 and 6 below would have a job there: there is no dominance for per se selection to be blind to (an inbred line's value is its additive merit alone – Step 3 makes this an identity, `varD = 0` exactly on a homozygous panel), no tester to cross a candidate onto, no general or specific combining ability to estimate (G. F. Sprague and L. A. Tatum [9]), and no second, deliberately divergent population to keep apart, because there is only one gene pool, not two heterotic ones held apart on purpose. What follows instead is specific to an allogamous, hybrid-product crop: the inbred line is not the product, the F1 is (Step 5); a line's breeding value depends on which pool it will be crossed to, not on itself alone (Step 6′s tester dependence); and the trait improvement a programme has made is realised only in a cross remade every season, never in either parental line by itself.

The pedagogical source is the AlphaSimR course of Gorjanc and colleagues [10]: its vignettes on simulating DNA, simulating traits, the DNA lottery, single- and multi-trait selection, and a plant breeding programme. The course has no hybrid programme of its own; the wheat programme it does build is the subject of Chapter 8.. This chapter takes the course's primitives and assembles them into the hybrid case.

The order matters. Everything in Step 8 is a wrapper in `R/13_alphasimr_bridge.R` – `rrs_config`, `alphasimr_founder_pools()`, `alphasimr_rrs_cycle()`, `alphasimr_make_hybrids()`, `alphasimr_to_stage1()`, `alphasimr_pipeline()` – and a wrapper is only readable once you know what it is wrapping. AlphaSimR is a `Suggests` dependency, guarded exactly like `quadprog` and `highs` elsewhere in the package; this chapter and Chapter 8. are the only ones that need it.

7.1 Step 1 – Founder genomes: what a founder is, and which engine to use

Every AlphaSimR simulation starts with founder haplotypes. Before choosing an engine, it is worth seeing what one produces, at a size small enough to read. `inbred = TRUE` makes every founder fully homozygous, which is what a panel of inbred lines is.

```
set.seed(1)
tiny <- quickHaplo(nInd = 3, nChr = 2, segSites = 4, inbred = TRUE)
SPt  <- SimParam$new(tiny)
tiny_pop <- newPop(tiny, simParam = SPt)
pullSegSiteHaplo(tiny_pop, simParam = SPt)
#>     1_1 1_2 1_3 1_4 2_1 2_2 2_3 2_4
#> 1_1   0   0   0   1   0   1   1   0
#> 1_2   0   0   0   1   0   1   1   0
#> 2_1   0   1   1   0   1   0   1   1
#> 2_2   0   1   1   0   1   0   1   1
#> 3_1   1   0   0   0   0   1   0   1
#> 3_2   1   0   0   0   0   1   0   1
```

One row per haplotype, two per diploid individual (`1_1` and `1_2` are the two chromosome copies of individual 1), one column per segregating site across both chromosomes (`1_3` is chromosome 1, site 3). Alleles are coded 0 and 1 – ancestral and derived under a coalescent engine, arbitrary labels under `quickHaplo()`. Because the founders are inbred, the two rows of each individual are identical. `pullSegSiteGeno()` sums the two rows into an allele dosage in `{0, 1, 2}`:

```
tiny_geno <- pullSegSiteGeno(tiny_pop, simParam = SPt)
tiny_geno
#>   1_1 1_2 1_3 1_4 2_1 2_2 2_3 2_4
#> 1   0   0   0   2   0   2   2   0
#> 2   0   2   2   0   2   0   2   2
#> 3   2   0   0   0   0   2   0   2
stopifnot(all(tiny_geno %in% c(0, 2)))
stopifnot(isTRUE(all.equal(diag(molecular_coancestry(tiny_geno / 2)),
                           rep(1, 3))))
```

The two assertions fix the coding for the rest of the book. AlphaSimR's dosage is 0/1/2; the contract of Chapter 4. codes the same information as 0/0.5/1, the within-individual frequency of the reference allele (AlphaSimR's derived allele plays that role), and every bridge function divides by two on the way across (`stage1_build()` also detects and rescales 0/1/2 on its own). On an inbred individual the molecular self-coancestry of Chapter 1. is exactly 1, which is the second assertion.

There are two engines, and the choice is not cosmetic. `runMacs()` runs a backward-in-time coalescent (G. K. Chen, P. Marjoram, and J. D. Wall [11]) under a species-specific demography – `species = "MAIZE"`, `"WHEAT"`, `"CATTLE"` – so the founders carry a realistic allele-frequency spectrum and realistic linkage disequilibrium. `quickHaplo()` samples 0s and 1s directly: instant, and with no history behind it.

```
tm <- function(expr) as.numeric(system.time(expr)["elapsed"])

# reproducibility: two calls from the same seed, and R's own stream after each
set.seed(7); qa <- quickHaplo(nInd = 20, nChr = 10, segSites = 50, inbred = TRUE)
rs_qa <- .Random.seed
set.seed(7); qb <- quickHaplo(nInd = 20, nChr = 10, segSites = 50, inbred = TRUE)
rs_qb <- .Random.seed
set.seed(7); ma <- runMacs(nInd = 20, nChr = 10, segSites = 50, inbred = TRUE,
                           species = "MAIZE")
rs_ma <- .Random.seed
set.seed(7); mb <- runMacs(nInd = 20, nChr = 10, segSites = 50, inbred = TRUE,
                           species = "MAIZE")
rs_mb <- .Random.seed
stopifnot(identical(rs_qa, rs_qb), identical(rs_ma, rs_mb))
```

```
# timing: one runMacs() call; quickHaplo() over n_rep_q calls, because a
# single call is below the clock's resolution
n_rep_q <- 100
t_m <- tm(runMacs(nInd = 20, nChr = 10, segSites = 50, inbred = TRUE,
                  species = "MAIZE"))
t_q <- tm(for (k in seq_len(n_rep_q))
          quickHaplo(nInd = 20, nChr = 10, segSites = 50, inbred = TRUE)) / n_rep_q

fmt(data.frame(engine              = c("quickHaplo", "runMacs"),
               seconds_per_call    = c(t_q, t_m),
               same_twice          = c(identical(qa@geno, qb@geno),
                                       identical(ma@geno, mb@geno)),
               R_stream_same_after = c(identical(rs_qa, rs_qb),
                                       identical(rs_ma, rs_mb))), 4)
```

engine	seconds_per_call	same_twice	R_stream_same_after
quickHaplo	0.0002	TRUE	TRUE
runMacs	2.7230	FALSE	TRUE

Read the last two columns together. **`runMacs()` is not reproducible from `set.seed()`, and R's own random stream is untouched by that.** `runMacs()` draws a seed for MaCS from R's stream (one `sample.int()` per chromosome) and hands it to the binary. The fourth column, asserted in the chunk, says that after two calls from the same seed R's `.Random.seed` is identical: the amount of R's stream the call consumes is fixed, and every draw R makes afterwards is the same in both runs. Yet the third column says the haplotypes differ. What was measured, then, is that MaCS's output is not a deterministic function of the seed it is handed. The only thing that moves between two runs is the founder haplotypes themselves – and everything computed *from* those founders. `quickHaplo()` is pure R sampling and is exactly reproducible.

(The `seconds_per_call` column is wall clock and will differ on your machine and between renders. `runMacs()` takes seconds; `quickHaplo()`, timed over 100 calls because one call is below the clock's resolution, takes about 0.25 ms per call.)

> ⚠ The founders are not seeded, and everything downstream is
>
> This is the single most expensive trap in AlphaSimR, because it fails silently: the code looks reproducible, runs without complaint, and returns different numbers. The non-reproducibility is documented in `alphasimr_founder_pools()`'s help page for `runMacs2()`; the table above shows it is not `runMacs2`-specific but true of MaCS generally.
>
> Two consequences, both because the founders move and not because of anything R's stream does:
>
> 1. **Found once, then branch.** Any experiment comparing two schemes must draw the founders a single time and start both arms from that one object. Draw them twice and the two arms start from different founders, and the comparison measures the founders as much as the schemes. Chapter 23.' linked arm is built this way, and so is Step 8's paired control below.
> 2. **Save what you cannot reseed.** `saveRDS(founderGenomes, ...)` is the only way to get the same coalescent history back in a later session.
>
> Everything after the founders – the crosses, the doubled haploids, the phenotyping error, the recombination – *is* controlled by `set.seed()`, given the founders. Reseeding downstream does not recover the founders and is not needed for anything else.

So what does `quickHaplo()` give up? Linkage disequilibrium among the founders. The honest way to measure that is against the sampling floor: with `n` individuals, two completely independent loci still show `r^2` of about `1 / (n - 1)` by chance alone.

```
ldstat <- function(fg) {
  SPx <- SimParam$new(fg)
  g <- pullSegSiteGeno(newPop(fg, simParam = SPx), simParam = SPx)
  g <- g[, apply(g, 2, stats::sd) > 0, drop = FALSE]
  chr <- sub("_.*$", "", colnames(g))           # "chr_site" column names
  r <- stats::cor(g)
```

```
  adj <- diag(r[-nrow(r), -1])                  # consecutive columns ...
  adj <- adj[chr[-length(chr)] == chr[-1]]      # ... on the same chromosome
  c(adjacent = mean(adj^2),                    # neighbouring sites, same chromosome
    overall  = mean(r[upper.tri(r)]^2),         # all pairs, mostly distant
    noise    = 1 / (nrow(g) - 1))               # what pure chance gives
}
set.seed(3); lq <- ldstat(quickHaplo(nInd = 100, nChr = 5, segSites = 200,
                                     inbred = TRUE))
set.seed(3); lm_ <- ldstat(runMacs(nInd = 100, nChr = 5, segSites = 200,
                                   inbred = TRUE, species = "MAIZE"))
fmt(rbind(quickHaplo = lq, runMacs = lm_))
```

	adjacent	overall	noise
quickHaplo	0.0103	0.0101	0.0101
runMacs	0.1359	0.0120	0.0101

`adjacent` is the mean `r^2` between neighbouring sites on the same chromosome (pairs that straddle a chromosome boundary are excluded); `overall` is over all pairs, most of them on different chromosomes. `quickHaplo()`'s adjacent-site `r^2` sits on the noise floor: there is no founder LD at all, as designed. `runMacs()`'s is 13 times the floor in this render, and its *overall* `r^2` is barely above it – so the LD it creates is local, decaying with map distance, which is what real LD does.

The `runMacs` row of that table is itself not reproducible, for the reason the row above it establishes; the `quickHaplo` row is, to the last digit.

What that does and does not imply for the rest of the chapter. R's stream is unaffected by MaCS, so any step that draws its own `quickHaplo()` founders from a fresh `set.seed()` is exactly reproducible, and so would be a step that did not reseed: the `runMacs()` calls above leave the stream exactly where a fixed number of `sample.int()` calls leaves it. What is not reproducible is anything computed *from* MaCS founders – the `runMacs` rows of both tables here, and all of Step 8. The chapter still calls `set.seed()` immediately before each step's own founder draw rather than once at the top, so that each step is reproducible on its own and not only as part of this chapter's execution order. With that in place, two independent executions agree on every number in Steps 1 to 7 except the wall-clock seconds and the `runMacs` rows.

That measurement decides the policy this chapter follows:

- **`quickHaplo(inbred = TRUE)` for Steps 2–7 and all of Chapter 8..** They teach the arithmetic of dominance, heterosis and response, where founder LD is not the point, and in exchange every number printed is reproducible from a clean clone. AlphaSimR still applies a real genetic map and real meiosis to every descendant, so linkage and the Bulmer effect of Chapter 23. still operate – only the founders' *history* is stylised.
- **`runMacs2(split = )` in Step 8.** There the divergence between two heterotic pools *is* the object of study, and no amount of speed compensates for not having it. Every Step 8 result is therefore replicated over founder draws.

Founder LD does matter where marker-QTL association is the mechanism, which is genomic prediction – see 3, and note that the accuracies there are computed on a different simulator for this reason.

7.2 Step 2 – `SimParam`, and defining a trait with dominance

`SimParam` holds everything global: the genetic map, which sites are QTL, the trait effects, the error variances. It is an R6 object, so its methods are called with `$`, and a variable named `SP` is picked up automatically by every AlphaSimR function that needs it.

A hybrid programme exists because of dominance: without it a line's cross performance is its own breeding value and there is nothing a testcross can see that the line per se could not (Chapter 13.). So the trait defined here, and used through Step 6, carries a dominance term from the start. This is exactly the reason an autogamous programme has no equivalent of this step: there, dominance is invisible by construction (a released, self-pollinated variety is homozygous, so no heterozygote is ever evaluated or sold) and `SP$addTraitA()` – additive effects only – is the whole model, the case Chapter 8. runs throughout.

```
# Seeded here, not once at the top of the chapter: Step 1 draws from R's random
# stream (each runMacs() call consumes a fixed amount of it), so everything
# downstream would move with any edit or re-execution of Step 1 unless the seed
# is reset immediately before the draw that matters.
set.seed(101)
founders <- quickHaplo(nInd = 60, nChr = 10, segSites = 100, inbred = TRUE)
SP <- SimParam$new(founders)
SP$addTraitAD(nQtlPerChr = 100, mean = 10, var = 1,
              meanDD = rrs_cfg[["mean_dd"]], varDD = rrs_cfg[["var_dd"]])
SP$setVarE(h2 = 0.4)
```

There are four trait constructors, and choosing between them is choosing a model of gene action:

Method	Adds	When a plant programme needs it
`SP$addTraitA()`	additive only	line per se performance; the course's teaching case [10], and Step 7 here
`SP$addTraitAD()`	1. dominance	testcross and hybrid value. This is what Steps 2–6 and Step 8 use
`SP$addTraitAG()`	1. genotype-by-environment	multi-location yield trials, where the same genotype is scored in several places. Chapter 8. uses it
`SP$addTraitADE()`	1. epistasis	available; used nowhere in this book

7.2.1 The dominance model, as AlphaSimR writes it

For a diploid with QTL $k = 1, \ldots, m$, let $x_{ik} \in \{0, 1, 2\}$ be the dosage of the derived allele of individual i at QTL k (the column of `pullQtlGeno()`). AlphaSimR's `TraitAD` genetic value is

$$G_i = \mu_0 + \sum_{k=1}^{m} a_k \, (x_{ik} - 1) + \sum_{k=1}^{m} d_k \, \mathbb{1}[x_{ik} = 1] \tag{7.1}$$

with μ_0 an intercept, $a_k \in \mathbb{R}$ the additive effect (half the difference between the two homozygotes, on the classical scale of D. S. Falconer and T. F. C. Mackay [12], Ch. 7, which puts the origin at the midpoint of the two homozygotes and assigns them $+a$ and $-a$ and the heterozygote d; no equation number is attributed here, the scale is stated in full) and $d_k \in \mathbb{R}$ the dominance deviation, the departure of the heterozygote from that midpoint. It is the model of Chapter 13. with AlphaSimR's coding, and the chunk checks it rather than trusts it:

```
tr <- SP$traits[[1]]
a  <- tr@addEff
d  <- tr@domEff
pop0 <- newPop(founders, simParam = SP)
x0   <- pullQtlGeno(pop0, simParam = SP)
gv_by_hand <- as.numeric(tr@intercept + (x0 - 1) %*% a + (x0 == 1) %*% d)
stopifnot(isTRUE(all.equal(gv_by_hand, as.numeric(gv(pop0)))))
```

`meanDD` and `varDD` parameterise the **dominance degree** $\delta_k = d_k / |a_k|$: AlphaSimR draws $\delta_k \sim N(\texttt{meanDD}, \texttt{varDD})$ and sets $d_k = \delta_k \, |a_k|$, so $\delta_k = 0$ is additivity, $\delta_k = 1$ complete dominance, $\delta_k > 1$ overdominance, and a *positive mean* is directional dominance: on average the heterozygote sits above the homozygote midpoint, whichever allele is favourable. Directional dominance is what makes heterosis positive and inbreeding depression a loss on the trait scale, and it is the standing hypothesis for yield in maize.

```
c(n_qtl = length(a), meanDD_requested = rrs_cfg[["mean_dd"]],
  meanDD_realised = mean(d / abs(a)), varDD_requested = rrs_cfg[["var_dd"]],
  varDD_realised = stats::var(d / abs(a))) |> fmt(3)
#>            n_qtl meanDD_requested  meanDD_realised  varDD_requested
#>         1000.000            0.400            0.378            0.100
#>   varDD_realised
#>            0.104
```

`nQtlPerChr` sets how polygenic the trait is. Setting it equal to `segSites` makes *every* segregating site a QTL, which is what this chapter does throughout – convenient, because then the marker matrix and the causal loci are the same set and there is no marker-versus-QTL gap to reason about. Real programmes have that gap; Chapter 12. is where the book takes it seriously.

`mean` fixes the mean genetic value of the *founders*. `var` fixes their **additive** variance (`useVarA = TRUE`, the default), not their total genetic variance – with inbred founders the two coincide, for a reason Step 3 shows. `corA` (Step 7) sets the genetic correlation between traits.

The environmental variance has two routes, and plant breeders use the second:

- `SP$setVarE(h2 = 0.4)` fixes an error variance once, from the founders' genetic variance (here 1.5), and every later `setPheno()` reuses it.
- `setPheno(varE = , reps = )` sets it per call. **`reps` is the plant idiom for locations**: `varE` is the plot-level error, and a genotype scored in `reps` plots has its error variance divided by `reps`, so the phenotype is an entry mean. That is how a staged programme raises selection accuracy from one stage to the next without ever changing the trait's heritability – the mechanism Chapter 8. is built on, and the one Step 6 uses for testers.

7.3 Step 3 – The base population, and where the variance components live

`newPop()` turns founder haplotypes into individuals and, because a trait is now defined, computes their genetic values immediately.

```
pop <- setPheno(newPop(founders, simParam = SP), simParam = SP)
c(n = pop@nInd, meanG = as.numeric(meanG(pop)), varG = as.numeric(varG(pop)),
  varP = as.numeric(varP(pop)),
  h2_realised = as.numeric(varG(pop) / varP(pop))) |> fmt(3)
#>           n       meanG        varG        varP h2_realised
#>      60.000      10.000       1.000       2.614       0.383
```

The realised heritability is near but not equal to the requested 0.4: `setVarE()` fixes a variance, and the ratio it produces in any finite sample is itself a random variable. This is a general and useful habit – **check what the simulation realised, do not assume what you requested.**

`genParam()` returns the full decomposition. Four of its fields are the ones a hybrid programme cares about, and they read differently on a panel of inbred lines and on a random-mating population made from them:

```
mixed <- pop
for (k in 1:3) mixed <- randCross(mixed, nCrosses = 1000, simParam = SP)

vrow <- function(p, lab) {
  g <- genParam(p, simParam = SP)
  data.frame(population = lab,
             meanG     = as.numeric(g[["mu"]]),
             varA      = as.numeric(g[["varA"]]),
             genicVarA = as.numeric(g[["genicVarA"]]),
             ratio     = as.numeric(g[["varA"]] / g[["genicVarA"]]),
             varD      = as.numeric(g[["varD"]]),
             genicVarD = as.numeric(g[["genicVarD"]]))
}
vtab <- rbind(vrow(pop,   "founders (fully inbred)"),
              vrow(mixed, "after 3 random matings"))
fmt(vtab)
```

population	meanG	varA	genicVarA	ratio	varD	genicVarD
founders (fully inbred)	10.0000	1.0000	0.4274	2.3397	0.0000	0.0496
after 3 random matings	14.3495	0.4503	0.4264	1.0560	0.0793	0.0494

```
stopifnot(vtab[["varD"]][1] < 1e-10)

# Among fully homozygous lines the locus value is a (x - 1), so at linkage
# equilibrium varA = sum 4 p q a^2, while genicVarA = sum 2 p q alpha^2 with
# alpha = a + d (q - p): under dominance the LE ratio is close to, not
# exactly, 2. What varA carries above varA_LE is disequilibrium.
p_f      <- colMeans(x0) / 2                        # founder allele frequencies
varA_f   <- vtab[["varA"]][1]
genic_f  <- vtab[["genicVarA"]][1]
varA_LE  <- sum(4 * p_f * (1 - p_f) * a^2)
ratio_LE <- varA_LE / genic_f
varA_LD  <- varA_f - varA_LE
ratio_LD <- varA_LD / genic_f
stopifnot(isTRUE(all.equal(as.numeric(bv(pop, simParam = SP)[, 1]),
                           as.numeric(sweep(x0, 2, 2 * p_f) %*% a))))
c(varA = varA_f, varA_LE = varA_LE, ratio_LE = ratio_LE,
  varA_LD = varA_LD, ratio_LD = ratio_LD) |> fmt(4)
#>     varA  varA_LE ratio_LE  varA_LD ratio_LD
#>   1.0000   0.8495   1.9875   0.1505   0.3522
```

`varA` is the realised additive variance; `genicVarA` is what it would be if the loci were independent and in Hardy-Weinberg proportions. Their ratio is the whole subject of Chapter 23., and the package's `genic_var()` computes the same quantity from a marker matrix directly.

The founders' ratio of 2.34 has two parts, and the chunk separates them. Among fully homozygous lines the value at QTL k is $a_k(x_{ik} - 1)$ with $x_{ik} \in \{0, 2\}$, and AlphaSimR's `bv()` on these founders is exactly $\sum_k a_k(x_{ik} - 2p_k)$ (asserted in the chunk). At linkage equilibrium its variance is `varA_LE` $= \sum_k 4p_k q_k a_k^2 = 0.8495$, against a genic value of 0.4274: a ratio of 1.99, exact given the effects and the frequencies. That part is **inbreeding** – a homozygous population has per-locus variance `4pq` where a Hardy-Weinberg one has `2pq`. It is close to 2 and not exactly 2 because `genicVarA` is written with the Hardy-Weinberg average effect $\alpha_k = a_k + d_k(q_k - p_k)$ defined below, whereas the homozygous lines carry a_k; twice the genic value equals `varA_LE` only where $d_k(q_k - p_k) = 0$ at every locus. The remainder, `varA_LD = varA - varA_LE` $= 0.1505$, or 0.35 in ratio units, is sampling (gametic-phase) disequilibrium among only 60 founders' worth of gametes. Three rounds of random mating restore Hardy-Weinberg and pull the ratio to 1.06; what remains is that same disequilibrium. A ratio away from 1 in an *outbred* population is the gametic-phase disequilibrium Chapter 3. defines and Chapter 23. measures; a ratio away from 1 in an inbred one is mostly the homozygosity.

The dominance columns carry the hybrid lesson. **`varD` among the inbred founders is exactly zero** (the assertion above): a homozygous line has no heterozygous locus, so its dominance contribution is zero whatever d is, and no amount of per se evaluation of inbred lines can see dominance. This single identity is the operational reason an autogamous programme stops at per se selection and never needs a testcross: its released lines are exactly this "founders" row, `varD = 0` for the whole life of the programme, so per se phenotype already is breeding value up to the additive/genic gap discussed below. After random mating `varD` appears. Two identities make that quantitative. With p_k the founders' frequency of the derived allele at QTL k and $q_k = 1 - p_k$, the population mean under Hardy-Weinberg proportions is [12, Ch. 7, Eq. 7.2]

$$M_{HW} = \mu_0 + \sum_k \left[\, a_k\,(p_k - q_k) + 2\,p_k q_k\,d_k \,\right] \tag{7.2}$$

while the inbred panel at the same frequencies has only the first term. The difference, $2\sum_k d_k p_k q_k$, is the inbreeding depression of Chapter 13. at $F = 1$, which the package computes as `inbreeding_depression(p, d, F_coef = 1)`. And the genic additive variance is $\sum_k 2p_k q_k \alpha_k^2$ with $\alpha_k = a_k + d_k\,(q_k - p_k)$ the average effect of a gene substitution. D. S. Falconer and T. F. C. Mackay [12] define α in Ch. 7 and the single-locus additive variance $2pq\alpha^2$ in Ch. 8; the equation numbers are not attributed here, so both identities are stated in this chapter's notation: $\alpha_k = a_k + d_k\,(q_k - p_k)$ and, summed over independent loci, $\sum_k 2p_k q_k \alpha_k^2$, which is the genic variance:

```
gp  <- genParam(pop, simParam = SP)
alpha_f <- a + d * (1 - 2 * p_f)
stopifnot(isTRUE(all.equal(as.numeric(gp[["mu_HW"]] - gp[["mu"]]),
```

```
                          inbreeding_depression(p_f, d, F_coef = 1))))
stopifnot(isTRUE(all.equal(as.numeric(gp[["genicVarA"]]),
                          sum(2 * p_f * (1 - p_f) * alpha_f^2))))
c(mean_inbred = as.numeric(gp[["mu"]]), mean_HW = as.numeric(gp[["mu_HW"]]),
  inbreeding_depression = inbreeding_depression(p_f, d, F_coef = 1)) |> fmt(3)
#>           mean_inbred               mean_HW inbreeding_depression
#>                10.000                14.355                 4.355
```

Both hold to machine precision against AlphaSimR's own `genParam()`. The gap between the two means is the amount a random-mating population built from these lines would sit *above* the lines themselves – and, read in the other direction, the amount an open-pollinated population loses on being driven to homozygosity. It is the same d that Step 4 turns into heterosis.

7.4 Step 4 – The DNA lottery, heterosis, and why an F1 is not a family

The course [10] demonstrates Mendelian sampling by crossing outbred founders and showing that full sibs vary. With **inbred** parents the result is different, and it is the fact hybrid breeding is organised around.

```
crosses <- list(c(1, 2), c(1, 3), c(2, 3))
lottery <- do.call(rbind, lapply(crosses, function(ij) {
  plan <- matrix(ij, ncol = 2)
  f1 <- makeCross(pop, plan, nProgeny = 200, simParam = SP)
  dh <- makeDH(makeCross(pop, plan, nProgeny = 1, simParam = SP),
               nDH = 200, simParam = SP)
  better <- max(gv(pop)[ij])
  data.frame(cross         = paste(ij, collapse = " x "),
             parent_mean   = mean(gv(pop)[ij]),
             better_parent = better,
             f1_mean       = as.numeric(meanG(f1)),
             f1_sd         = as.numeric(sqrt(varG(f1))),
             dh_sd         = as.numeric(sqrt(varG(dh))),
             dh_best       = max(gv(dh)),
             transgressive = mean(gv(dh) > better),
             # how far the better parent sits above the DH mean, in DH s.d.
           gap_in_dh_sd  = (better - mean(gv(pop)[ij])) / as.numeric(sqrt(varG(dh))))
}))
fmt(lottery, 3)
```

cross	parent_mean	better_parent	f1_mean	f1_sd	dh_sd	dh_best	transgressive	gap_in_dh_sd
1 x 2	10.655	12.193	14.997	0	0.788	12.749	0.025	1.952
1 x 3	10.912	12.193	15.350	0	0.584	12.249	0.010	2.193
2 x 3	9.374	9.632	14.029	0	0.568	11.276	0.320	0.453

```
stopifnot(all(lottery[["f1_sd"]] < 1e-8))
stopifnot(which.max(lottery[["transgressive"]])
== which.min(lottery[["gap_in_dh_sd"]]))
```

`f1_sd` is **zero**, asserted to machine precision. Two fully inbred parents are homozygous everywhere, so every gamete each one makes is identical, and every F1 is the same heterozygote. Two hundred F1 plants from a cross of two inbred lines are two hundred copies of one genotype. There is no lottery yet – and this uniformity is the first reason a single-cross hybrid is a saleable variety.

`f1_mean` sits well above `better_parent`. That is heterosis, and under the model of Step 2 it has an exact value: for inbred parents i and j with allele indicators $u_{ik}, u_{jk} \in \{0, 1\}$ (their dosage halved), the F1 is heterozygous exactly where they differ, so

$$H_{ij} = G_{ij} - \frac{1}{2}(G_i + G_j) = \sum_k d_k \, \mathbb{1}[u_{ik} \neq u_{jk}] \tag{7.3}$$

The additive terms cancel from the mid-parent difference because an inbred parent's additive contribution is exactly half the F1's. `heterosis_value(X, d)` is this sum, with `X` the F1's 0/0.5/1 genotype:

```
H <- vapply(seq_along(crosses), function(s) {
  ij <- crosses[[s]]
  f1 <- makeCross(pop, matrix(ij, ncol = 2), nProgeny = 1, simParam = SP)
  hv <- as.numeric(heterosis_value(pullQtlGeno(f1, simParam = SP) / 2, d))
  stopifnot(isTRUE(all.equal(as.numeric(gv(f1)) - mean(gv(pop)[ij]), hv)))
  hv
}, numeric(1))
names(H) <- lottery[["cross"]]
fmt(H, 3)
#> 1 x 2 1 x 3 2 x 3
#> 4.342 4.438 4.655
```

Over a *set* of crosses the same identity has a closed form in the allele frequencies. Take the 60 founders as two arbitrary halves, "pool A" (lines 1–30) and "pool B" (lines 31–60) – arbitrary because `quickHaplo()` gave them a common history and no divergence, which Step 8 will add – and cross every A line to every B line. `hybridCross(crossPlan = "testcross")` is the full factorial. With p_{Ak} and p_{Bk} the derived-allele frequencies of the two pools at QTL k, the mean of H_{ij} over the factorial is `heterosis_expected(pA, pB, d)`, the identity of Chapter 13.:

$$\bar{H} = \sum_k d_k \left(p_{Ak} + p_{Bk} - 2\, p_{Ak}\, p_{Bk} \right) \tag{7.4}$$

```
A <- pop[1:30]
B <- pop[31:60]
fact <- hybridCross(A, B, crossPlan = "testcross", simParam = SP)
U  <- pullQtlGeno(A, simParam = SP) / 2
V  <- pullQtlGeno(B, simParam = SP) / 2
pA <- colMeans(U)
pB <- colMeans(V)
H_bar_obs     <-     as.numeric(meanG(fact))     -     (as.numeric(meanG(A))     +
as.numeric(meanG(B))) / 2
H_bar_exp <- heterosis_expected(pA, pB, d)
stopifnot(isTRUE(all.equal(H_bar_obs, H_bar_exp)))
c(n_hybrids    =    fact@nInd,    mean_A    =    as.numeric(meanG(A)),    mean_B    =
as.numeric(meanG(B)),
  mean_hybrids = as.numeric(meanG(fact)), heterosis_observed = H_bar_obs,
  heterosis_expected = H_bar_exp,
      pct_of_midparent    =    100    *    H_bar_obs    /    ((as.numeric(meanG(A))    +
as.numeric(meanG(B))) / 2)) |>
  fmt(3)
#>          n_hybrids             mean_A             mean_B       mean_hybrids
#>            900.000             10.216              9.784             14.427
#> heterosis_observed heterosis_expected   pct_of_midparent
#>              4.427              4.427             44.275
```

Exact, on the full factorial, because every A x B pairing is present once. The size – 44.3 percent of the mid-parent – follows from `meanDD` and from `quickHaplo()`'s frequencies near one half, where $p_A + p_B - 2p_Ap_B$ is near its maximum; it is not calibrated to a crop. D. S. Falconer and T. F. C. Mackay [12], Ch. 14, derive the heterosis of the F1 between two *random-mating* populations as $H_{F1} = \sum_k d_k y_k^2$ with $y_k = p_{Ak} - p_{Bk}$ (stated here in this chapter's notation rather than by their equation number; it is twice the `divergence` term of `heterosis_partition()`). The two do not conflict; they use different mid-parents. Theirs carries each population's own random-mating heterozygosity; an inbred line carries none, so the whole F1 heterozygosity is heterosis and the expression above keeps the `shared` term as well – Chapter 13. works the reconciliation through.

The lottery starts one meiosis later. `dh_sd` is the standard deviation among doubled haploids drawn from that F1, and it is substantial – this is the segregation that F2, DH and RIL populations exist to sample. It is also why Chapter 5. is a chapter: the number of DH lines you pull from a cross is the number of tickets you buy, and no downstream selection stage can recover a combination that was never sampled.

`transgressive` is the fraction of DH lines exceeding the *better* parent per se: 0.025, 0.01 and 0.32 for the three crosses. It depends strongly on the cross, and `gap_in_dh_sd` says why: the DH lines are centred on the mid-parent, so what matters is how far the better parent sits above it in units of the DH standard deviation – 1.95, 2.19 and 0.45. The cross with the smallest gap is the one with the most transgressive lines (asserted), about a third of them; the other two, whose better parent is two DH standard deviations above the mid-parent, give a few percent. Crossing two parents whose favourable alleles sit at different loci produces progeny beyond either of them – transgressive segregation, which I. J. Mackay, J. Cockram, and P. Howell [13] argue is the same phenomenon as heterosis and inbreeding depression viewed from a third angle – and how often depends on the gap between the parents relative to the segregation variance, not on a constant.

7.5 Step 5 – Selfing, doubled haploids, and why hybrid seed is remade every year

A plant breeder has to get from a heterozygous F1 to a homozygous line. There are two routes, and AlphaSimR implements both. Following the same cross through selfing generations also answers a question every hybrid grower has asked: what happens if the F1 seed is saved and sown again.

This is the one step where the allogamous, hybrid case and the autogamous case use literally the same primitive – `makeDH()` is exactly what Chapter 8. calls to take its wheat F1s straight to a homozygous line, one generation instead of the six or seven selfing generations traced below – and where they diverge again in what the resulting inbred line is *for*. In an autogamous programme that homozygous line is commonly the release: it breeds true, so what leaves the DH pipeline is what a farmer eventually grows. In this chapter's programme the homozygous DH line is a parent, held inside a heterotic pool, testcrossed and recycled (Step 6, Step 8), and never sold on its own; the object a grower plants is the F1 these lines make together, which is the reason it cannot be saved.

Under selfing without selection, the fraction of the F1's heterozygous loci still heterozygous in generation F_t is $(1/2)^{t-1}$ in expectation, so the expected mean of the selfed generation is

$$\mathbb{E}[M_{F_t}] = \frac{1}{2}(G_i + G_j) + H_{ij}\,(1/2)^{t-1} \tag{7.5}$$

with H_{ij} the F1 heterosis of Step 4. At $t = 2$ this is the classical result that the F2 retains half the F1 heterosis, $H_{F2} = \frac{1}{2}H_{F1}$, which D. S. Falconer and T. F. C. Mackay [12] derive in Ch. 14 for the F2 of a cross between two populations (stated here in this chapter's notation rather than by their equation number); the form above extends it to later selfed generations by the same argument, since the expected heterozygosity at every locus is halved each generation.

```
homozygosity <- function(p)
  mean(pullSegSiteGeno(p, simParam = SP) %in% c(0, 2))

# Seeded here so that the |z| < 4 check below does not inherit the RNG state
# of Step 4: this z-statistic is not exactly Gaussian at this sample size, so
# an unrelated upstream edit that reseeded Step 4 could otherwise flip the
# check and break the render even though it passes for the seed used here.
set.seed(55)
ij <- c(1, 2)
midparent <- mean(gv(pop)[ij])
f1  <- makeCross(pop, matrix(ij, ncol = 2), nProgeny = 400, simParam = SP)
gen <- f1
traj <- data.frame(generation = "F1", homozygosity = homozygosity(f1),
                   meanG = as.numeric(meanG(f1)), sdG = as.numeric(sqrt(varG(f1))),
                   expected = midparent + H[["1 x 2"]])
for (t in 2:7) {
  gen  <- self(gen, nProgeny = 1, simParam = SP)
  traj <- rbind(traj, data.frame(generation = paste0("F", t),
                                 homozygosity = homozygosity(gen),
                                 meanG = as.numeric(meanG(gen)),
                                 sdG = as.numeric(sqrt(varG(gen))),
                                  expected = midparent + H[["1 x 2"]] * (1 / 2)^(t
```

```
- 1)))
}
dh1 <- makeDH(f1, nDH = 1, simParam = SP)
traj <- rbind(traj, data.frame(generation = "DH", homozygosity = homozygosity(dh1),
                               meanG = as.numeric(meanG(dh1)),
                               sdG = as.numeric(sqrt(varG(dh1))),
                               expected = midparent))
traj[["z"]] <- (traj[["meanG"]] - traj[["expected"]]) / (traj[["sdG"]] / sqrt(400))
traj[["z"]][1] <- 0   # F1: no variance, and the identity of Step 4 is exact
fmt(traj, 3)
```

generation	homozygosity	meanG	sdG	expected	z
F1	0.501	14.997	0.000	14.997	0.000
F2	0.753	12.798	0.667	12.826	−0.818
F3	0.876	11.742	0.759	11.740	0.045
F4	0.938	11.206	0.792	11.197	0.226
F5	0.970	10.951	0.792	10.926	0.616
F6	0.985	10.823	0.797	10.790	0.825
F7	0.993	10.744	0.790	10.723	0.533
DH	1.000	10.604	0.809	10.655	−1.244

```
stopifnot(all(abs(traj[["z"]]) < 4))
```

Three columns, three lessons.

`homozygosity`: `self()` halves the remaining heterozygosity every generation – the familiar `1 - (1/2)^(t-1)` approach to fixation, which never quite arrives. `makeDH()` gets there in one step, at 1.0 exactly, by doubling a haploid gamete.

`meanG` against `expected`: the F2 mean falls by half the heterosis, the F3 by three quarters, and the DH lines land on the mid-parent. Each row is 400 plants, one per F1 plant: `self(nProgeny = 1)` carries 400 parallel, independent selfing lineages forward, so within a row the 400 values are independent and `sdG / sqrt(400)` is the standard error of the row's mean. The check is statistical: `z` is the observed minus expected mean in units of that standard error, and the chunk asserts every `|z| < 4`. Across rows the same 400 lineages are followed – each F_t plant is the selfed progeny of one F_{t-1} plant – so successive rows are correlated, and the table is 400 lineages checked at seven generations, not seven independent samples. Nothing about the parents changed; the loss is the heterozygosity being halved, times d. This is why hybrid seed is remade from the two inbred parents every season and why a maize F2 crop yields visibly less than the F1 it was saved from.

`sdG`: the F1 has none and the F2 has plenty. The F2 is not a variety; it is a segregating population, and a uniform, high-yielding one cannot be recovered from it by anything but selection and re-fixation – which is the line development a breeding programme does, not something a grower can do.

The saving DH buys is seasons, and seasons are the currency of a breeding programme: six or seven selfing generations against one DH cycle. Step 8′s scheme uses `makeDH()` for precisely this reason, and the cost that never shows up in the simulation is that DH production is a laboratory operation with its own failure rate and its own genotype-dependent response.

The other operations a scheme needs are all one call each: `randCross()` for random mating within a pool, `makeCross()` for a specified crossing plan, `hybridCross()` for a full factorial or a testcross between two groups, and `mergePops()` or `c()` to combine populations.

7.6 Step 6 – Phenotyping and selection: per se, then on the testcross

Selection needs a criterion. `setPheno()` supplies the noisy one; `selectInd()` acts on it. This step does it twice: first on a line's own phenotype, as the course does [10], to check the breeder's equation; then on the testcross, which is how a hybrid programme actually ranks lines.

7.6.1 Per se selection and the breeder's equation

```
be_draw <- function(q, r) {
  set.seed(500 + r)
  mx    <- setPheno(mixed, simParam = SP)
  sel   <- selectInd(mx, nInd = round(q * mx@nInd), use = "pheno", simParam = SP)
  prog  <- randCross(sel, nCrosses = 1000, simParam = SP)
  S     <- as.numeric(meanP(sel) - meanP(mx))
  vA    <- as.numeric(varA(mx, simParam = SP))
  i_emp <- S / sqrt(as.numeric(varP(mx)))
  r_emp <- stats::cor(bv(mx, simParam = SP)[, 1], pheno(mx)[, 1])
  c(i_empirical = i_emp,
    R_S_h2      = S * vA / as.numeric(varP(mx)),
    R_i_r_sdA   = i_emp * r_emp * sqrt(vA),
    R_observed  = as.numeric(meanG(prog) - meanG(mixed)))
}
n_be <- 20
sweep_q <- do.call(rbind, lapply(c(0.05, 0.10, 0.20), function(q) {
  draws <- t(sapply(seq_len(n_be), function(r) be_draw(q, r)))
  gap   <- draws[, "R_observed"] - draws[, "R_S_h2"]
  data.frame(q = q, i_normal = sel_intensity(q),
             t(colMeans(draws)),
             R_observed_sd = stats::sd(draws[, "R_observed"]),
             gap_mean = mean(gap), gap_sd = stats::sd(gap))
}))
fmt(sweep_q, 3)
```

q	i_normal	i_empirical	R_S_h2	R_i_r_sdA	R_observed	R_observed_sd	gap_mean	gap_sd
0.05	2.063	2.068	0.657	0.653	0.654	0.074	−0.003	0.081
0.10	1.755	1.755	0.558	0.554	0.563	0.048	0.006	0.048
0.20	1.400	1.399	0.445	0.442	0.456	0.045	0.011	0.045

Both forms of the breeder's equation are here, each averaged over 20 independent draws of the phenotypes, the selection and the progeny (the population selected from is the same `mixed` every time, so `varA` is fixed and only `S`, the accuracy and the Mendelian sampling vary). `R = S * h2` uses the selection differential directly, with `h2` the *narrow-sense* heritability `varA / varP` of the population selected from; `R = i * r * sigma_A` decomposes it into intensity, accuracy and available variation – the form Chapter 3. builds the book's objective side on. Because the trait has dominance, the accuracy `r` is the correlation of the phenotype with the **breeding value** (`bv()`), not with the total genetic value: only the additive part is transmitted. `R_observed` is the realised change in *genetic* mean between the selected parents' random-mated progeny and the starting population, which is what the prediction is a prediction of.

`gap_mean` is the paired difference between the observed response and `R_S_h2`, with `gap_sd` its standard deviation over the draws. The mean gap is −0.5 percent of the prediction at `q = 0.05` and 2.5 percent at `q = 0.20`; its standard error over the 20 draws is `gap_sd / sqrt(20)`, and the largest gap in units of that standard error is 1.1. A single draw is another matter: `R_observed_sd` says that one realisation of the response scatters by around 11 percent of the prediction at `q = 0.05`, which is why the course's one-off comparison [10] "did not match perfectly" and why this one is replicated. The equation predicts the additive response to first order in the allele-frequency change; what it leaves out is second order in that change and the dominance contribution of the progeny's departure from Hardy-Weinberg proportions, both small with 1000 progeny from 50 or more parents. The empirical intensity agrees with the normal-theory value to within 0.2 percent: with 1000 candidates and a criterion that is a sum over 1000 loci plus normal error, the two assumptions of `sel_intensity()` – an infinite population and a normal criterion – are nearly met.

The book exports its own selection intensity, and AlphaSimR exports one too. They are independent implementations of `phi(z) / q`:

```
q <- 0.05
c(hybdiv = sel_intensity(q), AlphaSimR = selInt(q),
  difference = abs(sel_intensity(q) - selInt(q))) |> fmt(10)
```

```
#>     hybdiv  AlphaSimR difference
#>   2.062713   2.062713   0.000000
```

`selectInd()` takes `use = "pheno"`, `"gv"` or `"ebv"`. Selecting on `"gv"` is selecting with perfect accuracy – impossible in a real programme, and the correct way to compute the ceiling a scheme is being measured against. `use = "ebv"` selects on whatever has been written into the population's `ebv` slot, which is how genomic selection enters a simulation (Chapter 8.). `selectTop = FALSE` selects downwards, for traits where less is better.

`selectWithinFam()` is the plant-specific one. A programme that truncates globally on a set of families will take every entry from the two best crosses and nothing from the rest; capping the number retained per family holds the programme's genetic base open at some cost in immediate gain. It is the same trade-off as the `max_use` parental cap of Chapter 19., applied one stage earlier.

7.6.2 Selection on the testcross: general combining ability

A maize breeder does not rank inbred lines on their own yield. A line is crossed to a **tester** from the opposite pool, the testcross hybrids are grown in yield trials, and the line is ranked on the mean of its testcrosses. This subsection is the one with no autogamous counterpart at all: a self-pollinated variety is judged, released and grown as itself, so selection there stops at the per se phenotype of Step 6′s first half. A hybrid parent is judged by a cross it will never be sold as, which is the entire reason general and specific combining ability exist as concepts. The quantity being estimated is the line's **general combining ability** (GCA), the average performance of its crosses expressed as a deviation from the mean of all crosses (G. F. Sprague and L. A. Tatum [9]); what a particular cross does beyond the sum of its two parents' GCAs is the specific combining ability (SCA). Comstock, Robinson and Harvey's reciprocal recurrent selection (R. E. Comstock, H. F. Robinson, and P. H. Harvey [7]) is built on using the opposite pool as the tester so that selection captures both.

On the full factorial of Step 4 the GCA of every A line against pool B can be computed exactly, and under the model of Step 2 it has a closed form. With u_{ik} the line's allele indicator and $\bar{G}$ the factorial mean,

$$\mathrm{GCA}_i = \frac{1}{n_B}\sum_j G_{ij} - \bar{G} = \sum_k \alpha_k^{(B)}\,(u_{ik} - p_{Ak}), \qquad \alpha_k^{(B)} = a_k + d_k\,(1 - 2\,p_{Bk}) \tag{7.6}$$

$\alpha_k^{(B)}$ is the average effect of substituting the line's allele into a *pool-B* background – the same $\alpha = a + d(q-p)$ of Step 3, with the frequency of the tester pool, not the line's own pool. That is the algebraic content of the sentence "combining ability is a property of a line against a tester": change the tester pool and the weights change. Its dominance part, $\sum_k d_k(1-2p_{Bk})(u_{ik}-p_{Ak})$, is the `gca_A` of `heterosis_gca_sca()` (Section 13.10), and the interaction residual of the factorial is that function's `sca`:

```
gca_true <- tapply(gv(fact)[, 1], fact@mother, mean)[A@id] - as.numeric(meanG(fact))
gca_formula <- drop(sweep(U, 2, pA) %*% (a + d * (1 - 2 * pB)))
stopifnot(isTRUE(all.equal(as.numeric(gca_true), as.numeric(gca_formula))))

hg <- heterosis_gca_sca(U, V, d)
stopifnot(isTRUE(all.equal(as.numeric(gca_true) - as.numeric(sweep(U, 2, pA) %*% a),
                           as.numeric(hg[["gca_A"]]))))

G_ij <- tapply(gv(fact)[, 1], list(fact@mother, fact@father), mean)[A@id, B@id]
sca  <- sweep(sweep(G_ij, 1, rowMeans(G_ij)), 2, colMeans(G_ij)) + mean(G_ij)
stopifnot(isTRUE(all.equal(unname(sca), unname(hg[["sca"]]))))

var_gca <- mean(gca_true^2)
var_sca <- mean(sca^2)
c(var_GCA_A = var_gca, var_SCA = var_sca,
  cor_perse_gv_vs_GCA = stats::cor(gv(A)[, 1], gca_true)) |> fmt(4)
#>           var_GCA_A             var_SCA cor_perse_gv_vs_GCA
#>              0.2678              0.0486              0.9973
```

The last number is the one to hold on to. With pools that have no divergence, p_{Bk} sits near one half at most loci, $(1-2p_{Bk})$ is near zero, and the dominance contribution to GCA nearly vanishes: a line's own genetic value ranks it for combining ability almost perfectly. Step 8 measures the same correlation on

pools with a real split, and it is lower there. Divergence is what makes the testcross informative beyond the line itself, and it is the reason heterotic pools are kept apart.

7.6.3 How many testers, how many locations

A programme cannot cross every candidate to every line of the other pool. It uses t testers and r locations, and the estimate of GCA_i is the mean phenotype of the line's t testcrosses, each an entry mean over r plots. Writing the testcross phenotype as $\bar{G} + \mathrm{GCA}_i + \mathrm{GCA}_j + \mathrm{SCA}_{ij} + e_{ij}$ with e_{ij} the entry-mean error of variance σ^2_e/r, and averaging over the t testers (whose own GCAs are common to every candidate and drop out of the ranking), the estimate is $\widehat{\mathrm{GCA}}_i = \mathrm{GCA}_i + \bar{s}_i + \bar{e}_i$, with $\bar{s}_i$ the mean of the line's SCA with its t testers and $\bar{e}_i$ the mean error. The statistic the chunk computes is a *sample* correlation between $\widehat{\mathrm{GCA}}_i$ and GCA_i over the $n_A = 30$ fixed candidate lines, with the t testers drawn without replacement from the $n_B = 30$ lines of pool B. To first order its expectation is

$$\rho(t,r) = \sqrt{\frac{v_g}{v_g + v_s + v_e}}, \qquad v_g = \frac{1}{n_A - 1}\sum_{i=1}^{n_A} \mathrm{GCA}_i^2, \quad v_s = \frac{\sigma^2_{SCA}}{t} \cdot \frac{n_A}{n_A - 1} \cdot \frac{n_B - t}{n_B - 1}, \quad v_e = \frac{\sigma^2_\varepsilon}{t\,r} \tag{7}$$

where v_g is the sample variance ($n_A - 1$ divisor) of the candidates' true GCA, because the correlation is taken over exactly those n_A points; $\sigma^2_{SCA} = \frac{1}{n_A n_B}\sum_{ij} \mathrm{SCA}_{ij}^2$ is the SCA variance on the factorial (computed above), $(n_B - t)/(n_B - 1)$ is the finite-population correction for t testers drawn without replacement from n_B, $n_A/(n_A - 1)$ turns the mean of the per-candidate variances of $\bar{s}_i$ into the expected sample variance, and σ^2_e is `SP$varE`, the plot error of Step 2. The textbook form of this prediction replaces v_g by the population variance of GCA and v_s by σ^2_{SCA}/t: it assumes an infinite tester pool and population variances. It is kept in the table as `predicted_infinite`, so the two can be compared. Neither corrects the second-order term of a sample correlation (the Jensen term); it is measured directly, not assumed away, in the Monte Carlo chunk below the accuracy grid, and left uncorrected because it is small at this n_A. The simulation measures the thing itself: repeated draws of the tester set and of the field error, at every combination.

```
grid  <- expand.grid(testers = c(1, 2, 4, 8), reps = c(1, 4))
n_acc <- 20
acc   <- sapply(seq_len(n_acc), function(rep) {
  set.seed(300 + rep)
  testers <- B[sample(B@nInd, max(grid[["testers"]]))]
  tc <- hybridCross(A, testers, crossPlan = "testcross", simParam = SP)
  apply(grid, 1, function(g) {
    ph  <- setPheno(tc, reps = g[["reps"]], simParam = SP)@pheno[, 1]
    use <- tc@father %in% testers@id[seq_len(g[["testers"]])]
    est <- tapply(ph[use], tc@mother[use], mean)[A@id]
    stats::cor(est, gca_true)
  })
})
# The prediction, for the statistic actually computed above: a sample
# correlation over the n_A FIXED candidates, with t testers drawn WITHOUT
# replacement from the n_B lines of pool B.
#   est_i = GCA_i + sbar_i + ebar_i
#   v_g : the candidates' sample variance of GCA (n_A - 1 divisor), because
#         the correlation is taken over exactly these n_A points;
#   v_s : sbar_i is the mean of t values from row i of sca, whose n_B entries
#         average zero, so Var(sbar_i) = (s2_i / t) (n_B - t) / (n_B - 1) with
#         s2_i = mean_j sca_ij^2; the column means of sca are zero too, so the
#         sample mean of sbar over candidates is exactly zero and the expected
#         sample variance is mean_i Var(sbar_i) n_A / (n_A - 1)
#         = var_sca n_A / (n_A - 1) (n_B - t) / (n_B - 1) / t;
#   v_e : varE / (t r), independent of everything else.
# To first order E[cor] = sqrt(v_g / (v_g + v_s + v_e)). The second-order
# (Jensen) term of the sample correlation is not corrected here; it is
# measured by Gaussian Monte Carlo in the chunk below the accuracy grid and
# reported there (`jensen_max`) rather than asserted in advance.
```

```
# `predicted_infinite` is the textbook form -- population variances, infinite
# tester pool -- kept so the two can be compared at this replication.
n_A <- A@nInd; n_B <- B@nInd
var_gca_n1 <- stats::var(gca_true)
fpc <- (n_B - grid[["testers"]]) / (n_B - 1)
v_s <- var_sca * (n_A / (n_A - 1)) * fpc / grid[["testers"]]
v_e <- SP$varE / (grid[["testers"]] * grid[["reps"]])
grid[["predicted"]] <- sqrt(var_gca_n1 / (var_gca_n1 + v_s + v_e))
grid[["predicted_infinite"]] <- sqrt(var_gca / (var_gca + var_sca / grid[["testers"]]
+ v_e))
grid[["observed_mean"]] <- rowMeans(acc)
grid[["observed_sd"]]   <- apply(acc, 1, stats::sd)
grid[["z"]] <- (grid[["observed_mean"]] - grid[["predicted"]]) /
  (grid[["observed_sd"]] / sqrt(n_acc))
fmt(grid, 3)
```

testers	reps	predicted	predicted_infinite	observed_mean	observed_sd	z
1	1	0.389	0.384	0.371	0.132	−0.612
2	1	0.513	0.507	0.521	0.163	0.203
4	1	0.646	0.639	0.620	0.085	−1.388
8	1	0.768	0.762	0.752	0.044	−1.625
1	4	0.628	0.622	0.622	0.107	−0.258
2	4	0.753	0.747	0.733	0.084	−1.028
4	4	0.852	0.847	0.843	0.039	−0.922
8	4	0.918	0.914	0.922	0.019	0.905

```
acc_max_z <- max(abs(grid[["z"]]))
acc_max_z_infinite          <-          max(abs(grid[["observed_mean"]]          -
grid[["predicted_infinite"]]) /
                            (grid[["observed_sd"]] / sqrt(n_acc)))
acc_max_gap <- max(abs(grid[["predicted"]] - grid[["predicted_infinite"]]))
c(max_abs_z = acc_max_z, max_abs_z_infinite = acc_max_z_infinite,
  max_gap_between_predictions = acc_max_gap) |> fmt(3)
#>                   max_abs_z          max_abs_z_infinite
#>                       1.625                       1.913
#> max_gap_between_predictions
#>                       0.007
```

```
# The second-order term the first-order prediction leaves out: the bias of a
# sample correlation over n_A points, by Gaussian Monte Carlo per cell,
# y = x + N(0, v_s + v_e), E[cor(x, y)] minus the first-order value. In the
# table above the candidates are the SAME 30 lines in every draw, so `jensen`
# holds x fixed at gca_true and draws only the noise -- the design actually
# measured. `jensen_random_x` redraws x ~ N(0, v_g) as well, the textbook
# small-sample bias of a correlation between two random variables, which is
# a different (larger) quantity and not the one the table is exposed to.
set.seed(600)
n_mc <- 4000
cor_cols <- function(x, y) {
  xc <- sweep(x, 2, colMeans(x)); yc <- sweep(y, 2, colMeans(y))
  colSums(xc * yc) / sqrt(colSums(xc^2) * colSums(yc^2))
}
jensen_mc <- function(x_of) vapply(seq_len(nrow(grid)), function(i) {
  x <- x_of()
  y <- x + matrix(stats::rnorm(n_A * n_mc, 0, sqrt(v_s[i] + v_e[i])), n_A)
  mean(cor_cols(x, y)) - grid[["predicted"]][i]
}, numeric(1))
jensen <- jensen_mc(function() matrix(gca_true, n_A, n_mc))
```

```
jensen_random_x <- jensen_mc(function()
  matrix(stats::rnorm(n_A * n_mc, 0, sqrt(var_gca_n1)), n_A))
names(jensen) <- names(jensen_random_x) <- paste0("t", grid[["testers"]], "_r",
grid[["reps"]])
jensen_max <- max(abs(jensen))
fmt(rbind(fixed_candidates = c(jensen, max_abs = jensen_max),
          random_x = c(jensen_random_x, max_abs = max(abs(jensen_random_x)))), 4)
```

	t1_r1	t2_r1	t4_r1	t8_r1	t1_r4	t2_r4	t4_r4	t8_r4	max_abs
fixed_candidates	−0.0032	0.0013	−0.0013	0.0019	−0.0023	0.0024	0.0009	0.0023	0.0032
random_x	−0.0074	−0.0049	−0.0075	−0.0085	−0.0041	−0.0054	−0.0037	−0.0033	0.0085

The Jensen term for the design actually run (`fixed_candidates`, `gca_true` held fixed at the 30 candidates) is at most 0.003 in absolute value across the eight cells, small next to the accuracies themselves (0.39 to 0.92), which is what "left uncorrected" means quantitatively; `random_x` is the larger, textbook small-sample bias of a correlation between two random variables, shown only for contrast and not the quantity this chapter's statistic carries.

Accuracy rises with both, and the two arguments buy different things. More locations shrink only the error term; more testers shrink the error term *and* the SCA term, because a line's specific interactions with its testers average out. The table is 20 draws of the tester set and of the field error per cell; the corrected and the textbook predictions differ by at most 0.007 here. `z` is the observed mean minus the corrected prediction in units of the standard error of that mean over the 20 draws; the largest `|z|` over the eight cells is 1.63 (it would be 1.91 against the textbook form), so at this replication the observed accuracies agree with the prediction. Each `observed_sd` is across tester sets and error draws, on a fixed set of candidates: what a single testcross trial's ranking is worth depends on which testers were drawn, not only on how many; with one tester at one location the spread across tester sets is 0.35 of the accuracy itself.

In the scheme of Step 8 the testers are 2 individuals of the opposite pool and the trials have one location, which puts it in the low-accuracy corner of this table. A real maize programme would use elite inbred testers and several locations; A. R. Hallauer, M. J. Carena, and J. B. Miranda Filho [14] is the reference on the choice of tester.

7.7 Step 7 – Two traits, and what equal weighting assumes

`corA` makes the traits genetically correlated, which is where selection stops being one-dimensional. This step uses an additive trait pair on its own founder draw: index theory is written on breeding values, and the point being made does not need dominance.

```
set.seed(202)
f2 <- quickHaplo(nInd = 80, nChr = 10, segSites = 100, inbred = TRUE)
SP2 <- SimParam$new(f2)
SP2$addTraitA(nQtlPerChr = 100, mean = c(10, 10), var = c(1, 4),
              corA = matrix(c(1, -0.4, -0.4, 1), 2, 2))
SP2$setVarE(h2 = c(0.6, 0.1))

mt <- newPop(f2, simParam = SP2)
for (k in 1:3) mt <- randCross(mt, nCrosses = 1000, simParam = SP2)
mt <- setPheno(mt, simParam = SP2)

G <- varG(mt); P <- varP(mt); a2 <- c(1, 1)
c(cor_genetic = stats::cov2cor(G)[2, 1],
  cor_pheno   = stats::cov2cor(P)[2, 1],
  h2_trait1   = as.numeric(diag(G)[1] / diag(P)[1]),
  h2_trait2   = as.numeric(diag(G)[2] / diag(P)[2])) |> fmt(3)
#> cor_genetic   cor_pheno   h2_trait1   h2_trait2
#>      -0.416      -0.084       0.470       0.072
```

The genetic correlation lands near the requested `-0.4`; the phenotypic one is much weaker, because the environmental components were left uncorrelated (`SP$setCorE()` is where that would change). The two traits differ in variance and, sharply, in heritability – the ordinary situation.

The book's pipeline defaults to `weights = NULL`, meaning equal weight on every trait. Step 7 is where that default gets examined, using the index functions Chapter 3. introduced:

```
cor_with_H <- function(w)
  drop(crossprod(w, G %*% a2)) /
    sqrt(drop(crossprod(w, P %*% w)) * drop(crossprod(a2, G %*% a2)))

Bw <- rbind(equal       = a2,
            smith_hazel = index_smith_hazel(P, G, a2),
            restricted  = index_restricted(P, G, a2, R = cbind(c(0, 1))))
# expected response per unit selection intensity on the index I = b'x:
# the vector of phenotypic differentials is S = i P b / sqrt(b'Pb), and the
# multi-trait breeder's equation R = G P^-1 S gives R = i G b / sqrt(b'Pb)
dG <- t(apply(Bw, 1, function(w) drop(G %*% w) / sqrt(drop(crossprod(w, P %*% w)))))
colnames(dG) <- c("gain_trait1", "gain_trait2")
idx_tab <- cbind(Bw, dG, aggregate = rowSums(dG),
                 cor_with_H = apply(Bw, 1, cor_with_H))
fmt(idx_tab)
```

	Trait1	Trait2	gain_trait1	gain_trait2	aggregate	cor_with_H
equal	1.0000	1.0000	0.0108	0.3612	0.3720	0.2434
smith_hazel	0.0789	0.0589	0.0437	0.3289	0.3727	0.2438
restricted	0.1986	0.0386	0.3179	0.0000	0.3179	0.2080

```
stopifnot(isTRUE(all.equal(idx_tab[, "aggregate"],
                           idx_tab[, "cor_with_H"] * sqrt(drop(crossprod(a2, G
%*% a2))),
                           check.attributes = FALSE)))
```

`gain_trait1` and `gain_trait2` are the expected genetic change in each trait per unit of selection intensity on the index, `G b / sqrt(b' P b)`, and `aggregate` is their `a`-weighted sum – which equals `cor_with_H` times the standard deviation of the aggregate genotype, asserted. Three readings, in the order they matter:

1. **Equal weights are not the Smith-Hazel solution.** Smith-Hazel puts 0.75 units of weight on trait 2 for every unit on trait 1, not 1.00. The index is `b = P^-1 G a` (H. F. Smith [15]; L. N. Hazel [16]), with no further structure, and `b` is proportional to `a` if and only if `a` is an eigenvector of `P^-1 G`. A sufficient condition is `G = c P` – every trait with the same heritability `c` and the genetic correlations equal to the phenotypic ones – which says nothing about the traits being equally variable or uncorrelated: `index_smith_hazel()`'s help page gives a correlated-trait pair (`G = I, P = [[2, 0.5], [0.5, 2]]`) for which equal weights are exact. Here the heritabilities are 0.47 and 0.07 and the genetic and phenotypic correlations −0.42 and −0.08, so `G` is nowhere near a multiple of `P`, and the table shows that `(1, 1)` is not an eigenvector of `P^-1 G`.
2. **The cost on the objective is modest here; the composition of the gain is not.** Equal weighting loses 0.2 percent of the attainable correlation with the aggregate genotype. But it moves trait 1 by 0.011 per unit intensity where Smith-Hazel moves it by 0.044, 4.1 times as much; both indices put most of the aggregate into trait 2, whose variance is four times larger, and the weights decide how much trait 1 moves at all. An index that is nearly as good on the objective can still land the gain somewhere else.
3. **The restricted index holds trait 2 at exactly zero**, at a cost of 15 percent of the aggregate. That is Kempthorne and Nordskog's construction

   (1) and, as Chapter 3. argues, the classical ancestor of this book's `alpha` constraint – restrict a direction rather than price it.

Smith-Hazel is defined as the weight vector maximising that correlation, so the claim is checkable against a numerical optimum rather than taken on faith:

```
opt <- stats::optim(c(1, 1), function(w) -cor_with_H(w), method = "BFGS")
c(numerical = -opt$value, smith_hazel = cor_with_H(Bw["smith_hazel", ]),
  difference = abs(-opt$value - cor_with_H(Bw["smith_hazel", ]))) |> fmt(10)
#>   numerical smith_hazel  difference
#>   0.2438417   0.2438417   0.0000000
```

Multi-trait genomic selection in wheat, balancing yield against protein content and quality, is the standing plant example – S. Michel, F. Löschenberger, C. Ametz, B. Pachler, E. Sparry, and H. Bürstmayr [17]. In the hybrid scheme below only trait 1 enters the ranking; the second trait rides along in the contract so that `stage2_select()` has an index to build.

7.8 Step 8 – The parts assembled: reciprocal recurrent selection

Everything above was one operation at a time. A breeding scheme is those operations in a loop, and the loop for two heterotic pools is reciprocal recurrent selection: each pool is improved using the *other* pool as tester. Doubled-haploid lines are drawn from pool A, testcrossed to pool B (and vice versa), the testcross hybrids are phenotyped, and the lines with the best mean testcross performance – the GCA estimate of Step 6 – are recombined to found the next cycle's pool. Elite lines from both pools, at the end, are crossed directly to produce the commercial hybrids. It is exactly the asymmetry Chapter 22. argues for: selection happens on the hybrid, recycling happens within pool.

7.8.1 Founding two pools with a real coalescent history

This is the one place the chapter pays for `runMacs2()` rather than `quickHaplo()`. `AlphaSimR::runMacs2(..., split = )` draws founder haplotypes from a population-split coalescent history: one ancestral population, then two demes of equal size that stopped exchanging migrants `split` generations ago, with the first half of the sampled individuals from one deme and the second half from the other. That is the realistic analogue of `simulate_lines()`'s Balding-Nichols draw in Chapter 4., and between-pool divergence is the entire subject of Chapter 2., so a stylised substitute would not do.

```
n_split_draws <- 20   # founder draws per split time, below
n_dh_gca      <- 10   # DH lines per founder for the GCA measurement, below
fp <- alphasimr_founder_pools(rrs_cfg)
founder_geno  <-  pullSegSiteGeno(c(fp[["pop_A"]],  fp[["pop_B"]]),  simParam  =
fp[["SP"]])
c(pool_A = fp[["pop_A"]]@nInd, pool_B = fp[["pop_B"]]@nInd,
  markers = ncol(founder_geno),
  heterozygous_fraction = mean(founder_geno == 1),
  gd_A = alphasimr_gd(fp[["pop_A"]], fp[["SP"]]),
  gd_B = alphasimr_gd(fp[["pop_B"]], fp[["SP"]])) |> fmt(3)
#>               pool_A               pool_B              markers
#>               12.000               12.000              750.000
#> heterozygous_fraction                 gd_A                 gd_B
#>                0.223                0.213                0.213
```

Reading `alphasimr_founder_pools()` against Steps 1 and 2, it is `runMacs2(nInd = , nChr = , segSites = , split = )` followed by `SP$addTraitAD(nQtlPerChr = seg_sites, meanDD = 0.4, varDD = 0.1)` – the same dominance model as Step 2 – and `SP$setVarE(h2 = 0.4)`. Two things about it must be said before any number is read. **The founders are outbred**: `runMacs2()` is called without `inbred = TRUE`, so each pool is a sample of a random-mating population, heterozygous at the fraction of sites printed above, as an open-pollinated maize population would be; the inbred lines of the scheme are made from them by `makeDH()` inside each cycle. And there is no separate marker panel: every one of the 150 segregating sites per chromosome is a QTL, so the sites that carry the trait are the ones `pullSegSiteGeno()` reads for `X`, `f` and `fL` throughout. `alphasimr_gd()` is `1 - mean(f)`, the exact definition `gene_diversity()` uses, computed from that pull, so the two pools are already on the same scale as every diversity number in the rest of the book.

Because these founders come from MaCS, no number in the rest of this section is reproducible from its seed – see Step 1: the founders move between runs, and everything below is computed from them. The chapter's response is to replicate every Step 8 measurement over independent founder draws and report the spread.

What `split` does. The bridge's default is `split_gen = 100`. The sweep below draws 20 independent founder sets at each of four split times and measures three things on each: the Caballero & Toro partition of Chapter 2. over the two pools (`ct_partition()`, the general form of `gd_partition()`: the F-statistics of their Eq. 6 and the partition $GD_T = GD_{WI} + GD_{BI} + GD_{BS}$ of their Eq. 7, from which $F_{ST} = GD_{BS}/GD_T$ follows as an identity derived in Chapter 2., not stated in the paper), the share of expected heterosis that the divergence term carries (`heterosis_partition()`, Chapter 13.), and – the

Step 6 question with divergence – how much of a DH line's exact GCA against the other pool's DH lines its own genetic value can see. For that last one, 10 DH lines are drawn from every founder, the full factorial between the two DH sets is made, and the GCA of each pool-A line is split into its additive and dominance parts with `heterosis_gca_sca()`; the identity behind that split was asserted in Step 6. The chunk also records, per draw, an ascertainment bookkeeping that the reading of `GD_T` will need: the fraction of sites monomorphic within each pool, the fraction fixed for opposite alleles in the two pools, and the part of `GD_T` those sites carry.

```
split_draw <- function(split, seed) {
  cfg <- utils::modifyList(rrs_cfg, list(split_gen = split, seed = seed))
  fp_s <- alphasimr_founder_pools(cfg)
  SP_s <- fp_s[["SP"]]
  pool <- rep(c("A", "B"), c(cfg[["n_pool_A"]], cfg[["n_pool_B"]]))
  geno <- pullSegSiteGeno(c(fp_s[["pop_A"]], fp_s[["pop_B"]]), simParam = SP_s)
  ct   <- ct_partition(geno, pool)
  pA_s <- colMeans(geno[pool == "A", ]) / 2
  pB_s <- colMeans(geno[pool == "B", ]) / 2
  d_s  <- SP_s$traits[[1]]@domEff
  a_s  <- SP_s$traits[[1]]@addEff
  hp   <- heterosis_partition(pA_s, pB_s, d_s)
  # Ascertainment: runMacs2() returns sites that segregate in the POOLED
  # sample, so a site fixed for different alleles in the two pools is kept
  # and, with equal pool sizes, sits at pooled frequency 1/2 -- the maximum
  # of 2 p q. GD_T = mean 2 pbar qbar exactly (census weights, equal sizes),
  # so it splits into the part those fixed differences carry and the rest.
  mono_A <- pA_s %in% c(0, 1)
  mono_B <- pB_s %in% c(0, 1)
  fixed  <- mono_A & mono_B & (pA_s != pB_s)
  pbar_s <- (pA_s + pB_s) / 2
  h_s    <- 2 * pbar_s * (1 - pbar_s)
  stopifnot(isTRUE(all.equal(ct[["diversity"]][["GD_T"]], mean(h_s))))
  set.seed(seed)
  dh_A <- makeDH(fp_s[["pop_A"]], nDH = n_dh_gca, simParam = SP_s)
  dh_B <- makeDH(fp_s[["pop_B"]], nDH = n_dh_gca, simParam = SP_s)
  hyb  <- hybridCross(dh_A, dh_B, crossPlan = "testcross", simParam = SP_s)
 gca  <- tapply(gv(hyb)[, 1], hyb@mother, mean)[dh_A@id] - as.numeric(meanG(hyb))[1]
  U_s  <- pullQtlGeno(dh_A, simParam = SP_s) / 2
  hg   <- heterosis_gca_sca(U_s, pullQtlGeno(dh_B, simParam = SP_s) / 2, d_s)
  # GCA = additive part + dominance part (the Step 6 identity), so its
  # variance is var(add) + var(dom) + 2 cov(add, dom): three terms, not two.
  gca_add <- as.numeric(sweep(U_s, 2, colMeans(U_s)) %*% a_s)
  gca_dom <- as.numeric(hg[["gca_A"]])
  stopifnot(isTRUE(all.equal(as.numeric(gca), gca_add + gca_dom)))
  data.frame(split = split, seed = seed,
             F_ST  = ct[["global"]][["F_ST"]],
             GD_T  = ct[["diversity"]][["GD_T"]],
             GD_BS = ct[["diversity"]][["GD_BS"]],
             mono_A = mean(mono_A), mono_B = mean(mono_B),
             fixed_diff = mean(fixed),
             GD_T_fixed = sum(h_s[fixed]) / length(h_s),
             GD_T_other = sum(h_s[!fixed]) / length(h_s),
             maf_pooled = mean(pmin(pbar_s, 1 - pbar_s)),
             heterosis_divergence_share = hp[["divergence"]] / hp[["total"]],
             GCA_var = stats::var(gca),
             GCA_var_add = stats::var(gca_add),
             GCA_var_dom = stats::var(gca_dom),
             GCA_cov2 = 2 * stats::cov(gca_add, gca_dom),
             GCA_dominance_share = stats::var(gca_dom) / stats::var(gca),
             cor_perse_GCA = stats::cor(gv(dh_A)[, 1], gca))
}
splits <- c(0, 10, 100, 1000)
sw <- do.call(rbind, lapply(splits, function(s)
```

```
  do.call(rbind, lapply(seq_len(n_split_draws), function(r)
    split_draw(s, seed = 100 * s + r))))
sw_mean <- aggregate(sw[, -(1:2)], by = list(split = sw[["split"]]), FUN = mean)
sw_sd   <- aggregate(sw[, -(1:2)], by = list(split = sw[["split"]]), FUN = stats::sd)
rownames(sw_mean) <- rownames(sw_sd) <- paste0("split_", sw_mean[["split"]])
cols_div <- c("split", "F_ST", "GD_T", "GD_BS", "heterosis_divergence_share")
cols_asc <- c("split", "GD_T", "mono_A", "mono_B", "fixed_diff", "GD_T_fixed",
              "GD_T_other", "maf_pooled")
cols_gca <- c("split", "GCA_var", "GCA_var_add", "GCA_var_dom", "GCA_cov2",
              "GCA_dominance_share", "cor_perse_GCA")
fmt(sw_mean[, cols_div], 3)
```

	split	F_ST	GD_T	GD_BS	heterosis_divergence_share
split_0	0	0.023	0.308	0.007	0.023
split_10	10	0.047	0.305	0.014	0.046
split_100	100	0.258	0.295	0.076	0.205
split_1000	1000	0.569	0.333	0.190	0.364

```
fmt(sw_sd[, cols_div], 3)
```

	split	F_ST	GD_T	GD_BS	heterosis_divergence_share
split_0	0	0.003	0.009	0.001	0.003
split_10	10	0.005	0.006	0.002	0.006
split_100	100	0.014	0.007	0.004	0.014
split_1000	1000	0.014	0.005	0.007	0.012

```
fmt(sw_mean[, cols_asc], 3)
```

	split	GD_T	mono_A	mono_B	fixed_diff	GD_T_fixed	GD_T_other	maf_pooled
split_0	0	0.308	0.070	0.073	0.000	0.000	0.308	0.228
split_10	10	0.305	0.107	0.108	0.000	0.000	0.305	0.225
split_100	100	0.295	0.327	0.323	0.010	0.005	0.290	0.217
split_1000	1000	0.333	0.573	0.553	0.188	0.094	0.239	0.267

```
fmt(sw_mean[, cols_gca], 3)
```

	split	GCA_var	GCA_var_add	GCA_var_dom	GCA_cov2	GCA_dominance_share	cor_perse_GCA
split_0	0	0.531	0.491	0.049	−0.008	0.093	0.954
split_10	10	0.410	0.376	0.040	−0.006	0.096	0.952
split_100	100	0.397	0.312	0.060	0.025	0.153	0.923
split_1000	1000	0.216	0.189	0.040	−0.012	0.188	0.907

```
fmt(sw_sd[, cols_gca], 3)
```

	split	GCA_var	GCA_var_add	GCA_var_dom	GCA_cov2	GCA_dominance_share	cor_perse_GCA
split_0	0	0.124	0.101	0.015	0.054	0.025	0.013
split_10	10	0.097	0.090	0.018	0.033	0.034	0.017
split_100	100	0.208	0.152	0.040	0.054	0.054	0.028
split_1000	1000	0.153	0.127	0.029	0.055	0.068	0.033

```
# the contrast between the two extreme split times, for the two GCA columns
contrast <- function(m) {
  x <- sw[[m]][sw[["split"]] == max(splits)]
  y <- sw[[m]][sw[["split"]] == min(splits)]
  se <- sqrt(stats::var(x) / length(x) + stats::var(y) / length(y))
```

```
  c(at_split_max = mean(x), at_split_0 = mean(y), difference = mean(x) - mean(y),
    se = se, t = (mean(x) - mean(y)) / se)
}
ctr <- rbind(GCA_dominance_share = contrast("GCA_dominance_share"),
             cor_perse_GCA       = contrast("cor_perse_GCA"))
fmt(ctr, 3)
```

	at_split_max	at_split_0	difference	se	t
GCA_dominance_share	0.188	0.093	0.095	0.016	5.856
cor_perse_GCA	0.907	0.954	−0.047	0.008	−5.850

Read the first two tables together: the first is the mean over the 20 draws, the second their standard deviation. What was measured: `F_ST` rises from 0.023 at `split = 0` to 0.569 at `split = 1000`, and `GD_BS` from 0.007 to 0.19, each step many standard deviations wide; the divergence share of expected heterosis rises from 0.023 to 0.364. `GD_T` does not stay flat: it is 0.308 at `split = 0`, 0.295 at `split = 100` and 0.333 at `split = 1000`, and the rise from `split = 100` to `split = 1000`, 0.038, is 19 standard errors at this replication. The third table is the bookkeeping behind that rise. Across the same range the fraction of sites monomorphic within pool A goes from 0.07 to 0.573 (pool B: 0.073 to 0.553), the fraction of sites fixed for opposite alleles in the two pools from 0 to 0.188, the part of `GD_T` those sites carry (`GD_T_fixed`) from 0 to 0.094, and the part carried by every other site (`GD_T_other`) falls from 0.308 to 0.239.

What is inferred from those three tables: `runMacs2()` returns the sites that segregate in the *pooled* sample of the two demes, so a site fixed for opposite alleles in the two pools is kept and, with equal pool sizes, sits at pooled frequency one half – the maximum of $2\bar{p}\bar{q}$, which is exactly what `GD_T` averages over sites (asserted in the chunk). The rise in `GD_T` therefore comes entirely from sites fixed for opposite alleles, an ascertainment effect of how the founder sites are sampled, while diversity at every other site falls with divergence. At those other sites the reading is the one Chapter 13. gives for expected heterosis: drift moves diversity from within pools to between them and does not create it. How much of `GD_T` a marker panel shows under divergence therefore depends on how its sites were ascertained, which is a property of the panel and not of the pools.

The fourth and fifth tables are the combining-ability side, mean and standard deviation, and the sixth is the contrast between the two extreme split times, unpaired because the draws are independent. GCA is the sum of its additive and dominance parts (asserted in the chunk), so its variance is $\mathrm{var}(\mathrm{add}) + \mathrm{var}(\mathrm{dom}) + 2\,\mathrm{cov}(\mathrm{add}, \mathrm{dom})$, three terms. What was measured, from `split = 0` to `split = 1000`: `GCA_var` goes from 0.531 to 0.216, `GCA_var_add` from 0.491 to 0.189, `GCA_var_dom` from 0.049 to 0.04, and `GCA_cov2` ($2\,\mathrm{cov}$) from −0.008 to −0.012, the last with between-draw standard deviations of 0.054 and 0.055, so at every split it is within 2.1 standard errors of zero. The ratio `GCA_dominance_share` $= \mathrm{var}(\mathrm{dom})/\mathrm{var}(\mathrm{GCA})$ rises from 0.093 to 0.188 (`t = 5.9`). Which part drives it: holding the dominance part at its `split = 0` value and moving only the additive part to its `split = 1000` value gives a ratio of 0.214; holding the additive part and moving only the dominance part gives 0.076. And the per se-versus-GCA correlation falls from 0.954 to 0.907 (`t = −5.9`). Both correlations are below the 0.997 of Step 6′s undifferentiated halves, but that comparison crosses a change of founders, trait draw and line number, so the within-sweep contrast is the one that isolates `split`.

What is inferred: in this render the fall of the *additive* part of the GCA variance accounts for more of the rise in the ratio than the growth of the dominance part; the additive part falls with divergence because the sites that fix within a pool stop contributing to it, so the ratio can rise without the dominance part growing at all. The ratio is a partition of $\mathrm{var}(\mathrm{GCA})$ only insofar as the covariance term is negligible; here that term is within 2.1 standard errors of zero at every split, which is the only sense in which "share" is used. As the pools separate, the dominance weights $(1 - 2p_{Bk})$ of Step 6 move away from zero and a growing part of a line's combining ability is invisible to its own genetic value; the evidence for that at this scale is the `t` of the contrast, and nothing here separates how much of the between-draw spread is the trait drawn with each founder set and how much is the DH sampling. The limit of the inference is the scale: 12 individuals per pool, 750 sites, 20 draws per split, and a `split` in coalescent generations that is not calibrated to any named heterotic-pool pair. The direction transfers; the numbers do not.

7.8.2 One cycle

```
set.seed(11)
cy1 <- alphasimr_rrs_cycle(fp[["pop_A"]], fp[["pop_B"]], fp[["SP"]], rrs_cfg)
fmt(cy1[["summary"]], 4)
```

mean_tc_A	mean_tc_B	gd_A	gd_B
2.4984	1.7352	0.1799	0.1871

`alphasimr_rrs_cycle()` runs the same six steps for each pool, symmetrically (pool A tested on B, pool B tested on A), and every one is a primitive from above:

1. **Double haploids** (Step 5). `makeDH()` draws 3 fully homozygous lines from *each* individual in the current pool – at book scale, 12 parents x 3 DH each = 36 DH lines per pool, per cycle. Because the pool is outbred, the DH lines from one parent differ from each other: this is the lottery of Step 4, drawn one generation earlier than in a line programme.
2. **Pick reciprocal testers.** The first 2 individuals of the *current* opposite pool are used as testers – an arbitrary slice by index, not a random or merit-based draw, outbred individuals rather than inbred lines, and a different set each cycle because step 6 recreates the pool. This is the RRS-defining move: a line's value is judged by its cross onto the other heterotic pool, not by itself.
3. **Testcross and phenotype** (Step 6). `hybridCross()` forms the full factorial between every DH line and every tester – 36 DH x 2 testers = 72 testcross hybrids per pool – and `setPheno()` adds phenotyping error sized to the heritability set by `SP$setVarE(h2 = )`, at one location.
4. **GCA estimate** (Step 6). Testcross phenotypes are grouped by DH-line parent (`tapply(pheno[, 1], mother, mean)`) and averaged across the 2 testers each line was crossed to. Only trait 1 is used for this ranking even though this chapter's config simulates 2 traits per line; the second trait rides along in `X`/`hybrids` but plays no role in selection here. Step 7 is what it would take to use both.
5. **Keep the elites** (Step 6). The 6 DH lines per pool with the highest mean testcross phenotype are kept as `elite_A`/`elite_B`.
6. **Recombine** (Step 5). `randCross()` random-mates the elites back up to 12 individuals, founding the next cycle's pool A (symmetrically for B).

`elite_A`/`elite_B` are what step 6 mates – and, in the *last* cycle, what the final commercial cross uses instead of being recombined again. The cycle's `summary` mixes two different populations on purpose: `mean_tc_A`/`mean_tc_B` average over *all* DH lines tested that cycle (the population's testcross performance before selection), while `gd_A`/`gd_B` are gene diversity of only the 6 surviving elites (the population after selection). That asymmetry is exactly what the next section plots – response measured pre-selection, diversity measured post-selection.

7.8.3 The trajectory across cycles, replicated, against a paired control

`alphasimr_pipeline()` runs `alphasimr_rrs_cycle()` in a loop for 4 cycles, carrying `pop_A`/`pop_B` forward from one cycle's output to the next's input. Only the *last* cycle's `elite_A`/`elite_B` go on to found the commercial hybrid pool; every earlier cycle's elites are spent entirely on `randCross()`ing the next cycle's pool.

One run of that loop is an anecdote, and it would be an anecdote about one founder draw. The loop is written out here instead, so that three things can be added to it: a **paired no-selection control** (`random_elite = TRUE`, which draws the elites at random and leaves every other operation – the DH lines, the testcrosses, the recombination – identical, so the difference between the two arms is selection and nothing else), the between-pool partition of the *recombined* pools each cycle, and the genetic value of the actual product, the elite factorial.

```
run_rrs <- function(fp, cfg, random_elite = FALSE, seed = 1) {
  set.seed(seed)
  SP_r  <- fp[["SP"]]
  pop_A <- fp[["pop_A"]]; pop_B <- fp[["pop_B"]]
  pool  <- rep(c("A", "B"), c(cfg[["n_pool_A"]], cfg[["n_pool_B"]]))
  pools_row <- function(pop_A, pop_B) {
    ct <- ct_partition(pullSegSiteGeno(c(pop_A, pop_B), simParam = SP_r), pool)
    data.frame(gd_pool_A = alphasimr_gd(pop_A, SP_r),
               gd_pool_B = alphasimr_gd(pop_B, SP_r),
```

```
              GD_BS = ct[["diversity"]][["GD_BS"]],
              F_ST  = ct[["global"]][["F_ST"]])
  }
  rows <- list(cbind(cycle = 0, mean_tc_A = NA, mean_tc_B = NA, gd_A = NA, gd_B = NA,
                     hybrid_meanG = NA, pools_row(pop_A, pop_B)))
  for (i in seq_len(cfg[["n_cycles"]])) {
    cy <- alphasimr_rrs_cycle(pop_A, pop_B, SP_r, cfg, random_elite = random_elite)
    pop_A <- cy[["pop_A"]]; pop_B <- cy[["pop_B"]]
    prod  <- hybridCross(cy[["elite_A"]], cy[["elite_B"]], crossPlan = "testcross",
                         simParam = SP_r)
    rows[[i + 1]] <- cbind(cycle = i, cy[["summary"]],
                           hybrid_meanG = as.numeric(meanG(prod))[1],   # trait 1
                           pools_row(pop_A, pop_B))
  }
  do.call(rbind, rows)
}

n_rep <- 48
arms <- lapply(seq_len(n_rep), function(s) {
  fp_s <- alphasimr_founder_pools(utils::modifyList(rrs_cfg, list(seed = s)))
  list(sel = run_rrs(fp_s, rrs_cfg, random_elite = FALSE, seed = s),
       ctl = run_rrs(fp_s, rrs_cfg, random_elite = TRUE,  seed = s))
})
# pairing check: cycle 1 is identical between arms up to the elite pick
stopifnot(all(vapply(arms, function(a)
  isTRUE(all.equal(a[["sel"]][c("mean_tc_A", "mean_tc_B")][2, ],
                   a[["ctl"]][c("mean_tc_A", "mean_tc_B")][2, ])), logical(1))))
```

48 founder draws; each drawn once and both arms branched off it, with the same seed set before each arm so that cycle 1's DH lines and testcross phenotypes are identical between arms (asserted) and the two only diverge at the selection step. The first replicate's selected arm, cycle by cycle:

```
fmt(arms[[1]][["sel"]], 3)
```

cycle	mean_tc_A	mean_tc_B	gd_A	gd_B	hybrid_meanG	gd_pool_A	gd_pool_B	GD_BS	F_ST
0	NA	NA	NA	NA	NA	0.230	0.222	0.074	0.247
1	2.994	2.935	0.200	0.179	3.871	0.199	0.171	0.102	0.356
2	3.774	3.668	0.150	0.163	4.832	0.150	0.161	0.116	0.428
3	4.221	5.030	0.112	0.124	5.825	0.107	0.128	0.145	0.552
4	6.055	5.904	0.092	0.105	6.193	0.095	0.105	0.153	0.604

`gd_pool_A`/`gd_pool_B` are the gene diversity of the recombined pools that enter the next cycle, on the same `1 − mean(f)` scale as the elites' `gd_A`/ `gd_B`; `GD_BS` and `F_ST` are the between-pool components of those two pools; `hybrid_meanG` is the mean genetic value of the 6 x 6 elite factorial, the product the cycle would sell. All of them, over all replicates, at the last cycle, selected minus control:

```
last <- rrs_cfg[["n_cycles"]] + 1
# The two pools are exchangeable by design, so their mean is a legitimate
# pooled statistic; how much noise it removes is measured below (pool_tab).
arms <- lapply(arms, function(a) lapply(a, function(x) {
  x[["gd_elite_mean"]] <- (x[["gd_A"]] + x[["gd_B"]]) / 2
  x[["gd_pool_mean"]]  <- (x[["gd_pool_A"]] + x[["gd_pool_B"]]) / 2
  x
}))
metrics <- c("mean_tc_A", "mean_tc_B", "hybrid_meanG", "gd_A", "gd_B",
             "gd_elite_mean", "gd_pool_A", "gd_pool_B", "gd_pool_mean",
             "GD_BS", "F_ST")
arm_last <- function(arm, m) vapply(arms, function(a) a[[arm]][[m]][last],
numeric(1))
diffs <- function(m) arm_last("sel", m) - arm_last("ctl", m)  # one per founder draw
paired <- function(m) {
```

```
  dd <- diffs(m)
  c(selected = mean(arm_last("sel", m)), control = mean(arm_last("ctl", m)),
    diff_mean = mean(dd), diff_sd = stats::sd(dd),
    t = mean(dd) / (stats::sd(dd) / sqrt(length(dd))))
}
rep_tab  <- t(sapply(metrics, paired))
t_paired <- rep_tab[, "t"]                       # named: t_paired[["gd_elite_mean"]]
t_crit   <- stats::qt(0.975, df = n_rep - 1)
sig_paired <- abs(t_paired) > t_crit              # named logical, same names
fmt(rep_tab, 3)
```

	selected	control	diff_mean	diff_sd	t
mean_tc_A	4.791	2.218	2.573	1.516	11.753
mean_tc_B	4.786	2.252	2.534	1.178	14.904
hybrid_meanG	5.329	2.259	3.070	1.489	14.290
gd_A	0.093	0.109	−0.016	0.022	−5.221
gd_B	0.093	0.107	−0.015	0.032	−3.187
gd_elite_mean	0.093	0.108	−0.015	0.020	−5.281
gd_pool_A	0.093	0.108	−0.015	0.021	−4.995
gd_pool_B	0.092	0.106	−0.014	0.032	−3.094
gd_pool_mean	0.092	0.107	−0.015	0.020	−5.051
GD_BS	0.148	0.134	0.015	0.012	8.382
F_ST	0.617	0.556	0.062	0.068	6.323

```
# Pooling the two pools halves the variance only if their paired differences
# are uncorrelated across founder draws: measured, not assumed.
pool_pair <- function(mA, mB) {
  dA <- diffs(mA); dB <- diffs(mB)
  c(cor_AB = stats::cor(dA, dB), sd_A = stats::sd(dA), sd_B = stats::sd(dB),
    sd_pooled = stats::sd((dA + dB) / 2),
    sd_if_independent = sqrt((stats::var(dA) + stats::var(dB)) / 4))
}
pool_tab <- rbind(gd_elite = pool_pair("gd_A", "gd_B"),
                  gd_pool  = pool_pair("gd_pool_A", "gd_pool_B"),
                  mean_tc  = pool_pair("mean_tc_A", "mean_tc_B"))
fmt(pool_tab, 3)
```

	cor_AB	sd_A	sd_B	sd_pooled	sd_if_independent
gd_elite	0.118	0.022	0.032	0.020	0.019
gd_pool	0.126	0.021	0.032	0.020	0.019
mean_tc	0.912	1.516	1.178	1.318	0.960

```
stopifnot(isTRUE(all.equal(pool_tab["gd_elite", "sd_pooled"],
                           rep_tab["gd_elite_mean", "diff_sd"])))
```

```
loss <- function(arm, m) vapply(arms, function(a)
  (a[[arm]][[m]][1] - a[[arm]][[m]][last]) / a[[arm]][[m]][1], numeric(1))
loss_tab <- rbind(
    `pool A, selected` = c(mean = mean(loss("sel", "gd_pool_A")), sd =
stats::sd(loss("sel", "gd_pool_A"))),
    `pool A, control`  = c(mean = mean(loss("ctl", "gd_pool_A")), sd =
stats::sd(loss("ctl", "gd_pool_A"))),
    `pool B, selected` = c(mean = mean(loss("sel", "gd_pool_B")), sd =
stats::sd(loss("sel", "gd_pool_B"))),
    `pool B, control`  = c(mean = mean(loss("ctl", "gd_pool_B")), sd =
stats::sd(loss("ctl", "gd_pool_B"))))
fmt(loss_tab, 3)
```

	mean	sd
pool A, selected	0.578	0.083
pool A, control	0.509	0.075
pool B, selected	0.586	0.111
pool B, control	0.523	0.093

```
fig_cap <- paste0("Reciprocal recurrent selection over ", rrs_cfg[["n_cycles"]],
  " cycles at book scale, ", n_rep, " founder draws, selection (solid) against ",
  "the paired no-selection control (dashed); points are means over draws and ",
  "bars one standard deviation. Left: mean testcross phenotype per pool, ",
  "measured on all DH lines tested that cycle. Right: gene diversity of the ",
  "elite lines retained each cycle.")
```

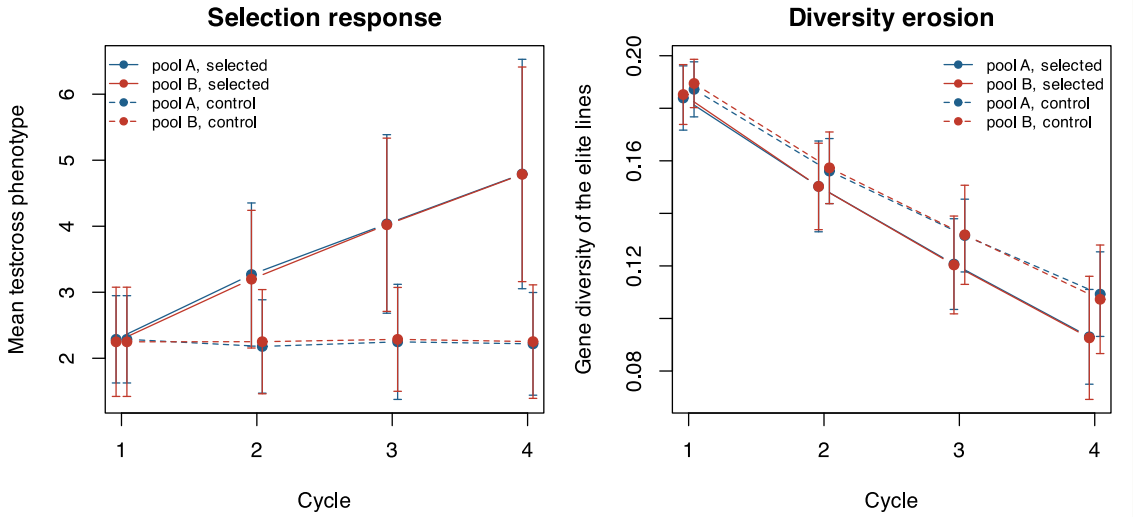

Figure 7.1: Reciprocal recurrent selection over 4 cycles at book scale, 48 founder draws, selection (solid) against the paired no-selection control (dashed); points are means over draws and bars one standard deviation. Left: mean testcross phenotype per pool, measured on all DH lines tested that cycle. Right: gene diversity of the elite lines retained each cycle.

What was measured (Figure 7.1 and the tables): over 48 paired replicates, the selected arm's mean testcross phenotype at cycle 4 differs from the control's by +2.57 (pool A) and +2.53 (pool B) trait units, with paired standard deviations of 1.52 and 1.18; the elite factorial's genetic value – the product – differs by +3.07 (`t = 14.3`). Gene diversity of the elite lines, averaged over the two pools, differs by −0.015 (paired s.d. 0.02, `t = -5.3`), and that of the recombined pools by −0.015 (`t = -5.1`). Between the pools, `GD_BS` differs by +0.015 (`t = 8.4`) and `F_ST` by +0.062 (`t = 6.3`). The two-sided 5% point of `t` with 47 degrees of freedom is 2.01. Against it, in this render, the response – both pools and the product – is beyond it; the pooled elite-line gene-diversity difference is beyond it; the pooled recombined-pool difference is beyond it; the `GD_BS` difference is beyond it; the `F_ST` difference is beyond it.

Pooling the two pools' differences is not free noise reduction, and `pool_tab` measures what it buys. Averaging halves the variance only if the two pools' paired differences are independent across founder draws. For the elite-line gene diversity they are correlated at 0.12, and the pooled standard deviation, 0.02, is close to the 0.019 it would be if they were independent: pooling does what it should there. For the testcross mean they are correlated at 0.91, and the pooled standard deviation is 1.32 against 0.96 if independent: pooling does not reduce that noise, because the founder draw moves both pools' response together. The per-pool rows are there to show the asymmetry between two exchangeable pools that any one set of 48 draws produces. The loss table says how much of the pools' loss is selection: the control loses gene diversity too – 6 elites recombined into 12 individuals is a bottleneck whether or not the elites were chosen – and the selected arm's mean loss exceeds the control's, with the pooled `t` just stated as the evidence.

What is inferred: with real meiosis and a dominance trait behind it, reciprocal recurrent selection responds in both pools, and at this replication the response is beyond `t_crit`. Whether it spends within-pool diversity faster than the same bottleneck under drift alone is what the pooled gene-diversity `t` decides, and in this render it does for the elite lines and does for the recombined pools: the selection component of the loss is of the same order as its paired standard deviation at this scale, so the conclusion, not only the number, is what the `t` decides. Where the sign holds, the control isolates that component from the drift part, which is the distinction Chapter 22. draws with the package's own faster simulator over a longer trajectory and a grid of selection intensities, and Chapter 8. draws a third time in an inbred-line programme. The between-pool rows: the `GD_BS` difference is beyond `t_crit`; where it is, and positive, RRS is moving the pools apart faster than drift does, which is what a scheme selecting on the cross is expected to do. `F_ST` rises partly because `GD_T` falls, so `GD_BS` is the row to read. What limits the inference is the scale – 12 founders per pool, 4 cycles, 2 testers at one location, 48 draws – and the fact that the founders come from MaCS, so both the numbers in this paragraph and any verdict that rests on a `t` near `t_crit` may differ between renders.

7.8.4 The final hybrid pool through `stage2_select()`, unchanged

`alphasimr_pipeline()` is the loop above without the instrumentation: founders, 4 cycles, then `alphasimr_make_hybrids()` testcrosses the final cycle's elites directly against each other – a full factorial between the 6 pool-A elites and 6 pool-B elites, so 36 commercial hybrids – and phenotypes them the same way every testcross in this chapter was phenotyped. `alphasimr_to_stage1()` then does the bookkeeping that makes this look like any other stage-1 output: it remaps AlphaSimR's internal line ids into the `1..n_A`, `n_A+1..n_A+n_B` convention `hybrids$line_A`/`line_B` and `fL` use elsewhere in the book, and builds `fL` by stacking the elite lines from *both* pools' genotypes into one matrix and computing a single joint coancestry matrix over all of them – not two separate per-pool matrices – which is what lets `theta_pools()` and the rest of the pipeline index into pool A and pool B lines with one consistent numbering.

```
res <- alphasimr_pipeline(rrs_cfg)
fmt(res[["cycles"]], 4)
```

cycle	mean_tc_A	mean_tc_B	gd_A	gd_B
1	3.4587	3.9198	0.1817	0.1928
2	4.5480	3.9639	0.1337	0.1504
3	6.6023	6.3502	0.1039	0.1322
4	8.5238	7.8568	0.0881	0.0882

```
st1 <- res[["stage1"]]
dim(st1[["X"]])
#> [1]  36 750
sel <- stage2_select(st1[["X"]], st1[["f"]], st1[["hybrids"]], n_sel = 8, fL =
st1[["fL"]],
                     verbose = FALSE)
fmt(sel[["metrics"]][, c("scenario", "alpha", "GD", "GD_BS", "F_ST", "index")], 4)
```

scenario	alpha	GD	GD_BS	F_ST	index
DE_alpha_0.00	−0.0034	0.2326	0.1510	0.6492	1.4574
DE_alpha_0.01	−0.0034	0.2326	0.1510	0.6492	1.4574
DE_alpha_0.02	−0.0034	0.2326	0.1510	0.6492	1.4574
DE_alpha_0.03	0.0003	0.2318	0.1526	0.6583	1.4605
ref_truncation	0.0003	0.2318	0.1526	0.6583	1.4605
ref_trunc_cap	0.0003	0.2318	0.1526	0.6583	1.4605
ref_max_div	−0.0291	0.2386	0.1499	0.6285	0.9246
ref_random	−0.0009	0.2320	0.1535	0.6614	−0.0673

This is one more founder draw, so its trajectory is one more replicate of the figure above, not the same run. Depending on the draw, `stage2_select()` may warn that the attainable `alpha` ceiling is at or below zero: with 36 hybrids over 12 lines, truncation on the index can pick a set no less diverse than a random one of the same size, and then the constraint has nothing to restrict and every `DE_alpha` row coincides

with `ref_truncation`. That is the same ceiling Chapter 19. computes on the book's own data, where the pool is large enough for it to be positive. Nothing here is AlphaSimR-specific: `stage2_select()` never sees where `X`, `f`, `hybrids` and `fL` came from, only that they satisfy the contract stage 1 promises. `GD_BS`, the between-pool divergence that carries the heterosis signal (Chapter 15.), is computed from `fL` exactly as it is everywhere else in the book.

7.9 Assumptions, and what this adds over Chapter 4.

What is more realistic here. Actual recombination and linkage (a genetic map per chromosome, not independent markers), a trait with directional dominance so that testcross selection has something real to select on and heterosis and inbreeding depression have a value, outbred founder pools with a coalescent split behind them, and multi-generation recurrent selection with a paired control rather than one static pool.

What is declared, and where. Steps 2–6 use inbred founders from `quickHaplo()` with every site a QTL and dominance degrees drawn at `meanDD = 0.4`, `varDD = 0.1`; the "pools" of Steps 4 and 6 are two undifferentiated halves of one founder set. Step 7 uses a separate additive founder draw. Step 8 uses outbred founders from `runMacs2(split = )`, again with every site a QTL; its DH lines are fully inbred, its testers are outbred individuals, and every loss of diversity is measured against the same run's founders (cycle 0) and against the paired no-selection control – not against a random-subset null of the same size, which is the reference Chapter 19.'s `alpha` uses.

What is still simplified. Testers are the first 2 individuals of the *current* opposite pool, an arbitrary slice by index – not chosen at random or by merit, and not the elite inbred tester a real programme would use; because `randCross()` recreates each pool every cycle, the testers change from cycle to cycle, so successive cycles' testcross means are not on a common tester and are comparable only through the pools' genetic level; there is one phenotyping environment (`setPheno()`'s error variance, no genotype-by-environment term – Chapter 8. adds one); the population-split coalescent history is a stylised divergence process, not calibrated to a named heterotic-pool pair; doubled-haploid production is instantaneous, skipping the field seasons Step 5 counted; and selection acts on trait 1 only – the 2 traits this chapter's config simulates are all carried through `X`/`hybrids`, but only the first ever enters the GCA ranking. The accuracy prediction of Step 6 is first order in the sample correlation – its finite-tester-pool and sample-variance corrections are applied, its second-order term is not – and the selfing trajectory of Step 5 follows 400 parallel lineages from one cross, checked at seven generations.

Scale. This chapter runs live at a book scale (12 + 12 founders, 750 markers, 4 cycles, 48 paired founder draws) – AlphaSimR is fast enough at this size that no caching is needed, unlike the MABC scans in Chapter 6.. Step 8's numbers may change on re-execution and do change between sessions, because its founders come from MaCS (Step 1); when they move, they move by about the amounts the standard deviations above say, and so may a conclusion that rests on a `t` near `t_crit`: between sessions, the paired `t` for the selection component of gene-diversity erosion has been seen both above and below `t_crit` at 24 pairs, which is why the chapter runs 48 pairs and writes each such inference conditionally on the render's own `t`. A heavier, production-scale run is `inst/scripts/run_alphasimr.R`:

```
Rscript inst/scripts/run_alphasimr.R --test          # smoke run, ~10 s
Rscript inst/scripts/run_alphasimr.R --out=DIR        # full run, ~2 min
```

It runs `alphasimr_pipeline()` at a larger configuration, feeds the resulting hybrid pool through `stage2_select()`, and writes the cycle trajectory and selection metrics as CSVs.

7.10 Where this connects

Chapter 4. and Step 8 build the same three-object contract from two different generative models – the fast one and the realistic one – which is the point of drawing that seam where the book does: everything past it, `build_ctx()` through `stage2_select()`, cannot tell which one it is looking at. Chapter 13. is the algebra Steps 3, 4 and 6 verify against AlphaSimR: inbreeding depression against `genParam()`, expected heterosis against a full factorial, and the GCA/SCA decomposition against the exact combining abilities. Chapter 3. supplies the breeder's equation Step 6 checks and the index theory Step 7 uses; Chapter 23. is where Step 3's `varA`-against-`genicVarA` gap becomes the subject rather than a diagnostic. Chapter 2. supplies the partition Step 8 measures across `split` and across cycles. Chapter 22. runs the same diversity-erosion argument at a longer horizon with the package's own faster simulator; this chapter is its cross-check

against an engine with real meiosis behind it, and Chapter 23. uses this chapter's linked arm for the same reason.

Chapter 8. is the other half of the method: the same primitives assembled into an inbred-line programme with staged yield trials, where the question is not which hybrids to sell but how fast a variety pipeline can run and what it spends to run faster.

Reproduce this chapter

All the code of this chapter, in order and with the explanation kept as comments, is one script: **04c-alphasimr-pipeline.R**. Download it, install `hybdiv`, and run it top to bottom in an empty directory; cached results are fetched on first use.

8. A complete plant breeding programme, and what it costs in diversity

Chapter 7. assembled AlphaSimR's primitives into a *hybrid* scheme: two heterotic pools, reciprocal testers, a commercial cross at the end. This chapter assembles the same primitives into the other kind of programme – an **inbred-line** pipeline, where the product is a variety rather than a hybrid, and where the organising problem is not which cross to sell but how many genotypes to carry through how many stages of field testing.

The scheme is the wheat programme of R. C. Gaynor *et al.* [18], reduced in scale. It is worth building because it is the shape most public and many commercial line-breeding programmes actually have, and because it exposes something the hybrid chapter cannot: **the trade between the number of genotypes tested and the precision with which each is measured.** A breeder has a fixed number of plots. Spending them on more entries at one location or fewer entries at eight locations is the same budget spent two ways, and the staged trial is the standard compromise.

Then the chapter does what the course this scheme comes from does not: it puts the book's diversity machinery over the result, and asks what the faster programme spends to be faster.

8.1 The scheme

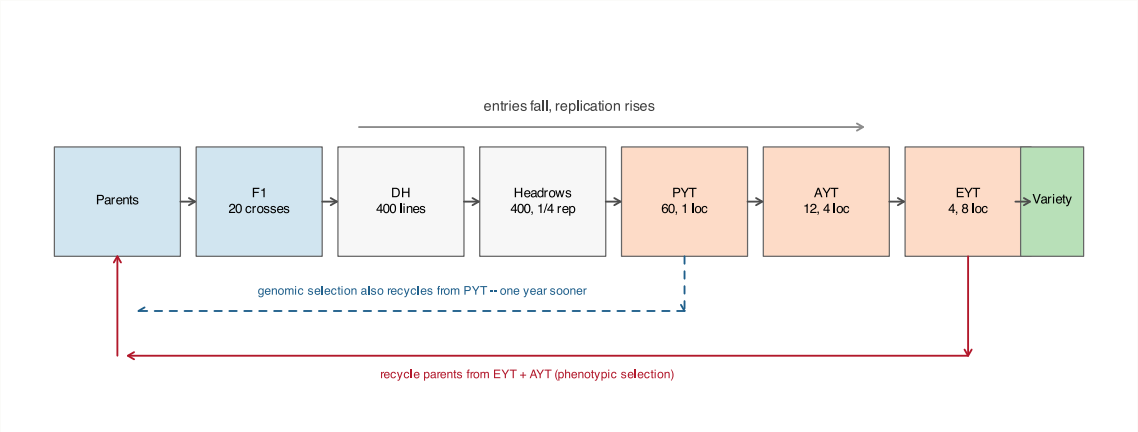

Figure 8.1: The staged inbred-line programme. Entries fall by roughly an order of magnitude at each stage while replication – the number of locations each entry is scored in – rises. Parents for the next cycle are recycled from the late trials (solid) or, under genomic selection, from the preliminary trial as well (dashed).

Seven stages, and the logic connecting them is a single idea: **at every step, entries fall and replication rises.** Four hundred doubled-haploid lines are scored once, cheaply, in unreplicated headrows; sixty survive into a preliminary yield trial at one location; twelve into an advanced trial at four; four into an elite trial at eight. Selection accuracy at the last stage is high because the error variance has been divided by eight locations – not because the trait became more heritable.

That is what `reps` does in `setPheno()`, and it is the argument Chapter 7.'s Step 2 flagged as the plant idiom. The configuration:

```
sch <- list(
  n_founders = 40, n_chr = 10, n_qtl = 60,   # founder genomes
  n_crosses  = 20, n_dh = 20, fam_max = 5,   # crossing and DH production
  n_pyt = 60, n_ayt = 12, n_eyt = 4,         # entries per yield trial
  rep_hdrw = 1/4, rep_pyt = 1, rep_ayt = 4, rep_eyt = 8,   # locations per stage
```

```
  var_e = 0.4, gs_boost = 3, n_burn = 7, n_years = 15
)
```

`rep_hdrw = 1/4` is the course's device for a stage *less* precise than a single plot: headrows are scored visually on a short row, which carries less information than one replicated plot, so the error variance is multiplied by four. `fam_max` caps how many DH lines any one cross may contribute to the preliminary trial – `selectWithinFam()` from Chapter 7.'s Step 6, holding the genetic base open against a truncation that would otherwise take the whole trial from the two best crosses.

8.2 Why a breeding programme needs a burn-in

The loop below advances every stage by one year, simultaneously. That only means something if all seven stages are already *occupied*, by cohorts of different ages: this year's elite trial entered the programme six years ago, this year's headrows entered last year. Starting the loop from an empty programme would compare a year-1 elite trial that does not exist against a year-1 headrow that does.

The fix is the staggered fill in the course's vignette, and it is the most confusing seven lines in it:

```
for (year in 1:7) {
  F1 <- randCross(Parents, n_crosses)
  if (year < 7) DH   <- makeDH(F1, n_dh)
  if (year < 6) HDRW <- setPheno(DH, reps = rep_hdrw)
  if (year < 5) PYT  <- ...
  if (year < 4) AYT  <- ...
  if (year < 3) EYT  <- ...
}
```

Read it as a triangle rather than a loop. On the first pass every stage runs, so every stage holds a cohort. On the second pass the elite trial is skipped, so the cohort sitting in it is a year older than the one behind it. By the seventh pass only the crossing block still runs. What comes out is a programme in which each stage holds a cohort of the right age relative to its neighbours – a *steady state*, which is the only state in which a per-year genetic gain is a meaningful number.

8.3 One programme, fifteen years

The whole scheme is one function. It is written out rather than packaged because the loop is the thing being taught, and because a reader adapting it to their own programme will edit the loop, not call it.

```
gd_of <- function(pop, SP)
  1 - mean(molecular_coancestry(pullSegSiteGeno(pop, simParam = SP) / 2))

run_programme <- function(cfg, seed, genomic = FALSE, gs_from = 1) {
  set.seed(seed)
  fg <- quickHaplo(nInd = cfg$n_founders, nChr = cfg$n_chr,
                   segSites = cfg$n_qtl, inbred = TRUE)
  SP <- SimParam$new(fg)
  SP$addTraitAG(nQtlPerChr = cfg$n_qtl, mean = 4, var = 0.1, varGxE = 0.2)
  ph <- function(p, r) setPheno(p, varE = cfg$var_e, reps = r, simParam = SP)

  # --- burn-in: fill every stage with a cohort of the right age ---
  Parents <- newPop(fg, simParam = SP)
  for (year in 1:cfg$n_burn) {
    F1 <- randCross(Parents, cfg$n_crosses, simParam = SP)
    if (year < 7) DH   <- makeDH(F1, cfg$n_dh, simParam = SP)
    if (year < 6) HDRW <- ph(DH, cfg$rep_hdrw)
    if (year < 5) {
      PYT <- selectWithinFam(HDRW, nInd = cfg$fam_max, use = "pheno", simParam = SP)
      PYT <- ph(selectInd(PYT, nInd = cfg$n_pyt, use = "pheno", simParam = SP),
                cfg$rep_pyt)
    }
```

```
    if (year < 4)
      AYT <- ph(selectInd(PYT, nInd = cfg$n_ayt, use = "pheno", simParam = SP),
                cfg$rep_ayt)
    if (year < 3)
      EYT <- ph(selectInd(AYT, nInd = cfg$n_eyt, use = "pheno", simParam = SP),
                cfg$rep_eyt)
  }

  # --- advancing years, last stage first so each reads the previous cohort ---
  out <- data.frame(year = 1:cfg$n_years, meanG = NA_real_, varG = NA_real_,
                    acc_pyt = NA_real_, gd_pyt = NA_real_, gd_parents = NA_real_)
  for (year in 1:cfg$n_years) {
    gs <- genomic && year >= gs_from

    Variety <- selectInd(EYT, nInd = 1, use = "pheno", simParam = SP)
    EYT <- ph(selectInd(AYT, nInd = cfg$n_eyt, use = "pheno", simParam = SP),
              cfg$rep_eyt)

    if (gs) {            # lever 1: a more accurate criterion at the PYT stage
      PYT@ebv <- setPheno(PYT, varE = cfg$var_e,
                          reps = cfg$rep_pyt * cfg$gs_boost,
                          onlyPheno = TRUE, simParam = SP)
      AYT <- ph(selectInd(PYT, nInd = cfg$n_ayt, use = "ebv", simParam = SP),
                cfg$rep_ayt)
    } else {
      AYT <- ph(selectInd(PYT, nInd = cfg$n_ayt, use = "pheno", simParam = SP),
                cfg$rep_ayt)
    }

    PYT <- selectWithinFam(HDRW, nInd = cfg$fam_max, use = "pheno", simParam = SP)
    PYT <- ph(selectInd(PYT, nInd = cfg$n_pyt, use = "pheno", simParam = SP),
              cfg$rep_pyt)
    HDRW <- ph(DH, cfg$rep_hdrw)
    DH   <- makeDH(F1, cfg$n_dh, simParam = SP)

    if (gs) {            # lever 2: recycle parents a year earlier, from the PYT
      PYT@ebv <- setPheno(PYT, varE = cfg$var_e,
                          reps = cfg$rep_pyt * cfg$gs_boost,
                          onlyPheno = TRUE, simParam = SP)
      EYT@ebv <- pheno(EYT); AYT@ebv <- pheno(AYT)
      Parents <- selectInd(c(EYT, AYT, PYT), nInd = cfg$n_eyt + cfg$n_ayt,
                           use = "ebv", simParam = SP)
    } else {
      Parents <- c(EYT, AYT)
    }
    F1 <- randCross(Parents, cfg$n_crosses, simParam = SP)

    out$meanG[year]      <- meanG(PYT)
    out$varG[year]       <- varG(PYT)
    out$acc_pyt[year]    <- stats::cor(PYT@gv[, 1], PYT@pheno[, 1])
    out$gd_pyt[year]     <- gd_of(PYT, SP)
    out$gd_parents[year] <- gd_of(Parents, SP)
  }
  out
}
```

Two details are worth pausing on.

The advancing loop runs backwards. The elite trial is filled *before* the advanced trial is refilled, which is before the preliminary trial is refilled. Written in the other order, `EYT` would be selected from an `AYT` that had already been overwritten with this year's cohort, and the programme would advance two stages per year without anyone noticing. Ordering is the whole correctness argument of a staged simulation.

Progress is reported at the PYT. It is the first stage with a yield phenotype and the last with enough entries for a variance to mean anything – `meanG` on four elite lines is noise. This is the course's choice and it is the right one.

```
ps <- run_programme(sch, seed = 1, genomic = FALSE)
fmt(ps[c(1, 5, 10, 15), ])
```

	year	meanG	varG	acc_pyt	gd_pyt	gd_parents
1	1	4.1209	0.1227	0.4863	0.4647	0.4353
5	5	4.6397	0.0868	0.1795	0.3985	0.4018
10	10	4.9367	0.0772	0.4830	0.3899	0.3644
15	15	5.0549	0.1296	0.4062	0.3778	0.3524

Genetic mean rises, genetic variance falls, and gene diversity of the preliminary trial falls with it. Nothing here is surprising; what matters is that all three are now being measured on the same run.

> **i** These numbers are reproducible
>
> The founders come from `quickHaplo()`, which is seeded by `set.seed()`, so `run_programme(sch, seed = 1)` returns the same data frame every time – unlike Chapter 7.'s Step 8, whose `runMacs2()` founders are not reproducible. That is what makes the paired comparison below legitimate: the phenotypic and genomic arms are run from the *same* seed, so they share founders, and every difference between them is attributable to the scheme.

8.4 Genomic selection: two levers, not one

Genomic selection is usually described as buying accuracy. It buys two things, and the second is normally the larger (R. C. Gaynor *et al.* [18]; T. H. E. Meuwissen, B. J. Hayes, and M. E. Goddard [19]):

1. **Accuracy earlier in the cycle.** A prediction is available on a line that has been genotyped but barely phenotyped, so the advance from preliminary to advanced trial can be made on better information than one location's yield.
2. **A shorter generation interval.** Parents can be recycled from the preliminary trial instead of waiting for the elite trial – and the breeder's equation of Chapter 3. is a rate *per unit time*, so cutting years off the cycle raises gain per year even at unchanged accuracy per selection.

This chapter mimics the prediction rather than fitting one: `ebv` is filled with a phenotype simulated at `gs_boost` times the replication, which is a stand-in for a genomic prediction of the corresponding accuracy. 3 is where the book actually fits prediction models and measures what accuracy is attainable; the honest statement here is that this chapter *assumes* an accuracy and studies its consequences, rather than earning it.

```
gs <- run_programme(sch, seed = 1, genomic = TRUE, gs_from = 6)
fmt(gs[c(1, 5, 10, 15), ])
```

	year	meanG	varG	acc_pyt	gd_pyt	gd_parents
1	1	4.1209	0.1227	0.4863	0.4647	0.4353
5	5	4.6397	0.0868	0.1795	0.3985	0.4018
10	10	4.9378	0.0663	0.3026	0.3969	0.3517
15	15	5.5676	0.0686	0.4497	0.3148	0.3038

Years 1 to 5 are identical to the phenotypic arm, digit for digit, because genomic selection only switches on in year 6. That is a matched control in the sense Chapter 23. uses the term: the two arms share a history, so the comparison is paired rather than between two independent runs.

8.5 The comparison, replicated

One run of one seed is an anecdote. Both arms are cheap enough to replicate.

```
seeds <- 1:24
reps <- do.call(rbind, lapply(seeds, function(s) {
  a <- run_programme(sch, s, genomic = FALSE)
  b <- run_programme(sch, s, genomic = TRUE, gs_from = 6)
  rate <- function(d) stats::coef(stats::lm(meanG ~ year,
                                            data = d[d$year >= 6, ]))[2]
  data.frame(seed = s,
             gain_ps = rate(a),      gain_gs = rate(b),
             gd_ps   = a$gd_pyt[nrow(a)], gd_gs = b$gd_pyt[nrow(b)],
             end_ps  = a$meanG[nrow(a)],  end_gs = b$meanG[nrow(b)])
}))

paired <- function(x, y) {
  d <- y - x
  c(mean_difference = mean(d), t = mean(d) / (stats::sd(d) / sqrt(length(d))))
}
fmt(rbind(`gain per year`      = paired(reps$gain_ps, reps$gain_gs),
          `gene diversity, yr 15` = paired(reps$gd_ps, reps$gd_gs),
          `genetic mean, yr 15`   = paired(reps$end_ps, reps$end_gs)))
```

	mean_difference	t
gain per year	0.0285	8.0851
gene diversity, yr 15	−0.0280	−2.8310
genetic mean, yr 15	0.2771	6.0496

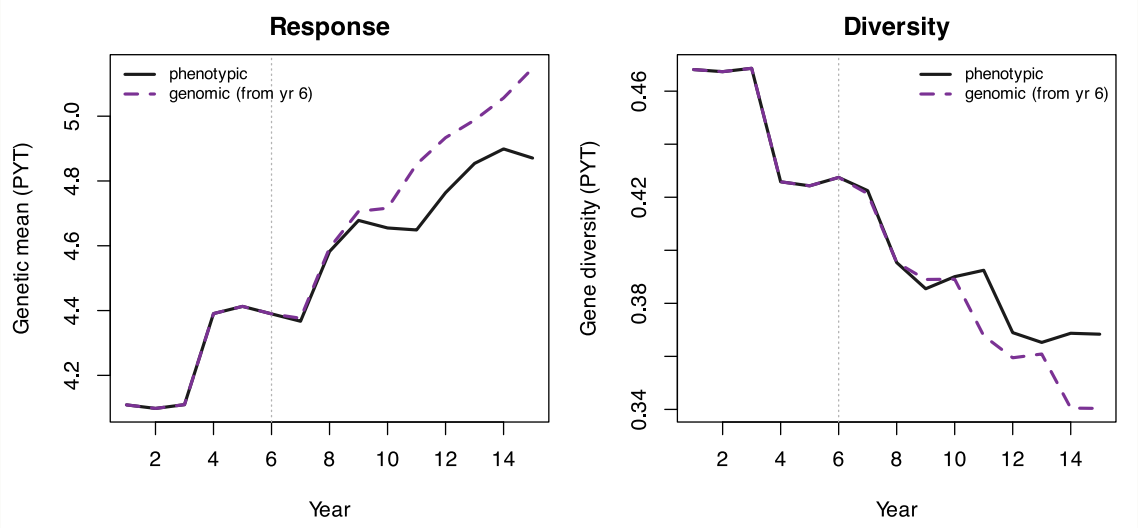

Figure 8.2: Twenty-four paired runs, genomic selection switched on in year 6. Left: genetic mean of the preliminary yield trial. Right: gene diversity of the same population. The faster programme is also the one spending diversity faster.

Genomic selection raises the rate of gain by roughly half, and the paired `t`-statistic leaves no doubt about the sign. That is the result R. C. Gaynor *et al.* [18] reports and the reason two-part strategies were adopted.

The second row is the one this book exists for. **Gene diversity in the preliminary trial is significantly lower under genomic selection at year 15**, and it is lower for a reason that is structural rather than incidental: recycling parents from the preliminary trial means recycling them a year before the trial data could have distinguished full sibs, so the parents chosen are more closely related than the ones the phenotypic arm would have chosen. A shorter generation interval is a higher rate of coancestry accumulation per unit time, not merely per cycle.

8.6 The diversity ledger

The measurement above used gene diversity of the preliminary trial. The parent set is the population that actually transmits, and it is the one an optimal contribution scheme would act on:

```
ledger <- data.frame(
  year       = yr,
  gd_pyt_ps  = avg("ps", "gd_pyt"),  gd_pyt_gs  = avg("gs", "gd_pyt"),
  gd_par_ps  = avg("ps", "gd_parents"), gd_par_gs = avg("gs", "gd_parents"))
fmt(ledger[c(1, 5, 10, 15), ])
```

	year	gd_pyt_ps	gd_pyt_gs	gd_par_ps	gd_par_gs
1	1	0.4681	0.4681	0.4418	0.4418
5	5	0.4243	0.4243	0.4158	0.4158
10	10	0.3901	0.3891	0.3812	0.3758
15	15	0.3684	0.3404	0.3628	0.3327

```
loss <- function(v) (ledger[[v]][1] - ledger[[v]][nrow(ledger)]) / ledger[[v]][1]
fmt(c(`PYT, phenotypic`   = loss("gd_pyt_ps"), `PYT, genomic`   = loss("gd_pyt_gs"),
             `parents,  phenotypic`  =  loss("gd_par_ps"),  `parents,  genomic`  =
loss("gd_par_gs")))
#>     PYT, phenotypic        PYT, genomic parents, phenotypic    parents, genomic
#>              0.2131              0.2729             0.1788              0.2470
```

Those four numbers are exactly the quantity this book calls `alpha`: the proportional loss of gene diversity, here measured against the programme's own starting point rather than against a random-selection baseline of the same size (Section 14.9 is where that distinction is made, and it matters – a same-size random baseline would attribute part of this loss to drift rather than to selection).

> **i** Status number, and why it is not the headline here
>
> `status_number()` on a set of fully inbred lines returns a value near or below 1, because a homozygous line has self-coancestry `f_ii = 1` and the diagonal dominates the mean. That is the same arithmetic Chapter 5. explains for a biparental cross, and it is correct rather than broken – but it makes `N_s` an unhelpful headline for a line-breeding programme. Gene diversity is the metric that stays interpretable across both inbred and outbred populations, which is why the ledger is written in it.

What would an `alpha` budget mean here? Not a constraint on which hybrids to sell, as in Chapter 19., but a constraint on **which parents to recycle**: the same optimal-contribution problem of Chapter 18., applied to the `c(EYT, AYT, PYT)` candidate set that the genomic arm selects from. The candidate set is small, the coancestry matrix is cheap, and the choice being constrained is annual. Chapter 22. is the argument for why that constraint pays back, and Chapter 23. is where the payback is predicted rather than observed.

8.7 Assumptions

What this models. Staged testing with replication rising across stages, genotype-by-environment interaction as a variance component (`SP$addTraitAG(varGxE = )`), doubled-haploid line development, within-family caps, a steady-state programme reached through a burn-in, and two distinct genomic-selection levers separated so their effects can be told apart.

What it does not. Genomic prediction is *mimicked* by adding replication, not fitted – so the accuracy is assumed rather than earned, and the training population, its size, and its relationship to the selection candidates play no role (3 is where they do). There is one trait, so the index questions of Chapter 7.'s Step 7 do not arise. Locations differ only through a variance component: there is no correlated environment structure, no year effect, and no location-by-year term, so a "location" here is an independent draw rather than a place. Doubled-haploid production is instantaneous and never fails. Founders come from `quickHaplo()`, so they carry no linkage disequilibrium – Chapter 7.'s Step 1 measures what that costs and argues it is the right trade for a chapter about response rather than about marker-QTL association.

Scale. 40 founders, 600 loci, 15 years, 24 paired seeds, all live at render. R. C. Gaynor *et al.* [18]'s programme is an order of magnitude larger in every dimension; the qualitative results here are the ones that survive the reduction, and the absolute rates are not comparable to a real wheat programme.

8.8 Where this connects

Chapter 7. supplies every primitive this chapter uses and builds them into the hybrid scheme instead; the two chapters are the same toolkit pointed at the two kinds of programme. Chapter 3. is where the breeder's equation says why a shorter generation interval raises gain per year. 3 is where the accuracy this chapter assumes is actually measured, and Chapter 12. is where marker-versus-QTL, the reason that accuracy is finite, is taken seriously.

The diversity result belongs to the book's spine. Chapter 22. shows the same tension – faster response, faster erosion – with the package's own simulator over a longer horizon; Chapter 23. explains why the erosion is what bounds the programme and when it can be predicted; Chapter 18. is the machinery for spending the budget deliberately rather than discovering afterwards what was spent.

Reproduce this chapter

All the code of this chapter, in order and with the explanation kept as comments, is one script: **04d-breeding-programme.R**. Download it, install `hybdiv`, and run it top to bottom in an empty directory; cached results are fetched on first use.

III Mating and testing design

9 How few hybrids are enough? Mating designs for a two-pool programme . . . 101

9.1 Why the crossing plan is the binding decision . 101
9.2 Combining ability as an estimation problem . . 102
9.3 The classical tester approach . . . 106
9.4 Sparse and incomplete factorials . . . 108
9.5 The analysis is part of the design . . . 111
9.6 Genomic information for deciding which crosses to make . . . 113
9.7 Seed production and the cost of a crossing plan . 116
9.8 A decision framework . . . 119
9.9 Open problems . . . 120
9.10 Reproducibility and assumptions . . . 121
9.11 Where this connects . . . 122
9.11 Reproduce this chapter . . . 122

10 Sparse testing: which hybrids go to which locations . . . 123

10.1 The allocation problem of stage 1 . . . 124
10.2 Foundations . . . 125
10.3 What sparse testing is, and where it came from . 126
10.4 Validation schemes: what each one answers . . 129
10.5 The empirical record across crops . . . 130
10.6 The minimum-overlap controversy . . . 132
10.7 Connectivity and checks . . . 133
10.8 Sparse testcrossing and the number of testers . 134
10.9 What sparsity does to variance components and to the BLUP . . . 135
10.10 Sparse testing with phenomics . . . 136
10.11 Simulation as a sizing tool . . . 137
10.12 Practical recommendations by stage . . . 145
10.13 Limitations of the evidence, and perspectives . 146
10.14 Reproducibility and assumptions . . . 147
10.15 Where this connects . . . 148
10.15 Reproduce this chapter . . . 149

11 Which crosses, at which locations? Joint sparse allocation . . . 151

11.1 Why the two decisions are one . . . 151
11.2 The joint model . . . 152
11.3 What the literature offers . . . 152
11.4 Design criteria computable before planting . . . 154
11.5 What the simulation finds . . . 155
11.6 Practical recommendations . . . 161
11.7 Open problems . . . 161
11.8 Reproducibility and assumptions . . . 161
11.9 Where this connects . . . 162
11.9 Reproduce this chapter . . . 162

9. How few hybrids are enough? Mating designs for a two-pool programme

Every chapter before this one took the hybrids as given. Chapter 7. made a commercial cross at the end of a reciprocal scheme, Chapter 4. derived populations from crosses already chosen, and the selection problems of the Optimisation part all start from a list of hybrids with predicted values. None of them asks where that list came from.

It comes from a crossing plan, and the crossing plan is the most constrained decision a hybrid programme makes. Two pools of 200 candidate lines each define a factorial of 40,000 single crosses. A field holds 1,000 to 3,000 plots. Every hybrid programme is therefore choosing a **sample** of a factorial whose cells are overwhelmingly empty, and the classical tester – every candidate crossed to one or two lines of the opposite pool – is one sample among many. This chapter treats the choice of sample as the design problem it is, and defends three results:

1. **At equal field size, a connected sparse factorial among the candidates estimates general combining ability better than a tester design**, at every budget studied, by 0.08 to 0.13 correlation points.
2. **The analysis matters more than the design.** Analysed by least squares, the sparsest factorials are the worst designs in the study; analysed by BLUP, they are among the best. The penalty for the wrong estimator is the largest single effect measured.
3. **Genomic information buys inference about lines that were never crossed, not better estimates for lines that were.** Where every candidate is in the field, genomic kernels did not raise GCA accuracy; where only a quarter of them were, they did.

The running example throughout is one simulated maize programme: two heterotic pools, 200 doubled-haploid candidates per pool, a field of 1,000 to 3,000 hybrid plots, and a winter nursery that must physically produce the seed of every cross that enters the field.

> **i** Where the numbers come from
>
> Two sources, and the chapter keeps them apart. **Live** results are computed at render time with the base-R design functions of `R/16_mating_design.R` – design generators, the connectivity diagnostic, least-squares and BLUP estimators with known variance components, and the cost model. **Cached** results come from a separate study (`sparsex`) that founded the pools in AlphaSimR, estimated variance components by REML, fitted genomic kernels and swept 24 designs x 3 plot budgets x 30 replicates x 3 models. That sweep needs AlphaSimR and an external REML check against `sommer`, and is not reproducible from this repository; its tables are in `book/data/mating_*.csv` and its figures in `book/figs/mating_*.png`. The phenotypic core of it *is* reproduced in-repo (`Rscript book/scripts/regenerate.R mating`), and the two are compared in Section 9.10.

9.1 Why the crossing plan is the binding decision

The crossing plan, not the statistical model and not the marker platform, sets the ceiling on what a hybrid programme can learn in a cycle. Which lines can be ranked, whether specific combining ability is estimable at all, how many plots each hybrid can occupy, whether untested combinations can be predicted – every one of these is fixed the moment the nursery plan is written. A mixed model fitted afterwards can only extract what the plan put there.

The arithmetic is unforgiving. A 200 x 200 factorial against 1,000 to 3,000 plots is a sampling fraction of 2.5 to 7.5 % at one plot per hybrid, and below 2 % once replication is required. Three decisions are usually conflated inside "the testing plan", and it pays to separate them:

- **which lines** enter the field at all – every candidate, or a subset chosen by some criterion;

- **which crosses** among those lines are made – to a common tester, to each other in a sparse pattern, or in complete blocks;
- **how many plots** each cross receives – replicates and locations.

The organising question of the chapter follows: *for a fixed budget, which sample of the factorial maximises the accuracy of combining-ability estimates and the selection response they support?* And one constraint is stated now, because it is the one most often forgotten: **a crossing plan is not a matrix, it is a nursery.** Each filled cell is a pollination that must yield enough seed for its plots.

The three canonical samples drawn over the same grid make the difference visible before any statistics. The plans below are produced by the package's generators, at 40 x 40 so that individual cells can be seen:

```
set.seed(3)
grids <- list("Tester, k = 2"     = design_tester(40, 40, k = 2),
              "Circulant, c = 4"  = design_circulant(40, 40, c = 4),
              "NC II, 10 x 10 sets" = design_nc2(40, 40, 10, 10))
op <- par(mfrow = c(1, 3), mar = c(1, 1, 2.2, 1))
for (nm in names(grids)) {
  M <- matrix(0, 40, 40)
  M[cbind(grids[[nm]]$parent1, grids[[nm]]$parent2)] <- 1
  image(t(M[40:1, ]), col = c("grey96", "#1f5f8b"), axes = FALSE,
        main = sprintf("%s  (%d crosses)", nm, nrow(grids[[nm]])), cex.main = 0.9)
  box(col = "grey60")
}
par(op)
```

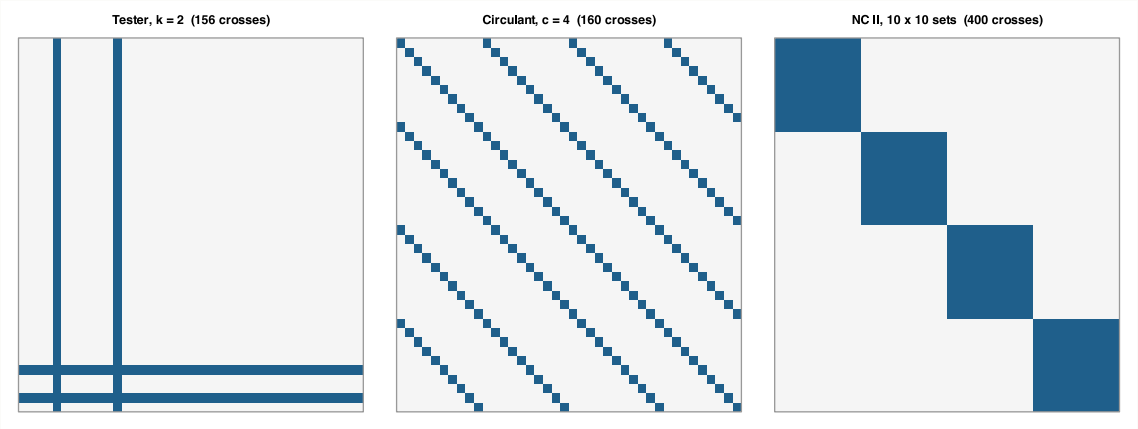

Figure 9.1: Three samples of the same 40 x 40 factorial, drawn by the package's own generators. Rows are pool-A lines, columns pool-B lines, a filled cell is a cross that goes to the field. Left: two testers per pool (156 crosses). Centre: a circulant sparse factorial with four crosses per line (160 crosses). Right: North Carolina Design II as four disconnected 10 x 10 blocks (400 crosses).

The tester fills two full columns and two full rows: every line meets the same partners, so lines are compared on a common background but the background is narrow. The circulant design spreads the same number of crosses as thin diagonals: every line meets different partners, every line is crossed exactly four times, and – the property that the next section shows to matter – every line is linked to every other through a chain of shared partners. NC II fills more cells than either and still cannot compare a line in the first block with a line in the second.

9.2 Combining ability as an estimation problem

General and specific combining ability are properties of a *reference population of crosses*, and a design estimates them only to the extent that it samples that population. G. F. Sprague and L. A. Tatum [9] defined them; B. Griffing [20] drew the distinction between fixed-effect inference about a specific set of parents and random-effect inference about the population they were drawn from; R. E. Comstock and H. F. Robinson [21] designed the North Carolina layouts to separate their variances, and R. E. Comstock, H.

F. Robinson, and P. H. Harvey [7] built reciprocal recurrent selection on them – the direct ancestor of the two-pool programme.

9.2.1 The model

At the level of a cross mean, the two-pool model is

$$\bar{y}_{ij} = \mu + g_i^A + g_j^B + s_{ij} + \bar{e}_{ij}, \qquad \operatorname{Var}(\bar{e}_{ij}) = \sigma_e^2 / r_{ij}, \tag{9.1}$$

with g^A, g^B the GCA of the pool-A and pool-B parent, s_{ij} the SCA of the pair, and r_{ij} the plots the cross received. Section 13.10 derived exactly this decomposition from allele frequencies, for the heterosis part of the hybrid value, and showed that a pool-A line's GCA is an average effect *evaluated at pool B's frequencies*. With an additive effect a_k added to the dominance deviation d_k, the general combining ability of pool-A line i is

$$g_i^A = \sum_k \left[\, a_k + d_k(1 - 2p_{Bk}) \right] \delta u_{ik}, \tag{9.2}$$

the classical structure of C. W. Stuber and C. C. Cockerham [22] for a cross between two populations. The additive part adds a term and changes nothing else, which is quick to confirm: the row effects of a complete factorial of genotypic values equal `cU %*% a` plus the `gca_A` of `heterosis_gca_sca()`, with no second implementation.

```
set.seed(7)
m  <- 300
pA <- runif(m, 0.1, 0.9); pB <- runif(m, 0.1, 0.9)
U  <- matrix(rbinom(30 * m, 1, rep(pA, each = 30)), 30)   # 30 inbred lines, pool A
V  <- matrix(rbinom(25 * m, 1, rep(pB, each = 25)), 25)   # 25 inbred lines, pool B
a  <- rnorm(m, 0, 0.3); d <- rnorm(m, 0.2, 0.2)
# Genotypic value of every single cross: a(u + v - 1) + d * heterozygosity
G  <- outer(seq_len(30), seq_len(25), Vectorize(function(i, j)
       sum(a * (U[i, ] + V[j, ] - 1) + d * (U[i, ] + V[j, ] - 2 * U[i, ] * V[j, ]))))
cs <- heterosis_gca_sca(U, V, d)
gcaA_formula <- drop(sweep(U, 2, colMeans(U)) %*% a) + cs$gca_A
c(max_abs_diff = max(abs((rowMeans(G) - mean(G)) - gcaA_formula)))
#> max_abs_diff
#>  2.04281e-14
```

Result. The row effects of the complete factorial and the closed form agree to machine precision. Interpretation. "True GCA" in this chapter is always this quantity – the row and column effects a breeder would estimate by growing every hybrid without error – and every accuracy reported is a correlation with it, over **all** candidates of a pool, crossed or not.

9.2.2 Four quantities that control a design

Table 9.1: The four quantities of a two-pool mating design, their closed forms and what controls them.

Quantity	Closed form	Controlled by
Rank of the GCA design matrix $[\mathbf{1} \mid Z_A \mid Z_B]$	$n'_A + n'_B -$ (connected components)	which crosses are made: the bipartite graph
Residual df for SCA at cross-mean level	$n_{\text{crosses}} -$ rank	crosses per line; zero for one tester per pool
LS sampling error of a GCA estimate, c crosses per line	$\sigma_s^2/c + \sigma_e^2/(c\,r)$	c buys down both terms, r only the second
LS accuracy of a GCA estimate	$\rho = \sqrt{\sigma_g^2/(\sigma_g^2 + \sigma_s^2/c + \sigma_e^2/(c\,r))}$	the same, as a correlation

The third row is the quantitative form of the argument of G. F. Sprague and L. A. Tatum [9] for using more than one tester, and the fourth is its square root as an accuracy: the square root of the heritability of a testcross mean [14]. Both describe **least squares**. A shrinkage estimator on a connected design borrows information across lines and exceeds them, which is why Section 9.5 exists.

The cached run validated the third and fourth rows against simulation in the complete 200 x 200 factorial, for $k = 1, 2, 4, 8$ testers and $r = 1, 2$ plots:

```
v4 <- subset(val, grepl("^V4", check))
fmt(data.frame(design = sub("cor\\(ghat_LS, GCA\\), ", "", v4$quantity),
               simulated = v4$observed, closed_form = v4$expected,
               rel_error_pct = 100 * v4$rel_error), 3)
```

Table 9.2: Least-squares GCA accuracy in a k-tester design: simulated (30-replicate mean, testers resampled) against the closed form. Cached from the `sparsex` validation suite, check V4.

design	simulated	closed_form	rel_error_pct
k=1 r=1	0.394	0.391	0.826
k=1 r=2	0.499	0.503	−0.840
k=2 r=1	0.509	0.515	−1.094
k=2 r=2	0.639	0.635	0.534
k=3 r=1	0.593	0.592	0.108
k=3 r=2	0.713	0.710	0.500
k=4 r=1	0.646	0.647	−0.119
k=4 r=2	0.758	0.758	−0.065
k=8 r=1	0.772	0.768	0.520
k=8 r=2	0.855	0.855	0.075

The largest relative error is 1.1 %. The expression is a reliable description of least squares on a tester design, and the cost optimiser of Section 9.7 uses it for that reason – knowing it is conservative for anything shrunk.

9.2.3 Connectivity, in one paragraph

! The rank rule

A mating design is a bipartite graph: pool-A lines on one side, pool-B lines on the other, an edge for every cross. GCA contrasts are estimable **within** a connected component of that graph and **not between** components, so the rank of the GCA design matrix is $n'_A + n'_B - (\text{components})$, one lost constraint per component. That is the formal content of what breeders call "ties between sets", and it was settled long ago: D. L. Weeks and D. R. Williams [23] gave the criterion for an N-way classification and R. Fernando, D. Gianola, and M. Grossman [24] an algorithm for the connected subsets of a two-way one.

`design_diagnostics()` computes both sides of the rule independently – the numerical rank by QR, the components by union-find over the graph – so a disagreement would be a bug, not a modelling choice:

```
set.seed(2026)
plans <- list("tester k=1"        = design_tester(n, n, 1),
              "tester k=2"        = design_tester(n, n, 2),
              "random sparse c=2" = design_random_sparse(n, n, 2),
              "circulant c=2"     = design_circulant(n, n, 2),
              "circulant c=5"     = design_circulant(n, n, 5),
              "NC II 10x10 sets"  = design_nc2(n, n, 10, 10))
diag_tab <- do.call(rbind, lapply(names(plans), function(nm) {
  d <- design_diagnostics(plans[[nm]], n, n)
  data.frame(design = nm, crosses = d$n_crosses,
             untested = 2 * n - d$lines_A - d$lines_B,
             components = d$components, rank_gca = d$rank_gca,
             rank_expected = d$rank_expected, df_sca = d$df_sca)
}))
```

```
stopifnot(all(diag_tab$rank_gca == diag_tab$rank_expected))
diag_tab
```

Table 9.3: Connectivity and estimability of five designs on the 200 x 200 running example. `rank_gca` is the numerical rank of the GCA design matrix; `rank_expected` is the graph prediction; `df_sca` is what is left at cross-mean level to separate SCA from error.

design	crosses	untested	components	rank_gca	rank_expected	df_sca
tester k=1	399	0	1	399	399	0
tester k=2	796	0	1	399	399	397
random sparse c=2	400	53	11	336	336	64
circulant c=2	400	0	1	399	399	1
circulant c=5	1000	0	1	399	399	601
NC II 10x10 sets	2000	0	20	380	380	1620

Result. The rank equals the graph prediction in every row. The NC II layout – twenty disconnected 10 x 10 blocks – has rank 380, not 399: twenty components, twenty lost constraints. The random sparse design at two crosses per line leaves 53 candidates with no cross at all and splits the rest into 11 components; the circulant design with the same number of crosses is a single component with every line crossed exactly twice. One tester per pool is connected and has **zero** residual df: $n_A + n_B - 1$ crosses for $n_A + n_B - 1$ GCA parameters. The GCA model is saturated and SCA is not merely imprecise, it is not identifiable.

What "not estimable" means is checked directly. A contrast between two lines of the same NC II block lies in the row space of the design matrix; a contrast between lines of different blocks does not:

```
nc <- plans[["NC II 10x10 sets"]]
l_within  <- c(0, 1, -1, rep(0, 2 * n - 2))           # A1 - A2, same block
l_between <- c(0, 1, rep(0, 9), -1, rep(0, 2 * n - 11)) # A1 - A11, next block
rbind(within  = unlist(design_estimable(nc, n, n, l_within)),
      between = unlist(design_estimable(nc, n, n, l_between)))
#>         estimable rel_residual
#> within          1 3.831056e-16
#> between         0 2.236068e-01
```

The within-block residual is rounding error; the between-block residual is 0.224, a contrast nowhere near the row space. Any least-squares ranking that crosses block boundaries in this design is a statement about the analyst's centring convention, not about the lines.

BLUP does rank across blocks, and in the simulations below NC II scores well on all-lines accuracy. It can, because a random-effects model supplies the missing constraint from its prior: every block's lines are assumed drawn from the same pool distribution, so every block mean is shrunk to the same value. That assumption is exactly true in a simulation that assigns lines to blocks at random, and exactly what fails when blocks are families, years or breeders' sub-programmes with real mean differences. Disconnection is not repaired by the estimator; it is replaced by an assumption, and the assumption should be stated.

9.2.4 SCA and the cross-level residual are the same thing in cross means

One more property of the model must be stated before any variance component is reported. Each cross has its own SCA, so in cross-mean data s_{ij} and $\bar{e}_{ij}$ occupy exactly the same position: only their sum $\sigma^2_s + \sigma^2_e/r$ is identified. The cached study separated them by fixing σ^2_e from the within-cross plot variance of stage 1 and estimating σ^2_s; an independent REML implementation (`sommer`) estimated a residual scale instead and put σ^2_s at zero, and the two agreed on the **sum** to 2e-06 relative error. A reported σ^2_{SCA} from cross means is only as good as the external plot-error estimate it was separated with. `fit_gca()` in this package sidesteps the question by taking the components as known, and weights each cross mean by $1/(\sigma^2_s + \sigma^2_e/r)$ for exactly this reason.

9.3 The classical tester approach

The tester design answers the sampling problem by collapsing one margin of the table. Each candidate is crossed to k testers of the opposite pool, and in the usual reciprocal arrangement each pool's candidates are tested against the other's testers, giving $k(n_A + n_B) - k^2$ crosses. The topcross method goes back to M. T. Jenkins and A. M. Brunson [25], the argument for general versus specific combining ability to G. F. Sprague and L. A. Tatum [9] and F. H. Hull [26], and the modern treatment of tester choice is A. R. Hallauer, M. J. Carena, and J. B. Miranda Filho [14].

9.3.1 What a tester is for, and what it trades

A tester is a **deliberate variance trade**. It buys cheap seed production and perfect connectivity – every line tied to every other through the common testers – at the price of confounding a line's GCA with its SCA *to the testers used*, and of a training set that covers the relationship space of the opposite pool very thinly. The testcross mean of line i estimates g_i^A plus the mean of k SCA deviations, so its error variance carries σ_s^2/k and $\sigma_e^2/(kr)$. Adding testers buys down both terms; adding replicates buys down only one. That is why the allocation literature treats tester number as a decision variable jointly with lines, locations and stages [27], and why it is not interchangeable with replication.

The closed form and its least-squares simulation on the running example, with the truth redrawn in every replicate so that tester SCA is resampled as the theory assumes:

```
set.seed(42)
tester_mc <- do.call(rbind, lapply(c(1, 2, 3, 4, 8), function(k) {
  do.call(rbind, lapply(1:2, function(r) {
    acc <- replicate(12, {
      tr <- simulate_factorial(n, n, vc$A, vc$B, vc$S)
      p  <- design_tester(n, n, k)
      p$reps <- r
      p  <- simulate_cross_means(p, tr, s2e)
      f  <- fit_gca(p, n, n, vc$A, vc$B, vc$S, s2e, "ls")
      mean(c(cor(f$gA, tr$gA), cor(f$gB, tr$gB)))
    })
    data.frame(k = k, r = r, acc = mean(acc), se = sd(acc) / sqrt(length(acc)))
  }))
}))
kk <- seq(1, 8, by = 0.1)
plot(NA, xlim = c(1, 8), ylim = c(0.3, 0.9), xlab = "testers per pool, k",
     ylab = "GCA accuracy (all 200 lines)", las = 1)
cols <- c("#1f5f8b", "#b2182b")
for (r in 1:2) {
  lines(kk, gca_accuracy_theory(kk, r, (vc$A + vc$B) / 2, vc$S, s2e),
        col = cols[r], lwd = 2)
  d <- tester_mc[tester_mc$r == r, ]
  arrows(d$k, d$acc - 2 * d$se, d$k, d$acc + 2 * d$se, angle = 90,
         code = 3, length = 0.03, col = cols[r])
  points(d$k, d$acc, pch = 19, col = cols[r])
}
legend("bottomright", c("1 plot per testcross", "2 plots per testcross"),
       col = cols, lwd = 2, pch = 19, bty = "n")
```

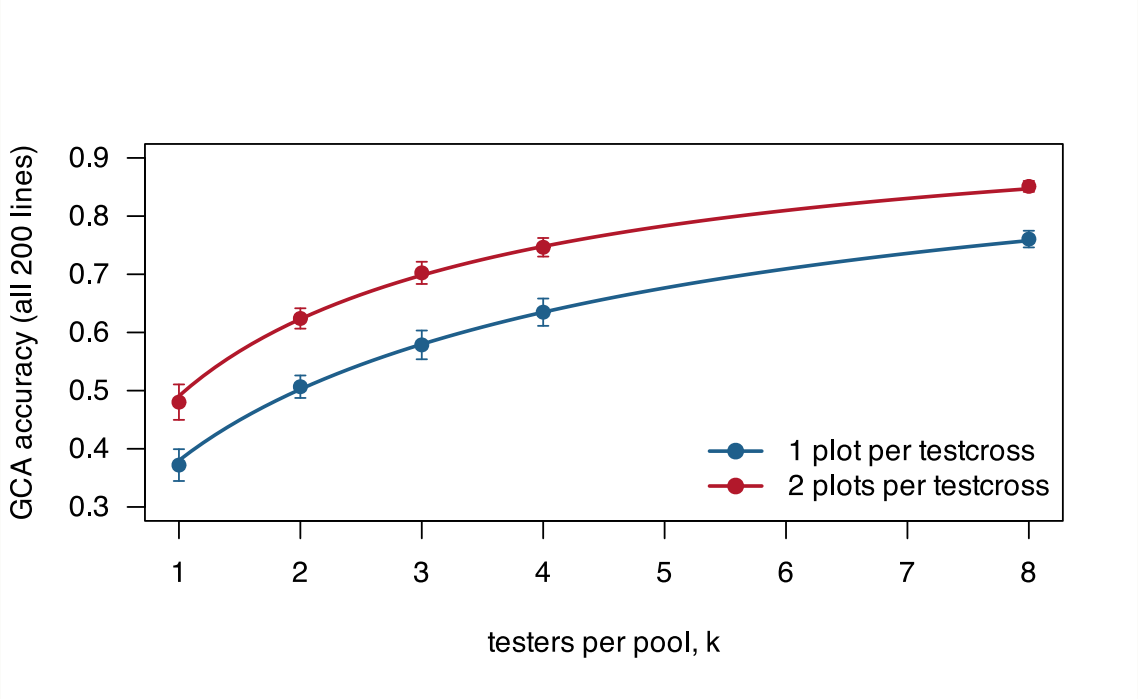

Figure 9.2: GCA accuracy of a k-tester design against the number of testers, at one and two plots per testcross. Lines: the closed form of Table 9.1. Points: least-squares estimates from `fit_gca()` on the 200 x 200 running example, mean of 12 replicates with the truth redrawn each time; bars are two standard errors.

Result. The simulated least-squares accuracies sit on the closed form at every k and both replication levels, which is the in-repo counterpart of the cached check V4. The returns to testers diminish fast: going from one to two testers at one plot raises accuracy from 0.38 to 0.5; going from four to eight adds 0.12. Interpretation. Here the plot error, $\sigma^2_e \approx 42.6$ at a single-plot heritability of 0.3, dwarfs $\sigma^2_s = 2.899$, so testers and replicates are nearly interchangeable *in this simulation*. In material where SCA is 20 to 30 % of the genetic variance [28], [29] the σ^2_s/k term is no longer negligible and testers buy strictly more than replicates. Section 13.12.6 gives the finite-population version of the same curve, with k testers drawn without replacement from the n_B available.

9.3.2 Choice of tester

The long-standing debate is between broad-base testers, elite inbreds and elite single crosses. The theoretical argument against a high-performing tester is O. S. Smith [30]'s: for many loci with substantial dominance, favourable dominant alleles carried by a good tester *mask* the candidate's alleles, reducing the variance among testcrosses and with it the correlation between testcross and line value. A good tester discriminates less. The counter-argument is operational: an elite tester produces hybrids that are themselves candidate products, so the testcross trial doubles as a product trial in the background where the line will actually be sold [14]. Programmes have resolved the tension by staging – discriminating testers early, elite testers late, single-cross evaluation last – and the number of testers per stage turns out to be among the highest-leverage parameters of the whole scheme: replacing a 4-1-1 tester plan with a 2-2-1 plan, with doubled haploids and higher selection intensity, raised gain per dollar by 43.1 % and cut the cost of a unit of gain by 28.5 % in a simulated African maize programme [31].

9.3.3 What testers cannot deliver

Three limitations recur, and the simulation adds numbers to two of them.

No SCA in the reference population. With $k = 1$ the GCA model is saturated and σ^2_{SCA} is not identifiable at all (`df_sca` = 0 in Table 9.3). With $k \geq 2$ it is identified, but the cached REML estimates of the SCA:GCA ratio were inflated by +14.8 % at $k = 2$ and +12.8 % at $k = 4$, against +0.3 % for a circulant factorial with three crosses per line. A tester design estimates SCA *with the testers*, not in the population of crosses the programme will sell.

Testcross ranking is not single-cross ranking. Comparing testcross progeny tests with randomly paired single-cross tests, B. Burdo, N. de Leon, and S. M. Kaeppler [32] found that the top 25 lines on testcross yield and on single-cross yield coincided for only 48 % of lines in the Stiff Stalk pool and 11 % in the Iowa Dent pool – almost disjoint selections – rising to 56 % in both with genomic prediction.

A thin training set. All hybrids of a candidate share one or two partners, so the design explores a narrow slice of the hybrid relationship space. Testcross was the worst of the designs compared as a training set for predicting the single crosses of a full diallel [33], and simulation agrees that the deficit grows with the SCA share [34].

9.3.4 When the tester is still right

None of this makes testers obsolete. They remain the right answer when seed production rather than plot capacity binds – a tester cross is made in bulk in an isolation block or on a male-sterile female, a sparse factorial needs many small hand pollinations (Section 9.7) – when the candidate set is so large that the first screen must be cheap per line, in programmes that cannot fit a mixed model (Section 9.5), and in the first cycle of a new pool, where there is no relationship information to exploit. D. González-Diéguez, G. Atlin, Y. Beyene, D. Wegary, D. Gemenet, and C. Werner [35] make the case for a hybrid of the two logics: three to five testers allocated *sparsely* across families rather than every tester crossed to every line.

9.4 Sparse and incomplete factorials

Once candidates are crossed to *each other* rather than to a common tester, the design becomes a sparse bipartite graph, and two things matter: how many crosses each line gets, c, and whether the graph is connected. Which particular partners are drawn matters much less.

9.4.1 The design family

The classical theory is the partial diallel of O. Kempthorne and R. N. Curnow [36]: each parent crossed to a fixed number s of others, with the sampling variances of the genetic components derived as functions of s. Its two-pool analogue is the incomplete factorial, each pool-A line crossed to c of the n_B pool-B lines. The members studied here:

- **random sparse** – $c \cdot n_A$ cells drawn at random. Lines receive a Binomial number of crosses, some none; the graph may be disconnected.
- **circulant (round-robin)** – pool-A line i crossed to pool-B lines $(i - 1 + o) \bmod n_B + 1$ for c offsets o. With $n_A = n_B$ every line in both pools has exactly c crosses, and choosing offsets with $\gcd(n_B, \Delta o) = 1$ guarantees one component. `design_circulant()` nudges the last offset until that holds.
- **NC II in sets** – complete $a \times b$ factorials on disjoint line sets [21]; deliberately disconnected.
- **criterion-optimised** – greedy exchange towards A-optimality, D-optimality or CDmean [37]; **relationship-guided** – partners chosen to minimise genomic similarity between parents. Both appear in the cached sweep only.

9.4.2 How sparse is sparse enough – the published evidence

Remarkably sparse, provided the design is structured and analysed well. The most informative single data point is operational rather than simulated: a commercial winter oilseed rape programme ran 4,000 hybrids over 61 unreplicated trials covering **1.67 % of all mother x father combinations**, and a GCA + SCA model still reached accuracies of 0.63, 0.84 and 0.80 for yield, oil and protein after a further 50 % of the observations were artificially removed [38]. Across maize, wheat and rice, training sets of 2 to 13 % of the hybrid set sufficed for genomic prediction [39]. A 363-hybrid dent x flint factorial predicted an independent 951-hybrid factorial as well as or better than tester designs at equal numbers of hybrids and lines [40]; NC II was the best and testcross the worst training set for predicting a full diallel [33]; and in simulation the factorial's advantage grew with the SCA share [34].

The design principle that emerges is to **spend the hybrid budget on parents, not on depth per parent** – up to the point where c is so small that connectivity and within-line precision fail. Where that point lies depends on heritability and SCA share. For hybrids with both parents untested at $c = 1$, A. Melchinger, R. Fernando, C. Stricker, C. Schön, and H. Auinger [41] reported attainable accuracies of 0.50 to 0.65 at $h^2 = 0.4$ and 0.70 to 0.85 at $h^2 = 0.8$, the low end belonging to the high-SCA scenario, and found raising c from 1 to 4 helped most exactly there.

9.4.3 Sparse versus tester, at equal plot budget

The cached sweep puts every design at the same three plot budgets and scores GCA accuracy over all 200 candidates per pool, analysed by phenotypic BLUP:

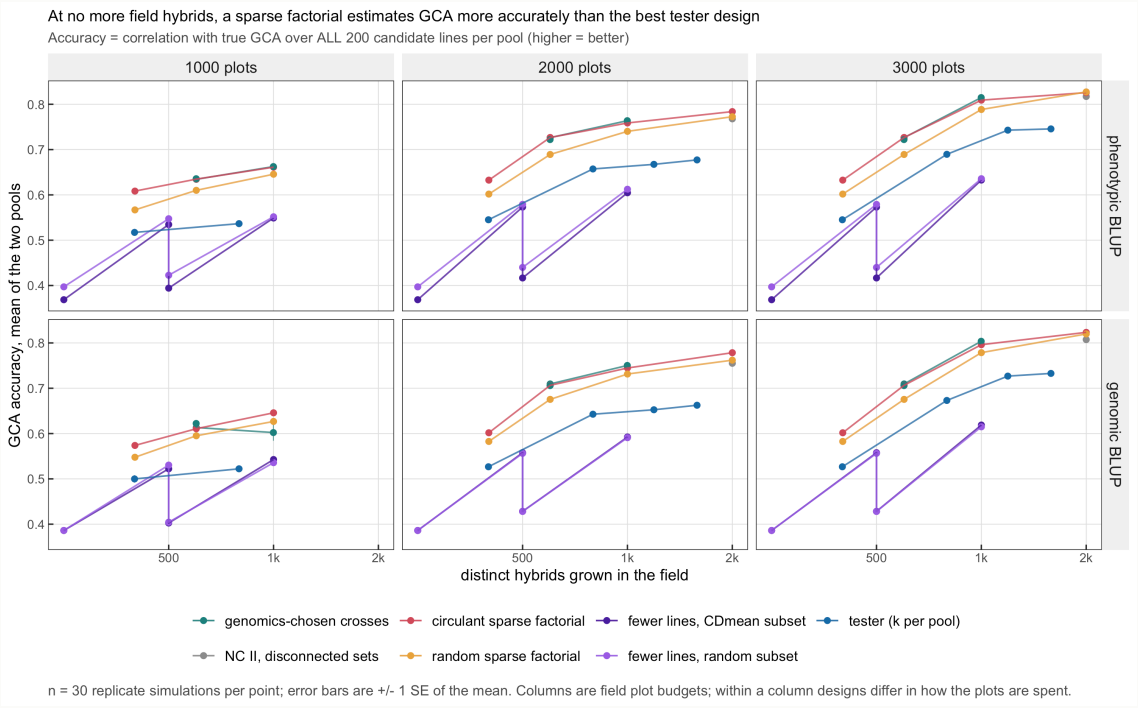

Figure 9.3: GCA accuracy against the number of distinct hybrids in the field, one line per design family, panelled by plot budget. Cached from the `sparsex` sweep: 30 replicates per cell, phenotypic BLUP, accuracy over all 200 candidates per pool.

```
sparse_fam <- c("sparse_circulant", "sparse_random")
h2h <- do.call(rbind, lapply(split(bl, bl$budget_plots), function(d) {
  t <- subset(d, family == "tester");       t <- t[which.max(t$acc_gca_all), ]
  s <- subset(d, family %in% sparse_fam);   s <- s[which.max(s$acc_gca_all), ]
  data.frame(plots = d$budget_plots[1],
             best_tester = t$scenario, tester_hybrids = round(t$n_crosses),
             tester_acc = t$acc_gca_all,
             best_sparse = s$scenario, sparse_hybrids = round(s$n_crosses),
             sparse_acc = s$acc_gca_all,
             difference = s$acc_gca_all - t$acc_gca_all,
             z = (s$acc_gca_all - t$acc_gca_all) /
                 sqrt(s$se_acc_gca_all^2 + t$se_acc_gca_all^2))
}))
fmt(h2h, 3)
```

Table 9.4: Best tester against best sparse factorial (circulant or random) at each plot budget, phenotypic BLUP. `z` is the difference in standard errors of the difference; anything above 3 is resolved by 30 replicates.

plots	best_tester	tester_hybrids	tester_acc	best_sparse	sparse_hybrids	sparse_acc	difference	z
1000	tester k=2	796	0.537	circulant c=5	1000	0.661	0.125	16.970
2000	tester k=4	1584	0.677	circulant c=10	2000	0.784	0.107	22.457
3000	tester k=4	1584	0.746	random sparse c=10	2000	0.827	0.082	21.904

Result. The sparse factorial wins at every budget, by 0.125, 0.107 and 0.082, each more than 16 standard errors. At 1,000 plots the relationship-guided design with five crosses per line is marginally ahead of the circulant one (0.662, a difference well inside the noise); it is excluded from the table only so that the comparison is between designs that need no markers. Interpretation. Two features of the table are mechanical and worth naming. The best tester never uses more than 1,584 crosses because $k = 4$ is the largest tester set swept, and at 3,000 plots it can only add replication, which buys down σ_e^2 but not σ_s^2. And the sparse winner at 3,000 plots is random rather than circulant because at $c = 10$ connectivity is no longer in question and the layouts converge.

The smallest fields that reach a given accuracy:

```
fewest <- do.call(rbind, lapply(c(0.60, 0.65, 0.70, 0.75, 0.80), function(tg) {
  d <- subset(bl, acc_gca_all >= tg)
  d <- d[order(d$n_crosses, d$budget_plots), ][1, ]   # ties: the smaller field
  data.frame(target = tg, design = d$scenario, plots = d$budget_plots,
             hybrids = round(d$n_crosses), pct_factorial = 100 * d$n_crosses / n^2,
             accuracy = d$acc_gca_all, se = d$se_acc_gca_all,
             cost = round(d$cost_total))
}))
fmt(fewest, 3)
```

Table 9.5: Fewest distinct field hybrids, over all designs and budgets, whose phenotypic-BLUP GCA accuracy reaches each target. Costs use the placeholder levels of Section 9.7.

target	design	plots	hybrids	pct_factorial	accuracy	se	cost
0.60	circulant c=2	1000	400	1.000	0.608	0.003	67800
0.65	relationship-guided c=3	2000	600	1.499	0.722	0.004	93261
0.70	relationship-guided c=3	2000	600	1.499	0.722	0.004	93261
0.75	relationship-guided c=5	2000	999	2.497	0.764	0.004	102292
0.80	relationship-guided c=5	3000	999	2.497	0.815	0.002	132193

Result. Four hundred hybrids – 1 % of the factorial – reach 0.608; about 600 (1.5 %) reach 0.722; about 1,000 (2.5 %) reach 0.815. Interpretation. This confirms the order of magnitude of the operational and published sampling fractions [38], [39] and adds the part they leave implicit: the fraction is set by the accuracy target, not by a universal threshold.

9.4.4 Crosses per line, and structure versus randomness

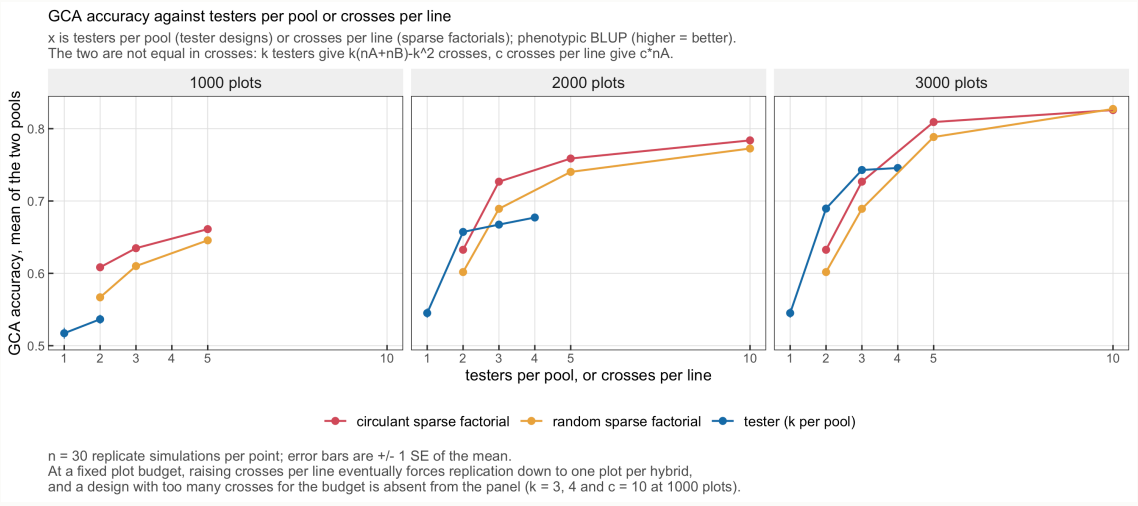

Figure 9.4: GCA accuracy against crosses per line c, separating circulant (structured) from random sparse layouts. Cached from the `sparsex` sweep.

The literature leaves one comparison open: structured against random layouts at matched c [D. L. Weeks and D. R. Williams [23] and R. Fernando, D. Gianola, and M. Grossman [24] supply the tools; no retrieved design paper reports the connectivity of the layout it used]. The sweep closes it at 2,000 plots:

```
mc <- subset(bl, budget_plots == 2000 & family %in% sparse_fam)
mc <- merge(subset(mc, family == "sparse_circulant")[, c("x_axis", "acc_gca_all",
"se_acc_gca_all")],
            subset(mc, family == "sparse_random")[, c("x_axis", "acc_gca_all",
"se_acc_gca_all")],
            by = "x_axis", suffixes = c("_circ", "_rand"))
mc <- data.frame(c = mc$x_axis, circulant = mc$acc_gca_all_circ,
                 random = mc$acc_gca_all_rand,
                 difference = mc$acc_gca_all_circ - mc$acc_gca_all_rand,
                 z = (mc$acc_gca_all_circ - mc$acc_gca_all_rand) /
                     sqrt(mc$se_acc_gca_all_circ^2 + mc$se_acc_gca_all_rand^2))
fmt(mc, 3)
```

Table 9.6: Circulant against random sparse at matched crosses per line, 2,000 plots, phenotypic BLUP.

c	circulant	random	difference	z
2	0.633	0.602	0.031	6.392
3	0.727	0.689	0.037	8.706
5	0.759	0.740	0.018	4.120
10	0.784	0.773	0.011	3.155

Result. The circulant layout is ahead at every c, and the gap shrinks as c grows, from 0.031 at $c = 2$ to 0.011 at $c = 10$; all four gaps exceed three standard errors of the difference. Interpretation. This is what Table 9.3 predicts: at $c = 2$ the random layout leaves lines uncrossed and splits the rest into many components, so its advantage is pure connectivity, and connectivity stops being binding once every line has enough partners. **A sparse factorial should be built, not sampled.**

9.4.5 What is lost

SCA is weakly estimated at low c. The circulant design with $c = 2$ has exactly 1 residual degree of freedom at cross-mean level (Table 9.3) – enough to fit a model and not enough to estimate a variance component – and the cached REML recovered the SCA:GCA ratio there with a +34.4 % error, the largest in the sweep. The recommendation therefore splits by objective: $c = 2$ to rank lines, $c \geq 3$ before any statement about SCA variance.

Sparsity in crosses also interacts with sparsity in environments. A parallel literature, mostly from CIMMYT, allocates entries to subsets of environments and predicts the unobserved line-by-environment cells [42], [43], [44], [45]. When the genetic correlation between managements is as low as 0.37 [43], a line observed only in one is weakly informative about the other. A design observing 2 % of the hybrid factorial and 30 % of the environment cells within that observes well under 1 % of the information space, and the two sparsities have not been studied together for hybrids (Section 9.9).

9.5 The analysis is part of the design

A sparse factorial is only a good design if it is analysed with a shrinkage estimator. This section exists because the single largest effect in the study is not a design effect at all.

The demonstration is live: one replicate of the running example at 2,000 plots, the same data analysed both ways.

```
set.seed(99)
ls_blup <- do.call(rbind, lapply(c("tester k=2", "circulant c=2", "circulant c=5"),
  function(nm) {
    tr <- simulate_factorial(n, n, vc$A, vc$B, vc$S)
    p  <- simulate_cross_means(design_reps(plans[[nm]], 2000), tr, s2e)
```

```
    acc <- sapply(c("ls", "blup"), function(meth) {
      f <- fit_gca(p, n, n, vc$A, vc$B, vc$S, s2e, meth)
      mean(c(cor(f$gA, tr$gA), cor(f$gB, tr$gB)))
    })
    data.frame(design = nm, crosses = nrow(p), plots_per_cross = mean(p$reps),
               least_squares = acc[["ls"]], blup = acc[["blup"]],
               blup_advantage = acc[["blup"]] - acc[["ls"]])
  }))
fmt(ls_blup, 3)
```

Table 9.7: One replicate of the 200 x 200 running example at 2,000 plots: the same phenotypes analysed by least squares and by BLUP with known variance components.

design	crosses	plots_per_cross	least_squares	blup	blup_advantage
tester k=2	796	2.513	0.649	0.658	0.008
circulant c=2	400	3.000	0.186	0.627	0.441
circulant c=5	1000	2.000	0.590	0.757	0.166

Result. On the tester design the two estimators agree to within 0.008. On the circulant design with two crosses per line, least squares collapses to 0.19 while BLUP reaches 0.63 on the same data. The cached sweep, averaged over 30 replicates, reproduces the pattern across all 24 designs:

```
lsb <- merge(subset(sm, model == "LS"    & budget_plots == 2000)[, c("scenario",
"n_crosses", "acc_gca_all")],
             subset(sm, model == "BLUP" & budget_plots == 2000)[, c("scenario",
"acc_gca_all")],
             by = "scenario", suffixes = c("_ls", "_blup"))
lsb$advantage <- lsb$acc_gca_all_blup - lsb$acc_gca_all_ls
lsb <- lsb[order(-lsb$advantage), ]
names(lsb) <- c("design", "hybrids", "least_squares", "blup", "advantage")
lsb$hybrids <- round(lsb$hybrids)
rownames(lsb) <- NULL
fmt(lsb, 3)
```

Table 9.8: BLUP advantage over least squares by design, 2,000 plots, ordered by size. Cached `sparsex` sweep, 30 replicates per cell.

design	hybrids	least_squares	blup	advantage
circulant c=2	400	0.134	0.633	0.498
circulant c=3	600	0.255	0.727	0.472
circulant c=5	1000	0.541	0.759	0.218
random sparse c=2	400	0.417	0.602	0.185
random sparse c=3	600	0.587	0.689	0.102
A-optimal c=3	600	0.626	0.726	0.100
100 lines/pool c=5 (random)	500	0.484	0.579	0.095
relationship-guided c=3	600	0.635	0.722	0.087
100 lines/pool c=5 (cdmean)	500	0.490	0.573	0.084
circulant c=10	2000	0.707	0.784	0.077
random sparse c=5	1000	0.689	0.740	0.051
NC II 10x10 sets	2000	0.737	0.768	0.031
50 lines/pool c=5 (random)	250	0.373	0.397	0.024
relationship-guided c=5	999	0.740	0.764	0.023
50 lines/pool c=5 (cdmean)	250	0.350	0.369	0.018
random sparse c=10	2000	0.756	0.773	0.016
100 lines/pool c=10 (random)	1000	0.598	0.612	0.015
100 lines/pool c=10 (cdmean)	1000	0.593	0.605	0.012
tester k=4	1584	0.670	0.677	0.007
tester k=3	1191	0.663	0.667	0.005
tester k=2	796	0.653	0.657	0.004
50 lines/pool c=10 (random)	500	0.437	0.440	0.003
50 lines/pool c=10 (cdmean)	500	0.414	0.417	0.002
tester k=1	399	0.544	0.545	0.001

Result. The penalty for least squares is 0.498 accuracy points at circulant $c = 2$ and 0.472 at $c = 3$; for the four tester designs it is 0.001 to 0.007. Testers are robust to a naive analysis; sparse factorials are not.

Interpretation. At low c each GCA estimate rests on two or three cross means, each carrying a full $\sigma^2_s + \sigma^2_e/r$ of noise, and the unshrunken estimate is dominated by it. BLUP borrows strength across the connected graph: a line's partners are themselves estimated from *their* partners, and the shrinkage towards zero is proportional to how little each line was observed. That the circulant design loses more to least squares than the random one at the same c is not a contradiction of the previous section. The circulant design's partners are each other's partners in long chains, which is exactly what BLUP exploits and least squares cannot.

The practical consequence is unconditional: **a programme that cannot or will not fit a mixed model should not move to a sparse factorial** – the design's advantage is entirely conditional on the estimator. The literature that motivated BLUP of single crosses [46], [47] and the allocation tools that assume it [48], [49] take this for granted; the design literature, in the review behind this chapter, never states it.

9.6 Genomic information for deciding which crosses to make

Genomic information changes the question from *how well can I estimate the lines I crossed?* to *which crosses should I make, and what can I say about the ones I did not?* Its value is in extrapolation and in the choice of what to field, not in improving estimates for lines that are already there.

9.6.1 From relatedness to hybrid prediction

The idea that untested single crosses can be predicted from tested ones through marker relatedness is R. Bernardo [46]'s: random subsets of 10 to 30 hybrids from a 54-hybrid factorial, and RFLP relationships recovering single-cross performance. R. Bernardo [47] scaled it to operational data, with correlations of 0.43 to 0.76 for yield in heterotic patterns with more than 100 tested crosses. The modern version models hybrid value as pool-specific additive kernels for GCA plus a dominance or product kernel for SCA [50], [51], [52]; 3 is where this book fits those models, and Section 17.4 where it validates them by masking hybrids.

The kernel the cached sweep used for SCA follows from the algebra of Section 9.2: $s_{ij} = -2\sum_k d_k\,\delta u_{ik}\,\delta v_{jk}$ is linear in the *elementwise product* of the two parents' centred genotypes, so the natural SCA kernel is the linear kernel on that product. It is **not** the Hadamard product of the two pool kernels, which adds cross-locus terms the model does not contain.

9.6.2 The T0 / T1 / T2 hierarchy

The standard currency for comparing designs is the split of untested hybrids by how many of their parents appear in the training set: both (T2), one (T1), neither (T0). Grain-yield accuracies of 0.91, 0.85 and 0.77 with 300 training hybrids [52], 0.92, 0.88 and 0.80 under a GCA model on the Hohenheim series [53], and 0.28 to 0.77 with the ordering T2 > T1-female > T1-male > T0 [54]. The hierarchy also explains why a tester is a poor training set: it generates only T1-type information for new lines, because the tester is always a tested parent and the candidate never is.

These figures share a property that must be stated: they come from material with strong family structure, where an untested line has close relatives in the training set. F. Technow [55] found the advantage of a better training-set composition shrank from 0.15 accuracy points for well-connected hybrids to 0.06 for T0 under a divergent heterotic structure. **Published T0 accuracies are not programme-independent**, and the simulated counter-example below makes the point concrete.

9.6.3 The negative result

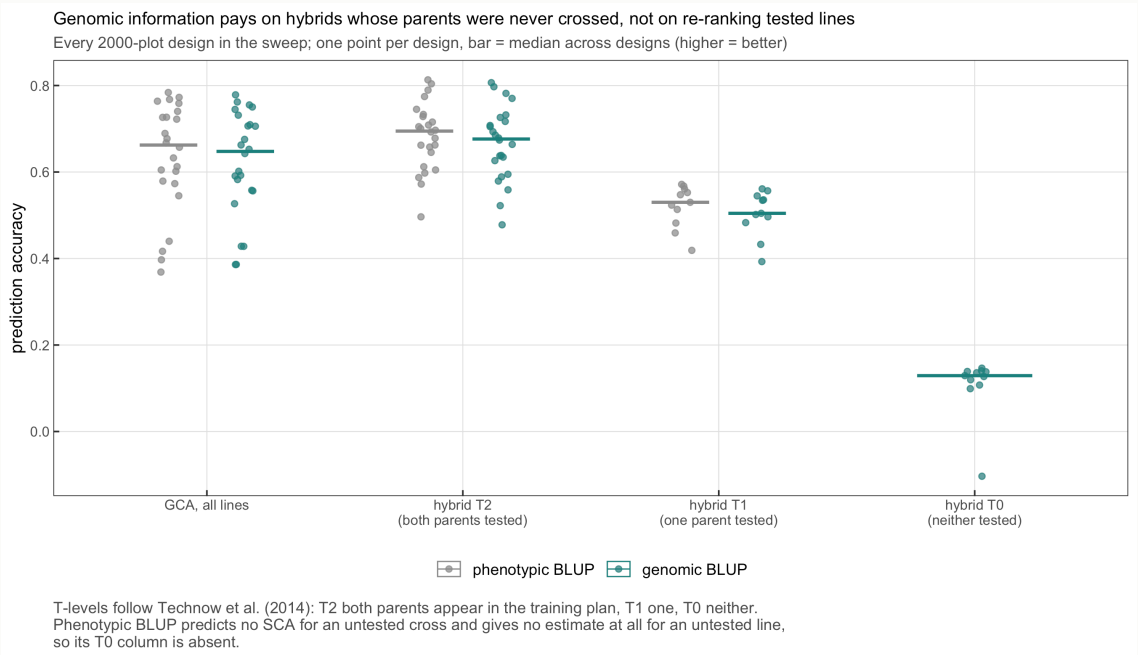

Figure 9.5: Genomic against phenotypic BLUP by design (left) and the genomic accuracy for hybrids whose parents were never crossed (right). Cached from the `sparsex` sweep, 2,000 plots.

```
gb <- merge(subset(sm, model == "GBLUP" & budget_plots == 2000)[, c("scenario",
"n_linesA_tested", "acc_gca_all", "se_acc_gca_all", "acc_T0")],
```

```
              subset(sm, model == "BLUP"  & budget_plots == 2000)[, c("scenario",
"acc_gca_all", "se_acc_gca_all")],
            by = "scenario", suffixes = c("_g", "_p"))
gb <- data.frame(design = gb$scenario, lines_tested = gb$n_linesA_tested,
                 phenotypic = gb$acc_gca_all_p, genomic = gb$acc_gca_all_g,
                 difference = gb$acc_gca_all_g - gb$acc_gca_all_p,
                 z = (gb$acc_gca_all_g - gb$acc_gca_all_p) /
                     sqrt(gb$se_acc_gca_all_g^2 + gb$se_acc_gca_all_p^2),
                 T0 = gb$acc_T0)
gb <- gb[order(-gb$difference), ]
rownames(gb) <- NULL
fmt(head(gb, 6), 3)
```

Table 9.9: Genomic (GBLUP) against phenotypic BLUP on all-lines GCA accuracy, 2,000 plots, the six largest differences. `z` in standard errors of the difference. `T0` is genomic accuracy for hybrids with neither parent crossed, defined only where some lines were left out.

design	lines_tested	phenotypic	genomic	difference	z	T0
50 lines/pool c=5 (cdmean)	50.0	0.369	0.386	0.017	4.524	0.136
50 lines/pool c=10 (cdmean)	50.0	0.417	0.428	0.012	4.248	0.146
circulant c=10	200.0	0.784	0.778	−0.005	−1.457	NA
random sparse c=5	198.9	0.740	0.731	−0.009	−1.719	−0.103
random sparse c=10	200.0	0.773	0.762	−0.011	−3.045	NA
50 lines/pool c=5 (random)	50.0	0.397	0.386	−0.011	−1.110	0.099

Result. Genomic kernels raised all-lines GCA accuracy in 2 of 24 designs, both of them reduced line sets of 50 per pool chosen by CDmean (+0.017 and +0.012, each beyond 3 standard errors). In every design that fielded all 200 candidates, the genomic model was level with or slightly below the phenotypic one. Genomic T0 accuracy where it is defined on more than a handful of lines was 0.1 to 0.15; the negative value in the random sparse rows is computed over the few lines the random draw happened to leave uncrossed and means nothing.

Interpretation. Two separate statements. First, when every candidate is phenotyped in a connected design, the marker kernel has little to add to what the crosses already say, and a misspecified SCA kernel (equal d^2 at every locus) costs a little. Second, the T0 accuracies are an order of magnitude below the published 0.77 to 0.80, because the simulated candidates are **unrelated doubled haploids from two pools diverged for 100 generations**, with no family structure to carry information from a tested line to an untested sibling. That is the condition under which genomics transfers least, and it is the condition of a programme sampling new, unrelated material. The genomic model earns its place where the phenotypic model has nothing to say at all – lines outside the field – and where the field holds only a fraction of the candidates, by deciding **which** lines go in: CDmean selection of the line set is where the gain appeared.

9.6.4 Which hybrids to phenotype, and which crosses to make

Once prediction is in place the design question becomes which cells to observe to calibrate it. CDmean [37] beat random sampling and PEVmean for inbred calibration sets because it accounts for the redundancy among related individuals; for hybrids, sets built by clustering, network analysis and mating-scheme considerations beat both [39], which suggests that the structure that matters for a hybrid training set is bipartite. A genomics-guided pre-selection that discarded 70 % of candidate single crosses retained 85 to 95 % of the top 5 % in a tropical maize programme [56].

A different question is which crosses to *make* to produce the next generation of lines. The usefulness criterion adds the within-cross selection differential to the predicted mean, and selecting on it added 0.14 genetic standard deviations of gain at 1 % selected per cross [57]; within-cross variance is usable for ranking but the marker model's estimate was biased downward by 28 to 60 % [58]. At the portfolio level, optimal cross selection chooses a *set* of crosses under a constraint on diversity: 15 to 78 % more long-term gain than truncation selection in G. Gorjanc, R. C. Gaynor, and J. M. Hickey [59], and more again once within-family variance enters the objective [60]. Constrained mate allocation with a mating permission

matrix [61] is what makes optimising a crossing list over a 200 x 200 candidate space, subject to nursery constraints, computationally routine.

The long-term caution is this book's own: cross selection controls inbreeding and the divergence of the pools, and a design chosen once should not be held fixed across cycles. A. Lorenzi *et al.* [29] observed the SCA share move between 0 and 30 % from one selection cycle to the next, so the variance structure every design calculation in this chapter assumes is not stationary.

9.6.5 What the simulation adds to the literature

Table 9.10: Where the simulated programme confirms, refines or contradicts the published expectation.

Published expectation	Simulated result	Verdict
Sparse factorials beat testers as training sets [33], [34], [40]	Sparse beats tester on GCA accuracy over all candidates at every budget, by 0.08 to 0.13 (Table 9.4)	Confirms, and extends beyond the training-set role
Spend on parents, c = 2 to 4 [41]	Supported for ranking; $c = 2$ leaves 1 residual df and inflates SCA:GCA by +34 %; the optimiser picks c = 3 to 5	Refines: $c = 2$ to rank, $c \geq 3$ to estimate variances
Sampling fractions of 1 to 5 % suffice [38], [39]	1 % gives 0.61, 1.5 % gives 0.72, 2.5 % gives 0.82 (Table 9.5)	Confirms the order of magnitude; the fraction follows the accuracy target
T0 penalty ~0.1 to 0.15 below T2 [52], [53]	Genomic T0 accuracy 0.10 to 0.15 *in absolute terms*	Tension: published values rest on family structure
Genomic kernels improve hybrid evaluation	Gain only in 2 of 24 designs, both reduced CDmean line sets (Table 9.9)	Refines: the gain is extrapolation and line choice
Structured vs random layouts at matched c not compared	Circulant ahead at every c, all four gaps resolved (Table 9.6)	Closes the gap
Testers lose SCA information [14], [32]	$k = 1$: SCA not identifiable; k = 2, 4 inflate SCA:GCA by +15 %, +13 %	Confirms and quantifies
Not in the retrieved literature	Least squares instead of BLUP costs 0.50 at $c = 2$, under 0.01 for testers (Table 9.8)	New, and the largest effect in the study

9.7 Seed production and the cost of a crossing plan

The crossing plan has to be made in a nursery. Treating hybrid seed production as a scalar cost term – as the published cost models do [62], [63] – hides the constraint that actually binds sparse designs: the seed quantity per cross limits how many plots and locations each hybrid can occupy.

9.7.1 Three cost centres

A mating design has three cost centres that scale differently. **Making** the crosses – pollinations, ears, nursery rows, isolation blocks – scales with the number of distinct crosses. **Testing** them scales with crosses x locations x replicates. **Genotyping** scales with the number of lines, not crosses, which is the structural reason genomics favours designs with many lines and few crosses each. A tester design makes few crosses with much seed each; a sparse factorial makes many crosses with little seed each. The two can occupy the same plots and need completely different nurseries.

`cross_cost()` writes the accounting chain from plots back to the nursery: seed needed per cross is plots x seed per plot; ears are that over seed per ear, never fewer than an insurance minimum; pollinations are ears over pollination success; female rows follow from the pollinations each pool-A parent carries, male rows from the pollinations each pool-B parent serves, several per plant. Four designs at the same 2,000 plots:

```
set.seed(8)
cost_plans <- list("tester k=4"       = design_tester(n, n, 4),
                   "circulant c=5"    = design_circulant(n, n, 5),
                   "circulant c=10"   = design_circulant(n, n, 10),
```

```
                  "NC II 10x10 sets" = design_nc2(n, n, 10, 10))
cost_tab <- do.call(rbind, lapply(names(cost_plans), function(nm) {
  cc <- cross_cost(design_reps(cost_plans[[nm]], 2000), n, n)
  data.frame(design = nm, t(cc[c("crosses", "plots", "pollinations",
                                 "nursery_rows", "genotyped", "total")]))
}))
cost_tab
```

Table 9.11: The cost vector of four designs at an equal budget of 2,000 plots. **Every monetary and biological level is an illustrative placeholder** (`cross_cost_par()`); the structure is the contribution, the numbers are not.

design	crosses	plots	pollinations	nursery_rows	genotyped	total
tester k=4	1584	2000	4752	552	400	108504
circulant c=5	1000	2000	3000	400	400	102300
circulant c=10	2000	2000	6000	600	400	112200
NC II 10x10 sets	2000	2000	6000	600	400	112200

Result. Under this accounting the four plans cost within 10 % of each other. The cheapest is the circulant design with five crosses per line, because it makes the fewest distinct crosses (1000) and spends the rest of the plots on replicates, which need more seed per cross but no more pollinations: the two-ear insurance minimum already produces enough seed for two plots. The ten-cross designs make twice as many crosses and need 6000 pollinations against 3000.

Interpretation. The tester is **not** cheaper here, and that is a property of the model, not of testers. `cross_cost()` charges every cross as hand pollinations, so it cannot credit the bulk production – an isolation block, a male-sterile female – that makes a tester cross cheap in practice, and the tester's 1,584 crosses each pay the insurance minimum. That missing term is gap 1 of Section 9.9, and it is the term a real programme must add before comparing designs on cost. The column that matters for a sparse factorial is the other one: seed per cross caps replication, so a plan at one ear per cross has no seed for a second plot. Operational sparse designs such as L. Thomsen *et al.* [38] are unreplicated for exactly this reason.

> ⚠ The cost levels are placeholders
>
> Every monetary and biological level in `cross_cost_par()` – seed per plot, seed per ear, pollination success, pollen uses per male plant, the price of a plot, a row, a pollination and a genotype – is an order of magnitude for a public maize programme, not a costed figure. The *structure* – which quantity multiplies which, which capacity binds – is what this section offers. Re-parameterise before any number below means anything for a real programme.

9.7.2 Budget-constrained optimisation

The cached optimiser searched over lines tested per pool, design family, k or c, plots per cross and whether to genotype every candidate, subject to field, nursery and money constraints, and maximised expected response $R = i\,\rho\,\sigma_{GCA}$ – not accuracy. That distinction is the trade-off: raising c raises ρ but, at fixed budget, lowers the number of lines tested and with it the selection intensity i. It scores ρ with the least-squares closed form, so it is conservative for sparse designs.

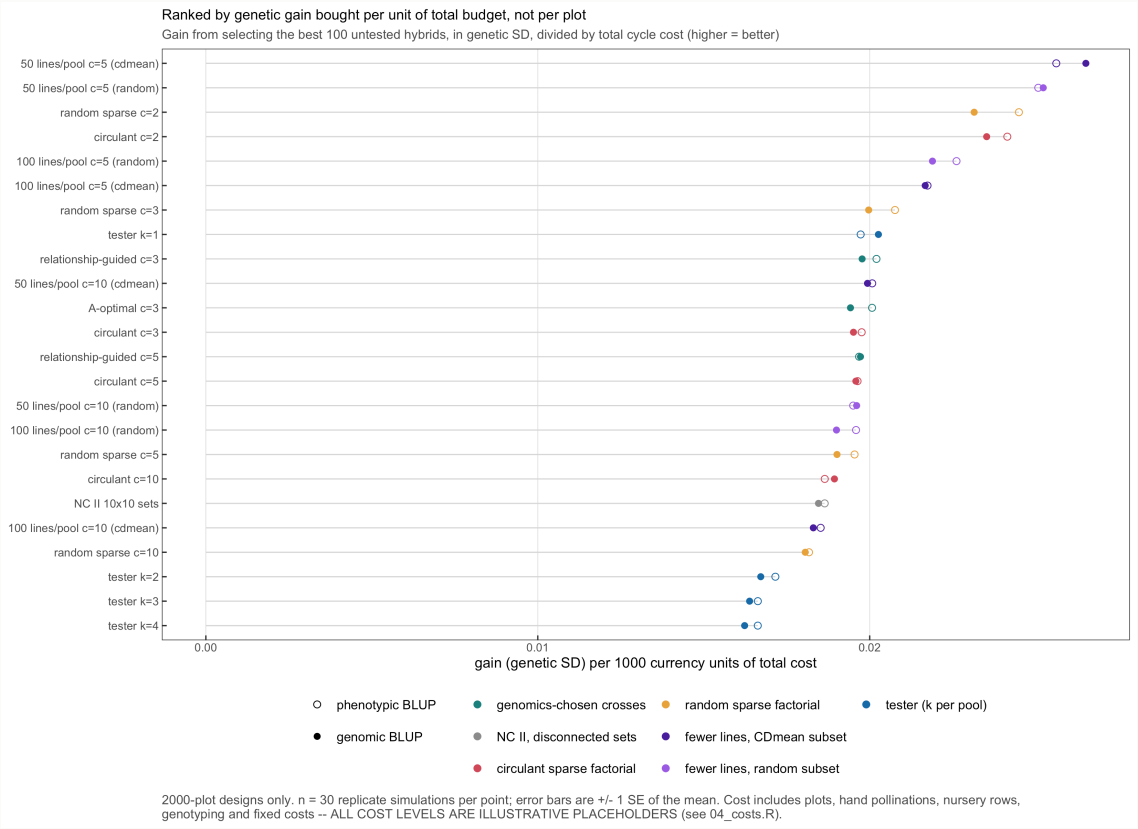

Figure 9.6: Left: expected response per unit cost by design. Right: the best feasible configuration at each total budget. Cached from the `sparsex` optimiser; cost levels are placeholders.

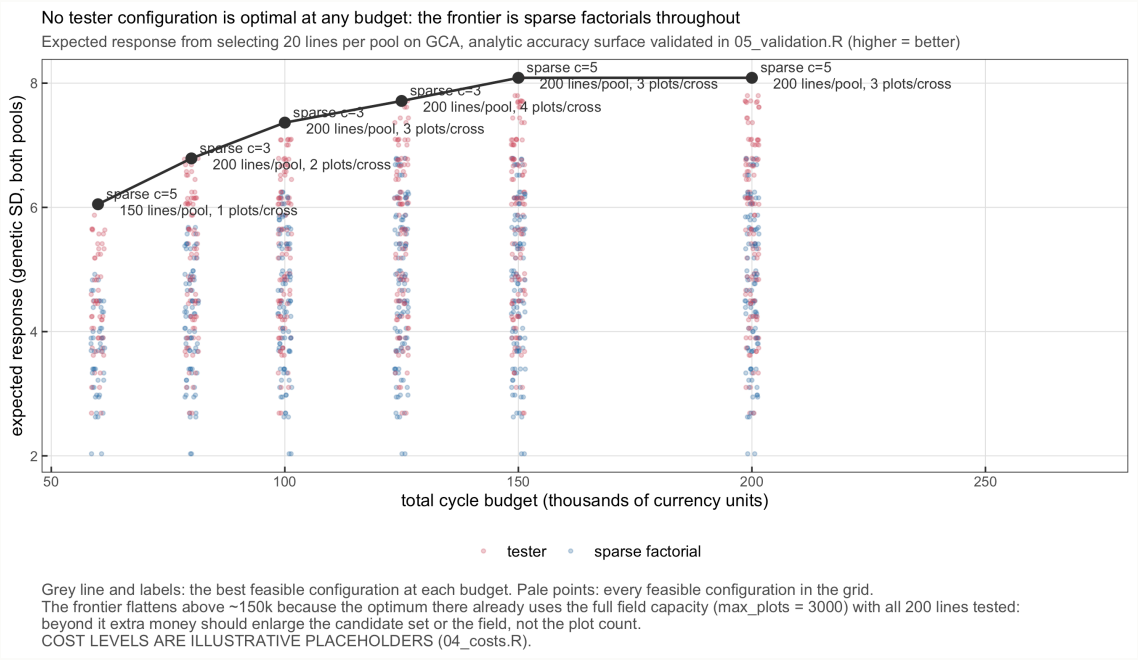

Figure 9.7: The optimiser's budget frontier. Cached from the `sparsex` optimiser.

```
ff <- subset(front, feasible)
best <- do.call(rbind, lapply(split(ff, ff$budget), function(d)
d[which.max(d$resp), ]))
best <- best[, c("budget", "design", "n_test", "c_or_k", "reps", "n_crosses",
                 "n_plots", "cost_total", "resp", "resp_per_1000")]
rownames(best) <- NULL
fmt(best, 3)
```

Table 9.12: The best feasible configuration at each total budget. Response in genetic units of GCA, summed over the two pools. Placeholder cost levels.

budget	design	n_test	c_or_k	reps	n_crosses	n_plots	cost_total	resp	resp_per_1000
60000	sparse	150	5	1	750	750	59225	6.050	0.102
80000	sparse	200	3	2	600	1200	75300	6.789	0.090
100000	sparse	200	3	3	600	1800	93300	7.365	0.079
125000	sparse	200	3	4	600	2400	111300	7.714	0.069
150000	sparse	200	5	3	1000	3000	132300	8.086	0.061
200000	sparse	200	5	3	1000	3000	132300	8.086	0.061

Result. A sparse factorial is on the frontier at every budget, and no tester configuration is optimal anywhere in the grid. The optimum moves from 150 lines per pool at $c = 5$ and one plot per cross at the smallest budget to all 200 lines at $c = 5$ and 3 plots per cross at the largest, where the 3,000-plot field cap binds and further money buys nothing. Response per 1,000 currency units falls from 0.102 to 0.061. Interpretation. Once the field is full, the next unit of money belongs in a larger candidate set – more lines to select among – not in more plots per cross.

The published allocation results agree in direction. A training population of 500 at one replicate was only 4 % more accurate than 125 at four, and a second replicate added 0.09 correlation points [64]; selection gain levels off quickly with budget, so additional money is better spent on more populations than on deeper testing [65]; and the optimum number of testers, locations and stages is a joint decision [27], [49], [62], [66].

9.8 A decision framework

The chapter collapses into a procedure a breeder can apply in a planning meeting.

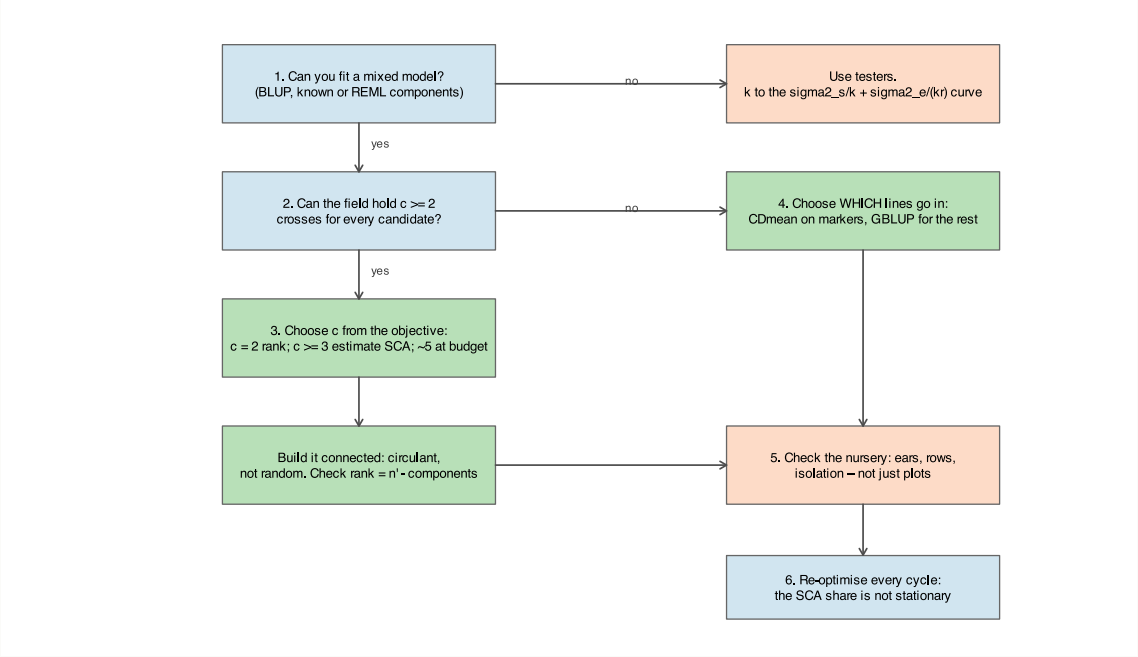

Figure 9.8: A decision procedure for the crossing plan of a two-pool programme. Each question is answered by a section of this chapter.

1. **Can you fit a mixed model, and do you have markers?** If neither, use testers: they are the only design robust to least squares (Table 9.8). Choose k from the curve of Figure 9.2.
2. **How many candidates must be ranked, and can the field hold a cross for each?** If it can hold $c \geq 2$ crosses per candidate, prefer a connected sparse factorial (Table 9.4).
3. **Choose c from the objective.** $c = 2$ ranks lines; $c \geq 3$ is required before any statement about SCA variance; $c \approx 5$ is where the optimiser sits at realistic budgets (Table 9.12). Build the layout, do not sample it, and confirm with `design_diagnostics()` that it is one component.
4. **Choose the line set, not just the crosses.** When the field cannot hold every candidate, CDmean-type selection of which lines to field is where genomic information paid, and GBLUP is the only route to the lines left out (Table 9.9).
5. **Check the nursery before committing:** ears, rows and isolation blocks, not just plots (Table 9.11).
6. **Re-optimise every cycle.** The variance structure moves [29], and so does the diversity of the pools (Section 9.11).

9.9 Open problems

Five gaps remain, each defined by the evidence that would close it.

1. **Seed production as an explicit constraint.** No retrieved study models seed yield per pollination, isolation capacity or seed per plot inside the optimisation of a mating design; cost models carry hybrid-seed production as a scalar [62], [63]. `cross_cost()` gives the structure; the levels are missing.
2. **Joint sparsity in crosses and environments.** The cross-sparsity literature [40], [41] holds the environment design fixed; the environment-sparsity literature [42], [44], [45] works with line or testcross entries. L. Thomsen *et al.* [38] is the closest to a joint treatment, and it is observational. Chapter 11. takes this gap up.
3. **Connectivity reported, structured layouts compared.** The tools are seventy years old [23], [24]; design papers rarely report the connectivity of the layouts they use. Table 9.6 closes the matched-c comparison in simulation only.
4. **Tropical maize.** Under-represented relative to its share of breeding effort; R. Fritsche-Neto *et al.* [56] and D. Chaingeni *et al.* [31] are the main exceptions.
5. **Multi-cycle validation.** The long-term consequences of genomics-driven cross selection for pool divergence are modelled [59], [60], [67] but not validated over real cycles, and the non-stationary SCA share of A. Lorenzi *et al.* [29] is the warning case.

9.10 Reproducibility and assumptions

What runs live. Every table and figure above built from `design_*()`, `design_diagnostics()`, `fit_gca()`, `cross_cost()` or `gca_accuracy_theory()` is computed at render, from base R, at the full 200 x 200 scale. The package's tests (`tests/testthat/test-mating.R`) assert the rank rule for every design family, the NC II estimability result, the $k = 1$ saturation, the Sprague-Tatum error variance by Monte Carlo, and that BLUP converges to least squares as the GCA variance grows.

What is cached. The 24-design x 3-budget sweep, the REML variance components, the genomic results and the optimiser frontier come from the external `sparsex` study, whose validation suite of 37 checks (closed form against AlphaSimR's own genotypic values to 6×10^{-14}; the sampling-variance and accuracy expressions to 1.5 %; in-house REML against `sommer` to 10^{-5}; the rank rule for every family) all passed. The phenotypic core of that sweep is re-run in-repo by `Rscript book/scripts/regenerate.R mating`, with the package's i.i.d. model and the variance components held at their true values:

```
cr <- subset(core, budget_plots == 2000)
cr <- reshape(cr[, c("scenario", "method", "acc_mean")], idvar = "scenario",
              timevar = "method", direction = "wide")
ca <- merge(subset(sm, budget_plots == 2000 & model == "LS")[, c("scenario",
"acc_gca_all")],
            subset(sm, budget_plots == 2000 & model == "BLUP")[, c("scenario",
"acc_gca_all")],
            by = "scenario", suffixes = c("_ls", "_blup"))
rep_tab <- merge(cr, ca, by = "scenario")
rep_tab <- data.frame(design = rep_tab$scenario,
                      ls_inrepo = rep_tab$acc_mean.ls, ls_cached = rep_tab$acc_gca_all_ls,
                                  blup_inrepo = rep_tab$acc_mean.blup, blup_cached
= rep_tab$acc_gca_all_blup)
fmt(rep_tab[order(rep_tab$blup_inrepo), ], 3)
```

Table 9.13: The in-repo reproduction (`mating_core_sweep.csv`: i.i.d. model, known components, 30 replicates) against the cached `sparsex` sweep (AlphaSimR founders, REML components), at 2,000 plots.

	design	ls_inrepo	ls_cached	blup_inrepo	blup_cached
9	tester k=1	0.550	0.544	0.551	0.545
7	random sparse c=2	0.420	0.417	0.600	0.602
2	circulant c=2	0.145	0.134	0.637	0.633
10	tester k=2	0.654	0.653	0.658	0.657
11	tester k=4	0.671	0.670	0.677	0.677
3	circulant c=3	0.219	0.255	0.710	0.727
8	random sparse c=5	0.697	0.689	0.749	0.740
4	circulant c=5	0.506	0.541	0.757	0.759
6	random sparse c=10	0.760	0.756	0.775	0.773
1	circulant c=10	0.615	0.707	0.776	0.784
5	NC II 10x10 sets	0.756	0.737	0.780	0.768

Result. The two agree on every qualitative claim of the chapter: sparse ahead of tester at equal plots, circulant ahead of random at low c, and a least-squares penalty that is large at low c and negligible for testers. They are not expected to agree to the third decimal: the cached run founded its pools in a coalescent simulation with linkage and finite QTL, estimated its components by REML, and scored against the realised factorial; the in-repo run draws i.i.d. effects and gives the estimator the true components.

Assumptions, stated in the chapter rather than a footnote.

- **Thirty replicates per cell.** The median standard error of a cell mean was about 0.004, so differences below about 0.018 accuracy points are inside the noise; every comparison quoted above clears three standard errors of the difference unless it says otherwise.
- **One trait, one marker density** (1,000 markers), no epistasis, no selection during line development, no spatial trend, no missing plots.

- **Genotype x location is folded into plot error.** An extra location and an extra replicate are treated as equivalent, which is exact under that assumption and optimistic when G x L variance is large.
- **Unrelated candidate panel.** The DH lines come from two pools diverged for 100 generations, with no family structure, which is why genomic T0 accuracy is low.
- **Placeholder costs.** Every level in `cross_cost_par()`.
- **Integer budgets.** Replication is `floor(plots / crosses)` capped at three, with the remainder spread at random, so "budget" is not a continuous axis: a one-tester design with 399 crosses stops at 1,197 plots.

9.11 Where this connects

The crossing plan decides **which lines are seen**, and that makes it a diversity decision before it is a statistical one. A design that fields 50 lines per pool selected by CDmean ranks those 50 and predicts the other 150 at a T0 accuracy of 0.1 – so the next cycle's parents will in practice come from the 50, and the loss of gene diversity that Chapter 16. tracks across the pipeline starts in the nursery plan, not at selection. A sparse factorial that crosses every candidate is the design that keeps the whole candidate set in contention.

Section 13.10 is where the GCA and SCA estimated here were derived from allele frequencies, and Section 13.12.6 is the finite-tester version of Figure 9.2; both show that the SCA:GCA ratio driving every design choice in this chapter falls as the pools diverge, so the number of testers and the crosses per line a programme needs are not constants of the crop. 3 fits the hybrid prediction models this chapter's genomic results rest on. Chapter 19. and Chapter 20. take the hybrids a crossing plan produced and select among them under the diversity constraint; Chapter 18. is the same optimal-contribution machinery that, applied to crosses rather than hybrids, becomes the optimal cross selection of G. Gorjanc, R. C. Gaynor, and J. M. Hickey [59]. And Chapter 22. is where the consequence of holding a design fixed while the pools drift is played out over many cycles.

Reproduce this chapter

All the code of this chapter, in order and with the explanation kept as comments, is one script: **04e-mating-design.R**. Download it, install `hybdiv`, and run it top to bottom in an empty directory; cached results are fetched on first use.

10. Sparse testing: which hybrids go to which locations

A hybrid programme's first field stage concentrates most of its selection pressure where it has the least information. It typically has thousands of testcrosses, seed for one or two plots of each, a handful of locations, and almost no replication. The previous chapter (Chapter 9.) asked **which crosses** to make from two pools. This one asks the next question: given the crosses, **which locations should each one be planted in?**

Sparse testing is the answer the literature has converged on. Not every hybrid is grown at every location. Different subsets go to different environments, and the missing hybrid x environment cells are *predicted*, not discarded, with a relationship matrix connecting what the field left unconnected. In Chapter 9. the sparsity was in the female x male grid. Here it is in the hybrid x environment grid. Real programmes face both at once, the joint problem Section 9.9 lists as its second gap.

The chapter covers the quantitative basis, the history and a taxonomy of the designs, the validation schemes used to judge them, and the empirical record across crops, including its main controversy. It then measures the question with a simulation that the primary literature had not run. The results it defends:

1. **At a fixed number of plots, spreading them over more locations beats concentrating them.** Spreading two plots per hybrid over ten locations instead of two raises accuracy by 0.06 to 0.09, with no extra plots.
2. **The choice among sparse layouts matters little, except at extreme sparsity.** At that point, a layout that puts *both pools in every location* helps the hybrids that were never tested.
3. **What connects a sparse trial is parent coverage, not shared hybrids.** With a relationship matrix in the model, the best layouts share *no* hybrid between some location pairs. This suggests one way to reconcile the contradictory published advice on overlap between environments.
4. **Prediction survives sparsity better than estimation.** The BLUP stays undispersed down to one location per hybrid. Variance components stop being reportable earlier, and the SCA variance is biased upward even in the complete design.

i Where the numbers come from

There are three sources, kept apart throughout.

- **Live** results are computed at render time with `R/17_sparse_testing.R`:
  - allocation generators;
  - connectivity diagnostics;
  - a base-R mixed-model solve with **known** variance components, checked against a literal GLS oracle (`ref_sparse_blup()`).
- **Cached** results come from a separate study and are not reproducible from this repository. That study estimated components by REML with `sommer` on 2,630 fits, ran a two-stage scenario, and added an AlphaSimR genomic layer. Its tables are in `book/data/sparse_*.csv`.
- **In-repo** results come from `Rscript book/scripts/regenerate.R sparse`, which reruns the phenotypic core with known components (`sparse_core_sweep.csv`). That sweep also adds an experiment the cached study did not run (Section 10.6). The two are compared in

  2.

The literature numbers are from six primary papers: Y. Li *et al.* [68], S. A. Atanda, V. Govindan, R. Singh, K. R. Robbins, J. Crossa, and A. R. Bentley [44], C. R. Werner, G. Covarrubias-Pazaran, D. Gonzalez-Dieguez, D. C. Gemenet, H. Gandhi, and G. N. Atlin [69], N. Lubanga, B. E. Ifie, R. Persa, I. Dieng, I. Y. Rabbi, and D. Jarquin [70], D. González-Diéguez, G. Atlin, Y. Beyene, D. Wegary, D. Gemenet, and C. Werner [35] and A. Hild Aono and A. Chawade [71]. Each one was read in full. **C. R. Werner, G. Covarrubias-Pazaran, D. Gonzalez-Dieguez, D. C. Gemenet, H. Gandhi, and G. N. Atlin [69] is a preprint that has not been peer reviewed.**

10.1 The allocation problem of stage 1

10.1.1 The economics of the first stage

Stage 1 is where a plot costs least and a decision is worth most. In CGIAR programmes most of the selection pressure is applied on a few research stations, often as visual selection in unreplicated nurseries [69], [72]. Two shifts have changed this stage:

- **Genotyping is no longer the bottleneck. Phenotyping is.** The cost of a marker has fallen steadily for two decades, while the cost of a yield plot has not [64].
- **Doubled haploids reversed the constraint.** A programme now makes more candidates than it can evaluate [66]. Allocating plots, not creating variation, has become the limiting factor.

What is left is the classical trade-off. More genotypes at fewer locations buys selection intensity and costs accuracy. Fewer genotypes at more locations does the opposite. At stage 1 the scarce resource is the plot, so the question is not *how much to test* but *how to spread what can already be tested* [68].

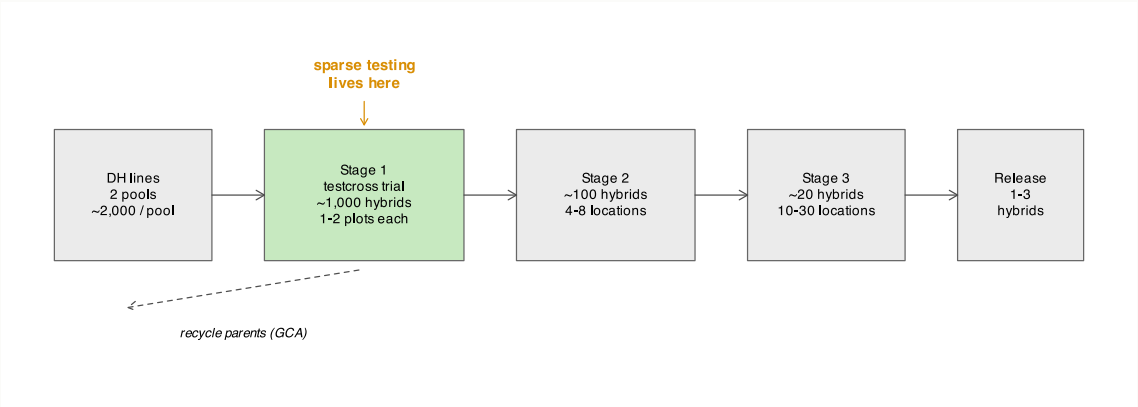

Figure 10.1: A two-pool hybrid pipeline. Population improvement (left) feeds product development (right). Candidate numbers fall by an order of magnitude per stage while plots per candidate rise. Sparse testing applies where the plots per candidate are fewest: the first testcross trial.

10.1.2 The breeder's equation as the decision criterion

The gain per unit time,

$$\Delta G_t = \frac{i\, r\, \sigma_A}{t}, \tag{10.1}$$

has four terms. A trial design moves at least three of them together:

- **intensity** i, through the number of candidates;
- **accuracy** r, through plots, locations, and how well the locations sample the target population of environments (TPE);
- **cycle length** t, when parents can be recycled from stage 1.

What a gain measured on a research station is worth depends on the genetic correlation between the selection environment and the TPE [73], [74]. No design is better in the abstract. A design is better according to whichever term limits that programme [65], [75].

Box 1 – Where each term of the breeder's equation meets the trial design

Term	Design decision that moves it
i (intensity)	number of candidates tested at a fixed plot budget
r (accuracy)	plots per candidate, locations per candidate, use of a relationship matrix
r (relevance)	how well the chosen locations sample the TPE
σ_A	indirectly: which parents are recycled, and how related they are (Chapter 18.)
t	whether stage-1 GCA predictions are good enough to recycle parents immediately

10.2 Foundations

10.2.1 The hybrid model: GCA, SCA, and what sparsity threatens

The value of the single cross between female i and male j splits as

$$g_{ij} = \mathrm{GCA}_i^{f} + \mathrm{GCA}_j^{m} + \mathrm{SCA}_{ij} \tag{10.2}$$

[9]. Section 13.10 derives this split from allele frequencies. Its consequence for this chapter is that a line's GCA includes dominance effects weighted by the *opposite* pool's frequencies. A GCA is therefore specific to the tester pool, not a property of the line alone [53], [76]. The ratio $\sigma^2_{SCA}/\sigma^2_{GCA}$ is a design parameter. It grows with the mean degree of dominance and with how close the pools are. In simulated maize it was 0.05, 0.23 and 0.50 for mean dominance degrees of 0.1, 0.5 and 0.9 [35].

GCA survives sparsity better than SCA because it is mostly additive and more stable across environments. Y. Li *et al.* [68] recovered GCA from a prediction-completed 50 % trial with correlations of 0.94 to 0.97 against the complete data. Over the same scenarios, phenotypic prediction accuracy was 0.43 to 0.59. **What stage 1 needs to estimate is GCA, and GCA is the part of the model most robust to missing cells.** That is the underlying reason sparse testing works in hybrids.

Quick glossary

- **GCA**: a line's average performance in crosses with the opposite pool.
- **SCA**: the deviation of a particular cross from the sum of its parents' GCAs.
- **Mid-parent heterosis**: the hybrid's advantage over the mean of its parents (Chapter 13.).
- **Heterotic pool**: a group of lines bred to cross well with another group.
- **Tester**: a line of the opposite pool used to evaluate candidates.
- **TPE**: the target population of environments the product is sold into.

10.2.2 Hybrid relationship matrices

With female and male relationship matrices $\mathbf{A}_f$ and $\mathbf{A}_m$, the three hybrid terms are distributed as

$$\mathrm{GCA}^f \sim N(\mathbf{0}, \mathbf{A}_f\sigma_f^2), \quad \mathrm{GCA}^m \sim N(\mathbf{0}, \mathbf{A}_m\sigma_m^2), \quad \mathrm{SCA} \sim N(\mathbf{0}, (\mathbf{A}_f \otimes \mathbf{A}_m)\sigma_s^2) \tag{10.3}$$

[46], [47], [50]. The practical point is that **the hybrid itself never needs genotyping**: its covariance is a Kronecker product of the parental matrices. Replacing pedigree $\mathbf{A}$ by marker $\mathbf{G}$ [77] gives the genomic version (3). There are two ways to write the additive and dominance effects: the "biological" form and the "statistical" form, which is orthogonal at the population's frequencies. Pool-specific marker effects are a third option [53], [78]. All of them lead to the same covariance structure.

The relationship matrix is what makes a sparse design analysable. It is the only link between cells that the field did not link. A hybrid with no plot gets a BLUP only because its parents are related to parents that do have plots. Without it, a sparse design yields comparisons that are not fair.

10.2.3 Multi-environment trials and mixed models

How many parameters the between-environment covariance spends ranges widely:

- identity, diagonal, or compound symmetry;
- heterogeneous AR(1);
- unstructured;
- factor-analytic [79], [80], [81], [82].

Factor-analytic models carry most of the information of the unstructured matrix at a fraction of its parameters, and extend directly to genomic relationships [83], [84], [85]. Reaction-norm and marker x environment models are cheaper alternatives [86], [87]. Only *crossover* G x E changes selection decisions. A scale effect does not.

In a sparse design the between-environment covariance stops being a statistical detail and becomes part of the design. It decides how much one environment can tell us about another. Two-stage analyses that carry the first-stage error forward are now as efficient as one-stage ones [88], [89]. Unreplicated trials still need spatial correction and checks [90], [91].

Box 2 – Which heritability to report in a sparse trial

Quantity	Formula	What it answers	Behaviour under sparsity
Plot-level h^2	$\sigma^2_G/(\sigma^2_G + \sigma^2_{GE} + \sigma^2_e)$	how informative one plot is	design-independent; it is what the simulations below fix
Entry-mean H^2	$\sigma^2_G/(\sigma^2_G + \sigma^2_{GE}/L + \sigma^2_e/(rL))$	how informative an entry mean is, if every entry had L locations and r reps	undefined when entries have different L, i.e. in every sparse design
Generalised (Cullis) H^2	$1 - \overline{\mathrm{PEV}}/(2\sigma^2_G)$ for pairwise differences [91], [92]	how well the analysed design ranks entries	the right one: it uses the actual PEVs

The entry-mean formula silently assumes balance. In a sparse trial, report Cullis'.

10.3 What sparse testing is, and where it came from

10.3.1 Definition and genealogy

Definition. A trial is sparse when not every genotype is evaluated in every environment *and* the missing cells are predicted rather than discarded. The idea developed in four steps:

1. **Incomplete blocks and lattices.** These already accepted partial evaluation *within* a location.
2. **Partially replicated designs (p-rep) and augmented p-rep.** These replaced replication of each entry with replicated checks and partial replication [91], [93].

3. **The genomic relationship replacing replication** [48]. With markers, the replicated unit is no longer the genotype but the haplotype, shared among relatives.
4. **Genomic sparse testing across environments, formalised** [42]. The design was later framed explicitly as a balanced incomplete block design in which the block is the environment [45].

Farm-as-incomplete-block is the limiting case. Each farmer's field is an incomplete block holding a handful of entries, and the TPE is sampled through hundreds of them [69], [74], [75].

Sparse testing is not a new method. It is incomplete-block design applied at the scale of many environments, made workable by genomic relationships acting as the connecting mechanism. The classical theory of incomplete blocks therefore carries over.

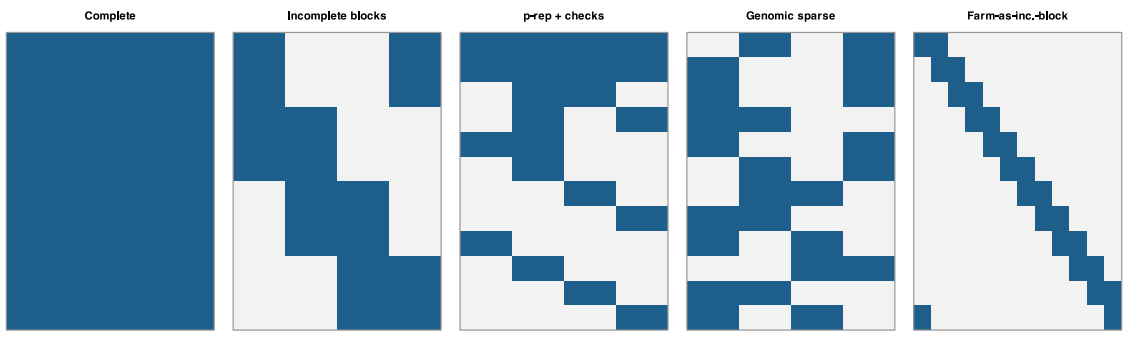

Figure 10.2: The genealogy of sparse testing, drawn as genotype (rows) x environment (columns) matrices; dark = observed. From the complete trial, to incomplete blocks within locations, to p-rep (a few entries and checks replicated), to genomic sparse testing (each entry in a subset of environments, connected through relationships), to farm-as-incomplete-block (many environments, very few entries each).

10.3.2 A taxonomy of sparse designs

The designs can be classified along three independent axes.

Axis of sparsity.

- *Environments*: the classical case.
- *Testers*: sparse testcrossing (Section 10.8).
- *Female x male combinations*: the incomplete factorials of Section 9.4.
- Any combination of these.

Overlap structure.

- full overlap, with a common core in every environment;
- no overlap;
- partial overlap, with a fixed core plus entries unique to each environment;
- unrestricted random allocation;
- restricted random allocation, for example keeping full-sibs out of the same location [69];
- allocation optimised against a formal criterion [94], [95].

Replicated unit. Genotype, family, or haplotype.

```
set.seed(5)
core6 <- function(n_core) {              # a common core + unique entries
  M <- matrix(0L, 32, 6); M[seq_len(n_core), ] <- 1L
  rest <- setdiff(1:32, seq_len(n_core))
  M[cbind(rest, rep_len(1:6, length(rest)))] <- 1L; M
}
tax <- list("Complete"          = sparse_alloc(8, 4, 6, strategy = "complete"),
            "Full overlap core" = { M <- matrix(0L, 32, 6); M[1:10, ] <- 1L; M },
            "Core + unique"     = core6(5),
            "No overlap"        = core6(0),
            "Random, k = 2"     = sparse_alloc(8, 4, 6, 2, "random"),
            "Parentage, k = 2"  = sparse_alloc(8, 4, 6, 2, "parentage"))
op <- par(mfrow = c(1, 6), mar = c(1, 0.5, 2.2, 0.5))
```

```
for (nm in names(tax)) {
   image(t(tax[[nm]][32:1, ]), col = c("grey95", "#1f5f8b"), zlim = c(0, 1), axes
= FALSE,
         main = sprintf("%s\n(%d plots)", nm, sum(tax[[nm]])), cex.main = 0.8)
  box(col = "grey60")
}
par(op)
```

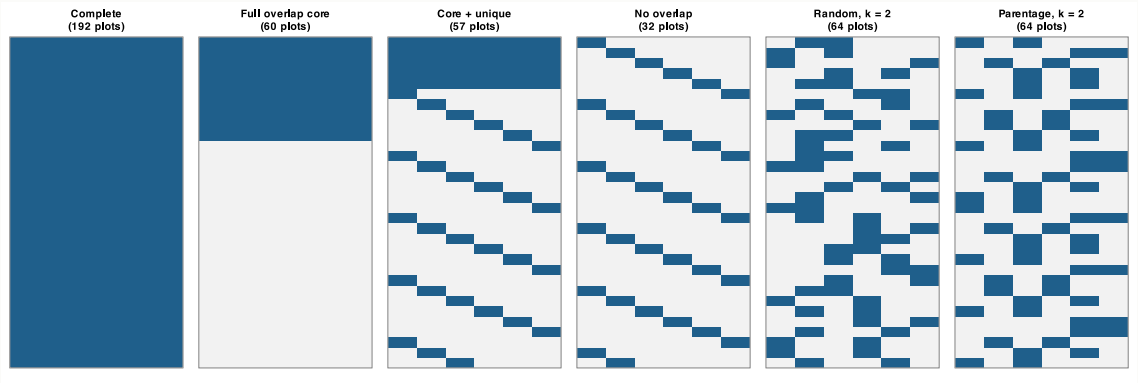

Figure 10.3: Six allocations of the same 8 x 4 factorial (32 hybrids, rows; male index fastest) to six locations (columns), each with the same number of plots except 'complete'. The last three are generated by the package's `sparse_alloc()`. 'Parentage' puts every female and every male in every location.

Table 10.1: The taxonomy mapped onto the designs in the literature. Different names often describe the same structure.

Design	Sparsity axis	Overlap	Example	Reference
Full overlap (common core only)	environments	full	Li et al. "CV1" allocation	Y. Li *et al.* [68]
Random cells, ≥ 1 per genotype	environments	random	Li et al. "CV2"; Jarquin's allocation schemes	Y. Li *et al.* [68]; D. Jarquin *et al.* [42]
Balanced by genotype (one environment each)	environments	none	Li et al. "CV3"; Lubanga NOL	Y. Li *et al.* [68]; N. Lubanga, B. E. Ifie, R. Persa, I. Dieng, I. Y. Rabbi, and D. Jarquin [70]
Core + unique entries	environments	partial	Li et al. "CV5"; Atanda overlap series	Y. Li *et al.* [68]; S. A. Atanda, V. Govindan, R. Singh, K. R. Robbins, J. Crossa, and A. R. Bentley [44]
Restricted random, full-sibs apart	environments	random	farm-as-incomplete-block	C. R. Werner, G. Covarrubias-Pazaran, D. Gonzalez-Dieguez, D. C. Gemenet, H. Gandhi, and G. N. Atlin [69]
One tester per line, sibs across testers	testers	none at line level	sparse testcrossing	D. González-Diéguez, G. Atlin, Y. Beyene, D. Wegary, D. Gemenet, and C. Werner [35]
Incomplete factorial	female x male	partial	sparse factorials	A. Seye, C. Bauland, A. Charcosset, and L. Moreau [34]; A. Lorenzi *et al.* [40]

⚠ A naming collision to watch for

Y. Li *et al.* [68] use **"CV1" to "CV5" as names of allocation strategies**, not of the classical validation schemes of Section 10.4. Their "CV1" is the full-overlap *design*. It is not the "new genotype" *validation* of J. Burgueño, G. de los Campos, K. Weigel, and J. Crossa [85]. Reading the two literatures together without noticing this produces wrong comparisons.

10.4 Validation schemes: what each one answers

The classical schemes are defined by two questions. Has the genotype been observed anywhere? Has the environment been observed for anyone? [42], [85]

- **CV2**: the genotype is observed in some environments and missing in others. The task is to *complete the table.*
- **CV1**: the genotype is new, never observed anywhere. The task is to *predict an unseen candidate.*
- **CV0**: the environment is new (next year, say) and the genotypes are known. The task is to *extrapolate to an untested environment.*
- **CV00**: both the genotype and the environment are new. This is the hardest case, and the most realistic for long-run decisions.

CV2 will always look better than CV1. Comparing accuracies across papers that use different schemes is therefore a common and serious error [68], [96]. The correlation is not the only metric:

- top-x % coincidence: top 20 % by a Smith-Hazel index in S. A. Atanda, V. Govindan, R. Singh, K. R. Robbins, J. Crossa, and A. R. Bentley [44], top 50 % in Y. Li *et al.* [68];
- mean squared error [70];
- Kendall's τ [71];
- PEV and reliability, and the dispersion slope b_1 of truth on prediction.

At stage 1 the operationally relevant number is the **rate of wrongly discarded superior candidates**. The correlation is only a proxy for it.

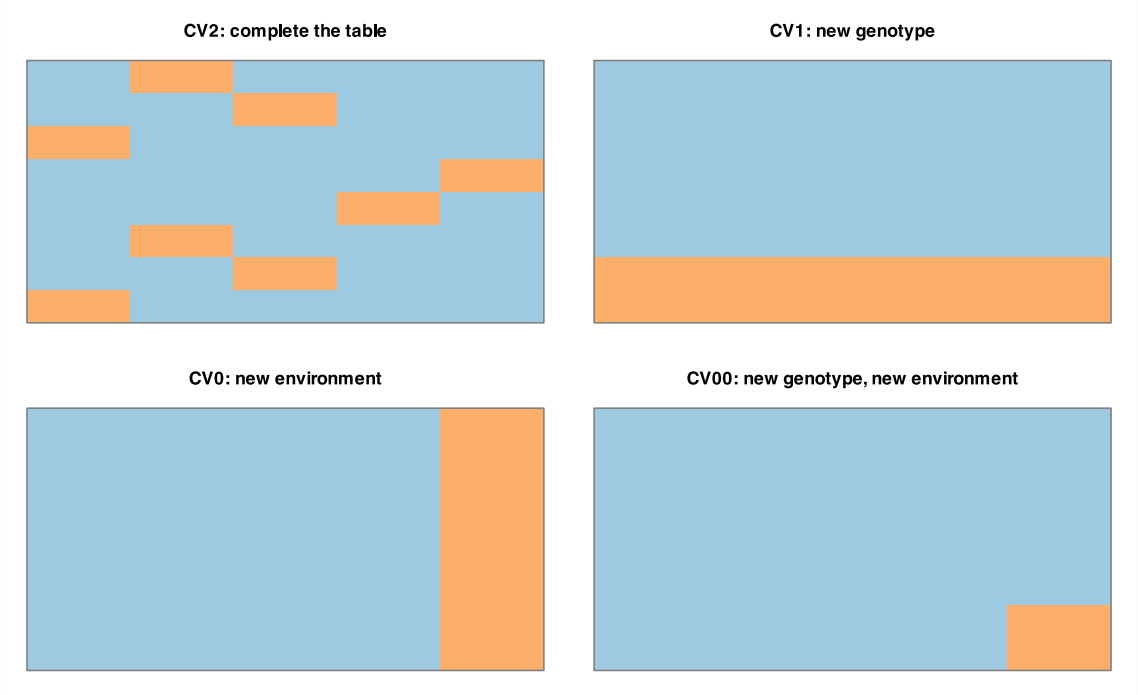

Figure 10.4: The four validation schemes as a 2 x 2 by whether the genotype (rows) and the environment (columns) were observed. Blue: training cells. Orange: the cells to be predicted.

Box 3 – CV0, CV1, CV2 and CV00

	Environment observed	Environment new
Genotype observed	**CV2**: fill a missing cell. *"How does hybrid H do at location L, where it was not planted?"*	**CV0**: *"How will the known hybrids do next year?"*
Genotype new	**CV1**: *"How good is a hybrid that was never planted?"* In this chapter's simulation, 20 % of hybrids per replicate	**CV00**: *"How good will an untested hybrid be next year?"*

In the simulation below, `acc_cv1` and `acc_cv2` are measured exactly in this sense.

10.5 The empirical record across crops

Table 10.2 puts the six primary studies side by side. There are two maize studies, one of which is a simulation of reciprocal recurrent selection. The others cover wheat, cassava and potato, and one simulates a self-pollinated crop. Supporting studies cover wheat [97], [98], sugarcane [99], soybean [100], tropical maize [43] and potato [101].

```
knitr::kable(ps[, c("study", "crop", "genotypes", "environments", "observed",
                    "model", "accuracy", "conclusion")],
             col.names = c("Study", "Crop", "Genotypes", "Environments", "Observed",
                           "Model", "Accuracy", "Conclusion"))
```

Table 10.2: The six primary studies. 'Observed' is the share of genotype x environment cells (or the equivalent) with data.

Study	Crop	Genotypes	Environments	Observed	Model	Accuracy	Conclusion
Li et al. 2026	maize	264	6 locations x 1 year	16.7-83.3%	multi-environment GBLUP (reaction norm best)	0.418 (full overlap) vs 0.479-0.504	Diminishing returns past ~50% of cells; full overlap is the worst allocation
Atanda et al. 2022	spring wheat	280	6 selection environments	16.4-58.9% per environment	GBLUP / PBLUP, XFA G x E, AR1 x AR1 spatial	+42% to +989% from 0 to 50% overlap	Substantial overlap of lines between environments recommended
Werner et al. 2025 (preprint)	simulated wheat	2000	1-440 locations	p-rep 0.1 on 2,200 plots	GBLUP with GRM, restricted randomisation	sparse 4.32-4.48 vs conventional 2.84-3.67	Every sparse scheme beat every conventional one; most of the gain by 5 locations
Lubanga et al. 2025	cassava	435	5 location-years	10.8-20.0% per environment	reaction norm, E + L + G + G x E	max 0.53 (dry matter) / 0.47 (yield)	More overlap HURT at fixed training size; 20% of records suffice
Gonzalez-Dieguez et al. 2026	maize hybrids, simulated RRS	1200 per pool	1-1200 testers	1 testcross per line	pool-specific additive + dominance RRBLUP	+3/11/23% GCA accuracy with 10 testers (low/mid/high dominance)	Plateau at 5-10 testers; 3-5 recommended
Hild Aono & Chawade 2026	potato	256	3 location-years	10-66%	BGLR with genomic and image-derived environment kernels	0.995-0.997 (image kernel) vs 0.612-0.798 (genomic), pairs at 50%	Image kernel wins from ~30% completeness; possible information leakage

The anchor numbers are as follows.

- **Maize** [68]. Mean accuracy was 0.418 under full overlap and 0.479 to 0.504 under every other allocation. Returns diminish beyond about 50 % coverage: going from 50 % to 67 % cost 33 % more phenotyping and returned 3.5 to 5.4 % accuracy. Top-50 % coincidence was above 61.9 % in all 25 strategies.
- **Wheat** [44]. Going from 0 to 50 % common lines raised predictive ability by 42 % to 989 %, depending on the environment. Plot h^2 was 0.18 in the worst environment. Genetic correlations between environments in the same year ranged from -0.04 to 0.67.
- **Cassava** [70]. With 20 % of records per environment, predictive ability was moderate (maxima of 0.53 for dry matter and 0.47 for fresh root yield). Phenotyping cost could be cut by up to 80 %.
- **Potato** [71]. For pairs of environments at 50 % completeness, r^2 was 0.995 to 0.997 with an image-derived environment kernel, against 0.612 to 0.798 with the genomic kernel. Section Section 10.10 covers the caveat.

The common denominator is the **genetic correlation between environments, and the size and diversity of the training set**. Together they explain most of the spread. In S. A. Atanda, V. Govindan, R. Singh, K. R. Robbins, J. Crossa, and A. R. Bentley [44], the environment with low heritability and weak correlations stayed poorly predicted under every design. In Y. Li *et al.* [68], plant height was the trait with the strongest between-environment correlations (0.36 to 0.67) and the best predicted. Grain yield had no

strong correlations and was predicted worst. Modelling G x E explicitly helped in every study that tested it [68], [70]. **Cutting phenotyping by 50 to 80 % without losing accuracy is feasible. The exact cut depends on the programme's own between-environment correlations, and does not transfer between programmes.**

10.6 The minimum-overlap controversy

The literature disagrees openly on how much overlap is needed:

- S. A. Atanda, V. Govindan, R. Singh, K. R. Robbins, J. Crossa, and A. R. Bentley [44] conclude that "a substantial overlap of lines between environments" is needed.
- L. Crespo-Herrera *et al.* [97] suggest 30 to 50 common lines.
- R. Persa *et al.* [100] report a gain with as few as 10.
- N. Lubanga, B. E. Ifie, R. Persa, I. Dieng, I. Y. Rabbi, and D. Jarquin [70] find that **more overlap made prediction worse**: predictive ability fell and MSE rose monotonically with the number of overlapping genotypes, and the fully non-overlapping design was the best.
- Y. Li *et al.* [68] sit in between. Full overlap was the worst design, and the four partial or non-overlapping designs tied.
- C. R. Werner, G. Covarrubias-Pazaran, D. Gonzalez-Dieguez, D. C. Gemenet, H. Gandhi, and G. N. Atlin [69] and D. González-Diéguez, G. Atlin, Y. Beyene, D. Wegary, D. Gemenet, and C. Werner [35] do without overlap altogether. All of their connectivity comes from relationships.

A proposed reconciliation, offered here as a hypothesis. The designs are not comparable because they confound overlap with the size and diversity of the training set:

- In N. Lubanga, B. E. Ifie, R. Persa, I. Dieng, I. Y. Rabbi, and D. Jarquin [70] the training set per environment is fixed in size. More overlap therefore means *fewer distinct genotypes observed.*
- In S. A. Atanda, V. Govindan, R. Singh, K. R. Robbins, J. Crossa, and A. R. Bentley [44] more overlap means a *larger* calibration set per environment, growing from 46-47 lines to as many as 165.

A second factor is family structure. Families of 1 to 9 individuals [44] give the relationship matrix little to work with, so physical overlap has to carry the connection. Families of 20 to 50 [35], [69] let the relationship matrix carry all of it.

The first half of the hypothesis is testable directly with the package. The experiment uses four locations and a training set **fixed at 80 hybrids per location** out of 400. A fraction of the 80 is a core common to all four locations, and the rest are unique to each location. Family size runs from unrelated lines to half-sib families of 10 (`regenerate.R sparse`, part c; 30 replicates per point; known components; plot $h^2 = 0.4$).

```
ov <- subset(core, part == "overlap")
fs <- sort(unique(ov$family_size))
cols <- c("grey40", "#1b9e77", "#d95f02")[seq_along(fs)]
op <- par(mar = c(4.2, 4.2, 3.2, 1))
plot(NA, xlim = c(0, 1), ylim = range(ov$acc_all + c(-2, 2) * ov$acc_all_se),
     xlab = "Overlap: fraction of each location's plots in the common core",
     ylab = "Accuracy, all hybrids")
for (i in seq_along(fs)) {
  d <- ov[ov$family_size == fs[i], ]; d <- d[order(d$overlap), ]
  lines(d$overlap, d$acc_all, type = "b", pch = 19, col = cols[i])
   arrows(d$overlap, d$acc_all - 2 * d$acc_all_se, d$overlap, d$acc_all + 2 *
d$acc_all_se,
         angle = 90, code = 3, length = 0.03, col = cols[i])
}
d0 <- ov[ov$family_size == fs[1], ]; d0 <- d0[order(d0$overlap), ]
axis(3, at = d0$overlap, labels = d0$n_distinct, cex.axis = 0.8)
mtext("distinct hybrids observed", 3, line = 2, cex = 0.8)
legend("bottomleft", bty = "n", lwd = 2, col = cols,
      legend = ifelse(fs == 1, "unrelated lines", paste("half-sib families of", fs)))
par(op)
```

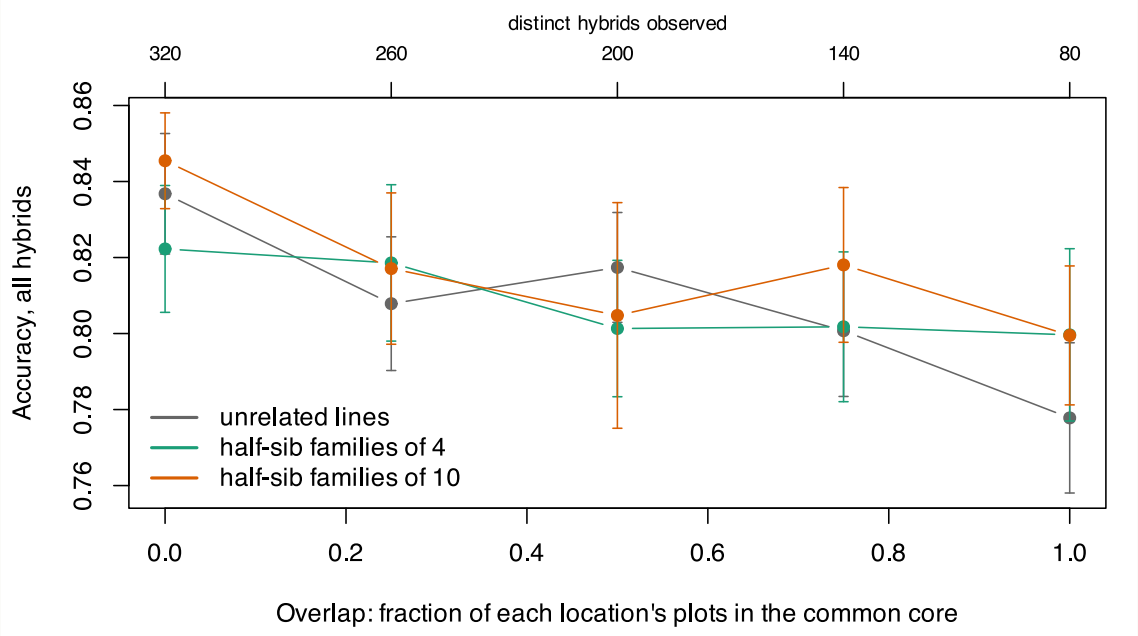

Figure 10.5: Accuracy over all 400 hybrids against the fraction of each location's 80 plots taken by a core common to all four locations, by half-sib family size. The training size is held fixed, so more overlap means fewer distinct hybrids observed (top axis). Bars are two standard errors over 30 replicates. In-repo sweep, known variance components.

With the training size held fixed, accuracy falls as overlap rises: it is 0.023 to 0.059 lower under full overlap than under none, 1.6 to 4.7 standard errors of the difference depending on family size. The cause is the confound, not overlap as such. Every hybrid moved into the common core is one fewer distinct hybrid observed (top axis). This is the N. Lubanga, B. E. Ifie, R. Persa, I. Dieng, I. Y. Rabbi, and D. Jarquin [70] pattern, reproduced with the mechanism exposed.

The second half of the hypothesis is **not** supported at this scale. The family-size curves cross, and they differ by no more than two standard errors. In hindsight the reason is structural. In a female x male factorial every line is already the parent of 20 hybrids, so the factorial carries much of the connectivity before any relationship *among lines* is added. Half-sib families add relatively little on top of that. In a line-breeding programme like S. A. Atanda, V. Govindan, R. Singh, K. R. Robbins, J. Crossa, and A. R. Bentley [44], each entry is its own genotype and no parent links them, so family structure should matter far more. This experiment cannot test that case. The experiment also holds the training size fixed, so it does not reproduce S. A. Atanda, V. Govindan, R. Singh, K. R. Robbins, J. Crossa, and A. R. Bentley [44]'s design, in which more overlap also meant a larger calibration set and the two effects pull in opposite directions.

What to do with this. The right question is not "how many common genotypes?". It is "how much *effective* connectivity does the design have, counting both physical overlap and relationships?". Physical overlap and genomic relationships are partial substitutes [102]. In a hybrid factorial, shared parents already supply much of the relationship side.

10.7 Connectivity and checks

Connectivity is not the same as overlap. Two designs with the same number of common genotypes can be connected very differently, depending on *how* those genotypes are distributed. Hybrids add a dimension. There is female x male connectivity, the subject of Section 9.2.3. There is genotype x location connectivity. And there is **female x male x location**, for which no metric is agreed. That is an open gap (Section 10.13). The available tools are:

- the rank of the incidence matrix;
- the PEV of pairwise contrasts, and the reliability derived from it;
- the connected components of the genotype-environment graph.

Checks do three jobs:

- anchor connectivity;
- estimate within-environment error in unreplicated trials, as in the augmented design of Y. Li *et al.* [68];
- control spatial trend [91], [93].

Restricted randomisation is a cheap connectivity tool. C. R. Werner, G. Covarrubias-Pazaran, D. Gonzalez-Dieguez, D. C. Gemenet, H. Gandhi, and G. N. Atlin [69] kept full-sibs apart within a location, but did not measure the restriction against unrestricted randomisation. Section 10.11.3 does measure a structured layout against unrestricted randomisation.

Connectivity can be checked before planting. Compute it at the planning stage, rather than finding out from a model that does not converge.

Box 4 – Computing the connectivity of a design before planting it

1. Build the genotype x environment incidence matrix.
2. Check the connected components of the bipartite graph. Every location should share hybrids, or at least parents, with every other.
3. Compute the mean PEV of the target values under the intended model, with provisional variance components.
4. Convert it to reliability, $1 - \overline{\mathrm{PEV}}/\sigma^2_G$.
5. Compare candidate designs on the same metric.

```
set.seed(8)
nF <- 20; nM <- 20; L <- 6
Af <- relmat_halfsib(nF, 5); Am <- relmat_halfsib(nM, 5)
vc <- sparse_vc(0.3)
recipe <- function(M) {
  cn  <- sparse_connectivity(M, nF, nM)                       # steps 1–2
  # PEV does not depend on y, so any phenotype vector will do (steps 3–4).
  cells <- which(M == 1L, arr.ind = TRUE)
  ph  <- data.frame(y = 0, hyb = cells[, 1], loc = cells[, 2])
  pev <- sparse_blup(ph, nF, nM, L, vc, Af, Am)$pev
    c(plots   =  sum(M),  fem_cover  =  cn$fem_cover,  disconnected_pairs  =
cn$frac_disconn,
    reliability = 1 - mean(pev) / vc$Vg)
}
fmt(rbind(complete_2loc = recipe(sparse_alloc(nF, nM, 2, strategy = "complete")),
          random_k2     = recipe(sparse_alloc(nF, nM, L, 2, "random")),
          parentage_k2  = recipe(sparse_alloc(nF, nM, L, 2, "parentage"))), 3)
```

	plots	fem_cover	disconnected_pairs	reliability
complete_2loc	800	1	0	0.625
random_k2	800	1	0	0.703
parentage_k2	800	1	0	0.705

All three designs cost 800 plots. Spreading them over six locations raises reliability *before a single plot is sown.* The PEV depends only on the design and the components, not on the data. That makes it a design tool and not only an evaluation metric.

10.8 Sparse testcrossing and the number of testers

The central idea of D. González-Diéguez, G. Atlin, Y. Beyene, D. Wegary, D. Gemenet, and C. Werner [35] is to treat the tester as an "environment". Each line is crossed to **one** tester, but full-sibs go to *different* testers. What gets evaluated against several testers is therefore the haplotype, not the genotype. This removes a real logistical constraint. The more accurate alternative, an incomplete factorial among candidates (Section 9.4), needs hand pollination at scale and synchronised flowering between pools.

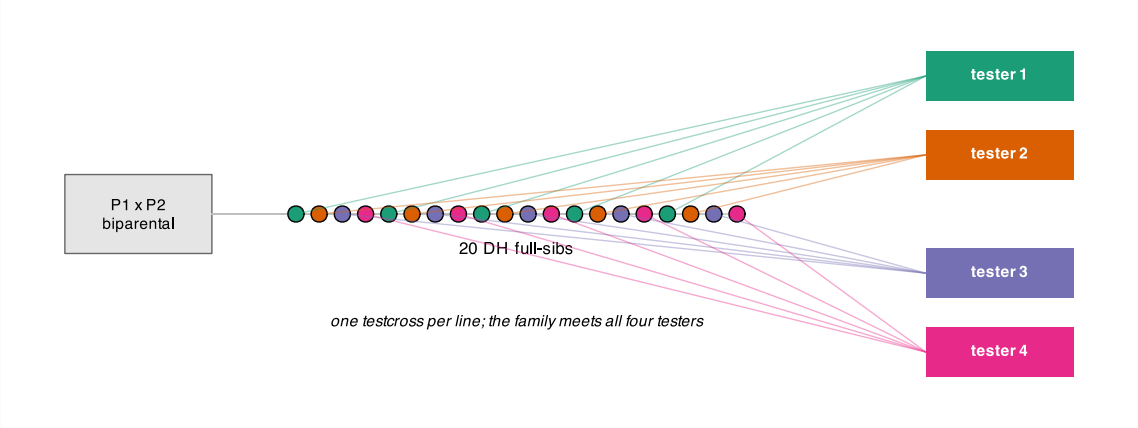

Figure 10.6: Sparse testcrossing. A biparental cross yields 20 DH lines; each line is crossed to one of four testers, so the family – not the line – meets every tester. The relationship matrix connects the full-sibs, and through them the four testers.

Table 10.3: Sparse testcrossing in D. González-Diéguez, G. Atlin, Y. Beyene, D. Wegary, D. Gemenet, and C. Werner [35]: accuracy and gain against a single tester at an equal number of testcrosses. Every sparse layout beat the single tester, and gains plateau at 5 to 10 testers; 3 to 5 is the practical recommendation.

Testers (1,200 testcrosses per pool)	Gain in GCA accuracy over 1 tester	Gain in hybrid genetic gain, cycle 15
2, medium dominance	+5.3 %	–
10, low / medium / high dominance	+3 % / +11 % / +23 %	+3 % / +6 % / +11 %
2 to 10, medium dominance	+0.7 % on top of the 2-tester gain	–

The advantage is largest in early cycles, while the pools are still close. That does **not** confine the method to emerging programmes. Real commercial pools sit at a minimum Nei distance of 0.17 [76] and an F_{ST} of 0.223 [103], which corresponds to cycles 2 to 4 of the simulation.

A useful side effect is that sparse testcrossing *slowed the erosion of within-pool variance.* It stops selection from piling onto closely related lines that happen to nick with one tester. The authors recommend combining it with optimum contribution selection (Chapter 18.; N. Santantonio and K. Robbins [104]).

The deterministic tradition reaches the same order of magnitude by another route [27]. The advocates of incomplete factorials [29], [34], [40], [67] reach a different conclusion. In D. González-Diéguez, G. Atlin, Y. Beyene, D. Wegary, D. Gemenet, and C. Werner [35], an approximate incomplete factorial matched 3 to 5 testers, with much heavier logistics. Two costs were not modelled. Five testers need ten isolation blocks. And in many public programmes hand pollination is already routine, which lowers the extra cost of the factorial.

Box 5 – Choosing testers: theory against practice

The classical recommendation is a *low-performing* tester, or one carrying recessive alleles at the loci of interest. Such a tester discriminates among candidates best [26], [105]. Commercial practice uses *elite* testers, chosen for seed production and *per se* performance, because the testcross may itself become a product. The consequence follows from Section 13.10: GCA is measured *against the tester's allele frequencies.* An elite tester carries the favourable allele at many loci and shrinks the visible differences there, so a few elite testers sample the opposite pool's frequencies poorly. Several testers spread across the pool, the sparse-testcross recipe, sample them better [27], [106].

10.9 What sparsity does to variance components and to the BLUP

Two questions are often merged in the literature, and they should be kept separate:

1. Do the predictions stay accurate?

2. Do the variance-component estimates stay unbiased?

The second is barely covered. **None of the six primary studies reports bias in σ^2_{GCA}, σ^2_{SCA} or σ^2_{GE} under sparsity.** The indirect evidence is thin:

- In A. Hild Aono and A. Chawade [71] the G x E term *degraded* prediction under high sparsity, because there were too few individuals to estimate it. The authors recommend dropping it in that regime.
- In Y. Li *et al.* [68] propagating prediction uncertainty barely changed the GCA estimates (correlation 0.9974 to 0.9998 with the point estimates). That is a sensitivity of GCA, not of the variance components.

Two diagnostics separate the questions.

- **The dispersion slope b_1**, from regressing truth on prediction. $b_1 = 1$ means no scale bias. $b_1 < 1$ means the predictions are too spread out, and $b_1 > 1$ means they are over-shrunk.
- **PEV and reliability.** These can be computed *before* any data exist (Box 4), so they serve as design tools.

Accuracy and estimation quality do not degrade at the same rate. **A design can rank candidates well and still estimate the SCA and G x E variances badly** [82], [84], [91], [92]. The simulation of Section 10.11.3 measures this directly. That is the gap in the primary literature it was designed to fill.

10.10 Sparse testing with phenomics

Phenomic data pose a specific problem. Markers can be read from a seedling whatever happens in the field. Sensor data exist only for plots that were actually grown, so they cannot serve as a covariate for an individual that was never observed. There are three ways out:

- multi-trait models with image indices as secondary traits;
- kernel regression on the indices;
- the proposal of A. Hild Aono and A. Chawade [71]: use the images to build an **environment** kernel.

The environment kernel is built in three steps. A PCA of the image traits is run *within each environment*. The first-component eigenvector of each environment is extracted. The kernel entries are the correlations between those eigenvectors.

The image kernel matched or beat the genomic kernel from about 30 to 33 % completeness, reaching $r^2 = 0.997$ on pairs of environments. Below about 20 % the genomic kernel was preferable again. **The authors warn of possible information leakage**, because the kernel came from the same trial that supplied training and validation. The result should be reported with that caveat.

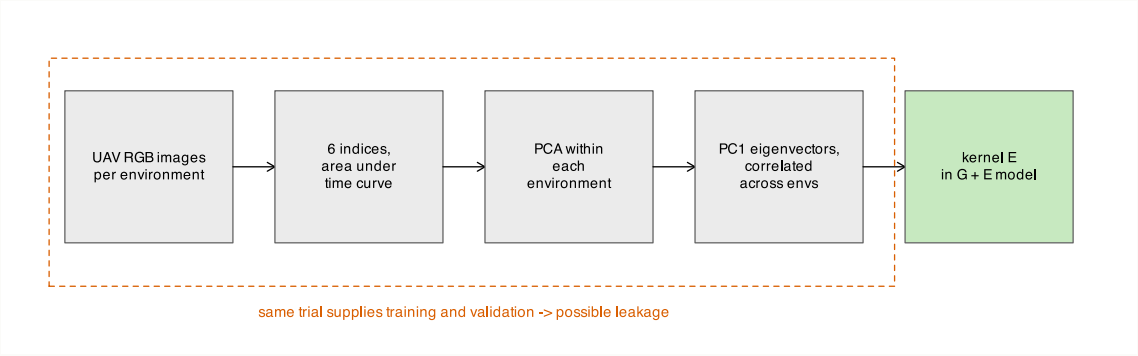

Figure 10.7: Building an environment kernel from images, after Hild Aono and Chawade (2026). The dashed box marks where training and validation share information – the leakage the authors flag.

The approach is attractive for programmes without routine genotyping. It is also promising for maize, where sensor phenotyping exists but has not been tested in this arrangement with hybrids. It connects to envirotyping. When locations are close together, coarse-grid weather covariates discriminate poorly, and images capture differences that the weather data miss [107], [108], [109]. NIRS-based phenomic prediction in maize points the same way, in a non-sparse setting [110].

Sensor data enter sparse testing as descriptors of environments, not of genotypes. This inversion solves the availability problem. In programmes that genotype little, it can partly replace the genomic matrix.

10.11 Simulation as a sizing tool

10.11.1 Why simulate before designing

None of the published numbers transfers directly to another programme. Each one depends on the between-environment correlations, the family structure, the $\sigma^2_{SCA}/\sigma^2_{GCA}$ ratio and the heritability of the programme that produced it, and the authors say so themselves [35], [69]. A simulation can do three things an experiment cannot:

- it knows the true values;
- it can rerun the programme a hundred times;
- it can isolate one factor at a time.

The tools exist. AlphaSimR covers the genetics , Chapter 7., FieldSimR covers plots and G x E [111], and `sommer` or BGLR fit the models [112]. Staged-allocation theory provides the deterministic cross-check [49], [66]. **Simulation does not replace the experiment. It replaces guessing at the design of the experiment.**

10.11.2 Realistic parameters: what the literature supplies

```
knitr::kable(lp[, c("parameter", "value", "source", "derived")],
             col.names = c("Parameter", "Value", "Source", "Derived"))
```

Table 10.4: Reference parameters from the six primary studies. 'Derived' marks a value computed from reported components rather than reported directly.

Parameter	Value	Source	Derived
Plot heritability	0.10 (founder population)	Werner et al. 2025 (simulation)	no
Plot heritability (worst environment)	0.18	Atanda et al. 2022 (wheat)	no
Broad-sense H2 over all hybrids	0.30	Gonzalez-Dieguez et al. 2026 (simulation)	no
Entry-mean H2	0.76-0.87	Li et al. 2026 (maize)	no
Entry-mean H2	0.61-0.87	Lubanga et al. 2025 (cassava)	no
Broad-sense H2 (maximum)	~0.67	Hild Aono & Chawade 2026 (potato)	no
s2SCA / s2GCA at cycle 1	0.05 / 0.23 / 0.50 for mean dominance 0.1 / 0.5 / 0.9	Gonzalez-Dieguez et al. 2026	no
s2GxE / (s2G + s2GxE)	0.56	Werner et al. 2025	no
Crossover share of G x E variance	0.53	Werner et al. 2025	no
Mean genetic correlation between TPE locations	0.60	Werner et al. 2025	no
Genetic correlation between locations (plant height)	0.36-0.67	Li et al. 2026 (maize)	no
Genetic correlation between environments (grain yield)	none strong detected	Li et al. 2026 (maize)	no
Genetic correlation between environments (same year)	−0.04 to 0.67	Atanda et al. 2022 (wheat)	no
s2_env / s2_g (grain yield)	8.1	Li et al. 2026 (maize; 1.13 / 0.14)	yes
Genotypes tested	264-2000	all six studies	no
Locations	2-440	all six studies	no
Testers	1-1200	Gonzalez-Dieguez et al. 2026	no
Cells observed	10-83%	all six studies	no

Two features of real data a simulation must respect:

1. **Environmental variance exceeds genetic variance by an order of magnitude.** For maize grain yield, $\sigma^2_{env}/\sigma^2_g \approx 8.1$ [68].
2. **Between-environment correlations for yield are often weak.** Scenarios should be anchored on measured parameters, not chosen for numerical convenience.

10.11.3 What the simulation finds

The model. The study simulated a female x male factorial of single crosses, tested with **one plot per hybrid x location cell**. The phenotype is

$$y = \mu + \text{loc} + \text{GCA}^f + \text{GCA}^m + \text{SCA} + \text{GCA}^f \times \text{loc} + \text{GCA}^m \times \text{loc} + \text{SCA} \times \text{loc} + e. \quad (10.4)$$

The target is the value in the TPE, $\text{GCA}^f + \text{GCA}^m + \text{SCA}$. In every replicate, **20 % of the hybrids get no plot at all.** These are the CV1 hybrids, predicted only through relationships.

The calibration is `sparse_vc()`:

- $\sigma^2_f = \sigma^2_m = 1$ and $\sigma^2_{SCA} = 0.25$;
- every interaction with location is half its main effect;
- plot h^2 of 0.20 or 0.40.

These choices imply a genetic correlation between locations of $V_G/(V_G + V_{GE}) = 0.667$. That sits inside the 0.36 to 0.67 measured in maize [68] and close to the 0.60 of C. R. Werner, G. Covarrubias-Pazaran, D. Gonzalez-Dieguez, D. C. Gemenet, H. Gandhi, and G. N. Atlin [69]. The SCA:GCA ratio of 0.25 lies among the values in D. González-Diéguez, G. Atlin, Y. Beyene, D. Wegary, D. Gemenet, and C. Werner [35], and the two heritabilities bracket C. R. Werner, G. Covarrubias-Pazaran, D. Gonzalez-Dieguez, D. C. Gemenet, H. Gandhi, and G. N. Atlin [69]'s 0.10 and the 0.30 of D. González-Diéguez, G. Atlin, Y. Beyene, D. Wegary, D. Gemenet, and C. Werner [35].

The four strategies, all at the same cost of k locations per hybrid:

- *complete*;
- *random*: each hybrid goes to k locations drawn at random;
- *optimised blocks*: an exchange algorithm on the mixed-model coefficient matrix, with replicated checks;
- *parentage*: the factorial is cut into cyclic groups, each holding every female and every male, and groups are dealt to locations so that **every location receives both whole pools**.

The last is `sparse_alloc(strategy = "parentage")`.

Validation of the estimator. Before any comparison, the estimator was checked on the complete design, where REML must recover the true components (Table 10.5).

```
v <- val; v$component <- sub("s2", "s2_", v$component)
knitr::kable(fmt(v[, c("h2_plot", "component", "true", "mean_est", "relbias_pct",
"se_relbias_pct", "t")], 3),
             col.names = c("Plot h2", "Component", "True", "Mean estimate", "Rel.
bias %", "SE %", "t"))
```

Table 10.5: Component recovery by REML in the COMPLETE design (20 x 20 hybrids, 6 locations, 30 replicates per heritability; cached). The residual target is $\sigma^2_{SCA\times L} + \sigma^2_e$, because with one plot per cell the two are confounded.

Plot h2	Component	True	Mean estimate	Rel. bias %	SE %	t
0.2	s2_f	1.000	1.026	2.63	7.88	0.33
0.2	s2_m	1.000	0.964	−3.57	6.70	−0.53
0.2	s2_s	0.250	0.350	39.94	18.50	2.16
0.2	s2_fl	0.500	0.540	7.97	6.13	1.30
0.2	s2_ml	0.500	0.499	−0.13	6.04	−0.02
0.2	s2_res	8.000	7.962	−0.48	0.53	−0.90
0.4	s2_f	1.000	0.979	−2.15	6.07	−0.35
0.4	s2_m	1.000	1.008	0.79	6.77	0.12
0.4	s2_s	0.250	0.303	21.31	20.91	1.02
0.4	s2_fl	0.500	0.488	−2.31	3.16	−0.73
0.4	s2_ml	0.500	0.496	−0.70	3.77	−0.19
0.4	s2_res	2.375	2.364	−0.44	0.73	−0.61

Every component except SCA is recovered without detectable bias ($|t| \leq 1.3$). σ^2_{SCA} is biased upward even here, by +39.9 % and +21.3 %. That is the expected direction for a small component (11 % of V_G) estimated by a procedure bounded below at zero. **The bias is a baseline, not an effect of sparsity.** Every later SCA number has to be read against it.

The grid has 2,630 fits:

- 100 and 400 hybrids;
- 2, 4, 6 and 10 locations;
- nominal sparsity from 20 % to 100 %;
- two heritabilities;
- 15 replicates per cell.

10.11.3.1 1. The cutting margin is wide, and wider at higher heritability

```
op <- par(mfrow = c(1, 2), mar = c(4.2, 4.2, 2, 1))
d <- subset(s4, h2_plot == 0.4 & strategy %in% c("parentage", "complete"))
Ls <- sort(unique(d$nLoc)); lc <- hcl.colors(length(Ls), "Dark 3")
plot(NA, xlim = range(d$n_plots_mean), ylim = range(d$acc_all_mean),
     xlab = "Plots", ylab = "Accuracy, all hybrids", main = "Cost-accuracy, h2 =
0.4", cex.main = 0.9)
for (i in seq_along(Ls)) {
  e <- d[d$nLoc == Ls[i], ]; e <- e[order(e$n_plots_mean), ]
  lines(e$n_plots_mean, e$acc_all_mean, type = "b", pch = ifelse(e$strategy ==
"complete", 1, 19), col = lc[i])
}
legend("bottomright", bty = "n", col = lc, lwd = 2, legend = paste(Ls, "locations"),
cex = 0.8)
sp <- subset(s4, k == 2 & strategy %in% c("complete", "random"))
plot(NA, xlim = c(2, 10), ylim = range(sp$acc_all_mean), xlab = "Locations sharing
the 2 plots per hybrid",
     ylab = "Accuracy, all hybrids", main = "Same 800 plots, spread wider", cex.main
= 0.9)
for (h in c(0.2, 0.4)) {
  e <- sp[sp$h2_plot == h, ]; e <- e[order(e$nLoc), ]
  lines(e$nLoc, e$acc_all_mean, type = "b", pch = 19, lty = ifelse(h == 0.2, 2, 1))
}
legend("bottomright", bty = "n", lty = c(1, 2), pch = 19, legend = c("plot h2 = 0.4",
"plot h2 = 0.2"), cex = 0.8)
par(op)
```

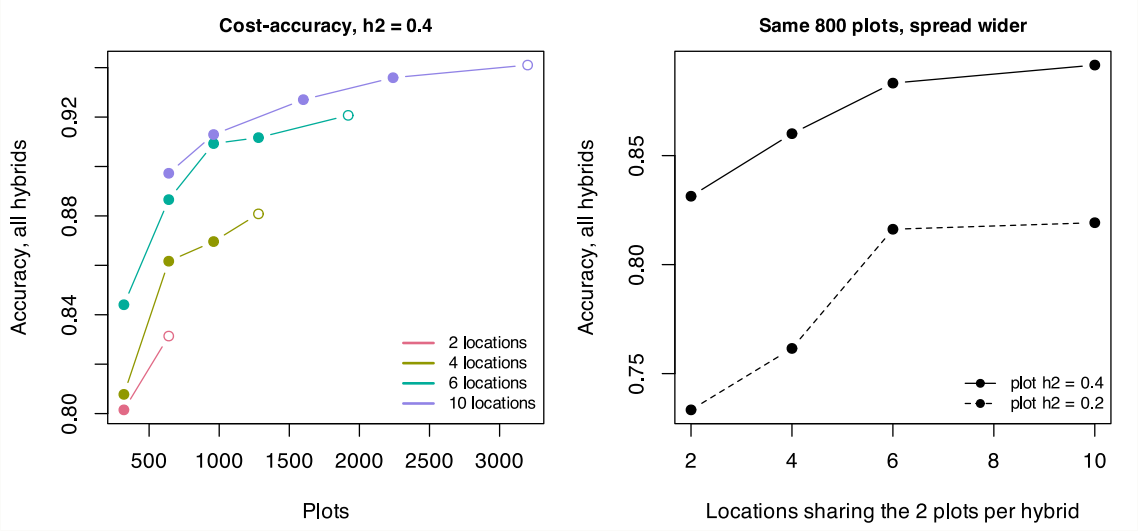

Figure 10.8: Left: accuracy against plots for 400 hybrids at plot h2 = 0.4, one curve per number of locations (parentage and complete designs; complete marked with an open circle). Right: accuracy at a FIXED cost of two plots per hybrid, spread over 2 to 10 locations, random allocation, both heritabilities. Cached study, REML components, 15 replicates.

Take 400 hybrids at $h^2 = 0.4$. The complete design over ten locations (3,200 plots) reaches an accuracy of 0.941. The **cheapest allocation within 5 % of that** is the parentage design with 2 locations per hybrid. It uses 640 plots, 20 % of the budget, and reaches 0.897, which is 95.3 % of the reference. Coincidence with the true top 5 % falls only from 70.0 % to 65.7 %. At $h^2 = 0.2$ the requirement roughly doubles to half the cells. Table 10.6 lists every scenario.

```
m <- mn; m$within <- ifelse(m$minimal_5, "5 %", "10 %")
m <- m[order(m$nHyb, m$h2_plot, m$sca_level, m$nLoc, m$within), ]
knitr::kable(fmt(m[, c("nHyb", "h2_plot", "sca_level", "nLoc", "within", "strategy",
"k",
                       "n_plots_mean", "acc_all_mean", "acc_rel", "cost_rel")], 3),
             row.names = FALSE,
                col.names = c("Hybrids", "Plot h2", "SCA", "Locations", "Within",
"Strategy", "k",
                              "Plots", "Accuracy", "Rel. accuracy", "Rel. cost"))
```

Table 10.6: Cheapest allocation within 5 % and within 10 % of the accuracy of the complete design at the largest number of locations in each scenario. Cached study.

Hybrids	Plot h2	SCA	Locations	Within	Strategy	k	Plots	Accuracy	Rel. accuracy	Rel. cost
100	0.2	low	4	10 %	parentage	3	240.000	0.779	0.903	0.3
100	0.2	low	4	5 %	complete	4	320.000	0.825	0.958	0.4
100	0.4	low	6	5 %	random	4	320.000	0.887	0.951	0.4
100	0.4	low	10	10 %	parentage	2	160.000	0.856	0.917	0.2
400	0.2	low	6	10 %	random	2	640.000	0.816	0.904	0.2
400	0.2	low	10	5 %	random	5	1600.000	0.874	0.967	0.5
400	0.4	high	10	10 %	random	2	640.000	0.825	0.901	0.2
400	0.4	high	10	5 %	optimised_blocks	5	1599.867	0.884	0.966	0.5
400	0.4	low	4	10 %	optimised_blocks	2	639.667	0.859	0.912	0.2
400	0.4	low	10	5 %	parentage	2	640.000	0.897	0.953	0.2

10.11.3.2 2. At fixed cost, spreading plots over locations beats concentrating them

This is the most actionable result (Figure 10.8, right). Each hybrid keeps **two plots**, the cost of a complete two-location trial. At $h^2 = 0.4$, accuracy rises from 0.831 with both plots in the same two locations to 0.860, 0.883 and 0.892 as those plots are spread across 4, 6 and 10 locations. At $h^2 = 0.2$ it rises from 0.733 to 0.819. Not one extra plot is spent. The gain saturates between six and ten locations, so the practical advice is to add locations up to about six before adding plots per hybrid. This is the replication-against-breadth trade-off of A. J. Lorenz [64], G. N. Atlin and B. F. Econopouly [72] and C. R. Werner, G. Covarrubias-Pazaran, D. Gonzalez-Dieguez, D. C. Gemenet, H. Gandhi, and G. N. Atlin [69], now measured for hybrids with GCA and SCA.

10.11.3.3 3. Among sparse designs the choice matters little, except at extreme sparsity

```
cc <- ct[ct$metric %in% c("acc_all", "acc_cv1", "acc_cv2") & ct$subset %in% c("all",
"k<=2"), ]
knitr::kable(fmt(cc[, c("layer", "contrast", "metric", "subset", "n", "diff", "se",
"p")], 4),
             row.names = FALSE,
           col.names = c("Layer", "Contrast", "Metric", "Subset", "n", "Difference",
"SE", "p"))
```

Table 10.7: Contrasts against random allocation. Paired by replicate in the fast layer (791 pairs; 385 with k <= 2) and in the genomic layer (32 pairs). acc_cv1: hybrids never tested; acc_cv2: missing cells of tested hybrids. Cached study.

Layer	Contrast	Metric	Subset	n	Difference	SE	p
fast	parentage - random	acc_all	all	791	0.0051	0.0029	0.0773
fast	optimised_blocks - random	acc_all	all	791	0.0011	0.0028	0.7018
fast	parentage - random	acc_cv1	all	791	0.0086	0.0039	0.0260
fast	optimised_blocks - random	acc_cv1	all	791	0.0018	0.0043	0.6698
fast	parentage - random	acc_cv2	all	791	0.0077	0.0026	0.0031
fast	optimised_blocks - random	acc_cv2	all	791	0.0060	0.0023	0.0103
genomic	parentage - random	acc_all	all	32	0.0226	0.0086	0.0137
genomic	optimised_blocks - random	acc_all	all	32	0.0152	0.0098	0.1296
genomic	parentage - random	acc_cv1	all	32	0.0139	0.0092	0.1410
genomic	optimised_blocks - random	acc_cv1	all	32	0.0111	0.0135	0.4197
fast	parentage - random	acc_cv1	k<=2	385	0.0187	0.0069	0.0070
fast	optimised_blocks - random	acc_cv1	k<=2	385	−0.0012	0.0079	0.8810

Over the whole grid, the parentage design beats random allocation on untested hybrids by +0.0086 $\pm$ 0.0039 (p = 0.026). That is real and small. Restricted to $k \leq 2$ locations per hybrid, the advantage roughly doubles, to +0.0187 $\pm$ 0.0069 (p = 0.007). The optimised blocks show nothing there (-0.0012 $\pm$ 0.0079). In the genomic layer, with marker relationships and real dominance, the parentage advantage on overall accuracy is +0.023 $\pm$ 0.009 (p = 0.014).

None of the six primary studies compared a structured allocation with a random one. Y. Li *et al.* [68] found their structured schemes tied with each other. C. R. Werner, G. Covarrubias-Pazaran, D. Gonzalez-Dieguez, D. C. Gemenet, H. Gandhi, and G. N. Atlin [69] used restricted randomisation without a random control. This result answers that open question for the hybrid case. The structure matters when plots are scarce, and only for the hybrids the field never saw.

```
a <- aggregate(acc_all ~ strategy + nLoc + p_real, gl, mean)
op <- par(mar = c(4.2, 4.2, 1, 1))
plot(NA, xlim = range(a$p_real), ylim = range(a$acc_all), xlab = "Share of cells
observed",
     ylab = "Accuracy, all hybrids")
for (s in names(strat_col)) for (L in unique(a$nLoc)) {
```

```
  e <- a[a$strategy == s & a$nLoc == L, ]; e <- e[order(e$p_real), ]
  if (nrow(e)) lines(e$p_real, e$acc_all, type = "b", pch = ifelse(L == 4, 19, 17),
                     lty = ifelse(L == 4, 1, 2), col = strat_col[s])
}
legend("bottomright", bty = "n", cex = 0.8, col = c(strat_col, "black", "black"),
       lty = c(1, 1, 1, 1, 1, 2), pch = c(NA, NA, NA, NA, 19, 17),
       legend = c(names(strat_col), "4 locations", "8 locations"))
par(op)
```

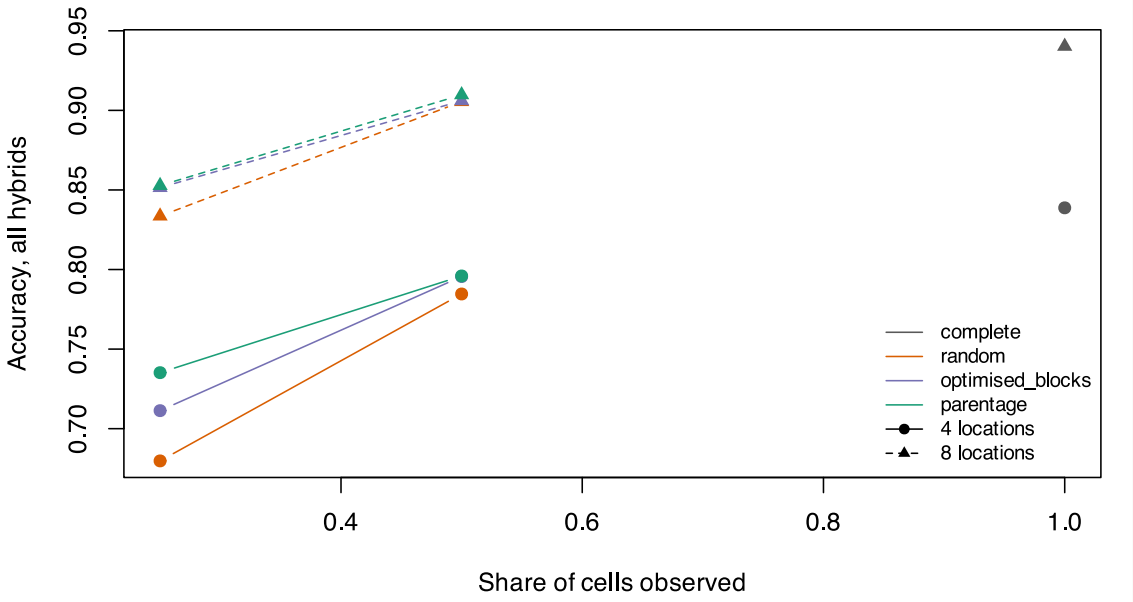

Figure 10.9: Genomic layer (AlphaSimR genomes, marker relationships, 8 replicates): accuracy against the share of cells observed, by strategy and number of locations. Cached study.

10.11.3.4 4. Parent coverage, not shared hybrids, carries the connectivity

Consider ten locations with two per hybrid at $h^2 = 0.4$. The best design there is the parentage design, with accuracy 0.897. Its harmonic-mean concurrence, the number of hybrids a pair of locations shares, is **0.000**: some pairs of locations share no hybrid at all. This is structural. Twenty groups at two locations each touch at most 20 of the 45 location pairs. Random allocation shares hybrids between almost every pair (harmonic mean 5.466) and is less accurate. What the parentage design keeps is **parent coverage**. The mean fraction of females present per location is 0.999, against 0.965 for random allocation.

Once a relationship matrix is in the model, what carries connectivity is representing both pools in every location, not repeating the same hybrids across locations. Put this together with Section 10.6 and it reads as one account of the S. A. Atanda, V. Govindan, R. Singh, K. R. Robbins, J. Crossa, and A. R. Bentley [44] versus N. Lubanga, B. E. Ifie, R. Persa, I. Dieng, I. Y. Rabbi, and D. Jarquin [70] disagreement. Shared hybrids are one way to connect environments. Shared parents are another, cheaper one, and in a hybrid factorial the shared parents come for free.

10.11.3.5 5. The BLUP is not dispersed; the SCA variance is not recoverable

```
med  <-  aggregate(cbind(relbias_s2f_mean,  relbias_s2fl_mean,  relbias_s2s_mean,
relbias_h2_mean) ~ k, sc, median)
op <- par(mfrow = c(1, 2), mar = c(4.2, 4.2, 1, 1))
matplot(med$k, 100 * med[, -1], type = "b", pch = 19, lty = 1,
        col = c("#1b9e77", "#7570b3", "#d95f02", "grey30"),
        xlab = "Locations per hybrid, k", ylab = "Median relative bias (%)")
abline(h = 0, lty = 3)
legend("topright", bty = "n", cex = 0.8, pch = 19, col = c("#1b9e77", "#7570b3",
```

```
"#d95f02", "grey30"),
       legend = c("GCA (female)", "GCA x location", "SCA", "plot h2"))
plot(jitter(sc$k, 0.4), pmin(pmax(sc$bias_all_mean, 0.5), 1.5), pch = 19, cex = 0.6,
     col = adjustcolor(ifelse(sc$frac_conv < 1, "#d95f02", "grey30"), 0.7),
    xlab = "Locations per hybrid, k", ylab = "Dispersion slope b1", ylim = c(0.5, 1.5))
abline(h = 1, lty = 2)
legend("topright", bty = "n", cex = 0.8, pch = 19, col = c("grey30", "#d95f02"),
       legend = c("all fits converged", "some fits did not"))
par(op)
```

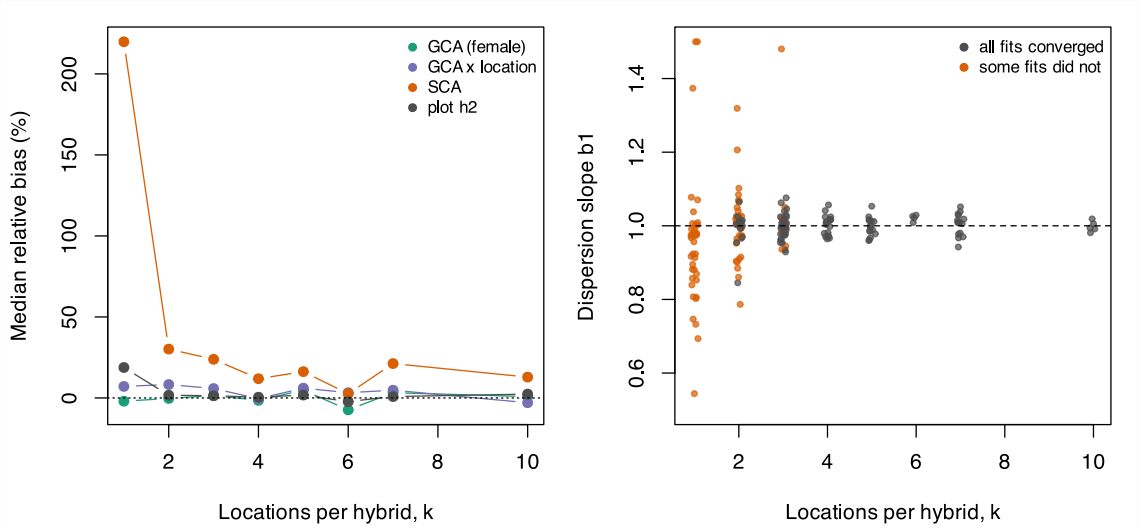

Figure 10.10: Left: median relative bias of REML components and of plot h2 by locations per hybrid k, over all cached scenarios. Right: the dispersion slope b1 (truth on BLUP), one point per scenario; the dashed line is no dispersion bias. Scenarios with k = 1 and incomplete REML convergence produce the outliers (axis clipped at 0.5-1.5).

Prediction holds up. Where every REML fit converged, the dispersion slope stays within 0.845 to 1.076 across the grid. With $k \geq 4$ it stays within 0.942 to 1.057. Sparsity costs accuracy but adds **no dispersion bias**, as long as the components are estimable. The GCA variance, the GCA x location variance and plot h^2 have median biases near zero at every k.

σ^2_{SCA} is the exception. Its median relative bias is +12 % at $k = 4$, +30 % at $k = 2$, and +220 % at $k = 1$. Much of this is present in the complete design too (Table 10.5), so it is mainly a problem of identifying a small component, and sparsity makes it worse. A programme that needs a reliable SCA is in product development, not population improvement. It cannot fix the problem by adding coverage to a stage-1 trial.

The hidden cost is convergence. REML converged in 100.0 % of fits with four locations per hybrid, 97.0 % with three, 88.5 % with two, and only **42.6 % with one**. That is 411 non-convergences in 2630 fits.

! Box – Why REML does not converge when each hybrid sees one location

With $k = 1$ each hybrid contributes a single plot in a single location. The data then carry no within-hybrid information to separate a hybrid's GCA from its GCA x location deviation. The likelihood is flat along that ridge. The algorithm walks to the boundary of the parameter space and stops without meeting its criterion. The BLUPs from those fits are still usable. They come from solving the mixed-model equations at the current components, which is exactly what `sparse_blup()` does with known ones. **The variance estimates are not usable, and should not be reported.** Predicting well and estimating well are different achievements. At $k = 1$ a trial can still do the first.

10.11.3.6 6. Two stages: test every candidate sparsely rather than a subset densely

The two-stage scenario works as follows:

- A pool of 400 candidate hybrids is available.
- Stage 1 tests N_1 of them sparsely and keeps the top q by BLUP.
- Stage 2 tests the survivors densely in new locations.
- The final analysis is joint.

The metric is the realised gain: the mean *true* value of the ten hybrids finally chosen, in genetic standard deviations. Under 700 total plots, the gain is 1.358 when stage 1 tests 100 candidates, 1.778 for 200, and 2.148 for all 400. Across all budgets, efficiency relative to an oracle that selects on the truth rises from 0.585 to 0.935. The share of the true top 5 % captured rises from 19.8 % to 72.1 %.

```
a <- aggregate(cbind(gain_sd_units, gain_max_sd_units) ~ N1 + plots_total, ts, mean)
cl <- c("100" = "#d95f02", "200" = "#7570b3", "400" = "#1b9e77")
op <- par(mar = c(4.2, 4.2, 1, 1))
plot(NA, xlim = range(a$plots_total), ylim = c(0, max(a$gain_max_sd_units)), log
= "x",
     xlab = "Total plots (log scale)", ylab = "Realised gain (genetic SD)")
o <- aggregate(gain_max_sd_units ~ plots_total, a, mean); o <-
o[order(o$plots_total), ]
lines(o$plots_total, o$gain_max_sd_units, col = "grey60", lwd = 3)
for (n in names(cl)) {
  e <- a[a$N1 == as.numeric(n), ]; e <- e[order(e$plots_total), ]
  lines(e$plots_total, e$gain_sd_units, type = "b", pch = 19, col = cl[n])
}
legend("bottomright", bty = "n", col = c(cl, "grey60"), lwd = c(1, 1, 1, 3), pch =
c(19, 19, 19, NA),
       legend = c(paste("N1 =", names(cl)), "oracle"), cex = 0.85)
par(op)
```

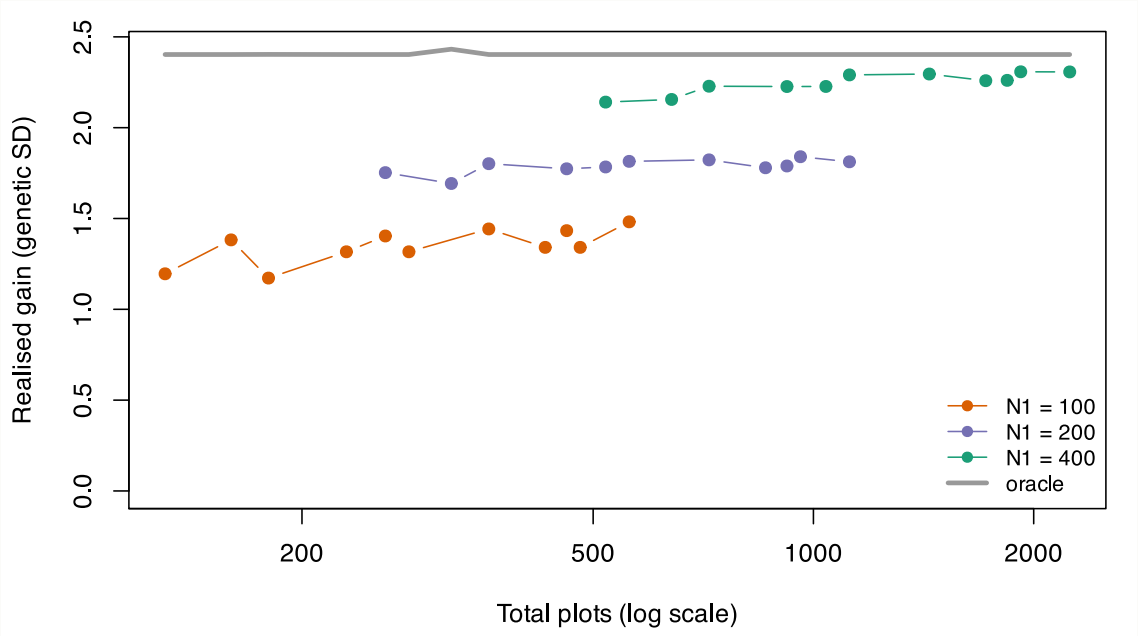

Figure 10.11: Realised gain (true value of the 10 hybrids finally selected, genetic SD units) against total plots, by the number of candidates tested in stage 1; mean over replicates and stage-2 settings at each budget. The grey line is the oracle that selects on the truth. Cached study.

With a fixed budget, **stage 1 should cover every candidate with few plots each, not a subset with many.** The best combinations among the cached settings used four stage-1 locations at 50 % sparsity, eight stage-2 locations, and kept the top 10 to 20 %.

10.11.3.7 Limitations of the simulation

The simulation has six limitations.

1. **One plot per cell.** $\sigma^2_{SCA\times L}$ is confounded with plot error, and the residual estimates their sum. Within-location replication was not simulated.
2. **G x E is full-rank and symmetric in the fast layer, and rank one in the genomic layer.** Neither layer has the factor-analytic structure of real trials or a crossover/non-crossover split [69], [85].
3. **The pools are symmetric and unselected.** There are no cycles of reciprocal recurrent selection, so the simulation says nothing about long-run pool trajectories (Chapter 22.) or about unbalanced pools.
4. **Cost is counted in plots, not money.** Opening a location costs more than adding plots to an existing one. The advice to spread plots assumes the marginal location is moderately priced (Section 9.7 has the cost structure).
5. **The grid is reduced.** It has 15 replicates, no 200-hybrid level, and the high-SCA block run separately. The two-stage layer has 4 replicates and the genomic layer 8.
6. **Components are checked by mean bias only.** No per-replicate confidence intervals were computed.

10.12 Practical recommendations by stage

Stage 1 (first agronomic testcross trial).

- Give priority to the number of candidates and the number of locations over replication. Keep the plots per hybrid fixed and widen the locations, up to about six (result 2).
- Use 3 to 5 testers in sparse testcrossing, with full-sibs spread across testers.
- Use low partial replication (p-rep of 0.1 to 0.2), with checks replicated in every block.
- When plots per hybrid fall to one or two, use a layout that puts both pools in every location (results 3 and 4).
- The estimation target is GCA. Do not expect a reliable SCA at this stage (result 5).

Stage 2 (pre-commercial hybrids).

- Test fewer candidates, and raise replication and environmental coverage.
- Keep a common core large enough to estimate the between-environment genetic correlations.
- The estimation target is now specific performance and stability, so SCA and G x E matter.

Decisions that span both stages.

- Genotype the parents, not the hybrids.
- Check connectivity before planting (Box 4).
- Match the between-environment covariance to the amount of data. Do not force an unstructured matrix on a very sparse design.
- Advance on GBLUP rather than BLUE when heritability is low to moderate and locations are few [44], [48].

What not to do.

- Put everything in one location, however many candidates that allows. It gave the most variable gain in C. R. Werner, G. Covarrubias-Pazaran, D. Gonzalez-Dieguez, D. C. Gemenet, H. Gandhi, and G. N. Atlin [69], with replicates of negative gain.
- Use one tester at stage 1 when there is infrastructure for three.
- Transfer a sparsity percentage from another crop or programme without your own simulation.
- Fit a G x E term under extreme sparsity [71].

The right sparse design changes from stage to stage because the parameter of interest changes: GCA in stage 1, specific performance and stability in stage 2.

Box 6 – A checklist for designing a sparse trial

1. Define the TPE and the market segment.
2. Estimate the genetic correlations between candidate locations from historical data.
3. Fix the budget in plots.
4. Choose the axis of sparsity: environments, testers, or both.
5. Fix the number of testers.
6. Fix the partial-replication rate.
7. Choose the checks and how often they appear.
8. Generate the allocation by restricted randomisation or a formal criterion. Where plots per hybrid are fewest, keep both pools in every location.
9. Compute connectivity and the expected PEV before planting (Box 4).
10. Simulate the design with the programme's own parameters (`sparse_vc()`, `sparse_simulate()`, `sparse_blup()`).
11. Fix the analysis model and the covariance structure *a priori*.
12. Fix the decision metric and the cut-off *a priori*.

10.13 Limitations of the evidence, and perspectives

The authors of the primary studies declare their own limitations:

- Y. Li *et al.* [68]: a narrow genetic base and a single year.
- S. A. Atanda, V. Govindan, R. Singh, K. R. Robbins, J. Crossa, and A. R. Bentley [44]: no way to identify the optimal overlap, and families of 1 to 9.
- C. R. Werner, G. Covarrubias-Pazaran, D. Gonzalez-Dieguez, D. C. Gemenet, H. Gandhi, and G. N. Atlin [69]: no cost model, and no hybrid structure.
- N. Lubanga, B. E. Ifie, R. Persa, I. Dieng, I. Y. Rabbi, and D. Jarquin [70]: a training set that was not optimised.
- D. González-Diéguez, G. Atlin, Y. Beyene, D. Wegary, D. Gemenet, and C. Werner [35]: no G x E, no epistasis, no diversity management, and the logistic cost ignored.
- A. Hild Aono and A. Chawade [71]: possible information leakage, and very similar environments.

As a research agenda, the open gaps are these.

- **SCA estimation, and prediction of specific hybrids, under sparsity.** This chapter's simulation shows the variance component is not recoverable at stage
  1. The accuracy of specific unseen crosses is still unmeasured.
- **A three-dimensional connectivity metric**, female x male x location. It would join Section 9.2.3 to Box 4. Chapter 11. defines one.
- **The long-run effect on pool divergence and within-pool variance**, in interaction with optimum contribution selection (Chapter 22.; Chapter 18.; N. Santantonio and K. Robbins [104]).
- **Phenomics-assisted sparse testing in maize hybrids.**
- **Pools unequal in size and breeding level**, which is the usual commercial case.
- **Cost-accuracy frontiers in money**, including isolation blocks and hand pollination.
- **The rate of wrongly discarded candidates** as the evaluation metric, in place of the correlation.
- **Prospective empirical validation.** Almost all the evidence for sparse testing comes from resampling complete data or from simulation, not from trials designed sparse from the start [113], [114].

Sparse testing changes the stage-1 question. It is no longer "which genotypes can I evaluate well?". It becomes "which haplotypes can I evaluate well, and against what sample of the opposite pool?". The evidence is sufficient to adopt sparse designs at stage 1 with confidence. It is not sufficient to prescribe a universal sparsity level. That decision remains specific to each programme, and should be sized by the programme's own simulation.

10.14 Reproducibility and assumptions

`regenerate.R sparse` reruns the phenotypic core of the cached study with `R/17_sparse_testing.R`. The scale and calibration are the same: 20 x 20 hybrids, half-sib families of four, and 20 % of hybrids untested. There are two differences:

- the variance components are **known**, not estimated by REML;
- the parentage allocation's exchange search minimises the number of disconnected location pairs before the harmonic concurrence. The cached study's criterion returned $-\infty$ for any disconnected start, which left its search no gradient.

```
cr <- subset(core, part %in% c("spread", "k1"))
cr$cached <- mapply(function(L, k, s, h) {
  x <- s4$acc_all_mean[s4$nLoc == L & s4$k == k & s4$strategy == s & s4$h2_plot == h]
  if (length(x)) x[1] else NA
}, cr$nLoc, cr$k, cr$strategy, cr$h2_plot)
cr$cached_se <- mapply(function(L, k, s, h) {
  x <- s4$acc_all_se[s4$nLoc == L & s4$k == k & s4$strategy == s & s4$h2_plot == h]
  if (length(x)) x[1] else NA
}, cr$nLoc, cr$k, cr$strategy, cr$h2_plot)
cr$z <- (cr$acc_all - cr$cached) / sqrt(cr$acc_all_se^2 + cr$cached_se^2)
cr <- cr[order(cr$h2_plot, cr$k, cr$nLoc, cr$strategy), ]
knitr::kable(fmt(cr[, c("h2_plot", "nLoc", "k", "strategy", "acc_all", "cached", "z",
"b1")], 3),
             row.names = FALSE,
             col.names = c("Plot h2", "Locations", "k", "Strategy", "In-repo acc.",
"Cached acc.",
                           "z", "In-repo b1"))
dk <- tapply(cr$acc_all - cr$cached, cr$k, mean, na.rm = TRUE)
```

Table 10.8: In-repo sweep (known components, 30 replicates) against the cached study (REML, 15 replicates), on overall accuracy for 400 hybrids. `z` is the difference in standard errors of the difference.

Plot h2	Locations	k	Strategy	In-repo acc.	Cached acc.	z	In-repo b1
0.2	4	1	parentage	0.711	0.692	0.690	0.957
0.2	4	1	random	0.726	0.702	1.075	0.962
0.2	6	1	parentage	0.760	0.720	1.911	1.031
0.2	6	1	random	0.728	0.727	0.036	0.976
0.2	10	1	parentage	0.749	NA	NA	0.953
0.2	10	1	random	0.742	NA	NA	0.990
0.2	2	2	complete	0.773	0.733	1.741	1.019
0.2	4	2	parentage	0.814	0.781	1.821	1.015
0.2	4	2	random	0.822	0.762	4.064	1.009
0.2	6	2	parentage	0.816	0.811	0.297	0.999
0.2	6	2	random	0.807	0.816	−0.592	1.006
0.2	10	2	parentage	0.823	0.820	0.221	0.982
0.2	10	2	random	0.810	0.819	−0.473	1.004
0.4	4	1	parentage	0.845	0.808	2.189	1.032
0.4	4	1	random	0.830	0.817	0.603	0.999
0.4	6	1	parentage	0.841	0.844	−0.256	0.976
0.4	6	1	random	0.845	0.836	0.627	0.994
0.4	10	1	parentage	0.858	NA	NA	1.023
0.4	10	1	random	0.865	NA	NA	0.986
0.4	2	2	complete	0.818	0.831	−1.229	1.003
0.4	4	2	parentage	0.870	0.862	0.744	1.027
0.4	4	2	random	0.872	0.860	0.959	1.014
0.4	6	2	parentage	0.866	0.887	−1.997	1.005
0.4	6	2	random	0.864	0.883	−1.827	0.990
0.4	10	2	parentage	0.891	0.897	−0.801	1.021
0.4	10	2	random	0.887	0.892	−0.558	0.999

At $k = 2$ the two sources agree within sampling error. The mean difference is +0.006, and the gain from spreading the same plots over more locations reproduces. At $k = 1$ the known-component values are higher by +0.017 on average. That is the cost of estimating the components where REML mostly fails to converge (result 5). The in-repo dispersion slopes all sit near 1. The in-repo sweep does **not** resolve the parentage-versus-random difference at $k = 1$: its 30 unpaired replicates per cell have standard errors near 0.01, the same size as the effect. The paired contrast of Table 10.7, on 385 pairs, is the evidence for result 3.

The live functions are verified in `tests/testthat/test-sparse.R`:

- `sparse_blup()` equals the literal GLS form of `ref_sparse_blup()` to 10^{-8}, including untested hybrids, and so do its PEVs.
- Every allocation puts each hybrid in exactly k locations.
- The parentage design covers both pools in every location.
- The dispersion slope of the known-component BLUP averages 1.

The cached numbers carry the assumptions listed in the limitations of the simulation (Section 10.11.3). Treat them as reproducible findings of that configuration, not as constants.

10.15 Where this connects

- Chapter 9. chose *which crosses* to make. This chapter chose *where to plant them*. The joint problem, sparse in crosses and in environments at once, is Section 9.9's second gap, and Chapter 11. treats it.

- The GCA/SCA decomposition and the tester-frequency argument behind Box 5 are derived in Section 13.10.
- The relationship matrices that make sparse designs analysable are those of

  3.
- The staged-trial trade-off of Chapter 8. is the line-breeding version of result 6.
- Spreading testers and parents to slow within-pool erosion is the same concern as Chapter 18. and Chapter 22.. Sparse testcrossing does by design what a diversity constraint does by optimisation.

Reproduce this chapter

All the code of this chapter, in order and with the explanation kept as comments, is one script: **04f-sparse-testing.R**. Download it, install `hybdiv`, and run it top to bottom in an empty directory; cached results are fetched on first use.

11. Which crosses, at which locations? Joint sparse allocation

Chapter 9. chose **which crosses** to make from two pools, with the field design held fixed. Chapter 10. chose **which locations** each hybrid goes to, with the whole factorial taken as given. A programme makes both decisions out of one budget, in the same season, and each changes what the other can do: a cross that is not made cannot be planted anywhere, and a cross planted in one location says nothing about the others except through its parents. This chapter treats the two as one decision.

Within the search reported in Section 11.3, the literature has not. It finds strong methods for each half – model-based designs that place entries in environments using genetic relatedness, and training-set criteria that choose which single crosses to phenotype – but no method that chooses the crosses and their locations together for a two-pool hybrid programme. That is an absence of evidence from one search, not a proof of absence. The nearest treatments either optimise the crosses and leave the environments to the trial manager [115], [116], or optimise the environments for a fixed entry list [117], [118]. Section 9.9 listed this as its second gap and Section 10.13 asked for the female x male x location connectivity metric that it needs; both are taken up here.

The results this chapter defends:

1. **At a fixed budget, distinct crosses beat locations per cross for predicting all hybrids.** Spending 160 plots as 160 crosses at one location each beat 80 at two, which beat 40 at four, in the direction of every scenario simulated; not every single step is resolved by 30 replicates, and the margin grows with SCA. For the crosses that were made the comparison reverses: 40 crosses at four locations estimate those better than 160 at one. The split of the budget is therefore a statement about what will be selected.
2. **Allocation strategy is a second-order decision.** With the plan and the locations per cross fixed, a placement that keeps as much of both pools as possible in each location raises parent coverage substantially, but its accuracy gain over random placement of the same plan is not detected: half a point at most on the point estimate, at 1.7 standard errors, and none in the tester design.
3. **The PEV criterion ranks plans, not placements.** The mean PEV of GCA, computable before planting, orders the budget splits much as accuracy does. An exchange search on it, run for 600 proposals, lowered PEV below its own greedy start by a fraction of a percent and bought no accuracy overall (the tester design is a tentative exception). The search had not converged, so this bounds what 600 proposals buy, not what placement optimisation can.

> **i** Where the numbers come from
>
> Everything here is reproducible from this repository.
>
> - **Live** results run at render time with `R/18_joint_allocation.R`, which builds the allocation matrix of `R/17_sparse_testing.R` with zero rows for crosses that are not made, so the R/17 simulator, mixed-model solve and connectivity diagnostics take it unchanged.
> - **In-repo** results come from `Rscript book/scripts/regenerate.R joint` (`joint_core_sweep.csv`, `joint_contrasts.csv`, `joint_optimiser.csv`; about 30 minutes of CPU): 20 x 20 pools in half-sib families of four, eight locations, **known** variance components, 30 replicates per cell. Strategy comparisons are **paired**: within a replicate the designs share the plan and the true values.
>
> There is no cached external study behind this chapter, unlike Chapter 9. and Chapter 10..

11.1 Why the two decisions are one

Write n_c for the number of distinct crosses made, k for the number of locations each cross is planted in, and r for plots per cross per location. The field budget is

$$B = n_c \, k \, r. \tag{11.1}$$

Chapter 9. varied n_c (through the crosses per line c, $n_c = c\, n_F$ for a circulant plan) at fixed k; Chapter 10. varied k and the layout at fixed $n_c = n_F n_M$. At $r = 1$, which is the stage-1 norm, fixing B ties the two together: halving the crosses doubles the locations each one can see. Neither chapter could ask which way to spend the budget, because each held the other term fixed.

The two terms buy different things. More crosses put more lines into data and sample more of the factorial, which is what predicts the crosses never made (Section 9.6; A. Melchinger, R. Fernando, C. Stricker, C. Schön, and H. Auinger [41]). More locations per cross sample more of the target population of environments, which is what shrinks the genotype x location noise around each GCA (Section 10.11.3, result 2; C. R. Werner, G. Covarrubias-Pazaran, D. Gonzalez-Dieguez, D. C. Gemenet, H. Gandhi, and G. N. Atlin [69]). Which matters more depends on the ratio of SCA to GCA variance, on the share of genotype x location in the genetic variance, and on how well the relationship matrix links lines across families – precisely the quantities a programme knows least about at stage 1.

11.2 The joint model

The model is Chapter 10.'s, unchanged:

$$y_{ijl} = \mu + \ell_l + g_i^F + g_j^M + s_{ij} + (g^F \ell)_{il} + (g^M \ell)_{jl} + (s\ell)_{ijl} + e_{ijl}, \tag{11.2}$$

with $g^F \sim N(0, A_F \sigma_F^2)$, $g^M \sim N(0, A_M \sigma_M^2)$, $s \sim N(0, (A_F \otimes A_M)\sigma_S^2)$ and the interactions with location carrying the same kernels crossed with an identity over locations. The target is the value in the target population of environments, $g_i^F + g_j^M + s_{ij}$.

One simplification is inherited from Chapter 10. and must be declared. With one plot per cross per location, $(s\ell)_{ijl}$ and e_{ijl} have the same incidence, so the analysis model cannot separate them and carries their sum as an i.i.d. residual of variance $\sigma^2_{S\ell} + \sigma^2_e$. The simulator draws $(s\ell)$ with the kernel $(A_F \otimes A_M)$, so the analysis model is misspecified, by an amount this chapter does not quantify: the BLUP and every PEV below are exact for the *analysis* model, not for the model that generated the data.

What is new is only the incidence. A crossing plan $P \subset \{1..n_F\} \times \{1..n_M\}$ and an allocation give a 0/1 matrix M of size $n_F n_M \times L$ in which a row is zero whenever $(i, j) \notin P$. Every row in P sums to k. The R/17 functions accept such an M directly; the unmade crosses are predicted through A_F, A_M and the GCA they share with made crosses – the CV1 prediction of Section 10.4, now produced by the crossing plan rather than by a mask.

```
p <- design_circulant(6, 6, c = 2)            # 12 of 36 possible crosses
M <- joint_alloc(p, 6, 6, nLoc = 3, k = 1, strategy = "coverage")
c(crosses_made = sum(rowSums(M) > 0), plots = sum(M), per_location = colSums(M))
#>  crosses_made         plots per_location1 per_location2 per_location3
#>            12            12             4             4             4
```

11.3 What the literature offers

The search was run in October 2026 over three sources: the Consensus index of peer-reviewed papers (Semantic Scholar, OpenAlex and PubMed), Crossref, and a general web search, with the bioRxiv and Crop Science / Plant Genome literature reached through them. Queries combined *sparse testing*, *multi-environment trial*, *allocation of genotypes / hybrids to environments*, *incomplete factorial*, *mating design*, *training set optimisation*, *CDmean*, *A-optimality* and *genetic relatedness*, with and without *hybrid* and *maize*. It looked for methods that allocate entries to environments, choose crosses or training hybrids, or both. Six families of approach emerge. None was built for the joint problem, but each supplies a piece. The training-set optimisation line of D. Akdemir, S. Rio, and J. Isidro y Sánchez [119], J. Isidro y Sanchez and D. Akdemir [94] and J. Fernandez-Gonzalez, D. Akdemir, and J. Isidro y Sanchez [95] is the nearest neighbour not in the table: its tools select genotypes for a training set, and could in principle be pointed at genotype-environment pairs, but none of the papers found does so for a crossing plan.

```
lit <- data.frame(
  Approach = c("Model-based design with relatedness",
```

```
               "CD / PEV criteria for multi-environment trials",
               "Choosing hybrids to create and phenotype",
               "Allocation by incomplete-block principle",
               "Environment structure in allocation",
               "Sparse factorials and sparse testcrossing"),
   Optimises = c("Entry placement in plots and environments (A-optimality on the
mixed-model equations)",
                 "Multi-environment design, forecast under a G x E model",
                 "Which single crosses to make and phenotype",
                 "Which lines go to which environment",
                 "Replication and sparsity across mega-environments",
                 "Crosses per line, testers per family"),
  `Held fixed` = c("The entry list", "The entry list", "The environments",
                   "The entry list", "The entry list", "The environments"),
  Borrowed = c("The criterion (PEV of GCA)", "Reliability forecast before planting",
               "Exchange search over a binary design", "Balance of location loads",
               "Locations are not exchangeable", "Parent coverage, connectivity"),
  Refs = c("butler2014; cullis2020; lisle2021", "rio2022",
           "heslot2020; kadam2021", "montesinoslopez2022ibd; montesinoslopez2023;
lell2021; mothukuri2025",
           "gonzalezbarrios2019; hoefler2026",
           "seye2020; lorenzi2022; melchinger2023; gonzalezdieguez2026"),
  check.names = FALSE)
knitr::kable(lit, row.names = FALSE)
```

Table 11.1: Current approaches, what each optimises, and what each holds fixed. The last column is the piece the joint problem borrows.

Approach	Optimises	Held fixed	Borrowed	Refs
Model-based design with relatedness	Entry placement in plots and environments (A-optimality on the mixed-model equations)	The entry list	The criterion (PEV of GCA)	butler2014; cullis2020; lisle2021
CD / PEV criteria for multi-environment trials	Multi-environment design, forecast under a G x E model	The entry list	Reliability forecast before planting	rio2022
Choosing hybrids to create and phenotype	Which single crosses to make and phenotype	The environments	Exchange search over a binary design	heslot2020; kadam2021
Allocation by incomplete-block principle	Which lines go to which environment	The entry list	Balance of location loads	montesinoslopez2022ibd; montesinoslopez2023; lell2021; mothukuri2025
Environment structure in allocation	Replication and sparsity across mega-environments	The entry list	Locations are not exchangeable	gonzalezbarrios2019; hoefler2026
Sparse factorials and sparse testcrossing	Crosses per line, testers per family	The environments	Parent coverage, connectivity	seye2020; lorenzi2022; melchinger2023; gonzalezdieguez2026

Model-based design with relatedness. D. G. Butler, A. B. Smith, and B. R. Cullis [120] and B. R. Cullis, A. B. Smith, N. A. Cocks, and D. G. Butler [117] generate field designs by optimising a criterion – the A-optimality of the predicted genetic effects – computed from the linear mixed model that will analyse the trial, with the relationship matrix inside it. Their in-silico studies show that designs built this way are more accurate than designs built for unrelated entries. C. Lisle, A. B. Smith, C. Birrell, and B. R. Cullis [121]

add an information-based diagnostic for whether a multi-environment design can estimate its genetic variance parameters at all. This is the most principled framework available, and the criterion this chapter uses is its restriction to the GCA effects. What it does not do is choose the entries: the list of varieties is an input.

CD and PEV criteria for multi-environment trials. S. Rio, D. Akdemir, T. Carvalho, and J. Isidro y Sánchez [122] extend the coefficient of determination, the CDmean of R. Rincent *et al.* [37], to G x E models, so that the reliability of a multi-environment design can be forecast before it is planted and the design optimised on it, for a given set of genotypes.

Choosing which hybrids to create. D. C. Kadam, O. R. Rodriguez, and A. J. Lorenz [116] apply PEV and CDmean to single crosses and show that optimising over *all possible* crosses, not only the ones already made, gives a better training set – the crossing half of the joint problem, with the environment design held fixed. N. Heslot and V. Feoktistov [115] are the closest methodological precedent. They optimise which maize hybrids to create and phenotype, to best predict the unobserved combinations, with a mixed binary-integer differential evolution, and extend the method to designing a connected population before the crosses are made. The environments are again outside the optimisation.

Allocation by incomplete-block principle. O. A. Montesinos-López, A. Montesinos-López, R. Acosta, R. K. Varshney, A. Bentley, and J. Crossa [118] allocate lines to locations with an incomplete block design and beat random allocation by 1.4 to 26.5 % in mean squared error; O. A. Montesinos-López, B. A. Mosqueda-González, J. Salinas-Ruiz, A. Montesinos-López, and J. Crossa [45] find balanced incomplete blocks and random allocation close. M. Lell, J. C. Reif, and Y. Zhao [123], on 1,604 wheat hybrids, find the best design splits the hybrids into equal subgroups each tested in an overlapping subset of environments. S. R. Mothukuri, Y. Beyene, M. Gültas, J. Burgueño, and S. Griebel [124] use the relationship between fully and sparsely tested maize lines to guide the split, with a weak positive correlation (0.20 to 0.31) between the relationship measures and accuracy. All four take the entries as given.

Environment structure. The mega-environmental design of P. González-Barrios, L. Díaz-García, and L. Gutiérrez [125] allocates entries so that each mega-environment is sampled, and reports a 40 % gain in response for the same resources; R. Hoefler and L. Gutierrez [126] optimise its replication and sparsity by simulation. Both are reminders that the locations here are treated as exchangeable, which a real target population of environments is not.

Sparse factorials and sparse testcrossing. A. Seye, C. Bauland, A. Charcosset, and L. Moreau [34], A. Lorenzi *et al.* [40] and A. Melchinger, R. Fernando, C. Stricker, C. Schön, and H. Auinger [41] establish that sparse factorials train hybrid prediction at least as well as tester designs, and that fewer crosses per line favours the lines never crossed; D. González-Diéguez, G. Atlin, Y. Beyene, D. Wegary, D. Gemenet, and C. Werner [35] spread three to five testers among full-sibs. L. Thomsen *et al.* [38] analyse oilseed-rape hybrids under sparse multi-environment testing, observationally. D. L. Schoemaker, D. C. Lima, N. de Leon, and S. M. Kaeppler [127] trade testers against environments in maize and find three to five well-chosen environments match ten random ones – the closest empirical look at one corner of the joint budget.

The gap is therefore precise. The criteria exist (PEV and CD with relatedness), the search methods exist (exchange algorithms, differential evolution), and both halves have been optimised separately. What is missing is the object that holds the crossing plan and the allocation together under one budget, and evidence on how the budget should be split.

11.4 Design criteria computable before planting

Two quantities can be read off M before a seed is sown.

The mean PEV of GCA. The left-hand side of the mixed-model equations depends on the design alone, so its inverse, and with it the prediction error variance of every GCA effect, is known before any phenotype exists. `joint_pev()` returns the mean over the $n_F + n_M$ GCA effects. It is the A-optimality criterion of D. G. Butler, A. B. Smith, and B. R. Cullis [120] restricted to the effects parents are selected on; with known components it is exact for the analysis model of Section 11.2, and Table 11.2 shows it ranking allocations of the same plan. It is blind to two things a programme may care about: SCA, and the difference between predicting crosses that were made and crosses that were not.

Two component counts. A single graph with females, males and locations as nodes is the obvious female x male x location metric, and it is wrong: a location node joins every parent planted there, so at one location it reports one component for any crossing plan, including two disconnected NC II blocks. A shared location links only *sums* of GCA. `joint_connectivity()` therefore returns two counts alongside the R/17 diagnostics:

- `n_comp_cross`, the components of the female x male graph of the crosses that receive a plot, pooled over locations. GCA is a main effect shared by every location, so this is what links GCA contrasts by data, and at one location it is exactly the bipartite count of Section 9.2.3.
- `n_comp_loc`, the components of the location graph in which two locations are joined when they share a parent. Location contrasts are linked by data only when it is 1.

Both equal to 1 is *necessary*, not sufficient: a relationship matrix can link families the data do not, and shared parents link locations only through those parents' GCA. This pair is the female x male x location metric Section 10.13 asked for.

```
set.seed(11)
vc <- sparse_vc(0.3); A <- relmat_halfsib(10, 2)
p <- design_circulant(10, 10, c = 3)
live <- do.call(rbind, lapply(names(strat_col), function(s) {
  M <- joint_alloc(p, 10, 10, nLoc = 4, k = 2, strategy = s, vc = vc,
                   Af = A, Am = A, n_iter = 100)
  cn <- joint_connectivity(M, 10, 10)
  data.frame(strategy = s, plots = sum(M), pev_gca = joint_pev(M, 10, 10, vc, A, A),
             fem_cover = cn$fem_cover, male_cover = cn$male_cover,
             n_comp_cross = cn$n_comp_cross, n_comp_loc = cn$n_comp_loc)
}))
knitr::kable(fmt(live, 4), row.names = FALSE)
```

Table 11.2: One crossing plan (circulant, 3 crosses per line, 10 x 10), three allocations over 4 locations at 2 locations per cross. Mean PEV of GCA (lower is better), share of each pool present per location, and the two component counts. Live.

strategy	plots	pev_gca	fem_cover	male_cover	n_comp_cross	n_comp_loc
random	60	0.6405	0.85	0.8	1	1
coverage	60	0.6341	1.00	1.0	1	1
optimised	60	0.6325	1.00	1.0	1	1

11.5 What the simulation finds

The sweep crosses six configurations that all spend 160 plots (the tester design 152) – circulant plans with 8, 4 and 2 crosses per line at 1, 2 and 4 locations per cross, random sparse plans with 8 and 4, and a two-tester design at two locations – with the allocation strategies, two plot heritabilities and two SCA levels (`sparse_vc()`: SCA a quarter of, or equal to, a GCA variance; genotype x location half of each main effect, so a genetic correlation of 2/3 between locations). Pools are 20 x 20 in half-sib families of four; eight locations; 30 replicates per cell.

The comparisons are paired. In each replicate one plan is drawn; `random` and `coverage` allocate that same plan; and every design is simulated from the same seed, so they share the true genetic values (`sparse_simulate()` draws every effect before it looks at the allocation). The exchange search is too slow to repeat per replicate, so per cell one plan gets a coverage allocation (`start`) and a search from it; those two fixed designs are then simulated in every replicate with the same seed, which pairs `optimised` with its own start.

11.5.1 1. The budget split: crosses against locations

```
lev <- c("circulant c=8, 1 loc", "circulant c=4, 2 loc", "circulant c=2, 4 loc",
         "random c=8, 1 loc", "random c=4, 2 loc", "tester k=2, 2 loc")
op <- par(mfrow = c(2, 2), mar = c(7.5, 4, 2, 0.5), mgp = c(2.4, 0.7, 0))
for (sc in c("low", "high")) for (h in c(0.2, 0.4)) {
  d <- js[js$sca == sc & js$h2_plot == h, ]
 plot(NA, xlim = c(0.5, length(lev) + 0.5), ylim = range(js$acc_all) + c(-0.04, 0.04),
      xaxt = "n", xlab = "", ylab = "Accuracy, all hybrids",
      main = sprintf("h2 = %.1f, SCA %s", h, sc), cex.main = 0.95, font.main = 1)
  axis(1, at = seq_along(lev), labels = lev, las = 2, cex.axis = 0.75)
```

```
  for (j in seq_along(strat_col)) {
    e <- d[d$strategy == names(strat_col)[j], ]
    x <- match(e$label, lev) + (j - 2) * 0.2
    arrows(x, e$acc_all - 2 * e$acc_all_se, x, e$acc_all + 2 * e$acc_all_se,
           angle = 90, code = 3, length = 0.02, col = strat_col[j])
    points(x, e$acc_all, pch = 19, col = strat_col[j])
  }
  if (sc == "high" && h == 0.2)
    legend("topright", legend = names(strat_col), col = strat_col, pch = 19,
           bty = "n", cex = 0.8)
}
par(op)
```

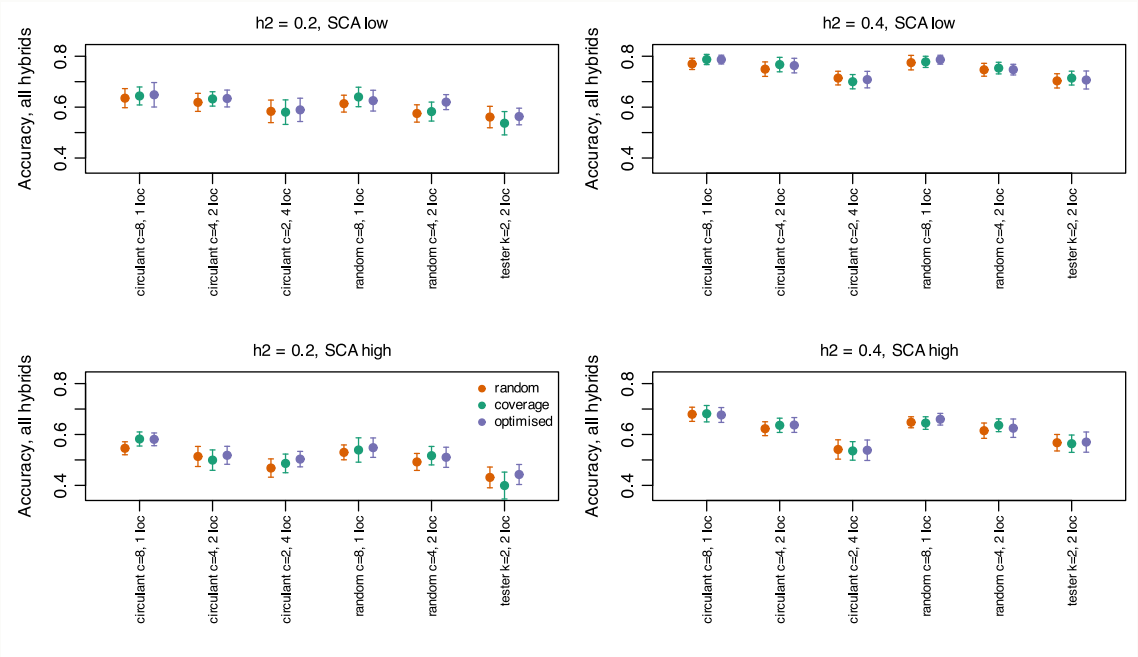

Figure 11.1: Accuracy over all 400 possible hybrids at 160 plots (tester 152), by configuration (crosses per line, locations per cross) and allocation strategy, in the four variance scenarios. Bars are two standard errors of the cell mean. `optimised` is one fixed design per cell. In-repo sweep, known components, 30 replicates.

Figure 11.1 reads left to right along the budget identity. Within the circulant family, with the coverage allocation, 160 crosses at one location each reach 0.787 at $h^2 = 0.4$ and low SCA, 80 crosses at two locations 0.767, and 40 at four 0.700. With high SCA the steps are larger – 0.682, 0.636, 0.535 – because a cross that is not made has an SCA no replication of other crosses can reveal. The order holds in all four scenarios. Its size is uneven: the step from 80 to 160 crosses is 0.5 to 3.4 standard errors of the difference across scenarios, the step from 40 to 80 0.5 to 4.4. The 160-versus-80 step at low SCA and the 80-versus-40 step at $h^2 = 0.2$ with high SCA are not resolved by 30 replicates; only their direction agrees with the rest. The two-tester design, with 76 crosses at two locations, sits below the circulant plan with 80 crosses at two locations in every scenario, at nearly the same budget (152 against 160 plots).

For the crosses that were made the comparison reverses (Table 11.3), though not step by step: at $h^2 = 0.2$ and high SCA, 80 crosses at two locations are no better than 160 at one. Forty crosses at four locations estimate *those* crosses at 0.845, against 0.807 for 160 crosses at one location ($h^2 = 0.4$, low SCA; 1.2 to 6.0 standard errors across scenarios), because each is seen in four environments rather than one. A programme that picks parents on GCA, or will choose among all possible single crosses with a prediction model, should spend the budget on distinct crosses. A programme that will advance only crosses it has made gets better estimates of those crosses from more locations – but it then chooses among 40 candidates instead of 160, and the response to selection depends on that intensity as well as on accuracy. This chapter measures accuracy only; it does not settle that trade.

```
tb <- cc[order(cc$sca, cc$h2_plot, -cc$c), ]
tb$crosses <- tb$c * 20
knitr::kable(fmt(tb[, c("sca", "h2_plot", "crosses", "k", "acc_all", "acc_all_se",
                        "acc_made", "acc_made_se", "acc_unmade")], 3),
             row.names = FALSE,
             col.names = c("SCA", "Plot h2", "Crosses", "Loc./cross", "All", "SE",
                           "Made", "SE", "Not made"))
```

Table 11.3: The circulant trade-off at 160 plots, coverage allocation: accuracy over all hybrids, over the crosses made, and over those not made, with standard errors of the cell means.

SCA	Plot h2	Crosses	Loc./cross	All	SE	Made	SE	Not made
high	0.2	160	1	0.582	0.014	0.618	0.014	0.558
high	0.2	80	2	0.499	0.020	0.610	0.018	0.467
high	0.2	40	4	0.486	0.018	0.709	0.018	0.455
high	0.4	160	1	0.682	0.016	0.742	0.012	0.637
high	0.4	80	2	0.636	0.014	0.783	0.009	0.595
high	0.4	40	4	0.535	0.018	0.837	0.010	0.490
low	0.2	160	1	0.644	0.018	0.669	0.017	0.627
low	0.2	80	2	0.632	0.014	0.693	0.013	0.616
low	0.2	40	4	0.580	0.024	0.704	0.023	0.565
low	0.4	160	1	0.787	0.010	0.807	0.010	0.775
low	0.4	80	2	0.767	0.014	0.824	0.011	0.753
low	0.4	40	4	0.700	0.014	0.845	0.011	0.683

The result of Chapter 10. that spreading a fixed number of plots per hybrid over more locations raises accuracy (Section 10.11.3, result 2) is not contradicted. There each hybrid kept two plots and the factorial was complete; here every extra location per cross is paid for with a cross. Spreading wins when it is free and loses, on all-hybrid accuracy, when it costs crosses. That was measured at one genetic correlation between locations, 2/3. A lower correlation makes each location carry less information about the others, which would be expected to move the optimum toward more locations per cross; this sweep did not test it.

11.5.2 2. Allocation strategy, with the plan fixed

With the plan and k fixed, the strategies are close (Figure 11.1). The paired contrasts (Table 11.4) resolve what the cell means cannot. Coverage does what it is built to do: in the factorial plans the share of females present per location rises from 0.676 under random placement to 0.874. Paired against random placement of the same plan, the accuracy difference over all hybrids, averaged over the 24 cells, is $+0.0057 \pm 0.0033$ ($z = 1.7$; 16 of 24 cells positive). That is below the two-standard-error bar this chapter uses, so the gain is **not detected**; only 2 of the 24 cells differ by more than two standard errors on their own, about what chance gives. The tester design is the exception in the other direction: there coverage moves the share of each pool present per location only from 0.494 to 0.549 – each tester already sits in most locations – and the paired difference is -0.012 ± 0.011. An earlier, unpaired version of this sweep had suggested the opposite for the tester design; with the plan and the truth held fixed, the suggestion does not survive.

```
st <- rbind(cbind(contrast = "coverage - random",
                  rbind(pc_plan, pc_all)),
            cbind(contrast = "optimised - start",
                  rbind(pooled("optimised - start", by = "plan"), po_all)))
knitr::kable(fmt(st, 4), row.names = FALSE,
         col.names = c("Contrast", "Plan family", "Cells", "diff", "SE", "Positive"))
```

Table 11.4: Paired contrasts in accuracy over all hybrids, pooled over the four variance scenarios within each plan family (and over everything in the last row of each block). `diff` is the mean paired difference, `SE` its standard error, `positive` the number of cells with a positive difference.

Contrast	Plan family	Cells	diff	SE	Positive
coverage - random	circulant	12	0.0075	0.0042	8
coverage - random	random	8	0.0119	0.0052	7
coverage - random	tester	4	−0.0123	0.0108	1
coverage - random	all	24	0.0057	0.0033	16
optimised - start	circulant	12	0.0023	0.0030	8
optimised - start	random	8	−0.0050	0.0035	5
optimised - start	tester	4	0.0189	0.0070	4
optimised - start	all	24	0.0027	0.0022	17

This replicates, with the plan now variable, the finding of Chapter 10. that structured layouts matter little except at extreme sparsity, and it sits with the mixed record of the literature: O. A. Montesinos-López, A. Montesinos-López, R. Acosta, R. K. Varshney, A. Bentley, and J. Crossa [118] found incomplete blocks better than random, O. A. Montesinos-López, B. A. Mosqueda-González, J. Salinas-Ruiz, A. Montesinos-López, and J. Crossa [45] found them close. When every line appears in several crosses spread over eight locations, random placement already puts most parents in most locations; whatever coverage adds – at most about half a point on the point estimate – is small next to the budget split.

11.5.3 3. What the criterion can and cannot see

```
jp <- js[js$strategy %in% names(strat_col), ]
op <- par(mar = c(4.2, 4.2, 1, 1))
plot(jp$pev_gca, jp$acc_all, pch = 19, col = strat_col[jp$strategy],
     xlab = "Mean PEV of GCA (before planting)", ylab = "Accuracy, all hybrids")
legend("topright", legend = names(strat_col), col = strat_col, pch = 19, bty = "n",
cex = 0.8)
par(op)
r_pev <- cor(jp$pev_gca, jp$acc_all)
# Rank agreement between PEV and accuracy over the six configurations, per scenario.
cvr <- js[js$strategy == "coverage", ]
rho <- sapply(split(cvr, paste(cvr$h2_plot, cvr$sca)),
              function(d) cor(d$pev_gca, d$acc_all, method = "spearman"))
rho_made <- sapply(split(cvr, paste(cvr$h2_plot, cvr$sca)),
                   function(d) cor(d$pev_gca, d$acc_made, method = "spearman"))
g300 <- median(1 - jo$pev_300 / jo$pev_start)
g600 <- median(1 - jo$pev_600 / jo$pev_start)
spread <- sapply(split(jp, paste(jp$label, jp$h2_plot, jp$sca)),
                 function(d) diff(range(d$pev_gca)) / min(d$pev_gca))
second <- (jo$pev_300 - jo$pev_600) / (jo$pev_start - jo$pev_300)
pot <- pooled("optimised - start", by = "plan")
pot <- pot[pot$group == "tester", ]
```

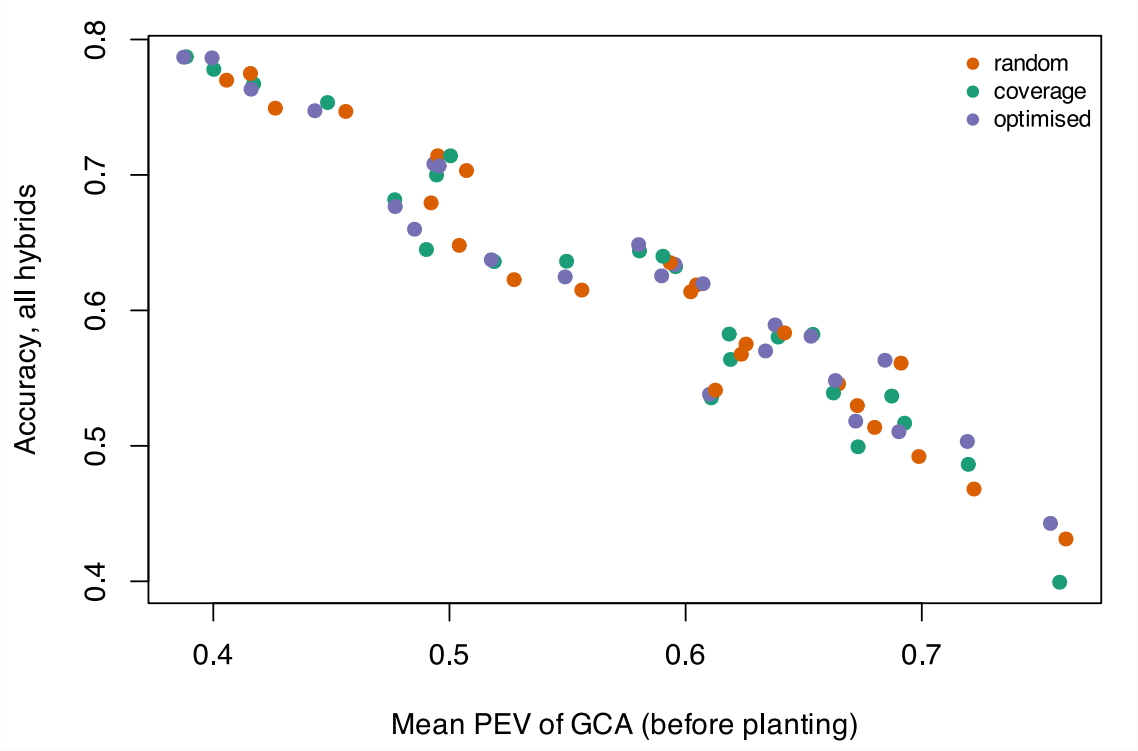

Figure 11.2: Mean PEV of GCA, computed before planting, against realised accuracy over all hybrids. One point per configuration x strategy x scenario; colour is the strategy. In-repo sweep.

Across all cells the mean PEV of GCA correlates with accuracy over all hybrids at −0.96 (Figure 11.2; lower PEV, higher accuracy). Almost all of that comes from the plan and the scenario: the clusters are configurations, and within a cluster the strategies differ by 0.3 to 4.7 % in PEV (median 1.8 %). Over the six configurations of one scenario, the rank correlation between PEV and all-hybrid accuracy is −1.00 to −0.89. The criterion is therefore a usable screen for *which plan and how many locations per cross* – it can be computed for every split of a budget in seconds – but only for that objective. Against accuracy on the crosses that were made its rank correlation is 0.20 to 0.66: mean PEV of GCA cannot see SCA or the made/unmade distinction, so it steers a programme that selects only among made crosses the wrong way.

As a guide to fine placement it is weak, and the exchange search shows it from the other side (Table 11.5). From the coverage start, 300 proposals lowered PEV by a median 0.10 % and 600 by 0.15 %, at about 47 seconds per design against a millisecond for the greedy start. Paired against its own start the optimised design changed accuracy by +0.0027 ± 0.0022 overall. The one family that stands out is the tester design, +0.019 ± 0.007 with all four cells positive, where the search also found the largest PEV reductions. It is tentative: each cell is one optimised design from one plan, and it is one of three plan families, with no correction for that. The search is first-improvement with no convergence test, and it had not converged: the second 300 proposals added a median 40 % of what the first 300 achieved, and in 8 of 24 cells 65 % or more. PEV was still falling. These results therefore say what 600 proposals buy – a fraction of a percent of PEV and no detected accuracy – not what a converged placement optimiser would buy. The optimised-block design of the external study behind Chapter 10. showed the same absence of gain on untested hybrids.

```
ot <- jo[order(jo$sca, jo$h2_plot, jo$plan, -jo$c), ]
ot$sec_600 <- ifelse(ot$sec_600 > 600, NA, round(ot$sec_600))
knitr::kable(fmt(ot[, c("plan", "c", "k", "h2_plot", "sca", "pev_start", "pev_300",
                        "pev_600", "sec_600")], 4), row.names = FALSE,
             col.names = c("Plan", "c", "Loc./cross", "Plot h2", "SCA", "PEV start",
                           "PEV 300", "PEV 600", "Seconds"))
```

Table 11.5: The exchange search on mean PEV of GCA, one design per cell: PEV of the coverage start and after 300 and 600 proposals. Seconds are CPU-bound wall time (the last three cells ran across a machine sleep and are omitted).

Plan	c	Loc./cross	Plot h2	SCA	PEV start	PEV 300	PEV 600	Seconds
circulant	8	1	0.2	high	0.6538	0.6534	0.6531	47
circulant	4	2	0.2	high	0.6726	0.6721	0.6720	47
circulant	2	4	0.2	high	0.7198	0.7195	0.7193	48
random	8	1	0.2	high	0.6642	0.6638	0.6634	47
random	4	2	0.2	high	0.6907	0.6903	0.6903	47
tester	2	2	0.2	high	0.7560	0.7548	0.7545	46
circulant	8	1	0.4	high	0.4774	0.4773	0.4770	48
circulant	4	2	0.4	high	0.5184	0.5179	0.5177	47
circulant	2	4	0.4	high	0.6107	0.6103	0.6101	47
random	8	1	0.4	high	0.4860	0.4855	0.4852	NA
random	4	2	0.4	high	0.5504	0.5496	0.5490	NA
tester	2	2	0.4	high	0.6355	0.6340	0.6338	NA
circulant	8	1	0.2	low	0.5801	0.5801	0.5801	47
circulant	4	2	0.2	low	0.5963	0.5956	0.5954	49
circulant	2	4	0.2	low	0.6385	0.6381	0.6379	48
random	8	1	0.2	low	0.5909	0.5902	0.5898	47
random	4	2	0.2	low	0.6082	0.6076	0.6073	47
tester	2	2	0.2	low	0.6869	0.6850	0.6844	46
circulant	8	1	0.4	low	0.3885	0.3877	0.3875	47
circulant	4	2	0.4	low	0.4172	0.4162	0.4160	47
circulant	2	4	0.4	low	0.4947	0.4937	0.4934	47
random	8	1	0.4	low	0.4006	0.3999	0.3994	47
random	4	2	0.4	low	0.4450	0.4439	0.4430	48
tester	2	2	0.4	low	0.4983	0.4962	0.4956	48

The component counts were 1 in almost every replicate. The exception is informative: in two of the 120 random sparse plans with 4 crosses per line (one replicate in each of two scenarios, appearing under both strategies that share the plan), the draw split the factorial into two components (`n_comp_cross = 2`) – a defect of the plan that no allocation can repair, visible before a seed is sown. Location connectivity (`n_comp_loc`) was 1 throughout: with several crosses per line over eight locations, locations always shared parents.

11.5.4 Limitations of the simulation

- **Known variance components.** The criterion and the BLUP use the true values; REML on a sparse trial is the weak point Section 10.11.3 found.
- **A misspecified analysis model**: SCA x location is absorbed into an i.i.d. residual (Section 11.2).
- **Exchangeable locations** with a genetic correlation of 2/3 between every pair. Mega-environment structure [125], [126] would favour allocations that sample each mega-environment, which the coverage rule does not see.
- **Half-sib relationships and one plot per cell.** No markers, no spatial trend, no replicated checks.
- **One budget, one scale**: 160 plots, 20 x 20 pools, eight locations.
- **Accuracy, not response.** Selection intensity differs between 40 and 160 candidates and is not modelled.
- **The optimised strategy** was searched once per cell, from one plan, with 600 first-improvement proposals.

11.6 Practical recommendations

A checklist for a joint crossing and testing plan

1. **Fix the budget in plots,** $B = n_c k r$, and the locations available. Everything below trades inside it.
2. **Decide what will be selected.** If parents are chosen on GCA, or any single cross by prediction, favour distinct crosses: in this sweep 160 crosses at one location beat 80 at two, which beat 40 at four, in direction in every scenario, though not every step was resolved. If only crosses already made will be advanced, more locations estimate them better, but at the cost of selection intensity; weigh both.
3. **Screen candidate splits with the mean PEV of GCA** (`joint_pev()`), with the relationship matrix and variance components of the previous cycle. It ranks splits for the GCA / all-hybrid objective; it is blind to SCA and should not be used for a made-crosses-only objective.
4. **Treat one location per cross with caution.** It was best here, but with the variance components known. With one plot per cross each cross is confounded with its environment and REML has little to work with (Section 10.11.3, result 5); two locations per cross is the safer floor when the components must be estimated.
5. **Place crosses with the coverage rule** (`joint_alloc(strategy = "coverage")`). It costs nothing, balances loads and raises parent coverage; an accuracy gain over random placement was not detected (point estimate half a point), and in the tester design there was none.
6. **Check `n_comp_cross == 1` and `n_comp_loc == 1`** before planting. A random sparse plan can come out disconnected; the fix is a new plan, not a new allocation.
7. **Reserve exchange optimisation of the placement for tester designs**, and treat even that as provisional: overall it bought no accuracy, and the tester gain rests on four cells and a search that had not converged; run it well beyond 600 proposals. Whether it helps with unequal pools or strongly structured environments is untested.

11.7 Open problems

- **A joint optimiser over the plan itself.** Here the plan family was enumerated and the placement optimised. Searching crosses and locations in one binary space is the problem N. Heslot and V. Feoktistov [115] solved for crosses alone; the criterion is ready, the search at programme scale is not.
- **A criterion that sees SCA and the selection objective.** Mean PEV of GCA ranks plans for one objective only; a criterion on the target values of the candidates that will actually be selected among, with intensity, is the missing piece.
- **Non-exchangeable environments.** Combining the mega-environment structure of P. González-Barrios, L. Díaz-García, and L. Gutiérrez [125] with a crossing plan, and the genetic correlation between locations at which the optimum moves toward more locations.
- **Estimated components.** How much of the budget split survives REML on the sparse trial, and whether the PEV screen should use a prior or the previous cycle's estimates [121].
- **Seed as a constraint.** A cross planted at k locations needs k plots of seed; the cost structure of `cross_cost()` (Section 9.7) has not been joined to the allocation.
- **Prospective validation.** As for sparse testing generally, no trial has been designed jointly sparse from the start and compared against a conventional plan.

11.8 Reproducibility and assumptions

`Rscript book/scripts/regenerate.R joint` writes `joint_core_sweep.csv` (cell means and standard errors), `joint_contrasts.csv` (paired contrasts) and `joint_optimiser.csv` (the exchange-search trace) to `book/data`, in about 30 minutes of CPU, almost two thirds of it in the exchange searches. It keeps the per-replicate results in `report/joint_raw.rds` (gitignored). The live example of Table 11.2 runs at render time. `tests/testthat/test-joint.R` checks that:

- every made cross gets exactly k plots and every unmade cross none;
- the coverage rule balances loads and covers more of each pool than random placement;

- `joint_pev()` equals the GCA block of the explicitly inverted coefficient matrix of `sparse_blup()`;
- the optimised allocation never scores worse than its start, whether the start is implicit or passed in;
- `n_comp_cross` is two for NC II blocks at one location, in separate locations and in shared locations, and equals the bipartite count of `design_components()`; `n_comp_loc` is two only when the blocks are grown apart.

The numbers are reproducible findings of this configuration, not constants: they will move with the budget, the pool sizes, the genetic correlation between locations and the SCA share.

11.9 Where this connects

- Chapter 9. chose the crosses with the field fixed and Chapter 10. the field with the crosses fixed; this chapter puts both under one budget and finds the crossing plan the larger lever.
- The coverage rule is result 4 of Section 10.11.3 – parent coverage, not shared hybrids, carries connectivity – turned into an allocation rule.
- Spending the budget on distinct crosses keeps more lines in the data, which is the diversity argument of Section 9.11: a line never crossed is a line the next cycle will in practice not use (Chapter 16.; Chapter 22.).
- The PEV screen is the same mixed-model machinery as the genomic prediction of 3, read before planting instead of after harvest.

Reproduce this chapter

All the code of this chapter, in order and with the explanation kept as comments, is one script: **04g-joint-allocation.R**. Download it, install `hybdiv`, and run it top to bottom in an empty directory; cached results are fetched on first use.

GCA 203
13.12 Running this on a real programme 206
13.13 What this does not change 212
13.14 Where this connects 213
13.14 Reproduce this chapter 213

IV Metrics

14 A catalogue of metrics on `f` 215
14.1 Group coancestry, and the off-diagonal problem 215
14.2 Gene diversity: θ and H_e are the same quantity . 216
14.3 Status number N_s 216
14.4 Effective number of parents 217
14.5 Allelic richness 217
14.6 Distances: E-NE and A-NE 218
14.7 Effective dimensionality 218
14.8 Decomposition by heterotic group 219
14.9 The correct baseline 220
14.10 Every metric against the null 221
14.10 Reproduce this chapter 221

15 Choosing metrics: collapse, cost, redundancy 223
15.1 Finding 1: the hybrid coancestry matrix is never needed 223
15.2 Finding 2: what each metric costs 225
15.3 Finding 3: the metric space is nearly one-dimensional 226
15.4 Finding 4: the encoding, not the constraint handler, decides whether DE works 229
15.5 Finding 5: the index is not the truth, and its error is not white 230
15.6 The recommendation, stated 232
15.7 Drop-in code 232
15.7 Reproduce this chapter 233

16 Metrics at each pipeline stage, and five traps 235
16.1 The kernel: one matrix, all four stages 235
16.2 S1: conserving the heterotic groups 236
16.3 S2: choosing crosses for DH induction 237
16.4 S3: selecting lines per se 239
16.5 S4: hybrid mining 241
16.6 Where the diversity actually goes 241
16.7 Cost summary 243
16.8 Checklist 243
16.8 Reproduce this chapter 243

17 Predicting breeding values: GBLUP, BayesA, BayesB 245
17.1 A training population with real noise 245
17.2 The truth this chapter is predicting 246
17.3 GBLUP: prediction from the relationship matrix 247
17.4 What the random split was measuring 249
17.5 BayesA and BayesB: prediction from marker effects 250
17.6 Verification: two routes agree 253
17.7 Which model wins, and when 253
17.8 From GEBVs to `stage1_build()` 255
17.9 Back to Finding 5: a measured accuracy, not an assumed one 255
17.9 Reproduce this chapter 256

12. Diversity metrics in the genomic era

In classical theory, inbreeding measured as **loss of heterozygosity** and inbreeding measured as **drift of allele frequencies** are the same thing. T. H. E. Meuwissen, A. K. Sonesson, G. Gebregiwergis, and J. A. Woolliams [6] show that inside genomic optimal-contribution schemes they stop being the same thing, and can diverge badly, whenever diversity is controlled through **identity by state** on a marker panel. Only when control is through **identity by descent** is the classical equivalence restored.

The practical consequence is uncomfortable:

> You hit exactly the target you wrote into the optimiser's constraint, and you pay an unmonitored price in the metric you did not write.

If you constrain $\frac{1}{2}\mathbf{c}'\mathbf{G}\mathbf{c}$ using a VanRaden $\mathbf{G}$, you are constraining **drift**. The loss of heterozygosity in your germplasm may be running at a different rate, in either direction, from the one you believe you are controlling.

This chapter is written for one kind of programme, stated in full in Section 12.1: an allogamous, maize-type hybrid programme running reciprocal recurrent selection over two heterotic pools. The first half defines the metrics; the second half is the sequence of steps that puts them into a cycle report, with the check and the failure mode of each step.

12.1 The programme this chapter is written for

Every number below is computed under these assumptions. They are declared here because several of the results are false outside them.

A1. The species is allogamous and the product is an F1 hybrid. Maize-type: two heterotic pools, A and B, crossed to produce single-cross hybrids. The commercial product carries the heterozygosity; the parents do not. Nothing here is addressed to a self-pollinated crop whose product is a line, and nothing here is addressed to a programme whose commercial unit is a synthetic or a population cross – both change which quantity is the product and therefore which loss matters.

A2. Parent lines are fully inbred, by DH induction or by selfing to fixation. This is a property of the *programme*, not of the mating system: an allogamous crop bred for hybrids still delivers inbred parents. In this simulation the lines are homozygous at every locus, which the coancestry kernel reports exactly:

```
c(het_fraction_lines   = mean(L / 2 == 0.5),
  self_coancestry_min  = min(diag(molecular_coancestry(L / 2))),
  het_fraction_F1      = mean(ctx$X == 0.5),
  twice_GD_WI_hyb      = 2 * gd_partition(seq_len(ctx$N), ctx)[["GD_WI_hyb"]])
#>  het_fraction_lines self_coancestry_min     het_fraction_F1     twice_GD_WI_hyb
#>           0.0000000           1.0000000           0.3616896           0.3616896
```

Line heterozygosity is zero and every line's self-coancestry is 1, so individual-level inbreeding coefficients are 1 by construction and carry no management information. The F1s carry 0.362 of their loci heterozygous, and that fraction is exactly twice `GD_WI_hyb` – the identity Chapter 13. derives. **The manageable quantity is therefore an allele frequency in a pool, never a homozygosity within an individual.** Everything below is computed on frequencies.

A3. Germplasm is recycled within pool; only information crosses the pool boundary. Lines are chosen on hybrid performance against the opposite pool (R. E. Comstock, H. F. Robinson, and P. H. Harvey [7]) and recycled inside their own pool. A DH taken from an $A \times B$ F1 would be a blend of both pools and would destroy the heterotic pattern, so it never happens. `run_cycles()` in Chapter 22. implements exactly this loop.

A4. Cycles are discrete. Pool sizes are constant across cycles and no generation overlap is modelled. Real programmes overlap cycles and reuse the same line for years, which biases any ΔF read off a cycle index; see Chapter 23..

A5. Markers are neutral, biallelic, coded $0/0.5/1$ as the frequency of the reference allele within the individual, and are in linkage equilibrium within pool because each line's genotype is drawn independently per locus (`simulate_lines()`). Linkage equilibrium makes the audit step of Section 12.7.5 a best case, and that is said again where the number is reported.

A6. The frequency base p_0 is frozen at the founding cycle and never recomputed. This is `ctx$p0`, set once in `build_ctx()`. Section 12.7.1 is that step and measures what happens without it.

A7. No dominance enters any metric in this chapter. θ, GD and α are functions of allele frequencies alone. The dominance side of the same germplasm – what the divergence between pools is worth in heterosis – is Chapter 13., and the two must not be mixed inside one constraint.

A8. The scale is the book's teaching scale: 25 + 25 lines, 2000 markers, 625 hybrids in the full factorial. The production scale is 71 + 71 lines and 25000 markers. Where a result is scale-dependent, it says so.

12.2 Three lenses, not three synonyms

Lens	What the metric actually sees	Inbreeding metric	Relationship matrix
Drift	squared allele-frequency change relative to a base	F_drift	G_VR2, G_VR1, G_i(p)
Homozygosity	proportion of heterozygosity remaining	F_hom	G_0.5 (molecular coancestry)
IBD	segments inherited by descent from a defined base	F_IBD	A, G_LA
Hybrid	haplotype homozygosity used as a proxy for IBD	F_ROH	G_ROH

> **!** Every inbreeding metric is a ratio to a reference population
>
> Without explicitly declaring your base – cycle 0 of the recurrent programme? a landrace panel? the founding heterotic pool? – none of the numbers below mean anything. This is the commonest error in plant applications: using the *current* generation's frequencies as the base, which zeroes the historical memory of erosion. Section 12.7.1 measures the size of that mistake: it is not an attenuation, it is an exact zero.

12.3 The metrics, one at a time

12.3.1 Rate of inbreeding and effective size

What you manage is not F but its rate of accumulation. F is a stock; ΔF is a flow.

$$\Delta F = \frac{F_t - F_{t-1}}{1 - F_{t-1}}, \qquad N_e = \frac{1}{2\Delta F} \tag{12.1}$$

with $F_t \in (-\infty, 1]$ the inbreeding coefficient of cycle t against the frozen base (negative values are possible for F_{hom}, see below), ΔF the per-cycle rate and N_e the effective size it implies [12, Ch. 3-4].

Since $1 - F_t = (1 - \Delta F)^t$, a regression of $\log(1 - F_t)$ on t has slope $\log(1 - \Delta F)$, so $\Delta F = 1 - e^{\text{slope}}$ and $-$ slope is only its first-order approximation. **The linearity of that regression is itself a diagnostic:** in T. H. E. Meuwissen, A. K. Sonesson, G. Gebregiwergis, and J. A. Woolliams [6], $\log(1 - F_{drift})$ was approximately linear under every scheme, while $\log(1 - F_{hom})$ showed marked curvature under some, notably the ROH-based one. Curvature means a non-constant rate – your N_e target is not being met steadily.

A tropical maize recurrent programme targeting $N_e = 50$ per cycle needs $\Delta F = 0.01$. Measuring F only at cycle 8 and dividing by 8 hides the possibility that cycles 1-3 were conservative and cycles 6-8 collapsed the base. Section 12.7.4 fits the regression on a case where the answer is known in advance, and then on one where the diagnostic has to fire.

12.3.2 `F_hom`, homozygosity-based

Wright defined an inbreeding coefficient to run from 0 to 1 as heterozygosity under random mating falls from its initial state to zero. So the metric is the fraction of heterozygosity already lost relative to the base:

$$F_{hom} = 1 - \frac{1}{N_{SNP}} \sum_{k} \frac{2p_{t,k}(1-p_{t,k})}{2p_{0,k}(1-p_{0,k})} = 1 - \frac{1}{N_{SNP}} \sum_{k} \frac{H_{t,k}}{H_{0,k}} \tag{12.2}$$

with $p_{t,k} \in [0,1]$ the frequency of the reference allele at marker k in the population being scored, $p_{0,k} \in (0,1)$ the same frequency in the frozen base, $H = 2p(1-p)$ the expected heterozygosity under random mating, and N_{SNP} the number of markers [6, their Eq. 1A defines it as the total marker count]. This book restricts the sum to markers polymorphic in the base, because $H_{0,k} = 0$ makes the ratio undefined otherwise – `freq_metrics()` does that with `keep <- ctx$p0 > eps & ctx$p0 < 1 - eps` – and the restriction changes nothing against their result, since their own panels were random samples of loci already segregating in the base population.

It can be negative. If management pushes frequencies toward 0.5, heterozygosity rises above the base and the "inbreeding coefficient" goes below zero. That is not a numerical error: it is the expected behaviour of any scheme that maximises heterozygosity on the monitored panel.

It protects against inbreeding depression and the expression of deleterious recessives in homozygosis. It is the right lens when the dominant risk is *vigour* – in this programme, the vigour of the segregating crossing block and not of the fixed lines, which is a narrower claim than it looks and is qualified at the end of this section.

12.3.2.1 What this lens is computed on, and what it is not

F_{hom} as `freq_metrics()` computes it takes p_t from `colMeans(X[idx, ])`: the reference-allele frequency of the **merged** hybrid set. With inbred parents that merged frequency is $\bar{p} = (\tilde{p}_A + \tilde{p}_B)/2$, with $\tilde{p}_A$ and $\tilde{p}_B$ the two pools' frequencies weighted by the line usage the selection induces. So the He column of the same call is $2\bar{p}(1-\bar{p})$, which is $GD_T = 1 - \bar{f}$ of the Caballero & Toro partition [1, Eq. 7] *identically*, not approximately: $\bar{f} = (\theta_A + 2\theta_{AB} + \theta_B)/4 = \bar{p}^2 + (1-\bar{p})^2$ under A2.

That is gene diversity of the notional merged pool, and it is not the heterozygosity of the product. The realised F1 heterozygosity is $2\,GD_{WI\text{-}hyb}$ – exactly, by the self-coancestry identity of assumption A2 – and on the full factorial it equals $GD_T + GD_{BS}$, the between-pool component counted twice (Section 13.3). The difference between the two quantities is therefore GD_{BS}, which the F_{hom} lens does not carry, and which moves with the selection:

```
set.seed(31)
sels <- list(random     = sample.int(ctx$N, 60),
             max_div    = sel_greedy(ctx, 60, w = 0),
             balanced   = sel_greedy(ctx, 60, w = 1),
             truncation = sel_truncation(ctx, 60))
two_levels <- t(sapply(sels, function(i) {
  gp <- gd_partition(i, ctx)
  fm <- freq_metrics(i, ctx)
  c(He_merged = fm[["He"]], GD_T = gp[["GD_T"]], F_hom = fm[["F_hom"]],
    GD_BS = gp[["GD_BS"]], F1_het = mean(ctx$X[i, ] == 0.5),
    two_GD_WI_hyb = 2 * gp[["GD_WI_hyb"]],
    GD_T_plus_BS = gp[["GD_T"]] + gp[["GD_BS"]])
}))
stopifnot(isTRUE(all.equal(two_levels[, "He_merged"], two_levels[, "GD_T"])),
          isTRUE(all.equal(two_levels[, "F1_het"], two_levels[, "two_GD_WI_hyb"])))
fmt(as.data.frame(two_levels), 4)
```

	He_merged	GD_T	F_hom	GD_BS	F1_het	two_GD_WI_hyb	GD_T_plus_BS
random	0.3257	0.3257	0.0115	0.0355	0.3607	0.3607	0.3612
max_div	0.3302	0.3302	−0.0113	0.0361	0.3707	0.3707	0.3664
balanced	0.3124	0.3124	0.0426	0.0524	0.3646	0.3646	0.3648
truncation	0.3124	0.3124	0.0426	0.0524	0.3646	0.3646	0.3648

Two identities hold to machine precision in that table and the `stopifnot()` enforces both on every render: $He = GD_T$, and the realised F1 heterozygosity equals $2\,GD_{WI\text{-}hyb}$. The third comparison is

not an identity on a selected subset: $GD_T + GD_{BS}$ departs from the realised heterozygosity by at most 0.0043 across these four rows, because GD_T and GD_{BS} are computed from line-usage weights – what a random re-pairing of the selected parents would give – while $GD_{WI\text{ - }hyb}$ scores the pairs actually chosen (Chapter 13. states the same limit from the other side).

Now read the ranking. Truncation has the **highest** F_{hom}, 0.0426 against 0.0115 for the random draw, which on the usual reading is "most heterozygosity lost". Yet its hybrids are *more* heterozygous than the random draw's: 0.3646 against 0.3607. The two readings are not in conflict because they are two quantities: truncation concentrated usage inside the pools (Section 12.7.6 measures that) and so lost merged-pool gene diversity, while the $A \times B$ pairs it kept happen to be more divergent – GD_{BS} = 0.0524 against 0.0355. Reported alone, F_{hom} would have called that plan the worst of the four for the very quantity – product heterozygosity – on which it beats the random draw.

Within a heterotic pool under reciprocal recurrent selection, F_{hom} on the *pool's* own frequencies is still the right lens for one thing, and that one thing is narrower than it is usually stated to be. It is the **segregating crossing block**, which is not inbred and in which inbreeding depression is therefore expressed: $ID = 2F \sum_k d_k p_k q_k$ [12, Ch. 14], with d_k the dominance deviation at locus k, p_k and $q_k = 1 - p_k$ the pool's current frequencies, and $F \in [0, 1]$ the block's inbreeding coefficient relative to random mating at those same *current* frequencies – which is **not** the F_t of this chapter's two lenses, measured against the frozen base p_0, and the two must not be substituted for one another. Section 13.5 verifies that quantity against `heterosis_partition()`'s shared term. It is built from the same per-locus product $p_k q_k$ that F_{hom} aggregates – but weighted by d_k, which F_{hom} does not carry, so F_{hom} locates the erosion and does not price it.

It does **not** anticipate the per-se value of the inbred lines themselves. Under A2 a line has no heterozygous locus, so its dominance contribution is exactly zero whatever the pool's frequencies are – Section 13.5 measures that exact zero – and a fully homozygous genotype's value is therefore the additive sum $\sum_k a_k(2x_{ik} - 1)$ alone, the genotypic-value model of Chapter 13. with $x_{ik} \in \{0, 1\}$ at every locus and a_k the per-locus additive effect. Its pool mean moves with the frequencies in whichever direction they move, so there is no directional decline there for F_{hom} to anticipate. It also says nothing about between-pool divergence (Chapter 13.). Computed instead on the homozygosity of individual inbred parents it is 1 by construction – assumption A2 – and is **useless for management**. The mating system of the species does not rescue that: an inbred maize line is as fixed as an inbred wheat line. What distinguishes the hybrid programme is that the pool of lines can be intermated, so the pool has a frequency vector worth managing even though none of its members does.

12.3.3 `F_drift`, drift-based

$$F_{drift} = \frac{1}{N_{SNP}} \sum_k \frac{(p_{t,k} - p_{0,k})^2}{p_{0,k}(1 - p_{0,k})} \tag{12.3}$$

with the same symbols [6].

Never negative. It has the algebraic form of the standardised-variance definition of F_{ST} – a variance of allele frequencies divided by $p_0(1 - p_0)$, the form S. Wright [128] introduced and K. E. Holsinger and B. S. Weir [129] separate from the several other quantities that carry the same name – but applied to *one population through time* rather than to a set of populations in space. That is an analogy of form, not an identity: nothing here is a between-subpopulation variance, and the chapter's own F_{ST} column (Section 12.7.6) is the Nei G_{ST} of `gd_partition()`, a different statistic on a different partition. F_{drift} is exactly the quantity constrained when `G_VR2` goes into the optimiser.

It protects against random change in phenotypic value for traits **outside** the selection objective, against rare deleterious recessives rising to dangerous frequencies, and against irreversible loss of rare alleles.

It must not be read as a measure of *this programme's* selection intensity, even though T. H. E. Meuwissen, A. K. Sonesson, G. Gebregiwergis, and J. A. Woolliams [6] offer exactly that reading for a locus under directional selection. They call $\delta p^2/[p_0(1 - p_0)]$ "an approximation to the squared total intensity (i^2) applied to the marker" (citing Liu & Woolliams 2010 for the underlying non-linearity), and where the only thing moving frequencies is truncation selection on a trait, that reading is theirs to make. But F_{drift} is accumulated over **every** source of displacement a scheme has – selection, pure drift, and a diversity constraint that pushes frequencies on its own – and the three are not separable from the statistic alone. Section 12.7.4 runs a crossing block with **no selection of any kind** and F_{drift} climbs over twelve cycles to a value the step reports, which under an unconditional intensity reading would be a large intensity applied by nothing; Section 12.7.2's `max_div` plan moves F_{drift} by minimising coancestry,

not by selecting on any trait. Even where selection is the only driver, the displacement is not a function of frequency alone: it depends on the average effect of a gene substitution at that locus, $a + d\,(q - p)$ [12, Ch. 7] – classically written α, which in this book is the diversity-loss statistic instead (Chapter 13. says the same) – and on the phenotypic standard deviation of the trait under selection, neither of which appears anywhere in F_{drift}. So an intensity read off `F_drift` alone, with no independent knowledge of what moved it, cannot be trusted, and this chapter does not attempt one. What constraining `F_drift` does constrain is the squared *displacement* of the genome from its base, whatever moved it.

A programme running six cycles of genomic selection with a VanRaden $\mathbf{G}$ and reporting $\Delta F \approx 1\%$ has reported ΔF_{drift} on the training panel. Quantitative disease-resistance loci not yet prioritised may have hitch-hiked to extreme frequencies alongside yield QTL, and none of that appears in the reported metric.

12.4 The classical equivalence, and the covariance that destroys it

Under purely random drift, with $E[\delta p_t \mid p_0] = 0$ for every p_0,

$$\frac{\mathrm{var}(p_{t,k})}{p_0(1-p_0)} = 1 - \frac{H_t}{H_0} \Rightarrow F_{drift} = F_{hom}. \tag{12.4}$$

The paper's central result quantifies the departure:

$$F_{hom} - F_{drift} = 2\,\mathrm{cov}\left(\frac{\delta p_{t,k}}{\sqrt{p_{0,k}(1-p_{0,k})}}, \frac{p_{0,k} - 1/2}{\sqrt{p_{0,k}(1-p_{0,k})}}\right) \tag{12.5}$$

[6, their Eq. 3], where $\delta p_{t,k} = p_{t,k} - p_{0,k}$ and the covariance is taken over markers with divisor N_{SNP} – which is why `freq_metrics()` computes it as `2 * mean(x * y)` and not with `cov()`. The two differ for two independent reasons: `stats::cov()` divides by $N_{SNP} - 1$, and $\overline{xy} = \mathrm{cov}_N(x, y) + \bar{x}\,\bar{y}$, so the product term vanishes only if *one of the two means* is zero – either will do, and neither is zero here.

In words: **if the direction of frequency change is correlated with the starting frequency, the two measures diverge.** The non-obvious part is that this covariance is induced by the *diversity management itself*, not by directional selection – T. H. E. Meuwissen, A. K. Sonesson, G. Gebregiwergis, and J. A. Woolliams [6] demonstrate this by running the scheme with random GEBVs, and the divergence persists.

12.4.1 Verifying the identity

`freq_metrics()` returns all three quantities in one marker sweep. The identity is asserted in `tests/testthat/test-lenses.R`; here it is on live data:

```
set.seed(31)
idx <- sample.int(ctx$N, 60)
fm <- freq_metrics(idx, ctx)
fmt(fm[c("F_hom", "F_drift", "cov_diag")], 6)
#>    F_hom  F_drift cov_diag
#> 0.011458 0.005192 0.006267
```

```
stopifnot(isTRUE(all.equal(fm[["cov_diag"]], fm[["F_hom"]] - fm[["F_drift"]])))
# The two standardised marker vectors of Eq. 3, on the markers polymorphic in
# the base, so `2 * mean(x * y)` can be put next to `2 * cov(x, y)`.
keep <- ctx$p0 > 1e-6 & ctx$p0 < 1 - 1e-6
sdv  <- sqrt(ctx$p0[keep] * (1 - ctx$p0[keep]))
x    <- (colMeans(ctx$X[idx, keep, drop = FALSE]) - ctx$p0[keep]) / sdv
y    <- (ctx$p0[keep] - 0.5) / sdv
stopifnot(isTRUE(all.equal(2 * mean(x * y), unname(fm[["cov_diag"]]))))
c(F_hom_minus_F_drift = unname(fm[["F_hom"]] - fm[["F_drift"]]),
  cov_diag            = unname(fm[["cov_diag"]]),
  residual          = unname(fm[["cov_diag"]] - (fm[["F_hom"]] - fm[["F_drift"]])),
  mean_x              = mean(x),
  mean_y              = mean(y),
```

```
  two_times_R_cov     = 2 * stats::cov(x, y))
#> F_hom_minus_F_drift            cov_diag            residual              mean_x
#>        6.266505e-03        6.266505e-03        8.673617e-18       -7.725594e-04
#>              mean_y     two_times_R_cov
#>       -1.005046e-02        6.254209e-03
```

Exact to machine precision, and the two `stopifnot()`s mean this page cannot render if it stops being. The last three entries are the point of the previous paragraph: neither marker-wise mean is zero, so `2 * cov(x, y)` returns 0.006254 where the quantity Eq. 3 defines is 0.006267. And when the selection *is* the base, both lenses must read zero:

```
fmt(freq_metrics(seq_len(ctx$N), ctx)[c("F_hom", "F_drift", "cov_diag")], 12)
#>    F_hom  F_drift cov_diag
#>        0        0        0
```

12.4.2 Why each matrix generates a sign

Matrix	Covariance	Mechanism
G_VR2	positive	the constraint penalises delta-p^2, so moving an allele to the *far* extreme costs more than to the near one; the optimiser pushes rare alleles to 0 and common ones to 1
G_0.5	negative	frequencies are measured as deviations from 0.5, so the optimiser buys 'diversity' by pushing everything to intermediate frequencies
G_VR1	positive, attenuated	G_VR2 reweighted by 2p0(1-p0), giving more weight to already-intermediate loci and less room at the extremes
G_i(p)	negative	uses accumulated intensity instead of delta-p^2; since di/dp = [p(1-p)]^-1/2, moves toward the extremes get more expensive

⚠ Your MAF spectrum decides how bad this is

T. H. E. Meuwissen, A. K. Sonesson, G. Gebregiwergis, and J. A. Woolliams [6] state this as an expectation rather than as a measured comparison: "the covariance generated is expected to be larger for WGS data where allele frequencies are extreme with typical MAF close to 0, than for SNP (chip) panels where allele frequencies are generally closer to 1/2" (their second Conclusion bullet), having observed that the strength of the covariance "depends on the distribution of $(p_0 - \frac{1}{2})$" (their Discussion). Their own simulation used four panels, all random samples of the same whole-genome sequence data (an SNP-BLUP panel, a neutral panel, a QTL panel and a second management panel) – none a SNP chip – so the comparison across a WGS panel and a chip panel is an inference from the algebra of Eq. 3 and not a result of their experiment either.

The algebra can be made a number, which is worth doing because it decides whether `G_VR2` is usable. Apply one deterministic management rule – move every frequency 5% of the way toward 0.5, the caricature of a heterozygosity-maximising constraint used again in Section 12.7.4 – to a sequence-like panel whose folded site-frequency spectrum has density proportional to $1/[p(1-p)]$, and then to MAF-filtered subsets of the *same* draw:

```
set.seed(21)
# Folded site-frequency spectrum with density proportional to 1/[p(1-p)] on
# [1e-3, 1 - 1e-3]: the logit of p is uniform. One draw, filtered four ways.
p_seq <- plogis(runif(2e5, qlogis(1e-3), qlogis(1 - 1e-3)))
maf_row <- function(floor, pull = 0.05) {
  p0 <- p_seq[pmin(p_seq, 1 - p_seq) >= floor]
  p1 <- p0 + pull * (0.5 - p0)                # the rule, applied exactly
  s2 <- p0 * (1 - p0)
  c(maf_floor = floor, markers = length(p0),
    W = mean((p0 - 0.5)^2 / s2),              # the panel's own weight
    F_hom = 1 - mean(p1 * (1 - p1) / s2),
    F_drift = mean((p1 - p0)^2 / s2),
    cov_diag = 2 * mean((p0 - 0.5) * (p1 - p0) / s2))
}
mafw <- as.data.frame(t(sapply(c(0.001, 0.01, 0.05, 0.20), maf_row)))
# Under this rule all three lenses are multiples of one panel statistic, W.
stopifnot(isTRUE(all.equal(mafw$F_drift, 0.05^2 * mafw$W)),
          isTRUE(all.equal(mafw$cov_diag, -2 * 0.05 * mafw$W)))
keep0  <- ctx$p0 > 1e-6 & ctx$p0 < 1 - 1e-6       # as freq_metrics() drops them
W_book <- mean((ctx$p0[keep0] - 0.5)^2 / (ctx$p0[keep0] * (1 - ctx$p0[keep0])))
fmt(mafw, 4)
```

maf_floor	markers	W	F_hom	F_drift	cov_diag
0.001	200000	35.7687	−3.4874	0.0894	−3.5769
0.010	132755	4.8542	−0.4733	0.0121	−0.4854
0.050	85382	1.1056	−0.1078	0.0028	−0.1106
0.200	40109	0.1761	−0.0172	0.0004	−0.0176

Under a rule of this form $\delta p = c\,(0.5 - p_0)$ the algebra is exact: $F_{drift} = c^2 W$ and `cov_diag` $= -2cW$ with $W = (p_0 - 0.5)^2/[p_0(1-p_0)]$, the panel statistic the `stopifnot()` above verifies. Everything about the panel enters through W, and W is 32.4 times larger at a MAF floor of 0.001 than at 0.05 on the same draw: `cov_diag` moves from −3.577 to −0.111 for the same 5% pull. For reference, this book's own hybrid panel has $W = 1.27$ over the 1934 markers polymorphic in the base, which sits at the chip-like end of that table – a consequence of the $U(0.05, 0.95)$ ancestral draw of `simulate_lines()`, not of anything biological.

Translated to plants, as a stated expectation and not as a measurement on real germplasm: GBS or skim-seq retaining the full spectrum puts a programme at the high-W end; a 20-50k chip filtered to MAF > 0.05 puts it at the low-W end. The rule above is a caricature, not an optimiser, so the *ratio* between panels is the transferable part and the absolute values are not.

12.5 The other matrices, briefly

`F_IBD` via `G_LA`. Instead of assuming a 50/50 transmission probability for each parental allele (as the pedigree matrix $\mathbf{A}$ does), linkage analysis over the markers infers which parental haplotype was actually transmitted along the chromosome. It requires pedigree **and** markers. It measures IBD relative to a base explicitly defined as unrelated and non-inbred, so frequency change is determined by the base population rather than by linkage disequilibrium generated during selection – and the problematic covariance disappears. It was the **only scheme tested that met both the ΔF_{hom} and ΔF_{drift} targets**, on the managed panel and at unmonitored neutral loci. On gain it was not the highest in absolute terms; what T. H. E. Meuwissen, A. K. Sonesson, G. Gebregiwergis, and J. A. Woolliams [6] report is that at the *same* averaged inbreeding – interpolating each scheme's gain to the mean of F_{hom} and F_{drift} – `G_LA` beat $\mathbf{A}$ in 65 of 100 replicates and `G_ROH` in 62 of 100 (their Genetic Gain section).

For a hybrid programme this is the point of greatest friction, and also the one where the news is best. In biparental populations and backcross families `G_LA` is natural and cheap – phase is known and the number of meioses since the base is small. In a maize recurrent-selection population with a complete recycling pedigree, which commercial programmes generally have because every DH line records its F1 parents, it is feasible. In a germplasm panel *without* pedigree it is inapplicable, and the honest alternative is segment-based IBD.

`F_ROH`. A run of homozygosity is an uninterrupted stretch of homozygous markers; the logic is that many consecutive markers are unlikely to be IBS by chance, so the stretch is probably IBD. The fundamental ambiguity is that `F_ROH` is IBD when runs are **long** (recent inbreeding) and homozygosity when they are **short** (ancient inbreeding, accumulated IBS). The minimum-length threshold is therefore a knob that selects the *age* of inbreeding you are measuring, and there is no well-defined reference base.

Empirically, `G_ROH` behaved more like `G_0.5` than like an IBD measure. It also is **not guaranteed positive semi-definite** – its elements are computed pairwise – and required substantial diagonal additions for inversion, which made it diagonally dominant and artificially inflated the number of parents selected.

In an inbred parent line the entire genome is one ROH and $F_{ROH} \approx 1$: the metric saturates for exactly the reason assumption A2 makes every individual-level coefficient useless. Applied instead to the *hybrids*, or to the segregating crossing block from which DH lines are induced, it is useful for diagnosing erosion of an elite pool. For *management* inside an optimiser, check positive definiteness carefully – Section 12.7.7 shows the check.

12.5.1 Side by side

Matrix	Class	Behaviour in the optimiser
A	IBD (expectation)	high gain, but exceeded targets by ~40% by not accounting for within-family LD
G_VR2	drift	meets the drift target *only on the managed panel*; F_hom runs away
G_VR1	drift	reweighted version; smaller bias, smaller gain
G_0.5	homozygosity	misses the drift target by the widest margin of the eight schemes; loses genetic variance early
G_i(p)	drift	inverts the sign of the bias; constraint only indirectly tied to F
G_LA	IBD	the only one meeting both targets; highest gain at equal averaged inbreeding
G_ROH	Homozygosity, read as IBD	high gain, uncontrolled drift, numerical problems

12.6 Reference results

Target $\Delta F = 0.005$ per generation ($N_e = 100$). Values at the **unmonitored neutral loci** (their Panel N), generation 20, as reported by T. H. E. Meuwissen, A. K. Sonesson, G. Gebregiwergis, and J. A. Woolliams [6]. Four columns are transcribed and one is derived:

- `dF_hom`, `dF_drift` and `gap` are the three Panel N columns of their Table 2. `gap` is the *level* $F_{hom} - F_{drift}$ reached in generation 20, not a rate and not a difference of the two rate columns; it is `cov_diag` of Eq. 3 accumulated over the 20 generations. Their reported standard errors are $< 2.5 \times 10^{-5}$ on the rates and $< 2.2 \times 10^{-4}$ on `gap`.

- `gain` is their Table 3: cumulative genetic gain after 20 generations, expressed in units of the **initial genetic standard deviation**, standard errors ≤ 0.004.
- `gain_per_dF` is computed here, not transcribed, and is **not** the device T. H. E. Meuwissen, A. K. Sonesson, G. Gebregiwergis, and J. A. Woolliams [6] use: their own interpolation (Genetic Gain section, their Figure 5) compares gains at a common generation-20 *level* of averaged inbreeding, $(F_{hom} + F_{drift})/2$ from their Table 3 – a stock – not at a common mean of the two *rate* columns used here. `gain_per_dF` is offered only as a same-chapter summary of the reference table above, and reading 1 below states explicitly which ordering it does and does not support.

Scheme	Class	dF_hom	dF_drift	gap	gain	gain_per_dF
G_VR2(M,M)	drift	0.0103	0.0068	0.054	7.124	833
G_VR2(M,D)	drift	0.0101	0.0068	0.050	7.107	841
G_VR1(M,M)	drift	0.0080	0.0069	0.021	6.680	897
G_i(p)(M,M)	drift	0.0065	0.0077	−0.031	7.111	1002
G_0.5(M,M)	homozygosity	0.0073	0.0176	−0.170	6.734	541
G_ROH(M,M)	homozygosity	0.0054	0.0088	−0.070	9.099	1282
G_LA(M,M)	IBD	0.0043	0.0049	−0.010	7.188	1563
A(M,~)	IBD	0.0070	0.0084	−0.021	9.890	1284

Four readings:

1. **No IBS scheme met both targets.** `G_LA` came closest: within about 2% of the 0.005 target on `dF_drift` on both panels M and N (their own figure for that lens), but 14% below target on `dF_hom` (0.0043 against 0.005) – the 2% is a drift-lens precision, not a joint one. It also has the largest `gain_per_dF`, 1563 against 1284 for $\mathbf{A}$ and 1282 for `G_ROH` – but that ordering is not robust to the denominator: against `dF_hom` alone `G_ROH` is marginally ahead (1685 against 1672). The claim that survives is the replicate-level one in reading 4, not a ratio of table means.
2. Pedigree $\mathbf{A}$ gave the highest absolute gain but overshot by ~40% – neutral loci hitch-hiking with QTL through within-family LD, which the pedigree expectation of IBD cannot capture.
3. Using a separate panel for management instead of the GEBV panel changed gain by **less than 0.3%**. Not worth the complexity as a *gain* measure; it is still worth it as an *audit* measure, which is a different use and is the subject of Section 12.7.5 below.
4. At equal mean inbreeding, `G_LA` beat $\mathbf{A}$ in 65 of 100 replicates and `G_ROH` in 62 of 100.

Those are animal-breeding numbers under the assumptions of Section 12.9, and none of them is reproduced here. What follows is.

12.7 The monitoring protocol

Seven steps, in the order a cycle report is assembled. Each one names the function to call, the quantity to check, and what a failure looks like.

12.7.1 Step 1 – Freeze p_0 at the founding cycle

Both lenses are ratios to a base, so the base is the first thing a cycle report has to pin down. In this book it is `ctx$p0`, the candidate-pool allele frequencies, set once in `build_ctx()` and carried forward unchanged by `run_cycles()`.

At the hybrid level the base is not a free choice: on the full factorial it is the average of the two pool bases, exactly.

```
stopifnot(isTRUE(all.equal(ctx$p0, as.vector((P0["A", ] + P0["B", ]) / 2))))
# Unequal pool sizes, still a complete factorial: each pool-A line appears in
# n_pool_B hybrids and each pool-B line in n_pool_A, so the internal weight
# stays uniform within each pool and the identity is untouched.
cfg_u <- utils::modifyList(book_cfg, list(n_pool_A = 10, n_pool_B = 40))
ctx_u <- simulate_data(cfg_u)
Pu    <- pool_freqs(simulate_lines(cfg_u) / 2,
                    rep(c("A", "B"), c(cfg_u$n_pool_A, cfg_u$n_pool_B)))
# A balanced INCOMPLETE factorial -- not the full n_A*n_B hybrids, but each
```

```
# line still used equally often within its own pool (a cyclic 1-to-3 design,
# the shape of a real NCII): the identity does not need completeness.
k <- 3
idx_bal <- unlist(lapply(seq_len(book_cfg$n_pool_A), function(a) {
  bpart <- ((a - 1 + 0:(k - 1)) %% book_cfg$n_pool_B) + 1
  which(ctx$ped$a == a & ctx$ped$b %in% (bpart + book_cfg$n_pool_A))
}))
p_bal <- colMeans(ctx$X[idx_bal, , drop = FALSE])
# An UNbalanced candidate set with concentrated line usage: it is not.
p_sub <- colMeans(ctx$X[sel_truncation(ctx, 60), , drop = FALSE])
c(complete_balanced     = max(abs(ctx$p0 - (P0["A", ] + P0["B", ]) / 2)),
  complete_unequal_pools = max(abs(ctx_u$p0 - (Pu["A", ] + Pu["B", ]) / 2)),
  incomplete_balanced   = max(abs(p_bal - (P0["A", ] + P0["B", ]) / 2)),
  truncation_subset     = max(abs(p_sub - (P0["A", ] + P0["B", ]) / 2)))
#>      complete_balanced complete_unequal_pools    incomplete_balanced
#>           1.110223e-16           1.110223e-16           1.110223e-16
#>      truncation_subset
#>           3.133333e-01
```

Each pool-A line appears in 25 hybrids and each pool-B line in 25, so a column mean over the factorial is $(p_A + p_B)/2$ with no weighting left over. Two consequences a breeder should read off this identity. First, freezing p_0 at the hybrid level freezes both pool bases *jointly*, so the hybrid-level lenses cannot attribute erosion to one pool – that is Section 12.7.6's job. Second – and this is the part that is easy to state wrongly – what the identity requires is **uniform line usage within each pool**, not completeness of the factorial as such. With 10 lines in one pool against 40 in the other, still complete, the identity holds to 1.1e-16. It holds just as exactly (1.1e-16) on a **balanced but incomplete** factorial – 75 of the 625 possible hybrids, each line used exactly 3 times, the shape of a real NCII design – because each pool's lines are still used equally often *within* that pool. What re-weights the two pools inside the base is unequal **line usage**, which a balanced partial factorial does not have and a selected subset typically does: the column means of the 60-hybrid truncation set depart from $(p_A + p_B)/2$ by up to 0.313 on a single marker, because truncation concentrates hybrids on a few high-value lines in each pool. A base computed on a set with unequal line usage is therefore a different base – which is the failure mode below, and not a property of the design being a factorial or not.

Now the failure mode. `build_ctx()` recomputes `p0 <- colMeans(X)` on every call, so rebuilding the context from the *selected* hybrids each cycle – which is what re-running a `G`-construction function per cycle amounts to – re-anchors the base on the population you were about to measure:

```
i_md   <- sel_greedy(ctx, 60, w = 0)
Xs     <- ctx$X[i_md, , drop = FALSE]
rebase <- build_ctx(X = Xs, f = molecular_coancestry(Xs), G = NULL,
                    traits = ctx$traits[i_md, , drop = FALSE],
                    ped = ctx$ped[i_md, ], n_lines = ctx$n_lines,
                    pool = ctx$pool, fL = ctx$fL, cfg = NULL)
fmt(rbind(frozen_base  = freq_metrics(i_md, ctx)[c("F_hom", "F_drift", "cov_diag")],
          rebased      = freq_metrics(seq_along(i_md), rebase)[c("F_hom", "F_drift",
"cov_diag")]), 6)
```

	F_hom	F_drift	cov_diag
frozen_base	−0.011338	0.006019	−0.017357
rebased	0.000000	0.000000	0.000000

Not attenuated – **identically zero, both lenses**. A rebased report is not a conservative report or a noisy one; it is a report that cannot detect erosion at all, and it will look clean for as many cycles as you run it.

The check has to be on the base itself, and it has to be the right check. If the base is stored per cycle, comparing each cycle's vector with cycle 0's catches the failure; collapsing the whole history into a `range()` does not, because `range()` discards the cycle dimension and returns the marker-wise spread of p_0, the same number whether the base moved or not – here 1, the whole unit interval, because 66 of the 2000 markers are fixed in the candidate pool:

```
chk <- function(h) {
  data.frame(
    all_identical_to_cycle0 = all(vapply(h, identical, logical(1), h[[1]])),
    max_abs_deviation       = max(vapply(h, function(p) max(abs(p - h[[1]])), 0)),
    collapsed_range         = diff(range(unlist(h))))
}
fmt(rbind(frozen  = chk(list(ctx$p0, ctx$p0, ctx$p0)),
          rebased = chk(list(ctx$p0, ctx$p0, rebase$p0))), 4)
```

	all_identical_to_cycle0	max_abs_deviation	collapsed_range
frozen	TRUE	0.0000	1
rebased	FALSE	0.1233	1

The first two columns separate the two histories; the third is the same number in both rows, which is the whole point.

Call. `build_ctx()` once at cycle 0; pass `p0` forward (`run_cycles()` does this through its internal `cycle_ctx()`). **Check.** `identical(ctx$p0, p0_cycle0)` at every cycle – one `TRUE`, with no tolerance. With a stored history, `max(vapply(p0_history, function(p) max(abs(p - p0_history[[1]])), 0))`, which must be exactly 0. Never a `range()` over the flattened history. **Warning sign.** F_{drift} near zero in a cycle where line usage was visibly concentrated, or an F_{drift} series that does not accumulate across cycles. **Threshold.** There is none to tune: the correct value of the base drift is exactly zero.

12.7.2 Step 2 – Compute both lenses in one marker sweep

`freq_metrics()` is the single sweep: it computes He, `alleles_lost`, F_{hom}, F_{drift}, `cov_diag`, `cor_dp_p0` and `rare_retained` from one `colMeans(X[idx, ])`. Anything else needing the selected frequencies belongs inside it, not in a second function with its own sweep.

The pure max-diversity greedy is the `G_0.5` scheme in miniature: it constrains molecular coancestry and nothing else. Its signature has to be read against a **resampled** baseline rather than against one random draw, for the reason the numbers below make explicit. The baseline is the package's own `null_distribution()`, 1000 resampled subsets of the same size, and the z-scores come from `discriminatory_power()`, so the comparison is the same machinery every scenario in Chapter 24. is reported through:

```
# `sels` is the set of four selections built in @sec-era-fhom.
cols   <- c("He", "F_hom", "F_drift", "cov_diag", "rare_retained")
null60 <- null_distribution(ctx, 60, B = 2000, B_full = 1000, seed = 7)
panel  <- t(sapply(sels, freq_metrics, ctx = ctx))[, cols]
fmt(rbind(null_mean = colMeans(null60$full[, cols]),
          null_sd   = apply(null60$full[, cols], 2, stats::sd),
          panel), 4)
```

	He	F_hom	F_drift	cov_diag	rare_retained
null_mean	0.3259	0.0069	0.0070	−0.0001	0.9583
null_sd	0.0008	0.0038	0.0014	0.0035	0.0229
random	0.3257	0.0115	0.0052	0.0063	0.9481
max_div	0.3302	−0.0113	0.0060	−0.0174	0.9481
balanced	0.3124	0.0426	0.0558	−0.0131	0.7338
truncation	0.3124	0.0426	0.0558	−0.0131	0.7338

```
z <- discriminatory_power(evaluate(sels, ctx, null60$gd_ref), null60)[, cols]
fmt(as.data.frame(z), 2)
```

	He	F_hom	F_drift	cov_diag	rare_retained
random	−0.25	1.20	−1.26	1.80	−0.45
max_div	5.58	−4.79	−0.67	−4.92	−0.45
balanced	−17.06	9.41	35.08	−3.72	−9.81
truncation	−17.06	9.41	35.08	−3.72	−9.81

Result. The max-diversity plan drives F_{hom} negative, −0.0113, which is 4.8 s.d. below a null mean of 0.0069: the selection is more heterozygous on the monitored panel than the frozen base. Its `cov_diag` is −0.0174, 4.9 s.d. below a null centred on −0.0001. Its F_{drift}, however, is 0.006 against a null mean of 0.007: a z of −0.67, and 76% of the 1000 resampled subsets pay **more** drift than it does. Truncation is the row with the drift bill: F_{drift} = 0.0558, z = 35.1.

This is where one draw would have misled. The `random` row is a single subset, and its F_{drift} sits 1.3 s.d. below the resampled mean – *further* below it than `max_div` is – so a two-row `random`-versus-`max_div` comparison reports max-diversity as paying more drift than random, which the 1000-subset null says it does not. The failure is not the metric, it is the baseline: with a null s.d. of 0.0014 on a quantity whose value is 0.006, no single draw can serve as a control.

Interpretation, and what it does not license. At this scale, on a single selection from a fixed candidate set, **minimising** molecular coancestry (the `max_div` plan, `w = 0`, which maximises gene diversity `GD = 1 − theta`) buys a measurable divergence between the two lenses – the negative covariance that the `G_0.5` row of the matrix-sign table predicts – and **no** measurable drift excess over random sampling. The drift excess T. H. E. Meuwissen, A. K. Sonesson, G. Gebregiwergis, and J. A. Woolliams [6] report for `G_0.5` ($\Delta F_{drift} = 0.0176$ against a 0.005 target) is a 20-generation *cumulative* result under recurrent optimal contributions, where each cycle's frequency change compounds on the last; one subset drawn once from one candidate pool is not that experiment, and the honest way to run it here is the multi-cycle arm of Section 12.7.4, where the same pull toward 0.5 does produce drift at more than twice the neutral rate. What survives at one cycle is the covariance, and a report carrying only F_{hom} would not show it.

One thing the table quietly says about the *weight* rather than the metric:

```
identical(sort(sel_greedy(ctx, 60, w = 1)), sort(sel_truncation(ctx, 60)))
#> [1] TRUE
```

At this scale `w = 1` is not a compromise between index and coancestry – it returns the truncation set exactly, because the index is standardised to unit variance (`sd(ctx$index)` = 1) while the spread of θ across candidate subsets of the same size is 7.58e-04, the standard deviation of the `theta` column over the 2000 cheap-metric resamples of the same null. That is a ratio of 1319: three orders of magnitude, not two. A linear weight has to be calibrated to that ratio, which is why `sel_greedy_budget()` sweeps a *logarithmic* grid of `w` instead of trusting any single value.

Call. `freq_metrics(idx, ctx)` per cycle; `metrics_cheap()` if you also need the coancestry block, `metrics_full()` post-hoc for the expensive ones. **Check.** Both lenses present in the report, side by side, with the sign of F_{hom} preserved (never take an absolute value or clamp at zero). **Warning sign.** A single "inbreeding" column in the cycle report. If you cannot tell which lens it is, it is the one your constraint used, and the other one is unmeasured. **Threshold.** Not the ΔF target. Both columns here are *stocks* accumulated against the frozen base, and a stock cannot be compared with a rate – Section 12.7.4 opens on exactly that point. Within one cycle the reference is the resampled null in the same table, i.e. the z columns above, read on **both** lenses separately and never on their average. The comparison with the programme's ΔF budget requires a trajectory and happens only through the regression of Section 12.7.4.

12.7.3 Step 3 – Report the covariance, not just the two lenses

`cov_diag` is the gap between the two lenses, and by Eq. 3 it is not an approximation of the gap – it *is* the gap, which the `stopifnot()` above enforces on every render. The operational point is that `cov_diag` is the only column that tells you *why* the two lenses disagree, and its sign identifies the mechanism from the matrix-sign table above: negative means the management pushed frequencies toward 0.5, positive means it pushed them to the extremes.

Call. `freq_metrics(idx, ctx)[["cov_diag"]]`, already computed by step 2 at no extra cost; `cor_dp_p0` from the same call is the scale-free version. **Check.** `cov_diag == F_hom − F_drift` to machine precision. If it does not, the base and the frequencies were computed on different marker sets. **Warning sign.** |`cov_diag`| of the same order as the stock it is meant to explain. In the panel of Section 12.7.2 the max-diversity plan reports `cov_diag` = −0.0174 against an F_{drift} of 0.006: the discrepancy between the two lenses is 2.9 times the quantity being reported. **Threshold.** No absolute one, and not the ΔF budget – `cov_diag` is a stock like the two lenses it separates. Compare it with F_{drift} in the same row, as just done, and with the resampled null: the `null_sd` row of Section 12.7.2's table puts the max-diversity plan 4.9 s.d. out, which is the statement that the covariance was induced by the management rather than by the sampling.

12.7.4 Step 4 – Regress $\log(1-F)$ on cycle, and read R^2

A single cycle's F is a stock and cannot be compared to a rate target. The rate comes from the trajectory. Before trusting the regression on a real programme, calibrate it on a case whose answer is known.

Let n_{gam} be the number of allele copies the pool samples per cycle. Drawing them at random from the previous cycle's frequencies gives $\mathbb{E}[H_t] = H_0\,(1-1/n_{gam})^t$ exactly, so $\Delta F = 1/n_{gam}$ and $N_e = n_{gam}/2$; with $n_{gam} = 2N$ this is the idealised population of [12, Ch. 3] and $\Delta F = 1/(2N)$. In a two-pool programme n_{gam} is not a matter of taste, and the two natural regimes differ by a factor of two:

- a **crossing block** of n plants intermated at random passes $2n$ gametes, so $\Delta F = 1/(2n)$ and $N_e = n$;
- a pool **re-derived as n doubled-haploid lines** passes only n distinct allele copies, because each DH is homozygous and contributes one allele identity per locus, so $\Delta F = 1/n$ and $N_e = n/2$. This is literally what `next_pool()` does in Chapter 22.: one Bernoulli draw per locus per new line.

```
# One heterotic pool, n_gam allele copies sampled per cycle, n_cyc cycles,
# `reps` independent replicates. `pull` first moves every frequency a fixed
# fraction toward 0.5: a caricature of a heterozygosity-maximising constraint,
# not an optimiser.
pool_traj <- function(n_gam, m, n_cyc, pull = 0, reps = 20, seed = 11) {
  out <- vector("list", reps)
  for (r in seq_len(reps)) {
    set.seed(seed + r)
    p0 <- runif(m, 0.05, 0.95)
    s2 <- p0 * (1 - p0)
    p  <- p0
    rows <- vector("list", n_cyc + 1L)
    for (t in 0:n_cyc) {
      rows[[t + 1L]] <- data.frame(
        rep = r, cycle = t,
        F_hom    = 1 - mean(p * (1 - p) / s2),
        F_drift  = mean((p - p0)^2 / s2),
        cov_diag = 2 * mean((p0 - 0.5) * (p - p0) / s2))
      p <- rbinom(m, n_gam, p + pull * (0.5 - p)) / n_gam
    }
    out[[r]] <- do.call(rbind, rows)
  }
  do.call(rbind, out)
}

per_cycle <- function(tr, n_gam) {
  a <- aggregate(cbind(F_hom, F_drift, cov_diag) ~ cycle, tr, mean)
  a$expected <- 1 - (1 - 1 / n_gam)^a$cycle
  # One s.d. per lens: the two lenses are not equally noisy across replicates,
  # so a deviation has to be compared with the s.d. of the lens it came from.
  a$sd_hom   <- aggregate(F_hom ~ cycle, tr, sd)$F_hom
  a$sd_drift <- aggregate(F_drift ~ cycle, tr, sd)$F_drift
  a
}

# dF from the slope, not from its first-order approximation.
dF_fit <- function(a, col) {
  d <- a[a$cycle > 0, ]                # cycle 0 is identically zero by construction
  f <- lm(log(1 - d[[col]]) ~ cycle, d)
  b <- coef(f)[["cycle"]]
  c(dF = 1 - exp(b), Ne = 1 / (2 * (1 - exp(b))), R2 = summary(f)$r.squared)
}

n_line <- 50
n_cyc  <- 12
block  <- per_cycle(pool_traj(2 * n_line, book_cfg$m, n_cyc), 2 * n_line)
```

```
dh     <- per_cycle(pool_traj(n_line, book_cfg$m, n_cyc), n_line)
fmt(block, 5)
```

cycle	F_hom	F_drift	cov_diag	expected	sd_hom	sd_drift
0	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000
1	0.01014	0.00992	0.00022	0.01000	0.00466	0.00046
2	0.02032	0.01971	0.00061	0.01990	0.00554	0.00065
3	0.02978	0.02963	0.00015	0.02970	0.00643	0.00099
4	0.03787	0.03898	−0.00111	0.03940	0.00772	0.00089
5	0.04759	0.04874	−0.00116	0.04901	0.00764	0.00112
6	0.05676	0.05802	−0.00126	0.05852	0.00852	0.00142
7	0.06557	0.06784	−0.00227	0.06793	0.00878	0.00179
8	0.07405	0.07722	−0.00317	0.07726	0.00918	0.00205
9	0.08414	0.08667	−0.00253	0.08648	0.00961	0.00293
10	0.09459	0.09600	−0.00141	0.09562	0.00995	0.00289
11	0.10342	0.10503	−0.00161	0.10466	0.01029	0.00284
12	0.11166	0.11411	−0.00246	0.11362	0.01160	0.00310

The `expected` column is the closed form, not a fit. Both lenses agree with it and with each other, in both regimes:

```
dev <- function(a) max(abs(c(a$F_hom, a$F_drift) - rep(a$expected, 2)))
fit_n <- rbind(block_F_hom   = dF_fit(block, "F_hom"),
               block_F_drift = dF_fit(block, "F_drift"),
               dh_F_hom      = dF_fit(dh, "F_hom"),
               dh_F_drift    = dF_fit(dh, "F_drift"))
fmt(rbind(crossing_block = c(target_dF = 1 / (2 * n_line), target_Ne = n_line,
                             max_deviation = dev(block),
                             sd_hom_final = block$sd_hom[nrow(block)],
                             sd_drift_final = block$sd_drift[nrow(block)]),
          dh_pool        = c(1 / n_line, n_line / 2, dev(dh),
                             dh$sd_hom[nrow(dh)], dh$sd_drift[nrow(dh)])), 5)
```

	target_dF	target_Ne	max_deviation	sd_hom_final	sd_drift_final
crossing_block	0.01	50	0.00321	0.01160	0.00310
dh_pool	0.02	25	0.00522	0.01136	0.00658

```
fmt(fit_n, 5)
```

	dF	Ne	R2
block_F_hom	0.00979	51.09840	0.99935
block_F_drift	0.01007	49.65000	0.99996
dh_F_hom	0.01941	25.76038	0.99992
dh_F_drift	0.01987	25.16253	0.99998

For the crossing block the fitted rates are 0.00979 and 0.01007 against a target of 0.01, giving N_e of 51.1 and 49.7 against the true 50. For the DH-derived pool of the same 50 lines they are 0.01941 and 0.01987 against a target of 0.02, giving $N_e \approx 25$ against the true 25. In both regimes the largest deviation of either lens from the closed form (0.0032 and 0.0052) falls on the homozygosity lens, and is within *its* between-replicate standard deviation at the last cycle (0.0116 and 0.0114, 20 replicates); the drift lens is the tighter of the two on both counts, deviating by at most 0.0005 and 0.0011. $R^2 \geq 0.999$ on all four fits.

Two things to carry out of that. The regression recovers N_e to within the sampling noise of 20 replicates of 2000 loci, which is what licenses the R^2 threshold used below – *at this scale only*: 20 replicates, 2000 loci, 12 cycles, one pool, one seed family. And a pool maintained as doubled-haploid lines drifts at twice the rate of a crossing block of the same count, so a programme that recycles 50 DH lines per pool per cycle is running at $N_e \approx 25$, not 50. That factor of two is structural, not a simulation artefact: it is the homozygosity of the DH itself.

A third thing, which is what the F_{drift} section above needed and could not compute at that point in the chapter. At cycle 12 the unmanaged crossing block stands at $F_{drift} = 0.1141$ and the DH pool at 0.2142, with **no selection applied in any cycle of either arm**: `pool_traj(pull = 0)` is a binomial resampling of the previous cycle's frequencies and nothing else. A statistic that reaches those values under pure drift is not an accumulated selection intensity, and reading it as one would attribute an intensity to a process that has none.

Now the crossing block again with the caricature constraint switched on: every frequency moved 5% of the way toward 0.5 before each drift step, which is what a heterozygosity-maximising objective does to the frequencies without needing an optimiser to do it.

```
managed <- per_cycle(pool_traj(2 * n_line, book_cfg$m, n_cyc, pull = 0.05),
                     2 * n_line)
fmt(managed[, c("cycle", "F_hom", "F_drift", "cov_diag", "sd_drift")], 5)
```

cycle	F_hom	F_drift	cov_diag	sd_drift
0	0.00000	0.00000	0.00000	0.00000
1	−0.05218	0.01222	−0.06440	0.00040
2	−0.09793	0.02661	−0.12454	0.00071
3	−0.13887	0.04265	−0.18153	0.00104
4	−0.17648	0.06062	−0.23710	0.00163
5	−0.20992	0.07989	−0.28981	0.00220
6	−0.24216	0.10079	−0.34295	0.00276
7	−0.26926	0.12213	−0.39139	0.00384
8	−0.29260	0.14298	−0.43558	0.00448
9	−0.31282	0.16482	−0.47764	0.00502
10	−0.33217	0.18661	−0.51878	0.00669
11	−0.35098	0.20884	−0.55982	0.00675
12	−0.36562	0.23091	−0.59653	0.00808

```
stopifnot(isTRUE(all.equal(managed$cov_diag, managed$F_hom - managed$F_drift)))
fit_m <- rbind(F_hom = dF_fit(managed, "F_hom"), F_drift = dF_fit(managed,
"F_drift"))
fmt(fit_m, 5)
```

	dF	Ne	R2
F_hom	−0.02335	−21.41636	0.96037
F_drift	0.02276	21.97079	0.99243

```
arms <- list(`crossing block` = block, `DH pool` = dh, `pull toward 0.5` = managed)
op <- par(mfrow = c(1, 3), mar = c(4, 4, 2.5, 1))
for (nm in names(arms)) {
  a <- arms[[nm]]
  plot(a$cycle, a$F_drift, type = "b", pch = 19, col = "#b2182b",
       ylim = c(-0.42, 0.26), xlab = "cycle", ylab = "F against frozen p0",
       main = nm)
  lines(a$cycle, a$F_hom, type = "b", pch = 17, col = "#1f5f8b")
  lines(a$cycle, a$expected, lty = 2, col = "grey40")
  abline(h = 0, col = "grey70")
  legend("bottomleft", c("F_drift", "F_hom", "closed form"), bty = "n",
         cex = 0.75, pch = c(19, 17, NA), lty = c(NA, NA, 2),
         col = c("#b2182b", "#1f5f8b", "grey40"))
}
par(op)
```

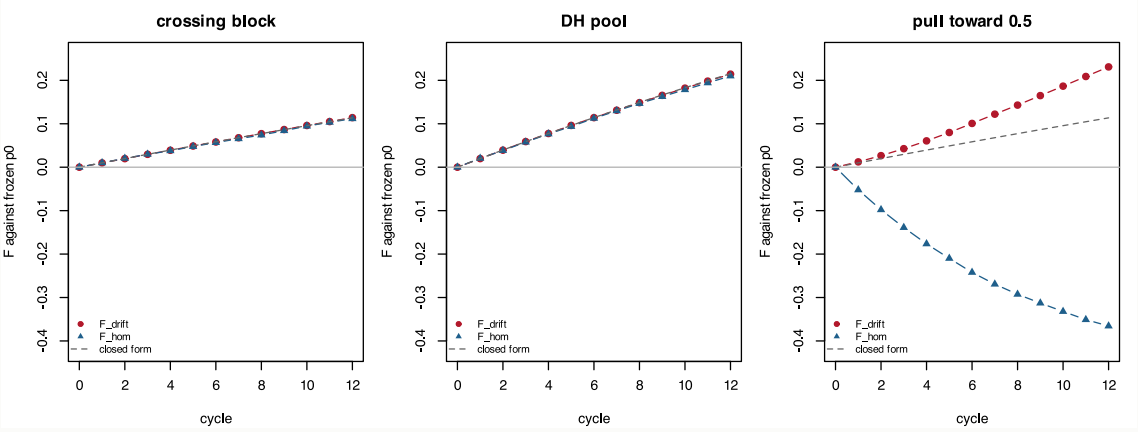

Figure 12.1: The two lenses on one heterotic pool over 12 cycles, averaged over 20 replicates, against a base frozen at cycle 0. Left: a crossing block of 50 plants (100 gametes per cycle). Centre: the same 50 lines re-derived as doubled haploids (50 allele copies per cycle), which drifts at twice the rate. In both, the closed form 1 - (1 - 1/n_gam)^t is the dashed line and the two lenses sit on it. Right: the crossing block with every frequency moved 5% toward 0.5 before each drift step, where the homozygosity lens reports a gain of heterozygosity against the frozen base while the drift lens reports more than twice the neutral rate; the dashed line is the same neutral closed form, here a reference both lenses leave. Nothing is selected in any panel; the divergence in the right-hand panel is produced by the management rule alone.

Figure 12.1 puts the three arms side by side. At cycle 12 the managed pool reports $F_{hom} = -0.366$ and $F_{drift} = 0.231$, a gap of −0.597 which is exactly `cov_diag`. The fitted drift rate is 0.0228, or 2.26 times the unmanaged crossing-block rate, implying $N_e = 22$ instead of 50. The fit on $\log(1 - F_{hom})$ returns a **negative** rate, −0.0233, and therefore a negative N_e, which is the arithmetic telling you the series it was handed is not an inbreeding trajectory at all. R^2 falls to 0.96 on that lens against 0.99935 in the unmanaged arm.

The inference, and its limits: a rule that pushes frequencies toward 0.5 generates exactly the negative covariance that the `G_0.5` row of the matrix-sign table predicts, and it does so with no selection whatsoever, which is the point T. H. E. Meuwissen, A. K. Sonesson, G. Gebregiwergis, and J. A. Woolliams [6] make with random GEBVs. What this arm does **not** establish is the magnitude – `pull = 0.05` is a stipulated caricature, not a fitted optimiser, and the factor of 2.26 would move with `pull`, with n_{gam} and with the ancestral frequency distribution. The direction is algebra; the size is this parameterisation.

Call. `lm(log(1 - F) ~ cycle)` on cycles $1..t$, separately for each lens; $\Delta F = 1 - e^{\text{slope}}$; Chapter 22. does this on `run_cycles()` output. **Check.** R^2 on both fits, and the sign of ΔF. **Warning sign.** A negative fitted ΔF (the series is not an inbreeding series), or an R^2 that is high while the two lenses disagree in sign – a smooth curve through the wrong quantity. **Threshold.** Calibrated above: a neutral trajectory at this scale gives $R^2 \geq 0.999$, so treat $R^2 < 0.995$ on an averaged series as a non-constant rate. R^2 says *that* the rate is not constant, not *why*: a decelerating, saturating series depresses it too, and Chapter 22. shows that case – there the diagnostic fires on the **unconstrained** run.

12.7.5 Step 5 – Audit on loci that did not enter the management

The constraint is met on the panel it is computed on. Whether it is met anywhere else is a separate measurement, and it needs loci the optimiser never saw. Split the marker panel in two, manage on one half, and read the report on the other:

```
mgmt  <- seq(1, ctx$m, by = 2)
audit <- seq(2, ctx$m, by = 2)
sub_ctx <- function(k) {
  build_ctx(X = ctx$X[, k], f = molecular_coancestry(ctx$X[, k]), G = NULL,
            traits = ctx$traits, ped = ctx$ped, n_lines = ctx$n_lines,
            pool = ctx$pool, fL = ctx$fL, cfg = NULL)
}
c_mgmt <- sub_ctx(mgmt)
c_audit <- sub_ctx(audit)
```

```
# The base of each half is the corresponding slice of the full base, and the
# index is untouched: only the marker set differs.
stopifnot(isTRUE(all.equal(c_mgmt$p0, ctx$p0[mgmt])),
          isTRUE(all.equal(c_mgmt$index, ctx$index)))

i_div <- sel_greedy(c_mgmt, 60, w = 0)     # maximise diversity on the mgmt half
i_bal <- sel_greedy(c_mgmt, 60, w = 1)
cols <- c("He", "F_hom", "F_drift", "cov_diag")
aud <- rbind(max_div_managed = freq_metrics(i_div, c_mgmt)[cols],
             max_div_audit   = freq_metrics(i_div, c_audit)[cols],
             index_managed   = freq_metrics(i_bal, c_mgmt)[cols],
             index_audit     = freq_metrics(i_bal, c_audit)[cols])
fmt(as.data.frame(aud), 4)
```

	He	F_hom	F_drift	cov_diag
max_div_managed	0.3333	−0.0300	0.0117	−0.0417
max_div_audit	0.3228	0.0221	0.0124	0.0097
index_managed	0.3120	0.0452	0.0580	−0.0128
index_audit	0.3129	0.0401	0.0536	−0.0135

```
gaps <- rbind(max_div = aud["max_div_audit", ] - aud["max_div_managed", ],
              index   = aud["index_audit", ]   - aud["index_managed", ])
fmt(as.data.frame(gaps[, c("F_hom", "F_drift")]), 4)
```

	F_hom	F_drift
max_div	0.0521	0.0007
index	−0.0051	−0.0043

Result. The plan that maximises diversity on the management half reports $F_{hom} = -0.0300$ there and $F_{hom} = 0.0221$ on the half it never saw: the sign reverses. Its drift readings are 0.0117 and 0.0124 – practically the same number on both halves, a gap of 0.0007 against 0.0521 on the homozygosity lens. The index-weighted plan, which optimises nothing about heterozygosity, has an F_{hom} gap of only −0.0051.

Interpretation. The homozygosity lens does not generalise off the panel it was maximised on, while the drift lens does; the gap is a property of having maximised, not of the marker set, which is why it nearly vanishes in the index-weighted row. Two limits on that inference. First, in this simulation loci are in linkage equilibrium within pool (assumption A5), so the two halves are as independent as a split panel can be. With real linkage disequilibrium the audit half is correlated with the management half, the measured gap shrinks, and the audit becomes *less* sensitive: the gap measured here is close to an upper bound on what a real split panel would report, not a typical value. Second, this is a one-cycle, single-seed measurement at the teaching scale; the cycle-by-cycle version of the same audit is Chapter 22..

Call. Two contexts over disjoint marker sets, both anchored on slices of the same frozen p_0; `freq_metrics()` on each. **Check.** The F_{hom} and F_{drift} gaps between managed and audit halves. **Warning sign.** A sign reversal in F_{hom}, as above. Also a *large* gap in F_{drift}, which would mean the management is not even controlling drift off its own panel. **Threshold.** The gap should be within the sampling noise of the split. A cheap empirical reference: the `index` row of the same table, which had no heterozygosity objective at all.

12.7.6 Step 6 – Read the two pools separately, and never F_{ST} alone

Everything so far is a single-population reading. A hybrid programme has two, and they must be constrained in opposite directions: within pool, limit ΔF; between pools, *preserve* divergence, because that is the heterosis engine (Chapter 13. proves the identity, Chapter 16. Trap 3 states the prescription). A single global $\frac{1}{2}\mathbf{c}'\mathbf{G}\mathbf{c}$ cannot express that.

```
pool_row <- function(i) {
  c(GD = gene_diversity(i, ctx), index = mean(ctx$index[i]),
    theta_pools(i, ctx)[c("theta_A", "theta_B")],
    gd_partition(i, ctx)[c("GD_T", "GD_BS", "F_ST")],
```

```
    ne_lines_pool(i, ctx), Ne_parents = ne_parents(i, ctx),
    max_line = max_line_use(i, ctx))
}
set.seed(31)
pr <- rbind(random     = pool_row(sample.int(ctx$N, 60)),
            truncation = pool_row(sel_truncation(ctx, 60)),
            max_div    = pool_row(sel_greedy(ctx, 60, w = 0)),
            capped     = pool_row(sel_greedy(ctx, 60, w = 1, max_use = 4)))
stopifnot(isTRUE(all.equal(unname(pr[, "GD"]), unname(pr[, "GD_T"]))))
fmt(as.data.frame(pr), 4)
```

	GD	index	theta_A	theta_B	GD_T	GD_BS	F_ST	Ne_lines_A	Ne_lines_B	Ne_parents	max_line
random	0.3257	−0.3178	0.7089	0.7108	0.3257	0.0355	0.1090	19.1489	20.2247	39.3443	5
truncation	0.3124	1.9919	0.7581	0.7219	0.3124	0.0524	0.1678	4.4888	11.8421	13.0199	23
max_div	0.3302	0.1343	0.7053	0.7066	0.3302	0.0361	0.1095	18.9474	18.3673	37.3057	5
capped	0.3248	0.8204	0.7096	0.7153	0.3248	0.0372	0.1146	15.5172	15.5172	31.0345	4

Two gene-diversity columns sit in that table and they are the **same** quantity reached by two routes, which the `stopifnot()` in the chunk enforces on every render: `GD` is $1 - \theta$ from $\mathbf{f}$ over the selected hybrids (Chapter 14.), `GD_T` is $1 - \bar{f}$ from the line-usage weights of the Caballero & Toro partition, and both reduce to $\bar{p}_k^2 + (1 - \bar{p}_k)^2$ with $\bar{p}$ the usage-weighted merged frequency, because under A2 a hybrid's marker row is the average of its two parents'. It is the same identity as $He = GD_T$ in Section 12.3.2, and `GD`, `GD_T` and Nei's He are one number on this germplasm. The columns that *are* a different quantity are GD_{BS} and F_{ST}: both are built from $\tilde{f} = (\theta_A + \theta_B)/2$, which no hybrid-level average can reproduce, and both are read below.

Result. Truncation on the index costs 4.1% of gene diversity against the random baseline – which is the α of Chapter 14., computed here on a single random draw rather than on the resampled `gd_ref` – and 5.4% against the max-diversity ceiling. Meanwhile θ_A rises to 0.7581 against $\theta_B = 0.7219$, and the effective number of lines used falls to 4.49 in pool A against 11.84 in pool B. One line appears in 23 of the 60 hybrids.

The dangerous column is F_{ST}. It *rises* from 0.1095 to 0.1678 under truncation, and so does GD_{BS}. Read as divergence, that looks like a gain in heterosis potential. It is not: by the Caballero & Toro definitions `gd_partition()` implements [1, Eq. 7], $GD_{BS} = \tilde{f} - \bar{f}$ with $\tilde{f} = (\theta_A + \theta_B)/2$ and $\bar{f} = (\theta_A + 2\theta_{AB} + \theta_B)/4$, so concentrating usage inside *one* pool raises $\tilde{f}$ twice as fast as it raises $\bar{f}$ and therefore raises the between-pool component – even though the two pools' lines are unchanged and their coancestry matrix $\mathbf{f}^L$ is fixed. All that moved is the weight the selection puts on each line. F_{ST} then divides by a GD_T that fell at the same time – the `GD_T` column of the table above, from 0.3302 to 0.3124 – which amplifies the artefact: the numerator rises by 45% while the denominator falls by 5%. A within-pool collapse and a genuine divergence are indistinguishable in F_{ST}, and only the separate θ_A, θ_B columns tell them apart.

The limitation to state here rather than hide: `freq_metrics()` computes its two lenses on `ctx$X` against `ctx$p0`, and Section 12.7.1 showed that hybrid base to be $(p_A + p_B)/2$. So F_{hom} and F_{drift} as this package computes them are **pooled over the two pools** and cannot, on their own, tell you which pool eroded. The pool-level attribution comes from three other places: θ_A and θ_B (`theta_pools()`), the effective number of lines used per pool (`ne_lines_pool()`), and the pool-level panel `s1_monitor()` returns from the line matrix:

```
s1 <- s1_monitor(L / 2, pool, cycle = 0)
fmt(s1, 4)
```

cycle	GD_within	GD_total	Fst_Wright	Gst_Nei	Ns_total	GD_A	GD_B	frac_opposite_fixed	mean_abs_delta_p	mean_sq_delta_p
0	0.2945	0.3281	0.1858	0.1024	0.7442	0.2953	0.2937	0	0.1982	0.0672

The two panels are the same quantities computed from the two ends of the contract, and on the full factorial – where line usage is uniform – they agree exactly:

```
gp_full <- gd_partition(seq_len(ctx$N), ctx)
stopifnot(isTRUE(all.equal(gp_full[["GD_T"]], s1$GD_total)),
```

```
          isTRUE(all.equal(gp_full[["F_ST"]], s1$Gst_Nei)))
c(gd_partition_GD_T = gp_full[["GD_T"]], s1_GD_total = s1$GD_total,
  gd_partition_F_ST = gp_full[["F_ST"]], s1_Gst_Nei = s1$Gst_Nei,
  s1_Fst_Wright = s1$Fst_Wright)
#> gd_partition_GD_T       s1_GD_total gd_partition_F_ST        s1_Gst_Nei
#>         0.3280928         0.3280928         0.1024003         0.1024003
#>     s1_Fst_Wright
#>         0.1857770
```

Note which two agree. `gd_partition()`'s `F_ST` is GD_{BS}/GD_T, which is the Nei G_{ST} form, and it equals `s1_monitor()`'s `Gst_Nei` to machine precision; `Fst_Wright` is the other definition and is 1.81 times larger on the same germplasm. That is Chapter 16. Trap 2, and it is the reason to name the definition in the report rather than the symbol.

Call. `theta_pools()`, `gd_partition()`, `ne_lines_pool()`, `max_line_use()`, `ne_parents()` on the selection; `s1_monitor()` on the line matrix of each cycle. **Check.** θ_A and θ_B separately; GD_{BS} held in a band, not maximised; the number of distinct lines used per pool. **Warning sign.** F_{ST} rising while GD_T falls – almost always a one-pool collapse, not divergence. An `Ne_lines` asymmetry between pools of the size in the `truncation` row above (4.5 against 11.8). **Threshold.** A per-line usage cap is the direct instrument: the `capped` row is `sel_greedy(ctx, 60, w = 1, max_use = 4)`, which holds $|\theta_A - \theta_B|$ at 0.0056 and both pools above 15.5 effective lines, against an index of 0.82 where truncation reached 1.99. That index difference is the price, and Chapter 22. is where it is paid back or not. `max_use` defaults to `NULL` everywhere in this package, so the cap is opt-in.

The diagnostic nobody runs. Schemes based on IBD selected many parents in T. H. E. Meuwissen, A. K. Sonesson, G. Gebregiwergis, and J. A. Woolliams [6]; VanRaden-style schemes selected few. If your optimal-contribution run is delivering high gain with a suspiciously small number of parents and ΔF on target, compute F_{hom} and F_{drift} separately on a panel **not used** in the management (Section 12.7.5). The difference between them is your hidden bill. This book's implementation is `ne_parents()`, which Chapter 24. reports alongside the two lenses for every scenario. The `era–pools` table above is the small version of the same diagnostic: between the `random` and `truncation` rows gene diversity falls by 4.1% while the effective number of parents falls from 39.3 to 13.0, which is 67% – the two relative drops differ by a factor of 16. A budget written on gene diversity alone is therefore a weak constraint on the parent count, and the `capped` row – the same greedy with `max_use = 4` – is what holding the parent count directly costs: 31.0 effective parents at an index of 0.82 where truncation reached 1.99. Measured on one candidate set at the teaching scale, with a single random draw as the `random` row.

12.7.7 Step 7 – Check the matrix you are about to invert

> **i** An implementation note that is routinely ignored

`G_VR1`, `G_VR2`, `G_0.5` and `G_i(p)` are all only **semi**-definite positive, so every one of them needs α added to the diagonal ($\alpha = 0.01$ in T. H. E. Meuwissen, A. K. Sonesson, G. Gebregiwergis, and J. A. Woolliams [6], their Methods) before it can be inverted and before the optimal-contribution algorithm will run. $\mathbf{A}$ and `G_LA` are positive definite by construction.

The mechanism is not the same for all four, and the difference matters here. For the centred VanRaden forms it is the centring: $\mathbf{Z}$ is centred on the *population's own* allele frequencies, so its columns sum to zero over individuals, the constant vector lies in the null space and one degree of freedom is gone by construction. For `G_0.5`, which is the molecular coancestry this book calls $\mathbf{f}$, it is **not** the centring – $\mathbf{f}$ is not centred on anything and stays in $[0, 1]$ by design (Chapter 1.). Its deficiency has a different cause, derived below. Both statements are checkable:

```
Zh  <- ctx$X - 0.5
rkf <- qr(ctx[["f"]])$rank
rkG <- qr(ctx[["G"]])$rank
c(f_equals_half_plus_gram = max(abs(ctx[["f"]] -
                                    (2 * tcrossprod(Zh) / ctx$m + 0.5))),
  f_rowsum_min            = min(rowSums(ctx[["f"]])),
  f_rowsum_max            = max(rowSums(ctx[["f"]])),
  G_sum                   = sum(ctx[["G"]]),
  G_rowsum_max_abs        = max(abs(rowSums(ctx[["G"]]))),
  rank_f = rkf, rank_G = rkG, n_lines = ctx$n_lines, N = ctx$N)
#> f_equals_half_plus_gram            f_rowsum_min            f_rowsum_max
#>            1.110223e-16            4.159937e+02            4.246250e+02
#>                   G_sum        G_rowsum_max_abs                  rank_f
#>           -4.178879e-11            1.163340e-13            4.900000e+01
#>                  rank_G                n_lines                       N
#>            4.800000e+01            5.000000e+01            6.250000e+02
```

$\mathbf{f} = 0.5 + \frac{2}{m}\mathbf{Z}_h\mathbf{Z}_h'$ with $\mathbf{Z}_h = \mathbf{X} - 0.5$, to machine precision – a Gram matrix plus the positive rank-one term $0.5\,\mathbf{1}\mathbf{1}'$, hence positive semi-definite, and with row sums nowhere near zero (`f_rowsum_min` above), so neither $\mathbf{1}$ nor any other constant vector is in its null space. VanRaden's $\mathbf{G}$ on the same hybrids has `sum(G)` and every row sum at numerical zero, which *is* the centring, and its rank comes out exactly one below $\mathrm{rank}(\mathbf{f})$ on this data set: one dimension is what centring costs, and it is the only part of either deficiency that centring explains. The other 576 missing dimensions have a different cause entirely, and the rest of this step is that cause.

In a hybrid programme the deficiency is far worse than one zero eigenvalue, and it is structural rather than numerical. A hybrid's marker row is the average of its two parents' rows, so $\mathbf{X} = \frac{1}{2}\mathbf{B}\mathbf{M}_L$ with $\mathbf{B}$ the $N \times L$ parent-usage incidence matrix (a 1 in the two columns of its parents) and $\mathbf{M}_L$ the $L \times m$ line matrix (the `L / 2` object of `simulate_lines()`; the subscript keeps it apart from the $\mathbf{G}_\bullet$ relationship matrices of this chapter). The hybrid coancestry matrix is a Gram matrix of that, $\mathbf{f} = [\mathbf{X}, 1 - \mathbf{X}][\mathbf{X}, 1 - \mathbf{X}]'/m$. Its column space lies in the span of the columns of $\mathbf{X}$ plus the constant vector, so its rank is bounded by $\min(\mathrm{rank}(\mathbf{B}), m) + 1$ however many hybrids you have. Measured:

```
incidence <- function(idx, ctx) {
  B <- matrix(0, length(idx), ctx$n_lines)
  B[cbind(seq_along(idx), ctx$ped$a[idx])] <- 1
  B[cbind(seq_along(idx), ctx$ped$b[idx])] <- 1
  B
}
set.seed(31)
# A separate set from the `sels` of @sec-era-fhom: these vary n_sel, which is
# the axis the rank depends on. Named apart so neither chunk shadows the other.
rk_sels <- list(full_factorial = seq_len(ctx$N),
                random_60      = sample.int(ctx$N, 60),
                truncation_60  = sel_truncation(ctx, 60),
```

```
                max_div_60     = sel_greedy(ctx, 60, w = 0),
                random_20      = sample.int(ctx$N, 20))
rk <- do.call(rbind, lapply(names(rk_sels), function(n) {
  i <- rk_sels[[n]]
  data.frame(selection = n, n_sel = length(i),
             lines_used = length(unique(c(ctx$ped$a[i], ctx$ped$b[i]))),
             rank_f = qr(ctx[["f"]][i, i])$rank,
             rank_incidence = qr(incidence(i, ctx))$rank)
}))
rk$deficiency <- rk$n_sel - rk$rank_f
fmt(rk)
```

selection	n_sel	lines_used	rank_f	rank_incidence	deficiency
full_factorial	625	50	49	49	576
random_60	60	47	46	46	14
truncation_60	60	33	32	32	28
max_div_60	60	43	41	41	19
random_20	20	31	20	20	0

```
stopifnot(all(rk$rank_f == rk$rank_incidence))
ev_min <- min(eigen(ctx[["f"]], symmetric = TRUE, only.values = TRUE)$values)
c(rank_full_factorial = rk$rank_f[1], n_lines = ctx$n_lines)
#> rank_full_factorial             n_lines
#>                  49                  50
c(min_eigenvalue_f = signif(ev_min, 3))
#> min_eigenvalue_f
#>        -4.28e-13
```

Result. In all five selections the rank of $\mathbf{f}$ equals the rank of the parent-usage incidence matrix exactly. On the full factorial that is $49 = L - 1$ for 50 lines, against 625 hybrids: a deficiency of 576, with a smallest eigenvalue at −4.3^{-13}. A subset of 20 hybrids drawn from 50 lines is full rank.

Interpretation. The deficiency is not a rounding artefact and does not shrink with more markers; it is the factorial structure. N hybrids built from L lines live in an at most L-dimensional space, so any hybrid-level optimiser that needs $\mathbf{f}^{-1}$ must regularise, and the amount of regularisation required is predictable *before* any eigendecomposition: compute $\mathrm{rank}(\mathbf{B})$, which costs one `qr()` on an $N \times L$ 0/1 matrix. The equality `rank_f == rank_incidence` is a measurement over five selections at one scale, not a theorem – the algebra above bounds $\mathrm{rank}(\mathbf{f})$ by $\mathrm{rank}(\mathbf{B}) + 1$, and the extra dimension the bound allows is not attained in any of the five. It is also the reason this book's selection machinery never inverts $\mathbf{f}$: `metrics_cheap()` and the DE fitness only ever evaluate the quadratic form $\mathbf{c}'\mathbf{f}\mathbf{c}$, which needs no inverse. The quadratic-programming routes do need a definite matrix, and both `optimal_contributions()` and `ocs_quadprog()` add `diag(1e-8, n)` for exactly this reason – four orders of magnitude above the numerical zero reported above, and small enough not to move the optimum. See Chapter 18. and Section 17.3.2.

Call. `qr()` on the usage incidence matrix, or on $\mathbf{f}$ restricted to the candidate set; `eigen(..., only.values = TRUE)` for the spectrum if you want the size of the numerical zero. **Check.** `rank(f) < n_sel`, and how far below. **Warning sign.** A quadratic-programming solver that returns without error but with contributions concentrated on a handful of hybrids: a singular matrix makes the objective flat along $N -$ rank directions, and the solver will pick a vertex. **Threshold.** The added diagonal α must exceed the largest magnitude of the numerically-zero eigenvalues, here about 4^{-13}, by several orders of magnitude; $\alpha = 0.01$ on a matrix whose diagonal averages 0.82 is a perturbation of about 1.2% and must be documented as such, because it changes the optimum.

12.8 What not to do

Four recommendations that do not need their own step. Each one is labelled with what backs it, because they are not backed equally: one is measured in this chapter, three are positions taken by T. H. E. Meuwissen, A. K. Sonesson, G. Gebregiwergis, and J. A. Woolliams [6] and carried forward here as *their* argument, and none of the three has been tested on plant germplasm anywhere in this book.

Do not maximise marker heterozygosity as an objective. *Measured here:* raising heterozygosity on the monitored panel does not raise it at unmonitored loci – Section 12.7.5 measures the sign reversal, under linkage equilibrium, which makes it a best case. *Their argument, not measured here:* doing so also raises IBD-based inbreeding, because "changes in allele frequency result from multiple copies of a subset of base generation alleles, so increasing frequency is promoting IBD based inbreeding" [6, Discussion, Management of Diversity]. This book has no IBD route at all – every matrix in it is identity-by-state – so that half is a reason to distrust the objective, not a result reproduced here.

Region-specific diversity targets rarely earn their complexity. *Their argument.* T. H. E. Meuwissen, A. K. Sonesson, G. Gebregiwergis, and J. A. Woolliams [6] give two grounds in the same paragraph: that causative alleles of quantitative traits are quite evenly distributed across the genome (their Discussion, citing Wood et al. 2014 on human stature – an observation about one species and one trait, not about maize), and that "what happens in a sample of loci does not necessarily predict what happens at loci outside that subset", which is the claim Section 12.7.5 does test, at the marker level. They name the major histocompatibility complex as the case that *does* earn a constraint of its own; the plant analogue offered here is R-gene-type resistance loci, where allelic diversity has direct and known value. That analogy is offered as such, and nothing in this book measures it.

Deleterious recessives are a load problem, not a diversity problem. *Their judgement:* for regions carrying known recessive defects, "direct inclusion of the known defects in the breeding goal seems more effective in controlling their frequencies" than prioritising the region for diversity management [6, Discussion]. Neither they nor this book compare the two routes quantitatively, so read it as the expectation of the authors of the scheme, not as a measured efficiency. In a hybrid programme there is a second and independent reason, which follows from A1 and A2 rather than from anyone's judgement: a recessive carried at low frequency in pool A is masked in every $A \times B$ hybrid unless pool B carries it too, so the load is expressed in the crossing block and in the DH induction, not in the product.

In a pure conservation scheme the problem is expected to be *worse*, not better. *Their expectation, stated as one:* "the potential covariance between the change in allele frequency and the initial frequency is expected to increase, since the inbreeding management term is more important in pure conservation schemes" [6, Discussion], which is why they argue the IBD recommendation is stronger there. Their simulations all carry selection, so no conservation scheme was run to test it; the sign of the mechanism is nevertheless the same algebra as everything above – a constraint that is the only thing moving frequencies is a constraint that generates the whole covariance.

12.9 Limits of the animal-to-plant transposition

Assumption in the paper	Situation in a hybrid crop programme
discrete generations, two sexes, one contribution per individual	same line reused for years, cloning of parents through DH maintenance, overlapping cycles, one line used in dozens of crosses
reliable, deep pedigree	recycling pedigree usually complete inside the programme, absent for introduced germplasm and genebank accessions
base = simulated founder population at mutation-drift equilibrium	base is an administrative choice (cycle 0) with pre-existing heterotic structure already in it
target Ne = 100 over 20 generations	shorter cycles with GS; this book measures no commercial elite Ne, but Section 12.7.6′s own teaching-scale example shows Ne_parents falling well below the pool size under plain truncation (39.3 to 13.0)
one population, one target	two heterotic pools with opposite-signed constraints, several target environments
heterozygosity is always desirable	desirable in the product, undesirable in the parents, irrelevant at the individual level once lines are fixed – and in none of the three cases is it what F_hom measures, which is merged-pool gene diversity (Section 12.3.2)

None of this invalidates the central conclusion. It is an **algebraic** property of the optimiser, not a biological property of the species. What it changes is *which matrix is practicable*, and – for a two-pool programme – that one global constraint is never enough.

12.10 Operational checklist

The protocol above, in one column, with the step that implements each item.

Check	Step
Is the base p_0 frozen at the founding cycle, and documented?	Section 12.7.1
Is the hybrid-level base the balanced average of the two pool bases?	Section 12.7.1
Are `F_hom` and `F_drift` computed **separately** and reported together?	Section 12.7.2
Is it recorded that `F_hom` is merged-pool gene diversity and **not** F1 heterozygosity, with GD_{BS} reported next to it?	Section 12.3.2
Is every single-cycle value compared with a **resampled** null rather than with one random draw?	Section 12.7.2
Is the sign of `F_hom` preserved rather than clamped?	Section 12.7.2
Is `cov_diag` reported as a diagnostic, with its sign?	Section 12.7.3
Is $\log(1 - F)$ regressed on cycle, with R^2 shown, for both lenses?	Section 12.7.4
Is ΔF read as $1 - e^{\text{slope}}$ rather than $-$ slope?	Section 12.7.4
Is there a panel of loci **not used in management**, for audit?	Section 12.7.5
Are θ_A and θ_B reported separately, never only F_{ST}?	Section 12.7.6
Is the number of parent lines used per pool biologically plausible?	Section 12.7.6
Is GD_{BS} held in a band rather than maximised?	Section 12.7.6
Is the matrix's rank checked before any inversion, and the added α documented?	Section 12.7.7
Is true genetic variance (or the best estimate of it) being tracked?	Chapter 3.
Where possible, is an IBD alternative compared in parallel?	Section 12.9

Reproduce this chapter

All the code of this chapter, in order and with the explanation kept as comments, is one script: **05-genomic-era.R**. Download it, install `hybdiv`, and run it top to bottom in an empty directory; cached results are fetched on first use.

13. Heterosis, dominance, and what divergence buys

Since Chapter 2. this book has repeated a claim without proving it: that the divergence between the heterotic pools is the engine of heterosis, and is therefore to be maintained rather than minimised. Chapter 14. will decompose diversity on that basis and Chapter 16. will build a monitoring panel around it. The claim has carried a lot of weight for an assertion.

This chapter derives it, and then corrects it. The derivation is short and the identity is exact. The correction is that the usual reading of the identity – *push the pools apart and you will get more heterosis* – is false: under neutral divergence expected heterosis does not move at all, and the simulation shows it.

The chapter then goes one step past the mean. Expected heterosis is what a pool pair is worth on average; what a breeder selects on is how heterosis is *distributed* across the crosses of the factorial. Those are different questions with different answers, and the second one is where general and specific combining ability come from.

The derivations come first and Section 13.12 turns them into the eight steps that compute them on a real programme's data, with the check and the failure mode of each one. Read that section first if the question is what to run rather than why it holds.

Three premises carry everything below. They are stated here once, because every result in the chapter uses all three and two of them are restrictive.

Parents are fully inbred. Every parent line is homozygous at every locus, so an F1′s marker row is the average of its two parents' rows and a locus is heterozygous in the F1 exactly when the parents differ. This is what makes the additive effects cancel from the mid-parent difference and what makes `GD_WI` zero. Note that this is a property of the *programme*, not of the mating system: the crop here is allogamous – maize-type, two heterotic pools under reciprocal recurrent selection – and it is DH induction or selfing to fixation that delivers homozygous parents from it. Nothing here transfers to a programme crossing non-inbred parents (an inter-population cross, a synthetic, S1 parents) without re-derivation, and Section 13.12.1 measures how large the error is if the formulae below are applied anyway: a factor of five, not a few per cent.

The genotypic model is dominance within loci and additivity between them. There is no epistatic term anywhere below: the genotypic value is a sum over loci, H_i is a sum of per-locus indicators, and the combining-ability expansion of Section 13.10 is bilinear only because no term couples two loci. M. R. Labroo, A. J. Studer, and J. E. Rutkoski [130] review epistasis alongside dominance as a proposed mechanism for heterosis; this model excludes it, and no result below should be read as evidence about it. What d_k does absorb is every *within-locus* story, as the next section sets out.

The marker panel is the set of dominance loci. d_k is indexed by the same k as the markers, so every locus carrying dominance is genotyped and the frequency statistics are computed on exactly the loci that generate heterosis. This is the strongest assumption of the three, and it is the one that carries the distance results of Section 13.6. A real panel tags causal loci through linkage disequilibrium rather than containing them, which weakens every statement below about markers predicting heterosis but does not change the algebra, since the algebra is written over whatever set k indexes.

13.1 The dominance model, minimally

13.1.1 The one-locus scale

The classical parameterisation puts the origin at the midpoint of the two homozygotes: one homozygote is worth $+a$, the other $-a$, and the heterozygote d. Under random mating at frequency p, with $q = 1 - p$, the population mean is [12, Eq. 7.2]

$$M = a\,(p - q) + 2pq\,d \tag{13.1}$$

The first term is the additive contribution, and the mid-parent of any cross already carries it. The second is the dominance contribution. It is the only part of M that depends on how many heterozygotes there are, and therefore the only part a cross can add over its own mid-parent.

> **i** a is a scale, not a quantity this chapter computes
>
> The symbols in that display are the textbook ones. This book's additive marker effects are β_k (3), and its α is the diversity-loss statistic of Chapter 14., not the classical average effect of a substitution. The per-locus a_k reappears three times below and never as a result: in the genotypic-value model used to show that it cancels, in the definition of overdominance as $|d| > |a|$, and in the average effect of a substitution $a + d(q - p)$. No quantity this chapter reports depends on it.

13.1.2 What d_k is, and what it is not

d_k is *functional* dominance: the departure of the heterozygote from the homozygote midpoint at locus k. It is a property of the locus. It does not move when allele frequencies move.

Fisher's *statistical* dominance deviation is a different object: the residual left after projecting genotypic values onto the best additive model **in a particular population**. It is a property of the population as much as of the locus, and it changes when frequencies change.

Both appear in this chapter, and keeping them apart is the whole trick. d_k is the input. The specific combining ability of Section 13.10 is the statistical dominance deviation that a given d_k *induces* over a given pair of pools. One is a constant; the other is an answer that depends on where the pools happen to sit.

The distinction also buys the model what generality it has, and marks where that generality stops. Dominance, overdominance (a heterozygote outside both homozygotes, $|d| > |a|$) and pseudo-overdominance (two dominant favourable alleles in repulsion phase, too close for any marker to separate) are three different biological stories, and M. R. Labroo, A. J. Studer, and J. E. Rutkoski [130] review the evidence for each. At marker resolution they are the same arithmetic: a locus at which the F1 gains something the mid-parent does not. A single d_k absorbs all three, and nothing below can distinguish them – nor needs to. Epistasis is the fourth story on M. R. Labroo, A. J. Studer, and J. E. Rutkoski [130]'s list and d_k does not absorb it, because a single per-locus number cannot represent a term that depends on two loci at once. That is the second premise above, and it is an exclusion rather than an approximation.

13.1.3 The mid-parent difference

Code a genotype by $x_k \in \{0, 0.5, 1\}$, the frequency of the reference allele within the individual, and take the genotypic value to be the additive part plus the dominance deviation at every heterozygous locus:

$$G(\mathbf{x}) = \sum_k a_k \,(2x_k - 1) + \sum_k d_k \,\mathbb{1}\{x_k = 0.5\} \tag{13.2}$$

The parents are fully inbred, so each is homozygous at every locus and the F1′s row is the average of its two parents' rows. The additive part is linear in $\mathbf{x}$, so the two parents' additive values average exactly to the F1′s own and cancel from the mid-parent difference. What survives is the dominance deviation at the loci where the F1 is heterozygous:

$$H_i = \sum_k d_k \,\mathbb{1}\{x_{ik} = 0.5\} \tag{13.3}$$

The additive effects never appear. That is the result, not an omission, and it is why `heterosis_value()` takes no additive-effect argument:

```
heterosis_value
#> function (X, d)
#> {
#>     stopifnot(length(d) == ncol(X))
#>     drop((X == 0.5) %*% d)
#> }
```

```
#> <bytecode: 0x750ce20cb8>
#> <environment: namespace:hybdiv>
```

Against the literal oracle of Appendix C., which loops over loci and compares the two parents' genotypes one at a time:

```
max(abs(heterosis_value(ctx$X[1:25, ], d) -
        ref_heterosis(L, head(ctx$ped, 25), d)))
#> [1] 0
```

The cancellation is worth seeing directly. Draw additive effects at random, score a real F1 and its two parents under $G(\mathbf{x})$, and subtract:

```
set.seed(9)                    # the cancellation is exact for any a; seed it anyway
a   <- rnorm(ctx$m, 0, 0.2)
val <- function(x) sum(a * (2 * x - 1)) + sum(d * (x == 0.5))
i   <- 7
mid <- (val(L[ctx$ped$a[i], ] / 2) + val(L[ctx$ped$b[i], ] / 2)) / 2
c(mid_parent_difference = val(ctx$X[i, ]) - mid,
  heterosis_value       = heterosis_value(ctx$X[i, , drop = FALSE], d))
#> mid_parent_difference       heterosis_value
#>             362.3386             362.3386
```

The additive effects entered the first number and cancelled out of it; they never entered the second.

13.2 Expected heterosis over a pool pair

Write p_{Ak} and p_{Bk} for the reference-allele frequencies at locus k in the two pools, each computed over that pool's lines. A locus is heterozygous in the F1 when the two gametes carry different alleles, which for a cross drawn at random between the pools happens with probability $p_A(1-p_B)+(1-p_A)p_B$. The expected heterosis over all $A \times B$ crosses therefore follows from the pool frequencies alone:

$$\mathbb{E}[H] = \sum_k d_k\,[\,p_{Ak} + p_{Bk} - 2p_{Ak}p_{Bk}\,] \tag{13.4}$$

The candidate pool here *is* the full factorial, every $A \times B$ pairing exactly once, and the frequencies are the means of those same lines. The closed form is then the mean of the realised values as an identity rather than an approximation, and the residual is at machine precision:

```
abs(heterosis_expected(pA, pB, d) - mean(heterosis_value(ctx$X, d)))
#> [1] 1.136868e-13
```

That is a mean over $N = 625$ crosses, and a mean is not a ranking. Hold on to the question of how the individual crosses are spread around it; Section 13.10 answers it exactly.

13.3 The identity

Now split the bracket. Writing $\bar{p} = (p_A + p_B)/2$ and $y = p_A - p_B$, the per-locus F1 heterozygosity separates exactly:

$$p_A + p_B - 2p_Ap_B = \underbrace{2\bar{p}(1-\bar{p})}_{\text{shared}} + \underbrace{y^2/2}_{\text{divergence}} \tag{13.5}$$

The first term is the heterozygosity a single merged pool at frequency $\bar{p}$ would produce under random mating. The second is what keeping the pools apart adds on top of it. Average both over loci with $d_k \equiv 1$, so that the weighting does not obscure the algebra, and they are exactly the GD_T and GD_{BS} of the Caballero & Toro partition that Chapter 2. derived and `gd_partition()` computes:

```
hp <- heterosis_partition(pA, pB, rep(1, ctx$m)) / ctx$m
gp <- gd_partition(seq_len(ctx$N), ctx)
```

```
fmt(data.frame(term       = c("shared", "divergence"),
               heterosis  = c(hp[["shared"]], hp[["divergence"]]),
               partition  = c(gp[["GD_T"]],   gp[["GD_BS"]]),
               residual   = c(abs(hp[["shared"]]     - gp[["GD_T"]]),
                              abs(hp[["divergence"]] - gp[["GD_BS"]]))), 12)
```

term	heterosis	partition	residual
shared	0.3280928	0.3280928	0
divergence	0.0335968	0.0335968	0

GD_{BS} weighted by dominance deviations **is** the divergence half of expected heterosis. "Between-pool divergence is the heterosis engine" is an identity, not a slogan.

One thing to be clear about before reading on. GD_T is the gene diversity of the *merged* population, and by Chapter 2. it already contains GD_{BS}: with inbred lines, $GD_T = GD_{BI} + GD_{BS}$. Expected F1 heterozygosity is therefore $GD_{BI} + 2\,GD_{BS}$, the between-pool component counted twice – once inside the shared term, once as the divergence term. That factor of two is where the classical result sits. Falconer & Mackay's heterosis of a cross between two *random-mating* populations is $\sum_k d_k y_k^2$ [12, Ch. 14], exactly twice the divergence term here. The two do not conflict; they use different mid-parents. Theirs is two random-mating populations, each carrying its own within-pool heterozygosity, so their H is the total above $\sum_k d_k(p_{Ak}q_{Ak} + p_{Bk}q_{Bk})$. Ours is two inbred lines carrying none, so the whole F1 heterozygosity is heterosis, and the shared term is the mean of the two pools' random-mating heterozygosities plus the half of $\sum_k d_k y_k^2$ that GD_T already holds.

That paragraph is an algebraic claim and it can be checked rather than asserted. Subtract the random-mating mid-parent from the F1's total dominance value and what is left is $\sum_k d_k y_k^2$, exactly twice this chapter's divergence term:

```
hpd  <- heterosis_partition(pA, pB, d)
mid_random_mating <- sum(d * (pA * (1 - pA) + pB * (1 - pB)))
stopifnot(isTRUE(all.equal(hpd[["total"]] - mid_random_mating,
                           2 * hpd[["divergence"]])),
          isTRUE(all.equal(2 * hpd[["divergence"]], sum(d * (pA - pB)^2))))
c(F1_total          = hpd[["total"]],
  mid_random_mating = mid_random_mating,
  difference        = hpd[["total"]] - mid_random_mating,
  twice_divergence  = 2 * hpd[["divergence"]],
  sum_d_y2          = sum(d * (pA - pB)^2))
#>          F1_total mid_random_mating        difference  twice_divergence
#>         355.66859         289.77816          65.89042          65.89042
#>          sum_d_y2
#>          65.89042
```

The first number is what an F1 between two inbred lines gains over *its* mid-parent of zero; the third is what the same F1 gains over a mid-parent of two random-mating pools. They differ by 289.8 units of dominance value that the random-mating parents already carry themselves. Which mid-parent is the right one is not a matter of convention: it is whichever one the programme actually plants. Section 13.12.1 measures the consequence of getting it wrong.

13.3.1 A third form: one minus the between-pool coancestry

The sum of the two components has a name of its own. $\theta_{AB} = \mathbf{w}_A' \mathbf{f}^L \mathbf{w}_B$ is the between-pool coancestry of Chapter 14.: on the full factorial, where the line weights are uniform, it is the probability that an allele drawn from a pool-A line and an allele drawn from a pool-B line are identical in state. Its complement is the whole bracket:

$$GD_T + GD_{BS} = 1 - \theta_{AB} \tag{13.6}$$

```
tp <- theta_pools(seq_len(ctx$N), ctx)
c(shared_plus_divergence = gp[["GD_T"]] + gp[["GD_BS"]],
  one_minus_theta_AB     = 1 - tp[["theta_AB"]])
#> shared_plus_divergence     one_minus_theta_AB
#>             0.3616896              0.3616896
```

This one is algebra rather than simulation. `gd_partition()` defines $GD_T = 1 - \bar{f}$ and $GD_{BS} = \tilde{f} - \bar{f}$ with $\bar{f} = (\theta_A + 2\theta_{AB} + \theta_B)/4$ and $\tilde{f} = (\theta_A + \theta_B)/2$, and the sum collapses to $1 - \theta_{AB}$. Which is the shortest statement the chapter has to make: **expected F1 heterozygosity is one minus between-pool coancestry.** Everything else here is a weighting of that quantity, or a way of splitting it.

13.4 A second identity, at the hybrid level

Chapter 14. reports `GD_WI_hyb`, the within-individual gene diversity actually carried by the F1s, separately from the line-level partition, because with inbred parents `GD_WI` is zero by construction and carries no signal. A hybrid's self-coancestry is $1 - h_i/2$, with h_i its heterozygous fraction, so `GD_WI_hyb` is half the mean F1 heterozygosity and the two levels are linked:

$$2 \cdot GD_{WI\text{ - }hyb} = GD_T + GD_{BS} \tag{13.7}$$

```
c(lhs = 2 * gp[["GD_WI_hyb"]], rhs = gp[["GD_T"]] + gp[["GD_BS"]],
  GD_WI = gp[["GD_WI"]])
#>           lhs           rhs         GD_WI
#>  3.616896e-01  3.616896e-01 -4.440892e-16
```

Exact on the full factorial, because every $A \times B$ pairing is present exactly once. On a *selected* subset it holds only approximately. There GD_T and GD_{BS} are computed from the line-usage weights, which is what a random re-pairing of the selected parents would give, while `GD_WI_hyb` scores the pairs actually chosen. The gap is the pairing structure of the subset. `tests/testthat/test-dominance.R` asserts the identity to `1e-12` on the full factorial and, on a *random* 40-hybrid subset at one scale (12 lines per pool, 800 markers), bounds the gap with `expect_equal(tolerance = 5e-3)`. That tolerance is **relative**, not absolute: on a gene-diversity value of order 0.4 the absolute bound it enforces is of order `2e-3`, and the gap the test actually sees is smaller again. A subset chosen by an optimiser, which concentrates line usage deliberately, is not covered by that bound; Section 12.3.2 is where it is measured, on four such subsets at this book's scale, and reports the largest departure of the four.

13.5 Inbreeding depression, and why it does not price α

The same model run backwards gives inbreeding depression. A locus at frequency p in a population with inbreeding coefficient F is heterozygous with probability $2pq(1 - F)$, so the mean declines linearly in F [12, Ch. 14]:

$$ID = 2F \sum_k d_k \, p_k q_k \tag{13.8}$$

At $F = 1$ that is exactly the *shared* term of `heterosis_partition()` evaluated at $p_A = p_B = p$. So the depression of a pool driven to complete homozygosity and the expected heterosis of a cross made *within* that same pool are one quantity seen from two sides. I. J. Mackay, J. Cockram, and P. Howell [13] treat heterosis and inbreeding depression together on that basis, alongside transgressive segregation, which this chapter does not take up. The identity needs no new machinery:

```
c(inbreeding_depression = inbreeding_depression(pA, d),
  shared_term           = heterosis_partition(pA, pA, d)[["shared"]],
  deviation             = abs(inbreeding_depression(pA, d) -
                              heterosis_partition(pA, pA, d)[["shared"]]))
#> inbreeding_depression          shared_term            deviation
#>             288.2869             288.2869                0.0000
```

The closed form is checked against simulated genotypes rather than against its own algebra, which is what `ref_inbreeding_depression()` exists for: it draws each genotype as identical by descent with probability F and as two independent alleles otherwise, so the agreement is to Monte Carlo error:

```
k <- 1:300
c(closed_form = inbreeding_depression(pA[k], d[k], 0.6),
  simulated   = ref_inbreeding_depression(pA[k], d[k], 0.6, n = 20000, seed = 2))
#> closed_form   simulated
#>    27.58563    27.57923
```

Now the part that matters, because the obvious next step is wrong.

It is tempting to argue: α budgets a loss of within-pool gene diversity; gene diversity is $\sum_k p_k q_k$; inbreeding depression is $\sum_k d_k\, p_k q_k$; therefore spending α costs yield in the current cycle, and *that* is the price of the budget. The algebra does not support it.

```
c(max_dominance_contribution_of_an_inbred_line = max(abs(heterosis_value(U, d))))
#> max_dominance_contribution_of_an_inbred_line
#>                                            0
```

Zero, exactly. A fully inbred line has no heterozygous loci, so its dominance contribution is zero **whatever the pool's allele frequencies are**. In a programme whose parents are inbred lines the depression was paid in full when the lines were made; it is a constant, and it does not move when the pool subsequently loses diversity. The formula above describes the gap between a hypothetical random-mating pool and its inbred derivatives, and that gap *shrinks* as $p_k q_k$ shrinks – but nothing in the programme stands at the random-mating end of it.

> ! Inbreeding depression is not the current-cycle price of the diversity budget
>
> This is the same fact that makes GD_{WI} exactly zero in Chapter 14.'s partition, and it has the same cause: with inbred parents there is no within-individual variation left to lose. Two consequences.
>
> The cost of α in this system remains what Chapter 22. measures – an option value, realised cycles later through the variance available to select on (Chapter 3.), not a yield loss booked this season.
>
> And inbreeding depression **is** the live quantity wherever the population is only partially inbred: F2, F3 and F4 in Chapter 5., where F is still climbing and the depression is still accumulating. That is the chapter it belongs to, not this one.

Note also what this does *not* say. Expected heterosis in the product is a separate question. Section 13.7 answers it for the neutral case: under drift the shared term lost and the divergence term gained cancel exactly, so hybrid heterosis is invariant. Under selection the frequencies move directionally, and what happens to expected heterosis is then the alignment question of Section 13.8, which can go either way. In neither case is the diversity loss itself an immediate dominance cost. The bill arrives later, and it arrives in the additive variance.

13.6 When genetic distance predicts heterosis exactly

Before the corrections start, one special case is worth having, because it is the only case in which marker distance and heterosis are the same number.

Set $d_k \equiv d$, constant across loci. Then H stops being a weighted count and becomes a plain one: d times the number of loci at which the two parents differ. For inbred lines that count is exactly what the coancestry kernel of Chapter 1. measures, so for a cross between lines a and b

$$H_{ab} = d\, m\, (1 - f^{L}_{ab}) = d\, m\, \mathrm{MRD}^2_{ab} \tag{13.9}$$

```
h1 <- heterosis_value(ctx$X, rep(1, ctx$m))
fL <- molecular_coancestry(L / 2)
ab <- cbind(ctx$ped$a, ctx$ped$b)
c(vs_coancestry = max(abs(h1 - ctx$m * (1 - fL[ab]))),
  vs_mrd        = max(abs(h1 - ctx$m * mrd_matrix(L / 2)[ab]^2)))
```

```
#> vs_coancestry        vs_mrd
#>  1.136868e-13  2.273737e-13
```

Both residuals are at machine precision. Under flat dominance, heterosis and molecular coancestry are one statistic seen twice – Trap 1 of Chapter 16. read from the other side – and the diversity kernel this whole book runs on turns out to be the right object for the heterosis question as well.

Flat d is also precisely the condition under which H is a function of marker distance alone. It is a strong condition, and it fails as soon as d varies: two crosses at the same distance then differ by *which* loci they differ at. With the gamma-distributed deviations used everywhere else in this chapter the relationship survives but stops being an identity:

```
c(cor_uniform_d = cor(h1, 1 - fL[ab]),
  cor_real_d    = cor(heterosis_value(ctx$X, d), 1 - fL[ab]))
#> cor_uniform_d    cor_real_d
#>     1.0000000     0.7809147
```

A correlation of 0.78 is a useful predictor and a poor identity. Two things move it, and both are properties of the germplasm rather than of the statistic: how d is spread over the loci at which the parents differ, which is what Section 13.8 is about, and – once the third premise is relaxed – how well the panel tags the causal loci. Under flat d on a panel that *is* the causal set the correlation is 1 by the identity above. Departing from either condition generally lowers it – not always, since a d that varies only across loci at which no pair of parents differs changes no H_i at all – and by how much is something this chapter does not attempt to predict.

13.7 Divergence is not a bonus

Read `divergence` as free money and you will reach for F_{ST}. Do not.

Draw the pools further apart and watch all three terms. Nothing here is selected. Each point re-draws both pools from the *same* ancestral frequencies – the ancestral vector is fixed by `seed = 5` and shared across the sweep, while the two pool-frequency vectors are fresh Balding-Nichols draws at each `fst` (Appendix A.) – so what varies along the sweep is how far the draw scatters and nothing else:

```
sweep_het <- function(fst) {
  cf <- modifyList(book_cfg, list(fst = fst, seed = 5))
  Lf <- simulate_lines(cf)
  pl <- rep(c("A", "B"), c(cf$n_pool_A, cf$n_pool_B))
  Pf <- pool_freqs(Lf / 2, pl)
  hp <- heterosis_partition(Pf["A", ], Pf["B", ], rep(1, cf$m)) / cf$m
  cm <- pool_complementarity(Lf / 2, pl)
  data.frame(fst = fst, shared = hp[["shared"]], divergence = hp[["divergence"]],
             total = hp[["total"]],
             frac_opposite_fixed = cm[["frac_opposite_fixed"]],
             mean_abs_delta_p = cm[["mean_abs_delta_p"]],
             mean_sq_delta_p = cm[["mean_sq_delta_p"]])
}
sw <- do.call(rbind, lapply(c(0.02, 0.05, 0.10, 0.20, 0.30, 0.45), sweep_het))
# Read again in @sec-heterosis-step5: no marker is fixed for opposite alleles
# anywhere below fst = 0.20, while the divergence term has already more than
# doubled over that span.
stopifnot(all(sw$frac_opposite_fixed[sw$fst <= 0.10] == 0))
fmt(sw, 4)
```

fst	shared	divergence	total	frac_opposite_fixed	mean_abs_delta_p	mean_sq_delta_p
0.02	0.3549	0.0107	0.3656	0.000	0.1138	0.0214
0.05	0.3489	0.0171	0.3659	0.000	0.1414	0.0341
0.10	0.3374	0.0246	0.3620	0.000	0.1715	0.0493
0.20	0.3231	0.0433	0.3664	0.002	0.2252	0.0867
0.30	0.3042	0.0595	0.3638	0.012	0.2589	0.1191
0.45	0.2792	0.0846	0.3639	0.041	0.3012	0.1693

Across the sweep the divergence term rises by a factor of 7.9. The comparison that matters is in absolute units, not in ratios: divergence gains 0.0739 per locus and the shared term loses 0.0756. The total does not move.

13.7.1 The two terms, term by term

The algebra is one line. Pool A and pool B are drawn *independently* around a common ancestral frequency p, so $\mathbb{E}[p_A] = \mathbb{E}[p_B] = p$ and $\mathbb{E}[p_A p_B] = p^2$. Therefore

$$\mathbb{E}[p_A + p_B - 2p_A p_B] = 2p(1-p), \tag{13.10}$$

which contains no F_{ST} at all. With ancestral frequencies drawn uniformly on $[0.05, 0.95]$ the predicted level is $\mathbb{E}[2p(1-p)] = 0.365$, against a simulated mean of 0.3646 across the whole sweep.

That the *sum* is flat is the headline, but the Balding-Nichols model predicts each half separately, which is the sharper statement. Under it $\mathrm{Var}(p_A) = \mathrm{Var}(p_B) = F_{ST}\,p(1-p)$, so $\mathrm{Var}(\bar{p}) = F_{ST}\,p(1-p)/2$ and $\mathrm{Var}(y) = 2F_{ST}\,p(1-p)$. Substituting:

$$\mathbb{E}[\bar{p}(1-\bar{p})] = 2p(1-p)\left(1 - \frac{F_{ST}}{2}\right), \qquad \mathbb{E}[y^2/2] = F_{ST}\,p(1-p) \tag{13.11}$$

The shared term loses $F_{ST}\,p(1-p)$ and the divergence term gains $F_{ST}\,p(1-p)$: the same expression with opposite signs, not approximately and not on average over some range. Both predictions are parameter-free once the ancestral distribution is fixed, so they can be laid straight on top of the sweep:

```
Ep <- 2 * (0.5 - (0.9^2 / 12 + 0.25))      # E[2p(1-p)] for p ~ U(0.05, 0.95)
sw$pred_shared     <- Ep * (1 - sw$fst / 2)
sw$pred_divergence <- Ep * sw$fst / 2
fmt(sw[, c("fst", "shared", "pred_shared", "divergence", "pred_divergence")], 4)
```

fst	shared	pred_shared	divergence	pred_divergence
0.02	0.3549	0.3614	0.0107	0.0036
0.05	0.3489	0.3559	0.0171	0.0091
0.10	0.3374	0.3468	0.0246	0.0182
0.20	0.3231	0.3285	0.0433	0.0365
0.30	0.3042	0.3102	0.0595	0.0548
0.45	0.2792	0.2829	0.0846	0.0821

```
plot(sw$fst, sw$total, type = "b", pch = 19, col = "#d98e04", ylim = c(0, 0.40),
      xlab = "target fst between pools", ylab = "expected heterosis per locus (d
= 1)")
lines(sw$fst, sw$shared, type = "b", pch = 17, col = "#1f5f8b")
lines(sw$fst, sw$divergence, type = "b", pch = 15, col = "#b2182b")
lines(sw$fst, sw$pred_shared, lty = 2, col = "#1f5f8b")
lines(sw$fst, sw$pred_divergence, lty = 2, col = "#b2182b")
abline(h = mean(sw$total), lty = 2, col = "grey45")
legend("right", c("total", "shared", "divergence", "predicted"), bty = "n",
       cex = 0.8, pch = c(19, 17, 15, NA), lty = c(NA, NA, NA, 2),
       col = c("#d98e04", "#1f5f8b", "#b2182b", "grey45"))
grid()
```

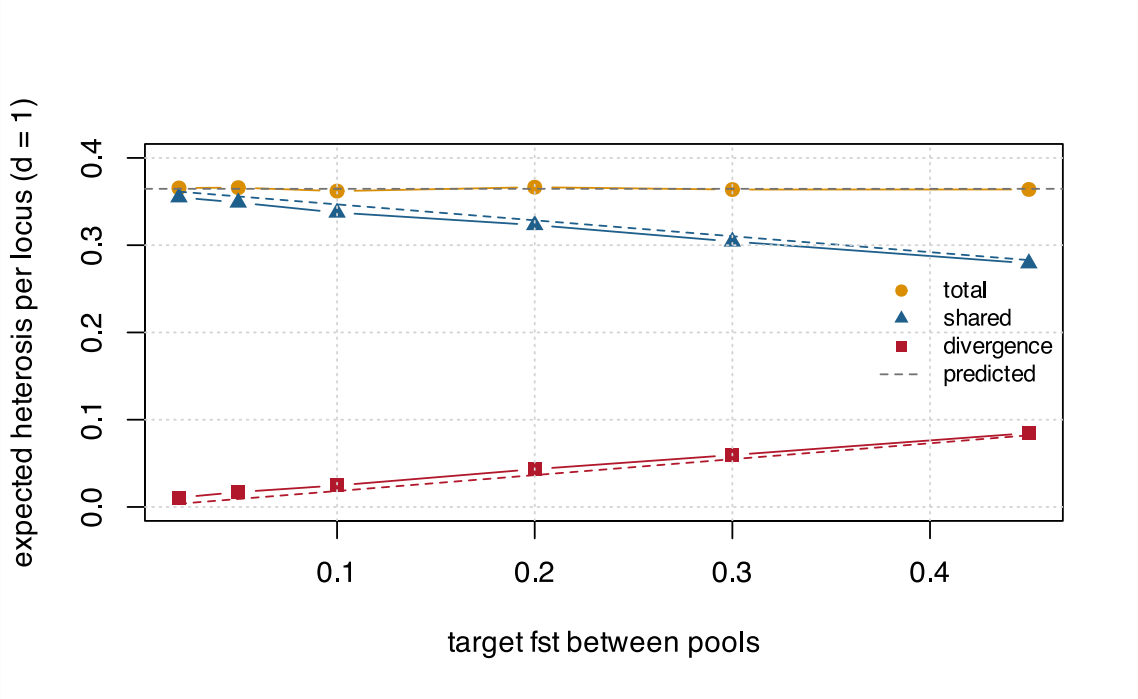

Figure 13.1: Expected heterosis and its two components as the pools are driven apart by drift alone. Dashed lines are the Balding-Nichols predictions for each component separately, with no fitted parameter. Each measured component is offset from its prediction – divergence above, shared below – by sampling error in the pool frequencies; that offset narrows along the axis, and the text quantifies both it and the measured gap. The divergence term grows several-fold; the shared term pays for all of it, unit for unit; the total is flat. Nothing is selected here – this is what neutral divergence does.

The measured divergence sits above its prediction at every `fst`, and the measured shared term about the same distance below it. That is not drift, it is estimation. The pool frequencies are estimated from 25 and 25 sampled lines, and a frequency estimated from n lines carries sampling variance $V = pq/n$. Conditional on the true pool frequencies, $\mathbb{E}[\hat{y}^2/2] = y^2/2 + (V_A + V_B)/2$ and $\mathbb{E}[2\hat{\bar{p}}(1-\hat{\bar{p}})] = 2\bar{p}(1-\bar{p}) - (V_A + V_B)/2$: equal and opposite, so the total estimated from sample frequencies is unbiased while each half is not.

The offset is not constant along the sweep. It is proportional to $\mathbb{E}[p_A q_A + p_B q_B]$, which the pools themselves erode as they diverge, so it shrinks with $(1 - F_{ST})$: about 0.0072 per locus at `fst` = 0.02, down to 0.004 at `fst` = 0.45, against measured gaps of 0.007 and 0.0025.

Those two pairs do not agree equally well. At `fst` = 0.02 prediction and measurement are within 2% of each other; at `fst` = 0.45 the measured gap is 38% *below* the predicted offset. The formula gives an expectation; each row of the sweep is one draw of 2000 loci at one seed, and the scatter of that draw is not constant either. $\mathrm{Var}(y)$ is $2F_{ST}\,p(1-p)$, so the per-locus divergence $y^2/2$ has a variance that grows with F_{ST} while the number of loci averaging it stays fixed – the high-`fst` rows are the noisy ones, and the agreement at the bottom of the sweep is the more informative of the two. The offset is the right order and the right sign at both ends; it is not a per-point prediction, and nothing below rests on it being one.

What changes by an order of magnitude along the sweep is not the offset but the signal it sits on: at `fst` = `0.02` the offset exceeds the predicted divergence of 0.0036, and by `fst` = `0.45` it is a small fraction of it. A pool pair's *apparent* divergence overstates its true divergence, most severely when the true divergence is small and the pools are few.

> **!** Every unit of divergence is paid for out of the shared term
>
> Under neutral divergence the trade is exactly one for one. Drifting the pools apart does not create heterosis; it relocates it from the shared term to the divergence term. A programme that maximises F_{ST} has bought nothing and spent its within-pool diversity to do it.
>
> This is the mechanism behind Chapter 16. Trap 3. That section states the symptom – "F_{ST} keeps rising as pools drift apart at loci that are *already* divergent, which buys no additional heterosis" – and points here for the reason. The sweep table and Figure 13.1 are that reason.

13.8 What actually buys heterosis

If divergence per se is worth nothing, why do heterotic groups work?

Because real pools are not diverged at random with respect to d. The divergence term is $\sum_k d_k y_k^2/2$, a *weighted* sum. It grows when the loci that diverge are the loci that carry dominance; under drift the two are independent, which is exactly why the total was flat.

13.8.1 Alignment is a covariance

That sentence can be a number rather than a word. A sum of products is m times a mean times a mean, plus m times a covariance – the population covariance with divisor m, which is why the chunk rescales `cov()`:

$$\sum_k d_k\, y_k^2/2 = \frac{m}{2}\left[\overline{dy^2} + \mathrm{cov}(d, y^2)\right] \tag{13.12}$$

```
y2 <- (pA - pB)^2
c(divergence = sum(d * y2 / 2),
  from_parts = ctx$m / 2 * (mean(d) * mean(y2) + cov(d, y2) * (ctx$m - 1) / ctx$m))
#> divergence from_parts
#>   32.94521   32.94521
```

The first bracketed term is what F_{ST} can see: average divergence times average dominance. The second is what it cannot. **Alignment is $\mathrm{cov}(d, y^2)$**, and no statistic computed from allele frequencies alone can contain it, because d does not appear in one.

13.8.2 The scenarios

Hold the pools fixed, hold F_{ST} fixed, hold the multiset of dominance deviations fixed, and change only which locus gets which d_k. `aligned` gives the largest d_k to the largest y_k^2, `anti-aligned` does the reverse, and `independent` is the draw as made:

```
mk <- function(dec) { v <- numeric(ctx$m); v[order(y2)] <- sort(d, decreasing =
dec); v }
scen <- list(`anti-aligned` = mk(TRUE), `independent` = d, `aligned` = mk(FALSE))
al <- do.call(rbind, lapply(names(scen), function(n) {
  h <- heterosis_partition(pA, pB, scen[[n]])
  data.frame(scenario = n, cor_d_y2 = cor(scen[[n]], y2),
                   shared = h[["shared"]], divergence = h[["divergence"]], total
= h[["total"]])
}))
al$relative <- al$total / al$total[al$scenario == "independent"]
fmt(al, 3)
```

scenario	cor_d_y2	shared	divergence	total	relative
anti-aligned	−0.608	270.538	11.283	281.821	0.792
independent	0.006	322.723	32.945	355.669	1.000
aligned	0.953	370.139	66.313	436.452	1.227

```
bp <- barplot(rbind(al$shared, al$divergence), beside = FALSE,
              names.arg = al$scenario, col = c("#1f5f8b", "#b2182b"),
              ylab = "expected heterosis", border = NA)
legend("topleft", c("shared", "divergence"), fill = c("#1f5f8b", "#b2182b"),
       bty = "n", cex = 0.8, border = NA)
text(bp, al$total, sprintf("%.2fx", al$relative), pos = 3, cex = 0.8, xpd = NA)
grid(nx = NA, ny = NULL)
```

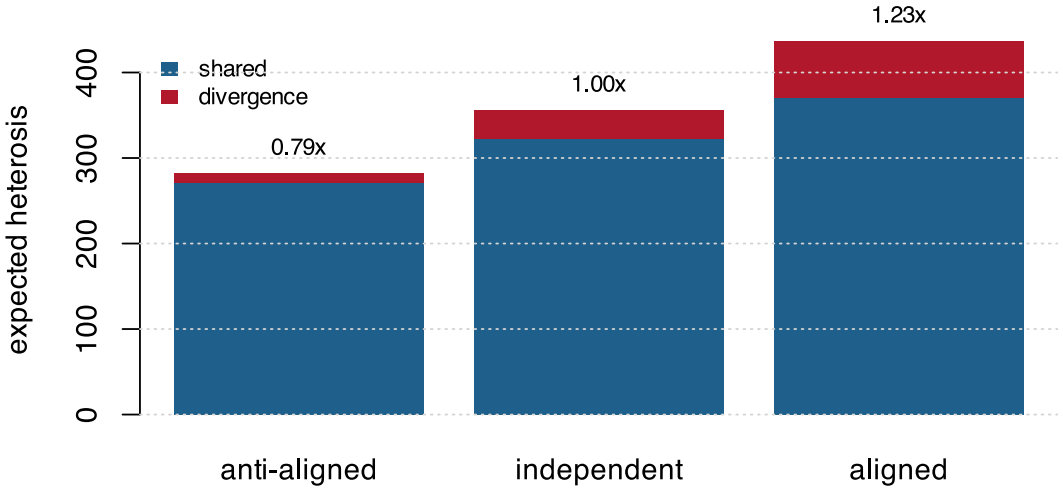

Figure 13.2: The same two pools, the same F_ST, and the same set of dominance deviations in all three bars – only their placement across loci differs. Expected heterosis moves by half again between the extremes. F_ST cannot see any of this, because d does not enter it.

F_{ST} is identical in all three columns of the table and in all three bars of Figure 13.2 – it is a function of the frequencies only, and d never enters it. Expected heterosis ranges over a factor of 1.55.

13.8.3 Isolating the divergence channel

One honest qualification. Sorting d against y^2 moves the *shared* term as well, because y^2 is not independent of $2\bar{p}(1-\bar{p})$: a locus with a large frequency difference tends to sit at an intermediate mean frequency. Part of the effect above therefore arrives through the shared half, not the divergence half.

The channel can be separated. Permute d only *within* groups of loci that share a value of the shared weight $2\bar{p}(1-\bar{p})$. The shared term is then invariant by construction – exactly, not approximately – and whatever moves is the divergence channel alone:

```
hz  <- 2 * ((pA + pB) / 2) * (1 - (pA + pB) / 2)
grp <- split(seq_len(ctx$m), round(hz, 9))
strat <- function(dec) {
  v <- numeric(ctx$m)
  for (i in grp) v[i[order(y2[i])]] <- sort(d[i], decreasing = dec)
  v
}
st <- do.call(rbind, lapply(c(aligned = FALSE, `anti-aligned` = TRUE), function(dec)
  as.data.frame(as.list(heterosis_partition(pA, pB, strat(dec))))))
fmt(st, 4)
```

	shared	divergence	total
aligned	322.7234	57.7211	380.4445
anti-aligned	322.7234	14.4716	337.1949

The shared column is identical to the digit; the divergence term still moves by a factor of 4 and the total by 13%. The alignment result survives its own control.

13.9 Reading the S1 panel again

Chapter 16. logs `frac_opposite_fixed` and `mean_abs_delta_p` every cycle as stand-ins for heterosis potential. The sweep above says how good they are.

`mean_sq_delta_p` is not a proxy at all. It is $\overline{y^2}$, so half of it is the unweighted divergence term GD_{BS}, exactly:

```
c(half_mean_sq = mean(sw$mean_sq_delta_p) / 2,
  divergence   = mean(sw$divergence))
#> half_mean_sq   divergence
#>     0.039983     0.039983
```

What it cannot carry is the weighting by d, which is the previous section's point; the frequency half of the divergence term it reports without loss.

`frac_opposite_fixed` is a different matter:

```
fmt(sw[, c("fst", "divergence", "frac_opposite_fixed", "mean_abs_delta_p",
           "mean_sq_delta_p")], 4)
```

fst	divergence	frac_opposite_fixed	mean_abs_delta_p	mean_sq_delta_p
0.02	0.0107	0.000	0.1138	0.0214
0.05	0.0171	0.000	0.1414	0.0341
0.10	0.0246	0.000	0.1715	0.0493
0.20	0.0433	0.002	0.2252	0.0867
0.30	0.0595	0.012	0.2589	0.1191
0.45	0.0846	0.041	0.3012	0.1693

Up to `fst = 0.10` it is exactly zero – not one marker in 2000 qualifies – and it first exceeds 1% of markers at `fst = 0.30`, near the top of the sweep. It counts *near-fixed* opposite loci, and the divergence lives at intermediate frequency differences, which the threshold discards.

Log `mean_sq_delta_p`. It is exact, it is monotone across the whole sweep, and it costs one squaring more than the statistic already being logged. `pool_complementarity()` and `s1_monitor()` return it.

13.10 General and specific combining ability

Everything so far has been about $\mathbb{E}[H]$. Now the spread.

The same expansion that produced the shared/divergence split produces the combining-ability decomposition, with no new machinery and no model fitting. Let $u_{ik} \in \{0, 1\}$ be the genotype of pool-A line i at locus k and v_{jk} that of pool-B line j, and write each as its pool frequency plus a deviation, $u = p_A + \delta u$ and $v = p_B + \delta v$. The per-locus heterozygosity of the cross, $u + v - 2uv$, expands to

$$u + v - 2uv = \underbrace{p_A + p_B - 2p_Ap_B}_{\text{mean}} + \underbrace{(1 - 2p_B)\,\delta u}_{\text{parent A}} + \underbrace{(1 - 2p_A)\,\delta v}_{\text{parent B}} - \underbrace{2\,\delta u\,\delta v}_{\text{both}} \tag{13.13}$$

Four terms: one constant, one linear in each parent, one bilinear. Weight by d_k, sum over loci, and the classical decomposition falls out,

$$H_{ij} = \mu + g_i^A + g_j^B + s_{ij} \tag{13.14}$$

with

$$g_i^A = \sum_k d_k (1 - 2p_{Bk})\, \delta u_{ik}, \qquad s_{ij} = -2 \sum_k d_k\, \delta u_{ik}\, \delta v_{jk} \tag{13.15}$$

and g_j^B symmetrically. What is linear in one parent is general combining ability: it is carried into every cross that parent enters. What is bilinear depends on *which two* lines met, and is specific combining ability. Because the deviations are taken from each pool's own line frequencies they sum to zero over that pool's lines, which is what makes the three components mean zero and mutually orthogonal. `heterosis_gca_sca()` is that expansion and nothing more:

```
heterosis_gca_sca
#> function (U, V, d)
#> {
#>     stopifnot(ncol(U) == ncol(V), length(d) == ncol(U))
#>     pA <- colMeans(U)
#>     pB <- colMeans(V)
#>     cU <- sweep(U, 2, pA)
#>     cV <- sweep(V, 2, pB)
#>     list(mu = heterosis_expected(pA, pB, d), gca_A = drop(cU %*%
#>         (d * (1 - 2 * pB))), gca_B = drop(cV %*% (d * (1 - 2 *
#>         pA))), sca = -2 * (cU * rep(d, each = nrow(U))) %*% t(cV))
#> }
#> <bytecode: 0x750d40aac0>
#> <environment: namespace:hybdiv>
```

It reconstructs every hybrid in the factorial exactly, and all three components are mean zero:

```
cs <- heterosis_gca_sca(U, V, d)
H  <- heterosis_value(ctx$X, d)
ia <- ctx$ped$a
ib <- ctx$ped$b - nA
c(max_residual = max(abs(H - (cs$mu + cs$gca_A[ia] + cs$gca_B[ib] +
                              cs$sca[cbind(ia, ib)]))),
  mu_vs_mean   = abs(cs$mu - mean(H)),
  mean_gca_A   = mean(cs$gca_A), mean_sca = mean(cs$sca))
#>  max_residual    mu_vs_mean    mean_gca_A      mean_sca
#>  7.958079e-13  1.136868e-13 -4.796163e-16 -1.264766e-15
```

This is not a decomposition that resembles the two-way analysis of variance. It *is* the two-way analysis of variance: μ, g^A and g^B are the grand mean and the row and column effects of the H_{ij} table, and s_{ij} is the interaction residual of the additive fit, to machine precision:

```
max(abs(resid(lm(H ~ factor(ia) + factor(ib))) - cs$sca[cbind(ia, ib)]))
#> [1] 6.966872e-12
```

R. E. Comstock, H. F. Robinson, and P. H. Harvey [7] designed reciprocal recurrent selection around the distinction between these two components. The algebra above says what each one is made of – and the next section says which of the two their scheme actually selects on.

13.10.1 GCA is an average effect in the other pool's background

Look at the weight on δu: it is $d_k(1 - 2p_{Bk})$, and $1 - 2p_B$ is $q_B - p_B$. That is the frequency factor of the classical average effect of a gene substitution, $a + d\,(q - p)$ [12, Ch. 7], with two differences: the additive part a is absent, because it belongs to the parental mean and not to H, and the frequencies are **pool B's**, not pool A's.

This is the whole reason combining ability is not a property of a line. A pool-A line's alleles are always substituted into a pool-B background, so its general combining ability for heterosis is a dominance effect *evaluated at the tester pool's frequencies*. Change the tester pool and every number changes. Two limits follow immediately:

- A locus at which the tester pool sits at $p_B = 0.5$ contributes **exactly zero** to g^A, whatever δu_{ik} is. It still contributes to g^B, through $(1 - 2p_A)\,\delta v_{jk}$, and to s_{ij}; what it cannot do is discriminate between pool-A parents.

- A locus at which the tester pool is fixed, $p_B \in \{0, 1\}$, has $\delta v \equiv 0$, so it contributes $\pm d_k\, \delta u_{ik}$ to g^A and nothing to either g^B or s_{ij}. The cross is then predictable from the pool-A parent by itself.

Between those two limits sits every real programme, and Section 13.11 is about which limit divergence pushes it towards.

This is also the exact sense in which R. E. Comstock, H. F. Robinson, and P. H. Harvey [7]'s scheme is built on the decomposition. Reciprocal recurrent selection evaluates each pool-A candidate on its cross performance against a *sample of the opposite pool*, and symmetrically for pool B. Averaging over testers removes s_{ij}, so of the heterosis in the decomposition above only g_i^A survives as a contrast between pool-A candidates. Note what that does *not* say: cross performance is a genotypic value, not a mid-parent difference, so it also carries the candidate's own additive contribution – the part that cancelled out of H in the mid-parent derivation above and is not in this decomposition at all. Selection on testcross means therefore acts on the additive part and g_i^A together. What the algebra here delimits is the dominance share of that response, and it says the share is general combining ability against the tester pool.

13.10.2 SCA is minus twice a weighted parental inner product

The bilinear term is $s_{ij} = -2\sum_k d_k\, \delta u_{ik}\, \delta v_{jk}$: minus twice the d-weighted inner product of the two parents' *centred* genotypes, a covariance up to the factor $1/m$. For $d_k > 0$, two parents that deviate from their pools in the same direction have a positive inner product and therefore a **negative** SCA.

Read in words: two parents that resemble each other – after removing what is typical of each pool – cross badly, and the resemblance that counts is weighted by d, not by marker count. Similarity enters negatively throughout this chapter – $1 - \theta_{AB}$, $d\,m\,(1 - f^L_{ab})$ – but everywhere else it is similarity measured against a fixed base. Here it is measured against each pool's own mean, which is what makes the term specific to a *pair* rather than a property either parent carries into all its crosses.

It is also Section 13.6 again. Since μ, g^A and g^B are the grand, row and column means of the H_{ij} table, s_{ij} is the double-centred H matrix. Under flat d, $H_{ij} = d\,m\,(1 - f^L_{ij})$, and double-centring annihilates the constant and flips the sign of what remains:

$$s_{ij} = -\,d\,m \cdot \mathrm{DC}\left(f^L\right)_{ij}, \tag{13.16}$$

with $\mathrm{DC}(M)_{ij} = M_{ij} - \bar{M}_{i\cdot} - \bar{M}_{\cdot j} + \bar{M}_{\cdot\cdot}$ over the $A \times B$ block. The factor $-dm$ is not cosmetic: s_{ij} is on the trait scale and f^L is a probability, and the minus is the same minus as in $-2\sum_k d_k \delta u\, \delta v$ – more coancestry than the two margins predict means *less* SCA. So the coancestry kernel determines SCA under exactly the condition it determines heterosis, up to that constant, and stops doing so under exactly the same condition: a d that varies across loci.

13.10.3 The variance components

Because the three components are orthogonal, the variance of heterosis over the factorial splits with them. Under the sampling model of the simulation – each line's genotype an independent Bernoulli(p_{Ak}) draw at every locus, so unrelated lines and linkage equilibrium within each pool – each piece has a closed form in the pool frequencies:

$$\sigma^2_{GCA_A} = \sum_k d_k^2\,(1 - 2p_{Bk})^2\, p_{Ak}q_{Ak}, \qquad \sigma^2_{SCA} = 4\sum_k d_k^2\, p_{Ak}q_{Ak}\, p_{Bk}q_{Bk} \tag{13.17}$$

```
qa <- pA * (1 - pA)
qb <- pB * (1 - pB)
fmt(data.frame(
  component = c("GCA_A", "GCA_B", "SCA"),
  observed  = c(var(cs$gca_A), var(cs$gca_B), var(as.vector(cs$sca))),
  closed_form = c(sum(d^2 * (1 - 2 * pB)^2 * qa),
                  sum(d^2 * (1 - 2 * pA)^2 * qb),
                  4 * sum(d^2 * qa * qb))), 2)
```

component	observed	closed_form
GCA_A	32.00	35.03
GCA_B	35.63	38.07

component	observed	closed_form
SCA	70.38	71.48

The closed forms take the sample frequencies as the pool frequencies; the observed column is one realisation of them. The GCA rows are variances over 25 lines, whose relative sampling error is about $\sqrt{2/n} \approx 28\%$, and the agreement is within that. That error rate belongs to the independent-Bernoulli sampling model declared immediately above, not to the statistic: it is the sampling error of a variance over n *independent* observations, and a recycled pool has no such n. Section 13.12.7 measures how far the agreement moves once the lines are related.

Note what each one requires. σ^2_{SCA} carries $p_A q_A\, p_B q_B$: it needs **both** pools to be polymorphic. Fix either pool at a locus and that locus contributes no specific combining ability at all, whatever its d_k. Within-pool diversity is not merely something the diversity constraint defends; it is the raw material of SCA.

13.11 Divergence destroys SCA faster than it builds GCA

Put the two closed forms next to the drift argument of Section 13.7 and the variance tells a different story from the mean. The mean was flat; the variance is not, and it does not merely change shape.

Under Balding-Nichols, $\mathbb{E}[p_A q_A] = p(1-p)(1-F_{ST})$ and $\mathbb{E}[(1-2p_B)^2] = (1-2p)^2 + 4F_{ST}\,p(1-p)$, and p_A and p_B are independent given the ancestral p. Per locus, in expectation over the draw,

$$\mathbb{E}\left[\,\sigma^2_{SCA,k}\,\right] = 4d_k^2\,p^2(1-p)^2\,(1-F_{ST})^2, \qquad \mathbb{E}\left[\,\sigma^2_{GCA_A,k}\,\right] = d_k^2\,p(1-p)\,(1-F_{ST})\big[(1-2p)^2 + 4F_{ST}\,p(1-p)\big] \tag{13.18}$$

Read the two expressions before reading the sweep, because they already say what it will show.

Both carry the erosion factor $(1-F_{ST})$, and both are **zero at $F_{ST}=1$**: a pool fixed at every locus has no two lines that differ, so it has no combining-ability variance of either kind. Divergence therefore cannot transfer variance from one component to the other indefinitely; it destroys both in the limit. SCA carries the factor twice, because it needs both pools polymorphic at the same locus. GCA carries it once, multiplied by the tester factor $(1-2p_B)^2$, whose expectation *grows* with F_{ST}.

The two expressions therefore behave differently in F_{ST}. The SCA expression is convex and decreasing on $[0,1]$: it is a positive constant times $(1-F_{ST})^2$, so it only ever falls. The GCA expression is a *concave* parabola, its F_{ST}^2 coefficient being $-4d_k^2p_k^2(1-p_k)^2$, and differentiating it gives the only turnover in this section. GCA variance rises from $F_{ST}=0$ at a locus if and only if $8p(1-p)>1$, that is $\tfrac{1}{2}(1-1/\sqrt{2}) < p < \tfrac{1}{2}(1+1/\sqrt{2})$, and turns over at

$$F_k^{\star} = \frac{8p_k(1-p_k)-1}{8p_k(1-p_k)}, \tag{13.19}$$

which is negative outside that interval – at such loci GCA variance falls from the start. A sum of concave parabolas is a concave parabola, and its maximum works out to the $d_k^2\,(p_kq_k)^2$-weighted mean of the $F_k^{\star}$. Whether that mean lands inside $[0,1]$ is a property of the ancestral frequency distribution, not a theorem: pools founded mostly at extreme frequencies would put it below zero and GCA variance would decline monotonically. Under the $U(0.05, 0.95)$ ancestral draw used here enough loci sit in the interval to place it inside the range, which is why the sweep below straddles a peak. The sweep's grid can show that the peak is interior; it cannot locate it.

What is monotone is the ratio,

$$\frac{\mathbb{E}[\sigma^2_{SCA,k}]}{\mathbb{E}[\sigma^2_{GCA_A,k}]} = \frac{4p(1-p)\,(1-F_{ST})}{(1-2p)^2 + 4F_{ST}\,p(1-p)}, \tag{13.20}$$

which falls in F_{ST} at every locus. So the defensible statement is about the *share*, not about a transfer. Expected heterosis, we already know, does not move at all. Sweeping the pools apart – same ancestral frequencies, fresh Balding-Nichols draws at each `fst`, as in Section 13.7 – and evaluating the closed forms on the realised frequencies:

```
reloc <- function(fst) {
  cf <- modifyList(book_cfg, list(fst = fst, seed = 5))
  Pf <- pool_freqs(simulate_lines(cf) / 2,
```

```
                  rep(c("A", "B"), c(cf$n_pool_A, cf$n_pool_B)))
  a <- Pf["A", ]; b <- Pf["B", ]
  va <- sum(d^2 * (1 - 2 * b)^2 * a * (1 - a))
  vb <- sum(d^2 * (1 - 2 * a)^2 * b * (1 - b))
  vs <- 4 * sum(d^2 * a * (1 - a) * b * (1 - b))
  data.frame(fst = fst, var_GCA = va + vb, var_SCA = vs,
             frac_SCA = vs / (va + vb + vs))
}
rl <- do.call(rbind, lapply(c(0.02, 0.05, 0.10, 0.15, 0.20, 0.30, 0.45), reloc))
fmt(rl, 3)
```

fst	var_GCA	var_SCA	frac_SCA
0.02	60.047	91.807	0.605
0.05	62.025	87.941	0.586
0.10	68.630	75.503	0.524
0.15	71.230	67.599	0.487
0.20	77.956	63.166	0.448
0.30	80.630	45.875	0.363
0.45	77.894	27.767	0.263

```
plot(rl$fst, rl$var_GCA, type = "b", pch = 19, col = "#d98e04",
     ylim = range(0, rl$var_GCA, rl$var_SCA),
     xlab = "target fst between pools",
     ylab = "variance of heterosis over the factorial")
lines(rl$fst, rl$var_SCA, type = "b", pch = 15, col = "#b2182b")
legend("right", c("GCA (both pools)", "SCA"), bty = "n", cex = 0.8,
       pch = c(19, 15), col = c("#d98e04", "#b2182b"))
grid()
```

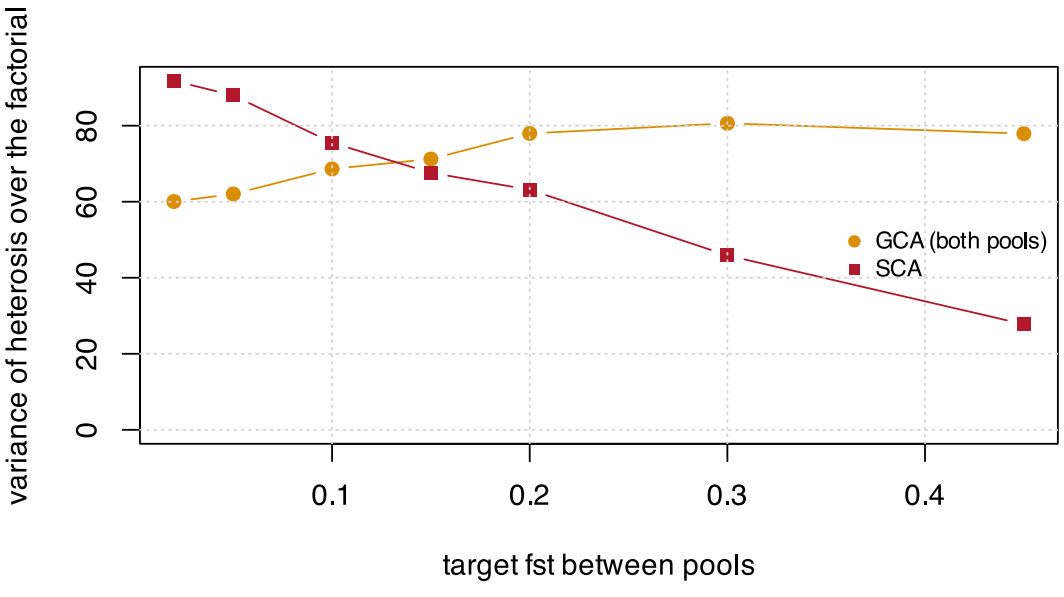

Figure 13.3: The same neutral divergence sweep, read through the combining-ability decomposition rather than through the mean. Expected heterosis is flat across this entire range (Figure 13.1); its decomposition is not. Specific combining ability falls steeply throughout. General combining ability rises, by much less than SCA loses, and itself turns over inside the range – both components vanish at F_ST = 1. The two curves do not sum to a constant: most of the SCA variance lost is destroyed rather than relocated, and the exact percentages are in the text.

Take the three columns in order, because the headline is in the arithmetic between them.

SCA variance falls by 70% across the sweep, a drop of 64 units. GCA variance ends 30% above where it started, a gain of 17.8 units, and peaks at an `fst` of 0.3 – interior, as the ancestral distribution used here leads one to expect, though a grid whose neighbouring points are 0.2 and 0.45 locates the peak only to that interval and cannot resolve it further. So the GCA gain recovers about 28% of what SCA lost. The rest is gone: total variance in heterosis over the factorial falls from 151.9 to 105.7, a 30% decline, while the mean stays fixed throughout.

Read those four percentages as one realisation, not as constants. They come from a single pair of seeds – `seed = 5` for the pool frequencies, `set.seed(4)` for d – with no replication and no dispersion reported, and they will move if either changes. What does not depend on the seed is the direction, which is already in the two closed forms above: σ^2_{SCA} is a positive multiple of $(1 - F_{ST})^2$ and can only fall, and both components vanish at $F_{ST} = 1$, so no amount of neutral divergence can hold the total up. The ordering of the columns is algebra; their magnitudes are this draw.

The share of that variance which is specific falls from 0.605 to 0.263. That statement – about the share – is the one the sweep supports; "the dominance expressed as SCA reappears as GCA" is not, and the two columns above are why.

The share result itself is not new in direction. J. C. Reif, F.-M. Gumpert, S. Fischer, and A. E. Melchinger [131] prove that the ratio of dominance to additive variance in a hybrid population falls as its two parent populations diverge, and conclude that superior hybrids then become predictable from general combining ability effects. The direction of this section's share result is theirs; the quantity is not the same one. Their ratio is σ^2_D/σ^2_A over the hybrid population, whose denominator contains the additive variance contributed by the parents – the part that cancelled out of H in the mid-parent derivation and is absent from everything computed here. The ratio in the table above is σ^2_{SCA} over $\sigma^2_{GCA} + \sigma^2_{SCA}$ *within heterosis only*. The two move the same way and are not interchangeable. What the sweep adds is the denominator, which a ratio hides: a ratio can fall because its numerator falls, because its denominator rises, or both, and the columns above say which. Here the numerator collapses and the denominator recovers only part of it, so the share falls and the total falls with it.

> *Neutral* divergence – drift, the only kind on this sweep – does not buy expected heterosis, and it does not move heterosis from one combining-ability component into the other. It **shrinks the part of hybrid variation that only a cross can reveal**, destroying SCA variance faster than it builds GCA variance. An additive model looks better against diverged pools mainly because there is less left for it to miss. Divergence that is *aligned* with dominance is a different matter and Section 13.8 is where it is measured; the sweep here holds alignment at zero by construction.

This sharpens a question 3 had to leave open. That chapter fits one additive effect per marker and notes that such a model cannot represent SCA at all. The sweep says how large the unrepresentable share is, and how it moves: against poorly separated pools most of the variance in heterosis is specific; against well separated ones a minority is. These are shares of the variance in *heterosis*, not of total hybrid value, whose parental-mean part is additive at every F_{ST}. F. Technow, T. A. Schrag, W. Schipprack, E. Bauer, H. Simianer, and A. E. Melchinger [52] report high accuracy for additive hybrid prediction in established, clearly separated dent and flint pools, which is the regime where this model puts the specific share at its lowest – consistent with the mechanism, though a single observational study in one crop cannot test it.

⚠ A smaller SCA share is not a better programme, and this is not a licence to maximise F_{ST}

Three things stay true at every point on that sweep. Expected heterosis does not change. The within-pool diversity spent to buy the divergence is spent for good. And $\sigma^2_{SCA} \to 0$ is the limit in which no cross is better than its parents predict – there is nothing left to *find* by testing combinations, which is the activity the whole factorial exists to support. Past the peak of the GCA curve in Figure 13.3 even the predictable half shrinks: at $F_{ST} = 1$ there is no combining-ability variance of either kind, because no two lines in a pool differ.

The falling SCA share is therefore a fact about what a predictor can express, not a gain. A programme reading it as one would be congratulating itself on having less to predict.

So this is a second, independent reason for Chapter 16. Trap 3's prescription to hold F_{ST} in a band rather than at a maximum. Section 13.7 gave the diversity-side reason. This is the prediction-side one, and it points the same way.

13.11.1 What this says about ranking crosses

The F1 of two fixed lines is genetically uniform, so the progeny-variance term that drives the usefulness criterion at Chapter 16. S2 is zero for the hybrid itself, and cross ranking reduces to ranking means. That is true, and it is why this book ranks hybrids on an index rather than on $\mu + ih\sigma$.

It does not follow that the ranking is easy. The *mean* of an F1 contains an SCA term that an additive $\mathbf{G}$ omits entirely, and Section 13.11 sized its share of the variance in heterosis: a majority of it against weakly separated pools, a minority against well separated ones, and zero only at the degenerate limit where one pool is fixed at every locus. The progeny variance collapses; the dominance deviation does not.

13.12 Running this on a real programme

Eight steps, in the order they have to be executed on a maize-type programme's own data. Each one names the function to call, the quantity to check and what failure looks like. Steps 1-5 need only genotypes and pool labels.

Steps 6-8 need more, and the requirement is worth naming because it is the reason these steps are rarely run. H_{ij} is a **mid-parent difference**, so fitting the decomposition on real data needs the hybrid trial *and* per-se phenotypes of the parent lines, measured on the same scale as the hybrid trial: a separate inbred-level trial, in the same environments, with a common check set or an overlapping design to put the two on one scale. Very few programmes run one, because inbred per-se performance has no commercial value. Without it what the two-way layout of step 6 fits is hybrid *performance*, not heterosis, and its row effects are then general combining ability **plus** the parent's own additive contribution – the term Section 13.10 shows is not in H at all. Both decompositions are legitimate; they are not the same decomposition, and the difference is exactly the parental additive part.

13.12.1 Step 1 – Verify the parents are inbred, and price it if they are not

Every formula in this chapter rests on the first premise. It is checkable in one line: an inbred line's self-coancestry is exactly 1, and its heterozygous fraction exactly 0.

```
c(min_self_coancestry = min(diag(molecular_coancestry(L / 2))),
  max_het_fraction    = max(rowMeans(L / 2 == 0.5)),
  dominance_value_of_the_most_heterozygous_line =
    max(abs(heterosis_value(rbind(U, V), d))))
#>                            min_self_coancestry
#>                                              1
#>                               max_het_fraction
#>                                              0
#> dominance_value_of_the_most_heterozygous_line
#>                                              0
```

If those numbers are not 1, 0 and 0, the parents are not fixed and the mid-parent already carries dominance of its own. The size of the resulting error is not small. Build parents that are *intra-pool* F1s instead of inbred lines – one random cross within each pool, so each parent sits at its pool's random-mating heterozygosity – then cross them across pools by drawing one gamete per parent per locus:

```
intra_pool_f1 <- function(W, n, seed) {
  set.seed(seed)
  (W[sample.int(nrow(W), n, TRUE), ] + W[sample.int(nrow(W), n, TRUE), ]) / 2
}
ZA  <- intra_pool_f1(U, nrow(U), 101)
ZB  <- intra_pool_f1(V, nrow(V), 102)
pd2 <- expand.grid(a = seq_len(nrow(ZA)), b = seq_len(nrow(ZB)))

# One gamete per parent per locus, five independent draws of the whole factorial.
draw <- t(vapply(1:5, function(r) {
  set.seed(200 + r)
  gA <- matrix(rbinom(length(ZA), 1, ZA), nrow(ZA))[pd2$a, ]
  gB <- matrix(rbinom(length(ZB), 1, ZB), nrow(ZB))[pd2$b, ]
  H  <- heterosis_value((gA + gB) / 2, d)
```

```
  mp <- (heterosis_value(ZA, d)[pd2$a] + heterosis_value(ZB, d)[pd2$b]) / 2
  c(hybrid_value = mean(H), mid_parent_gain = mean(H - mp))
}, numeric(2)))

closed_form <- heterosis_expected(colMeans(ZA), colMeans(ZB), d)
c(parent_het_fraction   = mean(c(ZA, ZB) == 0.5),
  closed_form           = closed_form,
  simulated_hybrid      = mean(draw[, "hybrid_value"]),
  simulated_gain        = mean(draw[, "mid_parent_gain"]),
  # The parental term is fixed across draws, so both columns carry the same s.d.
  sd_across_draws       = sd(draw[, "hybrid_value"]),
  exact_sum_d_y2        = sum(d * (pA - pB)^2),
  overstatement         = closed_form / mean(draw[, "mid_parent_gain"]))
#> parent_het_fraction         closed_form    simulated_hybrid      simulated_gain
#>            0.294260          355.428354          355.106663           67.222488
#>     sd_across_draws      exact_sum_d_y2       overstatement
#>            1.708396           65.890423            5.287343
```

Result. The parents carry 0.294 of their loci heterozygous. `heterosis_expected()` is still correct about the *hybrid*: it predicts 355.4 against a simulated 355.1 (s.d. 1.71 over five gamete draws), because that formula uses gamete frequencies only and does not care what the parents' own genotypes are. What it is not correct about is the *gain*: the mid-parent difference is 67.2, the same 1.71 standard deviation shifted down by the parents' own dominance value, so reading the closed form as heterosis overstates the gain by a factor of 5.29. The simulated gain agrees with the exact $\sum_k d_k y_k^2$ of Section 13.3, 65.9, to within one standard deviation of the gamete draw.

Interpretation, with its limit. The correction is not a fudge factor: it is the parents' own dominance contribution, 287.9 here, which `heterosis_value()` computes on the parent matrix directly and which is exactly zero under the inbred premise. Subtracting it recovers the mid-parent gain, and that gain matches the $\sum_k d_k y_k^2$ of Section 13.3 **in expectation only**: the realised intra-pool F1 parents are one draw, not their own expectation, so each replicate's mid-parent carries a sampling deviation of its own – which is why the agreement reported above is to within one standard deviation of the gamete draw and not exact. The factor of 5.29 itself is specific to this configuration – `fst` = 0.15, gamma-distributed d, parents at full random-mating heterozygosity – and it shrinks toward 1 as the parents approach fixation. Five gamete draws at one seed family give the dispersion reported and no more.

Call. `diag(molecular_coancestry(GL))`; `heterosis_value()` on the parent matrix, not only on the hybrids. **Check.** Self-coancestry 1 and heterozygous fraction 0 for every parent. **Warning sign.** Any parent with a heterozygous fraction above the marker error rate. Partially inbred parents (S1, S2, clonal maintenance of a heterozygous plant) invalidate the mid-parent cancellation, not the hybrid prediction. **Threshold.** `heterosis_value()` on the parent set must return zero to machine precision. If it does not, subtract it: the mid-parent gain is the hybrid value minus the parents' mean, always.

13.12.2 Step 2 – Compute pool frequencies, and keep them distinct from p_0

`pool_freqs(GL, pool)` returns one row per pool. These are the *current* cycle's frequencies and they are supposed to move; they are not the frozen base p_0 that Chapter 12.'s two inbreeding lenses are measured against. Both objects are frequency vectors over the same markers and confusing them produces a heterosis prediction anchored to a population that no longer exists.

Call. `pool_freqs(GL / 2, pool)` on the current cycle's line matrix. **Check.** `nrow() == 2` and the pool labels are the heterotic groups, not families or locations. **Warning sign.** A pool frequency vector computed over *hybrids* rather than lines. On the full factorial that gives $(p_A + p_B)/2$ for both pools – Section 12.7.1 proves the identity – which sets $y \equiv 0$ and reports the divergence term as zero. **Threshold.** None; this is a bookkeeping step, and the only failure is silent.

13.12.3 Step 3 – Decide what stands in for d_k, and say so

Nothing below step 4 can be computed without a weight per locus. Three defensible choices, in decreasing order of what they require:

1. **Estimated d_k** from a dominance model fitted on hybrid data. This is what the third premise assumes exists, and it is the only option that lets $\mathrm{cov}(d, y^2)$ be computed.

2. **Flat $d_k \equiv 1$**, which makes H proportional to marker distance and the whole calculation a function of the coancestry kernel (Section 13.6). Everything in this chapter's frequency algebra still holds; what is lost is alignment, which is exactly the term flat d sets to zero.
3. **No d at all**, reporting only GD_{BS} and `mean_sq_delta_p`, which is the frequency half of the divergence term and is exact (Section 13.8 says what is missing).

The cost of choice 2 is measurable, and it is the correlation already computed in Section 13.6: under flat d heterosis and marker distance are the same statistic, and under the gamma-distributed d used here the correlation between them is 0.78.

Call. `heterosis_value(X, d)` with whichever vector you have chosen, and record the choice next to the number. **Check.** `length(d) == ncol(X)` and d indexed in the same marker order. **Warning sign.** A report that gives expected heterosis without saying which of the three choices produced it. At the correlation just quoted the choices are not interchangeable for ranking crosses, and their absolute values are on different scales altogether. **Threshold.** If choice 2 is used, state that alignment is assumed zero, because that is what flat d asserts.

13.12.4 Step 4 – Partition, and cross-check against the diversity partition

`heterosis_partition(pA, pB, d)` returns `shared`, `divergence` and `total`. With $d \equiv 1$ and divided by m the first two are the GD_T and GD_{BS} of `gd_partition()`, and the chunk in Section 13.3 checks that to twelve digits. Running both is a consistency test worth the two lines it costs – but only if the two calls are given the **same line weights**, and by default they are not.

The algebra says why. For any weights w over the lines of a pool, with inbred lines, $\theta_A = \overline{p_{A,w}^2 + q_{A,w}^2}$ where $p_{A,w} = \sum_i w_i u_{ik}$ is the *weighted* pool frequency, and the same for B and for θ_{AB}. Substituting into `gd_partition()`'s definitions, $GD_T = 1 - \bar{f} = \overline{2\bar{p}(1-\bar{p})}$ and $GD_{BS} = \tilde{f} - \bar{f} = \overline{(p_{A,w} - p_{B,w})^2}/2$, which are exactly `heterosis_partition()`'s two terms evaluated at $p_{A,w}$ and $p_{B,w}$. So the identity holds for *any* weighting – as long as both sides use the same one. `gd_partition(idx, ctx)` weights each line by how often the selection uses it (`line_weights()`), while `pool_freqs()` is unweighted. On the full factorial usage is uniform and the two coincide; on a selected subset they do not, and the discrepancy is not small:

```
i_trunc <- sel_truncation(ctx, 60)     # index truncation: deliberately unbalanced
gp_sel  <- gd_partition(i_trunc, ctx)
GLh <- L / 2
wt  <- line_weights(i_trunc, ctx)
hp2 <- function(a, b) {
  heterosis_partition(a, b, rep(1, ctx$m))[c("shared", "divergence")] / ctx$m
}
cmp <- rbind(
  `all lines, unweighted`  = hp2(pA, pB),
  `lines used, unweighted` = hp2(colMeans(GLh[unique(ctx$ped$a[i_trunc]), ]),
                                 colMeans(GLh[unique(ctx$ped$b[i_trunc]), ])),
  `usage-weighted`         = hp2(as.vector(wt[["A"]] %*% GLh),
                                 as.vector(wt[["B"]] %*% GLh)),
  `gd_partition(idx, ctx)` = c(gp_sel[["GD_T"]], gp_sel[["GD_BS"]]))
stopifnot(isTRUE(all.equal(unname(cmp["usage-weighted", ]),
                           c(gp_sel[["GD_T"]], gp_sel[["GD_BS"]]))))
fmt(data.frame(cmp,
               shared_err_pct     = 100 * (cmp[, "shared"] / gp_sel[["GD_T"]] - 1),
               divergence_err_pct = 100 * (cmp[, "divergence"] / gp_sel[["GD_BS"]]
- 1)), 3)
```

	shared	divergence	shared_err_pct	divergence_err_pct
all lines, unweighted	0.328	0.034	5.020	−35.899
lines used, unweighted	0.326	0.039	4.351	−26.403
usage-weighted	0.312	0.052	0.000	0.000
gd_partition(idx, ctx)	0.312	0.052	0.000	0.000

```
c(lines_used_A = length(unique(ctx$ped$a[i_trunc])),
  lines_used_B = length(unique(ctx$ped$b[i_trunc])),
  max_line_use = max_line_use(i_trunc, ctx), n_sel = length(i_trunc))
#> lines_used_A lines_used_B max_line_use        n_sel
#>           10           23           23           60
```

Result. With usage weights on both sides the identity is exact to machine precision – the `stopifnot()` enforces it – while the unweighted pool frequencies of `pool_freqs()` overstate `shared` by +5.0% and understate `divergence` by 35.9% on this 60-hybrid truncation set, which uses only 10 pool-A lines with one of them in 23 of the 60 crosses. Restricting the unweighted mean to the lines actually used recovers part of the gap and not the rest, because the usage *within* the used set is itself far from uniform.

Interpretation. The two functions are not inconsistent; they answer the same question on two different populations – a notional pool in which every line is equally represented, and the pool as the selection actually weights it. Which one a report wants depends on the question, and the only error is comparing one with the other and calling the difference a bug. The order of magnitude quoted here is one selection at one scale; a plan with a usage cap (`max_use`) sits between the two rows.

Call. `heterosis_partition()` on the pool frequencies and `gd_partition(idx, ctx)` on the hybrid set – with the pool frequencies weighted by `line_weights(idx, ctx)` if the two are to be compared at all. **Check.** `shared / m == GD_T` and `divergence / m == GD_BS` under flat d, **both computed from usage-weighted pool frequencies**. On the full factorial, or under any uniform usage, the unweighted `pool_freqs()` satisfies this too. **Warning sign.** A disagreement on a selected subset is almost always the weighting above, not a coding error: check `line_weights()` first, and only then whether the lines are coded 0/1 or the pool labels differ between the calls. A GD_{WI} that is not zero is the separate failure – with inbred parents it must be, and a non-zero value means the line matrix contains heterozygotes, back to step 1. **Threshold.** Machine precision when both sides carry the same weights. With unweighted pool frequencies against a selected subset, expect tens of per cent on the divergence term, of the size measured above, and do not treat it as a tolerance to tighten.

13.12.5 Step 5 – Log `mean_sq_delta_p` every cycle

`s1_monitor(GL / 2, pool, cycle)` is the per-cycle panel. Of its three divergence columns only `mean_sq_delta_p` is exact – it is $\overline{y^2}$, so half of it is GD_{BS} – and `frac_opposite_fixed` is zero across most of the useful range, as the sweep in Section 13.7 shows.

Call. `s1_monitor()` once per cycle, appended to a single data frame. **Check.** `mean_sq_delta_p / 2` against `gd_partition()`'s `GD_BS`. **Warning sign.** `frac_opposite_fixed` used as the divergence statistic. In the sweep of Section 13.7 it is exactly zero at every `fst` up to 0.10 – not one marker in 2000 – while over the same span the divergence term has already risen by a factor of 2.3 (0.0107 to 0.0246 per locus at $d = 1$). The first non-zero entry is at `fst` = 0.2, by which point divergence has risen 4.1-fold. **Threshold.** A *band* on F_{ST}, not a maximum. Section 13.7 gives the diversity-side reason and Section 13.11 the prediction-side one.

13.12.6 Step 6 – Check that the crossing layout identifies what you intend to fit

This is the step most often skipped, and it is arithmetic, not opinion. The decomposition $H_{ij} = \mu + g_i^A + g_j^B + s_{ij}$ is the two-way layout with identified rows and columns. Count the degrees of freedom the layout actually provides:

```
layout_df <- function(n_testers) {
  k <- which(ib <= n_testers)
  a <- factor(ia[k]); b <- factor(ib[k])
  X_add <- if (n_testers > 1) model.matrix(~ a + b) else model.matrix(~ a)
  X_sat <- if (n_testers > 1) model.matrix(~ a * b) else X_add
  data.frame(testers = n_testers, crosses = length(k),
             rank_mu_gca = qr(X_add)$rank, rank_with_sca = qr(X_sat)$rank,
             sca_df = qr(X_sat)$rank - qr(X_add)$rank,
             residual_df = length(k) - qr(X_sat)$rank)
}
fmt(do.call(rbind, lapply(c(1, 2, 5, book_cfg$n_pool_B), layout_df)))
```

testers	crosses	rank_mu_gca	rank_with_sca	sca_df	residual_df
1	25	25	25	0	0
2	50	26	50	24	0
5	125	29	125	96	0
25	625	49	625	576	0

Result. With a single tester the $\mu + g^A$ model already has rank equal to the number of crosses, so `sca_df` is 0: specific combining ability is not estimable, not merely imprecise. From two testers onward `sca_df` is positive. `residual_df` is 0 in every row, because there is one observation per cross – SCA becomes *identifiable* from the layout at two testers and *testable* only when the trial adds replication.

How much a small tester set costs for general combining ability is a separate question with a closed-form answer. Averaging a pool-A line over k testers leaves it with g_i^A plus the mean of k specific deviations, so the correlation between the testcross mean and the true g^A is

$$r_k = \left[1 + \frac{\sigma^2_{SCA}}{\sigma^2_{GCA_A}} \cdot \frac{n_B - k}{k\,(n_B - 1)} \right]^{-1/2} \tag{13.21}$$

where the second factor is the finite-population correction for drawing k of the n_B available testers without replacement; at $k = n_B$ it vanishes and $r_k = 1$ exactly, because the row mean of the complete factorial *is* $\mu + g_i^A$. Twenty random tester sets per k:

```
Htab  <- matrix(H, nA, book_cfg$n_pool_B)   # rows = pool A, columns = pool B
stopifnot(isTRUE(all.equal(Htab, cs$mu + outer(cs$gca_A, cs$gca_B, "+") + cs$sca)))
v_gca <- var(cs$gca_A)
v_sca <- var(as.vector(cs$sca))
nBt   <- book_cfg$n_pool_B
tester_scan <- function(k, reps = 20) {
  r <- vapply(seq_len(reps), function(i) {
    set.seed(55 + 1000 * k + i)
    cor(rowMeans(Htab[, sample.int(nBt, k), drop = FALSE]), cs$gca_A)
  }, numeric(1))
  data.frame(testers = k, cor_mean = mean(r), cor_sd = sd(r),
             predicted = 1 / sqrt(1 + (v_sca / v_gca) * (nBt - k) / (k * (nBt - 1))))
}
ts <- do.call(rbind, lapply(c(1, 2, 3, 5, 10, nBt), tester_scan))
stopifnot(isTRUE(all.equal(ts$cor_mean[ts$testers == nBt], 1)))
# The residual is only interpretable where the tester sets differ at all; at
# k = n_B there is one possible set, so cor_sd is a rounding residue and the
# ratio would be 0/0 dressed up as a number.
ts$resid_sd <- ifelse(ts$cor_sd > 1e-12,
                      (ts$cor_mean - ts$predicted) / ts$cor_sd, NA)
fmt(ts, 4)
```

testers	cor_mean	cor_sd	predicted	resid_sd
1	0.4931	0.1250	0.5591	−0.5283
2	0.6972	0.1021	0.6978	−0.0063
3	0.7996	0.0448	0.7734	0.5843
5	0.8518	0.0407	0.8555	−0.0887
10	0.9403	0.0171	0.9376	0.1552
25	1.0000	0.0000	1.0000	NA

Result. A single tester ranks pool-A lines for general combining ability with a correlation of 0.49 (s.d. 0.12 over 20 tester choices); five testers give 0.85 and ten give 0.94. No k deviates from the closed form by as much as one standard deviation of the tester-set spread: the two largest residuals are $k = 1$ at −0.5 s.d. and $k = 3$ at 0.6 s.d., in opposite directions, and $k = 3$ is the larger of the two. Singling out $k = 1$ would be reading the *absolute* gap (0.066, the largest in the table) while ignoring that $k = 1$ also has by far the largest dispersion by construction – one tester, one draw of s_{ij}, nothing averaged. Measured against the standard error of a 20-replicate mean rather than against the single-draw spread, both residuals are larger than the noise (-2.4 and 2.6 standard errors), so the closed form is not exact at small k and this section does

not claim it is. Two reasons are identifiable and neither is modelled here: r_k is a function of an *expected* sampling variance, whereas the table averages 20 correlations, and $x \mapsto (1+x)^{-1/2}$ is convex, so the two are not the same quantity; and $\mathrm{cov}_i(g_i^A, \bar{s}_{i\cdot})$ vanishes only when the mean runs over all n_B testers, not over a subset. The scan is reported as a measurement with its dispersion, not as a validated formula.

Interpretation. The whole scan is driven by one ratio, $\sigma^2_{SCA}/\sigma^2_{GCA_A} = 2.2$ here. That ratio is what Section 13.11 shows falling as the pools diverge, so the number of testers a programme needs is not a constant of the crop: well separated pools need fewer. Two limits. This is heterosis only – the parental additive contribution to testcross performance is not in H and is not tested here – and it is one realisation of d and one pool pair, so the ratio, and with it every entry in the table, moves with the seed.

Call. `qr(model.matrix(...))$rank` on the layout you actually have, before fitting anything; `heterosis_gca_sca()` only once the layout supports it. **Check.** `sca_df > 0` if SCA is to be estimated; `residual_df > 0` if it is to be tested. **Warning sign.** A testcross file against one inbred tester per pool, analysed as though it estimated SCA. It estimates $\mu + g_i^A +$ (that tester's contribution) and nothing else. **Threshold.** Two testers to identify SCA; from the scan above, five to ten to rank GCA at $r > 0.85$ at this $\sigma^2_{SCA}/\sigma^2_{GCA}$ ratio – and recompute the requirement when the ratio changes.

13.12.7 Step 7 – Fit the decomposition and check it reconstructs

`heterosis_gca_sca(U, V, d)` is the expansion, not a model fit: it needs no solver and returns μ, `gca_A`, `gca_B` and the `sca` matrix. On simulated d it must reconstruct every hybrid exactly, and the chunk in Section 13.10 asserts that together with the `lm()` interaction equivalence. On real data d is unknown, so the route reverses: fit `lm(H ~ factor(a) + factor(b))`, take the row and column effects as g^A and g^B and the residual as s_{ij}. The two routes coincide only on a complete, balanced factorial.

Call. `heterosis_gca_sca()` when d is known (simulation); `lm()` with both parent factors when it is not. **Check.** The reconstruction residual, and that all three components are mean zero. **Warning sign.** A non-zero mean for `gca_A` or `sca`. The deviations are taken from each pool's own line frequencies, so they sum to zero over that pool by construction; a non-zero mean means the frequencies used were not the pool's. **Threshold.** Machine precision for the reconstruction. **For the variance components there is no threshold that transfers to a real pool**, and the $\sqrt{2/n}$ figure quoted in Section 13.10 is not one. That tolerance is the relative sampling error of a variance over n **independent** observations, and it is valid there only because the closed forms are derived under the simulation's sampling model – each line's genotype an independent Bernoulli(p_k) draw at every locus, hence unrelated lines and linkage equilibrium within pool. A recycled heterotic pool satisfies neither: assumption A3 of Section 12.1 makes within-pool recycling the normal state of this system, so its lines share ancestors.

The size of the failure can be measured, at least for the relatedness half of it. Re-derive each pool as 25 doubled haploids drawn from crosses among six of its own members – one Bernoulli draw per locus, so the lines are **related but carry no linkage disequilibrium** – and compare the closed forms with the observed variances on five such pairs of pools:

```
rel_dev <- function(A, B) {                    # % deviation, observed vs closed form
  p1 <- colMeans(A); p2 <- colMeans(B)
  cc <- heterosis_gca_sca(A, B, d)
  obs <- c(var(cc$gca_A), var(cc$gca_B), var(as.vector(cc$sca)))
  cf  <- c(sum(d^2 * (1 - 2 * p2)^2 * p1 * (1 - p1)),
           sum(d^2 * (1 - 2 * p1)^2 * p2 * (1 - p2)),
           4 * sum(d^2 * p1 * (1 - p1) * p2 * (1 - p2)))
  100 * (obs / cf - 1)
}
recycle <- function(W, n_found) {              # DH lines from crosses among founders
  fo <- W[sample.int(nrow(W), n_found), , drop = FALSE]
  t(vapply(seq_len(nrow(W)), function(i) {
    pr <- sample.int(n_found, 2)
    rbinom(ncol(W), 1, (fo[pr[1], ] + fo[pr[2], ]) / 2)
  }, numeric(ncol(W))))
}
dev <- t(vapply(1:5, function(r) {
  set.seed(500 + r)                              # seeded per draw, not once
  rel_dev(recycle(U, 6), recycle(V, 6))
}, numeric(3)))
```

```
dimnames(dev) <- list(paste0("draw", 1:5), c("GCA_A", "GCA_B", "SCA"))
fmt(rbind(dev, `independent lines` = rel_dev(U, V)), 1)
```

	GCA_A	GCA_B	SCA
draw1	0.3	46.1	5.5
draw2	−55.7	14.9	6.1
draw3	−31.9	−1.4	−2.4
draw4	−16.4	−29.9	8.8
draw5	−37.0	22.8	−10.3
independent lines	−8.6	−6.4	−1.5

Result. On the independently drawn lines the three closed forms are within 8.6% of the observed variances, inside the $\sqrt{2/n} = 28\%$ band. On the five recycled pairs 5 of the 15 components fall outside that band and the largest deviation is 56%. The two GCA columns carry almost all of it; the SCA column stays within 10.3%.

Interpretation, and its limits. A variance of a sum over loci picks up the between-line covariance that relatedness induces, which the closed forms – being sums of per-locus terms – do not contain; the GCA components are sums over loci weighted by the tester pool's frequencies and are the ones exposed. This says the tolerance must be recalibrated empirically on the programme's own pedigree, not that the closed forms are wrong: they remain exact statements about the sampling model they are derived under. The construction here is a caricature (six founders, one DH per F1, five draws, one seed family) and it isolates *relatedness only* – there is no linkage disequilibrium in it, because each locus is drawn independently. A real recycled pool violates the sampling model on both counts at once, and the size of the linkage contribution is not measured anywhere in this book.

13.12.8 Step 8 – Decide the F_{ST} band on alignment, not on F_{ST}

The output of steps 4-7 answers one management question: should the pools be pushed further apart? The chapter's answer is that F_{ST} cannot inform it (Section 13.7: the total does not move) and that the quantity that can is $\mathrm{cov}(d, y^2)$ (Section 13.8: a factor of 1.55 at constant F_{ST}).

Call. $\mathrm{cov}(d, y^2)$ with the population divisor, alongside $\bar{d}\,\overline{y^2}$; the two-term identity of Section 13.8 is the check. **Check.** The sign and magnitude of the covariance relative to $\bar{d}\,\overline{y^2}$. **Warning sign.** A proposal to raise F_{ST} justified by the divergence term alone. Under neutral divergence the shared term pays for all of it, unit for unit, and σ^2_{SCA} falls at the same time. **Threshold.** No universal one exists, and this chapter does not supply it: the missing weight λ between within- and between-pool diversity needs the two covariances listed in Section 13.13.1, and this book estimates neither.

13.13 What this does not change

> ⚠ The diversity constraint stays additive
>
> Nothing in this chapter reopens the optimisation. θ, GD and α are computed on molecular coancestry and are functions of the frequencies alone; no dominance term enters them, and Chapter 18., Chapter 19. and Chapter 20. are untouched. The combining-ability split of Section 13.10 sits outside the constraint for the same reason: it is a statement about the objective, not about the feasible set.
>
> A dominance model changes the **mean-performance ranking**, not the diversity term. Chapter 24. states the same separation from the other side. Do not conflate them, and do not read this chapter as licence to put a heterosis term inside the constraint.

What it does change is the interpretation of the constraint's output. Two plans with the same α and the same GD_{BS} can differ in expected heterosis by half again, if one of them concentrated its divergence on the dominance loci – that is the scenario table of Section 13.8, whose three columns share every frequency-based statistic. The partition cannot distinguish them. Only an estimate of d can.

13.13.1 What it would take to use d

Chapter 14. leaves a weight λ undetermined between within- and between-pool diversity and observes that the ratio of the two heterosis terms is what λ is trying to express. This chapter can now say what would have to be estimated to set it, and it is not much:

- **λ needs two covariances, not d itself.** Each heterosis term is m times a mean times a mean plus a covariance, so their ratio needs $\mathrm{cov}(d, y^2)$ and $\mathrm{cov}(d, 2\bar{p}(1-\bar{p}))$, each relative to $\bar{d}$, on top of what the frequencies already give. A programme does not need per-locus dominance deviations to weight the two diversity terms against each other; it needs to know whether its divergence is landing on its dominance loci.
- **Ranking crosses needs more.** s_{ij} is a d-weighted inner product between two specific parents, so it needs the deviations locus by locus. This is the extension Chapter 24. names and the book does not attempt.
- **Fitting it needs a factorial, and a testcross file is not one.** The decomposition above is the two-way layout with *identified* rows and columns: each pool-A line crossed to several named pool-B lines and each pool-B line to several named pool-A lines, which is what makes g^A, g^B and s_{ij} separately estimable and what the `lm()` above fits. Testcrosses against a single common tester give only a column of $\mu + g_i^A + \begin{pmatrix}\text{one tester's} \\ \text{contribution}\end{pmatrix}$ – no SCA is identifiable, because there is no second tester to contrast against. Programmes that already cross two pools in a partial factorial for hybrid evaluation have the layout; programmes that testcross against one inbred tester per pool do not.

13.14 Where this connects

Chapter 2. supplied the partition and Chapter 14. will use it; this chapter supplies the reason the GD_{BS} component is singled out for protection, and the reason protecting it is not the same as maximising it. Chapter 16. Trap 3 already recommended holding F_{ST} in a band: Section 13.7 is why a band, and Section 13.8 is what to watch instead of the band. Section 13.6 closes the loop with the coancestry kernel of Chapter 1.: under flat dominance, the diversity matrix and the heterosis prediction are the same object.

3 is the chapter this one supplies a missing term to. It predicts breeding values with an additive model on a trait that is additive by construction, and lists specific combining ability first among what it does not model. Section 13.10 is where that term comes from and Section 13.11 is what share of the variance in heterosis it holds; both are derived here rather than fitted, because on a simulated d they can be.

It also gives a reason why a heterotic *pattern* – which pool crosses well onto which – is not recoverable from how far apart the pools have drifted: Section 13.8′s three scenarios share every frequency-based statistic and differ by half again in expected heterosis. M. R. White, M. A. Mikel, N. de Leon, and S. M. Kaeppler [132] describe heterotic patterns and diversity separately in North American proprietary dent germplasm; this chapter says why the two need describing separately.

Forward, the alignment result is the argument for Chapter 22.. Drift moves frequencies independently of d, and selection on a line's own performance cannot respond to d either, because an inbred line's dominance contribution is exactly zero (Section 13.5) – which leaves selection on *cross* performance as the only process in this model that can move frequencies in a way that depends on where the dominance is. The alignment between divergence and dominance is therefore something each cycle either builds or erodes, not a standing asset. A one-cycle constraint cannot see that, and neither can F_{ST}. Chapter 7. runs that loop on a dominance trait and is the pipeline this chapter's algebra describes.

Reproduce this chapter

All the code of this chapter, in order and with the explanation kept as comments, is one script: **05b-heterosis-dominance.R**. Download it, install `hybdiv`, and run it top to bottom in an empty directory; cached results are fetched on first use.

14. A catalogue of metrics on $\mathbf{f}$

Every metric in this chapter has the same signature, `f(idx, ctx) -> scalar`, and every one operates on the molecular coancestry matrix from Chapter 1.. That uniformity is not tidiness: it is what makes metrics interchangeable inside an optimiser, and it is what lets Chapter 15. benchmark them against each other on equal terms.

The chapter also settles the question of what "no loss" means, which turns out to be less obvious than it sounds.

14.1 Group coancestry, and the off-diagonal problem

The group coancestry [133] of a set S of n individuals is the mean of **all** n^2 elements, diagonal included:

$$\theta_S = \frac{1}{n^2} \sum_{i \in S} \sum_{j \in S} f_{ij} = \frac{1}{n^2} \, \mathbf{1}' \mathbf{f}_S \mathbf{1}. \tag{14.1}$$

It is the probability that two alleles sampled at random *from the whole set* are identical – including the possibility that both come from the same individual. That is why the diagonal enters: it is the chance of sampling the same individual twice, which is entirely real when the set is used as a crossing pool.

Splitting the diagonal mean $\bar{f}_d$ from the off-diagonal mean $\bar{f}_o$,

$$\theta_S = \frac{\bar{f}_d}{n} + \frac{n-1}{n} \, \bar{f}_o. \tag{14.2}$$

So the off-diagonal mean – the metric many programmes already report – is one of two terms. When $\bar{f}_d$ varies little across individuals, which is the case in an all-F1 population, the two rank almost identically:

```
sels <- list(random     = sel_random(ctx, n_sel),
             trunc_cap  = sel_truncation_cap(ctx, n_sel),
             greedy_w1  = sel_greedy(ctx, n_sel, w = 1),
             greedy_div = sel_greedy(ctx, n_sel, w = 0),
             truncation = sel_truncation(ctx, n_sel))

cmp <- t(sapply(sels, function(s)
  c(theta = theta_group(s, ctx), offdiag = mean_offdiag(s, ctx))))
fmt(as.data.frame(cmp), 5)
```

	theta	offdiag
random	0.67426	0.67180
trunc_cap	0.67522	0.67280
greedy_w1	0.68759	0.68539
greedy_div	0.66976	0.66730
truncation	0.68759	0.68539

```
c(rank_correlation = cor(cmp[, "theta"], cmp[, "offdiag"], method = "spearman"))
#> rank_correlation
#>                1
```

The off-diagonal metric ranks correctly. What it does not give you is an interpretable scale or a defensible baseline – which is what the rest of this chapter supplies.

14.2 Gene diversity: θ and H_e are the same quantity

This is the result that simplifies everything downstream. Starting from the definition and swapping the order of summation, the double sum factors:

$$\frac{1}{n^2}\sum_i\sum_j x_{ik}x_{jk} = \Big(\frac{1}{n}\sum_i x_{ik}\Big)^2 = \bar{p}_k^2, \tag{14.3}$$

with $\bar{p}_k$ the **group's** allele frequency. Hence

$$\theta_S = \frac{1}{m}\sum_k \left(\,\bar{p}_k^2 + \bar{q}_k^2\,\right), \qquad GD_S = 1-\theta_S = \frac{1}{m}\sum_k 2\,\bar{p}_k\bar{q}_k \equiv H_e, \tag{14.4}$$

which is exactly Nei's gene diversity [134]. **Group coancestry and expected heterozygosity are not two independent checks. They are one quantity written two ways.**

```
c(via_submatrix = theta_group(idx, ctx),
  via_frequency = theta_from_freq(idx, ctx),
  difference    = theta_group(idx, ctx) - theta_from_freq(idx, ctx))
#> via_submatrix via_frequency    difference
#>     0.6734562     0.6734562     0.0000000

c(GD = gene_diversity(idx, ctx), He_Nei = he_nei(idx, ctx))
#>        GD    He_Nei
#> 0.3265438 0.3265438
```

Equality to machine precision is the implementation's strongest sanity check. If it fails, `f` is centered, rescaled, or built on the wrong marker coding.

Three practical consequences:

1. One fewer metric to justify to a technical committee.
2. "Losing 5% of diversity" acquires an exact meaning: 5% of the set's expected heterozygosity.
3. Two computation routes with the same answer and very different costs. The matrix route is $O(n^2)$; the frequency route is $O(nm)$. For $n = 200$ and $m = 25000$ that is forty thousand operations against five million.

```
reps <- 40
t_mat <- system.time(for (i in seq_len(reps)) theta_group(idx, ctx))[["elapsed"]]
t_frq <- system.time(for (i in seq_len(reps)) theta_from_freq(idx, ctx))[["elapsed"]]
c(us_matrix = 1e6 * t_mat / reps, us_frequency = 1e6 * t_frq / reps,
  ratio = t_frq / t_mat)
#>    us_matrix us_frequency        ratio
#>            0          175          Inf
```

The matrix route is the one that belongs in an optimiser's inner loop. Section 15.1 makes it cheaper still.

14.3 Status number N_s

D. Lindgren, L. D. Gea, and P. A. Jefferson [135] express group coancestry on the scale of individual counts:

$$N_s = \frac{1}{2\theta_S}. \tag{14.5}$$

N_s is the number of **unrelated, non-inbred** individuals that would have the same gene diversity as the observed set. The edge cases fix the intuition:

```
fake <- ctx
fake$f <- matrix(0.5, 5, 5)      # unrelated, p = 0.5
c(theta = theta_group(1:5, fake), Ns = status_number(1:5, fake))
#> theta    Ns
#>   0.5   1.0
```

```
fake$f <- matrix(1, 5, 5)        # all identical and homozygous
c(theta = theta_group(1:5, fake), Ns = status_number(1:5, fake))
#> theta    Ns
#>   1.0   0.5
```

⚠ N_s is a descriptor, not a drift projection

M. A. Toro, B. Villanueva, and J. Fernández [136] show that the **rate-based** effective size $N_e = 1/(2\Delta F)$ loses meaning under molecular diversity management: optimisation increases diversity in early generations, producing a negative ΔF and therefore a negative N_e, with no long-term asymptote.

This does not invalidate N_s. N_s is a monotonic reparametrisation of θ – a *static descriptor of a set* – while N_e is a *rate between generations*. Use N_s to compare sets; do not extrapolate it to project future drift. For the same reason, do not substitute the linkage-disequilibrium N_e here: that estimates population history, not the composition of a selected sample.

And, from Chapter 12.: θ tracks homozygosity, not drift. GD and N_s answer "how much heterozygosity does this selection preserve", which is the right question for next season's hybrid vigour. They do not certify that allele frequencies are moving as unmanaged drift would have moved them, which matters more across multiple recurrent cycles than for a single round of hybrid selection.

14.4 Effective number of parents

A metric independent of $\mathbf{f}$, computed purely from the pedigree. With p_i the frequency with which line i appears as a parent among the selected hybrids,

$$N_{e(\text{par})} = \frac{1}{\sum_i p_i^2}, \tag{14.6}$$

the inverse Simpson index. It captures precisely what coancestry blurs: a set of 200 hybrids can have an acceptable θ while drawing on only 12 lines, if those 12 happen to be genetically distant from each other.

```
c(sapply(sels, ne_parents, ctx = ctx), theoretical_max = ctx$n_lines)
#>          random       trunc_cap       greedy_w1      greedy_div      truncation
#>        39.34426        31.03448        13.01989        37.30570        13.01989
#> theoretical_max
#>        50.00000
```

Truncation collapses the parent base far more than it moves θ. Its cost is a `tabulate` call, which makes it a natural candidate to enter the objective function alongside θ – and in Chapter 15. it does.

14.5 Allelic richness

θ and H_e are averages across markers and are dominated by intermediate-frequency alleles. A lost rare allele barely moves them, yet the loss is irreversible. So count markers that were polymorphic in the population and became rare or fixed in the selection:

$$A_{\text{lost}} = \#\left\{k : \text{MAF}_k^{(S)} < 0.05 \wedge \text{MAF}_k^{(\text{pop})} \geq 0.05\right\}. \tag{14.7}$$

```
sapply(sels, alleles_lost, ctx = ctx)
#>     random  trunc_cap  greedy_w1 greedy_div truncation
#>         24         43        110         22        110
```

It is the only metric in this catalogue **genuinely non-redundant** with θ – a claim Chapter 15. tests rather than asserts.

E. López-Cortegano, R. Pouso, A. Labrador, A. Pérez-Figueroa, J. Fernández, and A. Caballero [137] compared, in a subdivided simulated population, strategies maximising *expected heterozygosity* (H_T, the same family as GD) against strategies maximising *allelic diversity* (A_T). Maximising A_T retained more alleles **and** controlled molecular inbreeding better than maximising H_T. The mechanism connects directly to Chapter 12.: heterozygosity-maximising management pushes allele frequencies toward 0.5, which favours rare alleles in the short run but does not stop them drifting to loss once they are no longer the rarest; allele-count management defends every segregating allele directly.

Their recommendation for structured conservation programmes was to make A_T, not H_T, the primary target. In this pipeline's terms: `alleles_lost` deserves a place next to θ as a *tracked outcome*, not only as a post-hoc sanity check, whenever the selection feeds several future cycles rather than a single hybrid release.

14.6 Distances: E-NE and A-NE

With distance $d_{ij} = 1 - f_{ij}$, Core Hunter 3 [138] defines two measures capturing opposite aspects that θ alone cannot separate:

$$\text{E-NE} = \frac{1}{n}\sum_{i \in S} \min_{j \in S,\, j \neq i} d_{ij}, \qquad \text{A-NE} = \frac{1}{N}\sum_{i=1}^{N} \min_{j \in S} d_{ij}. \tag{14.8}$$

- **E-NE**, entry-to-nearest-entry, measures **non-redundancy within the selection**. It distinguishes "200 individuals spread out" from "100 near-identical pairs spread out" – configurations that can share the same θ.
- **A-NE**, accession-to-nearest-entry, measures **representativeness of the population**. It detects a selection that optimises θ by concentrating in one region of genetic space and ignoring the rest.

```
fmt(t(sapply(sels, function(s) c(ENE = ene(s, ctx), ANE = ane(s, ctx)))), 4)
```

	ENE	ANE
random	0.2533	0.2471
trunc_cap	0.2536	0.2561
greedy_w1	0.2527	0.2523
greedy_div	0.2607	0.2504
truncation	0.2527	0.2523

Note the direction: A-NE is a distance to the *nearest selected entry*, so **lower is better** for representativeness.

14.7 Effective dimensionality

Over the eigenvalues of the submatrix, the participation ratio

$$D_{\text{eff}} = \frac{(\sum_i \lambda_i)^2}{\sum_i \lambda_i^2} \tag{14.9}$$

counts how many independent directions of genetic variation remain: 1 if all variation lies on a single axis, n if spread evenly.

```
fmt(sapply(sels, eff_dim, ctx = ctx), 2)
#>     random  trunc_cap  greedy_w1 greedy_div truncation
#>      28.59      23.99      12.72      27.92      12.72
```

The submatrix is **double-centered** before the eigendecomposition. On a raw all-positive `f` the leading eigenvalue is just $n\theta$, so an uncentered version would only re-measure θ under a more expensive name – a mistake worth naming because it is easy to make and produces a plausible-looking number.

14.8 Decomposition by heterotic group

In hybrid maize the diversity **between** pools is the engine of heterosis and cannot be spent; the diversity **within** each pool is the fuel for future progress. Chapter 13. derives the first half of that sentence – `GD_BS` weighted by dominance deviations *is* the divergence half of expected heterosis – and qualifies it: divergence is worth having where it lands on the dominance loci, and worth nothing where it does not. Chapter 22. demonstrates the second half. Following A. Caballero and M. A. Toro [1], compute molecular coancestry between lines and weight it by each line's usage among the selected hybrids:

$$\theta_A = \mathbf{w}_A' \mathbf{f}^L \mathbf{w}_A, \qquad \theta_B = \mathbf{w}_B' \mathbf{f}^L \mathbf{w}_B, \qquad \theta_{AB} = \mathbf{w}_A' \mathbf{f}^L \mathbf{w}_B. \tag{14.10}$$

```
fmt(t(sapply(sels, theta_pools, ctx = ctx)), 4)
```

	theta_A	theta_B	theta_AB
random	0.7090	0.7095	0.6393
trunc_cap	0.7096	0.7153	0.6380
greedy_w1	0.7581	0.7219	0.6352
greedy_div	0.7053	0.7066	0.6336
truncation	0.7581	0.7219	0.6352

Feeding those into the Caballero & Toro partition of Chapter 2.:

```
fmt(t(sapply(sels, gd_partition, ctx = ctx)), 4)
```

	GD_WI_hyb	GD_T	GD_WI	GD_BI	GD_BS	F_ST
random	0.1801	0.3257	0	0.2907	0.0350	0.1074
trunc_cap	0.1816	0.3248	0	0.2876	0.0372	0.1146
greedy_w1	0.1823	0.3124	0	0.2600	0.0524	0.1678
greedy_div	0.1853	0.3302	0	0.2941	0.0361	0.1095
truncation	0.1823	0.3124	0	0.2600	0.0524	0.1678

Two things to read here. `GD_WI` is **exactly zero**: the parent lines are inbred, so there is no within-individual diversity at line level – the algebra of Chapter 2. predicted this, and it is why `GD_WI_hyb`, the heterozygosity actually carried inside the F1s, is reported separately. And `GD_BS` *rises* under truncation: selection concentrates on the most complementary line pairs, which pushes the pools further apart.

> ! A blind optimiser erodes one pool while global θ looks fine
>
> Without this decomposition that goes unnoticed, and the consequence appears only in the following cycle. `ne_lines_pool()` is the cheap version of the same alarm – it needs no `fL`, only the pedigree:

```
t(sapply(sels, ne_lines_pool, ctx = ctx))
#>            Ne_lines_A Ne_lines_B
#> random      18.556701   20.93023
#> trunc_cap   15.517241   15.51724
#> greedy_w1    4.488778   11.84211
#> greedy_div  18.947368   18.36735
#> truncation   4.488778   11.84211
```

14.8.1 Where this could go next

θ_A, θ_B and θ_{AB} as computed here are **diagnostic**: reported, not fed back into the objective. The subdivided-population literature offers a ready-made way to promote them into a tunable target. A. Caballero and S. T. Rodríguez-Ramilo [139], used by E. López-Cortegano, R. Pouso, A. Labrador, A. Pérez-Figueroa, J. Fernández, and A. Caballero [137] to manage two heterotic-like pools simultaneously, partition total diversity as

$$H_T = D_G + \lambda H_S, \qquad A_T = D_A + \lambda A_S, \tag{14.11}$$

where H_S and A_S are the within-pool components, D_G and D_A the between-pool components, and $\lambda \in [0, \infty)$ is a weight chosen by the breeder. $\lambda = 0$ optimises only between-pool divergence – protects heterosis, ignores each pool's internal health. $\lambda \to \infty$ optimises only within-pool diversity – protects each pool's future options, ignores the cross.

That maps directly onto the two constraints a maize breeder already has in mind but applies only qualitatively: *don't blend the pools* versus *don't paint either pool into a corner*. Turning λ into a second explicit knob alongside $\alpha_{\max}$ is the natural next step if pool erosion shows up in practice. The current implementation reports the ingredients and leaves the weighting to the analyst.

λ need not stay a matter of taste. Chapter 13. shows that the value of between-pool divergence is not a free parameter but a d-weighted quantity: the divergence term contributes $\sum_k d_k y_k^2/2$ against the shared term's $\sum_k d_k \, 2\bar{p}_k(1-\bar{p}_k)$. Given an estimate of the dominance deviations, the ratio of those two sums is what λ is trying to express, and the reason a purely frequency-based partition cannot pin it down is that d does not appear in the partition at all.

14.9 The correct baseline

14.9.1 Why not compare against the whole population

The natural comparison – coancestry of the selected set against that of the whole population – is **biased by construction**. Applying equation (1) to both:

$$\theta_S - \theta_N = \left(\frac{1}{n} - \frac{1}{N}\right)(\bar{f}_d - \bar{f}_o) > 0, \tag{14.12}$$

since $\bar{f}_d > \bar{f}_o$ whenever individuals are not clones. A **random** sample already looks less diverse than the population it came from, with no selection whatsoever. The bias is pure sampling and grows as the sample shrinks.

```
fd <- mean(diag(ctx$f)); fo <- mean(ctx$f[upper.tri(ctx$f)])
predicted_bias <- (1 / n_sel - 1 / ctx$N) * (fd - fo)

set.seed(11)
observed_bias <- mean(replicate(400, theta_group(sample.int(ctx$N, n_sel), ctx))) -
                 theta_group(seq_len(ctx$N), ctx)

c(predicted = predicted_bias, observed = observed_bias)
#>   predicted    observed
#> 0.002222092 0.002276690
```

The formula is exact; the residual is Monte Carlo error in the resampling.

Note also that the **off-diagonal** mean of a random subset *is* an unbiased estimator of the population's off-diagonal mean. So the metric many programmes already use is free of this particular bias – but it is also not comparable across sample sizes on the N_s scale.

14.9.2 Operational definition of loss

The "0% loss" reference is the **distribution of random subsets of the same size**:

$$\alpha = \frac{GD_{\text{random}(n)} - GD_S}{GD_{\text{random}(n)}}. \tag{14.13}$$

$\alpha = 0$ means "as diverse as a random sample of the same size"; $\alpha < 0$ means more diverse than chance. Resampling also delivers the standard deviation, and hence a z-score for any strategy.

```
null <- null_distribution(ctx, n_sel, B = 400, B_full = 60)
c(GD_population = gene_diversity(seq_len(ctx$N), ctx),
  GD_reference  = null$gd_ref,
  sd            = null$gd_sd,
  apparent_loss_if_wrong_baseline =
    (gene_diversity(seq_len(ctx$N), ctx) - null$gd_ref) /
```

```
    gene_diversity(seq_len(ctx$N), ctx))
#>                  GD_population                     GD_reference
#>                   0.3280928000                     0.3258385719
#>                             sd apparent_loss_if_wrong_baseline
#>                   0.0007654486                     0.0068707028
```

That last number is what a programme would report as a diversity loss if it compared against the full pool. It is entirely a sampling artefact.

```
fmt(sapply(sels, alpha_loss, ctx = ctx, gd_ref = null$gd_ref), 5)
#>     random  trunc_cap  greedy_w1 greedy_div truncation
#>    0.00032    0.00326    0.04122   -0.01352    0.04122
```

Negative values are selections *more* diverse than chance, which is exactly what the max-diversity greedy is for.

14.9.3 How to set the α limit

The acceptable limit **should not be chosen a priori.** Pure truncation – the greediest possible selection – defines the maximum attainable α. If that ceiling is below your intended limit, the constraint restricts nothing and you have bought a false sense of control.

```
attainable <- alpha_loss(sel_truncation(ctx, n_sel), ctx, null$gd_ref)
c(attainable_alpha_max = attainable,
  intended_limit       = 0.05,
  constraint_is_active = attainable > 0.05)
#> attainable_alpha_max       intended_limit constraint_is_active
#>           0.04121618           0.05000000           0.00000000
```

At the full simulated scale that ceiling comes out near **3.6%**, so a 5% limit would be inoperative. The correct procedure is to trace the α-versus-gain frontier and choose its knee, which is Chapter 19..

14.10 Every metric against the null

A metric without a baseline says nothing. The z-score of each metric against the random-subset null is how this book reports all of them:

```
tab <- evaluate(sels, ctx, null$gd_ref)
fmt(as.data.frame(discriminatory_power(tab, null)[
  , c("GD", "Ns", "Ne_parents", "F_hom", "F_drift", "alleles_lost", "ENE", "ANE")]),
1)
```

	GD	Ns	Ne_parents	F_hom	F_drift	alleles_lost	ENE	ANE
random	0.0	0.0	1.4	0.6	−1.2	0.0	−1.0	−0.4
trunc_cap	−1.3	−1.3	−2.7	2.2	3.1	3.3	−0.7	17.9
greedy_w1	−17.9	−17.6	−11.4	9.5	30.1	14.9	−1.5	10.2
greedy_div	6.1	6.2	0.4	−5.2	−0.7	−0.3	5.5	6.3
truncation	−17.9	−17.6	−11.4	9.5	30.1	14.9	−1.5	10.2

Read the rows against each other. Truncation is many standard deviations from random on `F_drift` and on `Ne_parents` while being far less extreme on `GD` – the constraint that would catch it is not the one most programmes apply. That observation is what Chapter 15. turns into a recommendation.

Reproduce this chapter

All the code of this chapter, in order and with the explanation kept as comments, is one script: **06-metric-catalogue.R**. Download it, install `hybdiv`, and run it top to bottom in an empty directory; cached results are fetched on first use.

15. Choosing metrics: collapse, cost, redundancy

Chapter 14. built a catalogue. This chapter asks which of it to keep, and answers with two questions the catalogue could not: **what does each metric cost per evaluation**, and **how much of what it reports is already in θ**.

The recommendation is **two** metrics inside the optimiser and **three** at validation.

Role	Metric	Where it runs	Cost per evaluation
Constraint	Group coancestry theta of the selected set, as proportional loss of gene diversity against a same-size resampling baseline	inside DE, every evaluation	**5 us** (incremental) / 106 us (full)
Second constraint	Effective number of parent lines	inside DE, every evaluation	**7 us**, shares the state vector
Validation	Marker alleles retained (allelic richness)	on the returned plan only	0.9 ms
Validation	Within/between heterotic-pool decomposition of theta	on the returned plan only	62 us
Validation	Status number Ns = 1/(2 theta)	free, algebraic	0

Three findings drive this, and each is derived and verified below rather than asserted.

15.1 Finding 1: the hybrid coancestry matrix is never needed

15.1.1 Derivation

Let lines $a, b, \ldots$ be homozygous, with $\mathbf{f}^L$ their molecular coancestry matrix. The hybrid (ab) carries one genome from each parent. Molecular coancestry is the probability that two alleles, one drawn at random from each individual, are identical in state. Drawing at random from hybrid (ab) selects parental genome a or b with probability $\frac{1}{2}$ each, and likewise for (cd), so

$$f_{(ab),(cd)} = \frac{1}{4}\left(f^L_{ac} + f^L_{ad} + f^L_{bc} + f^L_{bd}\right). \tag{15.1}$$

Now let a selection of n hybrids use line a exactly k_a times across all its parental slots, $\sum_a k_a = 2n$, and write $w_a = k_a/(2n)$. Group coancestry of the selected hybrids is the mean of f over all ordered pairs; substituting the identity above and collecting terms by line gives

$$\theta_S = \mathbf{w}^\top \mathbf{f}^L \mathbf{w}, \tag{15.2}$$

with the selection allele frequencies following the same collapse, $\bar{\mathbf{p}} = \mathbf{w}^\top \mathbf{G}^L$.

15.1.2 Four consequences

The $N \times N$ hybrid matrix is never required. Memory drops from $O(N^2)$ to $O(L^2)$. At $L = 300$ lines and $N = 22,500$ hybrids that matrix would occupy 3.77 GB; in the benchmark below it was deliberately never formed, and every metric that needs it simply has no timing entry at that scale.

The vector w is a sufficient statistic. Group coancestry, gene diversity, status number, effective number of parents, selection allele frequencies and the heterotic-pool decomposition are all functions of $\mathbf{w}$ alone. One $O(L)$ tabulation per evaluation serves all of them.

Swapping one hybrid is a rank-2 update. Exchanging hybrid h for h' changes at most four entries of $\mathbf{w}$, so θ updates in $O(L)$ rather than $O(L^2)$.

The discrete hybrid problem is exactly optimal-contribution selection on lines, restricted to the lattice $w_a \in \{0, 1/(2n), 2/(2n), \ldots\}$. That connects it directly to the OCS literature of Chapter 18., with the caveat that the lattice restriction makes it combinatorial rather than the continuous quadratic programme OCS usually solves.

15.1.3 Verification

Every identity was checked against literal transcriptions of the reference implementations across $F_{ST} \in \{0.05, 0.15, 0.35\}$, two seeds and $n \in \{20, 100\}$, thirty replicate selections each.

	Identity	Worst deviation
max_pairwise_f	pairwise f: hybrid vs line formula	1.1e-16
max_theta_sub_vs_line	theta: line route vs f submatrix	2.2e-16
max_theta_sub_vs_freq	theta: line route vs allele-frequency route	1.1e-16
max_pbar	selection allele frequencies	0 (exact)
max_He	Nei He	0 (exact)
max_Neparents	effective number of parents	2.1e-14
max_alleles_lost	allele loss at MAF threshold	0 (exact)
max_theta_pools	pool decomposition	1.2e-15
max_swap_update	incremental swap vs full recompute	4.4e-16
bitset_vs_literal	bitset allelic richness vs literal count	0 (exact)

The same identity, live, on the book's own data:

```
ctx  <- fast_ctx(simulate_data(book_cfg))
set.seed(51)
idx  <- sample.int(ctx$N, 60)
c(via_hybrid_matrix = theta_group(idx, ctx),
  via_line_route    = fast_theta(idx, ctx),
  difference        = theta_group(idx, ctx) - fast_theta(idx, ctx))
#> via_hybrid_matrix    via_line_route        difference
#>         0.6745897         0.6745897          0.0000000
```

And the incremental swap, which is what makes a local search affordable:

```
st  <- theta_state(idx, ctx)
out <- idx[1]; inn <- setdiff(seq_len(ctx$N), idx)[1]
st2 <- theta_swap(st, out, inn, ctx)
c(after_swap_incremental = st2$theta,
  after_swap_recomputed  = theta_group(c(idx[-1], inn), ctx),
  difference             = st2$theta - theta_group(c(idx[-1], inn), ctx))
#> after_swap_incremental  after_swap_recomputed            difference
#>           6.747225e-01           6.747225e-01          2.220446e-16
```

⚠ Two implementation traps found while building this

R's `$` performs **partial matching** on lists, so `ctx$f` silently returns `ctx$fL` when the hybrid matrix is absent – a wrong answer with no error. Every accessor in the package uses `ctx[["f"]]`. Any pipeline carrying both matrices in one list is exposed to this.

R integers are signed 32-bit, so `bitwShiftL(1L, 31L)` is `NA` and the usual multiply-based popcount overflows. The bitset in `pack_lines()` packs 31 bits per word, and `popcount32()` counts them with a branch-free SWAR routine.

15.2 Finding 2: what each metric costs

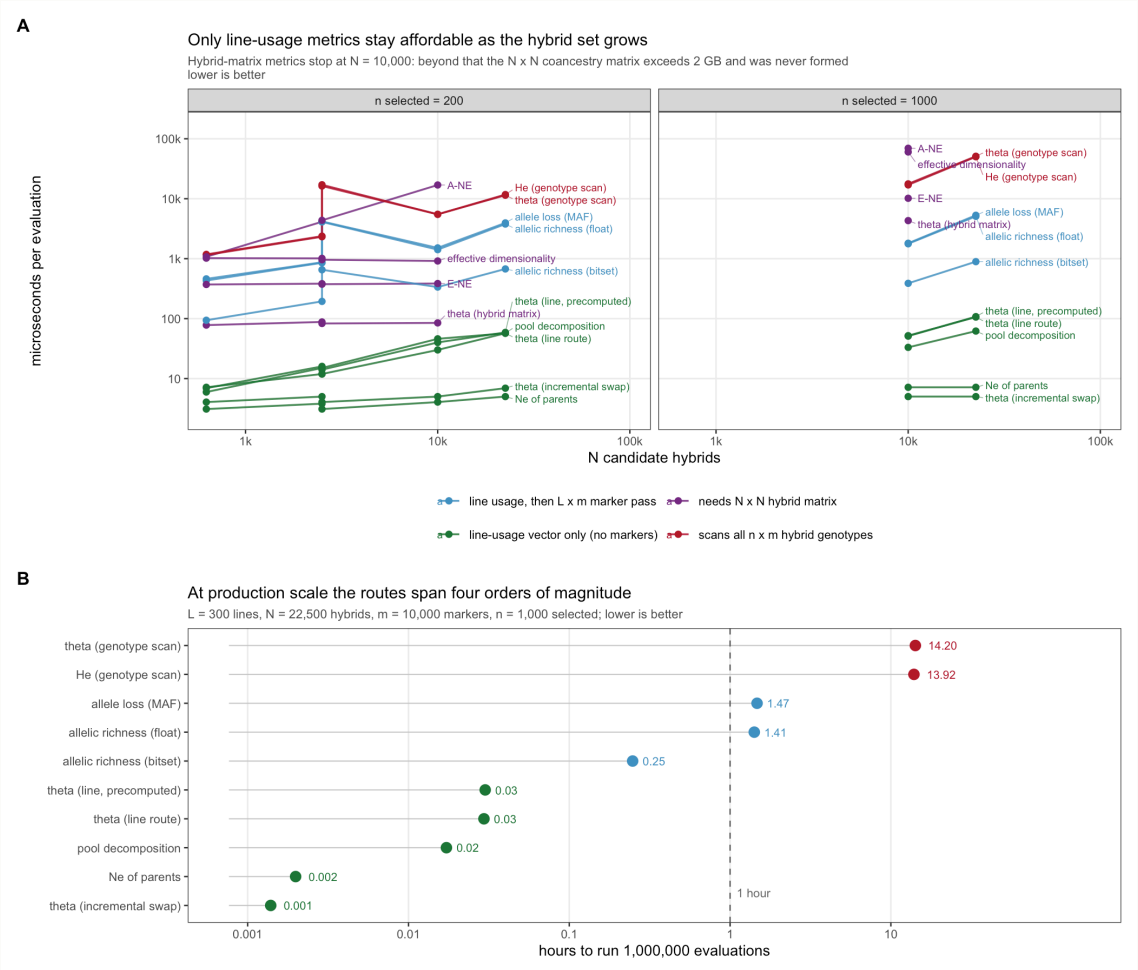

Figure 15.1: Cost per fitness evaluation. Panel A: scaling in the number of candidate hybrids. Panel B: hours to complete one million evaluations at production scale.

At $L = 300$ lines, $N = 22,500$ hybrids, $m = 10,000$ markers and $n = 1,000$ selected:

metric	us per evaluation	hours per 1e6 evals
theta: incremental swap	5.0	0.0014
Ne_parents	7.2	0.0020
pool decomposition	62.0	0.0172
theta: LINE route	106.1	0.0295
theta: line route precomp	108.0	0.0300
allelic richness: bitset	891.9	0.2478
allelic richness: float	5091.0	1.4142
allele loss (MAF thr)	5296.9	1.4714
He / gene diversity(freq)	50096.0	13.9156
theta: allele-freq route	51113.1	14.1981

Three thresholds matter:

- **The genotype-scan route is disqualified.** Computing θ from selection allele frequencies costs 14.2 hours per million evaluations. The line route costs 106 us – a **482x reduction** for an algebraically identical number.
- **The incremental swap is effectively free** at 5 us, or 1.4 seconds per million evaluations. $N_{e,\mathrm{par}}$ at 7 us shares the same state vector and is free in practice once θ has been computed.

- **Allelic richness cannot go inside the loop, and need not.** The bitset implementation is 5.7x faster than the floating-point version, but still 178x the cost of the line-route θ. It belongs at validation.

The same ordering is visible at the book's scale, where every route is affordable and so the comparison is about ratios, not feasibility:

```
fmt(cost_per_eval(ctx, n_sel = 60, reps = 40), 1)
```

metric	us_per_eval
theta_group	0
theta_from_freq	175
ne_parents	0
alleles_lost	150
ENE	50
ANE	475
eff_dim	100
Gst	1425

15.3 Finding 3: the metric space is nearly one-dimensional

621 selections were generated across four strategy families – random, truncation at varying intensity, greedy construction over a grid of diversity weights, and restricted line pools – and all 17 metrics computed on each.

Figure 15.2: Metric redundancy. A: Spearman correlations. B: PCA of rank-transformed metrics. C: the discriminating case, inside a narrow coancestry band.

PC1 absorbs 88% of the rank variance; PC2 adds 7%. Rank correlations with θ are $|\rho| \geq 0.93$ for status number, H_e, effective dimensionality, $N_{e,\mathrm{par}}$ and MAF-threshold allele loss. Reporting these alongside θ adds cost and length, not information.

But the marginal correlation is the wrong test. The question is whether a metric varies **at fixed θ** – because θ is the constraint, and any metric that is a deterministic function of it cannot discriminate among plans the optimiser treats as equivalent. Two analyses agree:

- After regressing each metric's ranks on θ (quadratic), **allelic richness retains 52% of its rank standard deviation** and $N_{e,\mathrm{par}}$ retains 36%. Effective dimensionality retains 25% – close to a restatement of θ.
- Inside a band $\theta \in [0.666, 0.670]$ containing 399 selections, **alleles retained still spans 171 markers** (11,680 to 11,851 of 12,000) and $N_{e,\mathrm{par}}$ spans **30.1 to 55.8**.

Plans the constraint cannot distinguish differ materially in how much allelic variation they carry forward. That is the empirical justification for the two-metric recommendation.

15.3.1 Reproducing the argument at book scale

```
set.seed(77)
grid <- do.call(rbind, lapply(seq(0, 4, length.out = 30), function(w) {
```

```
  s <- sel_greedy(ctx, 60, w = w)
  data.frame(w = w, theta = theta_group(s, ctx), Ne_par = ne_parents(s, ctx),
             lost = alleles_lost(s, ctx))
}))
rnd <- do.call(rbind, lapply(1:60, function(i) {
  s <- sample.int(ctx$N, 60)
  data.frame(w = NA, theta = theta_group(s, ctx), Ne_par = ne_parents(s, ctx),
             lost = alleles_lost(s, ctx))
}))
all <- rbind(grid, rnd)

plot(all$theta, all$Ne_par, pch = 19, col = ifelse(is.na(all$w), "grey60",
"#b2182b"),
     xlab = expression(theta), ylab = expression(N[e]~"parents"))
legend("topright", c("greedy sweep", "random"), pch = 19,
       col = c("#b2182b", "grey60"), bty = "n")
grid()
```

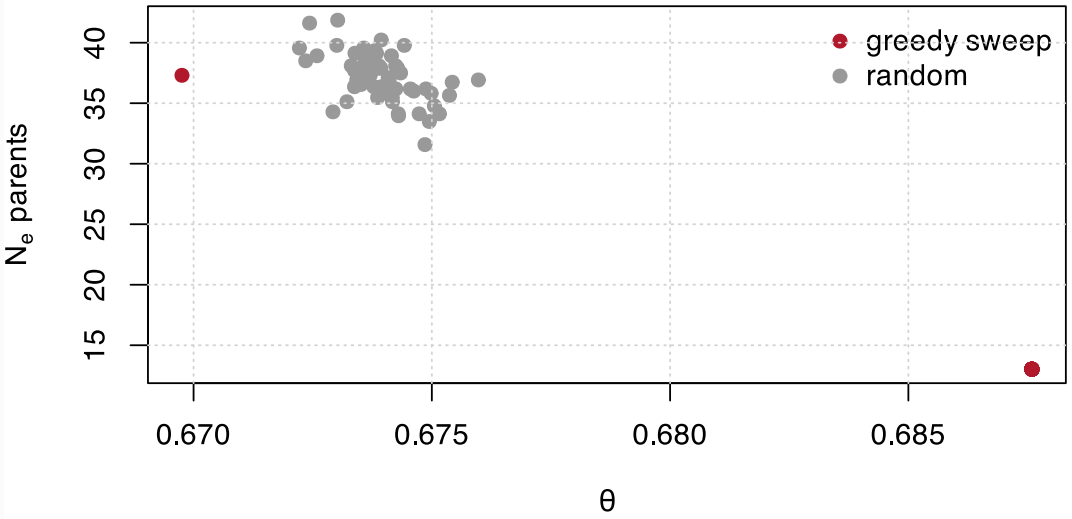

Figure 15.3: At fixed theta, the effective number of parents still varies by a factor of two. The constraint cannot see the difference; the second metric can.

```
band <- all[abs(all$theta - median(all$theta)) < 0.0015, ]
c(n_in_band = nrow(band),
  theta_span = diff(range(band$theta)),
  Ne_par_min = min(band$Ne_par), Ne_par_max = max(band$Ne_par),
  alleles_lost_span = diff(range(band$lost)))
#>         n_in_band        theta_span        Ne_par_min        Ne_par_max
#>      55.000000000       0.002502847      31.578947368      41.860465116
#> alleles_lost_span
#>      31.000000000
```

> **i** A caveat on transfer
>
> These numbers come from a two-pool simulation at $F_{ST} = 0.15$. The qualitative conclusion – near one-dimensionality, with allelic richness as the exception – is robust in direction, but the exact residual fractions shift with pool divergence and marker density. Re-running the redundancy analysis on real genotypes is a half-hour job and worth doing before treating the recommendation as final.

15.4 Finding 4: the encoding, not the constraint handler, decides whether DE works

The optimiser was run under four simultaneous restrictions: diversity loss $\leq 0.5\%$, $N_{e,\mathrm{par}} \geq 50$, at most 6 uses per line, and heterotic-pool balance within 5%. Budget fixed at 60,000 evaluations for every configuration, three seeds each.

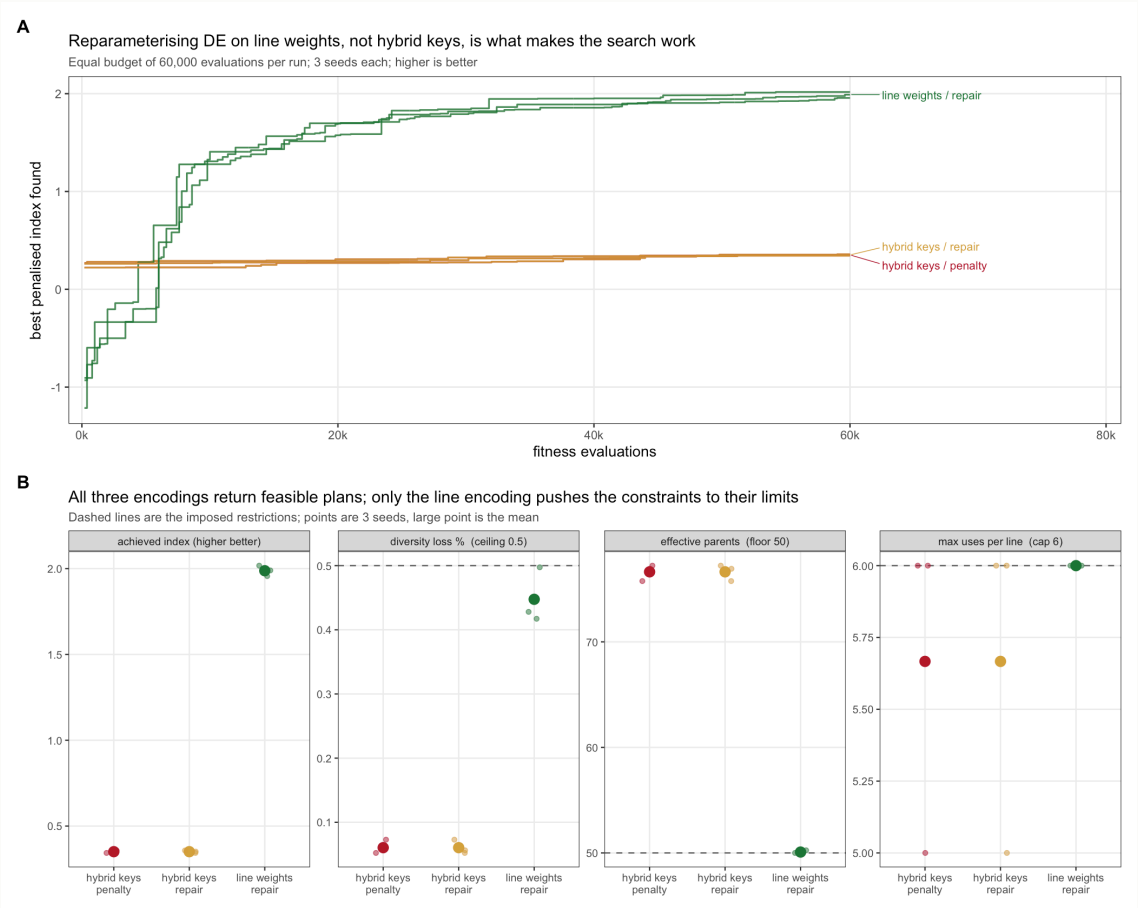

Figure 15.4: Differential evolution under four simultaneous restrictions at equal evaluation budget.

encoding	index	loss %	Ne parents	max uses	seconds
hybrid keys / penalty	0.351	0.060	76.634	5.667	6.550
hybrid keys / repair	0.351	0.060	76.634	5.667	9.621
line weights / repair	1.988	0.448	50.084	6.000	12.126

The encoding decides the outcome; the constraint handler does not. One random key per hybrid gives DE a 3,600-dimensional problem and it reaches index 0.351. One weight per *line* – dimension 120, available only because of the collapse – reaches **1.988 at the same budget**, a 5.7x improvement. Penalty

and repair handlers produced numerically identical solutions in the hybrid encoding, because at that dimension DE never approaches the constraint boundary where the handlers differ.

Only the line encoding makes the constraints bind. It finishes with $N_{e,\mathrm{par}} = 50.08$ against a floor of 50, max uses exactly 6 against a cap of 6, and loss 0.448% against a 0.5% ceiling. The hybrid encoding leaves every constraint slack – not because it found a better trade-off, but because it never searched far enough from random to reach the boundary.

A slack constraint at the end of a run is a diagnostic of a failed search, not of a conservative plan.

For calibration: unconstrained truncation reaches index 2.337 but violates everything (loss 1.62%, $N_{e,\mathrm{par}} = 33.2$, one line used 17 times). The line encoding recovers **85% of the truncation index while satisfying all four restrictions.**

A note on tuning. `DEoptim` warns that `NP` should be at least ten times the parameter count; at dimension 3,600 that is 36,000 population members, which is infeasible. At dimension 120 the recommendation is 1,200 and entirely reachable. The reparameterisation does not merely speed the search up – it moves the problem into the regime where DE's own tuning guidance can be followed.

15.5 Finding 5: the index is not the truth, and its error is not white

Findings 1 to 4 are about the diversity side of the objective. This one is about the other side, and it is the larger uncertainty of the two.

Everywhere in this book `ctx$index` is built from *known* marker effects (Chapter 4.), so every result so far assumes the breeding values are predicted without error. Real ones are not, and the error is not white noise: predictions for close relatives are wrong in the same direction, because they lean on the same training records and share the same haplotypes. The book already has a matrix of exactly that structure – the molecular coancestry $\mathbf{f}$ – so the error can be simulated without inventing anything. (3 fits an actual GBLUP model later and finds the same structure in its real residuals, at a comparable accuracy, without injecting anything.)

```
n_sel <- 60
null  <- null_distribution(ctx, n_sel, B = 400, B_full = 60)
gd    <- null$gd_ref
u     <- ctx$index
Lc    <- t(chol(ctx$f + diag(1e-8, ctx$N)))

# A predicted index at accuracy r. `kind` decides whether the error is
# correlated with relatedness or independent between hybrids.
predicted <- function(r, seed, kind = "correlated") {
  set.seed(seed)
  z <- rnorm(ctx$N)
  e <- if (kind == "correlated") as.numeric(Lc %*% z) else z
  e <- (e - mean(e)) / stats::sd(e)
  as.numeric(scale(r * u + sqrt(1 - r^2) * e))
}
```

Select on the prediction, then score the plan on the truth:

```
run <- function(r, seed = 1, kind = "correlated") {
  ch <- ctx; ch$index <- predicted(r, seed, kind)
  sel <- list(truncation = sel_truncation(ch, n_sel),
              greedy     = sel_greedy(ch, n_sel, w = 3e-4))
  do.call(rbind, lapply(names(sel), function(nm) data.frame(
    method = nm, r = r,
    reported = mean(ch$index[sel[[nm]]]),
    realised = mean(u[sel[[nm]]]),
    alpha    = alpha_loss(sel[[nm]], ctx, gd))))
}
acc <- do.call(rbind, lapply(c(1, 0.9, 0.7, 0.5), run))
fmt(acc, 4)
```

method	r	reported	realised	alpha
truncation	1.0	1.9919	1.9919	0.0412
greedy	1.0	1.5893	1.5893	0.0133
truncation	0.9	1.9247	1.9003	0.0406
greedy	0.9	1.4745	1.3965	0.0092
truncation	0.7	1.7529	1.5950	0.0356
greedy	0.7	1.3851	1.1464	0.0088
truncation	0.5	1.6223	1.1751	0.0301
greedy	0.5	1.2591	0.7965	0.0056

Two things move. The gap between `reported` and `realised` opens up – at $r = 0.5$ truncation reports an index of 1.62 and delivers 1.18. Selection on a noisy value selects partly on the noise, so the plan's own scorecard is biased upward. Nothing in the pipeline can detect this; it is invisible without the truth.

And α *falls* as accuracy drops. That is not the constraint working better. Noisy selection is closer to random selection, and random selection is what $\bar{H}_{\text{ref}}$ is defined against – so the plan drifts toward the baseline from which loss is measured.

15.5.1 The correlation structure matters more than the amount

Hold accuracy fixed at $r = 0.6$ and change only whether the error is correlated with relatedness, over twenty draws:

```
cmp <- do.call(rbind, lapply(c("correlated", "independent"), function(k)
  do.call(rbind, lapply(1:20, function(s) {
    ch <- ctx; ch$index <- predicted(0.6, s, k)
    tr <- sel_truncation(ch, n_sel)
    data.frame(kind = k, realised = mean(u[tr]),
               alpha = alpha_loss(tr, ctx, gd), max_line = max_line_use(tr, ctx))
  })))
fmt(aggregate(cbind(realised, alpha, max_line) ~ kind, cmp, mean), 4)
```

kind	realised	alpha	max_line
correlated	1.1508	0.0391	18.15
independent	1.2253	0.0121	12.60

Same accuracy, same method, same number selected. Correlated error costs roughly 3.2 times the diversity of independent error, and leans 1.4 times as hard on its favourite line.

The mechanism is direct. Independent error scatters the mistakes, which accidentally diversifies the plan. Error correlated with relatedness makes whole families look good together, so the plan takes families – and a family is exactly what the diversity constraint exists to stop it taking.

> ⚠ The worst case is not the noisiest one
>
> Truncation on a perfect index loses 4.12% of gene diversity and earns every bit of the index it reports. Under correlated error at $r = 0.6$ it still loses 3.91% – nearly the same price – while delivering 1.15 instead of 1.99.
>
> You pay the full diversity cost of aggressive selection and collect a fraction of the gain. That is the case to design against, and it is the ordinary case in a real programme.

15.5.2 What this does to the recommendation

Nothing, for the metric set. Findings 1 to 3 are statements about $\mathbf{f}$ and hold whatever the index is. But it changes what the numbers in Chapter 24. mean, and it reorders the priorities: the choice between two diversity metrics is worth a fraction of a percent of index, and prediction accuracy is worth tens of percent. Report α against the plan you actually planted, validate accuracy before tuning the constraint, and treat a reported index as an upper bound.

15.6 The recommendation, stated

15.6.1 Inside the optimiser

Use two metrics, both functions of the line-usage vector $\mathbf{w}$:

1. **Proportional loss of gene diversity**, $(\bar{H}_{\text{ref}} - H_S)/\bar{H}_{\text{ref}}$ with $H_S = 1 - \mathbf{w}^\top \mathbf{f}^L \mathbf{w}$ and $\bar{H}_{\text{ref}}$ the mean over same-size random subsets. Compute it by the line route, never by scanning genotypes.
2. **Effective number of parent lines**, $N_{e,\text{par}} = (\sum k_a)^2 / \sum k_a^2$. It costs 7 us, shares the state vector, and blocks concentration onto a few lines – a failure mode θ tolerates.

Reparameterise the search on **line weights**, and enforce the per-line usage cap in the decoder rather than by penalty.

15.6.2 At validation, on the returned plan only

3. **Marker alleles retained**, via the bitset route. The one metric carrying information θ does not.
4. **Within/between heterotic-pool decomposition**, to confirm the plan has not preserved diversity by degrading pool separation.
5. **Status number** $N_s = 1/(2\theta)$, free from θ, as the reportable descriptor. Do not report a rate-based N_e under active management.

15.6.3 Not recommended

Nei's H_e as a separate metric (it *is* $1 - \theta$); effective dimensionality ($|\rho| = 0.97$ with θ, and needs the hybrid matrix); MAF-threshold allele loss ($|\rho| = 0.93$, and 1.47 hours per million evaluations); Core Hunter's E-NE and A-NE (both require the $O(N^2)$ matrix and are unavailable at production scale – they are designed for core-collection assembly, a different problem).

15.6.4 Caveats

The choice of $\mathbf{f}^L$ remains open in the literature. Post-2018 work is consistent that molecular or IBD-flavoured coancestry beats a VanRaden-style drift matrix for diversity management, but reports no consensus on a single preferred formulation, and matrix choice demonstrably changes allele-frequency trajectories (Chapter 21.). The collapse derived here is a property of the F1 structure and holds for **any** symmetric line-level $\mathbf{f}^L$ – so if the matrix choice changes, every cost and identity result in this chapter survives unchanged.

Managing on molecular coancestry controls homozygosity, not drift; these diverge, and a plan optimal on one is not optimal on the other (Chapter 12.). Predicted Δf is biased downward, with the bias growing as the candidate pool shrinks – report realised θ on the chosen plan rather than a predicted rate.

15.7 Drop-in code

```
ctx <- fast_ctx(ctx, GL = line_genotypes, pack = TRUE)   # no N x N matrix needed

gd_ref <- mean(replicate(500, fast_gene_div(sample.int(ctx$N, n_sel), ctx)))

fit <- make_fitness_fast(ctx, n_sel, gd_ref, alpha_max = 0.005,
                         ne_par_min = 50, max_use = 6, penalty = 5,
                         encoding = "lines")

res <- DEoptim::DEoptim(fit,
                        lower = rep(0, ctx$n_lines), upper = rep(1, ctx$n_lines),
                       control = DEoptim::DEoptim.control(NP = 200, itermax = 300))

idx <- decode_lines(res$optim$bestmem, ctx, n_sel, max_use = 6)
c(theta      = fast_theta(idx, ctx),
  loss_pct   = 100 * (gd_ref - fast_gene_div(idx, ctx)) / gd_ref,
  Ne_parents = fast_ne_parents(idx, ctx),
```

```
  alleles    = alleles_retained_fast(idx, ctx),
  fast_theta_pools(idx, ctx))
```

Reproduce this chapter

All the code of this chapter, in order and with the explanation kept as comments, is one script: **07-choosing-metrics.R**. Download it, install `hybdiv`, and run it top to bottom in an empty directory; cached results are fetched on first use.

16. Metrics at each pipeline stage, and five traps

The previous chapters established *which* metrics to use. This one places them at the four stages of a hybrid maize programme, and names five traps – each one a thing that looks correct, returns a number, and is wrong.

Stage	What is selected
S1	conserving the heterotic groups (pool level)
S2	choosing parents and crosses for DH induction
S3	selecting lines per se
S4	hybrid mining

Two rules govern everything below. Every formula was verified numerically before it was written; where a result is an exact algebraic identity, the maximum deviation from simulation is reported next to it. And the metric set is the one established in Chapter 15.: molecular coancestry as the kernel, $GD = 1 - \theta$ as the scale, N_s as the interpretable size, allelic richness as the one non-redundant addition, and proportional loss measured against a same-size resampling baseline.

```
sim <- simulate_pools(n_A = 30, n_B = 30, m = 3000, fst = 0.15, seed = 12)
ctx <- ctx_from_pools(sim)
GL  <- sim$GL; pool <- sim$pool; fL <- sim$fL
c(lines = sim$L, markers = sim$m, hybrids = sim$N)
#>   lines markers hybrids
#>      60    3000     900
```

16.1 The kernel: one matrix, all four stages

Molecular coancestry f_{ij} is the probability that an allele drawn at random from individual i is identical in state to one drawn from j. For inbred lines coded 0/1 at m biallelic markers,

$$f_{ij} = 1 - \frac{1}{m}\sum_{k=1}^{m}(x_{ik} - x_{jk})^2. \tag{16.1}$$

Group coancestry of a set is the mean of the full submatrix **including the diagonal**, $\theta = n^{-2}\sum_i \sum_j f_{ij}$, with $GD = 1 - \theta$ and $N_s = 1/(2\theta)$.

The diagonal is not a technicality. It is why a small set always looks highly coancestral, and it is the reason the naive greedy constraint in S3 fails.

16.1.1 Trap 1: modified Rogers distance is the same statistic

Reporting both MRD and coancestry as separate evidence is reporting one number twice. The relation is exact:

```
D <- mrd_matrix(GL)
c(max_deviation = max(abs(coancestry_from_mrd(D) - fL)))
#> max_deviation
#>  1.110223e-16
```

$f_{ij} = 1 - \mathrm{MRD}_{ij}^2$. If a diversity report shows a distance tree and a coancestry table as mutual corroboration, they are the same analysis in two coordinate systems.

16.2 S1: conserving the heterotic groups

Diversity in a hybrid programme is **two quantities moving in opposite directions**: within-pool diversity erodes under selection, while between-pool divergence inflates. A single pool-wide statistic cannot express both, and a programme monitoring only total diversity can look stable while both pools collapse internally and drift apart.

The partition is exact for any grouping:

$$\theta_T = \sum_p w_p^2\,\theta_{pp} + \sum \text{over } p \neq q \text{ of } w_p w_q\,\theta_{pq} \tag{16.2}$$

```
d <- theta_decompose(GL, pool)
c(theta_T_from_partition = d$theta_T,
  theta_T_direct         = mean(fL),
  deviation              = d$theta_T - mean(fL))
#> theta_T_from_partition         theta_T_direct              deviation
#>             0.6614378              0.6614378              0.0000000
```

16.2.1 Trap 2: two F_{ST} definitions differ by up to 2x

Both are in routine use, both are computed from the same coancestry matrix, and they are **not interchangeable**:

$$F_{ST}^{(W)} = \frac{\theta_w - \theta_B}{1 - \theta_B} \qquad G_{ST} = \frac{\theta_w - \theta_T}{1 - \theta_T} \tag{16.3}$$

The first references *between-pool* coancestry; the second references *total* pooled-frequency coancestry and equals Nei's $G_{ST} = (H_T - H_S)/H_T$:

```
c(Fst_Wright = d$Fst_Wright,
  Gst_Nei    = d$Gst_Nei,
  Gst_classical = unname(gst_nei(GL, pool)[["Gst"]]),
  ratio      = d$Fst_Wright / d$Gst_Nei)
#>    Fst_Wright       Gst_Nei Gst_classical         ratio
#>     0.1824006     0.1003525     0.1003525     1.8175994
```

Because $\theta_T > \theta_B$ whenever the pools differ at all, $G_{ST} < F_{ST}$ **always**. Across simulated divergence they never converge:

fst_target	Fst_Wright	Gst_Nei	ratio	theta_B	theta_T	GD_within	frac_opp	mean_dp
0.02	0.0446	0.0228	1.9554	0.6372	0.6453	0.3466	0.0000	0.0991
0.05	0.0742	0.0385	1.9258	0.6361	0.6496	0.3369	0.0000	0.1280
0.10	0.1200	0.0638	1.8800	0.6367	0.6585	0.3197	0.0000	0.1607
0.15	0.1676	0.0915	1.8324	0.6337	0.6644	0.3049	0.0003	0.1907
0.20	0.2268	0.1279	1.7732	0.6289	0.6710	0.2869	0.0013	0.2211
0.30	0.3170	0.1883	1.6830	0.6413	0.6982	0.2450	0.0063	0.2504
0.40	0.4106	0.2584	1.5894	0.6341	0.7092	0.2157	0.0230	0.2840

State which definition you used, and never compare your value against a published threshold computed the other way.

16.2.2 Trap 3: F_{ST} is the wrong target to maximise

F_{ST} keeps rising as pools drift apart at loci that are *already* divergent, which buys no additional heterosis. Chapter 13. makes the reason exact: under neutral divergence expected heterosis does not depend on F_{ST} at all, because every unit gained in the divergence term is paid for out of the shared term. What does move it is divergence landing on the dominance loci. The frequency-side statistics decouple from F_{ST} accordingly:

```
sweep_fst <- do.call(rbind, lapply(c(0.05, 0.15, 0.25, 0.40), function(fst) {
  s <- simulate_pools(n_A = 25, n_B = 25, m = 2000, fst = fst, seed = 3)
  dd <- theta_decompose(s$GL, s$pool)
  cm <- pool_complementarity(s$GL, s$pool)
  data.frame(target_fst = fst, Fst = dd$Fst_Wright, GD_within = dd$GD_within,
             frac_opposite_fixed = cm[["frac_opposite_fixed"]],
             mean_abs_delta_p = cm[["mean_abs_delta_p"]],
             mean_sq_delta_p = cm[["mean_sq_delta_p"]])
}))
fmt(sweep_fst, 4)
```

target_fst	Fst	GD_within	frac_opposite_fixed	mean_abs_delta_p	mean_sq_delta_p
0.05	0.0907	0.3321	0.0005	0.1412	0.0331
0.15	0.1857	0.3003	0.0000	0.1995	0.0685
0.25	0.2711	0.2644	0.0045	0.2379	0.0984
0.40	0.4348	0.2106	0.0370	0.2940	0.1620

Buying oppositely-fixed loci costs within-pool gene diversity, and it costs a great deal of it. **Hold F_{ST} in a band, not at a maximum,** and monitor complementarity directly.

Of the three complementarity columns, `mean_sq_delta_p` is the one to log. Chapter 13. shows it is not a proxy at all but exactly twice the divergence term of expected heterosis, while `frac_opposite_fixed` sits at essentially zero across the whole range of divergence a real dent-versus-flint pair occupies, and so cannot discriminate between programmes.

16.2.3 The S1 monitoring panel

```
fmt(s1_monitor(GL, pool, cycle = 12), 4)
```

cycle	GD_within	GD_total	Fst_Wright	Gst_Nei	Ns_total	GD_A	GD_B	frac_opposite_fixed	mean_abs_delta_p	mean_sq_delta_p
12	0.3046	0.3386	0.1824	0.1004	0.7559	0.305	0.3042	0	0.2003	0.068

Log this every cycle and plot it as a time series. Defend `GD_within` per pool; watch `mean_sq_delta_p` for heterosis potential; treat F_{ST} as a descriptor, not an objective. Chapter 22. is what the time series looks like when it is actually run out to fifteen cycles.

The empirical anchor is Kadoumi et al. (2025), who tracked diversity erosion and heterotic-group divergence over decades of a commercial European dent maize programme – and found exactly the two opposite trends this panel is built to separate. See Chapter 21..

16.3 S2: choosing crosses for DH induction

This is where the largest share of pipeline diversity is lost, and also where the cheapest exact algebra is available.

16.3.1 Three verified identities

For a DH family from cross (i, j) of two inbreds, each locus is inherited from i or j with probability 1/2 and then doubled.

(a) Coancestry between two DH sibs of the same family: $\mathbb{E}[f] = (1 + f_{ij})/2$.

(b) Coancestry between a DH of family (i, j) and one of (k, l): $\mathbb{E}[f] = (f_{ik} + f_{il} + f_{jk} + f_{jl})/4$.

(c) Group coancestry of a DH pool from crosses $c = 1…C$, each contributing n_c lines, $D = \sum n_c$:

$$\theta_{pool} = \mathbf{w}^{\top}\mathbf{f}^{L}\mathbf{w} + \frac{1}{D}\left(1 - \overline{\left(\frac{1 + f_c}{2}\right)}\right) \tag{16.4}$$

with $\mathbf{w}$ the **parent-usage weight vector**. The second term is the self-coancestry correction; it is $O(1/D)$ and worth keeping.

```
set.seed(88)
A <- 1:30
crosses <- cbind(sample(A, 20, replace = TRUE), sample(A, 20, replace = TRUE))
crosses <- crosses[crosses[, 1] != crosses[, 2], , drop = FALSE]

c(with_self_term    = dh_pool_theta(crosses, n_per = 40, fL, self_term = TRUE),
  without_self_term = dh_pool_theta(crosses, n_per = 40, fL, self_term = FALSE),
  sib_coancestry_ex = dh_sib_coancestry(fL[crosses[1, 1], crosses[1, 2]]))
#>    with_self_term without_self_term sib_coancestry_ex
#>         0.7010910         0.7008837         0.8493333
```

! The key structural result of S2

The diversity of the DH pool is a quadratic form in *which parents you use and how often.* **The DH lines never need to be genotyped – or to exist – for their pool diversity to be optimised.** You can evaluate a crossing plan before making a single cross.

16.3.2 Progeny variance, and the usefulness criterion

Under an additive model only loci segregating in the cross contribute:

$$\sigma^2_{DH(i,j)} = \frac{1}{4} \sum_{k\,:\,x_{ik} \neq x_{jk}} \beta_k^2 \tag{16.5}$$

```
sig <- sigma_dh_crosses(crosses, GL, sim$beta)
mu  <- rowMeans(cbind(GL[crosses[, 1], ] %*% sim$beta,
                      GL[crosses[, 2], ] %*% sim$beta))
uc  <- usefulness_criterion(mu, sig, i = sel_intensity(0.05), h = 0.6)
c(sel_intensity_5pct = sel_intensity(0.05), textbook = 2.063)
#> sel_intensity_5pct           textbook
#>           2.062713           2.063000
fmt(head(data.frame(p1 = crosses[, 1], p2 = crosses[, 2],
                    mu = mu, sigma_dh = sig, UC = uc), 6), 3)
```

p1	p2	mu	sigma_dh	UC
23	27	0.930	4.609	6.634
13	11	−5.977	5.032	0.251
22	21	−3.481	5.264	3.034
7	19	−7.343	5.063	−1.076
19	1	−11.512	5.285	−4.971
2	12	−2.152	4.934	3.955

`sel_intensity(0.05)` returns 2.0627 against the textbook 2.063.

16.3.3 Trap 4: genetic distance is not a proxy for progeny variance

It is common practice to rank crosses by marker distance, on the argument that more distant parents segregate for more loci. Once the heterotic pattern is fixed, this fails:

stratum	cor	R2
inter-pool (A x B)	0.103	0.011
within-pool (A x A)	0.219	0.048
pooled	0.576	0.332

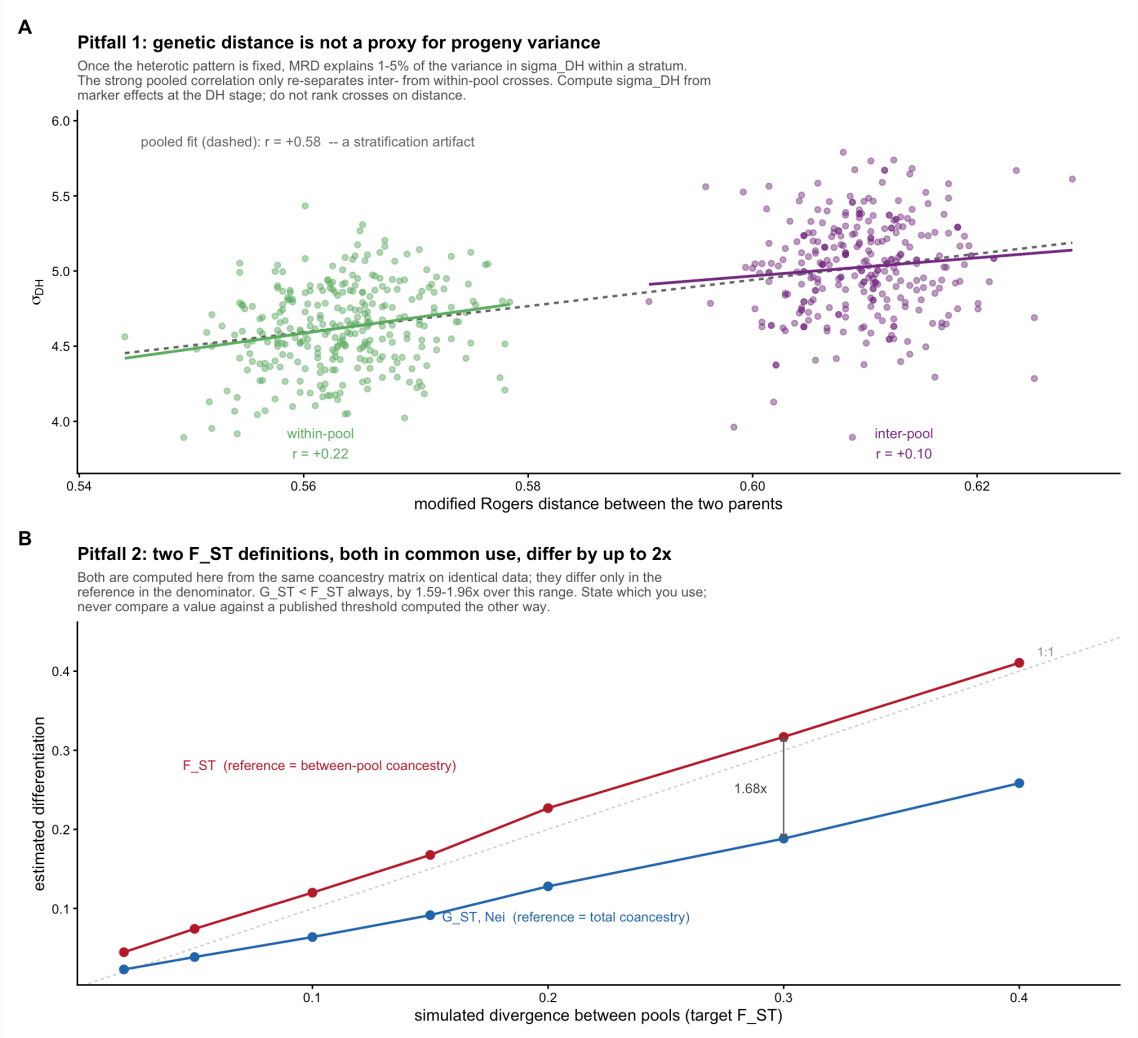

Figure 16.1: Traps 4 and 2, on the full simulation. Panel A: progeny standard deviation against modified Rogers distance – the strong pooled correlation only re-separates inter- from within-pool crosses and carries almost no information within either stratum. Panel B: the two F_ST definitions on identical data, differing only in the reference in the denominator.

A raw count of segregating loci explains the same variance – distance is adding nothing beyond "are these parents in different pools", which you already know.

Rank crosses on `sigma_dh_crosses()` computed from marker effects, not on distance.

16.4 S3: selecting lines per se

Two points of practice.

Apply the constraint within each pool separately. A pooled constraint can be satisfied by keeping the pools apart while both erode internally – exactly the failure mode S1 exists to prevent.

Measure loss against a same-size resampling baseline.

```
merit <- as.vector(GL %*% sim$beta)
merit <- (merit - mean(merit)) / sd(merit)
top   <- order(merit, decreasing = TRUE)[1:20]
fmt(loss_vs_resampling(top, fL, B = 300), 4)
#>    GD_selected GD_random_mean       loss_pct     percentile
#>         0.3245         0.3271         0.8029         0.0700
```

16.4.1 Trap 5: the incremental greedy constraint silently returns nothing

The natural implementation – "add the next-best line while θ stays below the ceiling" – **cannot work**, and fails silently by returning an empty set. Group coancestry includes the diagonal, so a single line has $\theta = 1$ and any pair has $\theta = (1 + f_{ij})/2$. A ceiling calibrated on a large pool is unreachable at small set sizes; the greedy never gets past the first candidate.

```
ceiling20 <- theta_ceiling(fL, n_sel = 20, alpha = 0.005)
c(ceiling             = ceiling20,
  theta_single_line   = fL[1, 1],
  theta_best_pair     = min((1 + fL[upper.tri(fL)]) / 2),
  any_pair_admissible = any((1 + fL[upper.tri(fL)]) / 2 <= ceiling20))
#>             ceiling   theta_single_line     theta_best_pair any_pair_admissible
#>           0.6746394           1.0000000           0.8008333           0.0000000
```

No single line and no pair can satisfy the ceiling. θ falls monotonically as the set grows, so **the constraint is only meaningful at the target size**:

```
sizes <- c(2, 3, 5, 8, 12, 20, 30, 45, 60)
th <- vapply(sizes, function(n) mean(replicate(50, {
  s <- sample.int(nrow(fL), n); mean(fL[s, s])})), numeric(1))
plot(sizes, th, type = "b", pch = 19, xlab = "set size", ylab = expression(theta))
abline(h = ceiling20, col = "#b2182b", lty = 2, lwd = 2)
legend("topright", c("mean theta of a random set", "ceiling for n = 20"),
       col = c("black", "#b2182b"), lty = c(1, 2), pch = c(19, NA), bty = "n")
grid()
```

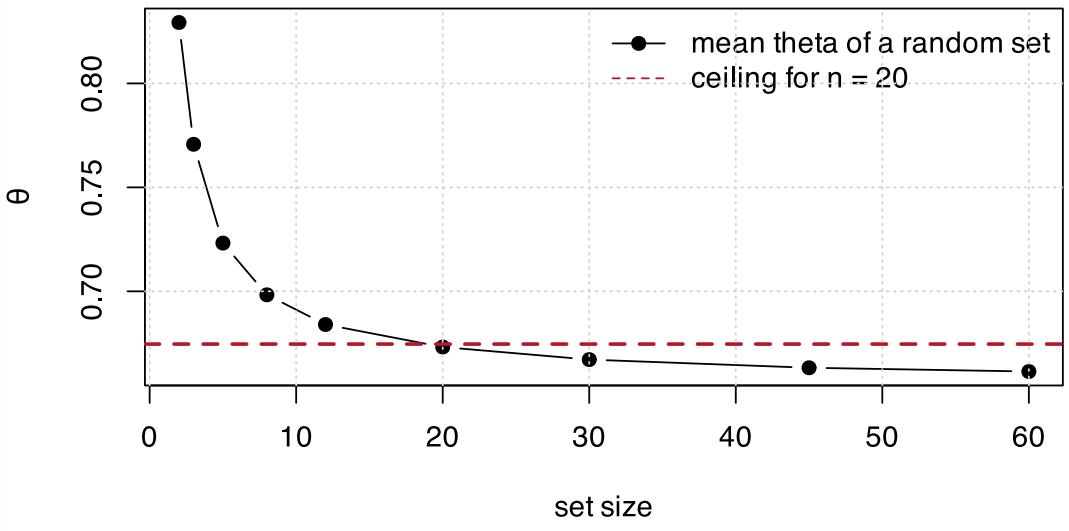

Figure 16.2: Group coancestry falls monotonically with set size. A ceiling calibrated at the target size is unreachable at every smaller size, which is why an incremental greedy stops at the first candidate.

The working route is **truncate-then-repair**: take the top `n_sel` by merit, then swap the most-related member for the least-related non-member until the ceiling is met, breaking ties on merit.

```
sel <- select_lines_constrained(merit, fL, n_sel = 20, theta_max = ceiling20)
c(n_selected   = length(sel),
  theta        = s3_theta(sel, fL),
```

```
  ceiling      = ceiling20,
  met          = s3_theta(sel, fL) <= ceiling20,
  mean_merit   = mean(merit[sel]),
  merit_of_top = mean(merit[top]))
#>   n_selected        theta      ceiling          met   mean_merit merit_of_top
#>   20.0000000    0.6755400    0.6746394    0.0000000    1.0724564    1.0724564
```

`theta_ceiling()` expresses the ceiling as a permitted **proportional loss** against the same-size baseline, so the constraint means the same thing at every set size.

16.5 S4: hybrid mining

With $\mathbf{u}$ the line-usage count vector over the n selected hybrids,

$$\theta_{hyb} = \frac{\mathbf{u}^\top \mathbf{f}^L \mathbf{u}}{(2n)^2} \tag{16.6}$$

```
set.seed(91)
hyb <- sample.int(ctx$N, 100)
u   <- line_usage(hyb, ctx$parents, ctx$n_lines)
c(theta_from_usage = hybrid_theta_from_usage(u, fL),
  theta_from_matrix = ref_theta(hyb, ctx),
  deviation = hybrid_theta_from_usage(u, fL) - ref_theta(hyb, ctx))
#>  theta_from_usage theta_from_matrix         deviation
#>         0.6620848         0.6620848         0.0000000
```

Deviation exactly zero. This is the collapse of Section 15.1: cost drops from $O(n^2)$ over an $N \times N$ hybrid matrix to $O(L^2)$ with no hybrid matrix formed at all.

Expected heterozygosity of the hybrid set needs no separate computation: for hybrids of fixed lines it is exactly $1 - \theta_{hyb}$.

```
c(theta = hybrid_theta_from_usage(u, fL),
  He    = 1 - hybrid_theta_from_usage(u, fL),
  Ne    = ne_parents_from_counts(u),
  pool_balance = pool_balance(hyb, ctx$parents, pool))
#>         theta            He            Ne pool_balance1 pool_balance2
#>     0.6620848     0.3379152    49.6277916     0.5000000     0.5000000
```

16.6 Where the diversity actually goes

The four stages were simulated end to end over **8 independent founder pools** (120 lines, 6000 markers, target $F_{ST} = 0.15$; 20 crosses, 40 DH each, 80 lines selected, 200 hybrids mined), under two cross-selection strategies: merit only, and merit plus a diversity term on parental coancestry.

Loss at each stage is proportional loss of gene diversity against that stage's **own same-size resampling baseline**, so the stages sit on one scale.

stage	loss_pct.UC + div	share_of_total.UC + div	loss_pct.UC only	share_of_total.UC only
S2 DH induction	3.43	35.51	9.40	77.46
S3 line per se	2.98	28.99	0.32	2.55
S4 hybrid mining	3.47	35.49	2.48	19.99

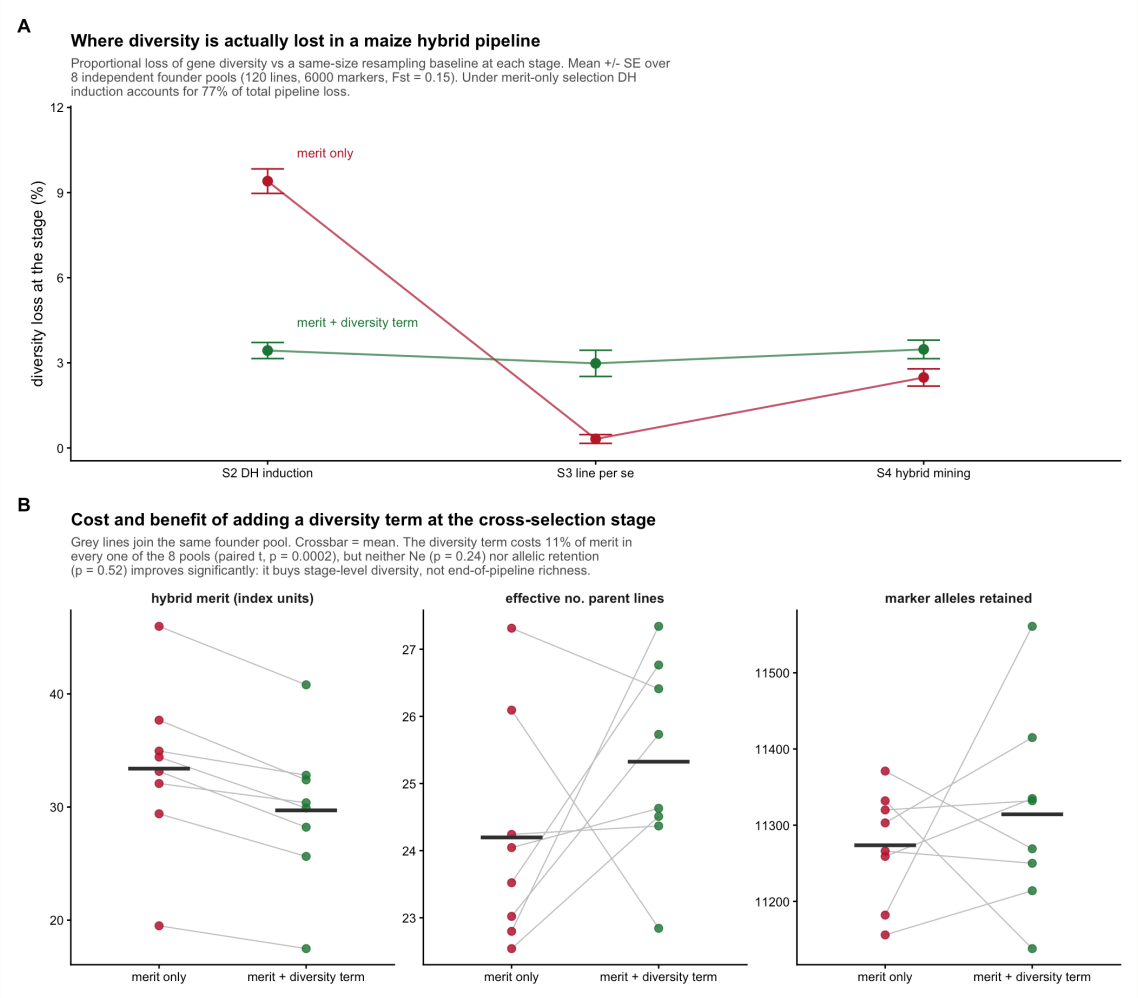

Figure 16.3: Where the diversity goes. DH induction dominates, and a diversity term applied there redistributes loss rather than removing it.

DH induction dominates. Under merit-only cross selection it accounts for **77% of all diversity lost in the pipeline** – more than three times the loss at hybrid mining, which is where optimisation effort conventionally goes.

The reason is structural: 20 crosses out of 1770 possible within-pool combinations is a far harsher bottleneck than 200 hybrids out of 1600, and the S2 collapse makes it the cheapest stage to constrain.

16.6.1 What the diversity term buys, and what it does not

Outcome	merit only	merit + diversity	paired t (n = 8)
hybrid merit	33.4	29.71	**p = 0.0002**
effective no. parent lines	24.2	25.33	p = 0.24
marker alleles retained	11274.0	11314.00	p = 0.52

The merit cost is **11%, consistent in all 8 pools**. Neither end-of-pipeline diversity outcome improves significantly. The diversity term redistributes loss across stages – it flattens the S2 spike – but with downstream selection still running on merit alone, the pipeline reconverges. Diversity spent at S2 is partly reclaimed at S3 and S4.

A diversity constraint applied at one stage buys stage-level diversity, not end-of-pipeline richness. If allelic richness in the commercial hybrid set is the target, the constraint has to be carried through all four stages, and the loss ledger is the instrument for allocating a budget across them.

16.7 Cost summary

Stage	Function	Cost	Notes
S1	`theta_decompose`	O(L^2) once per cycle	monitoring, not an inner loop
S2	`dh_pool_theta`	O(L^2) per plan	no DH genotypes needed
S2	`sigma_dh_crosses`	O(C m)	the real cost at this stage
S3	`select_lines_constrained`	O(swaps x n)	swaps typically under 20
S4	`hybrid_theta_from_usage`	O(L^2) per evaluation	no N x N matrix

The $O(L^2)$ quadratic form is the recurring shape. With L in the hundreds it is microseconds, which is why the whole pipeline can be optimised inside a metaheuristic. The dominant cost anywhere here is `sigma_dh_crosses` at S2, because it touches the marker matrix – cache it per candidate cross rather than recomputing inside the optimiser loop.

16.8 Checklist

1. Compute the coancestry matrix once per cycle; never report modified Rogers distance alongside it as separate evidence.
2. Monitor within-pool `GD` per pool, not pooled `GD`; state which F_{ST} definition you use; hold it in a band.
3. Rank crosses on σ_{DH} from marker effects, never on parental distance.
4. Evaluate the crossing plan's θ_{pool} *before* making the crosses – it is a quadratic form in parent usage.
5. Calibrate every diversity ceiling against a same-size resampling baseline, and apply it at the target set size.
6. Constrain within pool, at every stage – a single-stage constraint does not survive to the hybrid set.
7. At S4, use the line-usage collapse; the hybrid coancestry matrix is never needed.
8. Keep the per-stage loss ledger. It is the only way to see that DH induction, not hybrid mining, is where the diversity goes.

Reproduce this chapter

All the code of this chapter, in order and with the explanation kept as comments, is one script: **08-pipeline-stages.R**. Download it, install `hybdiv`, and run it top to bottom in an empty directory; cached results are fetched on first use.

17. Predicting breeding values: GBLUP, BayesA, BayesB

Three earlier chapters flagged the same gap and moved on. Chapter 4. built `ctx$traits` from *known* marker effects and called the result "an optimistic ceiling." Chapter 15.'s Finding 5 showed what happens once the index carries real error, but manufactured that error rather than fitting a model to produce it. Chapter 24. repeats the caveat and still moves on. This chapter is where the gap closes: it fits actual prediction models to a training population with real phenotypic noise, and hands the result to the same `stage1_build()` contract every later chapter already consumes.

`vanraden_G()` has said since Chapter 1. that its matrix "is the right matrix for GBLUP." Nothing in the package has used it that way until now.

The chapter also has a second job, which only became apparent once the models were fitted: an accuracy number means nothing until you say what was hidden from the model. Section 17.4 shows that the obvious split – hold out a random 40% of the hybrids – answers the easiest of the three questions a hybrid programme actually asks, and that the fit itself says so, before the truth is consulted.

Every model fitted below follows the same two-step logic, and it is worth stating plainly before any equation appears, because it is the entire economic case for genomic selection in an outcrossing, hybrid crop. **Step one, training:** markers and phenotypes recorded on a historical population – lines or hybrids already evaluated in earlier trials – are used to estimate a relationship between genotype and phenotype, either as marker effects (BayesA/BayesB/ridge) or as a variance structure over relatives (GBLUP). **Step two, prediction:** that same relationship is applied to the markers of candidate hybrids that have never been phenotyped, to rank them before any resource is committed to growing them out. In a self-pollinated crop the second step is a convenience; in a cross-pollinated, hybrid crop it is transformative, because Section 17.2 below shows that a hybrid's value is additive in its two parents' genotypes, so step two can score a line-by-line combination that has never even been made as a physical cross. Genotyping two parent lines already on hand is enough to rank a candidate that would otherwise cost a full generation to phenotype – the saving Chapter 16. calls hybrid mining (S4).

17.1 A training population with real noise

Everywhere else in this book, `ctx$index` *is* the truth. Here it plays a different role: it is the value a real breeding programme does not get to see, and can only estimate from phenotypes recorded on a subset of individuals. Building that subset takes three steps: manufacture a phenotype by adding noise to the true genetic value, split the population into the individuals that will carry a phenotype and those that will not, then hand only the first group's records to the models fitted below.

```
ctx <- simulate_data(book_cfg)
N   <- ctx$N
g   <- ctx$traits[, 1]          # the true genetic value -- known only to us
```

A phenotype is the genetic value plus environmental noise, $y = g + e$, and heritability is the fraction of phenotypic variance the genetic part accounts for, $h^2 = \mathrm{Var}(g)/(\mathrm{Var}(g) + \mathrm{Var}(e))$. `traits` is built by `scale()`, so $\mathrm{Var}(g) = 1$ exactly, and the noise variance that delivers a target h^2 is $(1 - h^2)/h^2$:

```
h2 <- 0.5
set.seed(9)
e <- rnorm(N, 0, sqrt((1 - h2) / h2))
y <- g + e
```

Split the population into a training set, whose phenotypes the model gets to see, and a candidate set – the hybrids actually being selected among – whose phenotypes it does not. This mirrors the two roles a real programme's germplasm plays at any one time: training stands in for hybrids already phenotyped

in earlier trials, and candidates stand in for this cycle's untested combinations, ranked here purely from markers:

```
set.seed(9)
train <- sample.int(N, floor(0.6 * N))
cand  <- setdiff(seq_len(N), train)
c(train = length(train), candidates = length(cand))
#>      train candidates
#>        375        250
```

17.2 The truth this chapter is predicting

Two properties of g decide everything that follows, and both are consequences of how Chapter 4. built it rather than choices made here.

17.2.1 The generative model is GBLUP's own assumption

`ctx$traits` is `scale(Z %*% B)` with the marker effects $\mathbf{B}$ drawn independently. For a single trait that is $\mathbf{g} = \mathbf{Z}\boldsymbol{\beta}$ with $\mathrm{Var}(\boldsymbol{\beta}) = \mathbf{I}\sigma^2_\beta$, so

$$\mathrm{Var}(\mathbf{g}) = \mathbf{Z}\mathbf{Z}'\sigma^2_\beta = \mathbf{G}\underbrace{2\sum_k p_k(1-p_k)\,\sigma^2_\beta}_{\sigma^2_u}, \tag{17.1}$$

because $\mathbf{G}$ is *defined* as $\mathbf{Z}\mathbf{Z}'$ over that same divisor (Chapter 1.). The second equality is exact, not approximate:

```
vr <- vanraden_G(ctx$X * 2)
s2 <- 2 * sum(vr$p * (1 - vr$p))
max(abs(tcrossprod(vr$Z) / s2 - ctx$G))
#> [1] 0
```

So the covariance GBLUP *assumes* for the breeding values is, up to the scalar σ^2_u, the covariance the breeding values actually have. This simulation is the infinitesimal model, exactly and not approximately, and that single fact is most of the answer to "which model wins" below. It is also the sense in which every accuracy in this chapter is a best case: no model here is misspecified.

17.2.2 A hybrid's value is the mean of two parental values

The hybrids are F1s of homozygous lines, so a hybrid's marker row is the average of its two parents' rows, and averaging is linear. For *any* additive effect vector whatsoever, the hybrid's genetic value is therefore the mean of the two parental line values – general combining ability, with no specific combining ability left over:

```
L  <- simulate_lines(book_cfg)
bb <- rnorm(ctx$m)                    # any effects at all; the identity is linear
vL <- as.numeric((L / 2) %*% bb)      # the parental line values they imply
max(abs(as.numeric(ctx$X %*% bb) - (vL[ctx$ped$a] + vL[ctx$ped$b]) / 2))
#> [1] 1.563194e-13
```

The residual is at machine precision because the statement is an identity. Two consequences run through the rest of the chapter. The mild one is that a "hybrid-specific" prediction has nothing to predict here: the dominance deviations of Chapter 13. are absent from `ctx$traits` by construction, so SCA is not merely unmodelled, it is not in the data. The sharp one is Section 17.3.2.

There is a third consequence, and it is the operational payoff of everything in this chapter. Because a hybrid's value is a fixed linear function of its two parents' marker genotypes, once marker effects are estimated from a training population of phenotyped hybrids (or lines), that same estimate scores *any* line A x line B combination – including combinations of lines that have never been crossed. Ranking a candidate hybrid this way costs a genotyping assay on two lines that are typically already on hand from earlier pipeline stages; the alternative, growing out the actual hybrid to phenotype it, costs a full crossing

and evaluation cycle. That difference is what makes genomic selection valuable specifically in a cross-pollinated, hybrid crop, where the space of candidate combinations is combinatorially larger than the number of lines ever phenotyped as hybrids.

17.3 GBLUP: prediction from the relationship matrix

GBLUP treats the breeding value as a random effect $\mathbf{u}$ with covariance proportional to the genomic relationship matrix, $\mathrm{Var}(\mathbf{u}) = \mathbf{G}\,\sigma^2_u$, and solves the mixed model equations. Fitting and using it is three operational steps, in this order: first write down the mixed model equations linking phenotypes, a relationship matrix and a variance ratio; second, solve that linear system once, for every individual in the population at the same time, training and candidates alike; third, convert the same solve's by-product – the inverse of its coefficient matrix – into a reliability for each prediction, which is the only part of the output a programme can trust without knowing the truth.

17.3.1 Where the mixed model equations come from

The model is $\mathbf{y} = \mathbf{1}\mu + \mathbf{Z}\mathbf{u} + \mathbf{e}$ with $\mathbf{u} \sim (\mathbf{0}, \mathbf{G}\sigma^2_u)$ and $\mathbf{e} \sim (\mathbf{0}, \mathbf{I}\sigma^2_e)$ independent. Treating $\mathbf{u}$ as a random variable rather than a parameter to be estimated, C. R. Henderson [140] maximises the *joint* density of what is observed and what is not,

$$-2\log p(\mathbf{y}, \mathbf{u}) \propto \frac{(\mathbf{y} - \mathbf{1}\mu - \mathbf{Z}\mathbf{u})'(\mathbf{y} - \mathbf{1}\mu - \mathbf{Z}\mathbf{u})}{\sigma^2_e} + \frac{\mathbf{u}'\mathbf{G}^{-1}\mathbf{u}}{\sigma^2_u}, \tag{17.2}$$

and the two stationarity conditions, multiplied through by σ^2_e, *are*

$$\begin{bmatrix} \mathbf{1}'\mathbf{1} & \mathbf{1}'\mathbf{Z} \\ \mathbf{Z}'\mathbf{1} & \mathbf{Z}'\mathbf{Z} + \mathbf{G}^{-1}\lambda \end{bmatrix} \begin{bmatrix} \hat{\mu} \\ \hat{\mathbf{u}} \end{bmatrix} = \begin{bmatrix} \mathbf{1}'\mathbf{y} \\ \mathbf{Z}'\mathbf{y} \end{bmatrix}, \qquad \lambda = \sigma^2_e/\sigma^2_u, \tag{17.3}$$

with $\mathbf{Z}$ the incidence matrix mapping training records to individuals. Three things fall out of that derivation that the equations alone do not say.

$\mathbf{u}$ is defined over *all* N individuals, training and candidates together – it enters through the prior, not through the data – so one solve produces predictions for both, and a candidate with no record of its own is predicted entirely by its $\mathbf{G}$ entries against individuals that do have one. Everything Section 17.4 finds is a consequence of that sentence.

λ is a variance *ratio*, not a tuning parameter: $\sigma^2_u = 1$ because `traits` is standardised, and `e` was built with variance $(1 - h^2)/h^2$, so $\lambda = (1 - h^2)/h^2$ is not a coincidence but the same number entered twice. Nothing in this chapter estimates it; a real programme would, by REML.

And under normality the joint maximiser is the conditional mean, $\hat{\mathbf{u}} = \mathbb{E}[\mathbf{u} \mid \mathbf{y}]$, which is why BLUP minimises mean squared error rather than merely being a sensible shrinkage rule. Dropping normality it remains best among *linear* predictors, which is what the acronym claims.

17.3.2 Why the ridge blend is not optional

Section 1.2 proves $\mathbf{1}'\mathbf{G}\mathbf{1} = 0$ and reads it as the reason $\mathbf{G}$ cannot measure diversity. It has a second consequence, for prediction. $\mathbf{G}$ is positive semi-definite, and a quadratic form of a PSD matrix vanishes only on its null space, so $\mathbf{1}'\mathbf{G}\mathbf{1} = 0$ forces

$$\mathbf{G}\mathbf{1} = \mathbf{0}. \tag{17.4}$$

$\mathbf{G}$ is singular for *every* population, whatever N and m are. The blend is not insurance against an awkward case; without it `solve()` has nothing to invert.

Here it is much worse than one lost dimension. By the identity of Section 17.2 every hybrid row is an average of two line rows, so the whole $N \times m$ matrix lives in the span of the L line genotypes. Each hybrid uses exactly one line from each pool, which makes the pool-A indicator columns sum to the pool-B ones and costs a further dimension, and centering costs one more:

```
c(max_abs_G_times_one = max(abs(ctx$G %*% rep(1, N))))
#> max_abs_G_times_one
#>         1.16334e-13
```

```
rank_G <- function(n_per_pool) {
    cfg  <-  utils::modifyList(book_cfg,  list(n_pool_A  =  n_per_pool,  n_pool_B  =
n_per_pool))
  G   <- simulate_data(cfg)$G
  c(lines = 2 * n_per_pool, hybrids = nrow(G),
    rank = sum(eigen(G, only.values = TRUE)$values > 1e-8))
}
```

lines	hybrids	rank
20	100	18
40	400	38
50	625	48

$\mathrm{rank}(\mathbf{G}) = L - 2$, independent of both the number of hybrids and the number of markers. At the book's scale that is 48 dimensions carrying 625 hybrids: 577 of the eigenvalues are exactly zero, and `0.95 * G + 0.05 * I` is what makes the equations solvable at all. Fifty lines were genotyped and crossed into 625 combinations, but there were only ever 48 independent dimensions to measure – and no amount of phenotyping the 625 adds a forty-ninth.

```
G <- ctx$G
gblup <- function(y, train, h2) {
  Gstar <- 0.95 * G + 0.05 * diag(N)
  n_t <- length(train)
  Z  <- matrix(0, n_t, N); Z[cbind(seq_len(n_t), train)] <- 1
  X1 <- matrix(1, n_t, 1); lambda <- (1 - h2) / h2
  LHS <- rbind(cbind(crossprod(X1), crossprod(X1, Z)),
               cbind(crossprod(Z, X1), crossprod(Z) + solve(Gstar) * lambda))
  RHS <- rbind(crossprod(X1, y[train]), crossprod(Z, y[train]))
  C   <- solve(LHS)                                  # kept: it carries the PEVs
  list(u   = (C %*% RHS)[-1],
       rel = 1 - diag(C)[-1] * lambda / diag(Gstar))
}
fit   <- gblup(y, train, h2)
u_hat <- fit$u
cat("GBLUP accuracy (candidates):", cor(u_hat[cand], g[cand]), "\n")
#> GBLUP accuracy (candidates): 0.9596232
```

17.3.3 Reliability: the accuracy you can compute without the truth

This is the third step promised above. Inverting the coefficient matrix rather than just solving against it costs one more decomposition and returns something a breeding programme can act on: a per-candidate confidence figure computed the same day the GEBV is, with no field trial required to check it. The $\mathbf{u}$ block of that inverse holds the prediction error variances, $\mathrm{PEV}_i = \sigma_e^2\,[\mathbf{C}^{uu}]_{ii}$, and the reliability of an individual's prediction is the fraction of its prior variance the data removed:

$$r_i^2 = 1 - \frac{\mathrm{PEV}_i}{\sigma_u^2\,\mathbf{G}^{*}_{ii}}. \tag{17.5}$$

This is the same r as the accuracy reported everywhere in this chapter, with one difference that is the entire point: g does not appear in it. A programme can compute it for candidates it has never phenotyped and never will.

```
c(mean_reliability = mean(fit$rel[cand]),
  implied_accuracy = sqrt(mean(fit$rel[cand])),
  realised_accuracy = cor(u_hat[cand], g[cand]))
#>  mean_reliability  implied_accuracy realised_accuracy
#>         0.8016810         0.8953664         0.9596232
```

The model under-states itself by about 0.06. Some of that is the blend's doing: $0.95\mathbf{G} + 0.05\mathbf{I}$ tells the equations that every candidate carries 5% of variance no relative can explain, and each r_i^2 is computed against that inflated prior. Dialling the blend down says how much:

```
blend <- function(w) {
  Gstar <- w * G + (1 - w) * diag(N)
  n_t <- length(train)
  Z  <- matrix(0, n_t, N); Z[cbind(seq_len(n_t), train)] <- 1
  X1 <- matrix(1, n_t, 1); lambda <- (1 - h2) / h2
  C  <- solve(rbind(cbind(crossprod(X1), crossprod(X1, Z)),
                    cbind(crossprod(Z, X1), crossprod(Z) + solve(Gstar) * lambda)))
  sqrt(mean((1 - diag(C)[-1] * lambda / diag(Gstar))[cand]))
}
c(implied_at_0.95 = blend(0.95), implied_at_0.999 = blend(0.999),
  realised = cor(u_hat[cand], g[cand]))
#>  implied_at_0.95 implied_at_0.999         realised
#>        0.8953664        0.9272495        0.9596232
```

Roughly half the gap, then, and the other half survives however little identity is mixed in. A mean reliability and a correlation taken across a *set* of candidates are not the same quantity: the second is helped by the spread of true values among the candidates, which the first knows nothing about. Both halves point the same way – the reliability is conservative – and Section 17.4 shows that the gap is this small only because this particular split was easy.

17.4 What the random split was measuring

The split at the top of this chapter took a random 60% of the *hybrids*. There are 625 hybrids but only 50 lines, so a sample that size contains every line several times over: each of the 250 candidates has **both of its parents** sitting in the training set, in other hybrids with recorded phenotypes. Given Section 17.2 – a hybrid's value is the mean of its two parental values – that split asks the model to interpolate inside a factorial whose margins it has already seen. It is not a hard question.

J. Burgueño, G. de los Campos, K. Weigel, and J. Crossa [85] named the schemes that distinguish the cases, and F. Technow, T. A. Schrag, W. Schipprack, E. Bauer, H. Simianer, and A. E. Melchinger [52] applied them to exactly this design, a factorial between two heterotic pools. Holding out *lines* rather than hybrids splits the candidates three ways: both parents seen (the random split, CV2), one parent unseen (CV1), neither parent seen (CV0). Removing 8 lines from each pool leaves 289 training hybrids and gives 272 CV1 and 64 CV0 candidates; the CV2 row below draws a random training set of the same 289, so the only thing that differs across the three rows is *what* was hidden, not how much.

```
cv_split <- function(seed, k = 8) {
  set.seed(seed)
  hoA <- sample(unique(ctx$ped$a), k)
  hoB <- sample(unique(ctx$ped$b), k)
  seen_a <- !(ctx$ped$a %in% hoA)
  seen_b <- !(ctx$ped$b %in% hoB)
  lin <- gblup(y, which(seen_a & seen_b), h2)          # lines held out
  rnd <- sample.int(N, sum(seen_a & seen_b))           # same size, random hybrids
  rd  <- gblup(y, rnd, h2)
  i2  <- setdiff(seq_len(N), rnd)
  i1  <- which(xor(seen_a, seen_b))
  i0  <- which(!seen_a & !seen_b)
  c(CV2 = cor(rd$u[i2],  g[i2]),  r2 = sqrt(mean(rd$rel[i2])),
    CV1 = cor(lin$u[i1], g[i1]),  r1 = sqrt(mean(lin$rel[i1])),
    CV0 = cor(lin$u[i0], g[i0]),  r0 = sqrt(mean(lin$rel[i0])))
}
cv <- t(sapply(1:20, cv_split))
```

scheme	accuracy	sd	reliability
CV2 – random hybrids held out	0.932	0.013	0.873
CV1 – one parent unseen	0.653	0.073	0.644

scheme	accuracy	sd	reliability
CV0 – both parents unseen	0.122	0.217	0.128

The accuracy the chapter opened with survives only in the first row. Ask the model about a hybrid with one unseen parent and it drops by a third; ask about one with two unseen parents and what is left is 0.12, with a spread across replicates nearly twice that – small, and not reliably distinguishable from nothing. D. Habier, R. L. Fernando, and J. C. M. Dekkers [141] separate the two things a genomic prediction can be using – the relationship between candidates and training individuals, and the marker effects estimated in their own right – and this table is that separation, measured. The first row is almost entirely relatedness. The third row is what is left when relatedness is removed, and it is the honest measure of what the 2000 markers bought.

That the third row collapses is not a defect of the fit. It is $\operatorname{rank}(\mathbf{G}) = L - 2$ again: the 289 training records carry at most 34 lines' worth of independent information, and asking them to pin down 2000 marker effects well enough to score a genotype no relative shares is asking for something that was never in the data. H. D. Daetwyler, B. Villanueva, and J. A. Woolliams [142] give the deterministic accuracy for exactly that regime, $r = \sqrt{nh^2/(nh^2 + M_e)}$, and with the units the simulation actually has – n the independent *lines*, and M_e the marker count, because these markers are drawn independently and carry no linkage to bundle them into segments – it agrees:

```
n_lines_train <- 2 * (book_cfg$n_pool_A - 8)
c(deterministic = sqrt(n_lines_train * h2 / (n_lines_train * h2 + book_cfg$m)),
  measured_CV0  = mean(cv[, "CV0"]))
#> deterministic  measured_CV0
#>    0.09180609    0.12191453
```

The most useful column is the last one in the table. The reliabilities computed from the mixed model equations – with no access to g – track the realised accuracies down the collapse, 0.64 against 0.65 for CV1 and 0.13 against 0.12 for CV0. The model knows. A programme that reports reliability alongside every GEBV does not need to discover this the way this chapter did; the warning is in the same solve that produced the prediction.

> ⚠ Report which parents the training set had seen
>
> An accuracy figure without its cross-validation scheme is not interpretable. The same fit, the same data and the same trait give 0.93, 0.65 or 0.12 here depending only on which hybrids were withheld. In a hybrid programme this is not a technicality: CV2 is the accuracy for filling in a partly-observed factorial, CV1 for a cross onto a new line from one pool, CV0 for a cross between two new lines. They are different questions, and only the first one is easy.

17.5 BayesA and BayesB: prediction from marker effects

GBLUP assumes every marker contributes an equal, infinitesimal share of the genetic variance. The Bayesian alphabet [19], [143] relaxes that one assumption at a time. **BayesA** gives every marker its own variance, drawn from a scaled-inverse-χ^2 prior – effects are shrunk individually rather than uniformly, which matters when a few markers carry much more signal than the rest. **BayesB** adds a point mass at zero: each marker has prior probability π of being excluded entirely, so the model performs variable selection as it fits. Both reduce to GBLUP's ridge-regression solution as a special case; they differ only in how much of the genetic architecture they let the data shape.

Fitting either is a different computational route to the same train-then-predict logic as GBLUP, in four steps rather than one linear solve: (1) declare a prior distribution for each marker effect β_k, the thing that separates the three models; (2) run a Gibbs sampler that alternates between drawing each β_k given the current residuals and every other parameter, and drawing the variance components given the current effects, on the training records only; (3) average the post-burn-in draws into a single point estimate $\hat{\boldsymbol{\beta}}$; (4) predict any candidate, phenotyped or not, by multiplying its marker dosages by $\hat{\boldsymbol{\beta}}$ – exactly the same last step as scoring a hybrid from GBLUP's $\hat{\mathbf{u}}$, just reached through markers instead of a relationship matrix.

17.5.1 The priors, explicitly

All three models write $\mathbf{g} = \mathbf{Z}\boldsymbol{\beta}$ and differ only in what they assume about β_k. Ridge regression, and therefore GBLUP, uses one variance for every marker; BayesA gives each marker its own, drawn from a scaled inverse chi-square; BayesB puts a spike at zero in front of it:

$$\text{ridge, and so GBLUP:} \quad \beta_k \sim N(0, \sigma^2_\beta) \qquad \text{BayesA:} \quad \beta_k \sim N(0, \sigma^2_k), \sigma^2_k \sim \chi^{-2}(\nu, S) \tag{17.6}$$

$$\text{BayesB:} \quad \beta_k \sim \begin{cases} 0 & \text{with probability } \pi \\ N(0, \sigma^2_k) & \text{otherwise.} \end{cases} \tag{17.7}$$

The scaled inverse chi-square is the conjugate prior for a normal variance, which is the whole reason it appears: it keeps the Gibbs sampler in closed form. Its two hyperparameters are the arguments the fits below pass. ν is `df = 5`, the prior weight in pseudo-observations – small, so the prior stays vague, but not zero, because the posterior is improper at $\nu = 0$. S is set from `R2 = 0.5`, the share of phenotypic variance the prior expects the markers to explain, spread over the markers by their observed variation. Neither is fitted; both are choices, and `R2` in particular is a statement about heritability made before seeing the data.

This book fits both with `bWGR::BayesA()`/`bWGR::BayesB()` [144] rather than by hand – unlike GBLUP, a Gibbs sampler over marker effects is not the few lines the mixed-model solve above was, and `bWGR` is exactly the tool `vanraden_G()`'s docstring pointed toward without naming:

```
library(bWGR)
M <- ctx$X * 2   # dosage 0/1/2, what bWGR expects
fitA <- BayesA(y[train], M[train, ], it = 1500, bi = 500, df = 5, R2 = 0.5)
fitB <- BayesB(y[train], M[train, ], it = 1500, bi = 500, pi = 0.95, df = 5, R2
= 0.5)
predA <- as.numeric(M[cand, ] %*% fitA$b)
predB <- as.numeric(M[cand, ] %*% fitB$b)
c(BayesA = cor(predA, g[cand]), BayesB = cor(predB, g[cand]))
#>    BayesA    BayesB
#> 0.9606263 0.9599031
```

Both fits produce a marker-effect vector `$b`; a candidate's GEBV is just its genotypes dotted with those effects. GBLUP produced the same quantity from the relationship matrix instead of the markers directly – two routes to the same thing, checked next.

17.5.2 What the sampler actually selects

Both fits also return the quantities the priors above are about – BayesA's per-marker variances in `$vb`, BayesB's posterior inclusion probabilities in `$d` – and on the polygenic trait there is nothing in them to look at. Rebuild the oligogenic architecture the cached sweep below uses, three markers of 2000 given effects and everything else exactly zero, and there is:

```
set.seed(book_cfg$seed + 42)
qtl  <- sample.int(book_cfg$m, 3)
Bq   <- rep(0, book_cfg$m); Bq[qtl] <- rnorm(3)
g_ol <- as.numeric(scale(vr$Z %*% Bq))
set.seed(31)
y_ol <- g_ol + rnorm(N, 0, sqrt((1 - h2) / h2))
round(apply(vr$Z[, qtl], 2, var) * Bq[qtl]^2 /
      sum(apply(vr$Z[, qtl], 2, var) * Bq[qtl]^2), 3)   # share of variance each
#> [1] 0.962 0.038 0.001
```

Worth knowing before reading anything into the sweep: the three effects were drawn from `rnorm(3)`, and this draw put 96% of the genetic variance on one of them. "Three causal markers" describes the construction; the trait is effectively monogenic, and the true π is nearer $1 - 1/2000$ than the $1 - 3/2000$ the caption below quotes.

```
oA <- BayesA(y_ol[train], M[train, ], it = 1500, bi = 500, df = 5, R2 = 0.5)
oB <- BayesB(y_ol[train], M[train, ], it = 1500, bi = 500, pi = 0.95, df = 5, R2
= 0.5)
```

```
big <- qtl[which.max(abs(Bq[qtl]))]
c(BayesB_d_at_QTL   = oB$d[big],  BayesB_d_median  = median(oB$d),
  BayesA_vb_at_QTL  = oA$vb[big], BayesA_vb_median = median(oA$vb))
#>  BayesB_d_at_QTL  BayesB_d_median BayesA_vb_at_QTL BayesA_vb_median
#>      0.999000013      0.179000005      0.011139854      0.002957477
```

```
par(mfrow = c(1, 2), mar = c(4, 4, 2, 1))
for (v in list(list(oB$d, "BayesB: posterior inclusion"),
               list(oA$vb, "BayesA: marker variance"))) {
  plot(v[[1]], pch = 16, cex = 0.3, col = "grey60", xlab = "marker", ylab = "",
       main = v[[2]], cex.main = 0.95)
  points(big, v[[1]][big], pch = 1, cex = 2, lwd = 2)
}
```

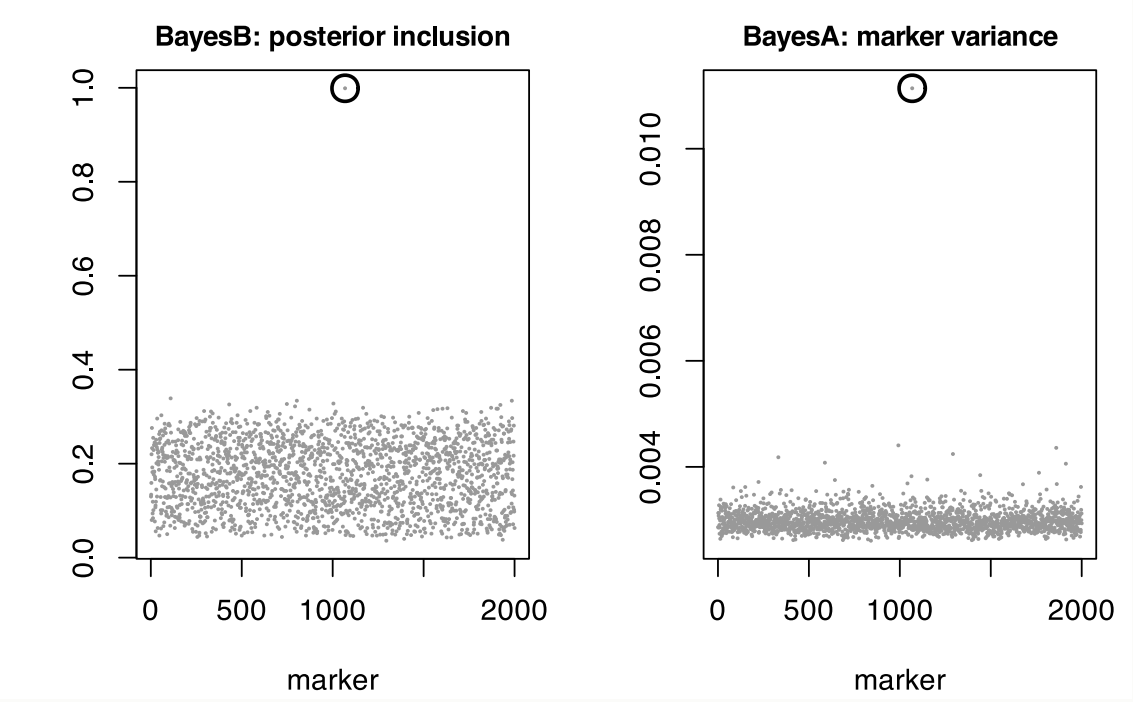

Figure 17.1: The two priors on the same oligogenic trait, with the marker carrying 96% of the genetic variance circled. Both find it. Only BayesB can then remove the other 1999 from the model.

17.5.3 What BayesB does that BayesA cannot

Both panels find the marker, which is worth saying plainly because the usual telling of this comparison implies BayesA does not: its variance for the QTL is about four times the median marker's and sits well clear of the background band. What separates the two priors is not detection but what each can do afterwards.

BayesA estimates a private variance σ_k^2 for every marker, and the only data bearing on σ_k^2 is the single effect β_k drawn from it. One observation never overwhelms a prior, so each σ_k^2 stays near the $\chi^{-2}(\nu, S)$ it started from however many individuals are phenotyped – D. Gianola, G. de los Campos, W. G. Hill, E. Manfredi, and R. Fernando [145] make this the central objection to describing the Bayesian alphabet as "learning the genetic architecture". The cost is not the missed QTL. It is that the other 1999 markers keep a variance they have not earned, and go on absorbing signal that belongs to the one marker that has. BayesB's spike deletes them, and it can afford to because it learns a single shared π from all 2000 markers at once rather than 2000 private variances from one observation each.

So the advantage is real but bounded, and shows up in accuracy rather than in the figure: on this architecture at $h^2 = 0.5$ the table below gives BayesB 0.952 against BayesA's 0.931 and GBLUP's 0.930. BayesA sits with GBLUP, not with BayesB, which is the practical summary of the paragraph above.

17.6 Verification: two routes agree

GBLUP with a flat prior on every marker's variance and the un-selected member of the Bayesian alphabet, Bayesian ridge regression (`iv = FALSE, pi = 0` in `bWGR::wgr()`), are the same model solved two different ways – one through the $N \times N$ relationship matrix, the other through the $N \times m$ marker matrix. The check below has three steps: substitute the marker-space model into the mixed model equations algebraically, read off the penalty each route implies, and then confirm numerically that the two solutions coincide to machine precision – not merely correlate.

They are the same model in a stronger sense than "they should agree". Substitute $\mathbf{u} = \mathbf{Z}\boldsymbol{\beta}$ and $\mathbf{G} = \mathbf{Z}\mathbf{Z}'/2\sum_k p_k(1-p_k)$ into the equations above and the two solutions are related by an identity: with t the training rows,

$$\hat{\mathbf{u}} = \mathbf{G}_{\cdot t}(\mathbf{G}_{tt} + \mathbf{I}\lambda)^{-1}(\mathbf{y}_t - \hat{\mu}) = \mathbf{Z}\hat{\boldsymbol{\beta}}, \qquad \hat{\boldsymbol{\beta}} = \mathbf{Z}_t'(\mathbf{Z}_t\mathbf{Z}_t' + \mathbf{I}\lambda_\beta)^{-1}(\mathbf{y}_t - \hat{\mu}), \tag{17.8}$$

with the two penalties differing by exactly the divisor that defines $\mathbf{G}$: $\lambda_\beta = \lambda \cdot 2\sum_k p_k(1-p_k)$. Getting that scalar wrong is the usual way this equivalence is reported as "approximate". It is not approximate:

```
lambda <- (1 - h2) / h2
yc  <- y[train] - mean(y[train])
u_G <- G[, train] %*% solve(G[train, train] + diag(lambda, length(train)), yc)
u_M <- vr$Z %*% crossprod(vr$Z[train, ],
         solve(tcrossprod(vr$Z[train, ]) + diag(lambda * s2, length(train)), yc))
max(abs(u_G - u_M))
#> [1] 1.354472e-14
```

Machine precision, over all 625 individuals. Note this route uses the raw $\mathbf{G}$: it needs no inverse of it, so Section 17.3.2's blend never enters, which is also why it is the cleaner statement of the identity.

The Gibbs sampler should land in the same place, and does, though only to sampling error – it is estimating λ from the data rather than being told it, and the blend used in `gblup()` has no exact marker-space equivalent:

```
yobs <- rep(NA_real_, N); yobs[train] <- y[train]
fit_brr <- wgr(yobs, M, iv = FALSE, pi = 0, it = 1500, bi = 500)
c(gblup_vs_gibbs = cor(u_hat, fit_brr$hat - mean(fit_brr$hat)))
#> gblup_vs_gibbs
#>      0.9992759
```

That correlation, 0.999, is the weaker of the two checks and is included because it exercises the code path the Bayesian fits above actually use. The identity is what proves the two routes are one model.

17.7 Which model wins, and when

`ctx$traits` is built from thousands of markers with independent, similarly sized effects – the textbook case GBLUP's infinitesimal-model assumption was designed for. A trait controlled by a handful of large-effect loci is a different regime, and it is the one BayesB's point-mass prior exists for. `book/scripts/regenerate.R genpred` reruns the fits above 15 times each, at two heritabilities, under both `ctx$traits`'s polygenic architecture and a book-local variant with genetic variance concentrated on three markers out of 2000:

arch	h2	GBLUP	BayesA	BayesB
oligogenic	0.2	0.789	0.782	0.788
polygenic	0.2	0.799	0.795	0.798
oligogenic	0.5	0.930	0.931	0.952
polygenic	0.5	0.935	0.935	0.934

```
sub <- reshape(acc, direction = "long", varying = c("GBLUP", "BayesA", "BayesB"),
               v.names = "accuracy", timevar = "method",
               times = c("GBLUP", "BayesA", "BayesB"))
sub$grp <- interaction(sub$arch, sub$h2, sep = "\nh2=")
```

```
boxplot(accuracy ~ method + grp, sub, las = 2, cex.axis = 0.7,
        col = rep(c("grey85", "grey65", "grey45"), 4),
        ylab = "accuracy (candidates)")
```

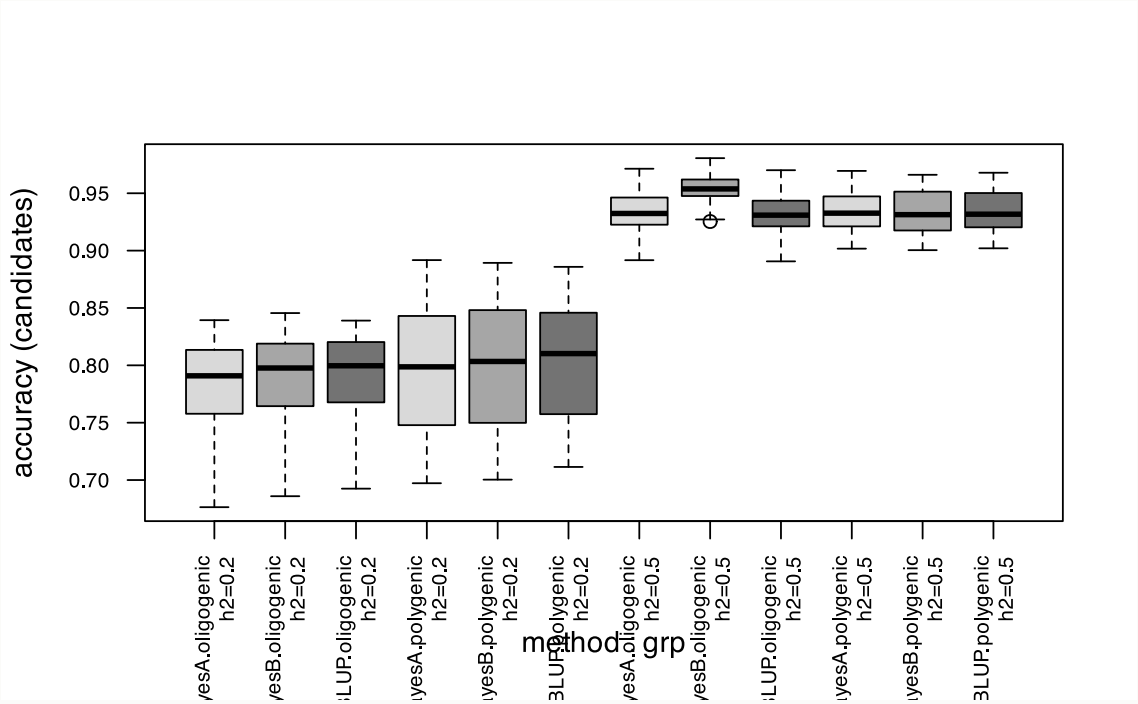

Figure 17.2: Prediction accuracy by architecture and heritability, 15 replicate splits each.

At $h^2 = 0.5$ under the oligogenic architecture, BayesB's mean accuracy is 0.022 higher than GBLUP's – the gap the model was built to close. Under the polygenic architecture the three methods are indistinguishable within replicate noise: with genetic variance spread over thousands of markers, BayesB's variable selection has nothing to select, and its point-mass prior buys nothing GBLUP was not already doing.

That result is not evidence that the alphabet rarely helps. It is Section 17.2 restated: on the polygenic arm the trait was *generated* from one shared marker variance, so GBLUP's prior is the true one and no competitor can do better than match it. What the comparison establishes is the weaker and more useful claim – that BayesA and BayesB cost nothing when they are wrong, and recover a real gap when they are right.

17.7.1 `pi` was fixed, not tuned

A real analyst does not know the true sparsity in advance. `genpred`'s second sweep holds the oligogenic architecture and $h^2 = 0.5$ fixed and only moves `pi`:

pi	BayesB
0.500	0.948
0.900	0.955
0.950	0.958
0.990	0.964
0.995	0.967

Accuracy improves monotonically as `pi` approaches the truth, but the degradation from being off is gradual, not catastrophic – `pi` is worth cross-validating, not agonising over.

The obvious alternative is to stop choosing it. BayesCπ keeps BayesB's spike-and-slab prior but adds one more level: instead of fixing π before the fit, it treats π itself as unknown and updates it by Metropolis-Hastings at every iteration, jointly with the marker effects and their inclusion indicators [144]. In principle

that removes the sweep above entirely, since the training data are left to say how sparse the architecture is. Fit it to both architectures:

```
cpi_poly <- BayesCpi(y[train],    M[train, ], it = 1500, bi = 500, df = 5, R2 = 0.5)
cpi_olig <- BayesCpi(y_ol[train], M[train, ], it = 1500, bi = 500, df = 5, R2 = 0.5)
c(pi_hat_polygenic  = cpi_poly$pi,   # truth: 0
  pi_hat_oligogenic = cpi_olig$pi,   # truth: ~0.9995
  acc_oligogenic    = cor(as.numeric(M[cand, ] %*% cpi_olig$b), g_ol[cand]),
  acc_BayesB_at_095 = cor(as.numeric(M[cand, ] %*% oB$b), g_ol[cand]))
#>  pi_hat_polygenic pi_hat_oligogenic    acc_oligogenic acc_BayesB_at_095
#>         0.4369196         0.4390892         0.9299389         0.9524220
```

It returns essentially the same π for a trait with 2000 causal markers as for one that is effectively monogenic, and predicts the oligogenic trait slightly worse than BayesB told the right answer. With 375 records and 2000 markers there is little in the data that speaks to π specifically – Section 17.3.2's 48 dimensions again – so what comes back is close to the prior. Estimating π is not a substitute for choosing it here; the sweep above remains the honest route.

17.8 From GEBVs to `stage1_build()`

Everything above ends where Chapter 4.'s contract begins, and this is where the opening paragraph's two steps close the loop. Training happened once, on the historical population's markers and phenotypes; the candidates below never contributed a phenotype to that fit, and their row in `hybrids_traits` is produced entirely from marker dosages – the same computation that, for a line-by-line combination not yet made as a physical cross, would score it before any seed exists. The candidates' predicted values, whichever model produced them, are exactly the `traits` argument `stage1_build()` expects – one column per trait, one row per hybrid:

```
hybrids_traits <- data.frame(hybrid = cand, trait1 = predB)
head(hybrids_traits)
```

hybrid	trait1
4	0.1398207
5	0.3260893
6	−0.1489823
11	1.6891953
13	1.3072349
16	0.2173326

Repeat the fit once per trait in the multi-trait index and this data frame is what feeds `stage1_build(X, ped, traits, fL)`. The second consumer is `ocs_quadprog(f, gebv, lambda, ...)` of Chapter 18., whose argument is literally named `gebv` and has until now been handed the truth.

Carry the reliabilities with them. Section 17.4's table is the argument for storing `fit$rel` next to every predicted value rather than discarding it: a candidate whose GEBV is unreliable is not a bad candidate, it is an unmeasured one, and the difference matters to a selection decision.

17.9 Back to Finding 5: a measured accuracy, not an assumed one

Chapter 15.'s Finding 5 assumed a prediction accuracy `r` and manufactured error correlated with relatedness to match it. The GBLUP fit above at $h^2 = 0.5$ gives a *measured* accuracy of 0.96 on the polygenic architecture, at the top of the range Finding 5 explored ($r \in \{0.5, 0.7, 0.9, 1\}$) – but Section 17.4 is the reason not to read that as the typical case. It is the CV2 number, the accuracy for a candidate whose two parents are already phenotyped in other combinations. The CV1 row of that table, 0.65, sits near the *bottom* of Finding 5's range, and the CV0 row falls well below anything it explored. Between them the two chapters bracket the same interval from opposite ends, and Finding 5's pessimistic cases are the realistic ones.

The errors are, in fact, structured by relatedness rather than white:

```
err <- g[cand] - u_hat[cand]
fcand <- ctx$f[cand, cand]
ut <- upper.tri(fcand)
cor(outer(err, err)[ut], fcand[ut])
#> [1] 0.1397751
```

Pairs of candidates that are more related have more correlated prediction errors – the mechanism Finding 5 assumed, confirmed here from an actual fitted model rather than injected. It is a modest correlation, not the extreme case Finding 5 stress-tested, because $h^2 = 0.5$ with `r length(train)` training records is a comparatively favourable regime; a smaller or less related training set would push it higher.

i What this chapter still does not model

Closing the "traits are given" gap opens several more, named here so they are not silently dropped:

- **Specific combining ability.** Section 17.2 showed there is none in `ctx$traits` to find: the trait is additive by construction, so a hybrid's value is exactly the mean of its parents'. SCA is not merely unmodelled here, it is absent from the data. A real hybrid programme has it – Chapter 13.'s dominance deviations are where it comes from – and would fit GCA and SCA jointly rather than one additive effect per marker.
- **Estimated variance components.** h^2 is supplied to `gblup()`, never estimated. A real fit gets λ from REML on the training data, and the reliabilities of Section 17.4 inherit that estimate's uncertainty.
- **Multivariate prediction.** `ctx$traits` has three correlated columns; this chapter predicted one. A multi-trait Bayesian model borrows information across traits and should out-predict fitting each one separately, especially for the traits with less training data.
- **Forward validation.** Section 17.4 splits by line, which is the right axis for a factorial, but every scheme in it still samples training and candidates from one generation. A programme selecting for the future validates forward in time, on a later cycle than it trained on.
- **Retraining across cycles.** Chapter 22. runs the *selection* side of a multi-cycle programme with a fixed, perfect index. A model trained once and never updated goes stale as the population it predicts drifts from the population it was trained on.
- **Genotyping cost and marker density.** Accuracy was measured here at a fixed 2000 markers; a real programme trades marker density against genotyping cost per individual, not just against training-set size.

Reproduce this chapter

All the code of this chapter, in order and with the explanation kept as comments, is one script: **08b-genomic-prediction.R**. Download it, install `hybdiv`, and run it top to bottom in an empty directory; cached results are fetched on first use.

V Optimisation

18 Optimal contributions and the α constraint 259
18.1 Constraint, not a weighted sum 259
18.2 Relation to the standard OCS formulation 260
18.3 Setting $\alpha_{\max}$ 262
18.4 A caveat carried over from the OCS literature . 265
18.5 Quadratic programming, when contributions are continuous 265
18.6 What belongs in the inner loop 266
18.6 Reproduce this chapter 266

19 Combinatorial selection 267
19.1 Four baselines before any optimiser 268
19.2 Greedy, with incremental updates 270
19.3 The encoding problem 273
19.4 Differential evolution 276
19.5 The full frontier 280
19.6 Which strategy to use 282
19.6 Reproduce this chapter 283

20 Advanced selection: structure, bounds and scale 285
20.1 Three facts about this problem 285
20.2 What lazy greedy would not buy 287
20.3 The move that was already paid for 287
20.4 The bound was not a bound 294
20.5 Rounding the relaxation into a plan 297
20.6 Exact, and how far the heuristics really are . . . 298
20.7 Scaling to 200 of 10,000 300
20.8 Multi-objective, and why not NSGA-II 304
20.9 What not to reach for 304
20.10 Which method to use 304
20.10 Reproduce this chapter 305

18. Optimal contributions and the α constraint

Everything so far has been measurement. This chapter turns the measurement into a constraint an optimiser can respect, and connects it to the optimal-contribution literature it specialises. `ctx$index` below is still the known-marker-effects version built in Chapter 4.; 3 shows how a real programme gets a `traits` matrix to hand `stage1_build()` in the first place, and what accuracy it can expect once it does.

18.1 Constraint, not a weighted sum

With $\mathbf{u}$ the multi-trait index and S the selected set of size n:

$$\max_{S,|S|=n} \frac{1}{n} \sum_{i \in S} u_i \qquad \text{subject to} \qquad \alpha(S) \le \alpha_{\max}. \tag{18.1}$$

The common alternative, $\max \bar{u} - \lambda\, \theta_S$, requires calibrating λ, which has no direct interpretation. The constraint form preserves an operational meaning: *"I accept losing at most $\alpha_{\max}$ of diversity."* That is a sentence a breeding committee can approve or reject. "Diversity is worth $\lambda = 3.7$ index units" is not.

The two are not equivalent in practice either. A weighted sum lets the optimiser buy index by spending diversity anywhere on the curve; a constraint fixes the budget and makes the optimiser find the best plan *inside* it. Since the acceptable loss is a policy decision and the exchange rate is not, the constraint form puts the decision in the right hands.

```
make_fitness
#> function (ctx, n_sel, alpha_max, gd_ref, penalty = 1000, max_use = NULL)
#> {
#>     N <- ctx$N
#>     f <- ctx[["f"]]
#>     ix <- ctx$index
#>     L <- ctx$n_lines
#>     pa <- as.integer(ctx$ped$a)
#>     pb <- as.integer(ctx$ped$b)
#>     function(par) {
#>         idx <- decode(par, N, n_sel)
#>         gd <- 1 - mean(f[idx, idx])
#>         viol <- max(0, (gd_ref - gd)/gd_ref - alpha_max)
#>         if (!is.null(max_use)) {
#>             cnt <- tabulate(c(pa[idx], pb[idx]), nbins = L)
#>             viol <- viol + sum(pmax(0, cnt - max_use))/(2 * n_sel)
#>         }
#>         -(mean(ix[idx]) - penalty * viol)
#>     }
#> }
#> <bytecode: 0x7bb10aa318>
#> <environment: namespace:hybdiv>
```

The violation enters as a *proportional* penalty, so a plan that overshoots by 10% of the budget is penalised in units the budget defines, whatever the absolute value of $\alpha_{\max}$.

18.1.1 This argument is older than it looks

The paragraphs above read as a claim about human decision-making: budgets are approvable, exchange rates are not. That is true, and it undersells the case. Kempthorne & Nordskog (1959) posed the identical

problem for selection indices – maximise aggregate gain subject to **zero expected change** in a specified direction – and solved it in closed form. Chapter 3. derives their solution; `index_restricted()` implements it. The distinguishing property is not softness:

```
P <- matrix(c(4.0, 1.2, 1.2, 1.0), 2)
G <- matrix(c(2.0, 0.9, 0.9, 0.3), 2)
a <- c(1, 1); R <- cbind(c(0, 1))
c(unrestricted = drop(crossprod(R, G %*% index_smith_hazel(P, G, a))),
  restricted   = drop(crossprod(R, G %*% index_restricted(P, G, a, R))))
#> unrestricted   restricted
#> 6.679688e-01 2.096025e-17
```

Exactly zero, not approximately. A penalty trades the restricted direction away at some price; a restriction refuses to trade it at any price. That is the distinction this section is drawing, and it has stood since 1959.

> **i** What is actually new here
>
> Not restriction over pricing. What is new is *what* is being restricted. Kempthorne and Nordskog restricted a trait – something with a genetic variance, an economic weight and a row in $\mathbf{G}$. The α constraint restricts the loss of molecular gene diversity, which has none of those: it is a property of a matrix, not of a phenotype, and it is bounded relative to a resampling baseline rather than held at zero.
>
> The algebra of restriction carries over. The object being restricted does not, which is why the rest of this chapter cannot simply cite 1959 and stop.

18.2 Relation to the standard OCS formulation

F. Gómez-Romano, B. Villanueva, J. Fernández, J. A. Woolliams, and R. Pong-Wong [4] write the general optimal-contribution problem, for a vector of continuous contributions $\mathbf{c}$, as

$$\text{minimise} \quad \mathbf{c}'\mathbf{G}\mathbf{c} \qquad \text{subject to} \qquad \mathbf{c}'\mathbf{s} = 0.5, \mathbf{c}'\mathbf{d} = 0.5, c_i \geq 0, \tag{18.2}$$

with $\mathbf{s}$ and $\mathbf{d}$ indicating sex and $\mathbf{G}$ the coancestry matrix chosen for managing diversity – here, $\mathbf{f}$.

The formulation of this book is that problem **restricted to $c_i \in \{0, 1/n\}$**: a fixed number n of candidates contribute equally, none partially. The minimisation of $\mathbf{c}'\mathbf{f}\mathbf{c}$ is turned into a constraint and $\bar{u}$ is promoted to the objective.

That restriction to equal, all-or-nothing contributions is exactly what makes the problem combinatorial, and what pushes the solution method from convex quadratic programming to a metaheuristic. It is also not arbitrary: a commercial hybrid programme plants a whole plot of a hybrid or does not plant it.

> **i** The sex constraint, in plants
>
> The per-sex sum constraint disappears in hermaphrodite and monoecious species (a single sum to 1) and reappears in dioecious crops and in schemes using cytoplasmic male sterility. In hybrid maize the natural problem is not OCS but **optimal cross selection**: $\mathbf{c}$ stops being an individual contribution and becomes a weight per cross, with the between-pool covariance entering explicitly. And there is an extra restriction the animal literature does not have – the **maximum number of crosses executable in a field**, which makes the problem mixed-integer.

18.2.1 The continuous relaxation as an upper bound

Dropping $c_i \in \{0, 1/n\}$ back to $0 \leq c_i \leq 1/n$ gives the continuous relaxation of *this* restricted problem. Every discrete solution is feasible under the relaxation, so the relaxation's optimum is a valid **upper bound**.

```
project_capped_simplex
#> function (v, cap)
#> {
```

```
#>     lo <- min(v) - 1
#>     hi <- max(v)
#>     for (i in 1:60) {
#>         tau <- (lo + hi)/2
#>         if (sum(pmin(pmax(v - tau, 0), cap)) > 1)
#>             lo <- tau
#>         else hi <- tau
#>     }
#>     pmin(pmax(v - (lo + hi)/2, 0), cap)
#> }
#> <bytecode: 0x7bb2d197e0>
#> <environment: namespace:hybdiv>
```

The cap is what makes the bound valid. Without $c_i \le 1/n$ the relaxation would put all weight on the single best candidate and the bound would be useless.

Turning the relaxation into a bound on the *constrained* problem needs one more step, and turning it into an actual selection needs a rounding: both are in Chapter 20..

```
lambdas <- 10^seq(-1, 2.5, length.out = 12)
relax <- ocs_relaxation(ctx, n_sel, lambdas)
fmt(as.data.frame(relax), 4)
```

lambda	index	theta	GD
0.1000	1.9919	0.6876	0.3124
0.2081	1.9919	0.6876	0.3124
0.4329	1.9919	0.6876	0.3124
0.9006	1.9919	0.6876	0.3124
1.8738	1.9911	0.6870	0.3130
3.8986	1.9900	0.6866	0.3134
8.1113	1.9885	0.6863	0.3137
16.8761	1.9831	0.6859	0.3141
35.1119	1.9270	0.6838	0.3162
73.0527	1.6173	0.6784	0.3216
151.9911	1.0023	0.6725	0.3275
316.2278	0.5729	0.6704	0.3296

Sweeping λ traces the merit-versus-diversity frontier. Each row is the best *continuous* plan at that exchange rate, and therefore an upper bound on what any discrete plan of the same diversity can achieve.

```
null <- null_distribution(ctx, n_sel, B = 400, B_full = 60)
a_relax <- (null$gd_ref - relax[, "GD"]) / null$gd_ref

sels <- list(random     = sel_random(ctx, n_sel),
             trunc_cap  = sel_truncation_cap(ctx, n_sel),
             greedy_w1  = sel_greedy(ctx, n_sel, w = 1),
             greedy_w3  = sel_greedy(ctx, n_sel, w = 3),
             truncation = sel_truncation(ctx, n_sel))
pts <- t(sapply(sels, function(s)
  c(alpha = alpha_loss(s, ctx, null$gd_ref), index = mean_index(s, ctx))))

plot(a_relax * 100, relax[, "index"], type = "b", pch = 1, lty = 2, col = "grey45",
     xlab = "gene diversity loss, alpha (%)", ylab = "mean index (s.d.)",
     xlim = range(c(a_relax, pts[, "alpha"]) * 100),
     ylim = range(c(relax[, "index"], pts[, "index"])))
points(pts[, "alpha"] * 100, pts[, "index"], pch = 19, col = "#b2182b")
text(pts[, "alpha"] * 100, pts[, "index"], rownames(pts), pos = 2, cex = 0.7)
abline(v = 0, lty = 3)
legend("bottomright", c("continuous OCS relaxation (upper bound)", "discrete
strategies"),
```

```
       col = c("grey45", "#b2182b"), pch = c(1, 19), lty = c(2, NA), bty = "n", cex
= 0.8)
grid()
```

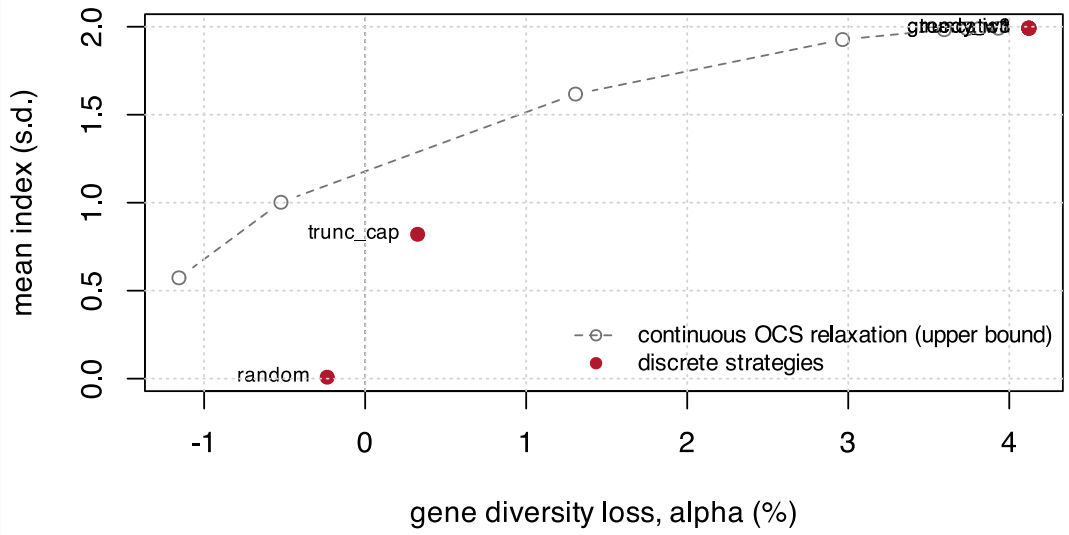

Figure 18.1: The continuous relaxation traces an upper bound. Discrete strategies must sit below and to the left of it; how far below is the convergence diagnostic.

> ⚠ Too few iterations gives an "upper bound" below the discrete optimum
>
> The relaxation is solved by projected gradient. Under-converged, it returns a value *smaller* than the discrete optimum – which is useless, and worse, silently so. The default of 4000 iterations is the minimum that converged in testing; re-check it whenever the dataset changes.

```
short <- ocs_relaxation(ctx, n_sel, lambdas = 10, iters = 50)
long  <- ocs_relaxation(ctx, n_sel, lambdas = 10, iters = 4000)
c(index_50_iters = short[, "index"], index_4000_iters = long[, "index"],
  greedy_w3 = mean_index(sels$greedy_w3, ctx))
#>   index_50_iters.index index_4000_iters.index             greedy_w3
#>               1.605937              1.987272              1.991937
```

18.3 Setting $\alpha_{\max}$

The limit **should not be chosen a priori**. Pure truncation – the greediest possible selection – defines the maximum attainable α. If that ceiling sits below your intended limit, the constraint restricts nothing.

```
attainable <- alpha_loss(sel_truncation(ctx, n_sel), ctx, null$gd_ref)
c(attainable_ceiling = attainable,
  a_5pct_limit_would_be_active = attainable > 0.05,
  a_1pct_limit_would_be_active = attainable > 0.01)
#>           attainable_ceiling a_5pct_limit_would_be_active
#>                   0.04121618                   0.00000000
```

```
#> a_1pct_limit_would_be_active
#>                   1.00000000
```

`stage2_select()` computes this ceiling automatically and warns about any requested α above it, rather than silently returning an unconstrained solution that looks constrained.

The correct procedure is to trace the frontier and choose its knee:

```
ws <- c(0, 0.25, 0.5, 0.75, 1, 1.5, 2, 3, 5, 8, 12)
front <- do.call(rbind, lapply(ws, function(w) {
  s <- sel_greedy(ctx, n_sel, w = w)
  data.frame(w = w, alpha = alpha_loss(s, ctx, null$gd_ref),
             index = mean_index(s, ctx), Ne_par = ne_parents(s, ctx))
}))
plot(front$alpha * 100, front$index, type = "b", pch = 19, col = "#1f5f8b",
     xlab = "gene diversity loss, alpha (%)", ylab = "mean index (s.d.)")
abline(v = attainable * 100, lty = 2, col = "#b2182b")
text(attainable * 100, min(front$index), "truncation ceiling", pos = 2, cex = 0.7,
     col = "#b2182b")
grid()
```

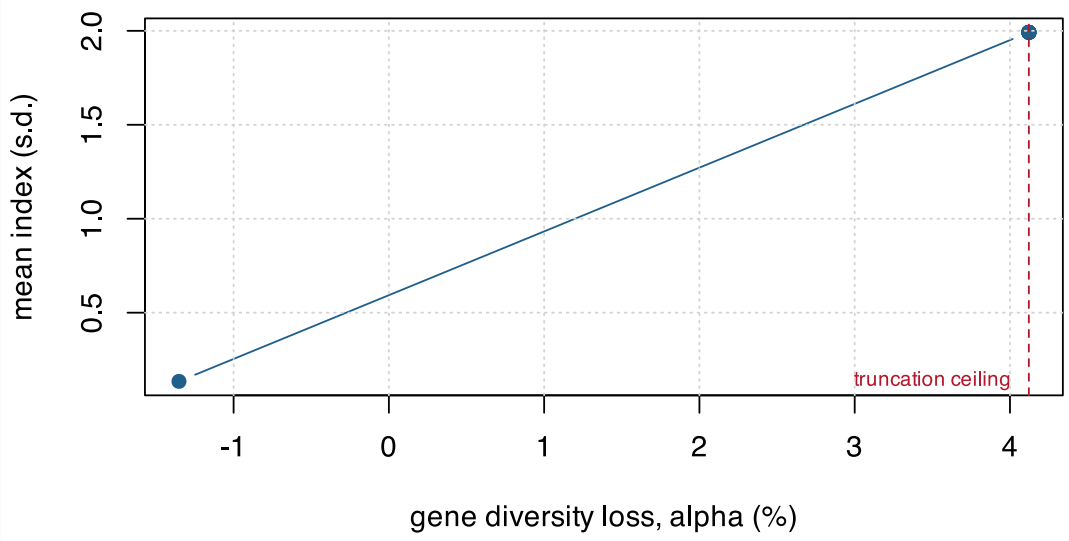

Figure 18.2: The greedy frontier, swept over the merit weight w. The knee is where further diversity costs index steeply.

```
fmt(front, 4)
```

w	alpha	index	Ne_par
0.00	−0.0135	0.1343	37.3057
0.25	0.0412	1.9919	13.0199
0.50	0.0412	1.9919	13.0199
0.75	0.0412	1.9919	13.0199
1.00	0.0412	1.9919	13.0199
1.50	0.0412	1.9919	13.0199
2.00	0.0412	1.9919	13.0199

w	alpha	index	Ne_par
3.00	0.0412	1.9919	13.0199
5.00	0.0412	1.9919	13.0199
8.00	0.0412	1.9919	13.0199
12.00	0.0412	1.9919	13.0199

18.3.1 How far $\alpha_{\max}$ moves when $\mathbf{f}$ moves

The ceiling above is quoted to four decimals, which invites more confidence than the marker panel supports. Chapter 12. showed that the MAF spectrum decides how badly the two inbreeding lenses disagree; the same spectrum feeds $\mathbf{f}$, and therefore $\bar{H}_{\text{ref}}$ and the ceiling. It is worth knowing which choices move the budget and which only look as though they should.

Rebuild everything downstream of the marker panel – $\mathbf{f}$, the null, the ceiling – under a filter, a density and an ascertainment scenario:

```
ceiling_for <- function(keep, label, B = 250) {
  Xs <- ctx$X[, keep, drop = FALSE]
  ck <- build_ctx(X = Xs, f = molecular_coancestry(Xs), G = NULL,
                  traits = ctx$traits, ped = ctx$ped, n_lines = ctx$n_lines,
                  pool = ctx$pool, fL = NULL, cfg = ctx$cfg)
  nl <- null_distribution(ck, n_sel, B = B, B_full = 20)
  data.frame(panel = label, markers = ncol(Xs), gd_ref = nl$gd_ref,
             alpha_max = alpha_loss(sel_truncation(ck, n_sel), ck, nl$gd_ref))
}
maf <- pmin(ctx$p0, 1 - ctx$p0)
polyA <- colMeans(simulate_lines(book_cfg)[seq_len(book_cfg$n_pool_A), ] / 2)
sens <- rbind(
  do.call(rbind, lapply(c(0, 0.01, 0.05, 0.10),
    function(t) ceiling_for(maf >= t, sprintf("MAF >= %.2f", t)))),
  ceiling_for(polyA > 0 & polyA < 1, "ascertained in pool A"))
fmt(sens, 4)
```

panel	markers	gd_ref	alpha_max
MAF >= 0.00	2000	0.3258	0.0411
MAF >= 0.01	1934	0.3369	0.0411
MAF >= 0.05	1780	0.3611	0.0404
MAF >= 0.10	1585	0.3881	0.0402
ascertained in pool A	1762	0.3535	0.0421

$\bar{H}_{\text{ref}}$ moves by 19% across these panels. $\alpha_{\max}$ moves by 4.7%. That is not a coincidence: α is a *ratio* of diversities measured on the same panel, so a filter that shifts the absolute level shifts numerator and denominator together and largely cancels out. **Filtering choices move the diversity level; they do not move the budget.**

Density does not move it either, but it does something worse: it makes the estimate noisy without making it wrong. Thinning the panel and averaging over six independent draws of which markers survive:

```
# Every subset is drawn BEFORE the loop. `null_distribution()` calls set.seed()
# internally, so a `sample.int()` written inside the loop would hand every
# replicate after the first the very same markers -- see the warning below.
set.seed(4)
subs <- lapply(c(250, 500, 1000),
               function(mm) lapply(1:6, function(i) sort(sample.int(ctx$m, mm))))
dens <- do.call(rbind, lapply(subs, function(sl) {
  v <- vapply(sl, function(k) ceiling_for(k, "", B = 200)$alpha_max, numeric(1))
  data.frame(markers = length(sl[[1]]), alpha_max = mean(v), sd = stats::sd(v))
}))
fmt(rbind(dens, data.frame(markers = ctx$m, alpha_max = attainable, sd = NA)), 4)
```

markers	alpha_max	sd
250	0.0445	0.0090
500	0.0461	0.0084
1000	0.0417	0.0067
2000	0.0412	NA

The means sit within one standard deviation of the full-panel ceiling all the way down to 250 markers. The standard deviation does not: at 250 it is about 20% of the estimate itself. A budget set from a thin panel is not biased, it is a coin flip – and a single thin-panel study has no way to tell which side it landed on.

So all three axes leave $\alpha_{\max}$ where it was. That is the useful finding, and it is worth stating positively: **the budget is a more portable quantity than the diversity level it is computed from.** $\bar{H}_{\text{ref}}$ is not comparable across panels; α largely is, provided the panel is dense enough for the estimate to be stable.

⚠ `null_distribution()` reseeds, and will freeze a resampling loop

It calls `set.seed(seed)` on entry, with `seed = 7` by default, so the random stream is reset every time it is invoked. Any `sample()` written *after* it inside a loop therefore draws the same values on every pass but the first, and a replication study silently becomes one replicate reported with a fabricated standard deviation.

This is not hypothetical: the table above returned two different wrong answers – once suggesting thin panels understate the ceiling, once that they overstate it – before the subsets were hoisted out of the loop. Draw randomisation up front, or pass an explicit varying `seed`.

Quote $\alpha_{\max}$ with the panel that produced it

A budget is not a property of the germplasm. It is a property of the germplasm *and* the marker panel, and only the second of those changes when the genotyping platform does. Re-derive the ceiling whenever the panel changes, and do not compare an α across two studies genotyped differently – Chapter 15. Finding 3's caveat on transfer applies here with more force, because this one is a constraint rather than a diagnostic.

18.4 A caveat carried over from the OCS literature

F. Gómez-Romano, B. Villanueva, J. Fernández, J. A. Woolliams, and R. Pong-Wong [4] show that the *predicted* rate of coancestry in the next generation, $E(\Delta f_{t+1}) = (\mathbf{c}'\mathbf{G}_t\mathbf{c} - f_t)/(1 - f_t)$, is systematically **biased downward** relative to the coancestry actually realised after offspring are generated – by roughly a factor of two in their simulations with small candidate pools, shrinking as the pool grows. The bias arises because $\mathbf{c}'\mathbf{G}\mathbf{c}$ is an expectation over Mendelian segregation, and the realised draw carries its own sampling variance on top of that expectation.

This does **not** affect the θ_S computed in this book. $\mathbf{f}$ is measured on real marker genotypes of an already-realised hybrid population; there is no forward-prediction step to bias.

It becomes relevant the moment the pipeline is extended to *plan a further generation* from the selected hybrids – to predict *their* offspring's coancestry from a contribution vector. At that point the naive $\mathbf{c}'\mathbf{f}\mathbf{c}$ prediction should be expected to underestimate realised coancestry, and the margin built into $\alpha_{\max}$ must account for it.

18.5 Quadratic programming, when contributions are continuous

Where the problem genuinely is continuous – composing a synthetic, assembling a core collection, setting seed proportions – solve it as a quadratic programme rather than with a metaheuristic. That is Chapter 2.'s `optimal_contributions()`, with `ocs_quadprog()` adding the merit term:

```
sub <- sample.int(ctx$n_lines, 20)
fLs <- ctx$fL[sub, sub]
merit <- as.vector(scale(rnorm(20)))
```

```
front_qp <- do.call(rbind, lapply(10^seq(-1, 2, length.out = 8), function(lam) {
  o <- ocs_quadprog(fLs, merit, lambda = lam, upper = 0.20)
  data.frame(lambda = lam, merit = o$merit, coancestry = o$coancestry,
             n_used = sum(o$c_i > 1e-6))
}))
fmt(front_qp, 4)
```

lambda	merit	coancestry	n_used
0.1000	1.1390	0.7254	5
0.2683	1.1390	0.7254	5
0.7197	1.1390	0.7254	5
1.9307	1.1324	0.7214	6
5.1795	1.1173	0.7165	6
13.8950	0.9831	0.7022	10
37.2759	0.6331	0.6865	17
100.0000	0.2734	0.6803	20

Note the `upper = 0.20` cap: an operational ceiling on how much any single line may contribute. Caps of this kind are usually easier to defend than the λ that produced them, and they are the continuous analogue of the per-line usage cap that Chapter 19. enforces in the decoder.

As λ falls the solution concentrates on fewer sources – the same mechanism, in continuous form, that makes $N_{e,\mathrm{par}}$ a necessary second constraint in the discrete problem.

18.6 What belongs in the inner loop

The optimiser performs on the order of 10^5 to 10^6 evaluations, so the cost per call decides what goes into the objective and what waits for post-hoc validation.

```
fmt(cost_per_eval(ctx, n_sel, reps = 20), 1)
```

metric	us_per_eval
theta_group	0
theta_from_freq	100
ne_parents	0
alleles_lost	150
ENE	50
ANE	350
eff_dim	100
Gst	1300

θ by the matrix route (or, better, the line route of Section 15.1) and $N_{e,\mathrm{par}}$ go into the objective. Allelic richness, A-NE and G_{st} are computed **once**, on the final solution. This is the same conclusion Chapter 15. reached from the production-scale benchmark, reproduced here at book scale so it can be re-run on your own data.

Reproduce this chapter

All the code of this chapter, in order and with the explanation kept as comments, is one script: **09-optimal-contributions.R**. Download it, install `hybdiv`, and run it top to bottom in an empty directory; cached results are fetched on first use.

19. Combinatorial selection

The problem is: choose n of N hybrids, maximising mean index subject to a diversity budget. This chapter builds up the strategies that solve it, from the two that need no optimiser at all to the differential evolution that beats them – and explains why the DE only beats them under one specific condition.

The decision is a concrete one, and it is the last of a cycle in a cross-pollinated crop. Two heterotic pools, A and B, hold L_A and L_B inbred (or doubled-haploid) lines. Every candidate is one cross $A_a \times B_b$, so the candidate set is the factorial $N = L_A \times L_B$ – in maize, the set of single crosses that could be made from the current elite lines without introducing anything new. The n you keep are the crosses that go forward: into multi-location testing, into seed increase, and – through their parents – into the next recycling cycle. Keeping too few distinct parents is not a statistical inconvenience, it is a decision to narrow the pool that the programme will breed from next.

What has to be in hand before any line of this chapter runs is the three-object contract of Chapter 16.:

- `X`, the $N \times m$ hybrid marker matrix, coded $\{0, 0.5, 1\}$ as the average of the two homozygous parents;
- `f`, the $N \times N$ molecular coancestry of A. Caballero and M. A. Toro [1], values in $[0, 1]$, built from `X` and never from a centred genomic relationship matrix (Section 1.2);
- `hybrids`, carrying each cross's parents and its predicted traits, from which `build_ctx()` forms the single index `ctx$index`, standardised to unit variance over the candidate pool (3).

Assumptions, declared once and not violated afterwards. Parent lines are homozygous, so a hybrid's genotype is the mean of its parents and `X` takes only the three values above. Markers are treated as neutral. The index is *frozen* before selection: nothing is re-predicted inside the optimiser, which matters because the index carries error that is correlated with relatedness (Chapter 15., finding 5). The allele-frequency base `ctx$p0` is the candidate pool and is set once in `build_ctx()`. Diversity loss α is always measured against a random subset of *the same size* `n_sel` (Section 14.9), never against the whole pool. Dominance appears nowhere in the objective: it is already inside the predicted hybrid performance, and the between-pool divergence that generates heterosis is a quantity to *preserve*, not to minimise (Chapter 13.).

The instance this chapter runs on:

```
c(candidates_N      = ctx$N,
  lines_A           = sum(ctx$pool == "A"),
  lines_B           = sum(ctx$pool == "B"),
  markers_m         = ctx$m,
  hybrids_kept      = n_sel,
  uses_per_line     = 2 * n_sel / ctx$n_lines,
  gd_ref            = null$gd_ref,
  gd_ref_sd         = null$gd_sd,
  alpha_ceiling_pct = 100 * alpha_loss(sel_truncation(ctx, n_sel), ctx, null$gd_ref))
#>      candidates_N           lines_A           lines_B         markers_m
#>      6.250000e+02      2.500000e+01      2.500000e+01      2.000000e+03
#>      hybrids_kept     uses_per_line            gd_ref         gd_ref_sd
#>      6.000000e+01      2.400000e+00      3.258386e-01      7.654486e-04
#> alpha_ceiling_pct
#>      4.121618e+00
```

Read it as a breeder would. 60 crosses are kept out of 625, which is 9.6% of the factorial. The 60 crosses consume $2n = 120$ parent slots spread over 50 lines, so a perfectly even plan would use each line 2.4 times – the scale against which any per-line cap has to be set. `gd_ref` is the gene diversity of a random subset of exactly this size, with its resampling standard deviation next to it, and the last row is the **attainable ceiling**: pure truncation loses 4.12% of it. Any $\alpha_{\max}$ above that ceiling constrains nothing at all, which is the first thing to check and the subject of Chapter 18..

19.1 Four baselines before any optimiser

Four families – random, truncation, truncation under a per-line cap, and greedy – in six rows, because greedy is shown at three merit weights.

```
sels <- list(
  random     = sel_random(ctx, n_sel),
  truncation = sel_truncation(ctx, n_sel),
  trunc_cap  = sel_truncation_cap(ctx, n_sel),
  greedy_div = sel_greedy(ctx, n_sel, w = 0),
  greedy_w1  = sel_greedy(ctx, n_sel, w = 1),
  greedy_w3  = sel_greedy(ctx, n_sel, w = 3)
)
fmt(t(sapply(sels, function(s) c(
  index      = mean_index(s, ctx),
  alpha_pct  = 100 * alpha_loss(s, ctx, null$gd_ref),
  Ns         = status_number(s, ctx),
  Ne_parents = ne_parents(s, ctx),
  max_line   = max_line_use(s, ctx)))), 3)
```

	index	alpha_pct	Ns	Ne_parents	max_line
random	0.008	−0.236	0.743	37.696	7
truncation	1.992	4.122	0.727	13.020	23
trunc_cap	0.820	0.326	0.740	31.034	4
greedy_div	0.134	−1.352	0.747	37.306	5
greedy_w1	1.992	4.122	0.727	13.020	23
greedy_w3	1.992	4.122	0.727	13.020	23

```
c(w1_is_truncation = setequal(sels[["greedy_w1"]], sels[["truncation"]]),
  w3_is_truncation = setequal(sels[["greedy_w3"]], sels[["truncation"]]))
#> w1_is_truncation w3_is_truncation
#>             TRUE             TRUE
```

Three of those six rows are the same plan. `greedy_w1` and `greedy_w3` return *exactly* the truncation set – same index, same α, same `max_line` – because both weights sit far above the crossover measured in the `w-units` chunk below, where the merit term swamps the coancestry term and the greedy argmax degenerates to the index ranking. Read them as one row until that measurement has been made; it is the reason a w grid of round numbers is a trap.

Read the columns in this order, because each answers a different question and three of them are routinely skipped:

1. `index`, in standard deviations of the candidate pool – what the plan is worth.
2. `alpha_pct`, the loss of gene diversity relative to a random subset of the same size – what it costs, on the lens the constraint will pin. Negative means *more* diverse than random.
3. `Ns`, the status number $1/(2\theta)$ (Section 14.3) – the same information as α on an absolute scale, kept because it is what the animal literature quotes.
4. `Ne_parents`, the effective number of parent lines, which is computed from counts alone and is the metric that catches "θ looks fine but twelve lines did all the work".
5. `max_line`, the largest number of selected crosses sharing one parent – the operational number, because that line is a seed parent and has to be increased, maintained and, in a CMS system, converted.

Truncation is the ceiling on gain and the floor on diversity. It is what every programme does by default, and the interesting question is not whether it loses diversity but how *little* index a constrained plan has to give up relative to it. Its `max_line` here is 23: one line parents that many of the 60 selected crosses, against 2.4 under even use. That is the concentration a cap is meant to remove, and it is visible without any coancestry matrix.

Truncation with a per-line cap is the cheap baseline that deserves more attention than it gets. It needs no coancestry matrix at all – only counting:

```
sel_truncation_cap
#> function (ctx, n_sel, cap = ceiling(2 * n_sel/ctx$n_lines) +
#>     1)
#> {
#>     ord <- order(ctx$index, decreasing = TRUE)
#>     used <- integer(ctx$n_lines)
#>     out <- integer(0)
#>     for (i in ord) {
#>         a <- ctx$ped$a[i]
#>         b <- ctx$ped$b[i]
#>         if (used[a] < cap && used[b] < cap) {
#>             out <- c(out, i)
#>             used[a] <- used[a] + 1L
#>             used[b] <- used[b] + 1L
#>             if (length(out) == n_sel)
#>                 break
#>         }
#>     }
#>     out
#> }
#> <bytecode: 0x7c096be580>
#> <environment: namespace:hybdiv>
```

The procedure, stated as a breeder would run it:

1. Rank all 625 candidate crosses by index, best first.
2. Walk down the ranking. Admit a cross only if *both* its parents are still below the cap.
3. Stop when `n_sel` crosses have been admitted, or when the ranking is exhausted – in which case the returned vector is **shorter than `n_sel`** and the cap was infeasible for this set size.
4. Feasibility is arithmetic, not luck: with L_A lines in pool A a cap of c admits at most $L_A c$ crosses, so the condition is $n \le \min(L_A, L_B)\, c$ – here 60 against

   100.

```
c(index_truncation = mean_index(sels$truncation, ctx),
  index_capped     = mean_index(sels$trunc_cap, ctx),
  index_retained_pct = 100 * mean_index(sels$trunc_cap, ctx) /
                             mean_index(sels$truncation, ctx),
  Ne_par_truncation = ne_parents(sels$truncation, ctx),
  Ne_par_capped     = ne_parents(sels$trunc_cap, ctx))
#>   index_truncation       index_capped index_retained_pct  Ne_par_truncation
#>          1.9919371          0.8203555         41.1838050         13.0198915
#>      Ne_par_capped
#>         31.0344828
```

```
cap <- ceiling(2 * n_sel / ctx$n_lines) + 1          # the function's default
ord <- order(ctx$index, decreasing = TRUE)
lines_used <- function(s) sum(tabulate(c(ctx$ped$a[s], ctx$ped$b[s]),
                                       ctx$n_lines) > 0)
c(cap                    = cap,
  reached_n_sel          = length(sels[["trunc_cap"]]) == n_sel,
  candidates_scanned     = max(match(sels[["trunc_cap"]], ord)),
  shared_with_truncation = length(intersect(sels[["truncation"]],
                                            sels[["trunc_cap"]])),
  lines_used_truncation  = lines_used(sels[["truncation"]]),
  lines_used_capped      = lines_used(sels[["trunc_cap"]]))
#>                    cap          reached_n_sel     candidates_scanned
#>                      4                      1                    413
#> shared_with_truncation  lines_used_truncation      lines_used_capped
#>                     16                     33                     32
```

The default cap is `ceiling(2 * n_sel / ctx$n_lines) + 1`, one above even use, which is 4 here. Filling 60 slots under it required scanning 413 of the 625 ranked candidates, and only 16 of the resulting crosses are also in the truncation set: the cap does not trim the top of the ranking, it reshuffles most of it. The last two rows count the same reshuffling in lines rather than in crosses, and they are the reason $N_{e,\mathrm{par}}$ is not a line count: the capped plan touches 32 distinct parent lines against truncation's 33, out of 50 available – to within one line, the same set size – while its effective number of parents is 31 against 13. What the cap changes is how evenly the 120 parent slots are spread over the lines that are used, not how many lines are used at all.

What that buys and what it costs, separately. The cap cuts the diversity loss from 4.12% to 0.33% and raises the effective number of parents from 13 to 31, at 41% of truncation's index. Whether that is a good trade is a programme decision, not a statistical one – but note that the capped plan is *feasible* at a 1% budget while truncation is not, and it was found by counting alone.

Before reaching for an optimiser, run this. If a per-line cap already meets your constraint at an acceptable index, you are done. The failure mode at this step is silent: a returned vector shorter than `n_sel`, which every downstream metric will happily evaluate on the wrong set size – `length()` it.

19.2 Greedy, with incremental updates

Adding hybrid j to an existing set changes the submatrix total by $2s_j + f_{jj}$, where s_j is the sum of f_{ij} over the already-chosen. That makes each step $O(N)$ instead of $O(n^2)$, and the whole construction affordable.

```
S <- sel_greedy(ctx, n_sel = 10, w = 1)
j <- setdiff(seq_len(ctx$N), S)[1]
f <- ctx[["f"]]
recomputed  <- sum(f[c(S, j), c(S, j)])                 # O(n^2), from scratch
incremental <- sum(f[S, S]) + 2 * sum(f[S, j]) + f[j, j] # O(n), the update
stopifnot(isTRUE(all.equal(recomputed, incremental)))
c(recomputed = recomputed, incremental = incremental)
#>  recomputed incremental
#>    87.07925    87.07925
```

```
sel_greedy
#> function (ctx, n_sel, w = 0, max_use = NULL)
#> {
#>     f <- ctx[["f"]]
#>     d <- diag(f)
#>     s <- numeric(ctx$N)
#>     avail <- rep(TRUE, ctx$N)
#>     out <- integer(n_sel)
#>     total <- 0
#>     if (!is.null(max_use)) {
#>         pa <- as.integer(ctx$ped$a)
#>         pb <- as.integer(ctx$ped$b)
#>         cnt <- integer(ctx$n_lines)
#>     }
#>     for (k in seq_len(n_sel)) {
#>         theta_new <- (total + 2 * s + d)/k^2
#>         score <- w * ctx$index - theta_new
#>         score[!avail] <- -Inf
#>         j <- which.max(score)
#>         if (!is.finite(score[j])) {
#>             out <- out[seq_len(k - 1L)]
#>             break
#>         }
#>         out[k] <- j
#>         avail[j] <- FALSE
#>         total <- total + 2 * s[j] + d[j]
#>         s <- s + f[, j]
#>         if (!is.null(max_use)) {
```

```
#>             for (a in c(pa[j], pb[j])) {
#>                 cnt[a] <- cnt[a] + 1L
#>                 if (cnt[a] >= max_use)
#>                   avail[pa == a | pb == a] <- FALSE
#>             }
#>         }
#>     }
#>     out
#> }
#> <bytecode: 0x7c0a37af58>
#> <environment: namespace:hybdiv>
```

One step k, in the order the code performs it:

1. `theta_new <- (total + 2 * s + d) / k^2` is the group coancestry the set *would* have, for every candidate at once, with `total` the current submatrix sum, `s` the running vector of coancestry sums against the chosen set and `d = diag(f)`. The normaliser is k^2, the size after the addition.
2. `score <- w * ctx$index - theta_new` – merit against relatedness.
3. Candidates already chosen are set to `-Inf`; with `max_use` set, so is every hybrid of a line that has reached its cap. The cap is therefore a **feasibility test on the move**, never a penalty on the result – the partition-matroid treatment of Chapter 20..
4. The argmax is taken, `total` and `s` are updated in $O(N)$, and the parent counts incremented.
5. If no finite score remains, the loop exits early and the returned vector is short – the same check as for the capped truncation.

The score is $w \cdot \text{index} - \theta$. At $w = 0$ this is pure diversity – the ceiling on diversity and near-zero gain. As $w \to \infty$ it converges to truncation. Sweeping w traces a frontier – but **only if the grid is in the right units**, and that is the trap at this step.

w is an exchange rate: index standard deviations per unit of group coancestry. The two quantities it converts between differ by four orders of magnitude, and that ratio is measurable before any sweep is run:

```
S30 <- sel_greedy(ctx, 30, w = 0)
theta_if_added <- (sum(f[S30, S30]) + 2 * colSums(f[S30, ]) + diag(f)) / 31^2
c(index_spread = diff(range(ctx$index)),
  theta_spread = diff(range(theta_if_added)),
  w_crossover  = diff(range(theta_if_added)) / diff(range(ctx$index)))
#> index_spread theta_spread  w_crossover
#> 6.055856e+00 3.293444e-04 5.438446e-05
```

`w_crossover` is the weight at which the two terms of the score have equal range. Below it the sweep returns the diversity solution; a decade or two above it, every point collapses onto truncation. A grid of $w \in \{0.25, 0.5, \ldots, 30\}$ – the obvious one, and the one this chapter used before it was measured – sits entirely above the transition and returns the *same* selection at every point, which makes greedy look an order of magnitude worse than it is. The grid below spans six decades around the crossover and is the one `book/scripts/regenerate.R` uses for the benchmark of Chapter 20., so the two chapters report the same greedy.

```
ws <- c(0, 10^seq(-5, 1, length.out = 19))
gf <- do.call(rbind, lapply(ws, function(w) {
  s <- sel_greedy(ctx, n_sel, w = w)
  data.frame(w = w, alpha = alpha_loss(s, ctx, null$gd_ref),
             index = mean_index(s, ctx))
}))
plot(gf$alpha * 100, gf$index, type = "b", pch = 19, col = "#1f5f8b",
     xlab = "gene diversity loss, alpha (%)", ylab = "mean index (s.d.)")
points(alpha_loss(sels$truncation, ctx, null$gd_ref) * 100,
       mean_index(sels$truncation, ctx), pch = 4, cex = 1.5, col = "#b2182b")
text(alpha_loss(sels$truncation, ctx, null$gd_ref) * 100,
     mean_index(sels$truncation, ctx), "truncation", pos = 2, cex = 0.75)
abline(v = 0, lty = 3); grid()
```

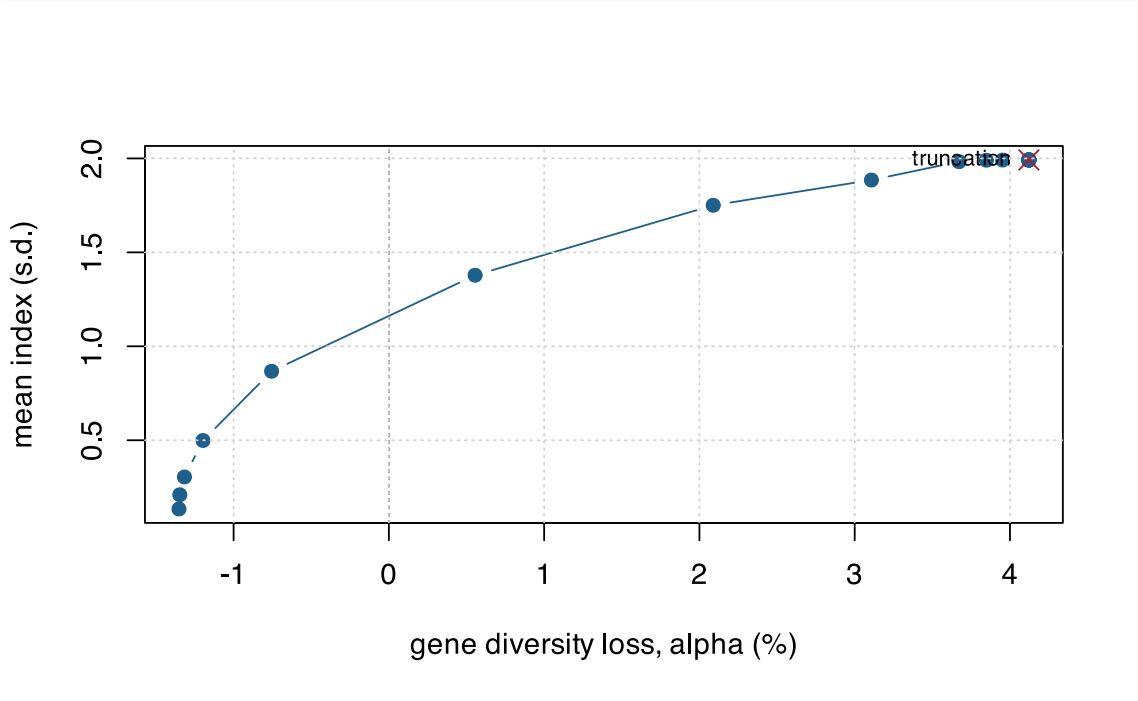

Figure 19.1: The greedy frontier. w = 0 is maximum diversity; large w converges to truncation. The useful range of w spans roughly 1e-5 to 1e-2 on this dataset, the same window the prose and the sweep table below quote: below it the sweep returns the maximum-diversity plan, above it the truncation plan, and every row outside the window is a duplicate of one of those two.

```
fmt(gf[gf$w <= 0.01, ], 5)
```

w	alpha	index
0.00000	−0.01352	0.13431
0.00001	−0.01347	0.20982
0.00002	−0.01317	0.30454
0.00005	−0.01197	0.49867
0.00010	−0.00755	0.86717
0.00022	0.00555	1.37820
0.00046	0.02089	1.75058
0.00100	0.03107	1.88486
0.00215	0.03672	1.98253
0.00464	0.03848	1.99043
0.01000	0.03952	1.99123

```
step <- which.max(gf$alpha > 0)          # first grid point that loses diversity
jump <- gf[c(step - 1L, step), ]
fmt(jump, 5)
```

	w	alpha	index
5	0.00010	−0.00755	0.86717
6	0.00022	0.00555	1.37820

```
# The upper edge of the useful window: the smallest w whose plan is already the
# truncation plan, above which every row of the sweep is a duplicate.
flat_from <- min(gf$w[gf$index >= max(gf$index) - 1e-12])
c(flat_from = flat_from, rows_above = sum(gf$w > flat_from))
```

```
#>  flat_from rows_above
#> 0.02154435 8.00000000
```

Two readings, and the second is the argument for everything that follows. First, the whole frontier lives in the three decades between $w = 10^{-5}$ – where the sweep is still moving, 0.13 at $w = 0$ against 0.21 at the first non-zero grid point – and $w \approx 10^{-2}$, by which the plan is within 0.04% of truncation's index. From $w = 0.0215$ upward every one of the remaining 8 rows is the *identical* selection, and a sweep whose rows are all the same selection is a sweep in the wrong units, not a flat frontier. Second, the sweep is parameterised by w and *the breeder's constraint is not.* Between the two adjacent grid points printed above, α jumps from −0.76% to 0.56% and the index from 0.87 to 1.38; a budget falling inside that interval cannot be requested of a greedy sweep at all, only stumbled upon. An optimiser parameterised by $\alpha_{\max}$ can be asked for it directly.

Greedy solutions are also what warm-start the differential evolution. That is not an optimisation detail – it is the difference between the DE working and not working.

19.3 The encoding problem

An N-dimensional 0/1 encoding is infeasible for differential evolution at this scale: at production size that is 5041 binary dimensions. Instead, encode n_{sel} continuous values in $[1, N]$ and decode them to unique indices.

```
decode
#> function (par, N, n_sel)
#> {
#>     k <- as.integer(par)
#>     k[k < 1L] <- 1L
#>     k[k > N] <- N
#>     dup <- duplicated(k)
#>     if (any(dup))
#>         k[dup] <- setdiff(seq_len(N), k[!dup])[seq_len(sum(dup))]
#>     k
#> }
#> <bytecode: 0x7c0631c078>
#> <environment: namespace:hybdiv>
```

The decode, step by step:

1. `as.integer(par)` truncates each coordinate toward zero, so the real interval $[i, i + 1)$ maps to candidate i. `sel_de()` searches the box $[1, N + 0.999]$, so candidates 1 to $N - 1$ each own a full unit interval and candidate N owns $[N, N + 0.999]$ – narrower by one part in a thousand, because the upper bound stops just short of $N + 1$ instead of reaching it.
2. Coordinates outside the box are clamped to the ends.
3. `duplicated()` marks every coordinate after the first that lands on an already-used candidate.
4. Each duplicate is replaced by the lowest index not otherwise in use, so the result always has exactly `n_sel` distinct candidates.

A worked case, small enough to check by hand:

```
d <- decode(c(3.9, 3.2, 1.1, 7.5), N = 10, n_sel = 4)
stopifnot(all(d == c(3, 2, 1, 7)))
d
#> [1] 3 2 1 7
```

Truncation gives $(3, 3, 1, 7)$; the second coordinate is the duplicate; the lowest index not in $\{3, 1, 7\}$ is 2; the repair returns $(3, 2, 1, 7)$. The same mechanism is what lets a warm start be injected exactly: a seed is written into the population as `index + 0.5`, the middle of its own interval, and decodes back to itself.

```
s0 <- sel_greedy(ctx, n_sel, w = 1)
c(box_lower      = 1,
  box_upper      = ctx$N + 0.999,
  width_typical  = 2 - 1,                      # candidate 1 owns [1, 2)
```

```
  width_last     = (ctx$N + 0.999) - ctx$N,     # candidate N owns [N, N+0.999]
  seed_roundtrip = all(decode(s0 + 0.5, ctx$N, n_sel) == s0))
#>      box_lower      box_upper  width_typical     width_last seed_roundtrip
#>          1.000        625.999          1.000          0.999          1.000
```

Duplicates are repaired **deterministically**. A stochastic repair would make the fitness noisy, and a noisy fitness stops DE converging – the same parameter vector must always score the same.

```
set.seed(3)
p1 <- runif(n_sel, 1, ctx$N + 1)
p2 <- runif(n_sel, 1, ctx$N + 1)
d1 <- decode(p1, ctx$N, n_sel)
d2 <- decode(p2, ctx$N, n_sel)
c(unique_1 = length(unique(d1)), unique_2 = length(unique(d2)),
  in_range = all(c(d1, d2) >= 1 & c(d1, d2) <= ctx$N),
  collisions_repaired = sum(duplicated(as.integer(p1))),
  deterministic = identical(d1, decode(p1, ctx$N, n_sel)))
#>            unique_1            unique_2            in_range collisions_repaired
#>                  60                  60                   1                   2
#>       deterministic
#>                   1
```

> **i** A known bias in the repair
>
> The repair fills in the lowest free indices, which introduces a slight bias toward low-numbered candidates. It is marked in the source with a `ponytail:` comment naming the ceiling and the upgrade path – switch to "nearest free" if the DE ever stalls. It has not mattered at these scales, but it is the kind of thing that should be written down rather than discovered later.

19.3.1 A third encoding: one weight per line

Section 15.1 showed that θ depends only on the line-usage vector. That makes a third encoding available, and one that changes what the coordinates *mean*: one weight per **line**, dimension L. Against the N-dimensional key vector that is a collapse from 3600 coordinates to 120 in the run below. Against the n_{sel} coordinates `sel_de()` actually searches it is close to a wash, and the numbers below say which of those two comparisons is which.

```
decode_lines
#> function (p, ctx, n_sel, max_use = NULL, w_div = 3)
#> {
#>     par <- ctx$parents
#>     sc <- ctx$index + w_div * (p[par[, 1]] + p[par[, 2]])
#>     ord <- order(-sc)
#>     if (is.null(max_use))
#>         return(ord[seq_len(n_sel)])
#>     sel <- integer(n_sel)
#>     cnt <- integer(ctx$n_lines)
#>     k <- 0L
#>     for (h in ord) {
#>         a <- par[h, 1]
#>         b <- par[h, 2]
#>         if (cnt[a] < max_use && cnt[b] < max_use) {
#>             k <- k + 1L
#>             sel[k] <- h
#>             cnt[a] <- cnt[a] + 1L
#>             cnt[b] <- cnt[b] + 1L
#>             if (k == n_sel)
#>                 break
#>         }
```

```
#>     }
#>     if (k < n_sel) {
#>         rest <- setdiff(ord, sel[seq_len(k)])
#>         sel[(k + 1L):n_sel] <- rest[seq_len(n_sel - k)]
#>     }
#>     sel
#> }
#> <bytecode: 0x7c0573de00>
#> <environment: namespace:hybdiv>
```

The line weight shifts each hybrid's score by the weights of its two parents, and hybrids are taken greedily subject to the per-line usage cap – so the cap is satisfied **by construction** rather than by penalty. In detail:

1. Each candidate scores `ctx$index + w_div * (p[a] + p[b])`, with `p` the line-weight vector being searched and `w_div = 3` by default. A line's weight therefore moves every cross it parents, which is what makes the decision a *line* decision.
2. Candidates are taken in decreasing score, skipping any whose parents have reached `max_use`.
3. If the cap makes `n_sel` unreachable, the selection is topped up ignoring it, and the fitness function's penalty term then sees the violation – so an infeasible cap degrades gracefully instead of returning a short vector.

At `p = 0` the decoder is exactly capped truncation, which is the natural zero of the encoding; Chapter 20. verifies that identity numerically where the encoding is actually driven.

The claim this chapter opened with – that an N-dimensional encoding is infeasible for DE at this scale – is measured rather than asserted, and the measurement is cached in `data/de_constraints.csv`. It is worth being exact about what it compares, because **neither arm of it is `sel_de()`**. Both are driven through `make_fitness_fast()`, on a pool of $N = 3600$ crosses from $L = 120$ lines, under the four simultaneous restrictions of Chapter 15.: diversity loss $\leq 0.5\%$, $N_{e,\mathrm{par}} \geq 50$, at most 6 uses per line, and heterotic-pool balance within 5%. One arm is `decode_rank()` – one random key per *hybrid*, dimension 3600, the N-dimensional encoding under indictment. The other is `decode_lines()`, dimension 120.

encoding	dim	index	index_min	index_max	loss %	Ne parents	max uses	seconds
hybrid keys / penalty	3600	0.351	0.343	0.358	0.060	76.634	5.667	6.550
hybrid keys / repair	3600	0.351	0.343	0.358	0.060	76.634	5.667	9.621
line weights / repair	120	1.988	1.956	2.017	0.448	50.084	6.000	12.126

```
#>   hybrid_keys_mean  hybrid_keys_range  line_weights_mean line_weights_range
#>              0.351              0.014              1.988              0.061
#>     ratio_of_means
#>              5.667
```

At dimension 3600 the search reaches index 0.351, at dimension 120 it reaches 1.988, and the seed-to-seed range within each arm (0.014 and 0.061) is an order of magnitude smaller than the distance between them, so the ordering is not a seed effect. Penalty and repair constraint handlers returned *numerically identical* plans at dimension 3600, because at that dimension the search never travels far enough from its random start to reach the boundary where the two handlers differ.

Three things that result does **not** say, each of which it has been used to say:

- It is not a comparison against the encoding this chapter runs. `sel_de()` searches n_{sel} coordinates, not N; in the book's own head-to-head benchmark (Chapter 20.) `sel_de()` finishes *ahead* of the line-encoded `sel_de_fast()` at both scales – 1.36 against 1.059 at book scale, and 1.446 against 1.336 at production scale. The line encoding beats the N-dimensional encoding; it does not beat this one.
- It is not a claim that one evaluation is cheaper. It is not, and Section 20.7 measures both.
- It is not a clean like-for-like. Four restrictions were active at once and the line decoder satisfies one of them – the per-line cap – by construction, so part of the margin is a penalty the other arm had to pay and the line arm did not.

The selected-set size of that cached run was not recorded alongside it, which is worth naming rather than papering over: the table is reproducible in direction, not to the digit.

The encoding decides whether the optimiser works. The constraint handler does not. Neither decides which of two *viable* encodings wins – that has to be measured, and Chapter 20. measures it.

19.4 Differential evolution

The run, as a procedure:

1. Fix `n_sel` first, then obtain `gd_ref` from `null_distribution()` **at that same size**. A reference computed at another size is a different quantity and every α downstream is wrong by a fixed factor.
2. Choose $\alpha_{\max}$ below the attainable ceiling printed at the top of this chapter, by the procedure of Chapter 18..
3. Call `sel_de()`. It builds its warm start, injects it into the initial population, and hands `DEoptim` the fitness $-(\bar{u}_S - 1000 \cdot \text{violation})$ to minimise.
4. Check the returned plan before using it: realised α against the budget, `max_line`, $N_{e,\text{par}}$, and the gap to the bound. The diversity constraint is a **penalty**, so a returned plan is not guaranteed feasible – the violation is only made expensive.

```
sel_de
#> function (ctx, n_sel, alpha_max, gd_ref, NP = 300, itermax = 2000,
#>     seeds = NULL, trace = FALSE, max_use = NULL)
#> {
#>     fit <- make_fitness(ctx, n_sel, alpha_max, gd_ref, max_use = max_use)
#>     if (is.null(seeds))
#>       seeds <- lapply(c(0, 0.25, 0.5, 1, 2, 4, 8, 16), function(w) sel_greedy(ctx,
#>             n_sel, w = w, max_use = max_use))
#>     seeds <- Filter(function(s) length(s) == n_sel, seeds)
#>     initial <- matrix(runif(NP * n_sel, 1, ctx$N + 1), NP, n_sel)
#>     for (i in seq_len(min(length(seeds), NP))) initial[i, ] <- seeds[[i]] +
#>         0.5
#>     ctrl <- DEoptim::DEoptim.control(NP = NP, itermax = itermax,
#>         trace = trace, initialpop = initial)
#>     res <- DEoptim::DEoptim(fit, lower = rep(1, n_sel), upper = rep(ctx$N +
#>         0.999, n_sel), control = ctrl)
#>     decode(res$optim$bestmem, ctx$N, n_sel)
#> }
#> <bytecode: 0x7c0a0a93b8>
#> <environment: namespace:hybdiv>
```

```
alpha_max <- 0.01
set.seed(9)
de_idx <- suppressWarnings(
  sel_de(ctx, n_sel, alpha_max = alpha_max, gd_ref = null$gd_ref,
         NP = 120, itermax = 250))

fmt(c(index      = mean_index(de_idx, ctx),
      alpha_pct  = 100 * alpha_loss(de_idx, ctx, null$gd_ref),
      budget_pct = 100 * alpha_max,
      Ns         = status_number(de_idx, ctx),
      Ne_parents = ne_parents(de_idx, ctx)), 3)
#>      index  alpha_pct budget_pct         Ns Ne_parents
#>      1.330      0.978      1.000      0.738     22.857
```

The realised loss sits just inside the budget, which is the sign that the constraint is active and the penalty correctly scaled: with `penalty = 1000` multiplying the *absolute* violation, one tenth of a percentage point of excess loss costs a full index standard deviation. A plan that comes back well inside the budget at a low index usually means the opposite – that the penalty dominated and the search never approached the boundary.

19.4.1 The warm start is not optional

The strong form of this claim was measured at the package's own simulation scale – 2500 candidates from 50 + 50 lines, $n = 100$, 4.5×10^5 evaluations, recorded as finding 7 of `README.md` – where a cold-started DE finishes *below* the greedy solutions it was supposed to improve on. That run is not reproduced here; this chapter runs 625 candidates and 3×10^4 evaluations, and the conclusion at this scale is weaker. What follows is the same comparison, measured rather than asserted.

```
budget <- list(NP = 120, itermax = 250)
set.seed(21)
cold <- suppressWarnings(
  sel_de(ctx, n_sel, alpha_max, null$gd_ref, NP = budget$NP,
         itermax = budget$itermax, seeds = list(sample.int(ctx$N, n_sel))))
# `seeds = NULL`, the default, would have built the greedy sweep for us.

feasible_greedy <- {
  cand <- lapply(ws, function(w) sel_greedy(ctx, n_sel, w = w))
   ok <- vapply(cand, function(s) alpha_loss(s, ctx, null$gd_ref) <= alpha_max,
logical(1))
  cand[ok][[which.max(vapply(cand[ok], mean_index, numeric(1), ctx = ctx))]]
}

res <- c(`cold DE` = mean_index(cold, ctx),
         `best feasible greedy` = mean_index(feasible_greedy, ctx),
         `warm DE` = mean_index(de_idx, ctx))
bp <- barplot(res, col = c("grey70", "#1f5f8b", "#b2182b"), ylab = "mean index
(s.d.)",
              ylim = c(0, max(res) * 1.2))
text(bp, res, sprintf("%.3f", res), pos = 3, cex = 0.8)
```

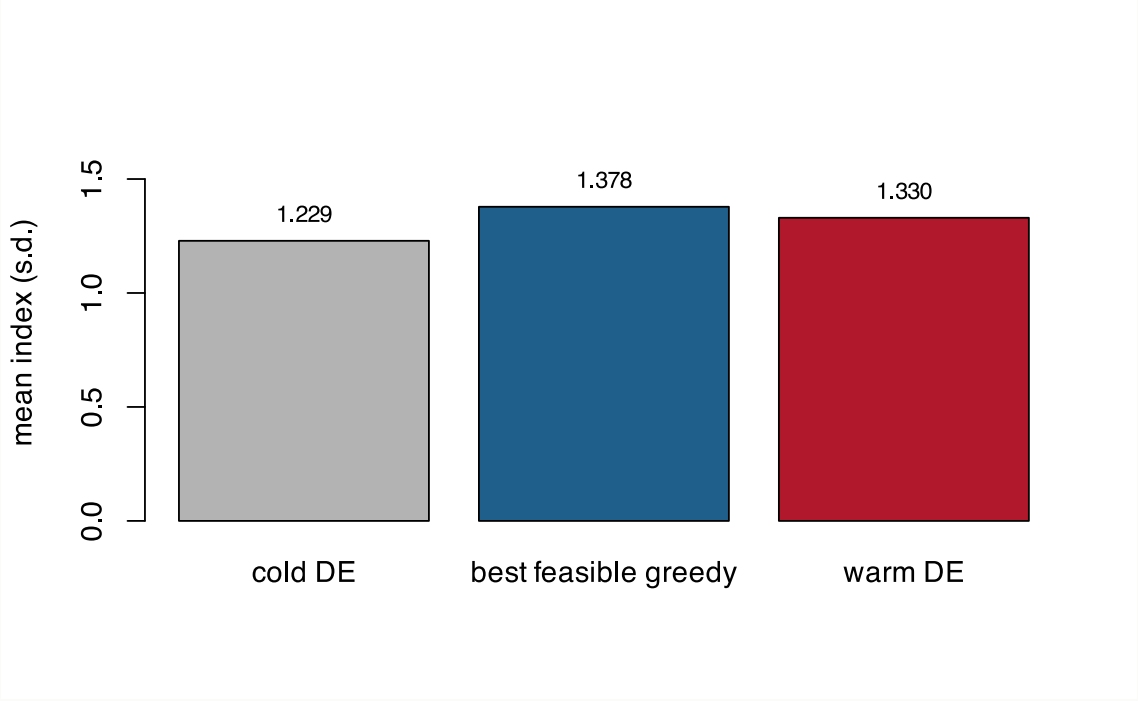

Figure 19.2: With and without a greedy warm start, at identical evaluation budget, at book scale. The cold start finishes below the best feasible greedy solution; the warm start finishes above the cold start but does not overtake greedy at this budget.

```
fmt(res, 4)
#>              cold DE best feasible greedy             warm DE
#>               1.2289              1.3782              1.3299
```

One run of each is not a comparison. Three seeds, with the cold and warm arms branched off the *same* seed so that the two differ only in the initial population:

```
reps <- do.call(rbind, lapply(c(9, 21, 33), function(sd_) {
  set.seed(sd_)
  warm <- suppressWarnings(
    sel_de(ctx, n_sel, alpha_max, null$gd_ref, NP = 120, itermax = 250))
  set.seed(sd_)
  cold <- suppressWarnings(
    sel_de(ctx, n_sel, alpha_max, null$gd_ref, NP = 120, itermax = 250,
           seeds = list(sample.int(ctx$N, n_sel))))
  data.frame(seed = sd_, warm = mean_index(warm, ctx),
             cold = mean_index(cold, ctx))
}))
fmt(reps, 4)
```

seed	warm	cold
9	1.3299	1.1589
21	1.3282	1.2289
33	1.3327	1.2016

```
fmt(c(warm_mean = mean(reps$warm), warm_sd = sd(reps$warm),
      cold_mean = mean(reps$cold), cold_sd = sd(reps$cold)), 4)
#> warm_mean   warm_sd cold_mean   cold_sd
#>    1.3303    0.0023    1.1965    0.0353
```

Result: over three seeds the warm start averages 1.33 with a standard deviation of 0.0023, the cold start 1.196 with 0.0353. Interpretation: the warm start is worth about 11% of mean index at equal evaluation budget, and rather more importantly it shrinks the run-to-run spread by an order of magnitude. A single cold run is not reportable: three cold runs span 0.07 index units against 0.004 for three warm ones – a spread that is 52% of the very difference being measured.

What the warm start actually contains at this scale is worth looking at, because it is weaker than the name suggests:

```
fit <- make_fitness(ctx, n_sel, alpha_max, null$gd_ref)
seed_w <- c(0, 0.25, 0.5, 1, 2, 4, 8, 16)        # the grid sel_de() hard-codes
seeds <- lapply(seed_w, function(w) sel_greedy(ctx, n_sel, w = w))
fmt(data.frame(
  w         = seed_w,
  index     = vapply(seeds, mean_index, numeric(1), ctx = ctx),
  alpha_pct = 100 * vapply(seeds, alpha_loss, numeric(1), ctx = ctx,
                           gd_ref = null$gd_ref),
  fitness   = vapply(seeds, function(s) fit(s + 0.5), numeric(1))), 4)
```

w	index	alpha_pct	fitness
0.00	0.1343	−1.3520	−0.1343
0.25	1.9919	4.1216	29.2242
0.50	1.9919	4.1216	29.2242
1.00	1.9919	4.1216	29.2242
2.00	1.9919	4.1216	29.2242
4.00	1.9919	4.1216	29.2242
8.00	1.9919	4.1216	29.2242
16.00	1.9919	4.1216	29.2242

Seven of the eight seeds are the *same* selection – truncation – because the hard-coded grid lies entirely above the crossover measured earlier, and at a 1% budget all seven are infeasible, which the penalty

converts into a fitness of about 29 on a scale where a random row scores near zero. The one feasible seed is the $w = 0$ diversity solution at index 0.134. So at this scale the warm start installs one feasible plan and seven copies of an infeasible one, and is still worth the difference measured above. Widening that grid is a change to the package rather than to the call, and it is not made here: every number in this book was produced with `sel_de()` as it stands.

The mechanism is dimensional. At roughly 60 integer dimensions the search space is far too large for 30,000 evaluations to find anything a construction heuristic has not already found. The warm start does not help the DE converge faster – it gives the DE something worth improving on.

`sel_de()` builds the warm start automatically from a greedy sweep, and **removing it will silently degrade every result in this book.**

The limitation, stated where the result is: at book scale the warm-started DE (1.33) is still below the best feasible greedy solution of the finely swept grid (1.378) at this budget. The DE's advantage here is not raw index but that it is parameterised by $\alpha_{\max}$, so it can be asked for a budget the greedy grid does not happen to land on. The method that beats both on index, at a fraction of the wall-clock, is in Chapter 20..

19.4.2 Checking convergence against the bound

Section 18.2.1 gives a relaxation. Turning it into a *bound* takes one more step, derived in Section 20.4: the relaxation at a given λ maximises a penalised objective rather than the index, so the best relaxation point inside the budget is not an upper bound on the best plan inside the budget. The Lagrangian combination is.

```
relax <- ocs_relaxation(ctx, n_sel, lambdas = 10^seq(-1, 2.5, length.out = 10))
a_relax <- (null$gd_ref - relax[, "GD"]) / null$gd_ref

bound <- ocs_bound(ctx, n_sel, alpha_max, null$gd_ref)

c(DE_index = mean_index(de_idx, ctx),
  upper_bound = as.numeric(bound),
  gap_pct = 100 * (as.numeric(bound) - mean_index(de_idx, ctx)) / as.numeric(bound))
#>    DE_index upper_bound     gap_pct
#>    1.329921    1.553244   14.377862
```

The four steps behind that one call, and the check that has to accompany it:

1. The budget is translated into group coancestry, $\theta_{\max} = 1 - \text{gd_ref}(1 - \alpha_{\max})$ – the largest θ a plan may have.
2. The relaxation is solved at each λ on the grid, giving $V(\lambda)$, its optimal penalised value.
3. The bound is $\min_\lambda [V(\lambda) + \lambda\,\theta_{\max}]$. Every λ gives a valid bound, so a coarse grid costs tightness, not validity.
4. `ocs_bound()` doubles its iteration count until the value stops moving and records whether it succeeded. **Read that flag before quoting any gap**: an under-converged relaxation reports $V(\lambda)$ too small, which makes the "bound" too small, and then it is not a bound at all (Section 20.7).

```
c(lambda_star = attr(bound, "lambda"),
  iterations  = attr(bound, "iters"),
  theta_max   = 1 - null$gd_ref * (1 - alpha_max))
#>  lambda_star   iterations    theta_max
#>   68.1292069 8000.0000000    0.6774198
attr(bound, "converged")
#> [1] TRUE
```

A large gap means **the optimiser has not converged**, not that the problem is intrinsically hard. A small gap means further optimiser effort is wasted. The gap here is 14.4%, against a bound the function certifies as converged; Chapter 20. closes most of it at the same budget with a cheaper move, which settles which of the two readings applies.

If the DE ever looks unconvincing on real data

H. De Beukelaer, G. F. Davenport, and V. Fack [138], optimising core-collection subsets with local search rather than DE, rank three stochastic searches on the same objective across four crop datasets – rice, coconut, maize and pea – in the Results section "Optimizing E-NE and A-NE with local searches": parallel tempering first, a genetic algorithm second, and random descent – a plain stochastic hill-climber – third, with all twelve pairwise comparisons significant under a Holm-adjusted Wilcoxon test. The genetic algorithm therefore *does* beat the hill-climber on raw score. What it does not do is win, and the method that wins is the one assembled from many simple local searches run in parallel, not the one with the more elaborate update rule.

The inference, stated separately from that result and limited by it: extra sophistication inside a *single-solution* algorithm bought a rank there and not the win, which is a weak reason to spend complexity on the optimiser and no reason at all to expect a DE to be the right one. It is one application, on an allele-coverage objective rather than an index under a coancestry budget, so it transfers as a prior about where effort pays and not as a result. That direction is taken up in Chapter 20., which builds the local search on the `O(L)` swap this package already had and measures it against the DE at equal wall-clock.

19.5 The full frontier

One optimisation per budget, which is the epsilon-constraint method: each point is a plan that can be put in front of a committee, described by the budget it respects rather than by an exchange rate nobody can interpret.

```
alphas <- c(0, 0.0025, 0.005, 0.01, 0.02)
set.seed(45)
front <- do.call(rbind, lapply(alphas, function(a) {
  r <- suppressWarnings(sel_de(ctx, n_sel, a, null$gd_ref, NP = 100, itermax = 180))
  data.frame(alpha_max = a, alpha = alpha_loss(r, ctx, null$gd_ref),
             index = mean_index(r, ctx), Ne_par = ne_parents(r, ctx))
}))

plot(front$alpha * 100, front$index, type = "b", pch = 19, lwd = 2, col = "#b2182b",
     xlab = "gene diversity loss, alpha (%)", ylab = "mean index (s.d.)",
     xlim = range(c(front$alpha, gf$alpha, a_relax) * 100),
     ylim = range(c(front$index, gf$index, relax[, "index"])))
lines(gf$alpha * 100, gf$index, type = "b", pch = 17, col = "#1f5f8b", cex = 0.7)
lines(sort(a_relax) * 100, relax[order(a_relax), "index"], type = "b", pch = 1,
      lty = 2, col = "grey55", cex = 0.7)
abline(v = 0, lty = 3)
legend("bottomright", c("constrained DE", "greedy sweep", "continuous OCS
relaxation"),
       col = c("#b2182b", "#1f5f8b", "grey55"), pch = c(19, 17, 1),
       lty = c(1, 1, 2), lwd = c(2, 1, 1), bty = "n", cex = 0.8)
grid()
```

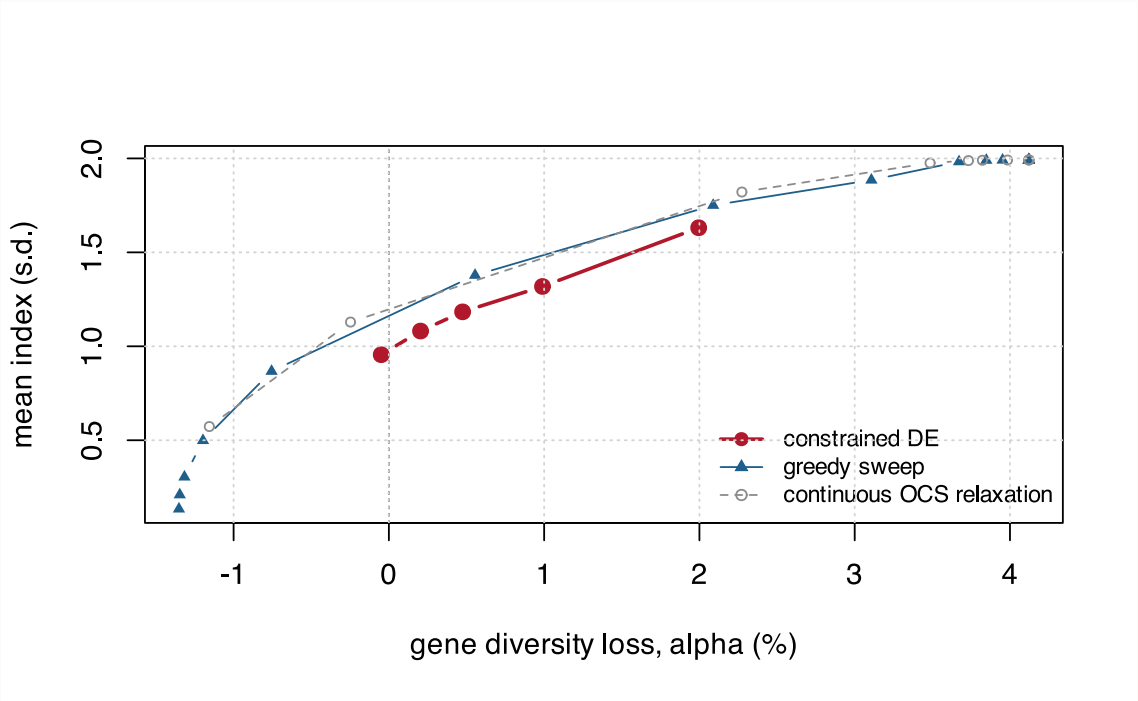

Figure 19.3: Everything together. The constrained DE, the greedy sweep, and the continuous OCS relaxation – which is a relaxation, not a bound at matched diversity (Section 20.4).

```
fmt(front, 4)
```

alpha_max	alpha	index	Ne_par
0.0000	−0.0005	0.9553	29.8755
0.0025	0.0020	1.0812	28.2353
0.0050	0.0047	1.1833	25.7143
0.0100	0.0099	1.3184	22.9299
0.0200	0.0200	1.6310	18.2741

```
best_greedy <- vapply(front$alpha_max, function(a) {
  ok <- gf$alpha <= a
  if (any(ok)) max(gf$index[ok]) else NA_real_
}, numeric(1))
fmt(data.frame(alpha_max = front$alpha_max, DE = front$index,
               greedy_sweep = best_greedy,
               DE_minus_greedy = front$index - best_greedy), 4)
```

alpha_max	DE	greedy_sweep	DE_minus_greedy
0.0000	0.9553	0.8672	0.0881
0.0025	1.0812	0.8672	0.2140
0.0050	1.1833	0.8672	0.3162
0.0100	1.3184	1.3782	−0.0598
0.0200	1.6310	1.3782	0.2528

Three things to read, in this order.

The constraint does its job. Realised `alpha` tracks `alpha_max` at every budget: the optimiser spends the diversity it was allowed and no more. A plan that lands far inside its budget has not been constrained by it, and whatever else is limiting it – the cap, the penalty scale, an unconverged search – has to be identified before the frontier is read as a price list.

The comparison against greedy is budget-dependent. The DE is ahead at 4 of the 5 budgets and behind at the rest. Where it is ahead, the greedy grid simply has no point near that budget – between two consecutive values of w the sweep steps over the whole interval, and the best *feasible* greedy plan is then one from far inside it. Where the grid does land well, greedy is at least as good for a fraction of the cost. Interpretation: at this scale the DE buys the ability to hit a requested budget, not a better plan; the statement that it dominates greedy on index was an artefact of a coarse w grid, and does not survive measurement.

The dashed curve is a relaxation, not a ceiling. It sits above the discrete points over most of the range, which is exactly what makes it tempting to read its height at a given α as an upper bound on what a plan there could be worth. It is not one: the best relaxation point whose own α falls inside the 1% budget reaches 1.129, *below* both the DE plan and the best feasible greedy plan at that budget, because each point maximises a penalised objective rather than the index at its own diversity. Section 20.4 falsifies that construction and gives the Lagrangian statement that is a ceiling – 1.553 here – and the vertical distance to *that* at $\alpha_{\max}$ is what Chapter 20. spends its effort on. Note that this figure runs one DE per budget with a fixed seed; the seed-to-seed spread measured above (0.002 for the warm start) is small against the steps between budgets here, but it is not zero and it should be quoted whenever two methods are compared at one budget.

19.6 Which strategy to use

Situation	Use
The constraint is inactive (ceiling below your limit)	truncation – nothing to optimise
A per-line cap already meets the constraint	`sel_truncation_cap()` – no matrix needed
One-off analysis, small pool	`sel_greedy()` swept over w
Production scale, a budget and a per-line cap	`sel_greedy(max_use = cap)` swept over w – the highest index of any *capped* route at both scales of Chapter 20.'s benchmark (1.112 at book scale, 1.123 at production), and cheap at both: at book scale it and `ocs_round(max_use = cap)` both cost milliseconds (0.008 s and 0.01 s, means of 3 seeds that single-call timing cannot separate – neither is reliably the cheaper), and at production it is second cheapest (0.329 s, behind `ocs_round(max_use = cap)`'s 0.214 s in every seed). `sel_local_search(max_use = cap, mode = "tempering")` – the only capped local-search mode in that benchmark – comes close without matching it: 2.5 thousandths of an index unit short at book scale, for roughly 36x the wall-clock, and 0.2 thousandths short at production
A restriction a swap can-	`sel_de_fast()` – the only route that accepts them; it trails the capped greedy on index (Chapter 20.)

Situation	Use
not test (Ne floor, pool balance)	

As an order of operations at the end of a cycle:

1. Build the contract objects and print the instance panel at the top of this chapter. Confirm `gd_ref` was computed at the `n_sel` you will actually use.
2. Compute the attainable ceiling from truncation. If it is below your intended $\alpha_{\max}$, stop: the constraint is inactive and truncation is the answer.
3. Run `sel_truncation_cap()`. It costs nothing and needs no matrix. If it meets the budget at an acceptable index, stop.
4. Sweep `sel_greedy()` over a w grid spanning the crossover measured from your own data, not a grid of round numbers. Read the frontier and locate the knee.
5. Only if a specific budget has to be hit, or several restrictions apply at once, run an optimiser – and warm-start it.
6. Whatever returned the plan, validate it on the second lens and on allelic richness before it leaves the analysis (Chapter 24.).

The honest summary is that an optimiser earns its place only in the last row, where a restriction rules the cheap routes out. In the first four – including the capped production case, once the capped benchmark is read on capped methods only – something simpler is either exactly as good or good enough, and Chapter 15.'s cost table is the reason to check which row you are in before writing any optimisation code.

Reproduce this chapter

All the code of this chapter, in order and with the explanation kept as comments, is one script: **10-combinatorial-selection.R**. Download it, install `hybdiv`, and run it top to bottom in an empty directory; cached results are fetched on first use.

20. Advanced selection: structure, bounds and scale

Chapter 19. stops at a warm-started differential evolution. That is a reasonable place to stop if the problem is a black box. It is not one. This chapter names the structure, and then spends it: on cheaper moves, on genuine approximation guarantees, on a bound that is actually a bound, and on an oracle that says how far from optimal the heuristics really are.

The target throughout is the scale the method has to survive: **200 hybrids chosen from 10,000**, a full factorial of 100 by 100 lines. That is an ordinary size for a hybrid maize programme at the end of a cycle: two heterotic pools of a hundred inbred or doubled-haploid lines each, every cross between them a legitimate candidate, and a few hundred of them going into the next round of multi-location testing.

That is a different – and deliberately larger – "production scale" from the one Chapter 19. quotes. There, production means the real dataset this package was written for, 71 + 71 lines and $N = 5041$ candidates, which is the size the encoding argument is made at. Here it is the 100 + 100 factorial the scaling study and the head-to-head benchmark are run at, because a cost curve needs a span of N wider than one point provides. Neither is wrong and they are not interchangeable: every per-evaluation cost and every memory figure in this chapter is at $N = 10,000$, and the dimension counts in Chapter 19. are at $N = 5041$.

What has to be in hand, beyond Chapter 19.'s three objects, is the line-level route: `fast_ctx()` adds `ctx$parents`, the $N \times 2$ matrix of parent indices, to a context that already carries `ctx$fL`, the $L \times L$ coancestry *between lines*. Everything in this chapter that scales is a function of those two and of nothing else.

```
c(candidates_N = ctx$N,
  lines_L      = ctx$n_lines,
  hybrids_kept = n_sel,
  budget_pct   = 100 * am,
  gd_ref       = gd,
  f_MiB        = as.numeric(object.size(ctx[["f"]])) / 2^20,
  fL_MiB       = as.numeric(object.size(ctx$fL)) / 2^20)
#> candidates_N      lines_L hybrids_kept   budget_pct       gd_ref        f_MiB
#> 625.00000000  50.00000000  60.00000000   1.00000000   0.32583857   2.98043823
#>       fL_MiB
#>   0.01927948
```

The assumptions are Chapter 19.'s, unchanged and worth restating because two of them are load-bearing here: parent lines are homozygous, which is what makes f_{ij} a function of the four parental coancestries and hence collapses the problem to L dimensions (Section 15.1); and `f` is molecular coancestry in $[0, 1]$, not a centred relationship matrix, which is what makes it a Gram matrix with the properties the next section needs.

20.1 Three facts about this problem

Everything below follows from three properties of `f`, each of which takes three lines to check and none of which has been stated in this book so far.

20.1.1 `f` is a Gram matrix

Molecular coancestry is $f_{ij} = \frac{1}{m}\sum_k [x_{ik}x_{jk} + (1-x_{ik})(1-x_{jk})]$, which is an inner product of the stacked vectors $[\mathbf{x}_i; \mathbf{1} - \mathbf{x}_i]$. So `f` is not merely symmetric, it is a **Gram matrix**, exactly:

```
V  <- cbind(ctx[["X"]], 1 - ctx[["X"]])
ev <- eigen(ctx[["f"]], symmetric = TRUE, only.values = TRUE)$values
```

```
c(max_abs_difference = max(abs(ctx[["f"]] - tcrossprod(V) / ctx$m)),
  min_eigenvalue     = min(ev),
  min_over_max_eig   = min(ev) / max(ev))
#> max_abs_difference     min_eigenvalue   min_over_max_eig
#>       0.000000e+00      -4.275667e-13      -1.018142e-15
```

The check is the whole argument: the difference is exactly zero, and the smallest eigenvalue is zero to the eigensolver's own rounding – its sign is arbitrary 15 orders of magnitude below the largest eigenvalue. A negative eigenvalue large enough to see on that scale would mean `f` had been centred, rescaled, or built on the wrong marker coding – and every method in this chapter would silently lose its justification.

Two consequences carry the rest of the chapter. First, $\mathbf{c}'\mathbf{f}\mathbf{c}$ is a **convex** quadratic, so the diversity budget is a convex constraint – which is what makes the exact method of Section 20.6 possible at all. Second, $d_{ij} = f_{ii} + f_{jj} - 2f_{ij}$ is a squared Euclidean distance, hence a distance of **negative type**, which is the precise hypothesis the strongest approximation results require.

20.1.2 Maximising diversity is minimising the submatrix sum

$\theta_S = \frac{1}{n^2}\sum_{i,j\in S} f_{ij}$, and at fixed n the normaliser is a constant:

```
s <- sel_greedy(ctx, n_sel, w = 1)
c(theta_times_n2 = theta_group(s, ctx) * n_sel^2,
  submatrix_sum  = sum(ctx[["f"]][s, s]),
  difference     = theta_group(s, ctx) * n_sel^2 - sum(ctx[["f"]][s, s]))
#> theta_times_n2  submatrix_sum     difference
#>   2.475329e+03   2.475328e+03   1.818989e-12
```

So the objective is $\lambda\sum_{i\in S} u_i - \sum_{i,j\in S} f_{ij}$: a **modular** term plus a **submodular** one, since the marginal cost of adding a hybrid grows with the set. Writing the same thing through d_{ij}, $\sum_{i<j} d_{ij} = n\,\mathrm{tr}(f_S) - n^2\theta_S$, identifies it as the **max-sum diversification** problem of A. Borodin, A. Jain, H. C. Lee, and Y. Ye [147]. I have not found it named that way in the plant-breeding literature, and this book makes no claim about what a systematic screen would find – no such screen was run. The structural identity is the load-bearing part, and it holds whether or not anyone has written it down before.

The practical content of "fixed n" is worth spelling out: it is *only* at a fixed set size that maximising gene diversity and minimising the submatrix sum are the same instruction. Comparing θ across selections of different sizes compares two different things, which is the same reason `gd_ref` has to be resampled at the size actually being used (Section 14.9).

20.1.3 The per-line cap is a partition matroid

"No line may parent more than `max_use` of the selected hybrids" is one partition matroid per heterotic pool. Together with the cardinality constraint that is an intersection of matroids – not an arbitrary combinatorial restriction, but the single best-understood constraint class in discrete optimisation.

The structure comes from the mating design, not from the mathematics. Every candidate is a cross between one pool-A line and one pool-B line, so each selected hybrid contributes exactly one use to a pool-A line and one to a pool-B line, and the counts within each pool sum to n by construction:

```
cnt <- line_counts(s, ctx)
c(uses_pool_A     = sum(cnt[ctx$pool == "A"]),
  uses_pool_B     = sum(cnt[ctx$pool == "B"]),
  n_sel           = n_sel,
  distinct_A_used = sum(cnt[ctx$pool == "A"] > 0),
  distinct_B_used = sum(cnt[ctx$pool == "B"] > 0))
#>     uses_pool_A     uses_pool_B           n_sel distinct_A_used distinct_B_used
#>              60              60              60              10              23
```

That is exactly the ground-set partition a partition matroid needs. It also gives the feasibility condition for any cap c directly: a plan of n crosses exists under the cap only if $n \le L_A c$ and $n \le L_B c$. In a programme the cap is rarely chosen for theoretical reasons – it is set by how many seed-production fields a female line can support – and the point of naming the structure is that a constraint arriving from the field turns out to be the one class of constraint discrete optimisation handles best.

i Why naming the structure is worth a section

Three named properties buy three concrete things, and it is worth being blunt about which is which.

Property	What it buys	Worth in practice
`f` positive semi-definite	a convex constraint, hence valid gradient cuts and a valid Lagrangian bound	**large** – Section 20.4 is a correction, not an ornament
submodular plus modular objective	$1 - 1/e$ under cardinality [148], $1/(p+1)$ under p matroids [149]; a PTAS for negative-type distances under a matroid [150]	**small** – worst-case bounds, and the measured gap is around 1%
line collapse $\theta = \mathbf{w}'\mathbf{f}^L\mathbf{w}$	memory $O(N^2) \to O(L^2)$, an $O(L)$ swap, and an L-dimensional contributions programme in place of an N-dimensional one	**large** – 0.745 GiB against 0.3 MiB, and the swap is what makes local search affordable

20.2 What lazy greedy would not buy

The obvious first move on a submodular objective is the accelerated, or "lazy", greedy of M. Minoux [151]: keep stale marginal gains in a priority queue and re-evaluate only the top one, which is valid precisely because submodularity makes a stale gain an upper bound on the current one. Two distinct results are easy to conflate here, so name them apart: the acceleration is Minoux's, while the worst-case guarantee for the plain greedy it accelerates is G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher [148]'s and is quoted separately below. It is the standard accelerator for this objective class, and here it is **worthless**.

`sel_greedy()` already carries the running vector `s` of coancestry sums against the chosen set, so every candidate's marginal gain is $O(1)$ and the whole step is one vectorised pass. Lazy evaluation optimises precisely the cost that is already free. Reaching for it would add a heap, add bugs, and save nothing.

The guarantee is worth stating even so, because it settles a question rather than improving a number: greedy under our constraints cannot do worse than $1/(p+1)$ of optimal, with p the number of matroids intersected – here the cardinality constraint plus one partition matroid per pool.

State it with its hypotheses, though, because they are not all met. The $1 - 1/e$ and $1/(p+1)$ results of G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher [148] and M. L. Fisher, G. L. Nemhauser, and L. A. Wolsey [149] are proved for **monotone, non-negative** submodular functions. Our objective is neither in general: the index is standardised to mean zero over the candidate pool, so $\bar{u}_S$ can be negative, and adding a hybrid increases the coancestry term, so the objective is monotone only where the merit term dominates. The guarantee therefore belongs in this chapter as a statement about the *constraint class* – the cap is not an arbitrary restriction – and not as a certificate for any particular run. The certificate this chapter actually uses is the Lagrangian bound of Section 20.4, and Section 20.6 measures what greedy does in fact achieve.

20.3 The move that was already paid for

Section 15.1 established $\theta_S = \mathbf{w}'\mathbf{f}^L\mathbf{w}$, and `theta_state()`/`theta_swap()` have been in the package ever since, maintaining group coancestry in $O(L)$ per swap. Nothing consumed them. Chapter 19. even ends with a callout recommending local search, citing H. De Beukelaer, G. F. Davenport, and V. Fack [138] – and then does not run one.

```
theta_swap
#> function (st, out_h, in_h, ctx)
#> {
#>     cnt <- st$cnt
#>     v <- st$v
#>     po <- ctx$parents[out_h, ]
#>     pi_ <- ctx$parents[in_h, ]
#>     for (a in po) {
```

```
#>         cnt[a] <- cnt[a] - 1L
#>         v <- v - ctx$fL[, a]
#>     }
#>     for (a in pi_) {
#>         cnt[a] <- cnt[a] + 1L
#>         v <- v + ctx$fL[, a]
#>     }
#>     s <- st$s
#>     list(cnt = cnt, v = v, s = s, theta = sum(cnt * v)/(s * s))
#> }
#> <bytecode: 0x7b852cd0e0>
#> <environment: namespace:hybdiv>
```

Four rank-one column updates. No `N x N` matrix is touched, and the cost depends on neither N nor m. The state it maintains is three objects: the line-count vector `cnt`, the product `v = fL %*% cnt`, and their inner product divided by s^2, which is θ. Removing a hybrid subtracts the two columns of `fL` belonging to its parents; adding one adds the two columns of the entrant's parents. Because a swap keeps the set size fixed, the normaliser $s = 2n$ never changes.

It is an exact update, not an approximation, and that is checkable in three lines:

```
st    <- theta_state(s, ctx)
out_h <- s[1]
in_h  <- setdiff(seq_len(ctx$N), s)[1]
st2   <- theta_swap(st, out_h, in_h, ctx)
after <- c(setdiff(s, out_h), in_h)
stopifnot(isTRUE(all.equal(st2$theta, theta_group(after, ctx))))
c(incremental = st2$theta, recomputed = theta_group(after, ctx))
#> incremental  recomputed
#>   0.6863819   0.6863819
```

If that identity ever fails, the state and the selection have drifted apart – the usual cause being a move that was accepted in the selection vector but not applied to the state, or the reverse.

That last property is the one that matters, and it is worth being precise about where the saving actually comes from, because the obvious answer is wrong. The line *encoding* does not make an evaluation cheaper. Taking two rows of Section 20.7's table:

route	at $N = 3600$	at $N = 10,000$
hybrid encoding, `mean(f[idx, idx])`	40 us	132 us
line encoding, full fitness	83 us	169 us
of which `decode_lines()`, which sorts all N candidates	77 us	159 us
incremental swap, no decode at all	**3.3 us**	**4.2 us**

The line encoding is *slower* per evaluation, because decoding a weight vector into a selection has to rank every candidate. What it buys is a smaller search space, not a cheaper objective. The order-of-magnitude saving belongs to the swap – and the swap is available only to a method that moves in the space of selections rather than decoding one from scratch each time.

```
args(sel_local_search)
#> function (ctx, n_sel, alpha_max, gd_ref, start = NULL, max_use = NULL,
#>     mode = c("hill", "anneal", "tempering"), iters = 20000, n_chains = 8,
#>     penalty = 5, t0 = NULL, swap_every = 50L, seed = NULL, trace = FALSE)
#> NULL
```

Three modes, one driver, differing only in the temperature schedule. The driver does the following, and each step is a decision a breeder can inspect:

1. **Start from a plan, never from noise.** `start = NULL` builds a warm start: capped truncation, plus the greedy sweep whenever an $N \times N$ `f` is present. The test is literally `nrow(ctx[["f"]]) == ctx$N`, and it is worth being accurate about which branch produced the numbers in this chapter: the production benchmark of Section 20.7 deliberately forms the 0.745 GiB matrix so that the routes needing it can be timed at all, so **its rows took the greedy branch**. Among the candidates it takes the highest-index

one that *meets the budget*, and if none does, the one with the smallest loss. As in `sel_de()`, `NULL` never means "start cold".

The greedy branch is guarded, but the ranking that follows it is not: the warm start scores every candidate with `alpha_loss()`, which reads `ctx$f`. So `start = NULL` needs the $N \times N$ matrix whichever branch it takes, and on a context built without one it fails – `subscript out of bounds`, measured in Section 20.7.1. Supplying an explicit `start =` skips the whole function, and from there the search is `fL`-only; that is the call to make if `f` does not fit in memory, and Section 20.7.1 is where the two are run side by side.

2. **Build the state.** `theta_state()` on the starting selection, plus the running index sum and a logical membership vector, so that every later question is $O(1)$ or $O(L)$.
3. **Calibrate the temperature.** With `t0 = NULL` and a stochastic mode, the driver proposes up to 200 random swaps, records the median absolute change in the objective, and uses that as the starting temperature. Nothing about the scale of the index is assumed, so the same call transfers between datasets. `mode = "hill"` skips this entirely, since it never accepts a worsening move.
4. **Propose a feasible move.** Pick one of the n selected crosses at random to leave; draw entrants at random until one is found whose parents stay under `max_use` once the leaving cross is removed, giving up after 20 attempts. The cap is tested, never punished.
5. **Score it in $O(L)$.** `theta_swap()` for the new θ, one addition and one subtraction for the new index sum, and the objective is $\bar{u}_S$ minus `penalty` times the budget violation *expressed in units of the budget itself* – the violation is divided by `alpha_max`, so a 5 becomes an effective slope of 500 index standard deviations per unit of relative loss at a 1% budget, and 5000 at 0.1%. The tighter the budget, the harder a violation is punished, which is the behaviour you want and is not what `make_fitness()`'s fixed multiplier of 1000 on the *absolute* violation does.
6. **Accept or reject.** Improving moves are always accepted; worsening ones with probability $\exp(\Delta/t_k)$ at the current temperature. The best-ever selection is tracked separately from the current one and is what is returned – so a chain that wanders away at high temperature loses nothing.

Two design points are worth stopping on.

The cap is tested, not punished. Whether a swap keeps every line under `max_use` is an $O(1)$ question given the count vector, so infeasible moves are never proposed. Penalising them instead spends the evaluation budget on being rejected – with a tight cap that was the difference between the search improving on its warm start and not moving at all.

The temperature calibrates itself. A constant would not transfer between datasets, so `t0 = NULL` samples the median absolute objective change over a short burn-in and uses that.

One run of each is not a comparison. Three seeds – 1, 2 and 3, the same three the cached benchmark two subsections below uses, so the two panels are read against the same draws:

```
modes <- c("hill", "anneal", "tempering")
ls_raw <- do.call(rbind, lapply(1:3, function(sd_) {
  do.call(rbind, lapply(modes, function(m) {
    # sel_local_search() sets the seed itself, immediately before its own draws.
    s <- sel_local_search(ctx, n_sel, am, gd, mode = m, iters = 20000, seed = sd_)
    data.frame(seed = sd_, mode = m, index = mean_index(s, ctx),
               alpha_pct = 100 * alpha_loss(s, ctx, gd),
               Ne_par = ne_parents(s, ctx), max_line = max_line_use(s, ctx))
  }))
}))
ls_res <- do.call(rbind, lapply(modes, function(m) {
  d <- ls_raw[ls_raw[["mode"]] == m, ]
  data.frame(mode = m, index = mean(d[["index"]]), index_sd = sd(d[["index"]]),
             alpha_pct = mean(d[["alpha_pct"]]), Ne_par = mean(d[["Ne_par"]]),
             max_line = max(d[["max_line"]]))
}))
# The same three modes and the same three seeds in the cached benchmark below,
# which runs 30,000 moves instead of 20,000.
bmode <- vapply(modes, function(m)
  mean(bench$index[bench$scale == "book" & bench$method == m]), numeric(1))
fmt(ls_res, 3)
```

mode	index	index_sd	alpha_pct	Ne_par	max_line
hill	1.526	0.002	0.999	20.553	13
anneal	1.520	0.002	0.996	20.514	14
tempering	1.493	0.004	0.993	20.854	13

```
fmt(rbind(here_20000 = setNames(ls_res[["index"]], modes),
          benchmark_30000 = bmode), 4)
```

	hill	anneal	tempering
here_20000	1.5264	1.5202	1.4933
benchmark_30000	1.5274	1.5276	1.5132

Note that `iters` is the total number of proposed moves *summed over chains*, so the three modes cost about the same wall-clock at equal `iters` and the comparison above is a fair one. In tempering mode those 20,000 moves are split across `n_chains` replicas on a geometric temperature ladder spanning three decades, exchanging adjacent rungs every `swap_every` moves.

The realised `alpha` sits against the 1% budget in every mode: the constraint is doing its job, and the search is spending the diversity it was allowed.

Three checks before a plan from this function is used, in this order: realised α against the budget (a plan far inside it means the search never reached the boundary, or the penalty was too steep); `max_line` against the cap you set; and $N_{e,\mathrm{par}}$, which is free and catches the case where θ is satisfied by a handful of lines used many times each.

Result. Over the three seeds the three schedules span 2.2% of index, against a within-mode seed-to-seed standard deviation of at most 0.24% – so the ordering at this iteration count, `"hill"` and `"anneal"` within 0.4% of each other and `"tempering"` behind both, is not a seed artefact.

Interpretation, and what limits it. Splitting a fixed move budget across tempering's replicas costs more than the exchanges return at this size – a statement about `iters = 20000` on 625 candidates, not about parallel tempering. The hill-versus-anneal ordering in particular does **not** survive a change in budget, which is why the second panel prints both: at 20,000 moves `"hill"` leads `"anneal"` by 6.2 thousandths of an index unit – more than the 3.7-thousandth within-mode standard deviation, so not noise at that budget – while at the benchmark's 30,000 moves the sign flips to 0.2 thousandths in `"anneal"`'s favour. That ordering is therefore a property of the move budget and not of the schedule. Read this panel as evidence that the temperature schedule is a second-order choice at fixed move budget – not as a ranking of the three, and not as a claim that the move rather than the schedule is what buys the index. That claim is a comparison against the other methods and it belongs to the benchmark below, which has the dispersion to support it.

20.3.1 Against the differential evolution, at equal wall-clock

Equal *evaluations* is the wrong comparison when one evaluation costs seventeen times what another does. The honest axis is seconds.

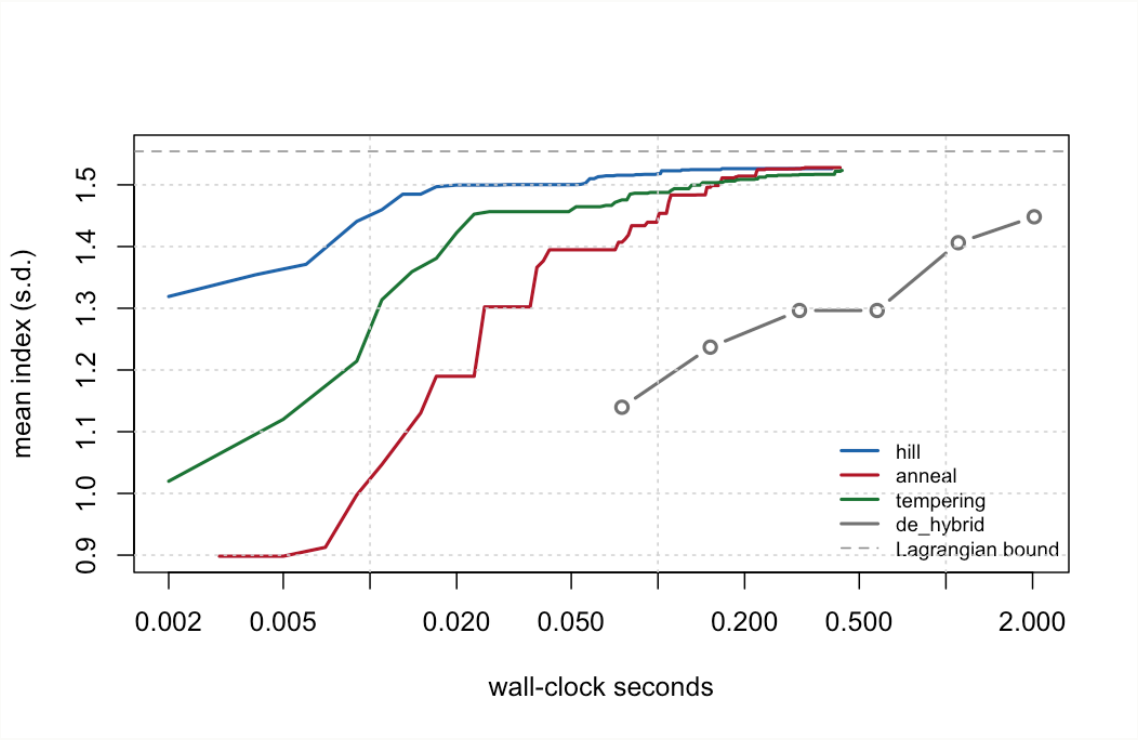

Figure 20.1: Best index reached against wall-clock seconds, at book scale under a 1% diversity budget. The dashed line is the Lagrangian bound of Section 20.4. Local search is at 1.52 within a tenth of a second; the differential evolution reaches only 1.45 after two seconds.

method	index	loss %	Ne parents	max uses	seconds	gap to bound %
anneal	1.528	0.999	20.594	13	0.232	1.718
hill	1.527	0.998	20.552	13	0.225	1.733
tempering	1.513	0.998	20.750	13	0.222	2.645
ocs_round	1.487	0.895	20.749	14	0.009	4.339
greedy	1.378	0.550	21.818	13	0.008	11.331
de_hybrid	1.360	0.960	22.314	14	0.846	12.517
greedy_cap	1.112	0.968	25.000	5	0.008	28.445
de_hybrid_cap	1.110	0.976	25.263	5	0.961	28.604
tempering_cap	1.110	0.976	25.263	5	0.286	28.604
de_lines_cap	1.106	0.881	25.714	5	1.563	28.870
ocs_round_cap	1.106	0.881	25.714	5	0.010	28.870
de_lines	1.059	0.996	16.314	11	0.800	31.843

Read that table by columns rather than by rank. `index` orders the methods; `loss %` says which of them actually spent the budget; `max uses` says which were capped, since the capped variants are solving a strictly harder problem and must not be compared with the uncapped ones on index alone; `seconds` is the axis the ranking is conditional on; and `gap to bound %` is the only column that says whether the leader is near optimal or merely ahead of the others. The greedy row is the one to keep in view: swept finely over w it reaches index 1.378 in 8 ms, which is 9.8% below the leading method at about 29x less wall-clock. Whether that shortfall is worth 0.23 seconds is a question about the programme, not about the algorithm – at production scale the same ratio is what decides whether a frontier of ten budgets is an afternoon or a minute.

This table is also where the move-versus-schedule question can actually be settled, because it has replication on both sides of it and both scales:

```
mv <- do.call(rbind, lapply(c("book", "production"), function(sc) {
  d   <- bench[bench$scale == sc, ]
```

```
  m   <- aggregate(index ~ method, d, mean)
  ls_ <- m[["index"]][m[["method"]] %in% modes]
  de_ <- m[["index"]][m[["method"]] %in% c("de_hybrid", "de_lines")]
  sds <- aggregate(index ~ method, d[d$method %in% modes, ], sd)
  data.frame(scale = sc, schedule_spread = diff(range(ls_)),
             best_swap_minus_best_de = max(ls_) - max(de_),
             ratio = (max(ls_) - max(de_)) / diff(range(ls_)),
             max_seed_sd = max(sds[["index"]]),
             hill_minus_anneal =
               mean(d[["index"]][d[["method"]] == "hill"]) -
               mean(d[["index"]][d[["method"]] == "anneal"]))
}))
fmt(mv, 4)
```

scale	schedule_spread	best_swap_minus_best_de	ratio	max_seed_sd	hill_minus_anneal
book	0.0144	0.1679	11.6579	0.0040	−0.0002
production	0.1267	0.3701	2.9217	0.0059	0.0024

Result. The three temperature schedules share one move and span 0.014 index units at book scale and 0.127 at production, with a within-mode seed-to-seed standard deviation of at most 0.0059. The best of them sits 0.168 units above the better of the two differential evolutions at book scale and 0.37 at production – 11.7x and 2.9x the spread among the schedules respectively, and in a fraction of the wall-clock in both cases – the `seconds` column above at book scale, and the production panel of Section 20.7 at the other.

Interpretation. What moves the index at both scales measured is the neighbourhood the search moves in – the $O(L)$ swap, available only to a method that does not decode a selection from scratch – rather than the schedule wrapped around it. Two things limit that: the ratio is 4 times smaller at production than at book scale, so it narrows with N rather than holding constant; and it rests on three seeds per cell, where the DE arms carry the larger dispersion. It is a comparison of two families at two sizes, not a law.

What the literature says about picking a metaheuristic

H. De Beukelaer, G. F. Davenport, and V. Fack [138], on core-collection subsets, found a plain stochastic hill-climber already competitive with a considerably more complex genetic algorithm, and parallel tempering winning by combining many simple local searches rather than by being individually cleverer.

S. Yadav and others [152] compared GA, DE, PSO and SA head to head on parent selection in wheat and ranked them by convergence AUC at 0.879, 0.860, 0.820 and 0.769, with DE running two to three times slower than GA for a marginally worse result. The current tools agree: TrainSel [119], which drives MateR [153], is a genetic algorithm hybridised with simulated annealing.

So the differential evolution of Chapter 19. is not the obvious choice, and never was. What it had going for it was the warm start. What it lacked was a cheap move.

20.3.2 The encoding, wired up at last

Chapter 19. measures the line encoding against a fully N-dimensional one – one random key per hybrid – and finds a 5.7x margin in the line encoding's favour at equal budget. That is a verdict on N-dimensional encodings, not a verdict on `sel_de()`, whose search space is n_{sel} coordinates wide; the head-to-head between the two encodings the package actually ships is in this chapter's own benchmark below, and it goes the other way. What is true regardless is that `decode_lines()` and `make_fitness_fast()` have been exported since the line route was written with nothing driving them. `sel_de_fast()` closes that:

```
sel_de_fast
#> function (ctx, n_sel, gd_ref, alpha_max, ne_par_min = NULL, max_use = NULL,
#>     pool_tol = NULL, NP = 300, itermax = 2000, seeds = NULL,
#>     trace = FALSE)
#> {
#>     if (is.null(ctx$parents))
#>         ctx <- fast_ctx(ctx)
#>     L <- ctx$n_lines
```

```
#>         fit <- make_fitness_fast(ctx, n_sel, gd_ref, alpha_max, ne_par_min =
ne_par_min,
#>          max_use = max_use, pool_tol = pool_tol)
#>     if (is.null(seeds)) {
#>         v <- -rowMeans(ctx$fL)
#>         v <- (v - min(v))/max(max(v) - min(v), 1e-12)
#>         seeds <- lapply(seq(0, 1, length.out = 8), function(s) s *
#>             v)
#>     }
#>     initial <- matrix(stats::runif(NP * L), NP, L)
#>     for (i in seq_len(min(length(seeds), NP))) initial[i, ] <- seeds[[i]]
#>     ctrl <- DEoptim::DEoptim.control(NP = NP, itermax = itermax,
#>         trace = trace, initialpop = initial)
#>     res <- DEoptim::DEoptim(fit, lower = rep(0, L), upper = rep(1,
#>         L), control = ctrl)
#>     decode_lines(as.numeric(res$optim$bestmem), ctx, n_sel, max_use)
#> }
#> <bytecode: 0x7b89440cf0>
#> <environment: namespace:hybdiv>
```

What the call does, in order:

1. Builds the fast fitness: decode the line weights into a selection, tabulate the parent counts, evaluate $\theta = \mathbf{w}'\mathbf{f}^{L}\mathbf{w}$ on the used lines only, and add one proportional penalty per active restriction – diversity budget, minimum $N_{e,\mathrm{par}}$, per-line cap, pool balance. No $N \times N$ matrix is formed or indexed anywhere.
2. Builds the seeds. They sweep a **line-diversity direction**, not a merit weight: `p` proportional to the negated mean coancestry of each line, at eight increasing strengths from zero to one.
3. Searches the box $[0, 1]^{L}$ with `DEoptim`, and decodes the best member.

The zero-strength seed is worth isolating, because it is the natural origin of the encoding: with every line weight at zero the decoder ranks on index alone and takes crosses greedily under the cap, which is capped truncation exactly.

```
cap <- ceiling(2 * n_sel / ctx$n_lines) + 1
zero_w <- decode_lines(rep(0, ctx$n_lines), ctx, n_sel, max_use = cap)
stopifnot(identical(as.integer(zero_w),
                    as.integer(sel_truncation_cap(ctx, n_sel, cap = cap))))
c(cap = cap, index = mean_index(zero_w, ctx),
  alpha_pct = 100 * alpha_loss(zero_w, ctx, gd),
  max_line = max_line_use(zero_w, ctx))
#>       cap     index alpha_pct  max_line
#> 4.0000000 0.8203555 0.3257287 4.0000000
```

The seeds sweep a line-diversity direction rather than a merit weight, but the principle is Chapter 19.'s unchanged: `seeds = NULL` builds the sweep and never means "start cold". The practical consequence of the identity above is that the population always contains at least one plan that respects the cap by construction, whatever the diversity penalty later does.

20.3.2.1 What it is actually for

`sel_de_fast()` is the only route in this package that accepts more than a budget and a cap, so the case for it has to be made where those extra restrictions are active. Here they are, all four at once, run rather than described:

```
cap4  <- ceiling(4 * n_sel / ctx$n_lines)     # the benchmark's cap, not the default
pool_share <- function(s) mean(ctx$pool[as.vector(ctx$parents[s, ])] == "A")

de_fast <- do.call(rbind, lapply(c(20, 25, 30), function(nemin) {
  set.seed(7)
  s <- suppressWarnings(
    sel_de_fast(ctx, n_sel, gd, am, ne_par_min = nemin, max_use = cap4,
                pool_tol = 0.05, NP = 100, itermax = 300))
```

```
  data.frame(Ne_floor = nemin, index = mean_index(s, ctx),
             alpha_pct = 100 * alpha_loss(s, ctx, gd),
             Ne_par = ne_parents(s, ctx), max_line = max_line_use(s, ctx),
             pool_A = pool_share(s))
}))
fmt(de_fast, 4)
```

Ne_floor	index	alpha_pct	Ne_par	max_line	pool_A
20	1.1056	0.8859	25.7143	5	0.5
25	1.1056	0.8859	25.7143	5	0.5
30	1.1056	0.8859	25.7143	5	0.5

```
c(distinct_plans_over_the_three_floors =
    length(unique(round(de_fast[["index"]], 12))))
#> distinct_plans_over_the_three_floors
#>                                    1
```

```
set.seed(7)
ls_capped <- sel_local_search(ctx, n_sel, am, gd, mode = "anneal",
                              iters = 20000, max_use = cap4, seed = 7)
fmt(c(de_fast_index          = de_fast[["index"]][de_fast[["Ne_floor"]] == 25],
      local_search_index     = mean_index(ls_capped, ctx),
      local_search_alpha_pct = 100 * alpha_loss(ls_capped, ctx, gd),
      local_search_Ne        = ne_parents(ls_capped, ctx),
      local_search_max_line  = max_line_use(ls_capped, ctx)), 4)
#>          de_fast_index     local_search_index local_search_alpha_pct
#>                 1.1056                 1.1102                 0.9614
#>        local_search_Ne  local_search_max_line
#>                25.0871                 5.0000
```

Three readings, in the order they narrow the recommendation.

The cap is satisfied by construction and the pool balance is free. `max_line` equals the cap in every row, because the decoder never proposes a plan that breaks it. `pool_A` is exactly 0.5 in every row, and not because the search found it: in a complete factorial every cross spends one pool-A slot and one pool-B slot, so the `matroid-check` panel above already forced it. Of the four restrictions `make_fitness_fast()` accepts, one is structural and cannot bind in this mating design.

The $N_{e,\mathrm{par}}$ floor does not shape the plan either. All three floors return the *same* selection, at $N_{e,\mathrm{par}} = 25.71$: met with room to spare at 20 and 25, missed at 30 with a penalty that does not recover it. The restriction that is actually deciding the plan is the cap – 120 parent slots over 50 lines is a mean usage of 2.4, against a cap of 5. That is the diagnosis of Section 20.7 arriving early: compare $2n/L$ with the cap before crediting any other restriction with the shape of a plan.

And it still does not lead on index. Under the same budget and the same cap, a plain annealed local search reaches 1.11 against `sel_de_fast()`'s 1.106. The honest recommendation is therefore narrow: reach for `sel_de_fast()` when a restriction exists that the swap neighbourhood cannot test – an $N_{e,\mathrm{par}}$ floor that actually binds, or an incomplete factorial where pool balance is a real degree of freedom – and not because it optimises better, because on every number in this chapter it does not.

20.4 The bound was not a bound

Section 18.2.1 introduces the continuous relaxation as an upper bound, and the convergence check of Chapter 19. was originally written to take the best relaxation point whose own `alpha` falls inside the budget. That construction is **not valid**, and it is worth showing rather than asserting.

The relaxation at a given λ maximises $\mathbf{c}'\mathbf{u} - \lambda\,\mathbf{c}'\mathbf{f}\mathbf{c}$ – a penalised objective, not the index. Its optimum happens to land at some diversity, but nothing makes it the best index achievable *at* that diversity. A discrete plan can beat it, and does:

```
lam   <- 10^seq(-1, 3, length.out = 12)
rel   <- ocs_relaxation(ctx, n_sel, lam, iters = 2000)
```

```
a_rel <- (gd - rel[, "GD"]) / gd
V     <- rel[, "index"] - lam * rel[, "theta"]     # the relaxation's dual value

cmp <- do.call(rbind, lapply(c(0.0025, 0.005, 0.01, 0.02), function(a) {
  ok  <- a_rel <= a
  old <- if (any(ok)) max(rel[ok, "index"]) else NA_real_
  new <- min(V + lam * (1 - gd * (1 - a)))  # the Lagrangian form, under-converged
  best <- max(vapply(
    Filter(function(s) alpha_loss(s, ctx, gd) <= a + 1e-12,
           c(list(sel_local_search(ctx, n_sel, a, gd, mode = "anneal",
                                   iters = 8000, seed = 1)),
             lapply(c(0, .5, 1, 2, 4, 8, 16), function(w) sel_greedy(ctx, n_sel, w
= w)))),
    mean_index, numeric(1), ctx = ctx))
  data.frame(alpha_max = a, feasible_plan = best, old_bound = old,
             lagrangian_2000 = new, old_valid = old >= best - 1e-9,
             lagrangian_valid = new >= best - 1e-9)
}))
fmt(cmp, 4)
```

alpha_max	feasible_plan	old_bound	lagrangian_2000	old_valid	lagrangian_valid
0.0025	1.2824	0.8459	1.3373	FALSE	TRUE
0.0050	1.3566	0.8459	1.4034	FALSE	TRUE
0.0100	1.5202	1.4879	1.5355	FALSE	TRUE
0.0200	1.7365	1.4879	1.7998	FALSE	TRUE

The experiment is a falsification, not an illustration. `feasible_plan` is the best plan any method in this book could find *inside* the stated budget; `old_bound` is what the discarded construction would have reported; a valid upper bound must be at least `feasible_plan` in every row. The `old_valid` column is FALSE at all four budgets tested – the "bound" is below a plan that exists, which is a contradiction and not a tight bound.

`lagrangian_2000` is the correct *form* evaluated at a deliberately cheap setting – 12 multipliers, 2,000 projected-gradient iterations – and it is here only to show that the form survives where the construction does not. **It is not a certified bound and nothing downstream of this chunk quotes it.** The certified value comes from `ocs_bound()` below.

The correct statement is a Lagrangian one, and it takes a line. Any selection S of size n has an equal-weight contribution vector feasible for the relaxation, so with $V(\lambda)$ the relaxation's optimal value,

$$V(\lambda) \geq \bar{u}_S - \lambda\,\theta_S \geq \bar{u}_S - \lambda\,\theta_{\max} \tag{20.1}$$

for every S meeting the budget, and therefore $\bar{u}_S \leq V(\lambda) + \lambda\,\theta_{\max}$ for **every** non-negative λ. The tightest such statement is the minimum over λ.

```
ocs_bound
#> function (ctx, n_sel, alpha_max, gd_ref, lambdas = 10^seq(-1,
#>     3, length.out = 25), iters = 4000, lines = NULL, tol = 0.001,
#>     max_doublings = 6L)
#> {
#>     theta_max <- 1 - gd_ref * (1 - alpha_max)
#>     at <- function(it) {
#>         rel <- ocs_relaxation(ctx, n_sel, lambdas, iters = it,
#>             lines = lines)
#>         b <- rel[, "index"] - lambdas * rel[, "theta"] + lambdas *
#>             theta_max
#>         j <- which.min(b)
#>         list(v = b[j], lam = lambdas[j], rel = rel, iters = it)
#>     }
#>     cur <- at(iters)
#>     converged <- FALSE
#>     for (k in seq_len(max_doublings)) {
```

```
#>         nxt <- at(2L * cur$iters)
#>         done <- abs(nxt$v - cur$v) <= tol * max(abs(cur$v), 1e-12)
#>         cur <- nxt
#>         if (done) {
#>             converged <- TRUE
#>             break
#>         }
#>     }
#>     if (!converged)
#>         warning("ocs_bound() did not converge in ", max_doublings,
#>             " doublings (last ", cur$iters, " iterations). An under-converged ",
#>             "relaxation understates V(lambda), so this may not be a valid ",
#>             "bound. Raise `iters`, `max_doublings`, or pass `lines = TRUE`.")
#>     structure(cur$v, lambda = cur$lam, relaxation = cur$rel,
#>         iters = cur$iters, converged = converged, names = NULL)
#> }
#> <bytecode: 0x7b881cc7e8>
#> <environment: namespace:hybdiv>
```

To use it: translate the budget into $\theta_{\max} = 1 - \mathrm{gd_ref}(1 - \alpha_{\max})$; solve the relaxation at each λ to get $V(\lambda)$; take the minimum of $V(\lambda) + \lambda\,\theta_{\max}$; and **read the `converged` attribute before quoting anything**. The function doubles its iteration count until the value stops moving because the value rises monotonically with iterations, so an under-converged run reports the bound too small – the one failure mode that turns a bound into a wrong number rather than a loose one.

```
bound <- ocs_bound(ctx, n_sel, am, gd)
fmt(c(hand_12_lambdas_2000_iters = min(V + lam * (1 - gd * (1 - am))),
      ocs_bound                  = as.numeric(bound),
      lambda_star                = attr(bound, "lambda"),
      iterations                 = attr(bound, "iters"),
      theta_max                  = 1 - gd * (1 - am)), 4)
#> hand_12_lambdas_2000_iters                  ocs_bound
#>                     1.5355                     1.5532
#>                lambda_star                 iterations
#>                    68.1292                  8000.0000
#>                  theta_max
#>                     0.6774
attr(bound, "converged")
#> [1] TRUE
```

The two numbers in the first two rows are the same formula at two iteration counts, and the gap between them is the failure mode named in the paragraph above, measured. The cheap sweep reports 1.5355; the converged run reports 1.5532, higher by 1.16%. Under-convergence pushes a Lagrangian bound *down*, which is the one direction that turns it into a wrong number rather than a loose one. Everything this chapter quotes as a bound from here on is `bound`, the certified object.

Two practical consequences. Every λ gives a valid bound, so a coarse grid costs tightness and never validity – unlike the old construction, where a coarse grid could silently produce a "bound" below the achievable. And the bound is now usable as a stopping rule: when the gap is small, further optimiser effort is provably wasted.

⚠ This changes a number in Chapter 19.

The convergence check in Chapter 19. once used the invalid construction, and the gap it reported was not a gap. That chapter now calls `ocs_bound()`, and the number there is the Lagrangian one; its qualitative conclusion survived the correction – the warm-started DE does sit below the frontier, and the size of the shortfall is what Section 20.3 closes. The lesson is the one that generalises: a quantity is only a bound if a feasible solution can never exceed it, and that is a property to test, not to assume.

Section 18.2.1 has not been rewritten to match, and the contradiction is named here rather than left to be discovered. That section still describes each row of its λ sweep as "an upper bound on what any discrete plan of the same diversity can achieve", and its figure caption still says discrete strategies must sit below and to the left of the curve. Both statements are the construction falsified above. Until that text is corrected, read Section 18.2.1's curve as a relaxation – a penalised optimum at each λ, nothing more – and take every bound in this book from `ocs_bound()`.

20.5 Rounding the relaxation into a plan

With a bound in hand, the relaxation stops being only a diagnostic. Solve the optimal-contribution programme in **line** space, where `quadprog` returns instantly because the dimension is L, round the contributions to integer line usage, and assign concrete crosses under both pools' caps. That two-stage shape – contributions first, mate allocation second – is exactly P. Waldmann [154]'s, whose mate-allocation stage solves in under 0.01 seconds.

The five steps inside `ocs_round()`, in the order they run:

1. **Line merit.** Each line is scored by the mean index of the hybrids it parents in the candidate set. That quantity is a general combining ability: a property of the line *against the opposite pool as represented in this factorial*, never a property of the line alone (Section 13.10). Change the tester pool and the score changes.
2. **Contributions.** `ocs_quadprog()` on `ctx$fL` with those merits, at a given λ, with each line's contribution capped at `max_use`$/2n$ – the cap expressed as a share of the $2n$ parent slots.
3. **Rounding.** Contributions times $2n$, floored, and the shortfall handed to the largest remainders; then clipped at `max_use` again.
4. **Mate allocation.** Walk the candidates in decreasing index and take each cross whose two parents still have quota and are under the cap – a greedy b-matching. If quota runs out before n crosses are found, top up on the cap alone, because the quota is a rounding artefact and the cap is the real restriction.
5. **Choose λ by budget, not by preference.** The whole sweep is evaluated and the plan returned is the best-index one whose realised α respects `alpha_max`. The exchange rate stays an internal detail; the budget remains the interface.

```
r <- ocs_round(ctx, n_sel, am, gd, bound = as.numeric(bound))
fmt(c(index = r[["index"]], alpha_pct = 100 * r[["alpha"]],
      lambda = r[["lambda"]], bound = r[["bound"]],
      gap_pct = 100 * r[["gap"]]), 4)
#>     index alpha_pct    lambda     bound   gap_pct
#>    1.4869    0.8998   52.7500    1.5532    4.2729
```

```
#>        here_index   benchmark_index       here_gd_ref        here_bound
#>           1.48688           1.48688           0.32584           1.55324
#>   benchmark_bound      here_gap_pct benchmark_gap_pct
#>           1.55432           4.27288           4.33929
```

That second panel exists because the benchmark table of Section 20.3 reports this same method and does not report exactly this gap. The plan is identical – both reach index 1.486876 – and the whole difference is in `gd_ref`: this chapter resamples the random reference at $B = 400$ and the cached benchmark at $B = 500$, so $\theta_{\max} = 1 - \text{gd_ref}(1 - \alpha_{\max})$ differs in the fourth decimal and the bound with it. Two draws of the same reference, not two results. What would be a real discrepancy, and was one until this section was corrected, is a gap computed against an *uncertified* bound: the cheap sweep above would have reported 3.17% here, more than a point tighter than the truth, because the quantity it divides by is too small.

```
fmt(r[["frontier"]], 4)
```

lambda	index	alpha
0.1000	1.9919	0.0412
0.2448	1.9919	0.0412
0.5995	1.9919	0.0412
1.4678	1.9919	0.0412
3.5938	1.9919	0.0412
8.7992	1.9919	0.0412
21.5443	1.9741	0.0365
52.7500	1.4869	0.0090
129.1550	0.7365	−0.0093
316.2278	0.4064	−0.0125

The frontier table is the step-5 diagnostic: at small λ the programme puts everything on the best lines and the realised loss exceeds the budget; at large λ it spreads contributions and the loss goes negative – more diverse than a random subset. The plan returned is the highest-index row that respects the budget, which because index falls monotonically with λ is the smallest feasible λ on the grid. If *no* row is feasible the function falls back to the least-loss one, which is a result to notice rather than to use – the returned `alpha` is then above `alpha_max` and nothing in the call will say so.

The rounding is visible in the plan itself. Line usage should track combining ability, and it does:

```
merit <- tapply(rep(ctx$index, 2),
                factor(c(ctx$parents[, 1], ctx$parents[, 2]),
                       levels = seq_len(ctx$n_lines)), mean)
used  <- line_counts(r[["idx"]], ctx)
isA   <- ctx$pool == "A"
c(cor_merit_usage_A = cor(merit[isA], used[isA]),
  cor_merit_usage_B = cor(merit[!isA], used[!isA]),
  lines_used_A      = sum(used[isA] > 0),
  lines_used_B      = sum(used[!isA] > 0),
  max_line          = max(used))
#> cor_merit_usage_A cor_merit_usage_B      lines_used_A      lines_used_B
#>         0.9392334         0.8125721        11.0000000        21.0000000
#>          max_line
#>        14.0000000
```

Result: usage correlates with line merit at 0.94 in pool A and 0.81 in pool B, over 32 lines of 50 actually used. Interpretation: the quadratic programme is doing what an optimal-contribution scheme is supposed to do – allocate more crosses to better general combiners and stop where relatedness makes the next cross expensive. Note that this call passes no `max_use`, so the plan's `max_line` is whatever the coancestry term alone permits; with a cap set, the bound stays valid but goes loose, because the relaxation it comes from carries no cap and therefore bounds an easier problem.

It is not the best method here, and it is not meant to be. What it gives is a plan and a certificate in one call, at a cost that does not grow with N in the part that matters – and a sanity check on everything else.

20.6 Exact, and how far the heuristics really are

Everything so far is a heuristic with a bound. The remaining question is what the true optimum is, and the honest answer requires solving the problem exactly at a scale where that is possible.

The obvious route is closed. Maximising a linear index subject to a **quadratic** diversity constraint over binary variables is a mixed-integer quadratically constrained programme, and HiGHS – like most open solvers – does not accept a quadratic term together with integer variables at all.

Convexity opens a different one. Because $\mathbf{f}$ is positive semi-definite, $\mathbf{x}'\mathbf{f}\mathbf{x}$ is convex, so its tangent at any point is a valid linear *under*-estimate, and

$$\bar{\mathbf{x}}'\mathbf{f}\bar{\mathbf{x}} + 2(\mathbf{f}\bar{\mathbf{x}})'(\mathbf{x} - \bar{\mathbf{x}}) \leq b \tag{20.2}$$

is a linear relaxation of $\mathbf{x}'\mathbf{f}\mathbf{x} \le b$. Solve the resulting mixed-integer **linear** programme, add a cut wherever the solution violates the true constraint, re-solve. This is outer approximation [155], it needs only a MILP solver, and it terminates: each cut strictly removes the infeasible point that generated it.

```
args(sel_exact)
#> function (ctx, n_sel, alpha_max, gd_ref, max_use = NULL, start = NULL,
#>     max_cuts = 200L, time_limit = 60)
#> NULL
```

The loop, step by step:

1. **Translate the budget.** The diversity constraint becomes $\mathbf{x}'\mathbf{f}\mathbf{x} \le n^2\,\theta_{\max}$ with $\theta_{\max} = 1 - \text{gd_ref}(1 - \alpha_{\max})$ – the same quantity as in the Lagrangian bound, expressed in submatrix-sum units because that is what the solver sees.
2. **Write the structural rows.** One cardinality row summing to n, and one row per line for the partition matroid when `max_use` is set. Those are linear and exact; only the quadratic needs approximating.
3. **Carry an incumbent from the start.** The warm start is the same `sel_local_search()` warm start; if it satisfies the budget it becomes the incumbent and is handed to the solver as a starting point, so that a run stopped by the clock still reports a real interval.
4. **Solve, cut, repeat.** If the MILP solution satisfies the true quadratic constraint, it is optimal for the current relaxation and the loop can stop; otherwise a gradient cut at that point is added and the MILP re-solved.
5. **Read the dual bound from the right field.** `objective_value` is an upper bound only for a MILP solved to optimality; the moment the solver hits its own time limit the bound must come from `mip_dual_bound`. Cuts only tighten the relaxation, so the running minimum over iterations stays valid.

```
c(theta_max       = 1 - gd * (1 - am),
  budget_xfx      = n_sel^2 * (1 - gd * (1 - am)),
  binary_vars     = ctx$N,
  structural_rows = 1 + ctx$n_lines)
#>       theta_max      budget_xfx     binary_vars structural_rows
#>       0.6774198    2438.7113298     625.0000000      51.0000000
```

It also returns more than a black-box call would. The MILP objective is a valid upper bound at every iteration and the warm start supplies a feasible incumbent, so a run stopped by the clock still hands back a **certified interval** containing the true optimum.

scale	N	n	budget %	local search	incumbent	dual bound	status	cuts	seconds	gap %
tiny	36	8	0.5	1.1215	1.1215	1.1215	optimal	40	7.135	0.0000
tiny	36	8	1.0	1.1277	1.1277	1.1277	optimal	73	14.667	0.0000
tiny	36	8	2.0	1.2500	1.2500	1.2500	optimal	20	1.846	0.0000
book	625	60	0.5	1.3804	1.3804	1.6424	time limit	4	300.487	15.9535
book	625	60	1.0	1.5280	1.5280	1.6995	time limit	4	300.509	10.0943

Read the two halves separately, because they say opposite things.

At tiny scale the loop **proves** optimality, and the local search matches the proven optimum to machine precision at every budget – the gap is not small, it is zero. That is the result worth having: on instances where the truth is knowable, the heuristic is not leaving anything on the table at all.

At book scale the loop does not converge, and would not at production scale either. Gradient cuts on a dense quadratic are weak, each MILP re-solve is expensive, and the number of cuts needed grows fast. P. Ahadi, B. Balasundaram, J. S. Borrero, and C. Chen [156], solving an integer programme for mate selection with a commercial solver, reach the same conclusion and say plainly that large datasets will need decomposition and "sub-optimal (but good quality) feasible solutions".

The scale caveat cuts both ways and belongs here rather than in a footnote: "the heuristic is exactly optimal" is measured at $N = 36$, on instances small enough to prove. It is evidence that the move and the objective are right, not evidence that the same holds at 625 or at 10,000, where the true optimum is unknown and only the interval between the best plan and the certified bound is reportable.

> **i** What the oracle is for
>
> Not production. `sel_exact()` forms the `N x N` matrix and scales like the mixed-integer programme it is. It belongs with the `ref_*` functions of Appendix C.: deliberately slow, deliberately literal, and the only way to know whether the fast route is right. Run it once, small, when the data changes; do not put it in a pipeline.

20.7 Scaling to 200 of 10,000

N	L	n_sel	f size (GiB)	hybrid (us)	line (us)	of which decode (us)	swap (us)
400	40	20	0.001	6.6	17.9	13.3	2.7
1600	80	32	0.019	7.7	36.3	30.5	3.0
3600	120	72	0.097	39.9	83.0	76.6	3.3
6400	160	128	0.305	78.2	147.8	140.9	3.9
10000	200	200	0.745	132.2	168.7	159.1	4.2
22500	300	450	3.772	NA	NA	NA	NA

```
#>        N_from          N_to    n_sel_from      n_sel_to growth_hybrid
#>        400.00      10000.00         20.00        200.00         20.11
#>  growth_lines growth_decode   growth_swap
#>          9.42         11.99          1.54
```

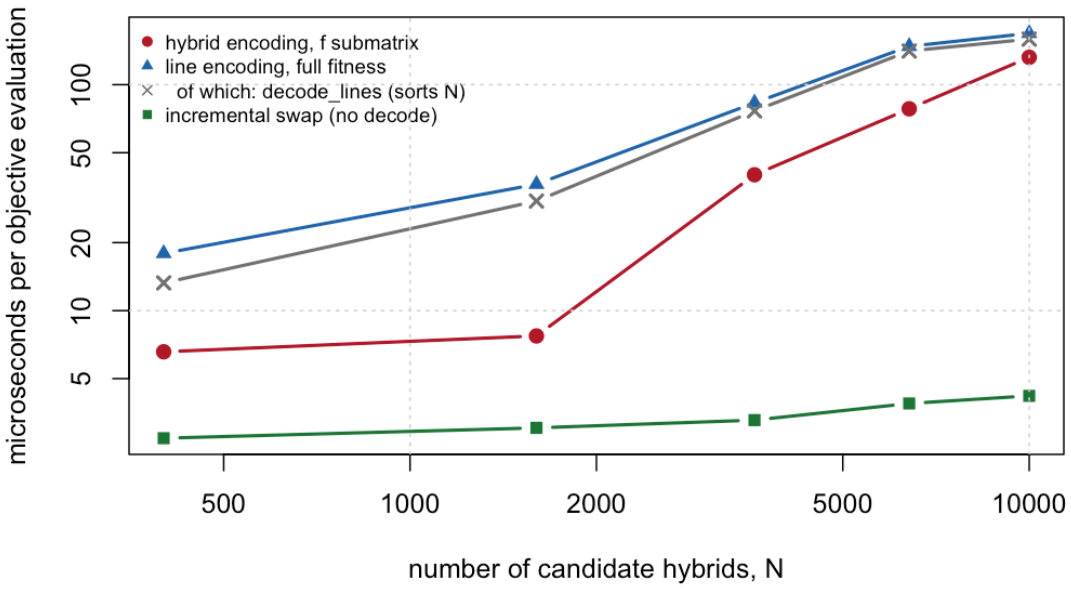

Figure 20.2: Cost per objective evaluation against the number of candidate hybrids. Only the incremental swap is flat. Both encodings grow: the line route by 9.4x, dominated by decode_lines()'s O(N log N) sort over all candidates, and the hybrid route by 20x - but that arm is confounded, because the sweep sets n_sel to 2 percent of N (20 crosses at N = 400, 200 at N = 10,000) and mean(f[idx, idx]) is quadratic in the selection, not in N.

Three things scale badly, and they are worth separating because only one of the three is fixed by the line encoding.

Memory. `f` is $8N^2$ bytes: 0.745 GiB at $N = 10,000$ and 3.8 GiB at $N = 22,500$, which is why Chapter 15.'s benchmark refuses to form it there. $\mathbf{f}^L$ is 200×200, or 0.3 MiB. **Fixed by the line route.**

Search dimension. Not what the line encoding fixes – not for the two differential evolutions, at any rate. `sel_de()` searches n_{sel} continuous coordinates and `sel_de_fast()` searches L; at this chapter's target both are **200**, which is also the figure Section 20.9 quotes. The 10,000-dimensional alternative is one key per hybrid, the encoding Chapter 19. rejects and no route in this package uses. What the collapse genuinely moves to L is the *contributions* space: `ocs_quadprog()` inside `ocs_round()` solves a 200-variable quadratic programme where `ocs_relaxation()` works on a 10,000-variable contribution vector, which is why `ocs_round()` (uncapped, no `max_use`) sweeps its ten λ values and returns a *plan* in 157 ms at production scale. That number is the rounding and nothing else: the benchmark passes `bound =` a value computed once, outside the timed block, and at $N = 10,000$ that value comes back **uncertified**, so the plan is what is fast here and there is no certificate at this scale to be fast about. The paragraph after the table below is where that is measured. **Fixed for the quadratic programme; already true for the DE.**

Cost per evaluation. `mean(f[idx, idx])` is $O(n^2)$ in the *selection*, so it degrades exactly as the problem gets interesting – but `decode_lines()` is $O(N \log N)$, so the line encoding does not fix this and at these sizes is slightly worse. **Fixed only by not decoding**, which is what the swap neighbourhood does.

Scaling up is therefore not a matter of buying time. It is a matter of not forming `f` at all: build `ctx$fL` and `ctx$parents`, and use the routes that read only those. That last clause is a real restriction and not a formality, because most of this package still reaches for the $N \times N$ matrix, and `fast_ctx()` does not change that – it *adds* `parents` to whatever context it is handed and neither builds `f` nor drops it. The constructor with the switch is `ctx_from_pools(sim, with_f = FALSE)`, and the next section is what actually happens on the context it returns.

20.7.1 Which routes actually run without `f`

```
sim <- simulate_pools(n_A = 10, n_B = 10, m = 400, seed = 1)
cxF <- ctx_from_pools(sim, with_f = FALSE)   # fL + parents, no N x N matrix
cxT <- ctx_from_pools(sim, with_f = TRUE)    # the same data, f as well
k   <- 12
gdk <- null_distribution(cxT, k, B = 50, B_full = 10)$gd_ref
st  <- sel_truncation_cap(cxF, k, cap = 3)   # a valid starting selection
set.seed(1); wts <- runif(cxF$n_lines)
# Line merit as ocs_round() defines it: mean index of the hybrids a line parents.
merit <- vapply(seq_len(cxF$n_lines), function(l)
  mean(cxF$index[cxF$parents[, 1] == l | cxF$parents[, 2] == l]), numeric(1))

runs <- function(e) tryCatch({ force(e); "runs" },
                     error = function(err) paste0("ERROR: ", conditionMessage(err)))
knitr::kable(data.frame(
  Route = c("`theta_state()` / `theta_swap()`", "`fast_theta()`",
            "`sel_truncation_cap()`", "`decode_lines()`", "`sel_de_fast()`",
            "`ocs_quadprog()`", "`ocs_bound()`",
            "`sel_local_search(start = st)`", "`sel_local_search(start = NULL)`",
            "`theta_group()` / `alpha_loss()`", "`null_distribution()`",
            "`sel_greedy()`", "`ocs_round()`"),
  `On a ctx with no f` = c(
     runs(theta_swap(theta_state(st, cxF), st[1], setdiff(seq_len(cxF$N), st)[1],
cxF)),
    runs(fast_theta(st, cxF)),
    runs(sel_truncation_cap(cxF, k, cap = 3)),
    runs(decode_lines(wts, cxF, k)),
    runs(suppressWarnings(sel_de_fast(cxF, k, gdk, am, NP = 20, itermax = 10))),
    runs(ocs_quadprog(cxF$fL, merit, lambda = 1, upper = 1)),
     runs(suppressWarnings(ocs_bound(cxF, k, am, gdk, iters = 200, max_doublings
= 1L))),
    runs(sel_local_search(cxF, k, am, gdk, start = st, iters = 500, seed = 1)),
    runs(sel_local_search(cxF, k, am, gdk, iters = 500, seed = 1)),
    runs(theta_group(st, cxF)),
    runs(null_distribution(cxF, k, B = 20, B_full = 5)),
```

```
    runs(sel_greedy(cxF, k, w = 1)),
    runs(ocs_round(cxF, k, am, gdk, bound = NA_real_))),
  check.names = FALSE), row.names = FALSE,
  caption = "Every route in this chapter, run on a context carrying fL and parents
but no N x N f.")
```

Route	On a ctx with no f
`theta_state()` / `theta_swap()`	runs
`fast_theta()`	runs
`sel_truncation_cap()`	runs
`decode_lines()`	runs
`sel_de_fast()`	runs
`ocs_quadprog()`	runs
`ocs_bound()`	runs
`sel_local_search(start = st)`	runs
`sel_local_search(start = NULL)`	ERROR: subscript out of bounds
`theta_group()` / `alpha_loss()`	ERROR: subscript out of bounds
`null_distribution()`	ERROR: subscript out of bounds
`sel_greedy()`	ERROR: argument is of length zero
`ocs_round()`	ERROR: subscript out of bounds

```
# The f-free replacement for null_distribution(): the same resampling, scored
# with fast_theta() instead of theta_group(). Same seed, same draws, same gd_ref.
set.seed(7)
gd_free <- mean(replicate(50, 1 - fast_theta(sample.int(cxF$N, k), cxF)))
stopifnot(all.equal(gd_free, gdk))

# What the matrix was buying the local search: only the starting point.
objs <- c(
  with_f_start_NULL = attr(sel_local_search(cxT, k, am, gdk, iters = 500, seed = 1),
                           "objective"),
  no_f_start_given  = attr(sel_local_search(cxF, k, am, gdk, start = st, iters = 500,
                                            seed = 1), "objective"))
```

Read the second column before designing a pipeline around the paragraph above it. The incremental swap, the line decoder, the line-space quadratic programme and the Lagrangian bound are genuinely `f`-free, and so is `sel_local_search()` **once it is given an explicit `start =`**: on this instance it reaches 1.5804 against 1.5804 for the default warm start with `f` present, so at this size the matrix was buying the starting point and nothing else. That is one instance, not a guarantee – the greedy warm start is there because at book and production scale it is worth having.

`sel_greedy()` is the honest failure: it opens with `f <- ctx[["f"]]` and dies on a `NULL`. The rest are the trap `ctx$f` is. `theta_group()` is `mean(ctx$f[idx, idx])`, and with no `f` in the list that `$` *partial-matches* `ctx$fL` – an $L \times L$ matrix, then indexed with hybrid subscripts, so what surfaces is `subscript out of bounds` and not a missing object. Through `gene_diversity()` and `alpha_loss()` that reaches `null_distribution()`, `ocs_round()`'s per-λ frontier, and `sel_local_search()` at its documented default of `start = NULL`: `ls_warm_start()` does guard the greedy branch with `nrow(ctx[["f"]]) == ctx$N`, but the ranking that follows it scores every candidate with `alpha_loss()`, and that call is unguarded. Had the selection been small enough to index a 20×20 matrix legally, the same line would have returned a diversity computed from the wrong matrix, silently.

So the honest scaling recipe has two branches. If `f` fits, form it: the benchmark below does, on purpose, and the whole chapter's numbers come from that branch. If it does not fit, the `f`-free subset is the *whole* pipeline you get – `sel_truncation_cap()` for a starting plan, `fast_theta()` over random subsets for `gd_ref` in place of `null_distribution()` (same seed, same draws, the same `gd_ref` to machine precision, which the chunk above asserts), `sel_local_search(start = ...)` and `sel_de_fast()` for the search, `ocs_quadprog()` and `ocs_bound()` for the contributions and the bound – and the greedy sweep, the resampled null and the rounding route are simply not available.

20.7.2 The head-to-head at production scale

Nothing broke at 10,000 and nothing here claims it did: the benchmark below forms the 0.745 GiB matrix on purpose, and the hybrid route still evaluates in 132 us. The wall is further out, and it is documented rather than predicted – at $N = 22,500$ the matrix is 3.8 GiB and Chapter 15.'s cost benchmark declines to form it at all, so every metric that needs it has no timing entry at that scale. The line route has one.

method	index	loss %	Ne parents	max uses	seconds
hill	1.816	0.999	46.796	20.333	0.341
anneal	1.814	0.998	47.014	19.000	0.343
ocs_round	1.789	0.832	45.121	20.000	0.157
tempering	1.689	0.999	56.383	15.333	0.352
greedy	1.500	0.227	54.348	18.000	0.221
de_hybrid	1.446	0.986	57.948	18.667	8.858
de_lines	1.336	0.988	28.925	29.667	6.314
greedy_cap	1.123	0.317	100.756	4.000	0.329
de_hybrid_cap	1.122	0.310	101.266	4.000	9.022
de_lines_cap	1.122	0.310	101.266	4.000	15.041
ocs_round_cap	1.122	0.310	101.266	4.000	0.214
tempering_cap	1.122	0.310	101.266	4.000	0.466

Two things in that table are worth more than the ranking.

The cap binds, the budget does not. Every capped method lands on essentially the same plan, at a diversity loss far inside the 1% allowed. With $n = 200$ over $L = 200$ lines the mean usage is already 2, so a cap of 4 leaves very little to choose; the restriction that is actually deciding the plan is the one nobody wrote a budget for. Check `max_line` and realised `alpha` together before concluding that a diversity constraint is doing any work.

Read down the capped rows only, as the rule above the book-scale table requires, and the ordering inverts. `greedy` ranks 5 of 7 among the uncapped rows and 1 of 5 among the capped ones: it tops them at 1.12252 in 0.329 s, ahead of `tempering_cap` at 1.12233 in 0.466 s and of both capped differential evolutions and of `ocs_round_cap`, all three of which land on the identical plan to the tempering here. The same holds at book scale, ranks 5 uncapped and 1 capped, where `greedy_cap` reaches 1.11220 in 0.008 s against `tempering_cap`'s 1.10972 in 0.286 s. A cap this tight leaves the optimiser nothing to find that a capped greedy sweep does not already reach.

The diagnosis is arithmetic and takes a minute: compare $2n/L$, the mean number of uses per line, against the cap. If the cap is a small multiple of the mean, it is the binding restriction and the diversity budget is decoration. Raising the cap, or enlarging the candidate pool, changes the plan; tightening the budget does not.

The certificate does not scale, and the bound is reported as uncertified. `ocs_bound()` doubles its iteration count until the value stops moving. At book scale it converges; at $N = 10,000$ it does not, within any budget worth spending. The reason is in the step size, $1/(2\lambda \max_i \sum_j f_{ij})$, whose denominator grows with N – so the same iteration count that converges on 625 candidates is nowhere near enough on 10,000. An under-converged relaxation reports $V(\lambda)$ *too small*, which makes the "bound" too small, and it stops bounding anything.

scale	N	bound	certified
book	625	1.5543	TRUE
production	10000	1.8378	FALSE

That is a finding, not a defect, and it is why the function warns instead of returning a number. At production scale you have heuristics that agree with each other and no proof that any of them is close – which is the honest position, and a better one than a gap column computed from a quantity that is not a bound.

20.8 Multi-objective, and why not NSGA-II

`stage2_select()` traces the frontier by sweeping `alpha_max` and running a full optimisation at each point. That is already the epsilon-constraint method, and it is the right one: each point is a plan a committee can approve, expressed in a budget rather than in an exchange rate (Chapter 18.).

The available saving is not a Pareto algorithm. It is that consecutive budgets have nearly identical solutions, so the plan found at α_k is the obvious warm start for α_{k+1} – which the sweep currently throws away. That is a `start =` argument, not a new optimiser.

NSGA-II would return the whole front in one run, at worse quality per point and with a crowding heuristic that has nothing to say about a constraint the breeder has already chosen. For a one-dimensional trade-off already parameterised by an interpretable budget, it is machinery in search of a problem.

20.9 What not to reach for

Reinforcement learning. BreedGym and its successors [157] cast the *breeding programme* as a Markov decision process: how to allocate resources across cycles, when to recycle, how many crosses to make each generation. Those are sequential decisions with delayed reward. Choosing 200 hybrids from a fixed candidate pool, once, with a closed-form objective, is not. The tool is aimed at a different question.

Bayesian optimisation. For expensive black-box objectives with a handful of continuous inputs. Ours costs 5 microseconds and has 200 integer dimensions.

Sampling from a determinantal point process. DPPs are the natural probabilistic model of a diverse subset, and $\log\det(\mathbf{I} + \beta\mathbf{f}_S)$ is genuinely submodular and monotone, so its greedy carries a $1 - 1/e$ guarantee [158]. But *sampling* from a DPP gives diversity in distribution when what is wanted is a single best plan, and the log-determinant lens is not θ – adopting it would quietly change the quantity the programme is constraining. Chapter 14.'s rule applies: one diversity lens, chosen deliberately, reported against a null.

20.10 Which method to use

Situation	Use
Any scale, first thing to run	`sel_greedy()` swept over w, then `sel_truncation_cap()` – Chapter 19.
Production scale, a budget and a per-line cap	`sel_greedy(max_use = cap)` swept over w – highest index of any *capped* route at both scales (1.112 in 0.008 s at book scale, 1.123 in 0.329 s at production); `sel_local_search(max_use = cap, mode = "tempering")` comes close without matching it – 2.5 thousandths of an index unit short at book scale, 0.2 at production – and is the only mode measured under a cap
A restriction the swap cannot test (Ne floor, pool balance)	`sel_de_fast()` – the only route that accepts them; it trails the capped greedy on index (Section 20.3.2.1)
You need to know whether to keep optimising	`ocs_bound()` – a valid bound, but check `converged`; a small gap means stop
You need to know the true optimum	`sel_exact()` on a subsample – an oracle, not a pipeline
Contributions are genuinely continuous	`ocs_quadprog()` – Chapter 18.

As a protocol at production scale:

1. Build `ctx` through `fast_ctx()`, and decide up front whether the $N \times N$ matrix is going to exist, because that decision selects the rest of the protocol (Section 20.7.1). `ctx$fL` and `ctx$parents` are required by every fast route; `f` is required by `null_distribution()`, `sel_greedy()`, `ocs_round()` and `sel_local_search()`'s automatic warm start. At $8N^2$ bytes it fits at $N = 10,000$ and the benchmark above forms it. If it does not fit, steps 2-4 change as noted, and the `f`-free subset is what you have.
2. Get `gd_ref` from a resampling at the target `n_sel`, and set $\alpha_{\max}$ below the attainable ceiling (Chapter 18.). With no `f`, `null_distribution()` is unavailable and the resampling has to be `mean(replicate(B, 1 - fast_theta(sample.int(ctx$N, n_sel), ctx)))`, which is `fL`-only and returns the same `gd_ref` at the same seed (Section 20.7.1).
3. Run the sweeps that cost milliseconds – `sel_truncation_cap()` and, when `f` exists, `sel_greedy()` over a w grid spanning your data's crossover, passing `max_use` if a cap applies – and record what they achieve. They are the baseline every later number is judged against, and under a cap they are also, at both scales measured here, the winner (the table above).
4. Run `sel_local_search()` for as many seconds as you are willing to spend, with the cap passed as `max_use` rather than enforced afterwards. With `f` present, leave `start = NULL` and let it build the warm start; with no `f`, pass `start = sel_truncation_cap(...)` explicitly, which is the only configuration of this route that runs without the matrix. The mode is the least consequential choice in this list: on the benchmark above `"hill"` and `"anneal"` are 0.2 thousandths of an index unit apart at book scale and 2.4 at production, with the sign of the difference changing between them, while `"tempering"` trails at both. Take the simplest one unless your own data says otherwise – but note that those three are the *uncapped* rows, and the only mode benchmarked under a cap is `"tempering"`.
5. Compute `ocs_bound()` and check its `converged` attribute. Certified and the gap small: stop. Certified and the gap large: keep going. Uncertified: report no gap at all, and judge the plan by agreement between independent methods instead.
6. Validate the returned plan on the lens the constraint does not pin – `F_drift` and allelic richness – and on the pool decomposition, before it is used (Chapter 24.).

The honest summary is that one change matters more than the rest put together, and it is not an algorithm and not an encoding either. The problem was always L-dimensional in its *state* – θ is a function of the line-usage vector and of nothing else – and that is what buys the $O(L)$ swap, the 0.3 MiB matrix and the 200-variable contributions programme. What it does not buy is a better parameter vector for a population method: `sel_de()` at $n_{sel} = 200$ and `sel_de_fast()` at $L = 200$ search boxes of the same width, and the measured difference between them is small and runs the other way. Everything else in this chapter – the guarantees, the corrected bound, the oracle – is there to tell you when you are done, which turns out to be much earlier than the size of the candidate set suggests.

> Name the structure before choosing the algorithm. The difficulty in this problem was in the state, not in the optimiser – and a claim about an encoding is only as good as the two things it was actually measured between.

Reproduce this chapter

All the code of this chapter, in order and with the explanation kept as comments, is one script: **10b-advanced-selection.R**. Download it, install `hybdiv`, and run it top to bottom in an empty directory; cached results are fetched on first use.

VI Evidence and practice

21 What the plant literature establishes 309
21.1 The headline qualification 309
21.2 Why plant practice differs from animal breeding 310
21.3 Heterotic groups 310
21.4 Selecting within the pool 311
21.5 Genebanks, core collections and introgression . 311
21.6 The animal-breeding corpus: which matrix to manage 312
21.7 Optimisation machinery reported in plants . . . 312
21.8 What remains unestablished 313
21.9 Consequences for the pipeline 313
21.9 Reproduce this chapter 314

22 Diversity across cycles 315
22.1 The cycle 315
22.2 Freezing p_0, and what happens if you do not . . 316
22.3 The trajectory 316
22.4 The rate of inbreeding, and what N_s is not . . . 318
22.5 Choosing a budget for a horizon 319
22.6 The per-line cap, at last measurable 319
22.7 Injection, and what it costs 320
22.8 What the corpus supports 320
22.9 Where this connects 321
22.9 Reproduce this chapter 321

23 The limit, and what the metrics cannot see 323
23.1 Robertson's limit 323
23.2 The R^2 column decides whether the theory applies 323
23.3 What the metrics cannot see 324
23.4 The two costs of selection 325
23.5 Where this connects 326
23.5 Reproduce this chapter 326

24 End to end, and the operational checklist 327
24.1 The whole pipeline in one script 327
24.2 Reading the frontier 330
24.3 What this simulation cannot tell you 330
24.4 The operational checklist 331
24.5 Where to go next 332
24.5 Reproduce this chapter 332

21. What the plant literature establishes

The preceding chapters derived a recommendation. This one asks what the applied literature actually does, where it agrees, and – more usefully – where it is silent.

The corpus behind this chapter is 687 screened plant-breeding records (2017-2026), of which 158 are directly relevant and 197 concern maize, plus 324 animal-breeding records over the same period. Appendix D. describes both and how to query them.

21.1 The headline qualification

Chapter 15. recommended economising to a minimal non-redundant set: θ and $N_{e,\mathrm{par}}$ as constraints, allelic richness at validation. That recommendation stands on its own evidence. **But it is not what plant breeding practice does, and the gap is worth stating plainly.**

metric	records	of which maize	% of 687
Population structure (STRUCTURE/PCA/admixture)	303	87	44.1
Fst / differentiation	117	36	17.0
Expected heterozygosity (He / gene diversity)	101	23	14.7
PIC (polymorphic information content)	97	29	14.1
Modified Rogers / genetic distance	83	36	12.1
Allelic richness / rare alleles	46	6	6.7
Marker coancestry / kinship / relatedness	46	22	6.7
Observed heterozygosity (Ho)	38	5	5.5
Haplotype / pan-genome diversity	37	7	5.4
Shannon / Simpson index	34	3	4.9
Inbreeding coefficient / ROH	22	5	3.2
Effective population size / status number	17	4	2.5

Three observations follow directly.

Applied plant work stacks metrics rather than economising. A representative sub-Saharan maize panel of 376 elite inbreds reports PIC = 0.39, gene diversity = 0.37, MAF = 0.29, Shannon = 6.86, Simpson = 949.09, allelic richness = 787.70 and mean Rogers distance = 0.303 – all in one analysis, presented as a reporting package rather than as competing alternatives. Roughly 20% of relevant records report three or more distinct metric families together.

The metrics recommended here are among the least used. Marker coancestry appears in 6.7% of records, allelic richness in 6.7%, and effective size or status number in 2.5%. Population structure analysis appears in 44% and F_{ST} in 17%. The applied field is oriented toward *describing* structure, not toward *constraining* a loss.

Nothing in the retrieved set computes θ or N_s on a maize hybrid-usage vector. The specific formalism of Section 15.1 is not established in the retrieved set. That is a gap this work fills rather than a contradiction – but it means the approach has **no published plant-breeding baseline to be validated against**, which is a real limitation and should be read as one.

> **i** These are different jobs, not a conflict
>
> For **monitoring and description**, plant practice deliberately reports many metrics because different audiences need different views. For **constraining an optimiser**, redundancy is a cost rather than a virtue.
>
> Report the wide package to collaborators and in publications; constrain the optimiser on the narrow set.

21.2 Why plant practice differs from animal breeding

Three structural facts separate the two literatures, and they explain nearly every divergence in metric choice.

Lines are homozygous. Observed heterozygosity in an inbred panel is a purity control, not a diversity descriptor – pearl millet parental lines report $H_o = 0.031$ against $H_e = 0.334$, and the gap is diagnostic of complete inbreeding rather than informative about the pool. Animal coancestry theory assumes a randomly mating heterozygous population; the plant analogue computes θ directly on fixed line genotypes, which is exactly the setting where the line-level collapse applies.

The product is the F1, not the line. Diversity is tracked separately for each parental pool and for cross-pool complementarity – a structure with no livestock analogue. GWAS models are now built to partition additive and dominance effects by *heterotic group of origin*.

Diversity is managed as two objects, not one. Within-pool and between-pool diversity trend in opposite directions under selection and require opposite management, which no single pool-wide statistic can express – the argument of Chapter 2., arrived at from the applied side.

21.2.1 Distance measures, and the absence of a standard

Modified Rogers distance dominates in maize (36 of 83 distance-reporting records), used chiefly for heterotic assignment and tester choice rather than for diversity monitoring. A recent methods paper works explicitly to unify similarity and distance measures for inbred comparison – the field still treats the choice as open. Recall from Chapter 16. that MRD and coancestry are the same statistic: $f_{ij} = 1 - \mathrm{MRD}_{ij}^2$.

One result is worth carrying into practice: in 298 African highland maize inbreds, **91% of line pairs differ at 30-36% of scored alleles, yet only 32% of pairs have kinship below 0.05.** The authors read the gap as allelic redundancy from repeated use of narrow source germplasm.

> A pool can look diverse by marker counts while being genealogically shallow. That is a direct argument for a coancestry-based constraint over a distance-based one.

Other calibration benchmarks: North American ex-PVP dent inbreds resolve into 8 admixture subgroups aligned with company of origin; China summer maize (490 inbreds, 525,141 SNPs) shows 66.05% of pairwise kinship values near zero; sweet corn shows systematically lower diversity, PIC and MAF than field corn across 514 versus 181 lines.

21.3 Heterotic groups

21.3.1 Assignment is not a solved problem

GCA-based, SCA-based, combined (HSGCA) and SNP-distance classification are compared repeatedly with no consistent winner. Several studies report HSGCA as most consistent with testcross performance; others find SNP-based and testcross-based classification **disagree**. Notably, SSR markers linked to yield QTL give **no advantage** over random SSRs for grouping – against the intuition that linked markers should be more predictive.

This matters for the pipeline: the line-level collapse assumes pool structure is given. Group assignment is itself contested, so the pool-balance constraint inherits whatever uncertainty the assignment method carries. Treat it as an uncertain input, not a fixed given.

21.3.2 The empirical trend that justifies the constraint

The single most load-bearing empirical result in this corpus: in a commercial European dent maize programme, **differentiation between Stiff Stalk and Non-Stiff Stalk groups increased significantly over time while genetic diversity within both groups declined.** This is longitudinal commercial data, not simulation.

It confirms the constraint is doing necessary work rather than solving a hypothetical problem, and it confirms the two-object framing: what erodes is within-pool diversity; what must be protected from erosion *and* from excessive inflation is between-pool divergence. That is exactly the panel `s1_monitor()` logs in Chapter 16..

Why divergence should be maintained rather than maximised now has a mechanistic basis: additive and dominance QTL effects for hybrid performance are **background-specific**, depending on allelic ancestry. Divergence pays through complementarity at specific loci, not through generic distance – so pushing it further does not add hybrid value and risks removing shared favourable alleles. That is Trap 3 of Chapter 16., with a mechanism attached.

The complication worth knowing: deliberate crossing *across* the heterotic boundary for germplasm enhancement generated substantial usable variation, with **10.8% of hybrids from one sub-population exceeding a high-yielding check.** The boundary is maintained for commercial hybrid-parent pools while being selectively porous for pre-breeding.

21.4 Selecting within the pool

21.4.1 The constraint alone is not sufficient

Optimal contribution and optimal cross selection are established across crops. But a load-bearing qualifier appears repeatedly: diversity depletion in closed elite programmes is described as **"ineluctable"** at realistic population sizes even under active management, with bridging populations argued as the necessary remedy.

A θ constraint on the existing pool cannot indefinitely substitute for new germplasm. Plan periodic external injections rather than treating the line pool as closed.

21.4.2 Cycle length, and what goes unreported

Rapid cycling delivers large gain-rate benefits: 12-88% relative gain advantage for oligogenic architectures at 2-3 cycles per year versus one; cycle time cut from 2-5 years to 1 year in intermediate wheatgrass with an explicit diversity-preserving sampling step. A maize two-cycle genomewide selection scheme reports larger gains at equal cost and time but **no diversity metric at all.**

The diversity cost per unit of cycle-time reduction is not established in the retrieved set. Since shortening cycles is the single most common current intervention in commercial programmes, that is a conspicuous hole.

Simulation work on hybrid rice reciprocal recurrent selection reports that larger population sizes give higher gain *and* less depletion of genetic variance – and that diversity is managed at the parental-pool level that feeds hybrids, which is exactly the line-usage level this book constrains.

21.4.3 What is hybrid-specific, and what is not

This distinction decides which literature transfers. **Every usefulness-criterion and progeny-variance study retrieved operates upstream, at the inbred-line development stage** – cross potential selection, UCPC for multi-parental crosses, barley validation against realised populations, soybean family mean and variance prediction, wheat progeny-variance formulas. None operates on F1 hybrids of fixed inbreds.

That confirms rather than challenges the decision to drop the progeny-variance term: the term is real and actively predicted *before* fixation, and vanishes after it. It is why `usefulness_criterion()` belongs at S2 and nowhere else.

One caution that is genuinely hybrid-specific: hybrid **mean** prediction benefits materially from additive-plus-dominance models, since dominance and SCA change predicted performance versus additive-only GEBVs. That is a separate axis from the diversity constraint – it does not reopen the variance term, but it means line-usage optimisation on θ should not be conflated with mean-performance ranking of the same crosses.

21.5 Genebanks, core collections and introgression

Core-subset construction converges on a shared objective – maximise retained diversity at minimum accession count – but diverges on method: GenoCore selecting 310 from 3,300+ sorghum accessions; distance-based optimisation on cassava; rice cores of 247 and 30 from 2,242 accessions; a lettuce core of 268 from 811 capturing 99.4% of total variation.

The quantitatively strongest result: **marker-based core extraction captures 3-78-fold more haplotypic diversity than geography- or environment-based surrogate sampling** across Arabidopsis, Populus and sorghum panels. If cores are being assembled, assemble them on markers.

Maize-specific core-collection algorithm choice is **not established in the retrieved set** – maize genetic-resource work is oriented toward bridging and targeted introgression instead:

- DNA-pool genotyping of 156 landraces (2,340 individuals) against 327 elite lines spanning the 20th century produced two indicators of a landrace's genomic contribution to elite germplasm, used to rank donors rather than introduce material blind.
- Haplotype mining in ~1,000 landrace-derived doubled haploids found landrace haplotypes **outperforming elite alternatives at the same loci**, stable across populations and environments.
- Rapid-cycling genomic selection *within* a landrace population (899 DH lines, 600K array) improves donor material before it contacts the elite pool.
- For capturing landrace variation, the **doubled-haploid route beats gamete capture** – higher prediction accuracy, and no masking of landrace variation behind a capture parent's background.

A θ- or N_s-based accounting of introgression cost against a same-size resampling baseline is not established in the retrieved set.

21.6 The animal-breeding corpus: which matrix to manage

On the animal side there is broad convergence that some form of optimal contribution or optimal cross selection on a **marker-based** coancestry matrix is the right optimisation *framework*. There is **no consensus on which specific genomic matrix** to place inside it, and multiple post-2018 papers report that the choice changes allele-frequency trajectories and diversity outcomes:

- IBD versus homozygosity- versus drift-based matrices give substantially different results unless management is IBD-based (Chapter 12.).
- A deviation-based coancestry matrix retained more genetic variability, while VanRaden's kept allele frequencies closer to the base – a genuine trade-off between "more heterozygosity" and "less frequency drift".
- A linkage-and-pedigree-informed matrix had the highest correlation with true IBD, the most accurate ΔF, and was the only scheme keeping frequency drift down while meeting the inbreeding target.
- ROH-based genomic relationship outperformed SNP-based relationship for limiting inbreeding accumulation at equal gain, in pigs.

The consensus, such as it is: molecular or IBD-flavoured coancestry outperforms simple VanRaden-style relationship for diversity management, but the field has not converged on a single preferred matrix. As Chapter 15. notes, the line-level collapse holds for **any** symmetric $\mathbf{f}^L$, so this open question does not destabilise the computational results.

On allelic richness the post-2018 work sharpens rather than contradicts the recommendation: allelic variation is far more sensitive to bottlenecks than heterozygosity, there is a linear relationship between census-size reduction and reduction in expected number of alleles at drift-mutation equilibrium, and standard N_e corrections do **not** predict allelic loss well. One paper explicitly recommends expanding the $N_e > 500$ conservation guideline to include an allelic-variation criterion.

And on effective size, two cautions beyond $N_s = 1/(2\theta)$: under *active* optimal molecular management, rate-based N_e is numerically pathological – negative in early generations, with no long-term asymptote – and N_e however estimated is informative about heterozygosity loss but not about allelic loss. Both reinforce reporting only a static status-number descriptor under active management.

21.7 Optimisation machinery reported in plants

This is where the corpus is thinnest, and it confirms the computational gap.

- **Optimal contribution methods** are applied across crops, but no abstract states solver type, runtime, or complexity class.
- **Core-subset search** is greedy or distance-based heuristic; only input and output set sizes are reported, never runtime.

- **Evolutionary algorithms** appear mainly *outside* plants. Within plants the closest analogue is a black-box numerical optimiser for progeny allocation, with no named algorithm, problem size, or convergence statistic.
- Problem sizes are reported as candidate-cross counts (330,078 possible barley parent combinations) or collection sizes, never as optimiser performance.

No retrieved plant study applies differential evolution – or any metaheuristic – to an OCS-type hybrid selection problem, and none reports a runtime or complexity comparison against quadratic-programming OCS solvers.

On the animal side, AlphaMate remains the most cited applied tool. The most directly relevant comparison benchmarked genetic algorithms, differential evolution, particle swarm and simulated annealing for parent selection in a 583-line wheat dataset: GA gave the best convergence speed and fitness; **DE was robust but required longer runtime and careful tuning**. That is consistent with Chapter 15.'s finding that the encoding, not the algorithm, is what decides the outcome.

21.8 What remains unestablished

Stated strictly, from the screened abstracts. These are the honest boundaries of what this book can claim support for. The distinction that matters here is between *unestablished in the literature* and *unaddressed in this book*: two of these items now have a simulated answer in Chapter 13. and Chapter 22., which narrows what the book leaves open without changing what the corpus establishes. A simulation under stated assumptions is not evidence from an operating programme, and the items below are still the honest boundary of the second.

- Application of DE or any metaheuristic to a θ-constrained line-usage selection problem in maize.
- Runtime or complexity comparison between OCS quadratic solvers and metaheuristics for any plant breeding problem.
- Empirical proportional diversity loss measured against a same-size resampling baseline in an operating maize hybrid programme. (Chapter 22. simulates the multi-cycle consequence and finds a gain crossover; no published maize programme reports the measurement itself.)
- Longitudinal joint reporting of allelic richness alongside θ or N_s for the same hybrid maize programme.
- Hybrid-genome-level (rather than parental-pool-level) coancestry optimisation for F1 maize.
- Whether a dominance-informed term would change optimal-cross outcomes relative to the mean-only approach. (Chapter 13. derives the term and shows expected heterosis is invariant to F_{ST} under neutral divergence, so a dominance-informed ranking must differ from a frequency-based one. Whether it changes *realised* outcomes in a programme is untested either way.)
- Quantitative diversity cost per unit of cycle-time reduction under rapid-cycling hybrid maize.
- ROH-based diversity metrics applied to maize hybrid pools (22 records use ROH, only 5 in maize, none for hybrid diversity management).
- Per-evaluation computational cost of diversity metrics, as a function of population size or metric type, anywhere in either corpus.
- Diversity metrics defined on hybrids rather than on lines or parents.
- Combinatorial equal-contribution (0/1) selection compared directly against continuous-contribution designs.

21.9 Consequences for the pipeline

1. Keep the narrow metric set inside the optimiser and report the wide package outside it.
2. Plan for periodic germplasm injection. A closed elite pool depletes regardless of how well the within-pool constraint is managed.
3. Prefer bridging over direct introduction for exotic material, and rank donors by their genomic contribution to the elite pool rather than by raw diversity.
4. Treat heterotic-group assignment as an uncertain input – the pool-balance constraint is only as good as the assignment behind it.
5. Constrain between-pool divergence within a band rather than maximising it.
6. Do not conflate the diversity constraint with mean prediction. Hybrid means need additive-plus-dominance models; the diversity constraint needs neither.
7. Report the diversity cost per cycle explicitly when shortening cycles.
8. If assembling a core or donor set, do it on markers rather than passport data.

9. Watch for allelic redundancy: high marker-based distance can coexist with shallow genealogy, so a coancestry constraint is safer than a distance-based one.
10. Validate on real genotypes before treating the redundancy result as settled.

Reproduce this chapter

All the code of this chapter, in order and with the explanation kept as comments, is one script: **11-plant-literature.R**. Download it, install `hybdiv`, and run it top to bottom in an empty directory; cached results are fetched on first use.

22. Diversity across cycles

Chapter 24. will end with a list of things its numbers cannot tell you, and the last item on it is the one that matters most: *one cycle, not a trajectory. A one-cycle constraint is not a multi-cycle policy.*

Every result in this book so far has been a single round of hybrid selection. But nobody pays an index cost for diversity in order to have more diversity. They pay it because of what next cycle can do with it, and that argument has been taken on faith for eleven chapters. This chapter runs the programme forward and checks.

22.1 The cycle

Selection happens at the hybrid level, on $A \times B$ crosses. Recycling happens *within* pool. That asymmetry is the whole design, and getting it backwards would quietly destroy the thing Chapter 13. says the programme is selling: a doubled haploid derived from an $A \times B$ hybrid is a blend of both pools, and a programme that recycled hybrids would merge its heterotic groups within a few cycles.

What crosses the pool boundary is *information* – which lines proved themselves in hybrid combination – and never germplasm.

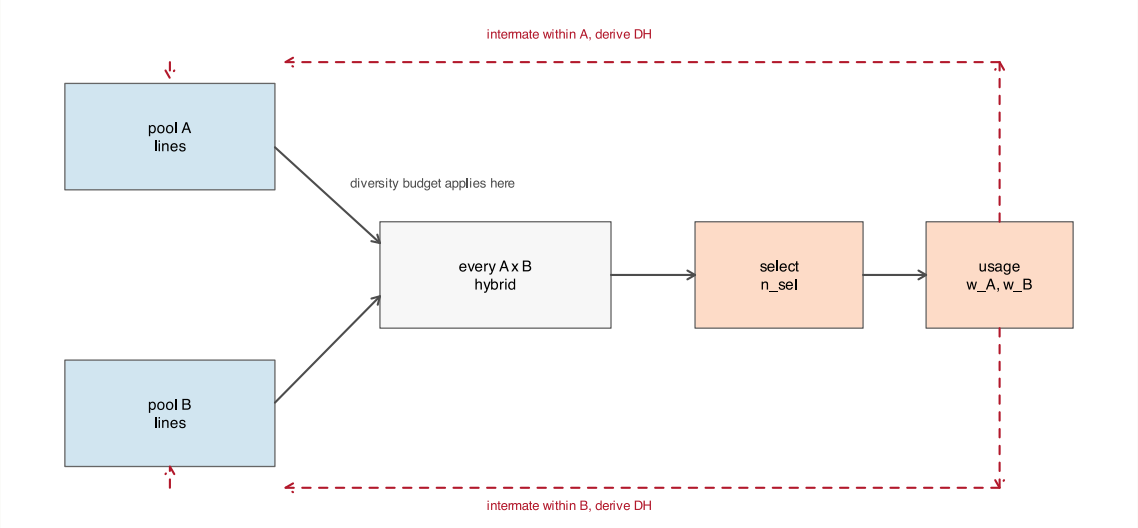

Figure 22.1: One cycle. Hybrids are evaluated across the pool boundary; lines are recycled inside it. The usage weights are the only channel between the two, and they are what `line_weights()` returns.

`run_cycles()` is the loop. The assumptions it runs under, stated once:

C1. Loci are unlinked. Every marker segregates independently, so no linkage disequilibrium builds up and there is no Bulmer effect on the genic side. This is the same abstraction Chapter 4. already makes; Chapter 5. is where linkage is modelled properly, and `build_map()`/`meiosis()` are there if this ever has to change.

C2. Recycling is by doubled haploid. Two parents are drawn within pool with probability proportional to usage, and one DH is derived per pair. Because the lines are inbred, `(GL[i, ] + GL[j, ]) / 2` lies in $\{0, 0.5, 1\}$ and a single Bernoulli draw per locus is an *exact* gamete, not an approximation.

C3. Pool sizes are constant. 25 lines per pool, every cycle. A real programme varies this, and the variation is itself a diversity decision.

C4. Marker effects are fixed. The same β throughout, so the index is comparable across cycles and gain accumulates on the founding cycle's scale. No new mutation, no changing selection target, no genotype-by-environment.

C5. The base p_0 is frozen at cycle 0 and carried forward. This is the subject of the next section.
C6. The pool is closed unless a germplasm injection is scheduled.
C7. The index is known without error. Chapter 15. Finding 5 measures what relaxing this does within a cycle; the effect compounds across cycles and is not modelled here. Every gain figure below is therefore an optimistic ceiling, for all budgets equally.
C8. Selection is on the hybrid only. No per-se line selection, no stage structure. Chapter 16. is where the multi-stage version lives.

22.2 Freezing p_0, and what happens if you do not

`build_ctx()` sets `p0 <- colMeans(X)` on every call. Run it fresh each cycle and the drift lens is re-anchored to the population it is supposed to be measuring drift *away from*, which reports approximately zero loss forever while the pool erodes underneath.

The demonstration needs two generations and nothing else:

```
sim <- simulate_pools(n_A = 25, n_B = 25, m = 2000, seed = 1)
p0  <- colMeans(sim$X)                       # the founding base, frozen here
GL5 <- sim$GL
set.seed(7)
for (t in 1:5) {                              # five generations of drift
  GL5 <- rbind(next_pool(GL5[1:25, ], rep(1, 25), 25),
               next_pool(GL5[26:50, ], rep(1, 25), 25))
}
ped5 <- expand.grid(a = 1:25, b = 25 + 1:25)
X5   <- (GL5[ped5$a, ] + GL5[ped5$b, ]) / 2

drift <- function(X, base) {
  p <- colMeans(X); k <- base > 1e-6 & base < 1 - 1e-6
  mean((p[k] - base[k])^2 / (base[k] * (1 - base[k])))
}
c(frozen_p0   = drift(X5, p0),
  reanchored  = drift(X5, colMeans(X5)))
#>  frozen_p0 reanchored
#> 0.08693168 0.00000000
```

The second number is zero by construction: a population is never drifted from itself. Five generations of real drift have been reported as none.

> ! This is the most common way to get a multi-cycle report wrong
>
> Chapter 12. named it as the single most frequent error in applying `F_hom` and `F_drift`, and Chapter 24. repeats it in the operational checklist. `run_cycles()` therefore takes `p0` once and overwrites it on every rebuilt context. It is one assignment, and every drift number in this chapter depends on it.

22.3 The trajectory

Three budgets, fifteen cycles, five seeds each. `alpha_max = Inf` is the unconstrained programme: it takes the best hybrids it can find and accepts whatever that costs.

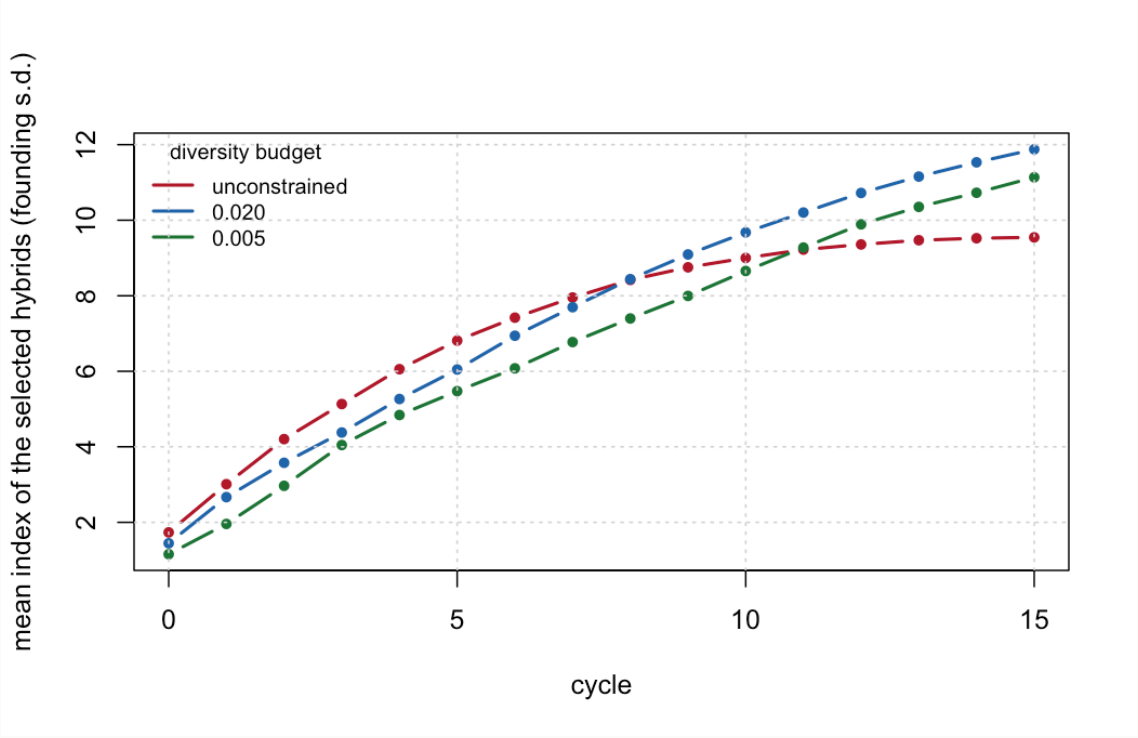

Figure 22.2: Mean index of the selected hybrids, in founding-cycle standard deviations, averaged over five seeds. The unconstrained programme leads for eight cycles and then stops climbing. Neither constrained programme has stopped by cycle 15.

cycle	alpha <= 0.005	alpha <= 0.02	unconstrained
0	1.16	1.45	1.73
3	4.05	4.38	5.13
5	5.47	6.05	6.81
8	7.40	8.44	8.42
10	8.66	9.68	9.00
12	9.89	10.72	9.36
15	11.14	11.88	9.55

The unconstrained run wins the first eight cycles and loses every cycle after that. By cycle 15 the $\alpha \leq 0.02$ programme is 2.33 standard deviations ahead – 24% more gain, from a programme that gave up 17% of its gain in year one.

The mechanism is visible in the diversity panel:

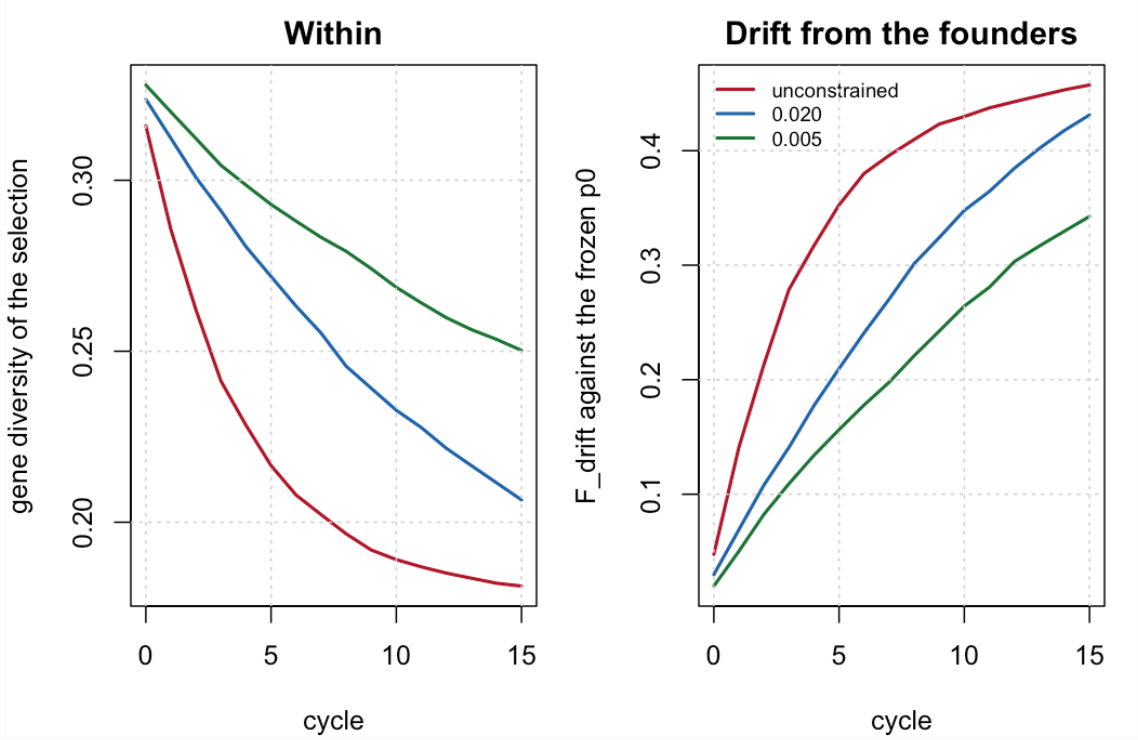

Figure 22.3: Left: gene diversity of the selection. Right: F_drift against the frozen founding base. The unconstrained programme spends nearly its whole diversity endowment by cycle 8 and has little left to convert afterwards.

The unconstrained programme is not out-selected. It is out of material. Its F_{drift} reaches 0.41 by cycle 8 and only 0.457 by cycle 15 – it has converted almost everything it had into gain, early, and then has nothing to convert.

22.3.1 It is not an artefact of the cheap selector

The trajectories above use a greedy sweep, which is what makes fifteen cycles by five seeds by three budgets affordable. Chapter 19.'s differential evolution finds slightly better plans per cycle. Re-running one seed with it:

cycle	0.020 de	0.020 greedy	unconstrained de	unconstrained greedy
0	1.39	1.37	1.53	1.53
4	5.53	5.98	5.22	6.60
8	8.44	9.56	7.40	9.08
12	10.47	11.73	7.99	9.84

The crossover reproduces. Under DE the constrained programme is ahead by 2.48 at cycle 12, against 1.89 under greedy. The cheap selector understates the case rather than manufacturing it.

There is a second thing in that table worth not glossing over: DE reaches a *lower* absolute index than greedy by cycle 12 in both columns, despite finding a better plan in every individual cycle. It uses more of the budget each time – $\alpha = 0.0200$ against greedy's 0.0177 at cycle 0 – and spends the difference out of the next cycle's inheritance. **Per-cycle optimality is not multi-cycle optimality.** This is one seed and should be read as a flag rather than a result, but it is the same trade the whole chapter is about, one level up.

22.4 The rate of inbreeding, and what N_s is not

Chapter 24.'s checklist says to regress $\log(1 - F)$ on cycle and check that it is linear, because curvature means a non-constant rate. Doing it:

```
rate <- do.call(rbind, lapply(unique(traj$budget), function(b) {
  d <- aggregate(F_drift ~ cycle, traj[traj$budget == b, ], mean)
  f <- lm(log(1 - F_drift) ~ cycle, d[-1, ])
  data.frame(budget = b, dF_per_cycle = -coef(f)[["cycle"]],
             Ne = 1 / (-2 * coef(f)[["cycle"]]), R2 = summary(f)$r.squared)
}))
fmt(rate, 4)
```

budget	dF_per_cycle	Ne	R2
unconstrained	0.0292	17.1146	0.8534
0.020	0.0358	13.9841	0.9931
0.005	0.0265	18.8898	0.9976

The diagnostic fires, and it fires on the unconstrained run: $R^2 = 0.853$ against 0.998 for the tightest budget. The unconstrained rate is not constant because it is decelerating – there is progressively less left to lose. Read naively its fitted ΔF is the *lowest* of the three, which is exactly backwards; the fit is meaningless for a saturating series, and the R^2 column is what says so.

> ⚠ N_s does not project this
>
> Section 14.3 insists that the status number is a static descriptor and not a drift projection. Here is the size of the discrepancy: N_s computed on the selected hybrids averages 0.67 across all of these runs, while the realised effective size implied by the drift they actually accumulated is between 14 and 19.
>
> They are not the same quantity and are not off by a constant. Report N_s as a descriptor of a plan, and measure ΔF from the trajectory.

22.5 Choosing a budget for a horizon

Every seed crosses over, and the cycle at which it happens is the useful number:

seed	alpha <= 0.02	alpha <= 0.005
1	8	11
2	12	15
3	10	12
4	7	10
5	5	9

The tighter budget always crosses later and always ends higher. That is the whole trade, and it makes the choice of $\alpha_{\max}$ a function of the planning horizon rather than a matter of taste:

Planning horizon	Budget	Why
3 cycles or fewer	do not constrain	the crossover has not happened yet; the constraint is pure cost
about 5 to 8 cycles	alpha ~ 0.02	the looser budget has crossed and the tighter one has not
10 cycles or more	alpha ~ 0.005 to 0.02	both have crossed; the tighter budget is still climbing
indefinite	constrain, and inject	no closed budget holds forever – see below

The caveat matters more than the numbers. `n_sel = 60` out of 625 candidates from 25 + 25 lines is a small, intensely selected programme, and the crossover arrives sooner the harder the selection is. What transfers is the shape: an unconstrained programme leads early, saturates, and is passed.

22.6 The per-line cap, at last measurable

Chapter 19. carries `max_use` through every selector, defaulting to `NULL` everywhere, because within a single cycle a usage cap only costs index. Across cycles it is buying something, and this is where it can be seen. Same budget ($\alpha \leq 0.02$), same seeds, three caps:

max_use	cycle	index	GD	F_drift	max_line
3	0	0.491	0.329	0.005	3.0
6	0	1.305	0.323	0.025	6.0
none	0	1.446	0.324	0.030	12.4
3	5	2.423	0.296	0.104	3.0
6	5	5.788	0.270	0.194	6.0
none	5	6.047	0.272	0.210	15.4
3	10	4.058	0.268	0.185	3.0
6	10	9.216	0.233	0.316	6.0
none	10	9.677	0.233	0.348	15.4
3	15	5.532	0.247	0.250	3.0
6	15	11.268	0.210	0.395	6.0
none	15	11.877	0.207	0.431	16.4

`max_use = 6` costs 5% of cycle-15 gain and holds F_{drift} to 0.395 against 0.431, while the uncapped programme leans on one line 16 times in its final cycle. `max_use = 3` is too tight: it halves the gain and is not compensated.

Note *which* lens the cap moves. Gene diversity at cycle 15 is essentially the same with the cap and without it (0.21 against 0.207), because α is already pinning that. The difference shows up in F_{drift}, which is the lens Chapter 12. established the constraint does *not* pin. The cap is a handle on the uncontrolled lens, which is exactly why it is worth having in addition to the budget and not instead of it.

This changes no default. `max_use` remains `NULL` throughout the package.

22.7 Injection, and what it costs

No closed budget holds indefinitely: all three trajectories are still losing diversity at cycle 15, only at different rates. Chapter 21. and Chapter 24. both prescribe periodic germplasm injection, and Chapter 6. is the machinery that performs it – introgression is how new material enters an elite pool without dragging the rest of the genome with it.

Replacing six of the twenty-five lines in each pool with founding-generation material at cycle 8:

regime	cycle	index	GD	F_drift
closed	7	7.697	0.255	0.270
injected	7	7.697	0.255	0.270
closed	8	8.440	0.246	0.301
injected	8	6.469	0.283	0.182
closed	10	9.677	0.233	0.348
injected	10	7.865	0.270	0.233
closed	12	10.722	0.222	0.385
injected	12	9.391	0.254	0.291
closed	15	11.877	0.207	0.431
injected	15	11.283	0.234	0.364

The cost is immediate and the return is slow. Gene diversity jumps from 0.246 to 0.283 and F_{drift} falls by 40%, but the index drops 1.97 standard deviations on the spot, and **has not repaid by cycle 15** (11.28 against 11.88).

That is the honest result and it should not be dressed up. Within this horizon injection buys a restored resource and not more gain; it is the same trade as the diversity budget itself, at a longer wavelength and a steeper entry price. The run that would repay it is longer than fifteen cycles, and the founders used here are the programme's own – unadapted material from a genebank would cost considerably more.

22.8 What the corpus supports

Appendix D.'s screened corpora hold 687 plant and 324 animal records, of which a substantial block concerns recurrent selection and long-term gain: two-part and rapid-cycling programme designs, optimal

cross selection evaluated over cycles rather than within one, and simulation studies of the long-term effects of genomic selection on both gain and variance.

The qualitative shape found here – early advantage to unconstrained selection, saturation, then a crossover – is the standard finding in that literature. G. Gorjanc, R. C. Gaynor, and J. M. Hickey [59] optimise cross selection explicitly for long-term gain in two-part programmes and report the same tension between per-cycle and multi-cycle optimality that the DE comparison above ran into. A. Allier, C. Lehermeier, A. Charcosset, L. Moreau, and S. Teyssèdre [60] separate the short-term and long-term components directly. Y. C. J. Wientjes, P. Bijma, M. P. L. Calus, B. J. Zwaan, Z. G. Vitezica, and J. van den Heuvel [159] track what genomic selection does to additive variance over many generations, which is the mechanism behind the saturation seen here, and A.-C. Doublet, P. Croiseau, S. Fritz, A. Michenet, C. Hozé, and C. Danchin-Burge [160] measure the diversity cost in three dairy cattle breeds where the cycles have actually been run. J. E. Rutkoski [161] is the counterpart caution: realised rates of gain are harder to estimate from an operating programme than a simulation makes them look. D. Müller, P. Schopp, and A. E. Melchinger [162] is the closest thing in the corpus to the selection criterion used here, chosen for its effect on later cycles rather than the current one. What Chapter 21. listed as unestablished was something narrower and remains so: nobody has published the crossover *for a θ-constrained hybrid maize programme measured against a same-size resampling baseline*. This chapter demonstrates it in simulation, under C1-C8, which is not the same as establishing it in an operating programme.

22.9 Where this connects

Chapter 13. argued that what builds a heterotic pattern is divergence aligned with dominance, and that selection on cross performance is the only process in the programme that searches for it. This chapter is that process running. The GD_{BS} column of `cycles_trajectory.csv` rises in every run – the pools do separate under reciprocal selection, as they should – while GD within them falls, which is the trade Chapter 14.'s partition was built to expose.

Chapter 24. is the single cycle, in operational detail. Read it as cycle 0 of this chapter, and read its final checklist – freeze p_0, keep the per-stage ledger, check $\log(1 - F)$ for curvature, plan germplasm injection – as the four things this chapter has now shown the consequences of skipping.

Reproduce this chapter

All the code of this chapter, in order and with the explanation kept as comments, is one script: **11b-diversity-across-cycles.R**. Download it, install `hybdiv`, and run it top to bottom in an empty directory; cached results are fetched on first use.

23. The limit, and what the metrics cannot see

Chapter 22. ran the programme forward and found the crossover: a constrained programme trails an unconstrained one for about eight cycles and leads thereafter. That is a measurement. This chapter asks the two questions a measurement leaves open – could it have been *predicted*, and is the quantity being managed the quantity that actually matters.

The answers are yes-with-a-condition, and no-not-entirely.

23.1 Robertson's limit

Chapter 3. wrote down the response to one cycle of selection. Robertson (1960) wrote down the sum of all of them. Selection fixes alleles; a finite population fixes some of the favourable ones in the wrong direction; and the total advance before everything is fixed is approximately

$$R_{\text{total}} \approx 2 N_e R_1 \tag{23.1}$$

with a half-life of about $1.4 N_e$ generations. The result is the theoretical ancestor of every optimal-contribution scheme in this book: it is *why* N_e is the thing to manage, and it says exactly what a unit of N_e is worth in units of cumulative gain.

Both inputs are already in Chapter 22.' trajectory. R_1 is the first cycle's response, and N_e comes from the rate of inbreeding, exactly as Chapter 22. computed it:

```
rob <- do.call(rbind, lapply(unique(traj$budget), function(b) {
  d  <- aggregate(cbind(index, F_drift) ~ cycle, traj[traj$budget == b, ], mean)
  f  <- lm(log(1 - F_drift) ~ cycle, d[-1, ])
  Ne <- 1 / (-2 * coef(f)[["cycle"]])
  R1 <- d$index[2] - d$index[1]
  data.frame(budget = b, R2 = summary(f)$r.squared, Ne = Ne, R1 = R1,
             limit = 2 * Ne * R1, half_life = 1.4 * Ne,
             observed_15 = d$index[16] - d$index[1],
             predicted_15 = 2 * Ne * R1 * (1 - (1 - 1 / (2 * Ne))^15))
}))
fmt(rob, 3)
```

budget	R2	Ne	R1	limit	half_life	observed_15	predicted_15
unconstrained	0.853	17.115	1.277	43.697	23.960	7.812	15.688
0.020	0.993	13.984	1.222	34.167	19.578	10.431	14.378
0.005	0.998	18.890	0.802	30.281	26.446	9.980	10.032

Read the last two columns against each other, and then read both against the first.

23.2 The R^2 column decides whether the theory applies

For the tightest budget the prediction is 10.03 and the observation is 9.98. Robertson's theory, published in 1960 and derived for a single locus under idealised drift, predicts fifteen cycles of a genomic hybrid programme to better than one percent.

For the unconstrained programme it predicts 15.69 against an observed 7.81 – too high by about half.

The theory did not fail arbitrarily, and Chapter 22. already said why. Robertson assumes a *constant* rate of inbreeding, and the regression of $\log(1 - F)$ on cycle is where that assumption is testable. Its R^2 is 0.998 for the tightest budget and 0.853 for the unconstrained run, whose drift decelerates because there is

progressively less left to lose. That low R^2 is not a nuisance. It is the theory declining to apply, in advance, and in a column you can read before you trust the number.

> ! Do not quote a Robertson limit without its R^2
>
> N_e here is a *fitted* quantity, and Chapter 22. showed the fit is meaningless for a saturating series – its fitted ΔF comes out lowest for the run that is actually losing diversity fastest, which is backwards. Feeding that N_e into $2N_eR_1$ propagates the error into a number that looks like a long-term plan.
>
> Report the R^2 next to every limit, and treat a limit computed from a fit below about 0.99 as an illustration rather than a projection.

That yields an argument for the constraint that this book did not previously have. Chapter 22. justified α by where the trajectories cross. The table above adds: the constrained programme is the one whose future can be *computed*. An unconstrained programme is not merely worse after cycle 8, it is unforecastable, because the theory that would forecast it needs a constant ΔF that it does not have.

23.3 What the metrics cannot see

Now the second question. Every diversity metric in this book – θ, GD, N_s, allelic richness, `F_hom`, `F_drift` – is a function of allele frequencies **one locus at a time**. Chapter 3. showed that additive variance is not:

$$\mathrm{Var}(\mathbf{X}\boldsymbol{\beta}) = V_{\mathrm{genic}} + 2\sum_{k<l} \beta_k \beta_l \ \mathrm{Cov}(x_k, x_l) \tag{23.2}$$

Directional selection drives that covariance term negative (Bulmer (1971)): it couples loci with like effects in repulsion, hiding genic variance from the next generation without moving a single allele frequency. No metric in the catalogue can detect it.

Chapter 22. assumed the effect away, in assumption **C1**: *loci are unlinked, so no linkage disequilibrium builds up and there is no Bulmer effect on the genic side.* That is worth testing rather than assuming, because Bulmer's 1971 treatment covers unlinked loci explicitly – free recombination halves the disequilibrium each generation, but selection regenerates it every generation, and the equilibrium is not zero.

23.3.1 The experiment

The gap $\mathrm{Var}(\mathbf{X}\boldsymbol{\beta}) - V_{\mathrm{genic}}$ is **not** on its own a measure of the Bulmer effect. A finite founder base leaves disequilibrium in the very first cycle, and two divergent pools add more. So the comparison is against a control: an arm identical in every respect – same founders, same seeds, same recycling, same population sizes – in which the hybrids are chosen at random instead of on the index.

```
gapc <- aggregate(gap ~ regime + cycle, vr, mean)
w <- reshape(gapc, idvar = "cycle", timevar = "regime", direction = "wide")
names(w) <- sub("gap.", "", names(w))
w$selection_effect <- w$unconstrained - w$random
fmt(head(w[, c("cycle", "random", "unconstrained", "constrained",
               "selection_effect")], 7), 4)
```

	cycle	random	unconstrained	constrained	selection_effect
1	0	−0.0261	−0.0261	−0.0261	0.0000
4	1	−0.0716	−0.2686	−0.1984	−0.1970
7	2	−0.0328	−0.2167	−0.1930	−0.1839
10	3	−0.0320	−0.2616	−0.0264	−0.2296
13	4	0.0935	−0.0931	−0.0976	−0.1866
16	5	−0.0453	−0.1069	−0.1171	−0.0615
19	6	0.0512	−0.0711	−0.1130	−0.1223

Cycle 0 is identical across all three arms, to the digit. Nothing has been selected yet, so the control is doing its job.

From cycle 1 the arms separate. Paired within seed and cycle:

```
p  <- reshape(aggregate(gap ~ regime + seed + cycle, vr, mean),
              idvar = c("seed", "cycle"), timevar = "regime", direction = "wide")
p  <- p[p$cycle > 0, ]
dd <- p$gap.unconstrained - p$gap.random
c(mean_effect = mean(dd), se = sd(dd) / sqrt(length(dd)),
  t = mean(dd) / (sd(dd) / sqrt(length(dd))), n_pairs = length(dd))
#> mean_effect          se           t     n_pairs
#> -0.08568088  0.02108980 -4.06266819 96.00000000
```

⚠ Assumption C1 of Chapter 22. is too strong, and is restated here

Selection builds negative disequilibrium among **unlinked** loci, at $t = -4.06$ over 96 paired observations. Unlinked loci do not prevent the Bulmer effect; they make it decay fast.

What C1 should say is that with free recombination the disequilibrium is halved every generation, so it reaches a low equilibrium and does not accumulate across cycles the way frequency change does. That is what makes Chapter 22.' *frequency* trajectories trustworthy, and it is the correct reason – not the absence of the effect.

The effect is largest in the early cycles and decays toward zero late, which is the expected shape: it is proportional to the genic variance still available to be hidden, and by cycle 12 there is much less of it.

23.3.2 With real linkage

The same question with real meiosis behind it. Chapter 7.'s reciprocal recurrent selection runs on a coalescent founder history with a genetic map, and AlphaSimR reports `varA` (which carries the disequilibrium) and `genicVarA` (which does not), so this arm needs no variance code of its own.

```
lk$rel <- ((lk$varA_A - lk$genicVarA_A) + (lk$varA_B - lk$genicVarA_B)) /
          (lk$genicVarA_A + lk$genicVarA_B)
pl  <- reshape(lk[, c("regime", "seed", "cycle", "rel")],
               idvar = c("seed", "cycle"), timevar = "regime", direction = "wide")
pl  <- pl[pl$cycle > 0, ]
ddl <- pl$rel.selected - pl$rel.random
c(mean_effect = mean(ddl), se = sd(ddl) / sqrt(length(ddl)),
  t = mean(ddl) / (sd(ddl) / sqrt(length(ddl))), n_pairs = length(ddl))
#> mean_effect          se           t     n_pairs
#> -0.16493928  0.04672916 -3.52968674 32.00000000
```

Same sign, larger in magnitude, and measured with fewer replicates so the evidence is weaker. Linkage does what linkage should: it slows the decay, so the disequilibrium selection creates persists longer than it does under free recombination.

ℹ A reproducibility trap in the linked arm

`alphasimr_founder_pools()` is **not** reproducible from `cfg$seed`. `AlphaSimR::runMacs2()` hands the coalescent to MaCS, which seeds itself outside R's RNG, so two calls with the same configuration return different founder haplotypes. Everything downstream of the founders is seeded normally.

The consequence for an experiment like this one is severe: founding the pools separately for each arm would have compared two different base populations and charged the difference to selection. `book/scripts/regenerate.R` founds once per replicate and branches both arms off that single result, which is why cycle 0 of the table above matches exactly.

23.4 The two costs of selection

The chapter's result stated once.

Selection pays for gain twice. It builds negative covariance between loci, which is a real loss of usable variance, and **recombination repays it** – halved every generation under free recombination, more slowly

under linkage. And it changes allele frequencies, moving them toward fixation, and **nothing repays that** short of mutation or migration.

The first cost is invisible to every metric in this book and is largely self-correcting. The second is what every metric in this book measures, and is permanent.

That is the justification for constraining the second and ignoring the first, and it is a stronger statement than "the constraint is what we can compute". It is worth being explicit that this ordering is a *consequence* of the two decay rates, not an assumption: if the disequilibrium did not decay – tight linkage, few chromosomes, short cycle times – the ranking would change, and a constraint written on allele frequencies would be managing the smaller of the two bills.

23.5 Where this connects

Chapter 3. supplies the variance decomposition and the breeder's equation this chapter runs forward. Chapter 22. supplies the trajectories, and has one assumption restated as a result of the test above. Chapter 7. supplies the linked arm, which is the only place in the book where a genetic map, rather than an assumption about one, decides how fast disequilibrium decays.

Chapter 24.'s checklist gains one line from this chapter: report the R^2 of the $\log(1 - F)$ regression next to any projection built on N_e, and treat a projection from a poor fit as an illustration.

Reproduce this chapter

All the code of this chapter, in order and with the explanation kept as comments, is one script: **11c-limits.R**. Download it, install `hybdiv`, and run it top to bottom in an empty directory; cached results are fetched on first use.

24. End to end, and the operational checklist

Everything the book has built, run once, on one dataset, in the order a programme would run it. Then the checklist.

24.1 The whole pipeline in one script

```
st1 <- stage1_simulate(book_cfg)
c(hybrids = nrow(st1$X), markers = ncol(st1$X),
  f_min = min(st1$f), f_max = max(st1$f))
#>    hybrids    markers      f_min      f_max
#>  625.00000 2000.00000    0.64475    0.83175
```

```
n_sel <- round(0.10 * nrow(st1$X))
res <- suppressWarnings(stage2_select(
  X = st1$X, f = st1$f, hybrids = st1$hybrids,
  n_sel   = n_sel,
  alphas  = NULL,     # NULL = automatic grid, 0 to the attainable ceiling
  weights = NULL,     # NULL = equal weight across traits
  fL      = st1$fL,   # optional: enables theta_A / theta_B / theta_AB
  B_null = 500, B_full = 80, NP = 100, itermax = 200, verbose = FALSE))
```

> **i** The same pipeline as a script
>
> This chapter runs at the book's reduced scale so it renders in seconds. The identical pipeline at the **production** scale of `sim_config`, writing its tables to `report/`, is `inst/scripts/run_all.R`:
>
> ```
> Rscript inst/scripts/run_all.R          # ~4 min
> Rscript inst/scripts/run_benchmark.R    # ~6 min; the numbers quoted in README.md
> ```

With real data only the first line changes:

```
st1 <- stage1_build(X = my_hybrid_matrix, ped = my_pedigree,
                    traits = my_predicted_traits, fL = my_line_coancestry)
```

24.1.1 The reference values

```
unlist(res$ref)
#>          gd_ref           gd_se       alpha_max           n_sel population_bias
#>    3.259845e-01    3.556567e-05    3.938109e-02    6.200000e+01    6.467346e-03
```

`alpha_max` is the attainable ceiling from Section 14.9: the loss pure truncation would incur. Any requested budget above it constrains nothing, and `stage2_select()` says so rather than returning a solution that merely looks constrained.

24.1.2 Metrics per scenario

```
fmt(res$metrics[, c("scenario", "alpha", "index", "Ns", "Ne_parents",
                    "Ne_lines_A", "Ne_lines_B", "max_line", "alleles_lost")], 4)
```

scenario	alpha	index	Ns	Ne_parents	Ne_lines_A	Ne_lines_B	max_line	alleles_lost
DE_alpha_0.00	0.0000	1.1004	0.7418	27.4571	12.4805	15.2540	10	63
DE_alpha_0.98	0.0098	1.4141	0.7383	22.3488	9.1962	14.2370	14	81
DE_alpha_1.97	0.0195	1.6767	0.7349	18.0894	7.3359	11.7914	16	106
DE_alpha_2.95	0.0295	1.8523	0.7314	15.8843	5.9505	11.9379	20	116
DE_alpha_3.94	0.0357	1.9600	0.7292	14.3701	5.2948	11.1744	21	125
ref_truncation	0.0394	1.9701	0.7280	13.4877	4.6878	12.0125	23	124
ref_trunc_cap	0.0028	0.7769	0.7408	31.6379	15.7541	15.8843	4	47
ref_max_div	−0.0131	0.1100	0.7465	36.9615	18.6602	18.3048	5	36
ref_random	0.0011	0.0682	0.7414	37.3204	17.9626	19.4141	6	31

Read the `index` column against `alpha`. That is the price list: what each percentage point of diversity costs in index units, on this dataset, at this selection intensity.

Now read `Ne_parents` against `Ns` in the same rows. This is the diagnostic Chapter 12. called for, and the reason `Ne_parents` is a second constraint rather than a report line:

```
d <- res$metrics
fmt(data.frame(scenario = d$scenario,
               Ns = d$Ns,
               Ne_parents = d$Ne_parents,
               Ns_relative = d$Ns / max(d$Ns),
               Ne_par_relative = d$Ne_parents / max(d$Ne_parents)), 3)
```

scenario	Ns	Ne_parents	Ns_relative	Ne_par_relative
DE_alpha_0.00	0.742	27.457	0.994	0.736
DE_alpha_0.98	0.738	22.349	0.989	0.599
DE_alpha_1.97	0.735	18.089	0.984	0.485
DE_alpha_2.95	0.731	15.884	0.980	0.426
DE_alpha_3.94	0.729	14.370	0.977	0.385
ref_truncation	0.728	13.488	0.975	0.361
ref_trunc_cap	0.741	31.638	0.992	0.848
ref_max_div	0.747	36.962	1.000	0.990
ref_random	0.741	37.320	0.993	1.000

The status number barely moves across scenarios while the effective number of parents collapses. A programme reporting only N_s would see a healthy pool and a shrinking parent base it never measured.

24.1.3 Both lenses, and the pool partition

```
fmt(res$metrics[, c("scenario", "alpha", "F_hom", "F_drift", "cov_diag",
                    "rare_retained", "GD_WI_hyb", "GD_BS", "F_ST")], 4)
```

scenario	alpha	F_hom	F_drift	cov_diag	rare_retained	GD_WI_hyb	GD_BS	F_ST
DE_alpha_0.00	0.0000	0.0038	0.0161	−0.0123	0.8636	0.1834	0.0393	0.1205
DE_alpha_0.98	0.0098	0.0132	0.0248	−0.0116	0.8312	0.1829	0.0417	0.1293
DE_alpha_1.97	0.0195	0.0224	0.0348	−0.0124	0.7727	0.1828	0.0452	0.1414
DE_alpha_2.95	0.0295	0.0352	0.0422	−0.0069	0.7143	0.1818	0.0471	0.1490
DE_alpha_3.94	0.0357	0.0377	0.0490	−0.0113	0.7143	0.1821	0.0499	0.1586
ref_truncation	0.0394	0.0412	0.0534	−0.0121	0.7338	0.1822	0.0514	0.1640
ref_trunc_cap	0.0028	0.0154	0.0114	0.0039	0.7273	0.1816	0.0370	0.1138

scenario	alpha	F_hom	F_drift	cov_diag	rare_retained	GD_WI_hyb	GD_BS	F_ST
ref_max_div	−0.0131	−0.0124	0.0063	−0.0187	0.9481	0.1853	0.0362	0.1096
ref_random	0.0011	0.0081	0.0066	0.0015	0.9481	0.1805	0.0357	0.1097

`alpha` constrains the first lens only. `F_drift` is the unmonitored bill, and `cov_diag` is exactly the gap between them – verify it:

```
c(max_deviation_from_Eq3 =
    max(abs(d$cov_diag - (d$F_hom - d$F_drift)), na.rm = TRUE))
#> max_deviation_from_Eq3
#>           6.678685e-17
```

`GD_BS` rising across scenarios is selection concentrating on the most complementary line pairs, which pushes the pools further apart. That is not automatically bad – it is heterosis – but Chapter 16. is the reason to watch it rather than celebrate it.

24.1.4 Distance from the random null

```
fmt(res$z_scores[, c("scenario", "z_GD", "z_Ne_parents", "z_F_drift",
                     "z_rare_retained", "z_GD_BS", "z_ANE")], 2)
```

scenario	z_GD	z_Ne_parents	z_F_drift	z_rare_retained	z_GD_BS	z_ANE
DE_alpha_0.00	0.00	−5.63	7.71	−4.27	6.57	6.96
DE_alpha_0.98	−5.20	−8.56	14.78	−5.69	11.05	9.97
DE_alpha_1.97	−10.35	−11.00	22.79	−8.24	17.33	19.86
DE_alpha_2.95	−15.63	−12.26	28.76	−10.80	20.87	17.76
DE_alpha_3.94	−18.88	−13.13	34.27	−10.80	25.81	16.12
ref_truncation	−20.85	−13.64	37.81	−9.94	28.51	9.80
ref_trunc_cap	−1.47	−3.23	3.95	−10.23	2.44	17.27
ref_max_div	6.91	−0.18	−0.16	−0.57	0.96	5.91
ref_random	−0.59	0.03	0.08	−0.57	0.12	−0.43

Every metric in standard deviations from a random selection of the same size. A metric near zero is not distinguishing this plan from chance, and cannot be serving as a constraint.

24.1.5 The selection table

```
scn <- setdiff(names(res$selection), c(names(st1$hybrids), "n_scenarios"))
head(res$selection[order(-res$selection$n_scenarios),
                   c("hybrid", "line_A", "line_B", scn, "n_scenarios")], 8)
```

hybrid	line_A	line_B	DE_alpha_0.00	DE_alpha_0.98	DE_alpha_1.97	DE_alpha_2.95	DE_alpha_3.94	ref_truncation	ref_trunc_cap	ref_max_div	ref_random	n_scenarios
35	35	10	27	1	1	1	1	1	1	1	0	8
23	23	7	35	1	1	1	1	1	1	1	0	8
66	66	6	50	1	1	1	1	1	1	1	0	8
31	31	6	27	0	1	1	1	1	1	1	0	7
32	32	7	27	1	1	1	1	1	1	0	0	7
57	57	7	28	1	0	1	1	1	1	0	1	7
82	82	7	29	1	1	1	1	1	1	1	0	7
90	90	15	29	0	1	1	1	1	1	1	0	7

```
de_cols <- grep("^DE_", scn, value = TRUE)
c(scenarios          = length(scn),
  chosen_in_all_DE   = sum(rowSums(res$selection[, de_cols, drop = FALSE]) ==
                           length(de_cols)),
```

```
  never_chosen       = sum(res$selection$n_scenarios == 0),
  total_candidates   = nrow(res$selection))
#>        scenarios chosen_in_all_DE     never_chosen total_candidates
#>                9               18              386              625
```

`n_scenarios` is the most directly useful column in the whole output. Hybrids chosen under **every** diversity budget are robust to the choice of α – they are the plan's backbone, and the argument for them does not depend on a policy number nobody wants to defend. Hybrids that appear only at the loosest budget are the ones being bought with diversity.

24.2 Reading the frontier

```
plot(d$alpha * 100, d$index, type = "b", pch = 19, col = "#b2182b", lwd = 2,
     xlab = "gene diversity loss, alpha (%)", ylab = "mean index (s.d.)")
text(d$alpha * 100, d$index, d$scenario, pos = 4, cex = 0.65, col = "grey30")
abline(v = 0, lty = 3)
grid()
```

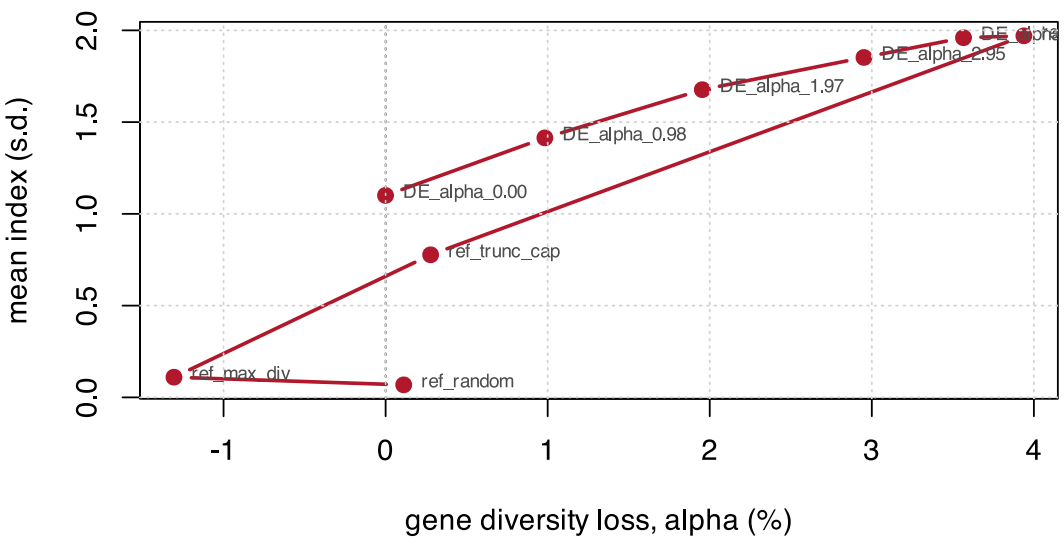

Figure 24.1: The price list. Each point is one diversity budget; the curve is what index it buys.

Choose the knee, not the endpoint. Past the knee, each additional unit of index costs disproportionately more diversity, and the diversity is not recoverable next cycle while the index gain is realised once.

24.3 What this simulation cannot tell you

Stated plainly, because the numbers above are precise and precision invites more confidence than the model supports.

The GEBVs carry no prediction error. The traits are built from *known* marker effects (Chapter 4.). Chapter 15. Finding 5 measures what happens when they do not: the reported index is biased upward, and error *correlated with relatedness* costs several times the diversity that independent error of the same size does. The plans below should be read as an optimistic ceiling.

The model is additive. An additive $\mathbf{G}$ misses heterosis and specific combining ability, which is precisely what a hybrid programme sells. The diversity constraint does not need a dominance model; the *mean-performance ranking* does (Chapter 13.). Do not conflate them, and do not use these `index` values as performance predictions.

The pools are symmetric by construction. Both are simulated from the same ancestral frequencies at the same F_{ST}. Real heterotic pools differ in size, history and internal structure, and a real programme will find one pool eroding faster than the other – which is why `ne_lines_pool()` is reported per pool.

N_s is not a drift projection. It is a static descriptor (Section 14.3). Do not extrapolate it forward.

One cycle, not a trajectory. Every result here is a single round of hybrid selection. Chapter 16. showed that a constraint applied at one stage buys stage-level diversity, not end-of-pipeline richness – and the same logic applies across cycles. A one-cycle constraint is not a multi-cycle policy: Chapter 22. runs fifteen of them and finds the unconstrained programme ahead for eight cycles and behind for the rest.

p_0 must be frozen. In this book it is set once in `build_ctx()`. In a real multi-cycle programme it must be the founder cycle's frequencies, propagated forward – the single most common error in applying `F_hom` and `F_drift` (Chapter 12.). Chapter 22. shows the failure directly: re-anchored, five generations of real drift are reported as exactly none.

24.4 The operational checklist

24.4.1 Setting up

- The base p_0 is frozen at the founding cycle and documented.
- Diversity is computed on molecular coancestry `f`, never on a centered `G`. Check `sum(G) == 0` once, to see why.
- The marker panel's MAF spectrum is known – it decides how severely the two lenses diverge.
- Heterotic-group assignment is recorded, with the method that produced it, and treated as uncertain.

24.4.2 Choosing the budget

- The attainable α ceiling is computed *before* choosing $\alpha_{\max}$. A budget above the ceiling constrains nothing.
- Loss is measured against a **same-size resampling baseline**, never against the full pool.
- The constraint is applied at the target set size, not incrementally.
- The frontier is traced and the knee chosen, rather than a round number.

24.4.3 Running the optimiser

- The search is encoded on **line weights**, not hybrid keys.
- The differential evolution is **warm-started** from greedy solutions.
- $N_{e,\mathrm{par}}$ is a second constraint, not a report line.
- The per-line usage cap is enforced in the decoder, not by penalty.
- The result is checked against the continuous relaxation. A large gap means the optimiser has not converged; a slack constraint means the search never reached the boundary.

24.4.4 Validating the plan

- `F_hom` and `F_drift` are computed **separately** and reported together, with `cov_diag` as the diagnostic.
- Allelic richness is computed on the returned plan – it is the one metric carrying information θ does not.
- The within/between pool decomposition confirms diversity was not preserved by degrading pool separation.
- The number of selected parent lines is biologically plausible.
- N_s is reported; a rate-based N_e is **not**, under active management.
- Every metric is reported against the random null, in standard deviations.

24.4.5 Across cycles

- The per-stage loss ledger is kept. DH induction, not hybrid mining, is usually where the diversity goes.
- The regression of $\log(1 - F)$ on cycle is checked for linearity. Curvature means a non-constant rate.

- Periodic germplasm injection is planned. A closed elite pool depletes however well the constraint is managed.
- The diversity cost per cycle is reported when cycle time is shortened.

24.5 Where to go next

Three extensions the material here supports but does not cover, each needing new work rather than new derivation:

Real data ingestion. `load_data()` is the single swap point and is currently a stub. A worked path from a VCF to the `X`/`f`/`hybrids` contract is what most readers will need first, and it is the one extension here that is pure engineering rather than new derivation.

Estimating the dominance deviations. Chapter 13. shows that expected heterosis depends on the alignment between divergence and d, and that no frequency-based statistic can see d. Everything downstream of that – ranking crosses on more than the mean, putting a number on the λ of Chapter 14. – waits on an estimate of d the book does not attempt.

Prediction error compounded across cycles. Chapter 15. Finding 5 measures it within one cycle and Chapter 22. assumes it away (C7). The two have not been run together, and the interaction is not obviously benign: correlated error concentrates the selection, and Chapter 22. shows concentration is what erodes the programme's future.

Reproduce this chapter

All the code of this chapter, in order and with the explanation kept as comments, is one script: **12-case-study.R**. Download it, install `hybdiv`, and run it top to bottom in an empty directory; cached results are fetched on first use.

Appendices

A. Notation and glossary

A.1 Symbols

Symbol	Meaning	In the code
N	candidate hybrids in the pool	`ctx$N`
n, n_{sel}	hybrids selected	`n_sel`
L	parent lines	`ctx$n_lines`
m	markers	`ctx$m`
x_{ik}	within-individual reference-allele frequency of individual i at marker k, in 0/0.5/1	`ctx$X[i, k]`
$\mathbf{X}$	hybrid marker matrix, N x m	`ctx$X`
$\mathbf{M}$	hybrid dosage matrix, 0/1/2	-
$\mathbf{G}^L$	line genotype matrix, L x m, coded 0/1	`ctx$GL`
p_k, $\bar{p}_k$	allele frequency; the bar denotes a group mean	-
$p_{0,k}$	**frozen** candidate-pool allele frequency: the base	`ctx$p0`

A.1.1 Matrices

Symbol	Meaning	Note
$\mathbf{f}$	molecular coancestry of the **hybrids**, N x N, in [0, 1]	the diversity kernel; never centered
$\mathbf{f}^L$	molecular coancestry **between lines**, L x L	the sufficient statistic in disguise – see the collapse
$\mathbf{G}$	VanRaden genomic relationship, centered – prediction only	sum of all elements is exactly 0; unusable for diversity
$\mathbf{A}$	pedigree relationship	expectation of IBD, no markers
$\mathbf{G}_{0.5}$	VanRaden with the centering frequency fixed at 0.5	**this is f**, arrived at from the VanRaden family

A.1.2 Coancestry and diversity

Symbol	Definition	Function
θ_S	$n^{-2}\sum_i\sum_j f_{ij}$, diagonal **included**	`theta_group()`, `fast_theta()`
GD, H_e	$1-\theta$; these are the same quantity	`gene_diversity()`, `he_nei()`
N_s	$1/(2\theta)$	`status_number()`
$N_{e,\text{par}}$	$1/\sum_i p_i^2$ over parent-line usage	`ne_parents()`
f_{ii}	coancestry within subpopulation i; $1-f_{ii}=H_{e,i}$	`ct_partition()`
s_i	self-coancestry; $F_i=2s_i-1$	`ct_partition()`
$\tilde{x}$	weighted mean **within** subpopulations	-
$\bar{x}$	mean over **all pairs**, between-subpopulation pairs included	-

Always $\bar{f}\le\tilde{f}\le\tilde{s}$. The differences between those three numbers *are* the partition of diversity.

A.1.3 The partition

Component	Definition	Meaning	In hybrid breeding
GD_T	$1 - \bar{f}$	total	the pool's diversity
GD_{WI}	$1 - \tilde{s}$	within individuals	0 for inbred lines; use `GD_WI_hyb` for the F1s
GD_{BI}	$\tilde{s} - \tilde{f}$	between individuals, within subpopulation	segregating variation available to selection
GD_{WS}	$1 - \tilde{f}$	within subpopulations	what erodes – defend it
GD_{BS}	$\tilde{f} - \bar{f}$	between subpopulations	the heterosis engine – **maintain**, do not maximise (Chapter 13.)

with $GD_T = GD_{WI} + GD_{BI} + GD_{BS}$ and $F_{ST} = GD_{BS}/GD_T$.

A.1.4 Heterosis and cycles

Symbol	Meaning	Function
d_k	dominance deviation at locus k	argument to `heterosis_value()`
H_i	mid-parent heterosis of hybrid i; $\sum_k d_k$ over its heterozygous loci	`heterosis_value()`
$\bar{p}_k$	$(p_{Ak} + p_{Bk})/2$, the merged-pool frequency	inside `heterosis_partition()`
y_k	$p_{Ak} - p_{Bk}$, the divergence at locus k	`pool_complementarity()$mean_sq_delta_p` is $\overline{y^2}$
g_i^A, g_j^B	general combining ability for heterosis; weighted by the **opposite** pool's frequencies	`heterosis_gca_sca()$gca_A`
s_{ij}	specific combining ability; $-2\sum_k d_k\, \delta u_{ik}\, \delta v_{jk}$	`heterosis_gca_sca()$sca`
σ^2_{SCA}	$4\sum_k d_k^2 p_{Ak} q_{Ak} p_{Bk} q_{Bk}$; falls as $(1 - F_{ST})^2$	closed form; see Section 13.11
λ	weight on within- against between-pool diversity	not implemented; see Chapter 14.
t	cycle index	`run_cycles()`
$p_0^{(0)}$	base frequencies, frozen at the founding cycle	`ctx$p0`, set once

Expected heterosis splits as $\sum_k d_k[\,2\bar{p}_k(1-\bar{p}_k) + y_k^2/2\,]$, whose two halves are GD_T and GD_{BS} when $d_k \equiv 1$, and whose sum is $1 - \theta_{AB}$ (Chapter 13.). Over a complete factorial the realised values split orthogonally as $H_{ij} = \mu + g_i^A + g_j^B + s_{ij}$.

A.1.5 Inbreeding, the three lenses

Symbol	Lens	Definition	Function
F_{hom}	homozygosity	$1 - \overline{H_t/H_0}$; **can be negative**	`freq_metrics()`
F_{drift}	drift	$\overline{\delta p^2/[p_0(1-p_0)]}$; never negative	`freq_metrics()`
F_{IBD}	identity by descent	segments inherited from a defined base	requires pedigree + linkage
F_{ROH}	hybrid	genome fraction in runs of homozygosity	not implemented
cov	-	$2\,\mathrm{cov}(\delta p/s, (p_0 - \frac{1}{2})/s) = F_{hom} - F_{drift}$	`freq_metrics()$cov_diag`

A.1.6 Selection and optimisation

Symbol	Meaning	Function
u_i	multi-trait index of hybrid i	`ctx$index`
α	$(GD_{\text{ref}} - GD_S)/GD_{\text{ref}}$; negative means more diverse than chance	`alpha_loss()`
$\alpha_{\max}$	the diversity budget – a policy decision	argument to `stage2_select()`
$\mathbf{c}$	continuous contributions, summing to 1	`ocs_quadprog()`

Symbol	Meaning	Function
$\mathbf{w}$	line-usage weights, $w_a = k_a/(2n)$	`line_weights()`, `line_usage_freq()`
k_a	times line a is used as a parent	`line_counts()`
λ	weight on the coancestry penalty in continuous OCS	`ocs_relaxation()`
σ^2_{DH}	progeny variance of a DH family	`sigma2_dh()`
UC	usefulness criterion, $\mu + ih\sigma$	`usefulness_criterion()`

A.1.7 Genomic prediction

Symbol	Meaning	Note
$\mathbf{u}$, $\hat{\mathbf{u}}$	breeding values; the BLUP of them	defined over **all** individuals, not just the phenotyped ones
β	marker effects; $\mathbf{u} = \mathbf{Z}\beta$	the same fit, two parametrisations – exactly (3)
σ^2_u, σ^2_e	genetic and residual variance	supplied here, never estimated; REML in practice
λ	σ^2_e/σ^2_u – **not** the OCS penalty above	a variance ratio, not a tuning knob
h^2	$\sigma^2_u/(\sigma^2_u + \sigma^2_e)$	the one number the whole fit is conditioned on
r	accuracy: $\mathrm{cor}(\hat{u}, g)$, needs the truth	meaningless without its CV scheme (Section 17.4)
r^2_i	reliability: $1 - \mathrm{PEV}_i/(\sigma^2_u \mathbf{G}^*_{ii})$, does not	the early warning: it collapses before the accuracy does
PEV_i	prediction error variance, from the inverse of the MME	$\sigma^2_e[\mathbf{C}^{uu}]_{ii}$
π	prior probability a marker has no effect (BayesB)	BayesCπ estimates it; at book scale, badly
ν, S	degrees of freedom and scale of the χ^{-2} prior; `df`, `R2`	hyperparameters, not fitted
M_e	effective number of independent chromosome segments	unlinked markers here, so M_e is the marker count

CV0, CV1, CV2 name what the training set had already seen of a candidate hybrid's *parents*: neither, one, or both. In a factorial between two pools these are three different questions, and only CV2 is easy – see Section 17.4.

A.1.8 Population sizing

Symbol	Meaning	Function
H	panel heterozygosity; 0.125 in an F4	`panel_het()`
D	diversity retained, $\overline{2p(1-p)}/0.5$	`diversity_retained()`
N_{eq}	$(1-H)/(1-D)$ – a communication device, **not** a Wright Ne	`n_equivalent()`
L (Morgans)	total map length	`map$total_cM / 100`
A_t	junctions per haplotype at generation t	`expected_junctions()`
r	recombination fraction, Haldane	`build_map()`

A.1.9 Mating design

Symbol	Meaning	In the code
g^A_i, g^B_j	general combining ability of a pool-A / pool-B line	`fit_gca()$gA`, `$gB`
s_{ij}	specific combining ability of the cross	`simulate_factorial()$S`

Symbol	Meaning	In the code
k	testers per pool	`design_tester(k =)`
c	crosses per line in a sparse factorial	`design_circulant(c =)`
r	plots per cross	`design_reps()`
ρ	least-squares GCA accuracy, $\sqrt{\sigma_g^2/(\sigma_g^2 + \sigma_s^2/c + \sigma_e^2/(cr))}$	`gca_accuracy_theory()`
T0 / T1 / T2	untested hybrids with neither / one / both parents in the training set	-

r here is plots per cross; in the next table it is the accuracy of a selection criterion. Chapter 9. keeps ρ for accuracy to avoid the clash.

A.1.10 Sparse testing

Symbol	Meaning	In the code
L	locations in the trial	`nLoc`
k	locations per hybrid (plots per hybrid, one plot per cell)	`sparse_alloc(k =)`
p	share of hybrid x location cells observed, k/L	`p_real`
CV1 / CV2 / CV0 / CV00	new genotype / missing cell of a tested genotype / new environment / both new	`acc_cv1`, `acc_cv2`
PEV	prediction error variance of a hybrid's target value	`sparse_blup()$pev`
b_1	slope of truth on prediction; 1 = no dispersion bias	`b1`
σ^2_{res}	residual of the analysis model, $\sigma^2_{SCA\times L} + \sigma^2_e$ under one plot per cell	`sparse_vc()$s2res`

Here k counts locations per hybrid; in Chapter 9. it counts testers per pool.

A.1.11 Response to selection

Symbol	Meaning	In the code
R	response to one cycle of selection	-
i	selection intensity for a selected fraction	`sel_intensity()`
r	accuracy of the selection criterion	-
σ_A	additive standard deviation	-
V_{genic}	variance if the loci were independent; the realised variance minus this is the disequilibrium term	`genic_var()`
β_k	marker effect at locus k	`sim$beta`
d_k	dominance deviation at locus k	`d`
ID	inbreeding depression, $2F\sum d_k p_k q_k$; a **constant** with fully inbred parents	`inbreeding_depression()`
$\mathbf{b}$	selection index weights	`index_smith_hazel()`
$\mathbf{a}$	economic weights on the aggregate genotype	-
$\mathbf{R}$	restriction matrix: directions held at zero expected gain	`index_restricted()`
$2N_eR_1$	Robertson's limit; half-life $1.4N_e$	-

$R = i\,r\,\sigma_A$ is a bivariate-normal result and runs above the realised response in a structured hybrid population – see Chapter 3., which measures the size of the gap rather than assuming it away.

A.2 Conventions

Coancestry always includes the diagonal. Group coancestry in the sense of Cockerham. That is what makes $1 - \theta$ exactly Nei's H_e and makes the algebra close. Drop the diagonal and none of the identities hold.

Everything is computed per locus and then averaged over loci. Never average frequencies first.

Marker coding. Lines are 0/1 (or 0/2) because they are homozygous. Hybrids are 0/1/2 as dosage, rescaled to 0/0.5/1 as within-individual frequency. `stage1_build()` auto-detects and rescales.

`f` is never centered, `G` always is. A negative coancestry is a symptom of the wrong matrix.
Loss is always relative to a same-size random baseline. Never to the full pool.

A.3 Terms

Balding-Nichols model. A Beta distribution with mean p and variance $F_{ST}\,p(1-p)$, used to draw pool-specific allele frequencies from an ancestral frequency. The parameter *is* the expected divergence.

Group coancestry. The probability that two alleles sampled at random from a whole set are identical in state, including the case of sampling the same individual twice.

Heterotic group / pool. A set of lines maintained as a breeding unit, crossed to lines of a complementary group to produce hybrids. Divergence between groups is where heterosis comes from – but only the divergence that coincides with dominance. Divergence produced by drift alone generates none, because it is paid for exactly out of the within-pool term. See Chapter 13..

Mid-parent heterosis. The F1′s value minus the mean of its two parents. With inbred parents the additive contributions cancel exactly, leaving only the dominance deviations at the F1′s heterozygous loci.

General and specific combining ability (GCA, SCA). The two halves of a hybrid's value that a two-way layout separates: what a line contributes to every cross it enters, and what depends on which two lines met. For heterosis both are exact functions of d and the pool frequencies, with GCA weighted by the *opposite* pool's frequencies – so combining ability is a property of a line against a tester pool, never of the line alone. See Section 13.10.

Reciprocal recurrent selection. Selecting on performance *in cross* and recycling the winners within their own pool. Selection information crosses the pool boundary; germplasm does not. See Chapter 22..

Identity by state (IBS) vs by descent (IBD). IBS is what markers observe: two alleles are the same allele. IBD adds the requirement of common ancestry from a defined base. The gap between them is where Chapter 12.'s whole argument lives.

Line-level collapse. The exact identity $\theta_S = \mathbf{w}'\mathbf{f}^L\mathbf{w}$ for F1 hybrids of homozygous lines, which removes the need for the $N \times N$ hybrid matrix. See Section 15.1.

Status number. $N_s = 1/(2\theta)$, the number of unrelated non-inbred individuals with the same gene diversity. A static descriptor.

Truncation selection. Take the top n by index, ignore everything else. The ceiling on gain and the floor on diversity.

Warm start. Seeding an optimiser's initial population with heuristic solutions. Not optional here: without it the differential evolution finishes below a plain greedy.

Reproduce this chapter

All the code of this chapter, in order and with the explanation kept as comments, is one script: **90-notation.R**. Download it, install `hybdiv`, and run it top to bottom in an empty directory; cached results are fetched on first use.

B. The `hybdiv` package

Every function used in this book lives in one package, whose source is the `R/` directory of this repository. This appendix is the index.

```
#> exported_objects
#>              213
```

B.1 Installation

```
install.packages(c("DEoptim", "quadprog"))
devtools::install(".")        # from the repository root
library(hybdiv)
```

Only `DEoptim` is a hard dependency, used solely by `sel_de()`. `quadprog` is suggested and needed only for `optimal_contributions()` and `ocs_quadprog()`. Everything else, including every plot, is base R.

B.2 File map

File	Role	Chapter
`R/00_data.R`	simulation, the two matrices, the `ctx` builder	Chapter 4.
`R/01_metrics.R`	the diversity metrics, all `f(idx, ctx) -> scalar`	Chapter 14.
`R/02_benchmark.R`	null distribution, discriminatory power, cost, plots	Chapter 14.
`R/03_de_select.R`	selection strategies: truncation, greedy, DE, OCS relaxation	Chapter 19.
`R/04_fast_metrics.R`	the line-level fast route – no N x N matrix	Chapter 15.
`R/05_pipeline_metrics.R`	stage-specific metrics for S1 to S4	Chapter 16.
`R/06_f2size.R`	F1 to F4 population sizing simulator	Chapter 5.
`R/07_caballero_toro.R`	Caballero & Toro subdivided-population analysis	Chapter 2.
`R/08_mabc.R`	marker-assisted backcrossing: drag, the selection index, sizing	Chapter 6.
`R/09_reference.R`	slow literal oracles, for verification	Appendix C.
`R/10_stage1.R`	stage 1: lines to the X / f / hybrids contract	Chapter 4.
`R/11_stage2.R`	stage 2: constrained selection across an alpha grid	Chapter 24.
`R/12_search.R`	structure-aware search: local search, the bound, the exact oracle	Chapter 20.
`R/13_dominance.R`	heterosis under a dominance model, and its link to the partition	Chapter 13.
`R/14_cycles.R`	recurrent selection across cycles	Chapter 22.
`R/15_response.R`	genic variance, and the Smith-Hazel and restricted indices	Chapter 3.
`R/16_mating_design.R`	mating designs: generators, connectivity, LS/BLUP GCA, seed cost	Chapter 9.
`R/17_sparse_testing.R`	sparse testing: allocation, connectivity, MME BLUP/PEV	Chapter 10.

Files are numbered because the package has no `Collate` field and sourcing order is alphabetical.

B.3 The two-stage contract

Everything downstream of stage 1 talks only through three objects. This is the seam for swapping in real data.

```
st1 <- stage1_simulate()                # or stage1_build(X, ped, traits, fL)
#   $X        N x m hybrid markers, 0/0.5/1
#   $f        N x N molecular coancestry
#   $hybrids  N x (3 + n_traits): id, line_A, line_B, traits
#   $fL       L x L line coancestry (optional)

res <- stage2_select(st1$X, st1$f, st1$hybrids, n_sel = 100, fL = st1$fL)
#   $selection  0/1 column per scenario, plus n_scenarios
#   $metrics    one row per scenario
#   $z_scores   each metric in s.d. from the random null
#   $ref        gd_ref, its Monte Carlo s.e., alpha_max, n_sel
```

B.4 Function index

B.4.1 Simulation and context

Function	Does
`sim_config`	simulation scale: lines per pool, markers, traits, divergence
`simulate_lines()`	Balding-Nichols line genotypes in two divergent pools
`simulate_data()`	lines, all A x B crosses, both matrices, predicted traits
`simulate_pools()`	lighter simulator returning raw pieces, with `GL` in 0/1
`build_ctx()`	assemble the context; freezes `p0` and the index
`build_ctx_from_stage1()`	same, from the stage-1 contract
`ctx_from_pools()`	context from `simulate_pools()`, carrying `parents` and `GL`
`load_data()`	the single swap point for real data
`stage1_simulate()` / `stage1_build()`	produce the stage-1 contract
`stage2_select()`	the whole constrained selection pipeline

B.4.2 The two matrices

Function	Does
`molecular_coancestry()`	**the diversity kernel**; one `tcrossprod`, stays in [0, 1]
`vanraden_G()`	**prediction only**; `sum(G) == 0` by construction
`mrd_matrix()` / `coancestry_from_mrd()`	modified Rogers distance, the same statistic

B.4.3 Diversity metrics – all `f(idx, ctx) -> scalar`

Function	Does
`theta_group()`	group coancestry, diagonal included
`gene_diversity()` / `he_nei()`	`1 - theta`; the same quantity, two routes
`theta_from_freq()`	theta via allele frequencies – validation only, ~100x slower
`status_number()`	`Ns = 1/(2 theta)`; a static descriptor
`ne_parents()`	effective number of parent lines; independent of `f`
`ne_lines_pool()`	the same, per pool; needs only the pedigree
`alleles_lost()`	markers polymorphic in the pool, rare in the selection
`freq_metrics()`	**one marker sweep**: He, F_hom, F_drift, cov_diag, rare_retained
`line_weights()` / `theta_pools()`	per-pool usage weights and coancestries
`gd_partition()`	Caballero & Toro Eq. 8 over the heterotic pools
`ene()` / `ane()`	Core Hunter entry- and accession-to-nearest-entry

Function	Does
`eff_dim()`	effective dimensionality, on the **double-centered** submatrix
`gst()`	Nei's Gst, selected against non-selected
`mean_offdiag()`	the common production metric, kept for comparison
`metrics_cheap()` / `metrics_full()`	the inner-loop panel and the post-hoc panel

B.4.4 The fast line route

Function	Does
`fast_ctx()`	add `parents`, `GL` and the packed bitset to a context
`line_counts()` / `line_usage_freq()`	the sufficient statistic
`fast_theta()` / `fast_gene_div()` / `fast_status_num()`	`theta = w' fL w`; no N x N matrix
`make_theta_fun()`	precompiled closure for ~1e6 evaluations
`theta_state()` / `theta_swap()`	O(L) incremental update per swap
`fast_ne_parents()` / `ne_parents_from_counts()`	free once the counts exist
`fast_pbar()` / `fast_he_nei()` / `fast_alleles_lost()`	frequencies from the line matrix
`pack_lines()` / `popcount32()` / `alleles_retained_fast()`	bitset allelic richness
`fast_theta_pools()`	pool decomposition from the same counts
`decode_rank()` / `decode_lines()`	the two encodings; **use `decode_lines()`**
`make_fitness_fast()`	DE fitness with several simultaneous restrictions

B.4.5 Selection strategies

Function	Does
`sel_random()`	the null strategy
`sel_truncation()`	ceiling on gain, floor on diversity
`sel_truncation_cap()`	per-line cap; no matrix needed, usually most of the benefit
`sel_greedy()`	incremental greedy; `w = 0` is max diversity
`decode()` / `make_fitness()` / `sel_de()`	differential evolution, warm-started by default
`project_capped_simplex()` / `ocs_relaxation()`	the continuous upper bound
`optimal_contributions()` / `ocs_quadprog()`	quadratic-programming OCS

B.4.6 Benchmarking

Function	Does
`null_distribution()`	**the correct baseline**: random subsets of the same size
`alpha_loss()`	proportional diversity loss against that baseline
`evaluate()` / `discriminatory_power()`	metric panel and its z-scores
`cost_per_eval()`	microseconds per metric; decides what goes inside the loop
`plot_null()` / `plot_correlation()` / `plot_frontier()`	base-R diagnostics; run by `inst/scripts/run_benchmark.R`, not by the book

B.5 Runnable scripts

The book runs everything at a reduced scale so it renders quickly. Three scripts ship with the package and run the same work at the production scale of `sim_config`, writing their tables and plots to `report/`:

```
Rscript inst/scripts/run_all.R          # the two-stage pipeline, ~4 min
Rscript inst/scripts/run_benchmark.R    # the metrics benchmark, ~6 min
Rscript inst/scripts/run_mabc.R         # the MABC scans, ~3 min
```

`run_benchmark.R` is the source of the numeric results quoted in `README.md`, and the only place the three `plot_*` helpers above are exercised. From an installed copy of the package:

```
source(system.file("scripts", "run_all.R", package = "hybdiv"))
```

B.5.1 Pipeline stages S1-S4

Function	Does
`theta_decompose()` / `gst_nei()`	**both** F_ST definitions, and Nei's Gst
`pool_freqs()`	allele frequencies per pool, one row each
`pool_complementarity()` / `s1_monitor()`	the S1 cycle panel; `mean_sq_delta_p` is the exact one
`dh_sib_coancestry()` / `dh_cross_coancestry()` / `dh_pool_theta()`	the three DH identities
`sigma2_dh()` / `sigma_dh_crosses()`	progeny variance from marker effects
`usefulness_criterion()` / `sel_intensity()`	UC – applies at S2, not S4
`s3_theta()` / `alleles_retained()`	line-level metrics
`loss_vs_resampling()` / `theta_ceiling()`	calibrate against a same-size baseline
`select_lines_constrained()`	truncate-then-repair; the incremental greedy fails
`hybrid_theta_from_usage()` / `line_usage()` / `pool_balance()`	S4 on the line route
`loss_ledger()`	loss per stage, on one scale

B.5.2 Heterosis and cycles

Function	Does
`heterosis_value()`	mid-parent heterosis of each hybrid, from `d` alone
`heterosis_expected()`	the same, expected over a pool pair
`heterosis_partition()`	shared and divergence terms – `GD_T` and `GD_BS`
`run_cycles()`	recurrent selection, one row per cycle
`next_pool()`	one generation of a pool: weighted intermating, DH derivation
`sel_greedy_budget()`	greedy sweep returning the best plan inside a budget

B.5.3 Mating design

Function	Does
`design_tester()` / `design_random_sparse()` / `design_circulant()` / `design_nc2()`	crossing plans over the two-pool factorial
`design_reps()`	spread a plot budget over the crosses
`design_diagnostics()` / `design_estimable()`	connectivity, rank = n' - components, df for SCA
`gca_accuracy_theory()`	least-squares GCA accuracy in closed form
`simulate_factorial()` / `simulate_cross_means()`	true GCA/SCA and the cross means a plan produces

Function	Does
`fit_gca()`	least squares or BLUP with known components
`cross_cost_par()` / `cross_cost()`	seed production and cost; **placeholder** levels

B.5.4 Sparse testing

Function	Does
`sparse_alloc()`	hybrid x location allocation: complete, random, parentage
`sparse_connectivity()`	parent coverage per location, concurrence, disconnected pairs
`sparse_vc()` / `relmat_halfsib()`	calibrated variance components; half-sib relationships
`sparse_simulate()`	true values and plot phenotypes for an allocation
`sparse_blup()`	mixed-model BLUP and PEV with known components
`ref_sparse_blup()`	the literal GLS oracle of `sparse_blup()`

B.5.5 Caballero & Toro

Function	Does
`ct_partition()`	subpopulation coancestries, F statistics, the Eq. 7 partition
`ct_loss_gain()`	loss **or gain** from removing each subpopulation

B.5.6 Population sizing

Function	Does
`maize_chr_len` / `f2_distorters` / `f2_opt`	defaults; **replace the distorters** with your data
`build_map()` / `marker_index()`	Haldane map on a regular grid
`meiosis()` / `meiosis_sel()` / `cross_indices()` / `make_F1()`	the vectorised engine
`make_distortion()` / `zyg_fitness()`	gametophytic and zygotic distortion, by exact rejection
`run_scheme()` / `reps_for_N()` / `bulk_weights()`	SSD, bulk, Syn schemes
`panel_freq()` / `panel_het()` / `diversity_retained()`	panel metrics
`min_minor_freq()` / `fixed_fraction()` / `skew_fraction()`	worst-region metrics
`n_equivalent()`	equivalent N – a communication device, **not** a Wright Ne
`block_lengths()` / `junctions_per_hap()` / `expected_junctions()`	map expansion
`multilocus_recovery()` / `f1_sizing()`	the objectives that keep paying for larger N
`f2_run_experiment()`	the full grid; minutes

B.5.7 Reference oracles

Function	Does
`ref_theta()`, `ref_gene_div()`, `ref_theta_freq()`, `ref_he_nei()`	literal transcriptions
`ref_status_num()`, `ref_ne_parents()`, `ref_alleles_lost()`, `ref_alleles_fixed()`	-
`ref_ene()`, `ref_ane()`, `ref_eff_dim()`, `ref_theta_pools()`	-

B.6 Full export list

alleles_lost	f1_sizing	mean_index	run_bc_program
alleles_retained	f2_distorters	mean_offdiag	run_cycles
alleles_retained_fast	f2_opt	meiosis	run_scheme
alpha_loss	f2_run_experiment	meiosis_sel	s1_monitor
alphasimr_founder_pools	fast_alleles_lost	metrics_cheap	s3_status_num
alphasimr_gd	fast_ctx	metrics_full	s3_theta
alphasimr_make_hybrids	fast_gene_div	min_minor_freq	sel_de
alphasimr_pipeline	fast_he_nei	molecular_coancestry	sel_de_fast
alphasimr_rrs_cycle	fast_ne_parents	mrd_matrix	sel_exact
alphasimr_to_stage1	fast_pbar	multilocus_recovery	sel_greedy
ane	fast_status_num	n_equivalent	sel_greedy_budget
backcross	fast_theta	n_for_count	sel_intensity
BITS_PER_WORD	fast_theta_pools	ne_lines_pool	sel_local_search
block_lengths	fit_gca	ne_parents	sel_random
build_ctx	fixed_fraction	ne_parents_from_counts	sel_truncation
build_ctx_from_stage1	foreground_ok	next_pool	sel_truncation_cap
build_map	freq_metrics	null_distribution	select_lines_constrained
bulk_weights	gca_accuracy_theory	ocs_bound	select_parent
coancestry_from_mrd	gd_partition	ocs_quadprog	selfing_step
col_cumsum	gene_diversity	ocs_relaxation	sigma_dh_crosses
cost_per_eval	genic_var	ocs_round	sigma2_dh
cp_hi	gst	optimal_contributions	sim_config
cp_lo	gst_nei	p_het	simulate_cross_means
cross_cost	haldane_r	p_same_segment	simulate_data
cross_cost_par	he_nei	pack_lines	simulate_factorial
cross_indices	heterosis_expected	panel_freq	simulate_lines
ct_loss_gain	heterosis_gca_sca	panel_het	simulate_pools
ct_partition	heterosis_partition	plot_correlation	sizing_table
ctx_from_pools	heterosis_value	plot_frontier	skew_fraction
decode	hybrid_theta_from_usage	plot_null	sparse_alloc
decode_lines	ibd_elite	pool_balance	sparse_blup
decode_rank	inbreeding_depression	pool_complementarity	sparse_connectivity
design_circulant	index_restricted	pool_freqs	sparse_simulate
design_diagnostics	index_smith_hazel	popcount32	sparse_vc
design_estimable	junctions_per_hap	project_capped_simplex	stage1_build
design_nc2	line_counts	ref_alleles_fixed	stage1_simulate
design_random_sparse	line_usage	ref_alleles_lost	stage2_select
design_reps	line_usage_freq	ref_ane	status_number
design_tester	line_weights	ref_eff_dim	strat_rec_until
dh_cross_coancestry	load_data	ref_ene	summarise_reps
dh_pool_theta	loss_ledger	ref_gene_div	target_indices
dh_sib_coancestry	loss_vs_resampling	ref_genic_var	theta_ceiling
discriminatory_power	mabc_opt	ref_he_nei	theta_decompose
diversity_retained	maize_chr_len	ref_heterosis	theta_from_freq
donor_dosage	maize_targets	ref_inbreeding_depression	theta_group
donor_specific	make_distortion	ref_ne_parents	theta_pools
drag_total	make_F1	ref_sparse_blup	theta_state

eff_dim	make_fitness	ref_status_num	theta_swap
ene	make_fitness_fast	ref_theta	usefulness_criterion
evaluate	make_shared_ibd	ref_theta_freq	vanraden_G
expected_donor	make_theta_fun	ref_theta_pools	zyg_fitness
expected_drag	marker_index	relmat_halfsib	
expected_ibd	marker_weights	reps_for_N	
expected_junctions	max_line_use	rrs_config	

Reproduce this chapter

All the code of this chapter, in order and with the explanation kept as comments, is one script: **91-package.R**. Download it, install `hybdiv`, and run it top to bottom in an empty directory; cached results are fetched on first use.

C. Verification

A derivation the reader cannot re-run against a naive implementation is a claim, not a verification. This appendix says how each result in the book was checked, and how to re-run the checks.

C.1 The method: slow oracles

Every fast route in the package has a **deliberately slow, literal** counterpart in `R/09_reference.R` that transcribes the definition with no algebraic shortcut. The fast route must match it to machine precision.

```
ref_theta
#> function (idx, ctx)
#> mean(ctx[["f"]][idx, idx])
#> <bytecode: 0x799d808e40>
#> <environment: namespace:hybdiv>
theta_group
#> function (idx, ctx)
#> mean(ctx$f[idx, idx])
#> <bytecode: 0x799d844318>
#> <environment: namespace:hybdiv>
fast_theta
#> function (idx, ctx)
#> {
#>     cnt <- line_counts(idx, ctx)
#>     u <- which(cnt > 0L)
#>     cu <- cnt[u]
#>     s <- sum(cu)
#>     as.numeric(crossprod(cu, ctx$fL[u, u, drop = FALSE] %*% cu))/(s *
#>         s)
#> }
#> <bytecode: 0x799d87bcb0>
#> <environment: namespace:hybdiv>
```

Three implementations of one quantity: a literal mean over the submatrix, the packaged version, and the line-route collapse that never forms the submatrix at all. They must agree.

```
sim  <- simulate_pools(n_A = 20, n_B = 20, m = 1500, seed = 7)
ctxp <- ctx_from_pools(sim)
set.seed(3); sel <- sample.int(ctxp$N, 50)

c(oracle     = ref_theta(sel, ctxp),
  packaged   = theta_group(sel, ctxp),
  line_route = fast_theta(sel, ctxp),
  max_dev    = max(abs(c(theta_group(sel, ctxp), fast_theta(sel, ctxp)) -
                       ref_theta(sel, ctxp))))
#>       oracle     packaged   line_route      max_dev
#> 6.708316e-01 6.708316e-01 6.708316e-01 1.110223e-16
```

The oracles exist so that this comparison is possible for a reader who does not trust the derivation – which is the correct posture.

```
fmt(rbind(
  he_nei       = c(oracle = ref_he_nei(sel, ctxp), packaged = he_nei(sel, ctxp)),
  theta_freq   = c(ref_theta_freq(sel, ctxp), theta_from_freq(sel, ctxp)),
  status_num   = c(ref_status_num(sel, ctxp), status_number(sel, ctxp)),
```

```
  ne_parents   = c(ref_ne_parents(sel, ctxp), ne_parents(sel, ctxp)),
  alleles_lost = c(ref_alleles_lost(sel, ctxp), alleles_lost(sel, ctxp)),
  ENE          = c(ref_ene(sel, ctxp), ene(sel, ctxp)),
  ANE          = c(ref_ane(sel, ctxp), ane(sel, ctxp))), 10)
```

	oracle	packaged
he_nei	0.3291684	0.3291684
theta_freq	0.6708316	0.6708316
status_num	0.7453435	0.7453435
ne_parents	26.4550265	26.4550265
alleles_lost	43.0000000	43.0000000
ENE	0.2581333	0.2581333
ANE	0.2503233	0.2503233

C.2 The identity sweep

The line-level collapse was checked across $F_{ST} \in \{0.05, 0.15, 0.35\}$, two seeds and two selection sizes, thirty replicate selections each.

	Identity	Worst deviation over the sweep
max_pairwise_f	pairwise f: hybrid vs line formula	1.1e-16
max_theta_sub_vs_line	theta: line route vs f submatrix	2.2e-16
max_theta_sub_vs_freq	theta: line route vs allele-frequency route	1.1e-16
max_pbar	selection allele frequencies	0 (exact)
max_He	Nei He	0 (exact)
max_Neparents	effective number of parents	2.1e-14
max_alleles_lost	allele loss at MAF threshold	0 (exact)
max_theta_pools	pool decomposition	1.2e-15
max_swap_update	incremental swap vs full recompute	4.4e-16
bitset_vs_literal	bitset allelic richness vs literal count	0 (exact)

C.3 The test suite

The suite is `tests/testthat/`, run with `devtools::test()`. It takes about a minute. What it asserts, and why each check earns its place:

Check	If it fails
theta via submatrix == theta via allele frequencies	**f is wrong**: centered, rescaled, or the wrong marker coding
1 - theta == Nei's He	the identity underpinning every 'loss of X%' statement is broken
f is bounded to [0, 1]	a centered matrix is being used as coancestry
sum(VanRaden G) == 0	the demonstration in Section 1.2 no longer holds on this data
theta and Ns analytical edge cases	the diagonal is being dropped somewhere
greedy < random < truncation on theta	there is no trade-off to optimise; the whole benchmark is pointless
decode() is unique, in range, deterministic	the DE fitness is noisy and cannot converge
fast line route == hybrid-matrix route	**the collapse is broken**; production scale becomes infeasible
incremental swap == full recomputation	any local search built on the state vector is wrong
popcount32 == naive bit count	allelic richness counts are wrong
F_hom and F_drift are 0 when the selection is the base	the base p0 is not frozen, or is being recomputed
cov_diag == F_hom - F_drift	one of the three lens formulas is wrong

Check	If it fails
max-diversity drives F_hom negative, F_drift positive	the two-lens argument of Chapter 12. does not reproduce
freq_metrics reproduces he_nei and alleles_lost	the consolidated marker sweep changed an existing number
the Caballero & Toro partition closes	the Eq. 8 algebra is broken
GD_WI == 0 for inbred lines, GD_WI_hyb large for their F1s	lines are not actually inbred, or the hybrid diagonal is wrong
eff_dim in (0, n - 1]	the double-centering was dropped and eff_dim re-measures theta
the two-stage output contract	downstream code reading `$selection` or `$metrics` will break
stage 2 runs without fL	the optional-fL path is broken
stage1_build accepts both marker codings	real data in the other coding would give different answers
genic_var == its literal oracle	the genic-variance route diverged from its definition
realised minus genic variance == the disequilibrium term	the disequilibrium decomposition of Chapter 3. is wrong
inbreeding depression == the shared term of the heterosis partition	Falconer's symmetry is broken, or `d` is being applied inconsistently
inbreeding depression == simulated genotypes (Monte Carlo)	the closed form does not describe genotypes anyone could actually draw
the restricted index moves the restricted trait exactly zero	**the restriction is a penalty**, and the Chapter 18. analogy is wrong
no restriction reduces the restricted index to Smith-Hazel	the degenerate case silently returns something other than Smith-Hazel
MRD and coancestry are one statistic	Trap 1 of Chapter 16. no longer holds
Gst < Fst always, and Gst matches the classical route	Trap 2 is wrong, or the two definitions were conflated
the DH identities	the S2 algebra is wrong and crossing plans cannot be pre-evaluated
hybrid theta from usage == the hybrid matrix	Trap 5′s repair route is broken
constrained line selection meets its ceiling	the constrained selection silently returns an infeasible plan
the oracles agree with the packaged metrics	a fast route diverged from its definition
map length and Haldane spacing	the sizing chapter's map is not the map it claims
heterozygosity halves per selfing	the simulator is not doing what selfing does
D == 1 - (1 - H)/N under neutrality	the identity every sizing number rests on is broken
n_equivalent inverts diversity_retained	N_eq is not the inverse it is described as
a Syn cycle adds half a map length of junctions	the intercrossing budget in Chapter 5. is wrong by 2x
distortion shifts ancestry away from 0.5	the distortion mechanism is inactive
f1_sizing halves the loss probability per plant	the F1 sizing argument is wrong
heterosis_value matches the literal oracle	the fast heterosis route diverged from its definition
additive effects cancel from the mid-parent difference	the derivation of Chapter 13. is wrong, or the parents are not inbred
shared and divergence == GD_T and GD_BS	**the central identity of Chapter 13. is broken**
2 * GD_WI_hyb == GD_T + GD_BS	the hybrid-level and line-level partitions have come apart
neutral divergence leaves expected heterosis unchanged	Chapter 13.'s correction is wrong and F_ST would predict heterosis
alignment between d and divergence is what moves heterosis	the alignment result, and the argument for Chapter 22. with it, is wrong
GCA + SCA reconstruct heterosis exactly and are mean-zero	**the combining-ability decomposition of Chapter 13. is not a decomposition**
SCA is the interaction residual of the two-way additive fit	SCA is a proxy for the interaction, not the interaction – the section overclaims
the closed-form GCA and SCA variance components match	the variance-component result of Section 13.11 rests on a wrong closed form

Check	If it fails
uniform-d heterosis is the coancestry kernel, exactly	genetic distance no longer predicts heterosis under flat dominance
GD_T + GD_BS == 1 - theta_AB	the between-pool coancestry reading of the identity is wrong
pool_freqs reproduces what its callers computed inline	a pool frequency route diverged after the refactor
the DH draw is an unbiased gamete; selfing an inbred returns it	recycling fabricates or destroys diversity; every trajectory is wrong
p0 is frozen, not re-anchored, across cycles	**both inbreeding lenses report no loss forever** – the trap of Chapter 22.
a closed programme loses diversity and gains index	the cycle loop is not selecting, or not recycling what it selected
the budget is respected; alpha_max = Inf is truncation	the budget is not binding across cycles
injection raises diversity relative to a closed run	injection is not actually introducing founder material
GCA rank == n' - components for every design family	**the connectivity argument of Chapter 9. is wrong**, or a generator is
NC II: between-set GCA contrast not estimable	the estimability check accepts what it should reject
k = 1 tester leaves zero df for SCA	the tester saturation claim of Section 9.3 is wrong
LS GCA error == s2s/k + s2e/(k r)	the Sprague-Tatum closed form no longer describes `fit_gca()`
sparse_blup == literal GLS oracle (BLUP and PEV)	**every live number of Chapter 10. is wrong**: the MME are assembled incorrectly
each hybrid in exactly k locations; parentage covers both pools everywhere	the allocation comparison of Section 10.11.3 compares different budgets
known-component BLUP has dispersion slope ~ 1	the BLUP is mis-scaled; the dispersion claims of Section 10.9 fail

```
Rscript -e 'devtools::test()'
```

C.4 Provenance of every figure and table

The book runs light examples live and reads heavy results from cache. This is the complete accounting.

C.4.1 Computed live at render

Chapter	Runs live
Chapter 1.	both matrices, the sum(G) demonstration, the f diagonal
Chapter 2.	the whole worked example, reproduced against the published tables
Chapter 3.	the breeder's equation against the realised response and its decomposition, the genic-variance oracle check, the GCA/SCA variance identity, both selection indices
Chapter 4.	the simulation, the fst sweep, the contract, the sampling-bias histogram
Chapter 5.	the theoretical identities, heterozygosity halving, map expansion, f1_sizing
Chapter 7.	the founder-engine reproducibility and LD comparison, the varA-against-genicVarA trajectory, the zero F1 variance of inbred parents, the selfing-versus-DH homozygosity trajectory, the breeder's equation over a selected-fraction sweep, sel_intensity against AlphaSimR selInt, Smith-Hazel against a numerical optimum, the RRS trajectory and the stage-2 metrics
Chapter 8.	the staged programme in both arms, the paired phenotypic-versus-genomic comparison over 24 seeds, the gene-diversity ledger

Chapter	Runs live
Chapter 9.	the design grids, the GCA-from-frequencies identity, the connectivity table and estimability check, the tester accuracy curve against its closed form, one LS-versus-BLUP replicate, the cost table, the decision tree
Chapter 10.	the allocation taxonomy panel, the pre-planting connectivity and reliability recipe
Chapter 12.	the Eq. 3 identity, the four-strategy lens panel
Chapter 13.	the heterosis oracle check, the additive cancellation, the shared/divergence identity, the coancestry-kernel identity, the fst sweep and its Balding-Nichols prediction, the alignment scenarios and their stratified control, the GCA/SCA decomposition and its variance components, the SCA/GCA relocation sweep
Chapter 14.	every metric, the sampling-bias formula, the null distribution
Chapter 15.	the collapse, the swap update, cost at book scale, the fixed-theta band, the prediction-error sweep
Chapter 16.	all five traps, the S1 panel, the DH identities, the S4 collapse
Chapter 18.	the OCS relaxation, the convergence check, the greedy frontier, the marker-panel sensitivity
Chapter 19.	all baselines, the encodings, the DE, the warm-start comparison
Chapter 20.	the Gram identity and PSD check, the submatrix-sum identity, the three local-search modes, the invalid-versus-Lagrangian bound comparison, ocs_round
Chapter 22.	the frozen-p0 demonstration, the delta-F regression
Chapter 23.	Robertson's limit against the observed trajectory, the paired selected-versus-random variance test in both the unlinked and the linked arm
Chapter 24.	the full two-stage pipeline
Appendix C.	the oracle comparisons

C.4.2 Read from cache

File	Why
`data/theta_identity_check.csv`	the full identity sweep, 12 configurations
`data/cost_per_eval.csv`	production scale: L = 300, N = 22500, m = 10000
`data/metric_redundancy.csv`	621 selections x 17 metrics
`data/de_constraints.csv`	9 DE runs at 60,000 evaluations each
`data/fst_definitions.csv`	the divergence sweep
`data/dist_vs_variance.csv`	600 candidate crosses with simulated DH
`data/pipeline_loss_ledger.csv`	8 founder pools, four stages, two strategies
`data/plant_metric_usage.csv`	687 screened plant records
`data/bibliography_*.csv`	the screened corpora themselves
`data/f2_results.csv`	5 schemes x 20 sizes, adaptive replication
`data/f2_multilocus.csv`	multilocus recovery across the grid
`data/mabc_*.csv`	the MABC grid: 4 policies x 4 gene numbers x 11 sizes x 3 s, 300 reps
`figs/mabc_sizing.png`	analytical sizing by stage
`figs/mabc_policies.png`	the four recombinant-selection policies
`figs/cost_scaling.png`	cost scaling at production scale
`figs/de_convergence.png`	DE convergence under four restrictions
`figs/metric_redundancy.png`	the redundancy panels
`figs/pipeline_loss.png`	the loss ledger

File	Why
figs/metric_pitfalls.png	Traps 4 and 2 on the full simulation
data/search_benchmark.csv	8 methods x 2 scales x 3 seeds, production scale N = 10000
data/search_scaling.csv	per-evaluation cost from N = 400 to N = 22500
data/search_exact_gap.csv	the outer-approximation oracle, proven small and time-limited large (target search_exact)
figs/search_anytime.png	anytime curves at equal wall-clock
figs/search_scaling.png	the encoding scaling wall
data/cycles_trajectory.csv	3 budgets x 16 cycles x 5 seeds (target cycles)
data/cycles_crossover.csv	the crossover cycle, per budget per seed
data/cycles_maxuse.csv	3 caps x 16 cycles x 5 seeds
data/cycles_injection.csv	closed against one injection at cycle 8, 5 seeds
data/cycles_selector.csv	greedy against DE, 2 budgets x 13 cycles, seed 1
figs/cycles_gain.png	cumulative gain by budget
figs/cycles_diversity.png	gene diversity and F_drift by budget
data/mating_core_sweep.csv	11 designs x 3 budgets x 30 reps, LS and BLUP (target mating)
data/mating_scenarios_summary.csv, data/mating_optimizer_frontier.csv, data/mating_validation.csv, figs/mating_*.png	**not regenerable here**: the external sparsex study (AlphaSimR founders, AI-REML, GBLUP, CDmean), 24 designs x 3 budgets x 30 reps x 3 models; its phenotypic core is the row above
data/sparse_core_sweep.csv	spreading, k = 1 strategies, and overlap x family size; 30 reps, known components (target sparse)
data/sparse_scenarios.csv, data/sparse_min_allocation.csv, data/sparse_contrasts.csv, data/sparse_two_stage.csv, data/sparse_genomic_layer.csv, data/sparse_validation.csv	**not regenerable here**: an external study (REML by sommer, 2,630 fits; two-stage layer; AlphaSimR genomic layer); its phenotypic core is the row above
data/sparse_primary_studies.csv, data/sparse_literature_parameters.csv	hand-extracted from the six primary papers of Chapter 10.

Nothing is read from report/ or data/ at the repository root: both are gitignored, so a clean clone would fail to render. Everything cached lives in book/data and book/figs, which are tracked.

C.5 Regenerating the cached results

```
Rscript book/scripts/regenerate.R          # everything; hours
Rscript book/scripts/regenerate.R f2size   # one target
Rscript book/scripts/regenerate.R search   # the advanced-selection benchmark
Rscript book/scripts/regenerate.R search_exact  # just the exact oracle
Rscript book/scripts/regenerate.R cycles   # the recurrent-selection trajectories
Rscript book/scripts/regenerate.R mating   # the mating-design core sweep (~6 min)
Rscript book/scripts/regenerate.R sparse   # the sparse-testing core sweep (~5 min)
```

The script is the single place where the expensive computations live. It is never invoked by quarto render.

C.6 Reproducing the book

```
Rscript -e 'devtools::document(); devtools::load_all()'
Rscript -e 'devtools::test()'
Rscript -e 'devtools::check()'
quarto render book
```

Quarto's `freeze: auto` means a chapter re-runs only when its source changes. To force a full recomputation, delete `book/_freeze/`.

C.7 Known limitations of the verification

Stated so they are not mistaken for oversights.

The oracles share the author's understanding. They check that the fast route computes what the slow route computes. They cannot check that the slow route implements the right definition. That is what the reproduction of the published Caballero & Toro tables in Chapter 2. is for – an external check against numbers this project did not produce.

Monte Carlo checks have Monte Carlo error. Where a test asserts agreement with a simulated expectation rather than an algebraic identity, the tolerance is loose by necessity, and a genuinely small bias would pass.

Everything is verified on simulated data. The identities are algebraic and will hold on real genotypes. The *empirical* results – which metrics are redundant, how much index a constraint costs – were derived at $F_{ST} = 0.15$ with simulated markers, and Chapter 15. says explicitly that they should be re-derived on real genotypes before being treated as settled.

One chapter's headline results come from outside this repository. The mating-design sweep of Chapter 9. (`mating_scenarios_summary.csv` and its siblings) was produced by an external study whose code needs AlphaSimR and `sommer`; it shipped with its own 37-check validation report, summarised in that chapter, but it is not re-run by anything here. What is re-run is its phenotypic core (`regenerate.R mating`), and Section 9.10 compares the two.

The sparse-testing chapter (Chapter 10.) has the same structure. Its REML grid, two-stage scenario and genomic layer are cached from an external study, which validated its own estimator on the complete design (Table 10.5). The in-repo `regenerate.R sparse` re-runs the known-component core, and 2 compares the two.

Reproduce this chapter

All the code of this chapter, in order and with the explanation kept as comments, is one script: **92-verification.R**. Download it, install `hybdiv`, and run it top to bottom in an empty directory; cached results are fetched on first use.

D. The bibliographic corpora

Chapter 21. draws on two screened corpora. This appendix describes them so the claims there can be checked, extended, or disagreed with.

D.1 What they are

Corpus	Records	Years	Focus
`bibliography_animal.csv`	324	2018-2026	optimal contribution, coancestry matrix choice, effective size, allelic richness
`bibliography_plant.csv`	687	2017-2026	diversity quantification, heterotic structure, genetic resources, optimisation

Both are annotated screens, not exhaustive searches. The plant corpus came from 2004 records across 29 plant-breeding queries on Crossref and Europe PMC, of which 687 were scored relevant.

D.2 Structure

```
names(pl)
#>  [1] "doi"              "year"             "authors"          "title"
#>  [5] "journal"          "citations"        "relevance"        "crop"
#>  [9] "metric_used"      "selection_method" "contribution"     "url"
```

Column	Meaning
`relevance`	3 = directly relevant, 2 = related background
`crop`	plant corpus only
`metric_used` / `metric_advocated`	the diversity metrics the record reports
`selection_method`	plant corpus only
`theme`	animal corpus only; multi-valued, semicolon-separated
`contribution` / `relevance_note`	a one-line note on what the record adds

D.3 Relevance and coverage

```
table(plant = pl$relevance)
#> plant
#>   2   3
#> 529 158
table(animal = ani$relevance)
#> animal
#>   2   3
#> 235  89
```

crop	records
maize	54
wheat	11
multi-crop	10
rice	9

crop	records
none	7
sorghum	5
barley	4
none (general crop)	4
potato	3
soybean	3

Maize dominates, which is the intended bias – but it also means every claim in Chapter 21. about crops other than maize rests on a handful of records.

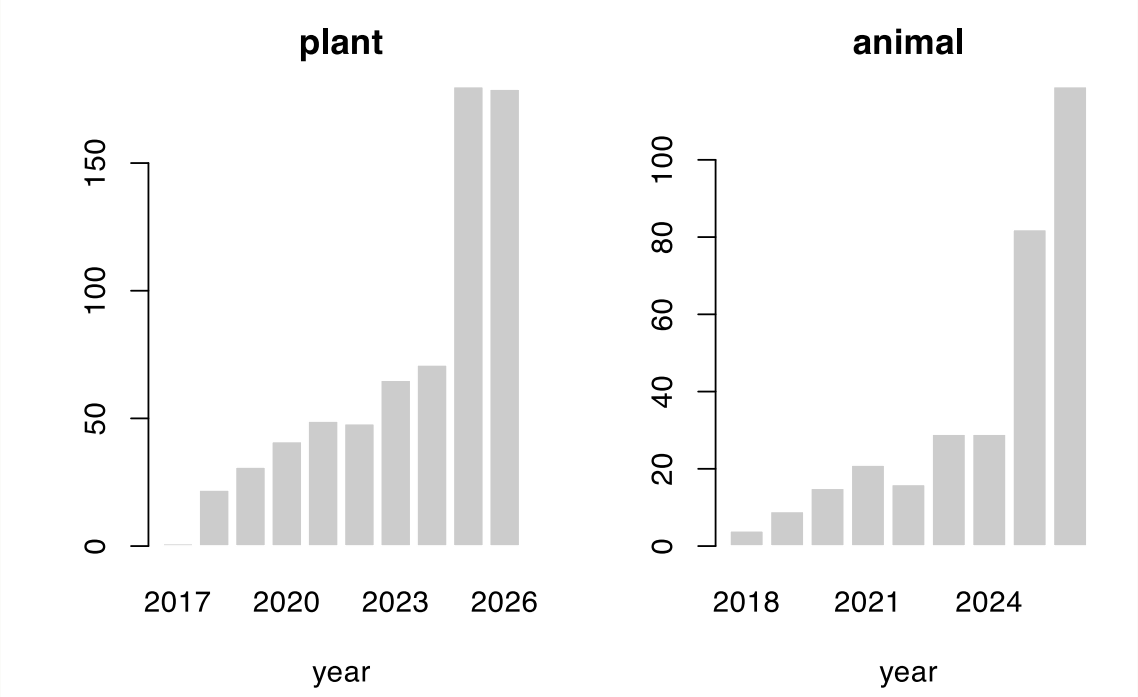

Figure D.1: Publication year of the screened records. Both corpora are deliberately post-2017.

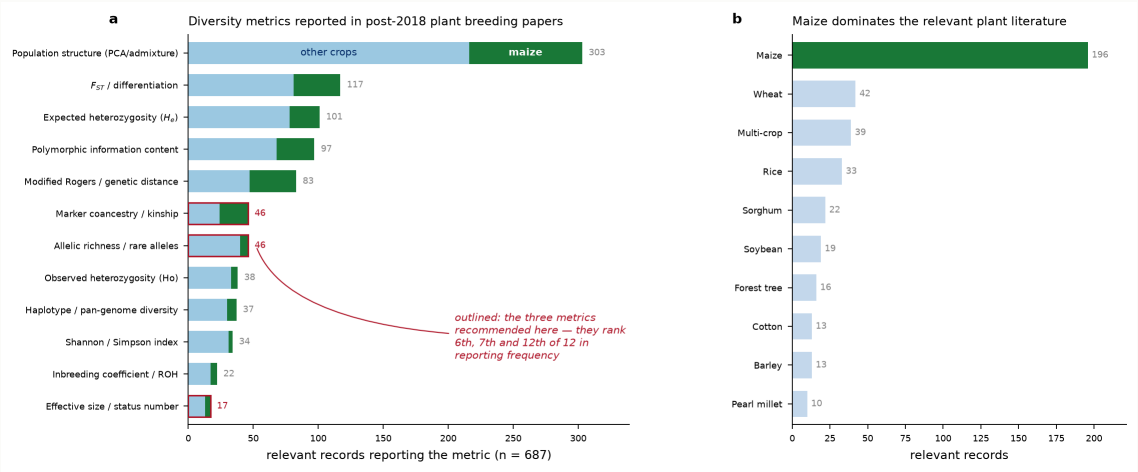

Figure D.2: Which diversity metrics the plant literature actually reports.

D.4 Themes in the animal corpus

The `theme` column is multi-valued. Splitting it:

```
th <- unlist(strsplit(ani$theme[ani$relevance == 3], ";\\s*"))
th <- trimws(th); th <- th[nzchar(th)]
sort(table(th), decreasing = TRUE)
#> th
#>         OCS / contribution optimisation Coancestry / relationship matrix choice
#>                                      68                                      61
#>          Optimisation algorithms / cost         Allelic richness / rare alleles
#>                                      27                                      20
#>                            Effective size         Heterozygosity / gene diversity
#>                                      18                                      15
#>      Core collection / subset selection
#>                                       9
```

D.5 Querying them

The corpora are ordinary CSVs, so the claims in Chapter 21. are checkable in one line each.

```
# records that report a coancestry or kinship metric
sum(grepl("coancestry|kinship|relatedness", pl$metric_used, ignore.case = TRUE))
#> [1] 12

# records using ROH, and how many of those are maize
roh <- grepl("ROH", pl$metric_used, ignore.case = TRUE)
c(roh_total = sum(roh), roh_maize = sum(roh & grepl("maize", pl$crop, ignore.case
= TRUE)))
#> roh_total roh_maize
#>         1         0

# the most-cited directly relevant maize records
mz <- pl[pl$relevance == 3 & grepl("maize", pl$crop, ignore.case = TRUE), ]
head(mz[order(-mz$citations), c("year", "title", "citations")], 6)
```

year	title	citations
2020	Diversity and heterotic patterns in North American proprietary dent maize germplasm	58
2020	Discovery of beneficial haplotypes for complex traits in maize landraces.	46
2021	Genetic diversity and selection signatures in maize landraces compared across 50 years of in situ and ex situ conservation	44
2021	Application of Genomic Selection at the Early Stage of Breeding Pipeline in Tropical Maize.	38
2020	Efficiencies of Heterotic Grouping Methods for Classifying Early Maturing Maize Inbred Lines	26
2021	Genetic variation and population structure in China summer maize germplasm.	25

```
# the animal-side records on matrix choice
mc <- ani[grepl("Coancestry / relationship matrix choice", ani$theme), ]
head(mc[order(-mc$citations), c("year", "title", "citations")], 6)
```

year	title	citations
2018	Optimal cross selection for long-term genetic gain in two-part programs with rapid recurrent genomic selection	156
2019	The impact of genomic selection on genetic diversity and genetic gain in three French dairy cattle breeds	100
2020	Management of Genetic Diversity in the Era of Genomics.	72
2019	Genomic Tools for Effective Conservation of Livestock Breed Diversity	71

year	title	citations
2020	Molecular genetic analysis of spring wheat core collection using genetic diversity, population structure, and linkage disequilibrium	65
2019	Improving Short- and Long-Term Genetic Gain by Accounting for Within-Family Variance in Optimal Cross-Selection.	59

D.6 Limitations

These are abstract screens. Relevance and the metric annotations were scored from titles and abstracts, not from full texts. A paper that computes a metric without naming it in the abstract is invisible to this table – which almost certainly understates how often coancestry is computed as an intermediate step.

The queries shaped the answer. 29 plant-breeding queries were run; a different query set would return a different landscape. The strong maize skew is by design, and the counts for other crops are too small to support a claim about those crops.

"Not established in the retrieved set" means exactly that. Every such statement in Chapter 21. is a claim about *this corpus*, not about the literature as a whole. It is a prompt to look further before assuming novelty, not a proof of it.

Citation counts are a snapshot at screening time and are already stale.

D.7 Classical references

The corpora are deliberately post-2017. The classical results the book rests on are cited from the source literature, and collected in `book/references.bib`:

Result	Source
group coancestry	Cockerham (1967)
molecular / IBS coancestry	Nejati-Javaremi et al. (1997)
gene diversity, Gst	Nei (1973)
the subdivided-population partition	Caballero & Toro (2000, 2002)
status number	Lindgren & Mullin (1997, 1998)
optimal contribution selection	Meuwissen (1997)
genomic relationship matrix	VanRaden (2008)
modified Rogers distance	Wright (1978)
usefulness criterion	Schnell & Utz (1975)
differential evolution	Storn & Price (1997)

Reproduce this chapter

All the code of this chapter, in order and with the explanation kept as comments, is one script: **93-corpora.R**. Download it, install `hybdiv`, and run it top to bottom in an empty directory; cached results are fetched on first use.

References

Bibliography

[1] A. Caballero and M. A. Toro, "Analysis of genetic diversity for the management of conserved subdivided populations," *Conservation Genetics*, vol. 3, no. 3, pp. 289–299, 2002, doi: 10.1023/A:1019956205473.

[2] A. Caballero and M. A. Toro, "Interrelations between effective population size and other pedigree tools for the management of conserved populations," *Genetics Research*, vol. 75, no. 3, pp. 331–343, 2000, doi: 10.1017/S0016672399004449.

[3] A. Nejati-Javaremi, C. Smith, and J. P. Gibson, "Effect of total allelic relationship on accuracy of evaluation and response to selection," *Journal of Animal Science*, vol. 75, no. 7, pp. 1738–1745, 1997, doi: 10.2527/1997.7571738x.

[4] F. Gómez-Romano, B. Villanueva, J. Fernández, J. A. Woolliams, and R. Pong-Wong, "The use of genomic coancestry matrices in the optimisation of contributions to maintain genetic diversity at specific regions of the genome," *Genetics Selection Evolution*, vol. 48, p. 2, 2016, doi: 10.1186/s12711-015-0172-y.

[5] M. A. Toro, B. Villanueva, and J. Fernández, "Genomics applied to management strategies for conservation of farm animal genetic resources," *Animal Genetics*, vol. 45, no. 3, pp. 1–10, 2014, doi: 10.1111/age.12140.

[6] T. H. E. Meuwissen, A. K. Sonesson, G. Gebregiwergis, and J. A. Woolliams, "Management of genetic diversity in the era of genomics," *Frontiers in Genetics*, vol. 11, p. 880, 2020, doi: 10.3389/fgene.2020.00880.

[7] R. E. Comstock, H. F. Robinson, and P. H. Harvey, "A breeding procedure designed to make maximum use of both general and specific combining ability," *Agronomy Journal*, vol. 41, no. 8, pp. 360–367, 1949, doi: 10.2134/agronj1949.00021962004100080006x.

[8] R. C. Gaynor, G. Gorjanc, and J. M. Hickey, "AlphaSimR: an R package for breeding program simulations," *G3: Genes, Genomes, Genetics*, vol. 11, no. 2, 2021, doi: 10.1093/g3journal/jkaa017.

[9] G. F. Sprague and L. A. Tatum, "General vs. specific combining ability in single crosses of corn," *Journal of the American Society of Agronomy*, vol. 34, no. 10, pp. 923–932, 1942.

[10] G. Gorjanc, I. Pocrnic, J. Obšteter, A. Ehsani, L. A. de C. Lara, and N. Lubanga, "AlphaSimR course vignettes: simulating DNA, traits, selection and breeding programmes." 2022.

[11] G. K. Chen, P. Marjoram, and J. D. Wall, "Fast and flexible simulation of DNA sequence data," *Genome Research*, vol. 19, no. 1, pp. 136–142, 2009, doi: 10.1101/gr.083634.108.

[12] D. S. Falconer and T. F. C. Mackay, *Introduction to Quantitative Genetics*, 4th ed. Harlow: Longman, 1996.

[13] I. J. Mackay, J. Cockram, and P. Howell, "Understanding the classics: the unifying concepts of transgressive segregation, inbreeding depression and heterosis and their central relevance for crop breeding," *Plant Biotechnology Journal*, vol. 19, no. 1, pp. 26–34, 2021, doi: 10.1111/pbi.13481.

[14] A. R. Hallauer, M. J. Carena, and J. B. Miranda Filho, *Quantitative Genetics in Maize Breeding*, 3rd ed., vol. 6. in Handbook of Plant Breeding, vol. 6. New York: Springer, 2010. doi: 10.1007/978-1-4419-0766-0.

[15] H. F. Smith, "A discriminant function for plant selection," *Annals of Eugenics*, vol. 7, no. 3, pp. 240–250, 1936, doi: 10.1111/j.1469-1809.1936.tb02143.x.

[16] L. N. Hazel, "The genetic basis for constructing selection indexes," *Genetics*, vol. 28, no. 6, pp. 476–490, 1943, doi: 10.1093/genetics/28.6.476.

[17] S. Michel, F. Löschenberger, C. Ametz, B. Pachler, E. Sparry, and H. Bürstmayr, "Combining grain yield, protein content and protein quality by multi-trait genomic selection in bread wheat," *Theoretical and Applied Genetics*, vol. 132, no. 10, pp. 2767–2780, 2019, doi: 10.1007/s00122-019-03386-1.

[18] R. C. Gaynor *et al.*, "A two-part strategy for using genomic selection to develop inbred lines," *Crop Science*, vol. 57, no. 5, pp. 2372–2386, 2017, doi: 10.2135/cropsci2016.09.0742.

[19] T. H. E. Meuwissen, B. J. Hayes, and M. E. Goddard, "Prediction of total genetic value using genome-wide dense marker maps," *Genetics*, vol. 157, no. 4, pp. 1819–1829, 2001, doi: 10.1093/genetics/157.4.1819.

[20] B. Griffing, "Concept of General and Specific Combining Ability in Relation to Diallel Crossing Systems," *Australian Journal of Biological Sciences*, 1956, doi: 10.1071/bi9560463.

[21] R. E. Comstock and H. F. Robinson, "The Components of Genetic Variance in Populations of Biparental Progenies and Their Use in Estimating the Average Degree of Dominance," *Biometrics*, 1948, doi: 10.2307/3001412.

[22] C. W. Stuber and C. C. Cockerham, "GENE EFFECTS AND VARIANCES IN HYBRID POPULATIONS," *Genetics*, 1966, doi: 10.1093/genetics/54.6.1279.

[23] D. L. Weeks and D. R. Williams, "A Note on the Determination of Connectedness in an N-Way Cross Classification," *Technometrics*, 1964, doi: 10.1080/00401706.1964.10490188.

[24] R. Fernando, D. Gianola, and M. Grossman, "Identifying All Connected Subsets in a Two-Way Classification Without Interaction," *Journal of Dairy Science*, 1983, doi: 10.3168/jds.s0022-0302(83)81951-1.

[25] M. T. Jenkins and A. M. Brunson, "Methods of Testing Inbred Lines of Maize in Crossbred Combinations1," *Agronomy Journal*, 1932, doi: 10.2134/agronj1932.00021962002400070004x.

[26] F. H. Hull, "Recurrent Selection for Specific Combining Ability in Corn1," *Agronomy Journal*, 1945, doi: 10.2134/agronj1945.00021962003700020006x.

[27] C. Longin, H. Utz, A. Melchinger, and J. Reif, "Hybrid maize breeding with doubled haploids: II. Optimum type and number of testers in two-stage selection for general combining ability," *Theor Appl Genet*, 2006, doi: 10.1007/s00122-006-0422-z.

[28] H. Giraud *et al.*, "Reciprocal Genetics: Identifying QTL for General and Specific Combining Abilities in Hybrids Between Multiparental Populations from Two Maize (Zea mays L.) Heterotic Groups," *Genetics*, 2017, doi: 10.1534/genetics.117.300305.

[29] A. Lorenzi *et al.*, "Portability of genomic predictions trained on sparse factorial designs across two maize silage breeding cycles," *TAG. Theoretical and applied genetics. Theoretische und angewandte Genetik*, 2024, doi: 10.1007/s00122-024-04566-4.

[30] O. S. Smith, "Covariance between Line per se and Testcross Performance1," *Crop Science*, 1986, doi: 10.2135/cropsci1986.0011183x002600030023x.

[31] D. Chaingeni *et al.*, "More bang for your buck: potential gains through optimizing maize breeding schemes in sub-Saharan Africa," *Frontiers in plant science*, 2025, doi: 10.3389/fpls.2025.1553272.

[32] B. Burdo, N. de Leon, and S. M. Kaeppler, "Testcross vs. randomly paired single-cross progeny tests for genomic prediction of new inbreds and hybrids derived from multiparent maize populations," *Crop Science*, 2021, doi: 10.1002/csc2.20545.

[33] R. Fristche-Neto, D. Akdemir, and J.-L. Jannink, "Accuracy of genomic selection to predict maize single-crosses obtained through different mating designs," *Theoretical and Applied Genetics*, 2018, doi: 10.1007/s00122-018-3068-8.

[34] A. Seye, C. Bauland, A. Charcosset, and L. Moreau, "Revisiting hybrid breeding designs using genomic predictions: simulations highlight the superiority of incomplete factorials between segregating families over topcross designs," *TAG. Theoretical and applied genetics. Theoretische und angewandte Genetik*, 2020, doi: 10.1007/s00122-020-03573-5.

[35] D. González-Diéguez, G. Atlin, Y. Beyene, D. Wegary, D. Gemenet, and C. Werner, "Sparse testcrossing for early-stage genomic prediction of general combining ability to increase genetic gain in maize hybrid breeding programs," *TAG. Theoretical and applied genetics. Theoretische und angewandte Genetik*, 2026, doi: 10.1007/s00122-026-05169-x.

[36] O. Kempthorne and R. N. Curnow, "The Partial Diallel Cross," *Biometrics*, 1961, doi: 10.2307/2527989.

[37] R. Rincent *et al.*, "Maximizing the Reliability of Genomic Selection by Optimizing the Calibration Set of Reference Individuals: Comparison of Methods in Two Diverse Groups of Maize Inbreds (Zea maysL.)," *Genetics*, 2012, doi: 10.1534/genetics.112.141473.

[38] L. Thomsen *et al.*, "Improving hybrid breeding efficiency in winter oilseed rape under sparse multi-environment testing," *TAG. Theoretical and applied genetics. Theoretische und angewandte Genetik*, 2026, doi: 10.1007/s00122-026-05341-3.

[39] T. Guo *et al.*, "Optimal Designs for Genomic Selection in Hybrid Crops," *Molecular Plant*, 2019, doi: 10.1016/j.molp.2018.12.022.

[40] A. Lorenzi *et al.*, "Genomic prediction of hybrid performance: comparison of the efficiency of factorial and tester designs used as training sets in a multiparental connected reciprocal design for maize silage," *Theoretical and Applied Genetics*, 2022, doi: 10.1007/s00122-022-04176-y.

[41] A. Melchinger, R. Fernando, C. Stricker, C. Schön, and H. Auinger, "Genomic prediction in hybrid breeding: I. Optimizing the training set design," *TAG. Theoretical and applied genetics. Theoretische und angewandte Genetik*, 2023, doi: 10.1007/s00122-023-04413-y.

[42] D. Jarquin *et al.*, "Genomic Prediction Enhanced Sparse Testing for Multi-environment Trials," *G3 (Bethesda, Md.)*, 2020, doi: 10.1534/g3.120.401349.
[43] S. A. Atanda *et al.*, "Scalable Sparse Testing Genomic Selection Strategy for Early Yield Testing Stage," *Frontiers in Plant Science*, 2021, doi: 10.3389/fpls.2021.658978.
[44] S. A. Atanda, V. Govindan, R. Singh, K. R. Robbins, J. Crossa, and A. R. Bentley, "Sparse testing using genomic prediction improves selection for breeding targets in elite spring wheat," *Theoretical and Applied Genetics*, 2022, doi: 10.1007/s00122-022-04085-0.
[45] O. A. Montesinos-López, B. A. Mosqueda-González, J. Salinas-Ruiz, A. Montesinos-López, and J. Crossa, "Sparse multi-trait genomic prediction under balanced incomplete block design," *The Plant Genome*, 2023, doi: 10.1002/tpg2.20305.
[46] R. Bernardo, "Prediction of Maize Single-Cross Performance Using RFLPs and Information from Related Hybrids," *Crop Science*, 1994, doi: 10.2135/cropsci1994.0011183x003400010003x.
[47] R. Bernardo, "Best Linear Unbiased Prediction of Maize Single-Cross Performance," *Crop Science*, 1996, doi: 10.2135/cropsci1996.0011183x003600010009x.
[48] J. B. Endelman *et al.*, "Optimal Design of Preliminary Yield Trials with Genome-Wide Markers," *Crop Science*, 2014, doi: 10.2135/cropsci2013.03.0154.
[49] X. Mi, H. F. Utz, F. Technow, and A. E. Melchinger, "Optimizing Resource Allocation for Multi-stage Selection in Plant Breeding with R Package Selectiongain," *Crop Science*, 2014, doi: 10.2135/cropsci2013.10.0699.
[50] F. Technow, C. Riedelsheimer, T. A. Schrag, and A. E. Melchinger, "Genomic prediction of hybrid performance in maize with models incorporating dominance and population specific marker effects," *Theoretical and Applied Genetics*, 2012, doi: 10.1007/s00122-012-1905-8.
[51] J. M. Massman, A. Gordillo, R. E. Lorenzana, and R. Bernardo, "Genomewide predictions from maize single-cross data," *Theoretical and Applied Genetics*, 2012, doi: 10.1007/s00122-012-1955-y.
[52] F. Technow, T. A. Schrag, W. Schipprack, E. Bauer, H. Simianer, and A. E. Melchinger, "Genome properties and prospects of genomic prediction of hybrid performance in a breeding program of maize," *Genetics*, vol. 197, no. 4, pp. 1343–1355, 2014, doi: 10.1534/genetics.114.165860.
[53] D. González-Diéguez *et al.*, "Genomic prediction of hybrid crops allows disentangling dominance and epistasis," *Genetics*, 2021, doi: 10.1093/genetics/iyab026.
[54] D. Kadam, S. Potts, M. Bohn, A. Lipka, and A. Lorenz, "Genomic Prediction of Single Crosses in the Early Stages of a Maize Hybrid Breeding Pipeline," *G3 (Bethesda, Md.)*, 2016, doi: 10.1534/g3.116.031286.
[55] F. Technow, "Use of F2 Bulks in Training Sets for Genomic Prediction of Combining Ability and Hybrid Performance," *G3 (Bethesda, Md.)*, 2019, doi: 10.1534/g3.118.200994.
[56] R. Fritsche-Neto *et al.*, "Optimizing Genomic-Enabled Prediction in Small-Scale Maize Hybrid Breeding Programs: A Roadmap Review," *Frontiers in plant science*, 2021, doi: 10.3389/fpls.2021.658267.
[57] C. Lehermeier, S. Teyssèdre, and C.-C. Schön, "Genetic Gain Increases by Applying the Usefulness Criterion with Improved Variance Prediction in Selection of Crosses," *Genetics*, 2017, doi: 10.1534/genetics.117.300403.
[58] L. Lian, A. Jacobson, S. Zhong, and R. Bernardo, "Prediction of Genetic Variance in Biparental Maize Populations: Genomewide Marker Effects versus Mean Genetic Variance in Prior Populations," *Crop Science*, 2015, doi: 10.2135/cropsci2014.10.0729.
[59] G. Gorjanc, R. C. Gaynor, and J. M. Hickey, "Optimal cross selection for long-term genetic gain in two-part programs with rapid recurrent genomic selection," *Theoretical and Applied Genetics*, vol. 131, 2018, doi: 10.1007/s00122-018-3125-3.
[60] A. Allier, C. Lehermeier, A. Charcosset, L. Moreau, and S. Teyssèdre, "Improving short- and long-term genetic gain by accounting for within-family variance in optimal cross-selection," *Frontiers in Genetics*, vol. 10, 2019, doi: 10.3389/fgene.2019.01006.
[61] B. P. Kinghorn, "An algorithm for efficient constrained mate selection," *Genetics Selection Evolution*, 2011, doi: 10.1186/1297-9686-43-4.
[62] J. J. Marulanda, X. Mi, A. E. Melchinger, J.-L. Xu, T. Würschum, and C. F. H. Longin, "Optimum breeding strategies using genomic selection for hybrid breeding in wheat, maize, rye, barley, rice and triticale," *Theoretical and Applied Genetics*, 2016, doi: 10.1007/s00122-016-2748-5.
[63] C. F. H. Longin, X. Mi, and T. Würschum, "Genomic selection in wheat: optimum allocation of test resources and comparison of breeding strategies for line and hybrid breeding," *Theoretical and Applied Genetics*, 2015, doi: 10.1007/s00122-015-2505-1.

[64] A. J. Lorenz, "Resource Allocation for Maximizing Prediction Accuracy and Genetic Gain of Genomic Selection in Plant Breeding: A Simulation Experiment," *G3 Genes|Genomes|Genetics*, 2013, doi: 10.1534/g3.112.004911.

[65] C. Riedelsheimer and A. E. Melchinger, "Optimizing the allocation of resources for genomic selection in one breeding cycle," *Theoretical and Applied Genetics*, 2013, doi: 10.1007/s00122-013-2175-9.

[66] C. F. H. Longin, H. F. Utz, J. C. Reif, W. Schipprack, and A. E. Melchinger, "Hybrid maize breeding with doubled haploids: I. One-stage versus two-stage selection for testcross performance," *Theoretical and Applied Genetics*, 2006, doi: 10.1007/s00122-005-0192-z.

[67] A. Melchinger and M. Frisch, "Genomic prediction in hybrid breeding: II. Reciprocal recurrent genomic selection with full-sib and half-sib families," *TAG. Theoretical and applied genetics. Theoretische und angewandte Genetik*, 2023, doi: 10.1007/s00122-023-04446-3.

[68] Y. Li *et al.*, "Multi-environment sparse testing strategies improve genomic prediction of hybrid performance and general combining ability in maize," *The Crop Journal*, 2026, doi: 10.1016/j.cj.2026.06.005.

[69] C. R. Werner, G. Covarrubias-Pazaran, D. Gonzalez-Dieguez, D. C. Gemenet, H. Gandhi, and G. N. Atlin, "Early-stage sparse testing strategies to increase genetic gain in plant breeding programmes." 2025. doi: 10.1101/2025.08.18.670883.

[70] N. Lubanga, B. E. Ifie, R. Persa, I. Dieng, I. Y. Rabbi, and D. Jarquin, "Sparse testing designs for optimizing resource allocation in multi-environment cassava breeding trials," *The Plant Genome*, 2025, doi: 10.1002/tpg2.20558.

[71] A. Hild Aono and A. Chawade, "Phenomics-assisted sparse testing for potato breeding," *Scientific Reports*, vol. 16, p. 22033, 2026, doi: 10.1038/s41598-026-59202-6.

[72] G. N. Atlin and B. F. Econopouly, "Simple deterministic modeling can guide the design of breeding pipelines for self-pollinated crops," *Crop Science*, vol. 62, pp. 661–678, 2022, doi: 10.1002/csc2.20684.

[73] G. N. Atlin, R. J. Baker, K. B. McRae, and X. Lu, "Selection response in subdivided target regions," *Crop Science*, vol. 40, pp. 7–13, 2000, doi: 10.2135/cropsci2000.4017.

[74] G. N. Atlin, M. Cooper, and A. Bjornstad, "A comparison of formal and participatory breeding approaches using selection theory," *Euphytica*, vol. 122, pp. 463–475, 2001, doi: 10.1023/A:1017557307800.

[75] C. R. Werner, M. Zaman-Allah, T. Assefa, J. E. Cairns, and G. N. Atlin, "Accelerating genetic gain through early-stage on-farm sparse testing," *Trends in Plant Science*, vol. 30, pp. 17–20, 2025, doi: 10.1016/j.tplants.2024.10.010.

[76] A. Legarra, D. O. Gonzalez-Dieguez, A. Charcosset, and Z. G. Vitezica, "Impact of interpopulation distance on dominance variance and average heterosis in hybrid populations within species," *Genetics*, vol. 224, p. iyad059, 2023, doi: 10.1093/genetics/iyad059.

[77] P. M. VanRaden, "Efficient methods to compute genomic predictions," *Journal of Dairy Science*, vol. 91, no. 11, pp. 4414–4423, 2008, doi: 10.3168/jds.2007-0980.

[78] Z. G. Vitezica, L. Varona, and A. Legarra, "On the additive and dominant variance and covariance of individuals within the genomic selection scope," *Genetics*, vol. 195, pp. 1223–1230, 2013, doi: 10.1534/genetics.113.155176.

[79] A. Smith, B. Cullis, and R. Thompson, "Analyzing variety by environment data using multiplicative mixed models and adjustments for spatial field trend," *Biometrics*, vol. 57, pp. 1138–1147, 2001, doi: 10.1111/j.0006-341x.2001.01138.x.

[80] A. B. Smith, B. R. Cullis, and R. Thompson, "The analysis of crop cultivar breeding and evaluation trials: an overview of current mixed model approaches," *The Journal of Agricultural Science*, vol. 143, pp. 449–462, 2005, doi: 10.1017/s0021859605005587.

[81] A. M. Kelly, A. B. Smith, J. A. Eccleston, and B. R. Cullis, "The accuracy of varietal selection using factor analytic models for multi-environment plant breeding trials," *Crop Science*, vol. 47, pp. 1063–1070, 2007, doi: 10.2135/cropsci2006.08.0540.

[82] A. B. Smith and B. R. Cullis, "Plant breeding selection tools built on factor analytic mixed models for multi-environment trial data," *Euphytica*, vol. 214, p. 143, 2018, doi: 10.1007/s10681-018-2220-5.

[83] H. Oakey, B. Cullis, R. Thompson, J. Comadran, C. Halpin, and R. Waugh, "Genomic selection in multi-environment crop trials," *G3 Genes|Genomes|Genetics*, vol. 6, pp. 1313–1326, 2016, doi: 10.1534/g3.116.027524.

[84] D. J. Tolhurst, K. L. Mathews, A. B. Smith, and B. R. Cullis, "Genomic selection in multi-environment plant breeding trials using a factor analytic linear mixed model," *Journal of Animal Breeding and Genetics*, vol. 136, pp. 279–300, 2019, doi: 10.1111/jbg.12404.

[85] J. Burgueño, G. de los Campos, K. Weigel, and J. Crossa, "Genomic prediction of breeding values when modeling genotype \times environment interaction using pedigree and dense molecular markers", *Crop Science*, vol. 52, no. 2, pp. 707–719, 2012, doi: 10.2135/cropsci2011.06.0299.

[86] D. Jarquin *et al.*, "A reaction norm model for genomic selection using high-dimensional genomic and environmental data," *Theoretical and Applied Genetics*, vol. 127, pp. 595–607, 2013, doi: 10.1007/s00122-013-2243-1.

[87] M. Lopez-Cruz *et al.*, "Increased prediction accuracy in wheat breeding trials using a marker x environment interaction genomic selection model," *G3 Genes|Genomes|Genetics*, vol. 5, pp. 569–582, 2015, doi: 10.1534/g3.114.016097.

[88] H. P. Piepho, J. Mohring, T. Schulz-Streeck, and J. O. Ogutu, "A stage-wise approach for the analysis of multi-environment trials," *Biometrical Journal*, vol. 54, pp. 844–860, 2012, doi: 10.1002/bimj.201100219.

[89] J. B. Endelman, "Fully efficient, two-stage analysis of multi-environment trials with directional dominance and multi-trait genomic selection," *Theoretical and Applied Genetics*, vol. 136, p. 65, 2023, doi: 10.1007/s00122-023-04298-x.

[90] K. R. Robbins, J. E. Backlund, and K. D. Schnelle, "Spatial corrections of unreplicated trials using a two-dimensional spline," *Crop Science*, vol. 52, pp. 1138–1144, 2012, doi: 10.2135/cropsci2011.08.0417.

[91] B. R. Cullis, A. B. Smith, and N. E. Coombes, "On the design of early generation variety trials with correlated data," *Journal of Agricultural, Biological, and Environmental Statistics*, vol. 11, pp. 381–393, 2006, doi: 10.1198/108571106x154443.

[92] H. P. Piepho and J. Mohring, "Computing heritability and selection response from unbalanced plant breeding trials," *Genetics*, vol. 177, pp. 1881–1888, 2007, doi: 10.1534/genetics.107.074229.

[93] E. Williams, H. P. Piepho, and D. Whitaker, "Augmented p-rep designs," *Biometrical Journal*, vol. 53, pp. 19–27, 2011, doi: 10.1002/bimj.201000102.

[94] J. Isidro y Sanchez and D. Akdemir, "Training set optimization for sparse phenotyping in genomic selection: a conceptual overview," *Frontiers in Plant Science*, vol. 12, p. 715910, 2021, doi: 10.3389/fpls.2021.715910.

[95] J. Fernandez-Gonzalez, D. Akdemir, and J. Isidro y Sanchez, "A comparison of methods for training population optimization in genomic selection," *Theoretical and Applied Genetics*, vol. 136, p. 30, 2023, doi: 10.1007/s00122-023-04265-6.

[96] V. H. Nguyen *et al.*, "Multi-environment genomic selection in rice elite breeding lines," *Rice*, vol. 16, p. 7, 2023, doi: 10.1186/s12284-023-00623-6.

[97] L. Crespo-Herrera *et al.*, "Genome-enabled prediction for sparse testing in multi-environmental wheat trials," *The Plant Genome*, vol. 14, p. e20151, 2021, doi: 10.1002/tpg2.20151.

[98] S. Sukumaran, D. Jarquin, J. Crossa, and M. Reynolds, "Genomic-enabled prediction accuracies increased by modeling genotype x environment interaction in durum wheat," *The Plant Genome*, vol. 11, p. 170112, 2018, doi: 10.3835/plantgenome2017.12.0112.

[99] J. Garcia-Abadillo *et al.*, "Sparse testing designs for optimizing predictive ability in sugarcane populations," *Frontiers in Plant Science*, vol. 15, p. 1400000, 2024, doi: 10.3389/fpls.2024.1400000.

[100] R. Persa *et al.*, "Improving predictive ability in sparse testing designs in soybean populations," *Frontiers in Genetics*, vol. 14, p. 1269255, 2023, doi: 10.3389/fgene.2023.1269255.

[101] S. Proma, J. Garcia-Abadillo, A. Shaik, and D. Jarquin, "Optimizing genomic prediction-based sparse testing designs for multi-environment trials in tetraploid potato cultivars," *Crop Science*, vol. 66, p. e70260, 2026, doi: 10.1002/csc2.70260.

[102] V. S. Windhausen *et al.*, "Effectiveness of genomic prediction of maize hybrid performance in different breeding populations and environments," *G3 Genes|Genomes|Genetics*, vol. 2, pp. 1427–1436, 2012, doi: 10.1534/g3.112.003699.

[103] R. Kadoumi *et al.*, "Evolution of population structure in a commercial European hybrid dent maize breeding program and consequences on genetic diversity," *Theoretical and Applied Genetics*, vol. 138, p. 233, 2025, doi: 10.1007/s00122-025-05008-5.

[104] N. Santantonio and K. Robbins, "A hybrid optimal contribution approach to drive short-term gains while maintaining long-term sustainability in a modern plant breeding program." 2020. doi: 10.1101/2020.01.08.899039.

[105] J. O. Rawlings and D. L. Thompson, "Performance level as criterion for the choice of maize testers," *Crop Science*, vol. 2, pp. 217–220, 1962, doi: 10.2135/cropsci1962.0011183x000200030012x.

[106] G. de Jong, O. Powell, G. Gorjanc, J. M. Hickey, and R. C. Gaynor, "Comparison of genomic prediction models for general combining ability in early stages of hybrid breeding programs," *Crop Science*, vol. 63, pp. 3338–3355, 2023, doi: 10.1002/csc2.21105.

[107] G. Costa-Neto, R. Fritsche-Neto, and J. Crossa, "Nonlinear kernels, dominance, and envirotyping data increase the accuracy of genome-based prediction in multi-environment trials," *Heredity*, vol. 126, pp. 92–106, 2020, doi: 10.1038/s41437-020-00353-1.

[108] J. Crossa *et al.*, "The modern plant breeding triangle: optimizing the use of genomics, phenomics, and enviromics data," *Frontiers in Plant Science*, vol. 12, p. 651480, 2021, doi: 10.3389/fpls.2021.651480.

[109] R. Rincent *et al.*, "Optimization of multi-environment trials for genomic selection based on crop models," *Theoretical and Applied Genetics*, vol. 130, pp. 1735–1752, 2017, doi: 10.1007/s00122-017-2922-4.

[110] A. J. DeSalvio *et al.*, "Near-infrared reflectance spectroscopy phenomic prediction can perform similarly to genomic prediction of maize agronomic traits," *The Plant Genome*, vol. 17, p. e20454, 2024, doi: 10.1002/tpg2.20454.

[111] C. R. Werner, D. C. Gemenet, and D. J. Tolhurst, "FieldSimR: an R package for simulating plot data in multi-environment field trials," *Frontiers in Plant Science*, vol. 15, p. 1330574, 2024, doi: 10.3389/fpls.2024.1330574.

[112] G. Covarrubias-Pazaran, "Genome-assisted prediction of quantitative traits using the R package sommer," *PLOS ONE*, vol. 11, p. e0156744, 2016, doi: 10.1371/journal.pone.0156744.

[113] A. Alemu *et al.*, "Genomic selection in plant breeding: key factors shaping two decades of progress," *Molecular Plant*, vol. 17, pp. 552–578, 2024, doi: 10.1016/j.molp.2024.03.007.

[114] J. Crossa *et al.*, "Genomic selection in plant breeding: methods, models, and perspectives," *Trends in Plant Science*, vol. 22, pp. 961–975, 2017, doi: 10.1016/j.tplants.2017.08.011.

[115] N. Heslot and V. Feoktistov, "Optimization of selective phenotyping and population design for genomic prediction," *Journal of Agricultural, Biological and Environmental Statistics*, vol. 25, no. 4, pp. 579–600, 2020, doi: 10.1007/s13253-020-00415-1.

[116] D. C. Kadam, O. R. Rodriguez, and A. J. Lorenz, "Optimization of training sets for genomic prediction of early-stage single crosses in maize," *Theoretical and Applied Genetics*, vol. 134, pp. 687–699, 2021, doi: 10.1007/s00122-020-03722-w.

[117] B. R. Cullis, A. B. Smith, N. A. Cocks, and D. G. Butler, "The design of early-stage plant breeding trials using genetic relatedness," *Journal of Agricultural, Biological and Environmental Statistics*, vol. 25, no. 4, pp. 553–578, 2020, doi: 10.1007/s13253-020-00403-5.

[118] O. A. Montesinos-López, A. Montesinos-López, R. Acosta, R. K. Varshney, A. Bentley, and J. Crossa, "Using an incomplete block design to allocate lines to environments improves sparse genome-based prediction in plant breeding," *The Plant Genome*, vol. 15, p. e20194, 2022, doi: 10.1002/tpg2.20194.

[119] D. Akdemir, S. Rio, and J. Isidro y Sánchez, "TrainSel: an R package for selection of training populations," *Frontiers in Genetics*, vol. 12, p. 655287, 2021, doi: 10.3389/fgene.2021.655287.

[120] D. G. Butler, A. B. Smith, and B. R. Cullis, "On the design of field experiments with correlated treatment effects," *Journal of Agricultural, Biological, and Environmental Statistics*, vol. 19, pp. 539–555, 2014, doi: 10.1007/s13253-014-0191-0.

[121] C. Lisle, A. B. Smith, C. Birrell, and B. R. Cullis, "Information based diagnostic for genetic variance parameter estimation in multi-environment trials," *Frontiers in Plant Science*, vol. 12, p. 785430, 2021, doi: 10.3389/fpls.2021.785430.

[122] S. Rio, D. Akdemir, T. Carvalho, and J. Isidro y Sánchez, "Assessment of genomic prediction reliability and optimization of experimental designs in multi-environment trials," *Theoretical and Applied Genetics*, vol. 135, pp. 405–419, 2022, doi: 10.1007/s00122-021-03972-2.

[123] M. Lell, J. C. Reif, and Y. Zhao, "Optimizing the setup of multienvironmental hybrid wheat yield trials for boosting the selection capability," *The Plant Genome*, vol. 14, p. e20150, 2021, doi: 10.1002/tpg2.20150.

[124] S. R. Mothukuri, Y. Beyene, M. Gültas, J. Burgueño, and S. Griebel, "Optimization of sparse phenotyping strategy in multi-environmental trials in maize," *Theoretical and Applied Genetics*, vol. 138, 2025, doi: 10.1007/s00122-025-04825-y.

[125] P. González-Barrios, L. Díaz-García, and L. Gutiérrez, "Mega-environmental design: using genotype x environment interaction to optimize resources for cultivar testing," *Crop Science*, vol. 59, pp. 1899–1915, 2019, doi: 10.2135/cropsci2018.11.0692.

[126] R. Hoefler and L. Gutierrez, "Simulation-based optimization of mega-environmental designs for sparse multi-environment testing," *Crop Science*, vol. 66, 2026, doi: 10.1002/csc2.70366.

[127] D. L. Schoemaker, D. C. Lima, N. de Leon, and S. M. Kaeppler, "Modeling the impact of resource allocation decisions on genomic prediction using maize multi-environment data," *Crop Science*, vol. 64, no. 5, pp. 2748–2767, 2024, doi: 10.1002/csc2.21305.

[128] S. Wright, "The genetical structure of populations," *Annals of Eugenics*, vol. 15, no. 1, pp. 323–354, 1951, doi: 10.1111/j.1469-1809.1949.tb02451.x.
[129] K. E. Holsinger and B. S. Weir, "Genetics in geographically structured populations: defining, estimating and interpreting F_{ST}", *Nature Reviews Genetics*, vol. 10, no. 9, pp. 639–650, 2009, doi: 10.1038/nrg2611.
[130] M. R. Labroo, A. J. Studer, and J. E. Rutkoski, "Heterosis and hybrid crop breeding: a multidisciplinary review," *Frontiers in Genetics*, vol. 12, 2021, doi: 10.3389/fgene.2021.643761.
[131] J. C. Reif, F.-M. Gumpert, S. Fischer, and A. E. Melchinger, "Impact of interpopulation divergence on additive and dominance variance in hybrid populations," *Genetics*, vol. 176, no. 3, pp. 1931–1934, 2007, doi: 10.1534/genetics.107.074146.
[132] M. R. White, M. A. Mikel, N. de Leon, and S. M. Kaeppler, "Diversity and heterotic patterns in North American proprietary dent maize germplasm," *Crop Science*, vol. 60, 2020, doi: 10.1002/csc2.20050.
[133] C. C. Cockerham, "Group inbreeding and coancestry," vol. 56, no. 1, 1967, pp. 89–104.
[134] M. Nei, "Analysis of gene diversity in subdivided populations," *Proceedings of the National Academy of Sciences*, vol. 70, no. 12, pp. 3321–3323, 1973, doi: 10.1073/pnas.70.12.3321.
[135] D. Lindgren, L. D. Gea, and P. A. Jefferson, "Status number for measuring genetic diversity," *Forest Genetics*, vol. 4, no. 2, pp. 69–76, 1997.
[136] M. A. Toro, B. Villanueva, and J. Fernández, "The concept of effective population size loses its meaning in the context of optimal management of diversity using molecular markers," *Journal of Animal Breeding and Genetics*, vol. 137, no. 4, pp. 345–355, 2020, doi: 10.1111/jbg.12457.
[137] E. López-Cortegano, R. Pouso, A. Labrador, A. Pérez-Figueroa, J. Fernández, and A. Caballero, "Optimal management of genetic diversity in subdivided populations," *Frontiers in Genetics*, vol. 10, p. 843, 2019, doi: 10.3389/fgene.2019.00843.
[138] H. De Beukelaer, G. F. Davenport, and V. Fack, "Core Hunter 3: flexible core subset selection," *BMC Bioinformatics*, vol. 19, no. 1, p. 203, 2018, doi: 10.1186/s12859-018-2209-z.
[139] A. Caballero and S. T. Rodríguez-Ramilo, "A new method for the partition of allelic diversity within and between subpopulations," *Conservation Genetics*, vol. 11, no. 6, pp. 2219–2229, 2010, doi: 10.1007/s10592-010-0107-7.
[140] C. R. Henderson, "Best linear unbiased estimation and prediction under a selection model," *Biometrics*, vol. 31, no. 2, pp. 423–447, 1975, doi: 10.2307/2529430.
[141] D. Habier, R. L. Fernando, and J. C. M. Dekkers, "The impact of genetic relationship information on genome-assisted breeding values," *Genetics*, vol. 177, no. 4, pp. 2389–2397, 2007, doi: 10.1534/genetics.107.081190.
[142] H. D. Daetwyler, B. Villanueva, and J. A. Woolliams, "Accuracy of predicting the genetic risk of disease using a genome-wide approach," *PLoS ONE*, vol. 3, no. 10, p. e3395, 2008, doi: 10.1371/journal.pone.0003395.
[143] G. de los Campos, J. M. Hickey, R. Pong-Wong, H. D. Daetwyler, and M. P. L. Calus, "Whole-genome regression and prediction methods applied to plant and animal breeding," *Genetics*, vol. 193, no. 2, pp. 327–345, 2013, doi: 10.1534/genetics.112.143313.
[144] A. Xavier, W. M. Muir, B. Craig, and K. M. Rainey, "bWGR: Bayesian whole-genome regression," *Bioinformatics*, vol. 36, no. 6, pp. 1957–1959, 2019, doi: 10.1093/bioinformatics/btz794.
[145] D. Gianola, G. de los Campos, W. G. Hill, E. Manfredi, and R. Fernando, "Additive genetic variability and the Bayesian alphabet," *Genetics*, vol. 183, no. 1, pp. 347–363, 2009, doi: 10.1534/genetics.109.103952.
[146] O. Kempthorne and A. W. Nordskog, "Restricted selection indices," *Biometrics*, vol. 15, no. 1, pp. 10–19, 1959, doi: 10.2307/2527598.
[147] A. Borodin, A. Jain, H. C. Lee, and Y. Ye, "Max-sum diversification, monotone submodular functions, and dynamic updates," *ACM Transactions on Algorithms*, vol. 13, no. 3, p. 41, 2017, doi: 10.1145/3086464.
[148] G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher, "An analysis of approximations for maximizing submodular set functions – I," *Mathematical Programming*, vol. 14, no. 1, pp. 265–294, 1978, doi: 10.1007/BF01588971.
[149] M. L. Fisher, G. L. Nemhauser, and L. A. Wolsey, "An analysis of approximations for maximizing submodular set functions – II," *Mathematical Programming Studies*, vol. 8, pp. 73–87, 1978, doi: 10.1007/BFb0121195.
[150] A. Cevallos, F. Eisenbrand, and R. Zenklusen, "Max-sum diversity via convex programming," in *32nd International Symposium on Computational Geometry (SoCG)*, 2016, p. 26:1–26:14. doi: 10.4230/LIPIcs.SoCG.2016.26.

[151] M. Minoux, "Accelerated greedy algorithms for maximizing submodular set functions," in *Optimization Techniques: Proceedings of the 8th IFIP Conference on Optimization Techniques*, J. Stoer, Ed., in Lecture Notes in Control and Information Sciences 7. Springer, 1978, pp. 234–243.

[152] S. Yadav and others, "Optimising parent selection in plant breeding: comparing metaheuristic algorithms for genotype building," *Theoretical and Applied Genetics*, vol. 138, no. 9, p. 242, 2025, doi: 10.1007/s00122-025-05028-1.

[153] J. Fernández-González, S. M. Metwally, and J. Isidro y Sánchez, "MateR: a novel genomic mating framework," *Genetics*, vol. 232, no. 4, p. iyag013, 2026, doi: 10.1093/genetics/iyag013.

[154] P. Waldmann, "Genomic optimum contribution selection and mate allocation using JuMP," *Bioinformatics Advances*, vol. 5, no. 1, p. vbaf259, 2025, doi: 10.1093/bioadv/vbaf259.

[155] M. A. Duran and I. E. Grossmann, "An outer-approximation algorithm for a class of mixed-integer nonlinear programs," *Mathematical Programming*, vol. 36, no. 3, pp. 307–339, 1986, doi: 10.1007/BF02592064.

[156] P. Ahadi, B. Balasundaram, J. S. Borrero, and C. Chen, "Development and optimization of expected cross value for mate selection problems," *Heredity*, vol. 133, pp. 80–90, 2024, doi: 10.1038/s41437-024-00697-y.

[157] O. G. Younis, L. Corinzia, I. N. Athanasiadis, A. Krause, J. M. Buhmann, and M. Turchetta, "Breeding programs optimization with reinforcement learning," *arXiv preprint arXiv:2406.03932*, 2024, doi: 10.48550/arXiv.2406.03932.

[158] L. Chen, G. Zhang, and H. Zhou, "Fast greedy MAP inference for determinantal point process to improve recommendation diversity," in *Advances in Neural Information Processing Systems 31*, 2018, pp. 5622–5633.

[159] Y. C. J. Wientjes, P. Bijma, M. P. L. Calus, B. J. Zwaan, Z. G. Vitezica, and J. van den Heuvel, "The long-term effects of genomic selection: 1. Response to selection, additive genetic variance, and genetic architecture," *Genetics Selection Evolution*, vol. 54, 2022, doi: 10.1186/s12711-022-00709-7.

[160] A.-C. Doublet, P. Croiseau, S. Fritz, A. Michenet, C. Hozé, and C. Danchin-Burge, "The impact of genomic selection on genetic diversity and genetic gain in three French dairy cattle breeds," *Genetics Selection Evolution*, vol. 51, 2019, doi: 10.1186/s12711-019-0495-1.

[161] J. E. Rutkoski, "Estimation of realized rates of genetic gain and indicators for breeding program assessment," *Crop Science*, vol. 59, 2019, doi: 10.2135/cropsci2018.09.0537.

[162] D. Müller, P. Schopp, and A. E. Melchinger, "Selection on expected maximum haploid breeding values can increase genetic gain in recurrent genomic selection," *G3: Genes, Genomes, Genetics*, vol. 8, 2018, doi: 10.1534/g3.118.200091.

[163] A. Robertson, "A theory of limits in artificial selection," *Proceedings of the Royal Society of London B*, vol. 153, no. 951, pp. 234–249, 1960, doi: 10.1098/rspb.1960.0099.

[164] M. G. Bulmer, "The effect of selection on genetic variability," *The American Naturalist*, vol. 105, no. 943, pp. 201–211, 1971, doi: 10.1086/282718.